

一只奶龙

寒枝OvO

打印版算法模板

生成日期: 2026-07-29

目录

- 赛时五步法：从读题到抄板 13
- 复杂度判定表：数据规模 -> 算法族 13
- 任务类型表：题目问什么 -> 先想什么 13
- 高频题型 -> 第一选择 14
- 本书小节的标准结构（新版格式） 14
- 「最小完整示例」长什么样 15
- 「传参要求」长什么样 15
- 区间顺序统计：别再拿错板子 15
- A 赛场入口与基础算法 15
 - 01 基础算法、数值工具与位运算 15
 - 通用头文件与主函数 15
 - 本地调试宏 16
 - 快读 FastScanner 16
 - il28 输入输出 17
 - 高精度整数 BigInt：非负大整数 18
 - 有理数 Frac：精确分数四则运算与比较 20
 - 快速幂 21
 - 最大公约数与最小公倍数 21
 - 无限重集合 k 子集和 gcd 22
 - 固定方向长度 k 连线增量计数 23
 - 整数平方根 24
 - 向下/向上整除 24
 - 常用 bit 内置函数 25
 - 位掩码基础：取位、置位、清位与最低位 25
 - 二进制集合：枚举子集、超集与集合拆分 26
 - Gosper's Hack：固定选 k 个元素的所有集合 27
 - Gray Code：相邻状态只翻转一位 28
 - 前缀异或：0(1) 区间 XOR 与 0 XOR 子数组计数 29
 - 静态区间 OR / AND：Sparse Table 30
 - 01 Trie：统计异或严格小于 K 的数对 30
 - 按位贪心：最大两数按位 AND 32
 - 子数组按位 OR / AND：不同结果集合 32
 - 按位贡献：所有无序数对的 XOR / OR / AND 和 33
 - OR / AND / XOR 卷积与子集变换：快速定位 34
 - bitset 01 背包：只问哪些和可达 35
 - 整数二分：最小可行值 36

整数二分：最大可行值	36
二分答案 + 区间覆盖判定	37
最大化最小距离：直线/圆周二分	38
实数二分	39
三分搜索	39
离散化	40
一维前缀和	41
一维差分	42
二维前缀和	43
二维差分	43
02 枚举、贪心、搜索与构造	44
贪心证明速查：邻项交换与反悔堆	44
分数背包：物品允许只取一部分	45
Huffman / 最优合并：每次合并最小两堆	46
定长滑动窗口：固定长度区间和/最大值	47
可变窗口：至多/恰好 K 种不同元素	48
正数数组双指针：最大长度且区间和不超过 S	48
排序双指针：数对和不超过 S 的数量	49
离线扫描 + 树状数组：区间中值落在 $[x, y]$ 的个数	50
DFS / 回溯骨架	52
双向 BFS	52
A* 最短路：第 k 短路骨架	53
IDA* 迭代加深搜索	54
DLX 精确覆盖	55
de Bruijn 序列：最短覆盖全部长度 n 串的循环串	57
折半搜索 Meet-in-the-Middle	58
B 数据结构与离线维护	59
03 基础数据结构与区间维护	59
并查集 DSU	59
单链表：头插、尾插、删除、反转与有序去重	60
双向循环链表：哨兵、 $O(1)$ 插删、移动与约瑟夫环	62
后继并查集：区间中跳过已处理位置	64
可撤销并查集	65
带权并查集	66
带权并查集例题：区间和约束判矛盾	68
关系并查集：食物链 mod 3	69
树状数组：单点加，区间和	70
树状数组求逆序对	72
树状数组：区间加，区间和	72
二维树状数组	73
区间颜色种类数：颜色很多且无修改	74
离线统计区间不同数字个数	75
莫队：区间不同数数量	76
莫队带修改	77

树上莫队	78
线段树：单点修改，区间最大值	80
线段树典题：单点修改，区间最大子段和	82
线段树：区间加，区间和/最大值	83
线段树：区间赋值，区间和	86
仿射懒标记线段树：区间乘、区间加、区间和	87
区间染色 + 查询区间颜色种类数	89
线段树 Beats：区间 chmin + 区间和	90
动态开点线段树：巨大值域单点加、区间和	92
线段树合并	93
分块：区间加 + 区间和	94
ST 表	95
单调队列	96
对顶堆：动态中位数、滑动窗口中位数与绝对偏差和	97
单调栈：左右第一个更小元素	99
单调栈例题：柱状图最大矩形	100
区间合并	101
启发式合并：小集合并入大集合	101
04 高级数据结构与离线分治	103
笛卡尔树	103
主席树：静态区间第 k 小 / 排名 / 前驱后继	104
动态区间顺序统计：值域线段树套 Fenwick (P3380 五操作)	107
普通 01 Trie：最大异或对与最大子段异或	111
可持久化 01 Trie：区间最大异或	112
Treap 平衡树	113
隐式 FHQ Treap：序列插入与第 k 个	114
左偏树：可并堆	116
Link-Cut Tree：动态森林	117
线性基	119
PBDS 有序树	120
CDQ 分治：三维偏序计数骨架	121
整体二分 Parallel Binary Search	122
离线动态连通性：线段树分治 + 可撤销并查集	122
C 树、图论与网络流	124
05 树上问题	124
树的直径	124
LCA：倍增	125
Tarjan 离线 LCA：并查集批量求最近公共祖先	127
长链剖分： $O(1)$ 第 k 级祖先	129
树链剖分 HLD	130
树上主席树：路径点权第 k 小	132
Euler 序 + 树状数组：子树加、单点查	134
Euler 序 + 树状数组：子树和查询	135
树上差分：路径加，点统计	137

树上路径交	138
树形 DP: 最大独立集	138
树上最大权匹配: 每个点至多选一条邻边	139
基环树最大点权独立集: 剥叶 + 树 DP + 环 DP	141
典题: 骑士互斥 / 环外树挂在一个环上	143
换根 DP: 所有点到其他点距离和	143
DSU on Tree	145
树的重心	146
点分治骨架	147
树上路径计数: 点分治统计恰好 K 条边	149
动态点分治: 点开关, 查询最近标记点	151
虚树 Virtual Tree	153
虚树例题: 关键点两两路径总长度	154
函数图倍增	154
树哈希: 无根树同构判定	155
06 图论基础: 遍历、最短路与生成树	156
图的邻接表	156
BFS 最短步数	157
网格 BFS	158
0-1 BFS	158
0-1 BFS 网格: 转向次数最少	159
Dijkstra	160
线段树优化建图: 点到区间 / 区间到点最短路	162
Bellman-Ford	163
Floyd	164
min-plus 矩阵快速幂: 恰好 / 至多 k 条边最短路	165
Johnson 全源最短路	166
SPFA 与负环判定	167
差分约束	168
最短路与次短路	169
所有最短路条数	170
Karp: 最小平均权环	171
最小生成树: Kruskal	171
最小生成树: Prim	172
最小 Steiner 树: 终端集状压 DP + 多源 Dijkstra	173
次小生成树	174
Kruskal 重构树	176
隐式 Kruskal 重构树: 增量网格连通块吞并阈值	177
朱刘算法: 有向图最小树形图	179
Stoer-Wagner 全局最小割	180
Matrix-Tree 定理: 生成树计数	180
07 图论进阶: 连通性、拓扑与结构	181
拓扑排序	181
DAG 最长路	182
非树边造好环 + BFS 定向构造	183

支配树: Lengauer-Tarjan	183
DAG 最小路径覆盖	184
Tarjan 强连通分量 SCC	185
割点与桥	186
边双连通分量 e-DCC	187
点双连通分量 v-DCC 与圆方树	189
仙人掌图找环	190
2-SAT	191
欧拉路径/回路: Hierholzer	192
bitset 传递闭包	193
最大团: Bron-Kerbosch bitset	194
三元环计数: 按度定向枚举	195
08 网络流、匹配与割建模	196
二分图染色判定	196
二分图最大匹配 Hopcroft-Karp	197
二分图最小点覆盖	199
Gale-Shapley: 稳定匹配 / 稳定婚姻	199
KM: 二分图最大权完美匹配	201
一般图最大匹配 Blossom	202
Dinic 最大流	204
HLPP: 最高标号预流推进最大流	206
最小费用最大流	207
最小割建模提醒	209
最大权闭合图: 最小割建模与选点恢复	209
上下界可行流 / 最大流 / 最小流	211
Gomory-Hu Tree: 无向图全点对最小割	212

D 字符串与序列 215

09 字符串与序列匹配	215
KMP 前缀函数	215
KMP 自动机: 单个禁串计数 DP	216
KMP 前缀数组计数字符串	218
Z 函数	219
序列自动机: 多模式子序列查询与最短缺失子序列	219
Trie 字典树	221
AC 自动机	223
AC 自动机: fail 树累计所有模式出现次数	224
AC 自动机: 批量收集所有模式串出现位置	226
典题: 子串出现覆盖的所有区间最大权和与总权和	227
AC 自动机 DP: 计数不含任一禁串的字符串	229
双哈希	230
双模二维滚动哈希: $O(1)$ 取任意子矩阵	231
典题 std: 在大网格中找出全部图案出现位置	233
Manacher	234
回文自动机 PAM	236

后缀数组 237

后缀数组 LCP RMQ 查询 238

后缀自动机 SAM 239

SAM: 子串出现次数与两串最长公共子串 240

多串最长公共子串: SAM 最小匹配长度 241

SAM: 字典序第 k 小本质不同子串 243

LCS 最长公共子序列 245

LCS 输出方案 245

最短公共超序列 SCS: 合并两条保持顺序的序列 246

Bit-parallel LCS: 只求长度 247

编辑距离 Levenshtein: 最少插入、删除、替换 247

最长公共上升子序列 LCIS 248

最长回文子序列与最少插入成回文 249

通配符匹配: $?$ 匹配一个, $*$ 匹配任意串 250

最小表示法 Booth 251

整数序列最小循环表示 Booth 251

Lyndon 分解 Duval 252

不重叠子序列匹配次数 253

字典序最小受限子序列 254

括号序列合法性与最长合法括号 255

E 数学、数论、几何与多项式 255

10 数论、组合计数与同余 255

 本册通用辅助函数 255

 ModInt 256

 线性求逆元与阶乘逆元 257

 扩展欧几里得 258

 线性筛 259

 线性筛: ϕ / μ / 约数个数 260

 试除分解质因数 261

 单个数欧拉函数 262

 枚举约数 262

 整除分块 263

 杜教筛: 大范围前缀 ϕ / μ 和 264

 Min_25 筛核心: $\pi(n)$ 与质数和 265

 Meissel-Lehmer: 超大范围质数个数 $\pi(n)$ 266

 Mobius 反演常见求和 268

 组合数预处理 269

 Lucas 定理 270

 exLucas: 组合数模任意合数 271

 Catalan 数 273

 Stirling 数 273

 Burnside / Polya 枚举 274

 Pruefer 序列: 带标号树编码、解码与凯莱公式 274

 扩展中国剩余定理 exCRT 276

线性同余：模 2^{64} 最小解	276
AtCoder floor_sum	277
BSGS 离散对数	278
exBSGS	279
原根 Primitive Root	279
二次剩余 Tonelli-Shanks	280
Miller-Rabin 素性测试	281
Pollard-Rho 因式分解	282
11 线性代数、多项式与生成函数	283
矩阵快速幂	283
有限状态转移矩阵快速幂	284
容斥原理：二进制枚举	285
高斯消元：实数线性方程组	286
单纯形法：线性规划最大化	287
GF(2) 高斯消元与异或方程组	289
模意义高斯消元：唯一解	289
吸收马尔可夫链期望方程	290
典题：标号图度数奇偶计数	291
行列式取模：任意模数版	292
康托展开	292
Lagrange 插值：连续点 $0..n-1$	293
FFT：任意整数卷积	294
NTT 卷积	295
形式幂级数核心：inv / ln / exp / sqrt / pow / divmod	296
积树：多点求值 $f(x_i)$	300
积树：快速插值 $f(x_i)=y_i$	301
FWT：AND / OR 卷积	301
FWHT：XOR 卷积	302
XOR 随机变量分布：FWHT 合并	303
Berlekamp-Massey + Kitamasa	304
Bostan-Mori：有理生成函数第 n 项	305
自适应辛普森积分：连续函数数值积分	306
12 计算几何、扫描线与空间结构	307
点与向量基础	307
三维点、向量、平面与体积	308
六边形网格 cube 坐标	309
线段相交与点到线段距离	310
多边形面积与点包含	311
简单多边形耳切三角剖分	311
Andrew 凸包	312
凸多边形闵可夫斯基和	313
旋转卡壳：凸包直径	315
最近点对	315
动态 K-D Tree：二维加点与矩形权值和	316
直线与圆交点	318

向量旋转与直线交点	319
一维移动墙反弹事件模拟	320
极坐标圆弧图最短路：关键角度建图	321
点在凸多边形内	321
Pick 定理	321
两圆交点	322
半平面交	323
曼哈顿距离变换	324
曼哈顿距离最小生成树	325
最小圆覆盖：随机增量	326
扫描线 + 线段树：矩形面积并	327

F 动态规划与状态优化 328

13 动态规划基础与状态设计	328
0/1 背包	328
完全背包	329
多重背包：二进制拆分	330
多重背包单调队列优化：数量很大、容量也大	331
分组背包	332
二维费用背包	332
背包方案数	333
整数分拆：把 n 写成若干正整数之和	334
有上界的方案计数：分糖果 / 分球到各人	334
依赖背包：树上选课骨架	335
LIS 最长严格上升子序列	336
LIS：恢复一组最长严格上升子序列	337
LIS 计数：最长严格上升子序列有多少条	338
概念辨析：子段、子序列与子串	339
带权区间选择：最大权不相交任务	339
最大子段和：无限制、环形、恰好 K 与长度区间 $[L, R]$	340
相邻区间合并：石子合并最小代价	342
矩阵链乘：相乘顺序的最小标量乘法次数	343
回文串最少分割：每段都是回文	344
区间 DP 骨架	345
Knuth 优化 DP	345
状压 DP：TSP 骨架	346
SOS DP：子集和变换	347
快速子集卷积 Fast Subset Convolution	348
小 k 枚举 + 区间行 DP：无左移网格避障	349
轮廓线 DP：骨牌覆盖	350
数位 DP 骨架	351
概率 DP：独立事件分布	352
期望 DP：含自环的移项公式	352
典题：随机吃寿司到空盘的期望步数	353
树上随机游走：到根的期望步数	354

均匀集卡与线性期望速查	355
有限状态零和 MDP: 值迭代差分收敛外推	356
14 DP 优化与高级状态模型	357
单调队列优化 DP	357
斜率优化: 单调队列凸包	357
Li Chao Tree	358
分治优化 DP	360
SMAWK: 全单调矩阵每行最优值	360
WQS 二分	362
Slope Trick: 分段线性凸函数 DP	363
最大子段和与最大子矩阵	364
全 1 最大矩形	365
G 交互、博弈、杂项与近期模型	366
15 交互、通信与特殊评测	366
2019-2026 XCPD 题源审查清单	366
特殊评测与 checker 思维: 输出任意合法方案	367
本地 checker 骨架: 构造题输出自检	367
典题记录: 2020 ICPC 银川 H Absolute Space	368
多数关系构造: McGarvey 投票序列	369
镜面反射网格: 状态图 + 0-1 BFS	370
计数构造工具: 三角数和矩形数贪心拆分	371
近年构造题自检: 随机验证与边界打印	372
交互题总策略: 协议、次数与故障处理	373
交互题通用协议: query / answer / flush	373
交互二分: 隐藏下标或阈值	374
交互比较排序: 用查询当比较器	375
交互容错: 重复询问多数投票	375
通信题信息量估算与设计清单	376
通信题 bit packing: 定长整数编码	376
通信题 XOR 翻一位传目标	377
通信题 Hamming(7,4): 一位纠错	378
通信题随机摘要: 集合一致性校验	379
16 博弈、随机、STL 与杂项速查	379
SG 函数	379
Nim 游戏	380
巴什博弈 / Bachet: 一堆每次取 $1..k$	381
Nim 必胜一步: 把异或和打成 0	381
反常 Nim / Misere Nim: 取最后一个输	382
Moore's Nim-k: 一次最多操作 k 堆	382
阶梯 Nim: 只看奇数层异或	383
Fibonacci Nim: 最小 Zeckendorf 项	383
Wythoff 博弈: 两堆 + 黄金分割	384
Multi-SG: 一步把一个子游戏拆成多个子游戏	385
树的删边游戏: Green Hackenbush 常用版	386

有向图游戏：胜 / 负 / 平局三态	387
二分图博弈：起点是否为最大匹配关键点	388
随机数	389
爬山法：邻域贪心近似搜索	390
模拟退火 Simulated Annealing	391
归并排序求逆序对	392
五张牌牌型比较：德州扑克基础评估	393
Josephus 约瑟夫问题	394
日期：基姆拉尔森公式	395
STL 常用函数速查	396
17 近期训练赛模型补充	396
赛后模型总览：2026 牛客暑期多校训练营 1	396
牛客多校 Round1 具体模型索引	397
赛后模型总览：2026 牛客暑期多校训练营 2	397
两集合异或和最大：总 XOR 固定位 + 线性基	398
逆序约束拓扑序计数：状压 DP	399
中位数替换最大和：排序 + 前缀和枚举中位	399
最大连通非边点对：大块加孤点	400
GCD 图最短路：质数间隙 + 容斥 + 短 DP	401
超立方删两点 Hamming-2 配对	402
交通期望：对角线随机游走首次撞边	403
树上绝对差赋点权：根相对值域 DP	404
少量特殊边动态最短路：普通图全源 BFS + 小图 Dijkstra	405
允许误差的向量询问：按方向分组 + 整数前缀近似	407
网格画同心矩形：偶短边拆块构造	408
羽毛球比分连胜：普通区与超分区分段	409
路径挂树按层生长计数 DP	410
随机四舍五入期望：连续 4 进位链与生成函数	410
赛后模型总览：2026 牛客暑期多校训练营 3	410
位对计数：全体按位操作后维护 1 段数量	411
Raney 引理：花光预算的合法序列计数	412
DAG 连续全 1 段：三组划分转最大权闭合图	413
两行扫雷条带 DP：按列 mask 统计方案数	413
矩阵同值分割：出现行前后缀列界 + 二维差分	414
一次交换最大邻差：局部贡献函数扫描	416
折线路径转向判定：叉积符号	416
严格升高移动博弈：按高度反向拓扑	417
树上约束重连：最深要求祖先 + 可并堆	417
有向边状态随机游走：树上无立即回头期望	418
赛后模型总览：2026 牛客暑期多校训练营 4	418
16 阶行列式构造：小元素矩阵表示任意目标值	418
平方和二次剩余构造：让 $p+q$ 成为平方数	420
重测队列：子集 DP + 高维前缀和	420
循环最小排列对抗构造：The Game	421
最短路比值只枚举边：DPRS 降边权询问	422

23 子序列：长度很小的区间最长合法子序列 423

拼接字符串加前缀改 a：只留 a 前缀长度和后缀 425

无 border 循环串出现计数：Rounddog 三分类 426

栈式最短路访问：Walk 区间 DP 427

任意点分树计数：子树路径长度背包 428

区间消除长度：极值折线线段树 430

凸多边形缩放内切圆覆盖：边 Voronoi + 二分 430

赛后模型总览：2026 河南萌新联赛第（一）场 430

二阶差分 gcd：加减同一个 x 变等差数列 431

区间 gcd 等于区间最小值计数 431

分层状态 BFS：一次破墙网格最短路 433

恰好 K 种且极差不超过 D 的子数组计数 434

字典序最小拓扑排序 435

区间减一到 0 的最少操作次数 435

所有数模 k 余数相同：枚举差值 gcd 的约数 436

删除一个元素后相邻奇偶交替 436

网格相邻不同染色：逐格 m 进制状态 DP 437

两种小权值物品凑最大公共和 438

赛前题源索引：码蹄杯 2022-2026 真题集 440

最大平均子段：长度至少 L 的二分答案 440

差分统计区间覆盖次数：重排后最大询问和 441

二分图染色：距离冲突两组分配 442

最小公倍数安全累积：上界剪枝 443

lowbit 拆分与最低位博弈速查 444

括号序列：线段树维护总和与前缀最小值 445

恢复回文与回文子序列：区间 DP 可恢复方案 447

网格寻路：一次破障 BFS 448

最终打印版原则：先看每节的「赛时先看」判断能不能用，再看「不会用就照抄 / API」接到 `solve()`，最后只在需要改功能时阅读实现。题面区间、内部递归区间、值域区间不是一回事；排名 k 从 0 还是 1 开始也必须以本节 API 为准。

一只奶龙 • 寒枝0v0

本册定位：赛时模板，不是教材。目标是“想到算法以后，不需要重新读实现，就能直接正确接入”。

赛时五步法：从读题到抄板

拿到一道题，按这个顺序走，不要跳步：

1. 读题，问自己三个问题：它要我求什么（最值 / 计数 / 判定 / 构造 / 第 k 小）？数据规模 n/m 是多少（决定复杂度上限）？操作是静态还是动态、在线还是离线？
2. 定位本质：把题面翻译成“要维护什么”。例：`[L,R]` 问第 k 小 $\rightarrow$ 维护值域上的出现次数；树上路径问颜色数 $\rightarrow$ 维护路径上集合；多次区间加最后求和 $\rightarrow$ 维护差分。
3. 锁复杂度：用下方「复杂度判定表」反推算法族，再用「高频题型表」的题面信号对号入座，翻到那一节。
4. 照抄接入：只看该节的「抄板清单」三步（构造 $\rightarrow$ 操作 $\rightarrow$ 取结果），把代码整段抄到 `solve()` 上面。
5. 按题改造：只改「改造点」列出的地方（下标起点、 k 从 0/1、比较器、模数、权值类型、附加维护量），其余一行都不要动。

最重要的一条：比赛时优先看“无脑调用”。`*_impl / dfs / push / pull / clone` 这类名字默认是内部实现；除非本节明确告诉你，否则不要从 `solve()` 直接调。

复杂度判定表：数据规模 $\rightarrow$ 算法族

数据规模（时限 1-2s）	可行复杂度	优先考虑的算法族
$n \leq 20$	$O(2^n n) / O(3^n n)$	状压 DP、子集枚举、Gosper、DLX
$n \leq 40$	$O(2^{n/2} n)$	折半搜索 Meet-in-the-Middle
$n \leq 500$	$O(n^3)$	Floyd、区间 DP、KM、高斯消元、单纯形、Steiner
$n \leq 5e3$	$O(n^2)$	普通 DP、LIS/LCS、 n^2 建图 + Dijkstra、SMAWK
$n \leq 1e5$	$O(n \log n)$	排序二分、BIT/线段树/ST、莫队、Dijkstra、Kruskal、Tarjan、SA/SAM、NTT
$n \leq 1e6$	$O(n)$ 或 $O(n \log n)$ 小常数	线性筛、单调栈/队列、双指针、KMP、Manacher、哈希
值域 $1e9 \sim 1e18$	$O(\log V) / O(\sqrt{n})$	二分答案、矩阵快速幂、BSGS、整除分块、Pollard-Rho、杜教筛
$n \leq 1e18$	$O(\log n) / O(k^2 \log n)$	快速幂、exgcd/CRT、BM+Kitamasa、Bostan-Mori、min-plus 矩阵幂

用法：先把题目最大的 n 填进表里看“能跑多重的复杂度”，再反过来淘汰算法。例如 $n=2e5$ 就不可能 $O(n^2)$ ，所以要么 $O(n \log n)$ 数据结构，要么 $O(n \sqrt{n})$ 莫队/分块。

任务类型表：题目问什么 $\rightarrow$ 先想什么

题目问什么	第一反应	对应章节
最小化最大值 / 最大化最小值 / 最小时刻	二分答案 + 判定函数	A 01 二分
方案数 / 满足条件的个数	DP / 组合数 / 容斥 / 莫队 / AC 自动机 DP	F 13、E 10
是否存在 / 输出任意方案	图论建模（流 / 2-SAT / 匹配）、贪心、构造	C 08、G 15
第 k 小 / 第 k 大	主席树、整体二分、SAM 第 k 子串、二分+计数	B 04、D 09
区间查询（静态数组）	前缀和 / ST / 主席树 / 离线 BIT	A 01、B 03
区间查询（带修改）	BIT / 线段树 / 树套树 / 莫队带修	B 03/04
树上路径 / 子树	LCA+差分 / HLD / 树上主席树 / 点分治 / Euler 序	C 05
字符串子串	哈希 / KMP / Manacher / SAM / SA	D 09

题目问什么	第一反应	对应章节
最优 DP 但转移慢	单调队列 / 斜率 / Li Chao / 决策单调 / WQS / Slope Trick	F 14
数据范围巨大但点数少	离散化 / 坐标压缩 / 动态开点线段树	A 01、B 03
交互 / 通信 / 特殊评测	交互二分、多数投票、bit packing、Hamming	G 15

高频题型 -> 第一选择

题面信号	第一选择	本质（在维护什么）	关键提醒
静态 [L,R] 第 k 小 / 中位数 / 排名	主席树 PersistentKth	值域上的历史版本前缀，区间 = 两版本相减	数组 1-indexed; k 从 1 开始; 不能修改
单点修改 + 区间排名 / kth / 前驱后继	动态区间顺序统计（树套树）	位置 BIT 上挂值域线段树，两维同时回答	先读完操作做坐标预处理
单点加 + 区间和	Fenwick	前缀和的可增量维护， bit[i] 沿 lowbit 链累加	1-indexed; range_sum=L..R
区间加 + 区间和/最大值	Lazy Segment Tree	节点存区间聚合 + 懒标记，查询时下推	懒标记含义和合并公式必须配套
静态 RMQ / gcd / OR / AND	Sparse Table	幂等操作可重叠合并，两段覆盖一区间	静态; 操作需满足对应性质
区间不同数，离线无修改	离线 BIT / 莫队	每个值只留最右一次出现，或窗口计数	查询多且无在线依赖
区间不同数，带修改	莫队带修改	窗口增删 + 时间轴移动	增加时间维
动态集合排名 / 第 k 小（不带区间）	PBDS / Treap	平衡树按值维护子树大小	PBDS 重复值要加唯一 id
树上路径第 k 小，点权静态	树上主席树	每个点到根的版本链，路径 = 四个版本相减	root[u]+root[v]-root[lca]-root[parent(lca)]
子树修改/查询	Euler 序 + BIT/线段树	子树映射为连续 dfn 区间	必须先 dfs 出 tin/tout
单源最短路，非负边	Dijkstra	每次确定一个最短路，松弛邻边	i64 dist ; 过期堆元素要跳过
0/1 边最短路	0-1 BFS	双端队列按代价排序扩展	0 权 push_front, 1 权 push_back
负边 / 差分约束	Bellman-Ford / SPFA 判环	边松弛不等式 x[v]<=x[u]+w 的可解性	注意负环与超级源
SCC / 2-SAT	Tarjan SCC	栈内 dfs 序维护强连通分量	SCC 编号方向与缩点拓扑要核对
最大流	Dinic	残量网络上的分层 + 增广路	有向边残量; 反向边容量初始 0
字符串单模式匹配	KMP / Z	维护“当前已匹配前缀长度”，失配跳 pi	pi/z 定义和返回下标先看约定
多模式匹配	AC 自动机	Trie + fail 跳转，一次扫描统计全部模式	insert -> build -> query 顺序
回文子串	Manacher / PAM	中心扩展 + 已覆盖最右回文复用信息	Manacher 求长度, PAM 适合在线扩展
组合数模质数	阶乘 + 逆元	预处理阶乘与阶乘逆元, C(n,k) O(1)	n < mod 时最直接; 否则看 Lucas
大整数判素/分解	Miller-Rabin + Pollard-Rho	概率判定 + 生日悖论分解	64 位乘法用 __int128
普通背包	0/1 / 完全 / 多重背包	容量维 DP, 循环方向决定能否重复取	容量循环方向决定“能否重复取”
DP 转移是直线最值	CHT / Li Chao	维护直线集合，按斜率/交点求最值	先确认斜率/查询 x 是否单调

本书小节的标准结构（新版格式）

每节统一为下面的骨架，赛时只按顺序读，不要乱翻：

```

> **赛时先看**
> - 题目信号
> - 本质
> - 复杂度判定
> - 维护的量
> - 接法 / 警告

**最小完整示例**
**传参要求**
**API / 入口函数**
**抄板清单**
**改造点**
**核心逻辑**
**改板时先认这几个量**
代码块
典题模型

< 30 秒决策区
< 题面出现什么词/特征 -> 就是它
< 这个算法到底在维护什么 / 在干什么
< O(...) n 多大能用 替代方案
< 赛中需要维护的关键成员
< 一句话接入 + 易错点

< 重点 能直接跑的最小代码 含结构体定义/调用/输出
< 每个函数/构造的参数含义 下标起点 范围 返回值
< 赛时只认这里的名字
< 三步 构造 -> 操作 -> 取结果
< 按题面改这几处 其它别动
< 改代码时不能破坏的部分
< 关键成员速查
< struct 头注释 维护的量 + 不变量
< 一道可对号的真实/典型题

```

每节必看的两个块：最小完整示例（照着抄就能跑）和 传参要求（每个参数怎么传）。
看不懂实现没关系，这两个块不看实现也能写对题。

「最小完整示例」长什么样

每个模板节都会有一小段“题目 + 完整调用代码”，格式固定为：结构体怎么定义（构造）→ 怎么初始化 → 怎么调用 → 输出什么。例如 Fenwick 节：

题目：n 个数，q 次操作，单点加 / 区间求和。

```
int n, q;
cin >> n >> q;
Fenwick<i64> fw(n); // 1. 结构体定义: Fenwick<i64>(长度 n)
for (int i = 1; i <= n; ++i) fw.add(i, a[i]); // 2. 初始化: 把 a[i] 放进下标 i
while (q--) {
    int op, x, y;
    cin >> op >> x >> y;
    if (op == 1) fw.add(x, y); // 3. 调用: 把 a[x] 加 y
    else cout << fw.range_sum(x, y) << '\n'; // 4. 调用: 输出 [x, y] 的和
}
```

输出示例：n=5, a=[1,2,3,4,5], add(2,10) 后查 [1,5] → 25。

「传参要求」长什么样

```
- Fenwick<T>(n) 构造 n = 数组长度 内部 1-indexed
- add(i, v) 把下标 i 的值加 v 要求 1 <= i <= n
- range_sum(l, r) 闭区间 [l,r] 的和 要求 1 <= l <= r <= n
- 返回值 和 类型 = 模板参数 T 答案可能超 int 时用 i64
```

下标起点、闭/开区间、k 从 0 还是 1、要不要先 build/dfs、返回值是原值还是编号——全部写在这里，照传即可。

区间顺序统计：别再拿错板子

- 静态数组：PersistentKth，查询 $O(\log M)$ ，最简洁。
- 动态全局集合：PBDS/Treap，支持插删和全局排名，但没有 $[L, R]$ 位置区间。
- 动态数组 + $[L, R]$ 顺序统计：使用本书新增“动态区间顺序统计：值域线段树套 Fenwick”。
- 树上路径静态 kth：使用“树上主席树”，不是数组主席树直接套。

A 赛场入口与基础算法

01 基础算法、数值工具与位运算

把比赛骨架、输入输出、调试、常用整数工具、位运算、二分、三分、离散化和前缀差分集中放在一起。

通用头文件与主函数

赛时先看

题目信号：任意题。

本质：比赛所有题目的默认开局。

复杂度判定：无。

警告：本书后续代码默认已经粘贴这份骨架，统一使用 i64 / ui64 / i128、INF / LIN 和 mod7 / mod9 / modn；不要再额外定义 ll、u64 或 #define int long long。

```
// 语言: C++。
// 寒枝0v0。

#include <bits/stdc++.h>

using namespace std;

using i64 = long long;
using ui64 = unsigned long long;
using i128 = __int128;
```

```

using pii = pair<int, int>;
using pll = pair<i64, i64>;

#define endl '\n'
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define eb emplace_back

const int INF = 0x3f3f3f3f;
const i64 LINF = 4e18;
const int mod7 = 1e9 + 7;
const int mod9 = 1e9 + 9;
const int modn = 998244353;

const int dx[4] = {0, 0, 1, -1};
const int dy[4] = {1, -1, 0, 0};

void solve()
{

}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr), cout.tie(nullptr);

    int T = 1;
    // 示例: cin >> T;

    while (T--)
    {
        solve();
    }

    return 0;
}

```

本地调试宏

赛时先看

题目信号：WA 后需要看中间状态。

本质：本地打印变量，提交时自动关闭。

复杂度判定：无。

警告：不要输出到 `stdout`；提交时不要定义 `LOCAL`。

【不会用就照抄】

```
dbg(x); // 本地打印 "x = 值" 到 stderr；提交（未定义 LOCAL）时空操作。
```

```

#ifdef LOCAL
#define dbg(x) cerr << #x << " = " << (x) << '\n'
#else
#define dbg(x) ((void)0)
#endif

```

快读 FastScanner

赛时先看

题目信号：读入整数数量达到几百万以上，担心 IO 卡常。

本质：输入规模极大时替代 `cin`。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；`solve()`

里先构造对象，再按下方“不会用就看这里”调用公开接口。

复杂度判定：0（输入长度）。

维护的量：`idx/size`（缓冲中已读字节数/本次读入的字节数）；`buf[S]`（ $1 \leq S \leq 20$ 的字符缓冲）。

警告：只适合读整数；字符串和浮点要另写。

【最小完整示例（先抄这一段就能跑）】

```

// 输入规模几百万的整数，求和后输出。
FastScanner fs;
int n;
fs.read_int(n);
i64 sum = 0, x;

```



```
while (n--) { fs.read_int(x); sum += x; }
cout << sum << '\n';
```

- 样例：输入 3 1 2 3 -> 输出 6。

【传参要求（照这个传不会错）】

- `read_int(T& out)`: 把下一个整数读进 `out`，成功返回 `true`、读到文件尾返回 `false`；自动处理负数；`T` 传 `int` / `i64` / `ui64` 均可。
- `getch()`: 底层读单个字符，文件结束返回 `0`；平时不用碰。
- 注意：本板子只负责整数；字符串和浮点请用 `cin`。

【不会用就照抄】

```
FastScanner fs;
i64 x;
fs.read_int(x); // 读一个整数到 x; 成功返回 true
```

- 你主要调用 `read_int(x)`；`getch()` 是底层读字符，平时不用碰。
- 不要和 `cin` 随意混用缓冲策略；这份板子只负责整数。

```
struct FastScanner {
    static const int S = 1 << 20;
    int idx = 0, size = 0;
    char buf[S];

    char getch() {
        if (idx >= size) {
            size = (int)fread(buf, 1, S, stdin);
            idx = 0;
            if (size == 0) return 0;
        }
        return buf[idx++];
    }

    template <class T>
    bool read_int(T& out) {
        char c = getch();
        if (!c) return false;
        while (c && c <= ' ') c = getch();
        T sign = 1, x = 0;
        if (c == '-') sign = -1, c = getch();
        while (c >= '0' && c <= '9') {
            x = x * 10 + (c - '0');
            c = getch();
        }
        out = x * sign;
        return true;
    }
};
```

i128 输入输出

赛时先看

题目信号：1e18 * 1e18、乘法比较、CRT 合并。

本质：中间结果超过 i64，但不需要高精度类。

复杂度判定：O(位数)。

维护的量：无额外结构；只重载了 `operator>>` / `operator<<` 两个全局函数。

警告：`cin/cout` 不能直接读写 i128。

【最小完整示例（先抄这一段就能跑）】

```
// 读入两个 i128 并输出它们的乘积。
i128 a, b;
cin >> a >> b;
cout << a * b << '\n';
```

- 样例：输入 12345678901234567890123 2 -> 输出 24691357802469135780246。

【传参要求（照这个传不会错）】

- `operator>>(istream& in, i128& x)`: 像普通整数一样读入 `x`；自动处理负号；和下方的 `operator<<` 重载一起整段抄。
- `operator<<(ostream& out, i128 x)`: 把 `x` 按十进制输出；`x == 0` 输出 `0`。
- 中间乘法防止先按 i64 算：写成 `(i128)a * b`。

【不会用就照抄】

```
i128 x;
cin >> x;
cout << x << '\n';
```

- 本节已经重载 `>>` / `<<`，所以把这两个重载函数一起抄上去后，就能像普通整数一样读写。
- 中间乘法需要 `i128` 时，记得在乘法发生前强转：`(i128)a * b`。

```
istream& operator>>(istream& in, i128& x) {
    string s;
    in >> s;
    int sign = 1, i = 0;
    if (s[0] == '-') sign = -1, i = 1;
    x = 0;
    for (; i < (int)s.size(); ++i) x = x * 10 + (s[i] - '0');
    x *= sign;
    return in;
}

ostream& operator<<(ostream& out, i128 x) {
    if (x == 0) return out << '0';
    if (x < 0) out << '-', x = -x;
    string s;
    while (x > 0) {
        s.push_back(char('0' + x % 10));
        x /= 10;
    }
    reverse(s.begin(), s.end());
    return out << s;
}
```

高精度整数 BigInt：非负大整数

赛时先看

题目信号：题面明确“不取模”；答案可能超过 `i64` / `i128`；需要输出完整十进制整数。

本质：处理非负大整数的输入输出、比较、加法、减法、乘法、除以 `int` 和模 `int`。适合计数答案巨大但不取模的题。

接法：读入字符串构造 `BigInt(s)`；普通计数 DP 里用 `+` 累加；需要乘小数或做大整数乘法时用

`*`。如果最终只需要对小模数取余，用成员函数 `mod_int(b)`。

复杂度判定：加减 $O(n)$ ，乘法 $O(nm)$ ，除以 `int` $O(n)$ ，其中 n 是 `base` 为 `1e8` 的块数。

维护的量：`d`（低位在前的块数组，每块存 `0..BASE-1`）；`BASE/WIDTH`（进制 `1e8` 与每块 8 位）。

警告：本模板只处理非负整数；减法要求左边不小于右边。若题目有负数，外层单独维护符号。

【最小完整示例（先抄这一段就能跑）】

```
// 读入两个大整数，输出和、差、积、除以 3 的商和模 3。
string sa, sb;
cin >> sa >> sb;
BigInt a(sa), b(sb);
cout << a + b << '\n'; // 加法
cout << a - b << '\n'; // 减法，要求 a >= b
cout << a * b << '\n'; // 乘法
cout << a / 3 << ' ' << a.mod_int(3) << '\n'; // 除 int 与模 int
```

- 样例：输入 `999999999999999999 1` -> 依次输出
`1000000000000000000`、`999999999999999998`、`999999999999999999`、`333333333333333333 0`。

【传参要求（照这个传不会错）】

- `BigInt(x)` / `BigInt(s)`：用 `i64` 或十进制字符串（仅非负）构造。
- `str()` / `operator<<`：输出完整十进制。
- `+`、`-`、`*`：`BigInt` 间运算；`-` 要求左操作数不小于右操作数。
- `/ int b`、`mod_int(b)`：除以正 `int`，`b > 0`；分别返回商与余数。
- `cmp(a, b)` / `<` / `==`：精确比较。

【改板时先认这几个量】

- `d`：低位在前，每个块存 `0..BASE-1`。
- `cur`：加减乘除时当前块上的临时累加值。

```
struct BigInt {
    static const int BASE = 100000000;
    static const int WIDTH = 8;
    vector<int> d; // 低位在前，每个块存 0..BASE-1。
};
```

```

BigInt(i64 x = 0) { *this = x; }
explicit BigInt(const string& s) { read(s); }

BigInt& operator=(i64 x) {
    d.clear();
    if (x == 0) return *this;
    while (x > 0) {
        d.push_back((int)(x % BASE));
        x /= BASE;
    }
    return *this;
}

BigInt& read(const string& s) {
    d.clear();
    for (int i = (int)s.size(); i > 0; i -= WIDTH) {
        int l = max(0, i - WIDTH);
        int x = 0;
        for (int j = l; j < i; ++j) x = x * 10 + (s[j] - '0');
        d.push_back(x);
    }
    trim();
    return *this;
}

void trim() {
    while (!d.empty() && d.back() == 0) d.pop_back();
}

bool is_zero() const { return d.empty(); }

string str() const {
    if (d.empty()) return "0";
    string s = std::to_string(d.back());
    for (int i = (int)d.size() - 2; i >= 0; --i) {
        string part = std::to_string(d[i]);
        s += string(WIDTH - part.size(), '0') + part;
    }
    return s;
}

friend ostream& operator<<(ostream& os, const BigInt& x) {
    return os << x.str();
}

friend int cmp(const BigInt& a, const BigInt& b) {
    if (a.d.size() != b.d.size()) return a.d.size() < b.d.size() ? -1 : 1;
    for (int i = (int)a.d.size() - 1; i >= 0; --i) {
        if (a.d[i] != b.d[i]) return a.d[i] < b.d[i] ? -1 : 1;
    }
    return 0;
}

friend bool operator<(const BigInt& a, const BigInt& b) { return cmp(a, b) < 0; }
friend bool operator==(const BigInt& a, const BigInt& b) { return cmp(a, b) == 0; }

friend BigInt operator+(const BigInt& a, const BigInt& b) {
    BigInt c;
    int n = max(a.d.size(), b.d.size());
    c.d.assign(n + 1, 0);
    i64 carry = 0;
    for (int i = 0; i <= n; ++i) {
        i64 cur = carry;
        if (i < (int)a.d.size()) cur += a.d[i];
        if (i < (int)b.d.size()) cur += b.d[i];
        c.d[i] = (int)(cur % BASE);
        carry = cur / BASE;
    }
    c.trim();
    return c;
}

friend BigInt operator-(const BigInt& a, const BigInt& b) {
    assert(!(a < b));
    BigInt c;
    c.d.assign(a.d.size(), 0);
    i64 borrow = 0;
    for (int i = 0; i < (int)a.d.size(); ++i) {
        i64 cur = (i64)a.d[i] - borrow - (i < (int)b.d.size() ? b.d[i] : 0);
        if (cur < 0) cur += BASE, borrow = 1;
        else borrow = 0;
        c.d[i] = (int)cur;
    }
    c.trim();
    return c;
}

```

```

    }

    friend BigInt operator*(const BigInt& a, const BigInt& b) {
        BigInt c;
        if (a.is_zero() || b.is_zero()) return c;
        c.d.assign(a.d.size() + b.d.size(), 0);
        for (int i = 0; i < (int)a.d.size(); ++i) {
            i64 carry = 0;
            for (int j = 0; j < (int)b.d.size() || carry; ++j) {
                i64 cur = c.d[i + j] + carry;
                if (j < (int)b.d.size()) cur += 1LL * a.d[i] * b.d[j];
                c.d[i + j] = (int)(cur % BASE);
                carry = cur / BASE;
            }
        }
        c.trim();
        return c;
    }

    friend BigInt operator/(const BigInt& a, int b) {
        assert(b > 0);
        BigInt c;
        c.d.assign(a.d.size(), 0);
        i64 rem = 0;
        for (int i = (int)a.d.size() - 1; i >= 0; --i) {
            i64 cur = a.d[i] + rem * BASE;
            c.d[i] = (int)(cur / b);
            rem = cur % b;
        }
        c.trim();
        return c;
    }

    int mod_int(int b) const {
        assert(b > 0);
        i64 rem = 0;
        for (int i = (int)d.size() - 1; i >= 0; --i) rem = (rem * BASE + d[i]) % b;
        return (int)rem;
    }
};

```

有理数 Frac: 精确分数四则运算与比较

赛时先看

题目信号: 题面要求精确输出分数；或者比较两个比例 a/b 与 c/d ，直接用 `double` 可能有精度误差。

本质: 维护形如 p/q 的有理数，支持化简、加减乘除和精确比较。常用于斜率、概率、期望、比例和几何中的有理比较。

接法: 构造 `Frac(a,b)`，自动约分；比较直接用 `<`、`==`；输出用 `num/den`，若 `den==1` 可以只输出整数。

复杂度判定: 每次运算 $O(\log V)$ 做 gcd；比较用 `i128` 防止交叉乘爆 `i64`。

维护的量: `num`（分子）；`den`（分母，始终为正，构造时自动约分）。

警告: 分母必须保持为正；乘法结果可能超过 `i64`，本模板适合中等范围整数，极大整数分数要换 `BigInt`。

【最小完整示例（先抄这一段就能跑）】

```

// 求 1/2 + 1/3 并以最简分数输出。
Frac a(1, 2), b(1, 3);
Frac c = a + b; // 自动约分
cout << c.num << '/' << c.den << '\n'; // 5/6

```

• 样例: `Frac(1,2) + Frac(1,3)` → 输出 `5/6`。

【传参要求（照这个传不会错）】

- `Frac(i64 n = 0, i64 d = 1)`: 分子 / 分母构造并自动约分；`d != 0`。
- `+`、`-`、`*`、`/`: 四则运算，结果自动约分；`/` 要求除数 `b.num != 0`。
- `<`、`==`: 精确比较，内部用 `i128` 交叉乘。
- 输出: 成员 `num`、`den` 公开，自行拼 `num/den`。

【改板时先认这几个量】

- `den`: 分母，始终为正。
- `g`: 约分用的最大公约数。

```

struct Frac {
    i64 num = 0, den = 1; // den 始终为正。

    Frac(i64 n = 0, i64 d = 1) : num(n), den(d) { normalize(); }

```

```

void normalize() {
    assert(den != 0);
    if (den < 0) num = -num, den = -den;
    i64 g = gcd(llabs(num), den);
    if (g) num /= g, den /= g;
}

friend Frac operator+(const Frac& a, const Frac& b) {
    return Frac(a.num * b.den + b.num * a.den, a.den * b.den);
}
friend Frac operator-(const Frac& a, const Frac& b) {
    return Frac(a.num * b.den - b.num * a.den, a.den * b.den);
}
friend Frac operator*(const Frac& a, const Frac& b) {
    return Frac(a.num * b.num, a.den * b.den);
}
friend Frac operator/(const Frac& a, const Frac& b) {
    assert(b.num != 0);
    return Frac(a.num * b.den, a.den * b.num);
}

friend bool operator<(const Frac& a, const Frac& b) {
    return (i128)a.num * b.den < (i128)b.num * a.den;
}
friend bool operator==(const Frac& a, const Frac& b) {
    return a.num == b.num && a.den == b.den;
}
};

```

快速幂

赛时先看

题目信号：指数很大；题目要求取模；需要费马小定理求逆元。

本质：求 $a^b \bmod m$ ，组合数学、矩阵、逆元常用。

接法：看到 $a^b \bmod m$ 且 b 很大就调用 `mod_pow(a,b,m)`；求质数模 p 下的逆元时用 `mod_pow(a,p-2,p)`，但前提是 p 为质数且 a 不是 p 的倍数。模数可能不是质数时，改翻扩展欧几里得。

复杂度判定： $O(\log b)$ 。

维护的量：无额外结构；只维护累积结果 `res` 与右移的指数 `b`。

警告：乘法可能爆 `i64` 时用 `i128`。

【最小完整示例（先抄这一段就能跑）】

```

// 求  $2^{100} \bmod 1e9+7$ ，以及 3 在模  $1e9+7$  下的逆元。
cout << mod_pow(2, 100, mod7) << '\n'; // 976371285
cout << mod_pow(3, mod7 - 2, mod7) << '\n'; // 333333336

```

• 样例：输出 976371285 与 333333336。

【传参要求（照这个传不会错）】

- `a`：底数，任意整数，内部先 `a %= mod`。
- `b`：指数，非负 `i64`（可到 $1e18$ ）。
- `mod`：模数， > 0 ，赛时直接传 `mod7 / mod9 / modn`。
- 返回值： $a^b \% \text{mod}$ (`i64`)；`mod` 为质数时 `mod_pow(a, mod - 2, mod)` 即 a 的逆元（要求 $a \% \text{mod} \neq 0$ ）。

```

i64 mod_pow(i64 a, i64 b, i64 mod) {
    i64 res = 1 % mod;
    a %= mod;
    while (b > 0) {
        if (b & 1) res = (i128)res * a % mod;
        a = (i128)a * a % mod;
        b >>= 1;
    }
    return res;
}

```

最大公约数与最小公倍数

赛时先看

题目信号：题面出现 `gcd/lcm/互质/整除/周期`。

本质：整除、约分、周期合并、数论题。

接法：约分、判断互质、合并周期时先想 `gcd`；两个周期同时对齐常用 `lcm`。多个数的 `gcd/lcm` 就从左到右累积。`lcm` 可能爆 `i64` 时先判断 `a / gcd(a,b) > LIMIT / b`。

复杂度判定： $O(\log \min(a,b))$ 。

维护的量：无额外结构；纯函数，只传 a/b 两个参数。

警告： $lcm = a / gcd(a,b) * b$ ，先除再乘防溢出。

【最小完整示例（先抄这一段就能跑）】

```
// 求 12 和 18 的 gcd/lcm，并用 gcd 约分 24/36。
cout << gcd_ll(12, 18) << ' ' << lcm_ll(12, 18) << '\n';
int g = gcd_ll(24, 36);
cout << 24 / g << '/' << 36 / g << '\n';
```

- 样例：输出 6 36 与 2/3。

【传参要求（照这个传不会错）】

- $gcd_ll(a, b)$ ：任意符号的整数，结果取非负；返回最大公约数。
- $lcm_ll(a, b)$ ：任一为 0 返回 0；内部先除后乘防溢出；返回最小公倍数。
- 多个数的 gcd/lcm：从左到右逐个累积。

```
i64 gcd_ll(i64 a, i64 b) {
    return b == 0 ? llabs(a) : gcd_ll(b, a % b);
}

i64 lcm_ll(i64 a, i64 b) {
    if (a == 0 || b == 0) return 0;
    return a / gcd_ll(a, b) * b;
}
```

无限重集合 k 子集和 gcd

赛时先看

题目信号：所有元素可重复选择；所有大小为 k 的和取 gcd； k 可以任意大。

本质：集合中每种数有无限个，问大小为 k 的可重子集和的 gcd，或问其最大值和最小达成 k 。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()`

里先构造对象，再按下方“不会用就看这里”调用公开接口。

复杂度判定： $O(n \log V)$ 。

维护的量： d （所有数与 $a[0]$ 差值的 gcd）；**best**（最大可达 gcd）与 **min_k**（达成它的最小 k ）；**infinite**（是否随 k 无界增长）。

警告：集合不能为空；gcd 取非负值。当所有数都相等时，只有公共值非零才会随 k 无界增长；全为 0 时所有和、所有 gcd 都是 0，并不无界。

结论：设 $d = gcd(|a_i - a_0|)$ ，并令 $f(k)$ 为所有“恰好选 k 个、允许重复”的和的 gcd。则 $f(k) = gcd(d, |k * a_0|)$ 。若 $d=0$ ，所有数相同： $a_0 \neq 0$ 时 $f(k) = |k * a_0|$ 无界； $a_0=0$ 时恒为 0。若 $d>0$ ，最大值为 d ，达成它的最小 k 是 $d/gcd(d, |a_0|)$ 。

```
struct SubsetGcdAnswer {
    bool infinite = false;
    i64 best = 0;
    i64 min_k = 0;
};

SubsetGcdAnswer max_subset_sum_gcd_with_repetition(const vector<i64>& a) {
    assert(!a.empty());
    i64 d = 0;
    for (i64 x : a) d = gcd(d, llabs(x - a[0]));
    if (d == 0) {
        if (a[0] == 0) return {false, 0, 1};
        return {true, 0, 0};
    }
    i64 g = gcd(d, llabs(a[0]));
    return {false, d, d / g};
}
```

【最小完整示例（先抄这一段就能跑）】

题目： $a = \{2, 6, 10\}$ ，每种数可重复选任意个，问所有大小为 k 的可重子集和的 gcd 的最大值，以及达成它的最小 k 。

```
// 构造：d = gcd(|6-2|, |10-2|) = 4, g = gcd(d, 2) = 2, 所以 best = d = 4, min_k = d/g = 2。
SubsetGcdAnswer ans = max_subset_sum_gcd_with_repetition({2, 6, 10});
cout << ans.best << ' ' << ans.min_k << '\n'; // 4 2: k=2 时首次达成最大 gcd=4
```

- 样例： $a = \{2, 6, 10\} \rightarrow$ 输出 4 2。

【传参要求（照这个传不会错）】

- `a`: `const vector<i64>&`, 非空; 元素可重复、可负、可为 0, 顺序无关。
- 返回值: `SubsetGcdAnswer`: `infinite` (`true` 表示 gcd 随 `k` 无界增长, 此时 `best=0`、`min_k` 无意义)、`best` (最大可达 gcd)、`min_k` (达成 `best` 的最小 `k`)。

固定方向长度 `k` 连线增量计数

赛时先看

题目信号: 五子棋、连珠、`k in a row`; 每次只新增一个点; 棋盘稀疏。

本质: 棋盘落子后, 维护横/竖/两条对角线方向上长度恰为 `k` 的连线数量。

接法: 把本节 `struct/class/函数组` 整段抄到 `solve()` 上面; `solve()`

里直接按下方“不会用就看这里”调用, 不需要自己猜内部参数。

复杂度判定: 每次 $O(k)$ 。

维护的量: `stones` (棋盘全部落子集合, 坐标用 `encode_cell` 编码); `current_total` (当前长度恰为 `k` 的连线总数); 四个方向增量表 `DX/DY`。

警告: 如果题意要求“极长连续段长度正好为 `k`”, 插入新点可能让旧连线失效, 要用 `after-before`; 如果只数长度为 `k` 的窗口, 把 `exact_segment=false`。

```
i64 encode_cell(int x, int y) {
    return (static_cast<i64>(x) << 32) ^ (unsigned int)y;
}

bool has_stone(const unordered_set<i64>& stones, int x, int y) {
    return stones.count(encode_cell(x, y));
}

int count_affected_lines(
    const unordered_set<i64>& stones,
    int x,
    int y,
    int k,
    bool exact_segment
) {
    static const int DX[4] = {1, 0, 1, 1};
    static const int DY[4] = {0, 1, 1, -1};
    int total = 0;
    for (int d = 0; d < 4; d++) {
        int from = exact_segment ? -k : -(k - 1);
        int to = 0;
        for (int start = from; start <= to; start++) {
            bool ok = true;
            for (int t = 0; t < k; t++) {
                int nx = x + (start + t) * DX[d];
                int ny = y + (start + t) * DY[d];
                if (!has_stone(stones, nx, ny)) {
                    ok = false;
                    break;
                }
            }
            if (!ok) continue;
            if (exact_segment) {
                int lx = x + (start - 1) * DX[d];
                int ly = y + (start - 1) * DY[d];
                int rx = x + (start + k) * DX[d];
                int ry = y + (start + k) * DY[d];
                if (has_stone(stones, lx, ly) || has_stone(stones, rx, ry)) continue;
            }
            total++;
        }
    }
    return total;
}

i64 add_stone_and_update_lines(
    unordered_set<i64>& stones,
    i64 current_total,
    int x,
    int y,
    int k = 5,
    bool exact_segment = true
) {
    int before = count_affected_lines(stones, x, y, k, exact_segment);
    stones.insert(encode_cell(x, y));
    int after = count_affected_lines(stones, x, y, k, exact_segment);
    return current_total + after - before;
}
```

【最小完整示例（先抄这一段就能跑）】

题目：五子棋模式连珠：空棋盘上依次在 $(0,0)$ $(1,0)$ $(2,0)$ 与 $(0,1)$ $(0,2)$ 落子， $k=3$ ，统计“极长段恰长 3”的连线数。

```
unordered_set<i64> stones;
i64 total = 0;
total = add_stone_and_update_lines(stones, total, 0, 0, 3);
total = add_stone_and_update_lines(stones, total, 1, 0, 3);
total = add_stone_and_update_lines(stones, total, 2, 0, 3); // 横线凑成, total = 1
total = add_stone_and_update_lines(stones, total, 0, 1, 3);
total = add_stone_and_update_lines(stones, total, 0, 2, 3); // 竖线也凑成, total = 2
cout << total << '\n'; // 2
```

- 样例：依次落 5 子后输出 2。

【传参要求（照这个传不会错）】

- `stones`: `unordered_set<i64>&`，存棋盘全部落子（坐标统一用 `encode_cell(x,y)` 编码）；函数内部会插入新点。
- `current_total`: `i64`，插入前的连线总数；返回值就是插入后的总数。
- `x, y`: `int`，新落子坐标，任意整数（原点自己定，全棋盘统一即可）。
- `k`: `int`，连线长度，默认 5。
- `exact_segment`: `bool`, `true` 只数“极长段恰为 k ”的连线（默认）；`false` 数“存在长为 k 子段”的连线。
- 返回值: `i64`，插入新点后长度满足要求的连线总数。

整数平方根

赛时先看

题目信号: n 接近 $1e18$ ，需要判断平方数或枚举到根号。

本质: 精确求 `floor(sqrt(n))`，避免 `double` 精度误差。

接法: 把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；`solve()` 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定: $O(1)$ 调整。

维护的量: 无额外结构；只维护近似根 x 并上下各调整一次。

警告: 判断 $(x+1)*(x+1)$ 可能溢出，用除法或 `i128`。

【最小完整示例（先抄这一段就能跑）】

```
// 求 n=1e18 的整数平方根，并判断 n 是否为完全平方数。
i64 n = 1000000000000000000LL;
i64 x = isqrt(n);
cout << x << '\n'; // 1000000000
cout << ((i128)x * x == n) << '\n'; // 1, n 是完全平方数
```

- 样例: $n = 1e18 \rightarrow$ 输出 1000000000 与 1。

【传参要求（照这个传不会错）】

- `n`: 非负 `i64`，可接近 $1e18$ 。
- 返回值: `floor(sqrt(n))` (`i64`)，即最大的 x 满足 $x*x \leq n$ 。

```
i64 isqrt(i64 n) {
    i64 x = sqrt((long double)n);
    while ((i128)(x + 1) * (x + 1) <= n) x++;
    while ((i128)x * x > n) x--;
    return x;
}
```

向下/向上整除

赛时先看

题目信号: 二分边界、数论不等式、坐标计算中可能出现负数。

本质: 处理负数除法、数学推式中的 `floor(a/b)` 和 `ceil(a/b)`。

接法: 把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；`solve()` 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定: $O(1)$ 。

维护的量: 无额外结构；纯函数实现数学意义的 `floor/ceil` 除法。

警告: C++ 整数除法是向 0 取整，不等于数学 `floor`。

【最小完整示例（先抄这一段就能跑）】


```
// 负数除法按数学意义取整，常用于二分边界与推式子。
cout << floor_div(-7, 2) << ' ' << ceil_div(-7, 2) << '\n';
cout << floor_div(7, -2) << ' ' << ceil_div(7, -2) << '\n';
```

- 样例：输出 -4 -3 与 -4 -3。

【传参要求（照这个传不会错）】

- a: 被除数，可为负。
- b: 除数， $b \neq 0$ ，可为负（内部自动统一符号）。
- 返回值：floor_div 返回数学下取整 $\text{floor}(a/b)$ ；ceil_div 返回数学上取整 $\text{ceil}(a/b)$ （均 i64）。

```
i64 floor_div(i64 a, i64 b) {
    assert(b != 0);
    if (b < 0) a = -a, b = -b;
    if (a >= 0) return a / b;
    return -((-a + b - 1) / b);
}

i64 ceil_div(i64 a, i64 b) {
    assert(b != 0);
    if (b < 0) a = -a, b = -b;
    if (a >= 0) return (a + b - 1) / b;
    return -((-a) / b);
}
```

常用 bit 内置函数

赛时先看

题目信号：状压、异或、按位统计。

本质：二进制状态、lowbit、位数、集合 DP。

接法：把本节 struct/class/函数组 整段抄到 solve() 上面；solve() 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定：通常 $O(1)$ 。

维护的量：无额外结构；一组针对整数的位运算包装函数。

警告：__builtin_clz(0) 未定义；64 位整数用 ll 后缀版本（popcountll、clzll）。

【最小完整示例（先抄这一段就能跑）】

```
// 统计 12 (二进制 1100) 的 1 个数、最低位与 floor(log2)。
cout << popcount_int(12) << '\n'; // 2
cout << lowbit(12) << '\n'; // 4
cout << floor_log2(12) << '\n'; // 3
```

- 样例：输出 2、4、3。

【传参要求（照这个传不会错）】

- lowbit(x): int; 返回最低位 1 的值 ($x=0$ 时返回 0)。
- popcount_int(x) / popcount_ll(x): 32 / 64 位整数；返回二进制中 1 的个数。
- floor_log2(x) / floor_log2_ll(x): 要求 $x > 0$ ；返回最大的 k 使 $2^k \leq x$ 。

```
int lowbit(int x) { return x & -x; }

int popcount_int(int x) { return __builtin_popcount((unsigned)x); }
int popcount_ll(i64 x) { return __builtin_popcountll((ui64)x); }

int floor_log2(unsigned int x) {
    assert(x > 0);
    return 31 - __builtin_clz(x);
}

int floor_log2_ll(ui64 x) {
    assert(x > 0);
    return 63 - __builtin_clzll(x);
}
```

位掩码基础：取位、置位、清位与最低位

赛时先看

题目信号：题目有少量开关、颜色、字母、人物、状态或特征；需要按位判断、合并、翻转，或用整数表示集合。

本质：用一个无符号整数表示大小不超过 64 的集合；第 b 位为 1 表示元素 b 被选中。

接法：有 $n \leq 60$ 个开关，每次翻转若干位，问某个状态是否出现过或合并后有多少种颜色。用 ui64 state 保存状态，合并是 |，共同拥有是 &，差异是 ^，计数是 __builtin_popcountll(state)；当 $n > 64$ 时改用 bitset

或动态位集。

复杂度判定：每个操作 $O(1)$ 。

维护的量：无额外结构；所有操作都是对 `ui64 mask` 的纯函数。

警告：掩码优先用 `ui64`，不要依赖负数的右移语义；写位常量要用 `1ULL << b`，而不是 `1 << b`；`~mask` 会把高位全变成 1，若只关心 `n` 位集合请写 `full_mask ^ mask`；`ctz(0)`、`clz(0)` 都未定义。

【最小完整示例（先抄这一段就能跑）】

```
// 维护 n<=60 个开关的状态，并统计已打开的开关数。
ui64 mask = set_bit(0, 2);           // 打开第 2 个
mask = set_bit(mask, 5);            // 打开第 5 个
mask = flip_bit(mask, 2);           // 翻转第 2 个
cout << __builtin_popcountll(mask) << '\n'; // 1
cout << has_bit(mask, 5) << '\n';      // 1
```

- 样例：输出 1 与 1。

【传参要求（照这个传不会错）】

- `bit_at(b)`: `b` in $[0, 64]$ ；返回 `1ULL << b`。
- `low_bits_mask(bits)`: `bits` in $[0, 64]$ ；返回低 `bits` 位全 1 的掩码。
- `has_bit(mask, b)`: 返回 `bool`；`set_bit / clear_bit / flip_bit(mask, b)`: 返回新掩码 `ui64`，`b` in $[0, 64]$ 。
- `lowbit(x)`: 最低位 1 的值；`erase_lowbit(x)`: 删除最低位 1。
- `set_bit_positions(mask)`: 从低到高返回所有为 1 的位，`vector<int>`。

【API / 入口函数（赛时只认这里列的名字）】

- `set_bit_positions(ui64 mask)` -> 从低到高返回所有为 1 的位（`mask` 为 0 时返回空）。返回 `vector<int>`。

```
ui64 bit_at(int b) {
    assert(0 <= b && b < 64);
    return 1ULL << b;
}

ui64 low_bits_mask(int bits) {
    assert(0 <= bits && bits <= 64);
    return bits == 64 ? ~0ULL : (1ULL << bits) - 1;
}

bool has_bit(ui64 mask, int b) { return (mask & bit_at(b)) != 0; }
ui64 set_bit(ui64 mask, int b) { return mask | bit_at(b); }
ui64 clear_bit(ui64 mask, int b) { return mask & ~bit_at(b); }
ui64 flip_bit(ui64 mask, int b) { return mask ^ bit_at(b); }

ui64 lowbit(ui64 x) { return x & -x; }           // 最低位的 1; lowbit(0)=0.
ui64 erase_lowbit(ui64 x) { return x & (x - 1); } // 删除最低位的 1.

// 从低到高枚举所有为 1 的位；循环中 mask 必须非零。
vector<int> set_bit_positions(ui64 mask) {
    vector<int> pos;
    while (mask) {
        int b = __builtin_ctzll(mask);
        pos.push_back(b);
        mask &= mask - 1;
    }
    return pos;
}
```

典题模型：有 $n \leq 60$ 个开关，每次翻转若干位，问某个状态是否出现过或合并后有多少种颜色。用 `ui64 state` 保存状态，合并是 `|`，共同拥有是 `&`，差异是 `^`，计数是 `__builtin_popcountll(state)`；当 $n > 64$ 时改用 `bitset` 或动态位集。

二进制集合：枚举子集、超集与集合拆分

赛时先看

题目信号：状态是

`mask`；转移形如“任选当前集合的一个子集”“把任务集合拆成两组”“枚举包含已选元素的所有集合”。这是状压 DP、Steiner Tree、子集卷积前最常见的循环。

本质：在给定集合 `mask` 内枚举所有子集，或枚举全集 $0..n-1$ 中包含 `mask` 的所有超集；也可将 `mask` 拆成两个互不相交部分。

接法：有不超过 20 个任务，`dp[mask]` 表示完成集合 `mask` 的最优值，转移需要选择 `mask` 的一个非空真子集 `sub` 先处理。用 `for_each_submask(mask, ...)`，跳过 `sub=0` 和 `sub=mask`；若转移把两个部分视为无序，把其中一半（如

`sub < (mask ^ sub)`) 跳过以免重复。

复杂度判定：单个 `mask` 的子集数是 $2^{\text{popcount}(\text{mask})}$ ；遍历所有 `mask` 的所有子集总复杂度是 $O(3^n)$ ；枚举超集总复杂度同样是 $O(3^n)$ 。

维护的量：无额外结构；三个模板函数各自维护循环变量 `sub` / `sup`。

警告：`sub = (sub - 1) & mask` 在 `sub=0` 时会回到 `mask`，所以包含空集的写法必须显式 `break`；超集循环里 `full = (1 << n) - 1`，用之前确保 $1 << n$ 不会爆 `int`；无序二分拆分会把 `(A,B)` 与 `(B,A)` 都枚举到，需要额外去重。

【最小完整示例（先抄这一段就能跑）】

```
// 枚举 mask=5 (101) 的所有子集，以及 3 位全集中包含第 0 位的所有超集。
for_each_submask(5, [&](int sub) { cout << sub << ' '; });
cout << '\n'; // 5 4 1 0
for_each_supermask(1, 3, [&](int sup) { cout << sup << ' '; });
cout << '\n'; // 1 3 5 7
```

- 样例：输出 5 4 1 0 与 1 3 5 7。

【传参要求（照这个传不会错）】

- `for_each_submask(mask, visit)`：按降序枚举 `mask` 的所有子集（含 0，回调 `visit(sub)`）。
- `for_each_supermask(mask, n, visit)`：在 `n` 位全集 `[0,n)` 中枚举包含 `mask` 的所有超集（回调 `visit(sup)`）；确保 $1 << n$ 不爆 `int`。
- `for_each_ordered_split(mask, visit)`：枚举有序二分 `(a, b)`，其中 `b = mask ^ a`、交集为空；若两侧无序要自己去重。

```
// 按降序枚举 mask 的所有子集，包括 0。
template <class F>
void for_each_submask(int mask, F&& visit) {
    for (int sub = mask; sub = (sub - 1) & mask; sub) {
        visit(sub);
        if (sub == 0) break;
    }
}

// 在 n 位全集 [0,n) 中枚举 mask 的所有超集。
template <class F>
void for_each_supermask(int mask, int n, F&& visit) {
    int full = (1 << n) - 1;
    for (int sup = mask; sup <= full; sup = (sup + 1) | mask) {
        visit(sup);
        if (sup == full) break;
    }
}

// 有序拆分：a 与 b 的并集为 mask，交集为空。
template <class F>
void for_each_ordered_split(int mask, F&& visit) {
    for_each_submask(mask, [&](int a) {
        int b = mask ^ a;
        visit(a, b);
    });
}
```

典型模型：有不超过 20 个任务，`dp[mask]` 表示完成集合 `mask` 的最优值，转移需要选择 `mask` 的一个非空真子集 `sub` 先处理。用 `for_each_submask(mask, ...)`，跳过 `sub=0` 和 `sub=mask`；若转移把两个部分视为无序，把其中一半（如 `sub < (mask ^ sub)`）跳过以免重复。

Gosper's Hack：固定选 k 个元素的所有集合

赛时先看

题目信号：组合大小固定，如“从 $n \leq 62$ 个点中恰好选 `k` 个”“一轮必须选固定数量机器/边/列”，并且每个组合都要单独计算。

本质：在 `n` 个位置中只枚举恰好选 `k` 个的掩码，避免枚举 2^n 个状态后再判断 `popcount`。

接法：`n <= 40` 个城市，必须选恰好 `k=5` 个建立中继站，给每个选择集合计算代价。用 `for (ui64 mask : masks_with_exactly_k_bits(n,k))`；需要访问第 `b` 个城市时判断 `(mask >> b) & 1ULL`。

复杂度判定：恰好枚举 $C(n,k)$ 个掩码，每次求后继 $O(1)$ 。

维护的量：无额外结构；`mask` 为当前枚举掩码，`limit = 1ULL << n` 是上界。

警告：下面代码限制 $0 \leq n \leq 62$ ，避免 $1ULL << 64$ 和后继溢出；`k=0` 是唯一的空集；固定大小组合远大于可承受范围时，Gosper 也救不了复杂度。

【最小完整示例（先抄这一段就能跑）】

```
// n=5 个点中恰好选 k=2 个，输出全部 C(5,2)=10 种选择掩码。
for (ui64 mask : masks_with_exactly_k_bits(5, 2)) {
    cout << mask << ' ';
}
cout << '\n';
```

- 样例：输出 3 5 6 9 10 12 17 18 20 24。

【传参要求（照这个传不会错）】

- `masks_with_exactly_k_bits(n, k)`: $0 \leq n \leq 62$, $0 \leq k \leq n$; 返回所有恰好 k 位为 1 且 $1 \leq k \leq n$ 的掩码（升序，`vector<ui64>`）； $k=0$ 时返回 `{0}`。
- 判断第 b 个元素是否被选：`(mask >> b) & 1ULL`。
- `next_same_popcount(mask)`: 内部求后继用，`mask != 0`；一般不用直接调。

```
ui64 next_same_popcount(ui64 mask) {
    // 要求 mask != 0; 返回下一个更大的、popcount 相同的整数。
    ui64 low = mask & -mask;
    ui64 raised = mask + low;
    return raised | (((mask ^ raised) >> 2) / low);
}

vector<ui64> masks_with_exactly_k_bits(int n, int k) {
    assert(0 <= n && n <= 62 && 0 <= k && k <= n);
    if (k == 0) return {0};
    ui64 limit = 1ULL << n;
    ui64 mask = (1ULL << k) - 1;
    vector<ui64> result;
    while (mask < limit) {
        result.push_back(mask);
        ui64 next = next_same_popcount(mask);
        if (next >= limit) break;
        mask = next;
    }
    return result;
}
```

典题模型： $n \leq 40$ 个城市，必须选恰好 $k=5$ 个建立中继站，给每个选择集合计算代价。用 `for (ui64 mask : masks_with_exactly_k_bits(n,k))`；需要访问第 b 个城市时判断 `(mask >> b) & 1ULL`。

Gray Code: 相邻状态只翻转一位

赛时先看

题目信号：要遍历所有子集，但从一个集合切到另一个集合时希望仅加入或删除一个元素；题目本身要求输出 Gray code；硬件/开关模拟。

本质：按顺序遍历所有 n 位掩码，保证相邻状态恰好只有一个 bit 不同；适合状态修改/回滚代价只与“翻转一个元素”有关的题。

接法：有 $n \leq 22$ 个元素，需要统计每个子集的函数值，而增删一个元素可以 $O(1)$ 更新。按 `all_gray_codes(n)` 遍历，维护前一个 `mask` 与当前 `mask` 的异或 `changed = prev ^ cur`, `__builtin_ctzll(changed)` 就是唯一改变的元素。

复杂度判定： 2^n 个状态，每个状态和相邻变化都可 $O(1)$ 计算。

维护的量：无额外结构；只有编号 `index` 与灰码 `gray_code(index)` 的对应。

警告：这是遍历顺序技巧，不会降低 2^n 的总状态数； n 要保证 `1ULL << n` 合法；从编号 $i-1$ 到 i 时翻转的位置是 `ctz(i)`，因为 `gray(i) = i ^ (i >> 1)`。

【最小完整示例（先抄这一段就能跑）】

```
// n=3 时输出全部 8 个 Gray Code。
auto order = all_gray_codes(3);
for (ui64 g : order) cout << g << ' ';
cout << '\n';
```

- 样例：输出 0 1 3 2 6 7 5 4。

【传参要求（照这个传不会错）】

- `all_gray_codes(n)`: $0 \leq n \leq 62$; 返回 2^n 个 n 位灰码的完整序列 (`vector<ui64>`)。
- `gray_code(index)`: `index >= 0`; 返回编号 `index` 对应的灰码 `index ^ (index >> 1)`。
- 相邻状态只差一位：翻转的位置是 `__builtin_ctzll(prev ^ cur)`。

【API / 入口函数（赛时只认这里列的名字）】

- `all_gray_codes(int n) ->` 枚举所有 n 位 Gray Code；要求 n 在 $[0, 62]$ 。返回 `vector<ui64>`。

```

ui64 gray_code(ui64 index) {
    return index ^ (index >> 1);
}

// 枚举所有 n 位 Gray Code; 要求 n 在 [0,62]。
vector<ui64> all_gray_codes(int n) {
    assert(0 <= n && n <= 62);
    vector<ui64> order;
    order.reserve(1ULL << n);
    for (ui64 i = 0; i < (1ULL << n); ++i) order.push_back(gray_code(i));
    return order;
}

```

典型模型：有 $n \leq 22$ 个元素，需要统计每个子集的函数值，而增删一个元素可以 $O(1)$ 更新。按 `all_gray_codes(n)` 遍历，维护前一个 `mask` 与当前 `mask` 的异或 `changed = prev ^ cur`，`__builtin_ctzll(changed)` 就是唯一改变的元素。

前缀异或： $O(1)$ 区间 XOR 与 0 XOR 子数组计数

赛时先看

题目信号：题面出现连续子数组 xor、区间异或、两段前缀异或相等、找 xor 为 0 或固定值 K 的子数组个数。

本质：异或对应“加法”，逆运算仍是自己。前缀异或可在 $O(1)$ 查询区间 XOR，也能计数异或值满足条件的子数组。

接法：给数组 `a`，问 xor 恰为 K 的连续子数组数量。将输入放进 1-indexed `vector<ui64>`，直接输出 `count_subarrays_with_xor(a,K)`；当 $K=0$ 时，等价于统计相等前缀异或对。

复杂度判定：预处理和单次区间查询 $O(n)$ / $O(1)$ ；哈希计数 $O(n)$ 期望。

维护的量：`pre[i]`（前 i 个元素的异或前缀，1-indexed）；`frequency`（哈希表，各前缀异或值的出现次数）。

警告：区间 $[l,r]$ 的 xor 是 `pre[r] ^ pre[l-1]`；统计固定 xor K 时，在加入当前前缀前先累加此前 `pre ^ K` 的出现次数；答案可能到 $O(n^2)$ ，用 `i64`。

约定：`a` 使用 1-indexed，`n = (int)a.size() - 1`。

【最小完整示例（先抄这一段就能跑）】

```

// a = [_, 1, 2, 3] (1-indexed)，求区间 [2,3] 的 xor 与 xor 恰为 0 的子数组数。
vector<ui64> a = {0, 1, 2, 3};
auto pre = build_prefix_xor(a);
cout << range_xor(pre, 2, 3) << '\n'; // 2^3 = 1
cout << count_subarrays_with_xor(a, 0) << '\n'; // [1,3] 一个

```

- 样例：输出 1 与 1。

【传参要求（照这个传不会错）】

- `a`: 1-indexed，`a[0]` 是占位符；`n = a.size() - 1`。
- `build_prefix_xor(a)`: 返回长度 `n+1` 的 `pre`，`pre[0] = 0`。
- `range_xor(pre, l, r)`: 闭区间， $1 \leq l \leq r \leq n$ ；返回 `pre[r] ^ pre[l-1]`。
- `count_subarrays_with_xor(a, target)`: 统计 xor 恰为 `target` 的连续子数组个数（`i64`，最大 $O(n^2)$ ）。

```

vector<ui64> build_prefix_xor(const vector<ui64>& a) {
    int n = (int)a.size() - 1; // a 使用 1-indexed。
    vector<ui64> pre(n + 1, 0);
    for (int i = 1; i <= n; ++i) pre[i] = pre[i - 1] ^ a[i];
    return pre;
}

ui64 range_xor(const vector<ui64>& pre, int l, int r) {
    return pre[r] ^ pre[l - 1];
}

i64 count_subarrays_with_xor(const vector<ui64>& a, ui64 target) {
    unordered_map<ui64, i64> frequency;
    frequency.reserve(a.size() * 2 + 1);
    frequency.max_load_factor(0.7F);
    ui64 pre = 0;
    i64 answer = 0;
    frequency[0] = 1;
    for (int i = 1; i < (int)a.size(); ++i) {
        pre ^= a[i];
        auto it = frequency.find(pre ^ target);
        if (it != frequency.end()) answer += it->second;
        ++frequency[pre];
    }
    return answer;
}

```

典题模型：给数组 `a`，问 `xor` 恰为 `k` 的连续子数组数量。将输入放进 1-indexed `vector<ui64>`，直接输出 `count_subarrays_with_xor(a,k)`；当 `k=0` 时，等价于统计相等前缀异或对。

静态区间 OR / AND: Sparse Table

赛时先看

题目信号：没有修改，询问很多；每次询问 `[l,r]`

的所有数按位或/按位与；或二分一个区间端点时需要快速判断该区间的 OR/AND。

本质：静态数组上反复查询一个区间的按位 OR 或按位 AND。它们和 `min/max/gcd`

一样满足幂等性，所以可用两段有重叠的 ST 表在 `O(1)` 查询；不能用前缀和相减，OR/AND 没有一般意义的逆运算。

接法：有 `q` 次静态区间询问，求 `[l,r]` 的 OR，或找最短/最长区间使区间 OR 达到某个掩码。先 `BitwiseSparseTable st(a)`，再用 `st.query_or(l,r)`；若条件随区间扩张单调，可在外层二分端点。

复杂度判定：预处理 `O(n log n)`，单次查询 `O(1)`，空间 `O(n log n)`。

维护的量：`n`（长度）；`lg[i]`（`floor(log2(i))`）；`st_or[k][l]` / `st_and[k][l]`（从 `l` 起长 `2^k` 的区间 OR / AND）。

警告：数组以下标 `0` 开始；这里查询是闭区间 `[l,r]`；OR 和 AND 都能把重叠两段再合并，但 XOR 不满足幂等性，区间 XOR 应使用前缀异或或不重叠 ST 表。

【最小完整示例（先抄这一段就能跑）】

```
// a = {1, 6, 8, 12}, 静态查询区间 OR 与区间 AND.
BitwiseSparseTable st(a);
cout << st.query_or(1, 3) << '\n'; // 6|8|12 = 14
cout << st.query_and(0, 2) << '\n'; // 1&6&8 = 0
```

- 样例：输出 14 与 0。

【传参要求（照这个传不会错）】

- 构造 `BitwiseSparseTable(a)`：`a` 为 `vector<ui64>`，0-indexed。
- `query_or(l, r)` / `query_and(l, r)`：闭区间 `[l,r]`，`0 ≤ l ≤ r < n`；分别返回区间按位 OR / AND（`ui64`）。

```
struct BitwiseSparseTable {
    int n = 0;
    vector<int> lg;
    vector<vector<ui64>> st_or, st_and;

    explicit BitwiseSparseTable(const vector<ui64>& a) {
        n = (int)a.size();
        lg.assign(n + 1, 0);
        for (int i = 2; i <= n; ++i) lg[i] = lg[i >> 1] + 1;
        int levels = n ? lg[n] + 1 : 0;
        st_or.assign(levels, vector<ui64>(n));
        st_and.assign(levels, vector<ui64>(n));
        if (!n) return;
        st_or[0] = st_and[0] = a;
        for (int k = 1; k < levels; ++k) {
            int len = 1 << k;
            for (int l = 0; l + len <= n; ++l) {
                st_or[k][l] = st_or[k - 1][l] | st_or[k - 1][l + (len >> 1)];
                st_and[k][l] = st_and[k - 1][l] & st_and[k - 1][l + (len >> 1)];
            }
        }
    }

    ui64 query_or(int l, int r) const {
        assert(0 <= l && l <= r && r < n);
        int k = lg[r - l + 1];
        return st_or[k][l] | st_or[k][r - (1 << k) + 1];
    }

    ui64 query_and(int l, int r) const {
        assert(0 <= l && l <= r && r < n);
        int k = lg[r - l + 1];
        return st_and[k][l] & st_and[k][r - (1 << k) + 1];
    }
};
```

典题模型：有 `q` 次静态区间询问，求 `[l,r]` 的 OR，或找最短/最长区间使区间 OR 达到某个掩码。先 `BitwiseSparseTable st(a)`，再用 `st.query_or(l,r)`；若条件随区间扩张单调，可在外层二分端点。

01 Trie: 统计异或严格小于 K 的数对

赛时先看

题目信号：题面要求 $a[i] \text{ xor } a[j] < K$ 、 $\leq K$ 、异或值排名/第 k

小，或需要按顺序处理前缀中的异或阈值对。普通最大异或 Trie

只能贪心选相反位，出现比较符号时要改用本节的“贴着 K 走”的查询。

本质：在线插入整数，统计此前有多少个数满足 $x \text{ xor } y <$

K ；据此求数组中满足异或阈值条件的无序数对数，也可作为“第 k 小两数异或”二分答案的判定器。

接法：求第 k 小的 $a[i] \text{ xor } a[j]$ ($i < j$)。答案具有单调性： $\text{count_pairs_xor_leq}(a, \text{mid}) \geq k$ 时第 k 小不大于 mid ，对 mid 在无符号范围二分；若题目仅问异或小于 K 的对数，直接输出 $\text{count_pairs_xor_less}(a, K)$ 。

复杂度判定：插入和单次计数均为 $O(B)$ ， $B=64$ ；总计 $O(nB)$ ，空间 $O(nB)$ 最坏。

维护的量： tr （节点数组，每个节点含 $\text{ch}[2]$ 两个孩子与 cnt 子树计数）； $\text{LOG}=63$ （从最高位 63 向下插入）。

警告：代码统计的是严格小于 limit ；要统计 $\leq K$ 应调用 $\text{count_pairs_xor_leq}$ ，其中 $K=\sim 0\text{ULL}$ 需单独处理以免 $K+1$ 溢出；数值按无符号 64 位解释，若题目给的是非负 long long 可直接转成 ui64 。

【最小完整示例（先抄这一段就能跑）】

```
// a = {1, 2, 3}, 统计异或严格小于 2 与不超过 1 的无序数对数。
vector<ui64> a = {1, 2, 3};
cout << count_pairs_xor_less(a, 2) << '\n'; // 1 2^3=1 < 2
cout << count_pairs_xor_leq(a, 1) << '\n'; // 1 2^3=1 <= 1
```

- 样例：输出 1 与 1。

【传参要求（照这个传不会错）】

- XorLessTrie trie : 先构造对象； $\text{trie.add}(x)$ 插入一个 ui64 ； $\text{trie.count_xor_less}(x, \text{limit})$ 统计已插入的 y 中满足 $(x \text{ xor } y) < \text{limit}$ 的个数（ ui64 ，严格小于）。
- $\text{count_pairs_xor_less}(a, \text{limit})$: 整组 a ($\text{vector}<\text{ui64}>$)，返回无序对数 ($i < j$) 中异或严格小于 limit 的个数。
- $\text{count_pairs_xor_leq}(a, \text{limit})$: 统计 $\leq \text{limit}$ ； $\text{limit} == \sim 0\text{ULL}$ 时直接返回全部对数。

【API / 入口函数（赛时只认这里列的名字）】

- $\text{add}(\text{ui64 } x) \rightarrow$ 插入一个数 x
- $\text{count_xor_less}(\text{ui64 } x, \text{ui64 } \text{limit}) \rightarrow$ 统计已插入的 y 中满足 $(x \text{ xor } y) < \text{limit}$ 的个数。返回 ui64 。

```
struct XorLessTrie {
    static constexpr int LOG = 63;

    struct Node {
        int ch[2] = {-1, -1};
        int cnt = 0;
    };

    vector<Node> tr{Node{}};

    void add(ui64 x) {
        int u = 0;
        ++tr[u].cnt;
        for (int b = LOG; b >= 0; --b) {
            int c = (x >> b) & 1ULL;
            if (tr[u].ch[c] == -1) {
                tr[u].ch[c] = (int)tr.size();
                tr.push_back(Node{});
            }
            u = tr[u].ch[c];
            ++tr[u].cnt;
        }
    }

    // 统计已插入的 y 中满足 (x xor y) < limit 的个数。
    ui64 count_xor_less(ui64 x, ui64 limit) const {
        int u = 0;
        ui64 answer = 0;
        for (int b = LOG; b >= 0 && u != -1; --b) {
            int xb = (x >> b) & 1ULL;
            int kb = (limit >> b) & 1ULL;
            if (kb) {
                int same = tr[u].ch[xb]; // 这一位让 xor 取 0，小于 limit 的 1。
                if (same != -1) answer += tr[same].cnt;
                u = tr[u].ch[xb ^ 1]; // 保持当前前缀仍等于 limit。
            } else {
                u = tr[u].ch[xb]; // 这一位必须让 xor 取 0。
            }
        }
        return answer;
    }
};
```

```

i64 count_pairs_xor_less(const vector<ui64>& a, ui64 limit) {
    XorLessTrie trie;
    i64 answer = 0;
    for (ui64 x : a) {
        answer += trie.count_xor_less(x, limit);
        trie.add(x);
    }
    return answer;
}

i64 count_pairs_xor_leq(const vector<ui64>& a, ui64 limit) {
    if (limit == ~0ULL) {
        i64 n = (i64)a.size();
        return n * (n - 1) / 2;
    }
    return count_pairs_xor_less(a, limit + 1);
}

```

典型模型：求第 k 小的 $a[i] \text{ xor } a[j]$ ($i < j$)。答案具有单调性： $\text{count_pairs_xor_leq}(a, \text{mid}) \geq k$ 时第 k 小不大于 mid ，对 mid 在无符号范围二分；若题目仅问异或小于 k 的对数，直接输出 $\text{count_pairs_xor_less}(a, k)$ 。

按位贪心：最大两数按位 AND

赛时先看

题目信号：题目出现“选两个数”“最大 $a[i] \& a[j]$ ”“最大公共置位集合”；有时会扩展成至少选 k 个数，其判定条件就是“满足候选掩码的数至少有 k 个”。

本质：求数组中任意两个不同元素按位 AND 的最大值。逐位从高到低尝试把答案该位置为

1，只要仍有至少两个数包含当前候选掩码即可保留。

接法：给 n 个非负数，选两个数使 AND 最大。直接调用 $\text{maximum_pairwise_and}(a)$ ；若要求选 k 个数的 AND 最大，把代码中 $++\text{count} == 2$ 与 $\text{count} \geq 2$ 都改成 k 。

复杂度判定： $O(nB)$ ，这里 $B=64$ ；额外空间 $O(1)$ 。

维护的量： answer （已确定的高位答案）与 candidate （本轮尝试把第 b 位置 1 的候选掩码）。

警告：这是求数值最大，必须从最高位往最低位试；不要每一位独立选最大出现次数；题目要求 $n \geq 2$ ，并且应使用 ui64 ，避免第 63 位的有符号移位问题。

【最小完整示例（先抄这一段就能跑）】

```

// a = {3, 5, 7}，选两个数使 AND 最大。
vector<ui64> a = {3, 5, 7};
cout << maximum_pairwise_and(a) << '\n'; // 5 5 & 7 = 5

```

- 样例：输出 5。

【传参要求（照这个传不会错）】

- $\text{maximum_pairwise_and}(a)$ ：入参 a （ $\text{vector}<\text{ui64}>$ ，非负数， $\text{size}() \geq 2$ ）；返回 ui64 ，最大两数 AND 值。
- 要选 k 个数：把代码里 $++\text{count} == 2$ 与 $\text{count} \geq 2$ 中的 2 改成 k 。

```

ui64 maximum_pairwise_and(const vector<ui64>& a) {
    assert(a.size() >= 2);
    ui64 answer = 0;
    for (int b = 63; b >= 0; --b) {
        ui64 candidate = answer | (1ULL << b);
        int count = 0;
        for (ui64 x : a) {
            if ((x & candidate) == candidate && ++count == 2) break;
        }
        if (count >= 2) answer = candidate;
    }
    return answer;
}

```

典型模型：给 n 个非负数，选两个数使 AND 最大。直接调用 $\text{maximum_pairwise_and}(a)$ ；若要求选 k 个数的 AND 最大，把代码中 $++\text{count} == 2$ 与 $\text{count} \geq 2$ 都改成 k 。

子数组按位 OR / AND：不同结果集合

赛时先看

题目信号：题目问“不同子数组 OR/AND 的数量”“所有子数组按位或/与的值集合”“是否存在 OR/AND 恰为某数”；数组元素是非负整数，且不能枚举 $O(n^2)$ 个子数组。

本质：统计所有连续子数组的不同 OR 值或不同 AND 值。固定右端点时，所有后缀的 OR 只会不断把 1 位变多，AND 只会不断把 1 位变少，所以不同结果数量至多 $B+1$ 个。

接法：数组长度 $n=2e5$ ，求所有子数组按位 OR 的不同结果数。不要枚举左右端点；直接输出 `count_distinct_subarray_or(a)`。若题目问某个值 K 是否出现，可把全局 `unordered_set` 保留并查询 `contains(K)` (C++17 写 `find(K) != end()`)。

复杂度判定：每个右端点最多保留 $B+1$ 个值；本实现为 $O(n B \log B)$ ，在 64

位数下近似线性，空间为答案集合大小。若只问存在性或计数，也可以把 `all_values` 改成相应的聚合结构。

维护的量：`previous`（固定右端点时的所有不同后缀结果，已排序去重）与 `all_values`（全局不同结果集合）。

警告：这个“不同后缀结果只有 $O(B)$ 个”的结论对 OR/AND 成立，对 XOR 不成立；不同子数组 XOR 的个数可以达到 $\Theta(n^2)$ 。代码每轮排序去重，不容易因为维护顺序出错。

【最小完整示例（先抄这一段就能跑）】

```
// a = {1, 2, 3}, 统计不同子数组 OR / AND 的结果个数。
vector<ui64> a = {1, 2, 3};
cout << count_distinct_subarray_or(a) << '\n'; // 3 ({1, 2, 3, 3, 3} 去重)
cout << count_distinct_subarray_and(a) << '\n'; // 4 ({1, 2, 3, 0, 2, 0} 去重)
```

- 样例：输出 3 与 4。

【传参要求（照这个传不会错）】

- `count_distinct_subarray_or(a)` / `count_distinct_subarray_and(a)`：入参 `a` (`vector<ui64>`，非负数)；返回 `ui64`，不同结果个数。
- 问某个值 K 是否出现：把全局集合 `all_values` 保留下来并查询 `contains(K)` (C++17 写 `find(K) != end()`)。

```
template <class Combine>
i64 count_distinct_subarray_bitwise_values(const vector<ui64>& a, Combine combine) {
    vector<ui64> previous;
    unordered_set<ui64> all_values;
    all_values.reserve(a.size() * 8 + 1);
    all_values.max_load_factor(0.7F);

    for (ui64 x : a) {
        vector<ui64> current;
        current.reserve(previous.size() + 1);
        current.push_back(x);
        for (ui64 value : previous) current.push_back(combine(value, x));
        sort(current.begin(), current.end());
        current.erase(unique(current.begin(), current.end()), current.end());
        for (ui64 value : current) all_values.insert(value);
        previous.swap(current);
    }
    return (i64)all_values.size();
}

i64 count_distinct_subarray_or(const vector<ui64>& a) {
    return count_distinct_subarray_bitwise_values(
        a, [](ui64 x, ui64 y) { return x | y; }
    );
}

i64 count_distinct_subarray_and(const vector<ui64>& a) {
    return count_distinct_subarray_bitwise_values(
        a, [](ui64 x, ui64 y) { return x & y; }
    );
}
```

典型模型：数组长度 $n=2e5$ ，求所有子数组按位 OR 的不同结果数。不要枚举左右端点；直接输出 `count_distinct_subarray_or(a)`。若题目问某个值 K 是否出现，可把全局 `unordered_set` 保留并查询 `contains(K)` (C++17 写 `find(K) != end()`)。

按位贡献：所有无序数对的 XOR / OR / AND 和

赛时先看

题目信号：出现“所有数对”“按位运算结果之和”“分别求 XOR/OR/AND

总和”，且答案不取模或模数允许最后取模。遇到这类题，不要枚举 $O(n^2)$ 个数对。

本质：快速求 $\sum_{i<j}(a[i] \text{ xor } a[j])$ 、 $\sum_{i<j}(a[i] | a[j])$ 、 $\sum_{i<j}(a[i] \& a[j])$ 。每一位彼此独立，只需统计该位为 1 的个数。

接法：对每组数组输出全部无序数对 OR 的和。写 `auto result = sum_pairwise_bitwise(a)`，然后输出 `result.or_sum`；若题目要求模 `MOD`，可以在每次加法后取模，或最后把 `i128` 对 `MOD` 取模。

复杂度判定： $O(nB)$ ，空间 $O(1)$ 。

维护的量：PairwiseBitwiseSums (xor_sum / or_sum / and_sum 三个 i128 累加器) 与每位的 ones (该位为 1 的个数)。

警告：XOR 的该位贡献是 ones * zeros; OR 是全部数对减去两端都是 0 的数对; AND 是两端都是 1 的数对。总和可能超过 i64, 这里使用 i128, 输出时搭配本章 01 小节的 i128 输入输出重载。

【最小完整示例 (先抄这一段就能跑)】

```
// a = {1, 2, 3}, 求所有无序数对的 XOR / OR / AND 总和。
vector<ui64> a = {1, 2, 3};
auto result = sum_pairwise_bitwise(a);
cout << result.xor_sum << '\n'; // 6 1^2 + 1^3 + 2^3 = 3+2+1
cout << result.or_sum << '\n'; // 9 1|2 + 1|3 + 2|3 = 3+3+3
cout << result.and_sum << '\n'; // 3 1&2 + 1&3 + 2&3 = 0+1+2
```

- 样例：输出 6、9 与 3。

【传参要求 (照这个传不会错)】

- sum_pairwise_bitwise(a): 入参 a (vector<ui64>, 非负数); 返回 PairwiseBitwiseSums, 字段 xor_sum / or_sum / and_sum 均为 i128, 直接取字段输出。
- 输出 i128: 需要先抄本章「i128 输入输出」小节的输出重载。
- 要取模: 每次加法后对 MOD 取模, 或最后把 i128 结果对 MOD 取模。

```
struct PairwiseBitwiseSums {
    i128 xor_sum = 0;
    i128 or_sum = 0;
    i128 and_sum = 0;
};

PairwiseBitwiseSums sum_pairwise_bitwise(const vector<ui64>& a) {
    i128 n = (i128)a.size();
    i128 all_pairs = n * (n - 1) / 2;
    PairwiseBitwiseSums answer;
    for (int b = 0; b < 64; ++b) {
        i128 ones = 0;
        for (ui64 x : a) ones += (x >> b) & 1ULL;
        i128 zeros = n - ones;
        i128 bit_value = (i128)1 << b;
        answer.xor_sum += ones * zeros * bit_value;
        answer.or_sum += (all_pairs - zeros * (zeros - 1) / 2) * bit_value;
        answer.and_sum += ones * (ones - 1) / 2 * bit_value;
    }
    return answer;
}
```

典题模型: 对每组数组输出全部无序数对 OR 的和。写 auto result = sum_pairwise_bitwise(a);, 然后输出 result.or_sum; 若题目要求模 MOD, 可以在每次加法后取模, 或最后把 i128 对 MOD 取模。

OR / AND / XOR 卷积与子集变换: 快速定位

赛时先看

题目信号: 数组长度是 2^m (通常 $m \leq 20$); 需要统计所有有序对/两组选法合并后的掩码, 或出现 $A \mid B = S$ 、 $A \& B = S$ 、 $A \oplus B = S$ 、 $T \text{ subset } S$ 等精确关系。

本质: 当状态下标是掩码 S , 并且两个状态按 $|\cdot|$ 、 $\&$ 、 $\wedge$ 合并时, 用相应 FWT/FWHT 把朴素 $O(4^m)$ 的卷积降为 $O(m \cdot 2^m)$; 当题目求所有子集/超集的贡献和时用 SOS DP。

接法: 有两个掩码多重集合, 问任选一个掩码后其 OR 恰为 S 的有序对数量。把频率写入长度 2^m 的数组 a, b , 调用 E 章 11 的 or_convolution(a, b, mod); 若合并符号改成 XOR, 转到同节的 xor_convolution。

复杂度判定: FWT / FWHT / SOS DP 均为 $O(m \cdot 2^m)$, 空间 $O(2^m)$ 。

维护的量: 无本地结构; 对象是各章的频率数组 a, b (长度 2^m , $a[S]$ = 掩码 S 出现次数) 与对应的变换函数 (fwt_or / fwt_and / XOR FWT / SOS DP)。

警告: 这是掩码数组的整体变换, 不是普通区间 OR/AND 查询; OR 与 AND 的正反变换方向不同, XOR 逆变换还需要乘长度的逆元。完整可抄代码不要重复粘在本节, 直接翻下列位置:

【最小完整示例 (先抄这一段就能跑)】

```
// 两集合掩码频率 a, b, 问任选一个掩码后 OR 恰为 S 的有序对数量 (E 章 11 抄 or_convolution)。
int m = 3; // 掩码位数, 数组长度 2^m
vector<i64> a(1 << m, 0), b(1 << m, 0);
a[1] = 2; b[2] = 1; // 填频率: 掩码 1 出现 2 次、掩码 2 出现 1 次
auto conv = or_convolution(a, b, MOD);
cout << conv[3] << '\n'; // 2 (1|2 = 3 的有序对: 2*1)
```

- 样例：输出 2。

【传参要求（照这个传不会错）】

- `or_convolution(a, b, mod)`: a/b 为长度 2^m 的频率数组 (`vector<i64>`, $a[S]$ = 掩码 S 出现次数), `mod` 为模数; 返回 `vector<i64>` 卷积结果, 下标 S 即合并掩码。
- `and_convolution(a, b, mod)` / `xor_convolution(a, b, mod)`: 签名同上, 合并符号分别为 `&`、`^`。
- 底层变换: `fwf_or(a, mod, inverse)` / `fwf_and(a, mod, inverse)` 原地操作, `inverse=false` 正变换、`true` 逆变换; XOR 逆变换注意乘长度的逆元。
- $A \mid B = S$: E 章 11「FWT: AND / OR 卷积」中的 `fwf_or`。
- $A \& B = S$: E 章 11「FWT: AND / OR 卷积」中的 `fwf_and`。
- $A \text{ xor } B = S$: E 章 11「FWHT: XOR 卷积」。
- `sum f[T]`, $T \text{ subset } S$ 或 $S \text{ subset } T$: F 章 13「SOS DP: 子集和变换」。

典题模型: 有两个掩码多重集合, 问任选一个掩码后其 OR 恰为 S 的有序对数量。把频率写入长度 2^m 的数组 a, b , 调用 E 章 11 的 `or_convolution(a, b, mod)`; 若合并符号改成 XOR, 转到同节的 `xor_convolution`。

bitset 01 背包: 只问哪些和可达

赛时先看

题目信号: 每个物品至多选一次; 问子集和、能否分成相等两组、最接近目标但不超过目标的和; 重量上界较大但仍能开位集 (常见 $1e5 \sim 1e7$, 取决于内存和时限)。

本质: 0/1 选择若干非负重量, 只关心某个和能否凑出、最大不超过上界的可达和, 而不关心方案数或最大价值。用一个位集并行模拟整个布尔 DP。

接法: 给 n 个正整数, 问是否能选一些数使和恰为 `target`, 或把数组分成两组且和最接近。若总和 `sum <= S`, 取 `auto reachable = subset_sum_01_bitset<S>(w)`; 等分判断是 `sum` 偶数且 `reachable[sum/2]`。

复杂度判定: 若上界为 S , 时间 $O(nS / \text{word_size})$, 空间 $O(S)$ bit。C++ `bitset` 的长度必须是编译期常量。

维护的量: `reachable (bitset<MAX_SUM+1>, reachable[s] = 1` 表示和 s 可达; `reachable[0]` 恒为 1)。

警告: `reachable |= reachable << w` 对应 0/1 背包; 完全背包不能直接照抄; w 必须非负且不超过

`MAX_SUM`; `std::bitset` 不是动态长度, 运行时上界用 `vector<ui64>` 或其他动态位集。

【最小完整示例（先抄这一段就能跑）】

```
// weights = {2, 3, 5}, 问能否凑出 10, 并求不超过 9 的最大可达和。
constexpr int S = 1000; // 编译期上界, 取 sum 或题给上界
vector<int> weights = {2, 3, 5};
auto reachable = subset_sum_01_bitset<S>(weights);
cout << reachable[10] << '\n'; // 1 (2+3+5=10 可达)
for (int s = 9; s >= 0; --s)
    if (reachable[s]) { cout << s << '\n'; break; } // 8 3+5=8
```

- 样例: 输出 1 与 8。

【传参要求（照这个传不会错）】

- `subset_sum_01_bitset<MAX_SUM>(weight)`: 模板参数 `MAX_SUM` 是编译期上界 (总和 `sum <= MAX_SUM` 即可, 最大取约 $1e7$); 入参 `weight (vector<int>`, 每个 $0 \leq w \leq \text{MAX_SUM}$); 返回 `bitset<MAX_SUM + 1>`, `reachable[s]` 为 1 表示和 s 可达。
- 等分判断: `sum` 为偶数且 `reachable[sum / 2]`。
- 完全背包不能照抄 `reachable |= reachable << w`; 运行时才知道上界时改用动态位集。

```
template <int MAX_SUM>
bitset<MAX_SUM + 1> subset_sum_01_bitset(const vector<int>& weight) {
    bitset<MAX_SUM + 1> reachable;
    reachable[0] = 1;
    for (int w : weight) {
        assert(0 <= w && w <= MAX_SUM);
        reachable |= reachable << w;
    }
    return reachable;
}

// 示例:
// 示例: constexpr int S = 200000;
// 示例: auto reachable = subset_sum_01_bitset<S>(weights);
// 示例: bool can_make_target = reachable[target];
// 示例: int best = -1;
// 示例: for (int s = target; s >= 0; --s) if (reachable[s]) { best = s; break; }
```

典题模型：给 n 个正整数，问是否能选一些数使和恰为 $target$ ，或把数组分成两组且和最接近。若总和 $sum \leq S$ ，取 `auto reachable = subset_sum_01_bitset<S>(w)`；等分判断是 sum 偶数且 `reachable[sum/2]`。

整数二分：最小可行值

赛时先看

题目信号：题面出现“最小的最大值”“最早时间”“至少多少”；判定 `check(x)` 随 x 单调（ x 越大越容易可行）。这类“答案单调 + 可判定”直接二分。

本质：把“求最优答案”转成“猜一个答案 mid ，验证是否可行”；单调性保证猜的方向唯一，二分把 O （答案范围）压缩成 $O(\log V)$ 次判定。

复杂度判定： $O(\log V * \text{check})$ ； V 是答案范围（ $1e18$ 也没问题）；若 `check` 本身 $O(n)$ ， $n \leq 2e5$ 时约 30-60 次判定可过；若题目不单调，不能二分，改枚举或三分。

维护的量：无额外结构；只维护二分边界 l/r 与判定函数 `check`。

接法：把 `check(mid)` 写成题目的可行性判定，然后一行 `min_true(1, r, check)`。

警告：必须先证明 `check(x)` 单调；`mid = 1 + (r-1)/2` 防溢出。

【抄板清单（照着做就行）】

1. 抄哪段：整个 `min_true` 模板函数。
2. 构造：把 l 设成“一定不可行”， r 设成“一定可行”。
3. 调用：`i64 ans = min_true(1, r, check)`；
4. 取结果：`ans` 就是最小可行值，直接输出。

【改造点（按题目改这几处）】

- 判定函数：把 `check` 换成题目的可行性判定（这是唯一要写的内容）。
- 边界： l/r 从题面数据范围取，通常 $l = 0$ 或 1 ， $r = 1e18$ 或已知上界。
- 返回类型：答案不是 `i64` 时改模板参数或外面转。
- 要找“最大可行值”：用下一节 `max_true`（上取中位数，防死循环）。

```
// 维护的量：二分边界 l/r 与判定函数 check (check(x) 表示 x 是否可行)。
// 不变量：l 一定不可行、r 一定可行；区间长度每次减半，结束时 l = r 即最小可行值。
template <class Check>
i64 min_true(i64 l, i64 r, Check check) {
    while (l < r) {
        i64 mid = 1 + (r - l) / 2; // 下取中位数，l + r 不会溢出。
        if (check(mid)) r = mid; // mid 可行，答案落在 [l, mid]。
        else l = mid + 1; // mid 不可行，答案落在 (mid, r]。
    }
    return l;
}
```

【最小完整示例（先抄这一段就能跑）】

题目：速度 $v=3$ ，距离 $d=100$ ，求最早到达（ $3*t \geq 100$ ）的最小整数秒数。

```
// check(t): t 秒能走 3t，够 100 就可行；l=0 一定不可行，r=1e18 一定可行。
auto check = [](i64 t) { return 3LL * t >= 100; };
i64 ans = min_true(0, 10000000000000000LL, check);
cout << ans << '\n'; // 34: 3*33=99 不够，3*34=102 够
```

- 样例：输出 34。

【传参要求（照这个传不会错）】

- l : `i64`，一定不可行的下界（含），一般取 0 或 1 。
- r : `i64`，一定可行的上界（含），一般取 $1e18$ 或题面已知上界。
- `check`: `bool check(i64 x)`，返回 x 是否可行；必须随 x 单调（ x 越大越容易可行）。
- 返回值：`i64`，最小可行值 x ，满足 `check(x)=true` 且 `check(x-1)=false`。

整数二分：最大可行值

赛时先看

题目信号：题面问“最多能多少”“最大的最小值”“距离至少多少”，答案越大越难满足。

本质：与最小可行值完全对称：`check(x)` 表示“ x 可行”， x 越大越难可行，二分找最大的可行 x 。唯一区别是中位数要上取整，否则区间长为 2 时会死循环。

复杂度判定： $O(\log V * \text{check})$ ；用法与数据规模限制同上一节 `min_true`。

维护的量：二分边界 l/r (l 一定可行、 r 一定不可行) 与判定函数 `check`。

接法：`check(mid)` 写成题面可行性判定，然后 `i64 ans = max_true(1, r, check);` 输出 `ans`。

警告：必须用 `mid = l + (r-l+1)/2` 上取中位数，否则 `l=mid`

无法推进；与上一节配对的“答案范围单调”判断不要写反。

【抄板清单（照着做就行）】

1. 抄哪段：整个 `max_true` 模板函数（和上一节 `min_true` 一起抄，两者常配对使用）。
2. 构造：`l` 设成“一定可行”的下界，`r` 设成“一定不可行”的上界。
3. 调用：`i64 ans = max_true(1, r, check);`
4. 取结果：`ans` 即最大可行值，直接输出。

【改造点（按题目改这几处）】

- 判定函数：换成题目的可行性判定（唯一要写的内容）。
- 中位数：不要改成下取整， $(r-l+1)/2$ 是防死循环的关键。
- 答案单调方向反了：把“可行”的定义取反，或用上一节 `min_true`。

```
// 维护的量：二分边界 l/r (l 一定可行、r 一定不可行) 与判定函数 check。
// 不变量：check(l)=true 且 check(r)=false；结束时 l 是最大的可行值。
template <class Check>
i64 max_true(i64 l, i64 r, Check check) {
    while (l < r) {
        i64 mid = l + (r - l + 1) / 2; // 上取中位数：保证 l=mid 时区间内仍会缩小。
        if (check(mid)) l = mid;      // mid 可行，答案落在 [mid, r]。
        else r = mid - 1;             // mid 不可行，答案落在 [l, mid-1]。
    }
    return l;
}
```

【最小完整示例（先抄这一段就能跑）】

题目：速度 $v=3$ ，距离 $d=100$ ，求不超过距离 ($3*t \leq 100$) 的最大整数秒数。

```
// check(t): 3*t <= 100 即可行；l=0 一定可行，r=1e18 一定不可行。
auto check = [](i64 t) { return 3LL * t <= 100; };
i64 ans = max_true(0, 10000000000000000LL, check);
cout << ans << '\n'; // 33: 3*33=99 <= 100, 3*34=102 超过了
```

- 样例：输出 33。

【传参要求（照这个传不会错）】

- `l`: `i64`，一定可行的下界（含），一般取 0 或 1。
- `r`: `i64`，一定不可行的上界（含），一般取 $1e18$ 或题面已知上界。
- `check`: `bool check(i64 x)`，返回 `x` 是否可行；必须随 `x` 单调（`x` 越大越难可行）。
- 返回值：`i64`，最大可行值 `x`，满足 `check(x)=true` 且 `check(x+1)=false`。

二分答案 + 区间覆盖判定

赛时先看

题目信号：每个源点随时间向两侧扩展；要求覆盖 $[0, k]$ 所有整数点；答案具有单调性。

本质：若干事件在时间 t 后覆盖一段区间，求最早覆盖完整目标区间的的时间。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；`solve()`

里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定：单次判定 $O(n \log n)$ ，二分后 $O(n \log n \log V)$ 。

维护的量：`segs`（时间 t 下每个源点展开的覆盖区间 $[l, r]$ ）与 `cur`（当前最小未覆盖整数点）。

警告：如果覆盖的是整数点，维护“当前最小未覆盖整数”`cur`；区间右端覆盖到 `r` 后，下一个未覆盖点是 `r + 1`。

【最小完整示例（先抄这一段就能跑）】

```
// 事件（出现时间，位置），t 时刻覆盖 [pos-(t-出现时间), pos+(t-出现时间)]，求最早盖满 [0, 12] 的时间。
vector<pair<i64, i64>> events = {{0, 2}, {3, 10}};
cout << earliest_full_cover(events, 12) << '\n'; // 5 (t=5 时覆盖 [0, 7] 与 [8, 12] 正好接满)
```

- 样例：输出 5。

【传参要求（照这个传不会错）】

- `cover_integer_segment(events, k, t)`: `events` 为 `vector<pair<i64, i64>>` (`first`=出现时间, `second`=位置)，`k` 为目标右端（覆盖 $[0, k]$ ），`t` 为当前时间；返回 `bool` 是否已全覆盖。

- `earliest_full_cover(events, k)`: 入参同上, 返回最早可行时间 (`i64`); 内部自动倍增上界再二分。
- 覆盖的是整数点: 一段盖到 `r` 后下一个未覆盖点是 `r + 1`。

```
bool cover_integer_segment(const vector<pair<i64, i64>>& events, i64 k, i64 t) {
    vector<pair<i64, i64>> segs;
    for (auto [appear_time, pos] : events) {
        if (t < appear_time) continue;
        i64 d = t - appear_time;
        i64 l = max<i64>(0, pos - d);
        i64 r = min<i64>(k, pos + d);
        segs.push_back({l, r});
    }
    sort(segs.begin(), segs.end());
    i64 cur = 0;
    for (auto [l, r] : segs) {
        if (r < cur) continue;
        if (l > cur) return false;
        cur = r + 1;
        if (cur > k) return true;
    }
    return cur > k;
}

i64 earliest_full_cover(vector<pair<i64, i64>> events, i64 k) {
    i64 lo = 0, hi = 1;
    while (!cover_integer_segment(events, k, hi)) hi <= 1;
    while (lo < hi) {
        i64 mid = (lo + hi) >> 1;
        if (cover_integer_segment(events, k, mid)) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
```

最大化最小距离：直线/圆周二分

赛时先看

题目信号: 求最大最小距离; 选点集合固定; 判定答案 `d` 可行具有单调性。

本质: 从若干位置中选 `k` 个, 使任意相邻选点的距离至少为 `d`, 常用于“最大化最小距离”。

接法: 把本节 `struct/class/函数组` 整段抄到 `solve()` 上面; `solve()`

里直接按下方“不会用就看这里”调用, 不需要自己猜内部参数。

复杂度判定: 直线判定 $O(n)$; 圆周判定 $O(n \log n)$ 。

维护的量: 直线版只需贪心游标 `last` 与计数 `cnt`; 圆周版额外维护

`nxt` (每个位置向后最早可选点)、`up` (倍增跳表)、`cur` (当前选点下标)。

警告: 直线只需贪心; 圆周必须额外检查最后一个选点到第一个选点绕回去的距离。

【最小完整示例 (先抄这一段就能跑)】

```
// 直线: {1, 5, 9} 中选 2 个点, 间距至少 d。
cout << can_pick_on_line({1, 5, 9}, 2, 5) << '\n'; // 1 (选 1 与 9, 间距 8)
cout << can_pick_on_line({1, 5, 9}, 2, 9) << '\n'; // 0 (最大间距 8 < 9)
// 圆周: 周长 12 的圆上 {0, 4, 8} 选 3 个点, 含绕回一圈的距离。
cout << can_pick_on_circle({0, 4, 8}, 12, 3, 4) << '\n'; // 1 (间距 4,4,4 均满足)
cout << can_pick_on_circle({0, 4, 8}, 12, 3, 5) << '\n'; // 0 (8 绕回 0 只有 4)
```

- 样例: 输出 `1`、`0`、`1`、`0`。

【传参要求 (照这个传不会错)】

- `can_pick_on_line(x, k, d)`: `x` (`vector<i64>`) 候选点, 函数内会排序, 可传任意顺序)、`k` (要选的点数)、`d` (最小间距); 返回 `bool`, 从左到右贪心能选够 `k` 个即 `true`。
- `can_pick_on_circle(p, circumference, k, d)`: `p` (圆上点坐标, $0 \leq p[i] < circumference$)、`circumference` (周长)、`k`、`d`; 返回 `bool`; `k <= 1` 时只要 `n >= k`, `d == 0` 时直接 `true`。
- 求最大可行间距: 外层对 `d` 二分, 如 `max_true(0, 上界, [&return can_pick_on_line(x, k, d);])`。

【改板时先认这几个量】

- `up`: 倍增跳表, 表示从当前位置再跳若干次选点。
- `cur`: 当前选点下标。
- `nxt`: 每个位置向后能跳到的最早选点。

```
bool can_pick_on_line(vector<i64> x, int k, i64 d) {
    sort(x.begin(), x.end());
```



```

int cnt = 0;
i64 last = -(1LL << 60);
for (i64 v : x) {
    if (v - last >= d) {
        cnt++;
        last = v;
    }
}
return cnt >= k;
}

bool can_pick_on_circle(vector<i64> p, i64 circumference, int k, i64 d) {
    sort(p.begin(), p.end());
    int n = (int)p.size();
    if (k <= 1) return n >= k;
    if (n < k) return false;
    if (d == 0) return true;
    vector<i64> q = p;
    for (i64 v : p) q.push_back(v + circumference);
    int m = (int)q.size();
    vector<int> nxt(m, m);
    for (int i = 0; i < m; i++) {
        nxt[i] = lower_bound(q.begin(), q.end(), q[i] + d) - q.begin();
    }
    int LOG = 1;
    while ((1 << LOG) <= k) LOG++;
    vector<int> up(LOG, vector<int>(m + 1, m));
    up[0] = nxt;
    up[0].push_back(m);
    for (int j = 1; j < LOG; j++) {
        for (int i = 0; i <= m; i++) up[j][i] = up[j - 1][up[j - 1][i]];
    }
    for (int start = 0; start < n; start++) {
        int cur = start;
        int steps = k - 1;
        for (int j = 0; steps; j++, steps >>= 1) {
            if (steps & 1) cur = up[j][cur];
        }
        if (cur < start + n && q[cur] - q[start] <= circumference - d) return true;
    }
    return false;
}

```

实数二分

赛时先看

题目信号：几何、速度、概率、浮点答案。

本质：连续答案，误差允许 $1e-6$ 或 $1e-9$ 。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()`

里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定：固定迭代 100 次。

维护的量：二分边界 l/r (double) 与判定函数 `check` (单调的 `double -> bool`)。

警告：输出精度要比题目要求多 2 位。

【最小完整示例（先抄这一段就能跑）】

```

// 求 sqrt(2): check 单调递增, 返回最小满足  $x^2 \geq 2$  的  $x$ 。
double ans = real_binary(0.0, 10.0, [](double x) { return x * x >= 2.0; });
cout << fixed << setprecision(10) << ans << '\n'; // 1.4142135624

```

- 样例：输出 1.4142135624。

【传参要求（照这个传不会错）】

- `real_binary(l, r, check)`: l/r 为 double 二分边界 (l 侧 `check` 为 false、 r 侧为 true)，`check` 是 `double -> bool` 的单调判定；固定迭代 100 次，返回 double。
- 输出精度：题目要 $1e-6$ 就至少输出 8 位小数（比要求多 2 位）。

```

template <class Check>
double real_binary(double l, double r, Check check) {
    for (int it = 0; it < 100; ++it) {
        double mid = (l + r) / 2.0;
        if (check(mid)) r = mid;
        else l = mid;
    }
    return (l + r) / 2.0;
}

```

三分搜索

赛时先看

题目信号：连续变量，函数先降后升或先升后降。

本质：单峰/单谷函数求极值。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定：固定迭代 120 次。

维护的量：三分点 `m1/m2`（`l` 与 `r` 的三等分点）与单峰函数 `f (double -> double)`。

警告：只适用于单峰；整数三分最后要枚举小区间。

【最小完整示例（先抄这一段就能跑）】

```
// f(x) = (x-1)^2 + 3 在 x=1 取最小值，三分返回极小值点。
double ans = ternary_min(-10.0, 10.0, [](double x) { return (x - 1) * (x - 1) + 3; });
cout << fixed << setprecision(6) << ans << '\n'; // 1.000000
```

- 样例：输出 1.000000。

【传参要求（照这个传不会错）】

- `ternary_min(l, r, f)`： `l/r` 为 `double` 搜索区间（须包含唯一极小值点），`f` 是 `double -> double` 的单谷函数；固定迭代 120 次，返回极小值点（`double`）。
- 求极大值：对 `-f` 调用 `ternary_min` 后返回点不变，或换求 `f` 取反后的最小值点。
- 整数变量：三分后要在最后的小区间内枚举所有整数点取最值。

```
template <class F>
double ternary_min(double l, double r, F f) {
    for (int it = 0; it < 120; ++it) {
        double m1 = l + (r - l) / 3.0;
        double m2 = r - (r - l) / 3.0;
        if (f(m1) < f(m2)) r = m2;
        else l = m1;
    }
    return (l + r) / 2.0;
}
```

离散化

赛时先看

题目信号：坐标/值域范围 `1e9/1e18`，但实际出现的点只有 `1e5`；题目只关心大小关系（排序、当数组下标、开 BIT/线段树）而不关心具体值。

本质：把巨大且稀疏的坐标压缩成连续的 `1..m`；排序去重后 `id(x)` 拿排名、`value(id)`

还原原值；从此“按值操作”全部变成“按下标操作”。

复杂度判定：排序去重 $O(n \log n)$ ，单次 `id/value` 查询 $O(\log n)$ （`lower_bound`）；`n` 到 `2e5`、`1e6` 都轻松，别用 `map` 拖慢。

维护的量：`xs`（排序去重后的原坐标表，编号 `i` 对应 `xs[i-1]`）。

接法：先把所有会被访问的坐标 `add` 进去，`build()` 后用 `id(x)`

作为树状数组/线段树下标。区间覆盖和扫描线如果要算真实长度，不能只用压缩后的编号相减，要用 `value(id+1)-value(id)` 这种原坐标差。

警告：区间覆盖题常要额外加入 `l`、`r`、`r+1`；长度计算要用原坐标。

【API / 入口函数（赛时只认这里列的名字）】

- `add(T x)` -> 收集一个坐标（`build` 前必须加完全部会被访问的值）
- `build()` -> 完成建树或预处理
- `size()` -> 查询集合大小 返回 `int`。

【抄板清单（照着做就行）】

1. 抄哪段：整个 `Compressor` 结构体。
2. 收集：把所有会被访问到的坐标全部 `add` 一遍（区间题把 `l/r/r+1` 也加进去）。
3. 建表：调用一次 `build()` 完成排序去重。
4. 使用：`id(x)` 得到 `1..m` 的编号当下标；要还原原坐标用 `value(id)`。

【改造点（按题目改这几处）】

- 收集范围：漏加任何一个会被查询的坐标都会让 `id` 错位，宁多勿少。
- 模板参数：坐标是 `int` 还是 `i64`，决定 `Compressor<int>` 还是 `Compressor<i64>`。
- 只比大小：直接拿编号用；要算长度/差值：用 `value(...)` 还原原坐标再减。
- 离散化后的值域：`size()` 返回 `m`，开数组就开到 `m+1` (1-indexed)。

```
// 维护的量: xs = 排序去重后的原坐标表, 编号 i (1-indexed) 对应原坐标 xs[i-1]。
// 不变量: xs 严格递增, id(value(i)) = i, 且 id 返回值落在 [1, size()]。
template <class T>
struct Compressor {
    vector<T> xs;

    void add(T x) { xs.push_back(x); } // 收集坐标; build 前必须加完全部会被访问的值。

    void build() {
        sort(xs.begin(), xs.end()); // 排序后去重, 得到从小到大、互不相同的坐标表。
        xs.erase(unique(xs.begin(), xs.end()), xs.end());
    }

    int id(T x) const {
        return int(lower_bound(xs.begin(), xs.end(), x) - xs.begin()) + 1; // 排名 +1 得 1..m 编号。
    }

    T value(int id) const {
        return xs[id - 1]; // 编号还原成原坐标。
    }

    int size() const {
        return (int)xs.size(); // 不同坐标个数 m, 开数组开到 m+1。
    }
};
```

【最小完整示例（先抄这一段就能跑）】

题目：坐标 {10, 5, 5, 1000000000} 离散化到 1..3, 并演示编号与还原。

```
Compressor<i64> cp;
for (i64 x : {10LL, 5LL, 5LL, 1000000000LL}) cp.add(x);
cp.build();
cout << cp.size() << '\n'; // 3: 去重后 3 个不同坐标
cout << cp.id(5) << ' ' << cp.id(10) << '\n'; // 1 2: 排名从 1 开始
cout << cp.value(cp.id(1000000000LL)) << '\n'; // 1000000000: 编号还原原坐标
```

- 样例：依次输出 3、1 2、1000000000。

【传参要求（照这个传不会错）】

- `add(T x)`: `x` 是任意会被查询的坐标 (`Compressor<int>` 或 `Compressor<i64>`)；`build()` 前加完全部，重复加没关系。
- `build()`: 无参数；排序去重，之后才可用 `id/value/size`。
- `id(T x)`: `x` 必须已 `add` 过；返回 `int` 编号，落在 `1..m` (1-indexed)。
- `value(int id)`: `id` 取值 `1..size()`；返回原坐标 `T`。
- `size()`: 返回 `int`，不同坐标个数 `m`；开数组开到 `m+1`。

一维前缀和

赛时先看

题目信号：数组不修改，多次问 `[l,r]` 的和。

本质：静态数组区间和查询。

接法：数组不修改且反复问区间和，就先 `pre = build_prefix(a)`，每次输出 `range_sum(pre,l,r)`。如果有修改，前缀和失效，改用树状数组或线段树。

复杂度判定：预处理 $O(n)$ ，查询 $O(1)$ 。

维护的量：`pre`（前缀和数组，`pre[i] = a[1..i]` 的和，1-indexed，长度 `n+1`）。

警告：建议 1-based，`sum(l,r)=pre[r]-pre[l-1]`。

约定：`a` 使用 1-indexed，`n = (int)a.size() - 1`。

【最小完整示例（先抄这一段就能跑）】

```
// a = {0, 1, 2, 3} (1-indexed, a[0] 占位)，问区间和。
vector<i64> a = {0, 1, 2, 3};
auto pre = build_prefix(a);
cout << range_sum(pre, 1, 3) << '\n'; // 6 1+2+3
cout << range_sum(pre, 2, 3) << '\n'; // 5 2+3
```

- 样例：输出 6 与 5。

【传参要求（照这个传不会错）】

- `build_prefix(a)`: 入参 `a` 必须 1-indexed (`a[0]` 占位, 元素从下标 1 开始), 长度 `n+1`; 返回 `pre` (`vector<i64>`, 长度 `n+1`, `pre[i] = a[1..i]` 的和)。
- `range_sum(pre, l, r)`: `l/r` 为 1-indexed 闭区间, `1 <= l <= r <= n`; 返回 `i64` 区间和。
- 有修改时前缀和失效, 换树状数组或线段树。

【API / 入口函数（赛时只认这里列的名字）】

- `range_sum(const vector<i64>& pre, int l, int r) ->` 查询闭区间和 返回 `i64`。

```
vector<i64> build_prefix(const vector<i64>& a) {
    int n = (int)a.size() - 1; // 公式/约定: a is 1-based
    vector<i64> pre(n + 1, 0);
    for (int i = 1; i <= n; ++i) pre[i] = pre[i - 1] + a[i];
    return pre;
}

i64 range_sum(const vector<i64>& pre, int l, int r) {
    return pre[r] - pre[l - 1];
}
```

一维差分

赛时先看

题目信号: 多次对 `[l,r]` 加 `x`, 最后输出每个位置。

本质: 离线区间加, 最后一次性还原数组。

接法: 所有区间加操作都提前知道、且只在最后问最终数组时, 用差分最短。每次 `[l,r] += x` 调用 `add(l,r,x)`; 全部操作结束后 `restore()` 得到最终数组。若中途穿插查询, 不能用纯差分, 翻 BIT/线段树。

复杂度判定: 修改 $O(1)$, 还原 $O(n)$ 。

维护的量: `diff` (差分数组, `diff[i] = a[i]-a[i-1]`, 长度 `n+2`) 与 `n` (元素个数)。

警告: `diff[r+1] -= x`, 数组要开到 `n+2`。

【最小完整示例（先抄这一段就能跑）】

```
// 长度为 5 的数组: 区间 [2,4] 加 3, 区间 [3,5] 减 1。
Difference1D d(5);
d.add(2, 4, 3);
d.add(3, 5, -1);
auto a = d.restore();
cout << a[2] << ' ' << a[3] << ' ' << a[4] << ' ' << a[5] << '\n'; // 3 2 2 2
```

- 样例：输出 3 2 2 2。

【传参要求（照这个传不会错）】

- `Difference1D(n)` (构造) 或 `init(n)`: `n` 为数组长度, 下标 `1..n`, 内部数组开 `n+2`。
- `add(l, r, x)`: 对闭区间 `[l,r]` 加 `x` (`i64`), `1 <= l <= r <= n`, 可多次调用。
- `restore()`: 返回最终数组 (`vector<i64>`, 1-indexed, 长度 `n+1`)。
- 中途穿插查询不能用纯差分, 换树状数组/线段树。

【API / 入口函数（赛时只认这里列的名字）】

- `add(int l, int r, i64 x) ->` 对区间 `[l,r]` 加 `x`
- `init(int n_) ->` 初始化/清空结构

```
struct Difference1D {
    int n;
    vector<i64> diff;

    Difference1D(int n = 0) { init(n); }

    void init(int n_) {
        n = n_;
        diff.assign(n + 2, 0);
    }

    void add(int l, int r, i64 x) {
        diff[l] += x;
        diff[r + 1] -= x;
    }
}
```

```
vector<i64> restore() const {
    vector<i64> a(n + 1, 0);
    for (int i = 1; i <= n; ++i) a[i] = a[i - 1] + diff[i];
    return a;
}
};
```

二维前缀和

赛时先看

题目信号：矩阵不修改，多次查询矩形区域和。

本质：静态矩阵子矩形和。

接法：把 `a` 填成 1-indexed 矩阵（第 0 行/列占位），先 `auto pre = build_prefix_2d(a);`，之后每次询问直接 `rect_sum(pre, x1, y1, x2, y2)`。

复杂度判定：预处理 $O(nm)$ ，查询 $O(1)$ 。

维护的量：`pre[i][j]`（左上角 $(1,1)$ 到 (i,j) 的子矩形和）；输入 `a` 为 1-indexed，`a[0]` 行与 `a[i][0]` 列是占位。

警告：容斥公式四项别写反。

【最小完整示例（先抄这一段就能跑）】

```
// a 为 3x3 矩阵（1-indexed，第 0 行/列是占位），多次查询子矩形和。
vector<vector<i64>> a = {
    {0, 0, 0, 0},
    {0, 1, 2, 3},
    {0, 4, 5, 6},
    {0, 7, 8, 9}
};
auto pre = build_prefix_2d(a);
cout << rect_sum(pre, 2, 2, 3, 3) << '\n'; // 5+6+8+9 = 28
```

- 样例：输出 28。

【传参要求（照这个传不会错）】

- `a`：1-indexed 矩阵，`n = a.size()-1` 行、`m = a[1].size()-1` 列；`a[0]` 行与 `a[i][0]` 列是占位（值为 0）。
- `build_prefix_2d(a)`：返回 $(n+1)*(m+1)$ 的 `pre`；`pre[0][*]`、`pre[*][0]` 恒为 0。
- `rect_sum(pre, x1, y1, x2, y2)`：闭矩形 $[x1..x2] \times [y1..y2]$ 的和；要求 $1 \leq x1 \leq x2 \leq n$ 、 $1 \leq y1 \leq y2 \leq m$ 。

```
// 维护的量：pre[i][j] = 左上角 (1,1) 到 (i,j) 的子矩形和，1-indexed。
// 不变量：pre[i][j] = pre[i-1][j] + pre[i][j-1] - pre[i-1][j-1] + a[i][j]。
vector<vector<i64>> build_prefix_2d(const vector<vector<i64>>& a) {
    int n = (int)a.size() - 1;
    int m = (int)a[1].size() - 1;
    vector<vector<i64>> pre(n + 1, vector<i64>(m + 1, 0));
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            pre[i][j] = pre[i - 1][j] + pre[i][j - 1] - pre[i - 1][j - 1] + a[i][j]; // 容斥：上+左-左上角
        }
    }
    return pre;
}

// 维护的量：只读 pre，不新增结构。
// 不变量：矩形和 = 右下 - 左下 - 右上 + 左上（四角容斥，方向别写反）。
i64 rect_sum(const vector<vector<i64>>& pre, int x1, int y1, int x2, int y2) {
    return pre[x2][y2] - pre[x1 - 1][y2] - pre[x2][y1 - 1] + pre[x1 - 1][y1 - 1];
}
```

二维差分

赛时先看

题目信号：多次对矩形区域加 `x`，最后输出矩阵。

本质：离线子矩形加值，最后还原整个矩阵。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要自己直接改内部数组。

复杂度判定：每次修改 $O(1)$ ，还原 $O(nm)$ 。

维护的量：`n/m`（行列数）；`diff[i][j]`（二维差分数组，开 $(n+2)*(m+2)$ ，四角打标记）。

警告：四个角容斥，数组要开到 `n+2, m+2`。

【最小完整示例（先抄这一段就能跑）】

```
// 4x4 矩阵初始全 0；两次矩形加后输出最终矩阵。
```

```
Difference2D d(4, 4);
d.add(1, 1, 2, 2, 3); // (1,1)-(2,2) 每个格子加 3
d.add(2, 2, 4, 4, 1); // (2,2)-(4,4) 每个格子加 1
auto a = d.restore();
cout << a[2][2] << '\n'; // 3+1 = 4
cout << a[1][1] << '\n'; // 只被第一次覆盖: 3
```

- 样例：输出 4 与 3。

【传参要求（照这个传不会错）】

- Difference2D(n, m): 构造；n/m = 行数/列数，内部自动开 n+2, m+2。
- init(n_, m_): 重置为 n_ 行 m_ 列并全部清零。
- add(x1, y1, x2, y2, v): 闭矩形 [x1..x2] × [y1..y2] 整体加 v；要求 1 ≤ x1 ≤ x2 ≤ n、1 ≤ y1 ≤ y2 ≤ m。
- restore(): 返回 (n+1)*(m+1) 的最终矩阵 a (a[i][j] 即 (i,j) 的最终值)，a[0] 行/列恒为 0。

【API / 入口函数（赛时只认这里列的名字）】

- init(int n_, int m_) -> 初始化/清空结构
- add(int x1, int y1, int x2, int y2, i64 v) -> 对矩形 (x1,y1)-(x2,y2) 加 v

```
// 维护的量: diff[i][j] (二维差分数组, 比原矩阵多开一圈避免越界)。
// 不变量: restore() 时对 diff 做二维前缀和, 得到的 a 就是所有 add 叠加后的最终矩阵。
struct Difference2D {
    int n, m;
    vector<vector<i64>> diff;

    Difference2D(int n = 0, int m = 0) { init(n, m); }

    void init(int n_, int m_) {
        n = n_;
        m = m_;
        diff.assign(n + 2, vector<i64>(m + 2, 0)); // 多开一圈, add 的 +1 角不会越界
    }

    void add(int x1, int y1, int x2, int y2, i64 v) {
        // 四角打标记: 容斥之后, 只有矩形内部的点在还原时收到 v
        diff[x1][y1] += v;
        diff[x2 + 1][y1] -= v;
        diff[x1][y2 + 1] -= v;
        diff[x2 + 1][y2 + 1] += v;
    }

    vector<vector<i64>> restore() const {
        vector<vector<i64>> a(n + 1, vector<i64>(m + 1, 0));
        for (int i = 1; i ≤ n; ++i) {
            for (int j = 1; j ≤ m; ++j) {
                a[i][j] = diff[i][j] + a[i - 1][j] + a[i][j - 1] - a[i - 1][j - 1]; // 前缀和还原
            }
        }
        return a;
    }
};
```

02 枚举、贪心、搜索与构造

小数据、状态空间、精确覆盖、构造和贪心证明先翻这里；折半搜索也放在本章，方便和 DFS/BFS 一起定位。

贪心证明速查：邻项交换与反悔堆

赛时先看

题目信号：题目要求选择若干项最大/最小；每项有截止时间、收益、花费；排序后局部交换能证明更优。

本质：处理排序贪心、区间选择、任务安排、可反悔选择最大收益等题。

接法：把本节 struct/class/函数组 整段抄到 solve() 上面；优先调用下面 API 列出的入口，其余 helper 默认不要从 solve() 直接调。

复杂度判定：通常排序 $O(n \log n)$ ，反悔堆 $O(n \log n)$ 。

维护的量：pq（已选任务收益的小根堆，堆项是最该反悔掉的）；sum（当前已选任务总收益）；按截止时间排序后的 jobs。

警告：贪心要能证明。常用证明是邻项交换、包含关系、反悔替换、前缀最优。

【最小完整示例（先抄这一段就能跑）】

题目：每个任务耗时 1，截止时间/收益为 (1,20),(2,10),(2,15),(3,5)，求截止前能完成的任务最大总收益。

```
vector<pair<int, int>> jobs = {{1, 20}, {2, 10}, {2, 15}, {3, 5}};
i64 ans = schedule_with_deadline(jobs); // 任务数超过截止时间就反悔掉堆顶最小收益
cout << ans << "\n";
```

样例：ans = 35（选收益 20、10、15，放弃 5）。

【传参要求（照这个传不会错）】

- jobs: vector<pair<int,int>>, 每个元素 {d, v} 是截止时间 d 和收益 v；内部会先按 d 排序，传乱序没关系；d、v 都应在 int 范围内。
- 返回值：i64，能在各自截止时间前完成的任务最大总收益；收益全为正、任务做不完时，放弃的正好是堆里收益最小的几个。

【API / 入口函数（赛时只认这里列的名字）】

- schedule_with_deadline(vector<pair<int, int>> jobs) -> 反悔堆模型：每个任务有截止时间 d 和收益 v，每个任务耗时 1，最多在 d 前完成。选择总收益最大的一批任务。返回 i64。

```
// 反悔堆模型：每个任务有截止时间 d 和收益 v，每个任务耗时 1，最多在 d 前完成。
// 选择总收益最大的一批任务。
```

```
i64 schedule_with_deadline(vector<pair<int, int>> jobs) {
    sort(jobs.begin(), jobs.end()); // 按截止时间排序。
    priority_queue<int, vector<int>, greater<int>> pq;
    i64 sum = 0;
    for (auto [d, v] : jobs) {
        pq.push(v);
        sum += v;
        if ((int)pq.size() > d) {
            sum -= pq.top();
            pq.pop();
        }
    }
    return sum;
}
```

分数背包：物品允许只取一部分

赛时先看

题目信号：液体、粮食、矿石、可切割材料等，题面明确可以拆开或按比例获取；这时不是 0/1 背包，按单位重量价值贪心即可。

本质：容量有限，但每个物品可以取任意比例，求最大价值并恢复每个物品取了多少。

接法：把本节 struct/class/函数组 整段抄到 solve() 上面；solve()

里先构造对象，再按下方“不会用就看这里”调用公开接口。

复杂度判定：排序 $O(n \log n)$ 。

维护的量：result.max_value（已取物品总价值）；result.picks（每件取走的实际重量 {id, amount}）；capacity（剩余容量，循环里递减）。

警告：只有“允许分割”才能按价值密度贪心，完整物品不可拆时必须回到 DP。权重需为正；比较密度时用 long double 或交叉乘避免精度误判。

【最小完整示例（先抄这一段就能跑）】

题目：容量 50，物品（重量，价值，id）为 (10,100,0),(30,120,1),(20,40,2)，求最大总价值与每件取多少。

```
FractionalKnapsackResult res =
    fractional_knapsack(50, {{10, 100, 0}, {30, 120, 1}, {20, 40, 2}});
cout << res.max_value << "\n"; // 最大总价值
for (auto p : res.picks) cout << p.id << " " << p.amount << "\n";
```

样例：max_value = 240；picks = {(0,10),(1,30),(2,10)}（第 3 件只取 10 重量）。

【传参要求（照这个传不会错）】

- capacity: long double，背包总容量，非负即可。
- items: vector<FractionalItem>, 每件 {weight, value, id}: weight > 0、value >= 0，id 是原编号（会原样出现在结果里，用来恢复方案）。
- 返回值：FractionalKnapsackResult，max_value 是最大总价值，picks 按贪心顺序装 {id, amount}，amount 是实际取走的重量。

```
struct FractionalItem {
    long double weight, value;
```

```

    int id;
};

struct FractionalPick {
    int id;
    long double amount; // 取走的实际重量。
};

struct FractionalKnapsackResult {
    long double max_value = 0;
    vector<FractionalPick> picks;
};

FractionalKnapsackResult fractional_knapsack(
    long double capacity, vector<FractionalItem> items
) {
    sort(items.begin(), items.end(), [](const auto& a, const auto& b) {
        return a.value / a.weight > b.value / b.weight;
    });
    FractionalKnapsackResult result;
    for (const auto& item : items) {
        if (capacity <= 0) break;
        long double amount = min(capacity, item.weight);
        result.max_value += amount / item.weight * item.value;
        result.picks.push_back({item.id, amount});
        capacity -= amount;
    }
    return result;
}

```

Huffman / 最优合并：每次合并最小两堆

赛时先看

题目信号：反复从任意两堆中合并，代价是两堆和，最后合成一堆；题面没有“只能合并相邻两堆”的限制。

本质：有若干文件/果子/区间块，合并两个的代价等于两者当前大小之和，且合并后新块还会继续参与，求最小总代价；也等价于给定字符频率求 Huffman 前缀码的最小带权路径长度。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；优先调用下面 API 列出的入口，其余 helper 默认不要从 `solve()` 直接调。

复杂度判定： $O(n \log n)$ 时间、 $O(n)$ 空间。

警告：只有“任意两堆可合并”才是 Huffman；要求“只能合并相邻”时翻相邻区间合并 DP；输入须非负，累计代价可能到 i64。

维护的量：heap（当前所有堆大小的小根堆，每次弹两个最小的）；answer（累计合并代价）。

【最小完整示例（先抄这一段就能跑）】

题目：洛谷 P1090 合并果子，各堆数量 `[1,2,3,4]`，求最小总合并代价。

```

vector<i64> weights = {1, 2, 3, 4};
i64 ans = huffman_optimal_merge_cost(weights); // 每次合并最小的两堆
cout << ans << "\n";

```

样例：ans = 19 (1+2=3, 3+3=6, 4+6=10, 总代价 3+6+10)。

【传参要求（照这个传不会错）】

- `weights`: `const vector<i64>&`，各堆当前大小（字符频率），要求非负（内部有 `assert(weight >= 0)`）；允许为空或只有一个元素，此时返回 0。
- 返回值：i64，最小总合并代价；累计代价可能超出 int，读入时用 i64。

【API / 入口函数（赛时只认这里列的名字）】

- `huffman_optimal_merge_cost(const vector<i64>& weights)` -> Huffman 编码 / 合并果子：每次合并当前最小的两堆。返回 i64。
- 只要题目要求“相邻”合并，就不是 Huffman，应翻相邻区间合并/石子合并 DP。
- 输入非负；累计代价可能到 long long，先看数据范围。
- 一堆或零堆不需要合并，答案是 0。

```

// Huffman 编码 / 合并果子：每次合并当前最小的两堆。
i64 huffman_optimal_merge_cost(const vector<i64>& weights) {
    priority_queue<i64, vector<i64>, greater<i64>> heap;
    for (i64 weight : weights) {
        assert(weight >= 0);
        heap.push(weight);
    }
}

```

```

i64 answer = 0;
while (heap.size() >= 2) {
    i64 merged = heap.top();
    heap.pop();
    merged += heap.top();
    heap.pop();
    answer += merged; // 确认题目数据使总和不超过 i64。
    heap.push(merged);
}
return answer;
}

```

典题：洛谷 P1090《合并果子》。读入每堆数量到 `weights` 后，输出 `huffman_optimal_merge_cost(weights)`。例如 `[1,2,3,4]` 的最优过程是 `1+2=3`, `3+3=6`, `4+6=10`，总代价 19。

定长滑动窗口：固定长度区间和/最大值

赛时先看

题目信号：题面明确子数组/连续区间长度固定为 `k`；问连续 `k` 天/`k` 个位置的最大总和、最小总和、平均值或某个可增量维护的统计量。

本质：长度固定的窗口从左往右滑，每一步只“进一个出一个”：新窗口 = 旧窗口 + 进 - 出，`O(1)` 增量更新，不必每个窗口重新求和。

复杂度判定：`O(n)` 时间、`O(1)` 额外空间；`n` 到 `1e6/1e7` 都直接过。窗口长度不固定（至多/至少 `k`）翻“可变窗口”；窗口内还要查最值翻单调队列。

维护的量：`current`（当前窗口的和），`answer`（扫过的所有窗口结果的最值）。

接法：给 `n` 天收益，必须连续工作恰好 `k` 天，求最大收益。将收益放入 0-indexed `vector<i64> a`，直接调用 `maximum_fixed_window_sum(a,k)`；若要求最小收益，把 `max` 改成 `min` 即可（答案初值已经是第一个窗口）。

警告：`len` 必须在 `[1,n]`；窗口右移时先减掉离开的 `a[i-len]`，再加进入的 `a[i]`；元素与答案都可能需要 `i64`。

【抄板清单（照着做就行）】

1. 抄哪段：整个 `maximum_fixed_window_sum` 函数。
2. 构造：把数组放进 0-indexed `vector<i64> a`，长度 `len = k`。
3. 调用：`i64 ans = maximum_fixed_window_sum(a, k);`
4. 取结果：`ans` 就是所有固定长度窗口和的最大值，直接输出。

【改造点（按题目改这几处）】

- 求最小：把 `max(answer, current)` 改成 `min(answer, current)`，其余不动。
- 维护的量不是和：把 `current += a[r] - a[r-len]` 换成题目要的增量更新式；要窗口内最值翻单调队列。
- 平均值：`ans / len` 即可；元素含负数、浮点都没问题，模板无假设。
- 多个不同长度：套循环分别调用，或翻“一维前缀和” `range_sum`。

```

// 维护的量: current = 当前长度为 len 的窗口和, answer = 所有窗口和的最大值。
// 不变量: 窗口右移一格时, current 先减掉离开的 a[r-len], 再加进进入的 a[r]。
i64 maximum_fixed_window_sum(const vector<i64>& a, int len) {
    int n = (int)a.size();
    assert(1 <= len && len <= n);
    i64 current = 0;
    for (int i = 0; i < len; ++i) current += a[i]; // 先求第一个窗口 [0, len) 的和。
    i64 answer = current;
    for (int r = len; r < n; ++r) {
        current += a[r] - a[r - len]; // 窗口右移: 进 a[r], 出 a[r-len], O(1) 更新。
        answer = max(answer, current);
    }
    return answer;
}

```

【最小完整示例（先抄这一段就能跑）】

题目：`a = {1, 3, -2, 5, 2}`，必须连续工作恰好 `k=3` 天，求最大收益。

```

vector<i64> a = {1, 3, -2, 5, 2};
i64 ans = maximum_fixed_window_sum(a, 3); // 窗口和依次为 2、6、5
cout << ans << '\n'; // 6: 最大窗口和

```

- 样例：输出 6。

【传参要求（照这个传不会错）】

- `a`: `const vector<i64>&`, 0-indexed, 元素可为负；`n >= 1`。
- `len`: `int`, 窗口长度，必须满足 `1 <= len <= n`（不满足会触发 `assert`）。

- 返回值: `i64`, 所有长度恰为 `len` 的连续子数组和的最大值。

典型模型: 给 n 天收益, 必须连续工作恰好 k 天, 求最大收益。将收益放入 0-indexed `vector<i64> a`, 直接调用 `maximum_fixed_window_sum(a,k)`; 若要求最小收益, 把 `max` 改成 `min` 即可 (答案初值已经是第一个窗口)。

可变窗口: 至多/恰好 K 种不同元素

赛时先看

题目信号: 连续子数组、不同值种类数、至多/恰好 K

个颜色/数字/字符; 窗口右端点从左到右扫, 约束在左端点右移后不会更难满足。

本质: 统计不同元素数不超过 K 的子数组数量, 或利用 “恰好 $K = \text{至多 } K - \text{至多 } K-1$ ” 统计恰好 K 种不同元素的子数组数量。

接法: $n \leq 2e5$, 求元素互不重复的子数组数。这不能直接写成一个固定 K ; 把窗口条件改成某值频率不超过 1 即可, 或直接用 `count_subarrays_at_most_k_distinct(a,k)` 处理 “至多/恰好 K 种” 的原型。CSES Distinct Values Subarrays 是典型的窗口计数题。

复杂度判定: 哈希表实现期望 $O(n)$, 每个位置最多进窗口、出窗口各一次; 空间 O (不同值数)。

维护的量: `frequency` (窗口内每个值出现次数); `distinct` (当前窗口不同值个数); `left/right` (窗口两端); `answer` (累计合法子数组数)。

警告: `count_subarrays_at_most_k_distinct(a,-1)` 必须返回 0 ——“恰好 K 种”的差分要用到 $k-1=-1$, 这样 $K=0$ 时也安全; 每个右端点合法左端点是连续的一段, 贡献为 $r-l+1$; 不要把它直接套到 “区间和不超过 S 且允许负数” 的题, 负数会破坏窗口单调性。

【最小完整示例 (先抄这一段就能跑)】

题目: `a = [1,2,1,3]`, 求不同元素至多 2 种、以及恰好 2 种的子数组个数。

```
vector<i64> a = {1, 2, 1, 3};
i64 at_most = count_subarrays_at_most_k_distinct(a, 2);
i64 exactly = count_subarrays_exactly_k_distinct(a, 2); // 恰好 = 至多2 - 至多1
cout << at_most << " " << exactly << "\n";
```

样例: `at_most = 8, exactly = 4`。

【传参要求 (照这个传不会错)】

- `a`: `const vector<i64>&`, 0-indexed 数组, 元素就是 “种类” 的编号, 任意整数都行。
- `k`: `int`, 允许的不同种类数上限; $k < 0$ 时返回 0 , 恰好 K 种的内部差分会传 $k-1 = -1$, 这是安全的。
- 返回值: `i64`, 满足条件的连续子数组个数。

```
i64 count_subarrays_at_most_k_distinct(const vector<i64>& a, int k) {
    if (k < 0) return 0;
    unordered_map<i64, int> frequency;
    frequency.reserve(a.size() * 2 + 1);
    frequency.max_load_factor(0.7F);

    int left = 0, distinct = 0;
    i64 answer = 0;
    for (int right = 0; right < (int)a.size(); ++right) {
        if (++frequency[a[right]] == 1) ++distinct;
        while (distinct > k) {
            if (--frequency[a[left]] == 0) --distinct;
            ++left;
        }
        answer += right - left + 1;
    }
    return answer;
}

i64 count_subarrays_exactly_k_distinct(const vector<i64>& a, int k) {
    return count_subarrays_at_most_k_distinct(a, k)
        - count_subarrays_at_most_k_distinct(a, k - 1);
}
```

典型模型: $n \leq 2e5$, 求元素互不重复的子数组数。这不能直接写成一个固定 K ; 把窗口条件改成某值频率不超过 1 即可, 或直接用 `count_subarrays_at_most_k_distinct(a,k)` 处理 “至多/恰好 K 种” 的原型。CSES Distinct Values Subarrays 是典型的窗口计数题。

正数数组双指针: 最大长度且区间和不超过 S

赛时先看

题目信号: 元素保证非负/正; 区间和与一个阈值比较; 右扩窗口只会让和不减, 左缩窗口只会让和不增。

本质：数组元素非负时，求和不超过 `limit` 的最长连续子数组；也可改为统计和不超过 `limit` 的所有子数组数。

接法：所有货物重量非负，问最长连续装载段总重量不超过 `S`。直接输出

`longest_nonnegative_subarray_sum_at_most(weight,S)`；若问合格连续段数量，改调

`count_nonnegative_subarrays_sum_at_most`。

复杂度判定： $O(n)$ 时间， $O(1)$ 额外空间。

维护的量：`sum`（当前窗口 `[left,right]` 的元素和）；`left/right`（窗口两端，`right` 固定时收缩 `left` 直到合法）；`answer`（最长合法长度或合法子数组个数）。

警告：有负数时不能直接用本模板；`limit < 0`

时没有非空合法段；“最长”在每次修复合法窗口后更新，若改成“计数”则每个右端点贡献 `right-left+1`。

【最小完整示例（先抄这一段就能跑）】

题目：货物重量 `[3,1,2,1]` 均非负，求总重量不超过 4 的最长连续段长度，以及合格连续段数量。

```
vector<i64> weight = {3, 1, 2, 1};
int len = longest_nonnegative_subarray_sum_at_most(weight, 4);
i64 cnt = count_nonnegative_subarrays_sum_at_most(weight, 4);
cout << len << " " << cnt << "\n";
```

样例：`len = 3`（取 `[1,2,1]`），`cnt = 6`。

【传参要求（照这个传不会错）】

- `a`: `const vector<i64>&`，0-indexed 数组，元素必须非负（内部有 `assert(a[right] >= 0)`）。
- `limit`: `i64`，区间和的上界；`limit < 0` 时没有合法非空段，返回 0。
- 返回值：最长版返回 `int`（最大合法长度），计数版返回 `i64`（合法子数组个数）。

```
int longest_nonnegative_subarray_sum_at_most(const vector<i64>& a, i64 limit) {
    if (limit < 0) return 0;
    int left = 0, answer = 0;
    i64 sum = 0;
    for (int right = 0; right < (int)a.size(); ++right) {
        assert(a[right] >= 0);
        sum += a[right];
        while (sum > limit) sum -= a[left++];
        answer = max(answer, right - left + 1);
    }
    return answer;
}

i64 count_nonnegative_subarrays_sum_at_most(const vector<i64>& a, i64 limit) {
    if (limit < 0) return 0;
    int left = 0;
    i64 sum = 0, answer = 0;
    for (int right = 0; right < (int)a.size(); ++right) {
        assert(a[right] >= 0);
        sum += a[right];
        while (sum > limit) sum -= a[left++];
        answer += right - left + 1;
    }
    return answer;
}
```

典型模型：所有货物重量非负，问最长连续装载段总重量不超过 `S`。直接输出

`longest_nonnegative_subarray_sum_at_most(weight,S)`；若问合格连续段数量，改调

`count_nonnegative_subarrays_sum_at_most`。

排序双指针：数对和不超过 `S` 的数量

赛时先看

题目信号：任意两个数、和与阈值比较、只需要数量或是否存在；数组可以排序，原下标通常不重要。

本质：统计无序数对 (i,j) ， $i < j$ 满足 $a[i] + a[j] \leq \text{limit}$ 的数量；可作为“两数和第 k 小”或三数和计数的基础子过程。

接法：求第 k 小的两数和。对答案 `mid` 二分，判定 `count_pairs_with_sum_at_most(a,mid) >= k`；若只问不超过 `S` 的数对数，直接输出本函数结果。

复杂度判定：排序 $O(n \log n)$ ，双指针扫描 $O(n)$ ，空间取决于是否复制数组。

维护的量：`left/right`（排序后从两端向中间收的指针）；`answer`（满足 $a[i]+a[j] \leq \text{limit}$ 的无序数对个数）。

警告：数对数量用 `i64`；两数相加可能溢出 `i64`，比较时用 `i128`；当 $a[\text{left}]+a[\text{right}] \leq \text{limit}$ 时，固定 `left` 的所有右端点 `left+1..right` 都合法，共有 `right-left` 对。

【最小完整示例（先抄这一段就能跑）】

题目：a = [1,2,3,4,5]，求两数和不超过 6 的无序数对 (i,j), i<j 个数。

```
vector<i64> a = {1, 2, 3, 4, 5};
i64 cnt = count_pairs_with_sum_at_most(a, 6); // 内部会先排序，原下标不重要
cout << cnt << "\n";
```

样例：cnt = 6 ((1,2), (1,3), (1,4), (1,5), (2,3), (2,4))。

【传参要求（照这个传不会错）】

- a: 按值传入 vector<i64> (会先 sort, 原数组顺序被破坏), 0-indexed。
- limit: i64, 两数和的上界; 比较时内部用 i128, 不用担心相加溢出。
- 返回值: i64, 满足 $a[i]+a[j] \leq \text{limit}$ 的无序数对个数。

```
i64 count_pairs_with_sum_at_most(vector<i64> a, i64 limit) {
    sort(a.begin(), a.end());
    int left = 0, right = (int)a.size() - 1;
    i64 answer = 0;
    while (left < right) {
        if ((i128)a[left] + a[right] <= limit) {
            answer += right - left;
            ++left;
        } else {
            --right;
        }
    }
    return answer;
}
```

典题模型：求第 k 小的两数和。对答案 mid 二分，判定 $\text{count_pairs_with_sum_at_most}(a, \text{mid}) \geq k$ ；若只问不超过 s 的数对数，直接输出本函数结果。

离线扫描 + 树状数组：区间中值落在 [x,y] 的个数

赛时先看

题目信号：数组没有修改；查询同时限制位置区间和值域区间；本质是二维矩形数点，n,q 到 $2e5$ 级别，不能逐查询扫区间。

本质：静态数组 a，多次查询索引区间 [l,r] 中值也落在 [low,high] 的元素数量。将每个 a[i] 看成平面点 (i,a[i])，按值扫描，用树状数组维护已经激活的位置。

接法：给静态数组与 q 个询问 (l,r,x,y)，求 [l,r] 中落在值域 [x,y] 的元素数。构造 0-indexed ValueRangeQuery{1,r,x,y}，批量调用

$\text{count_values_in_index_and_value_ranges}(a, \text{queries})$ ；二维点带权或需要在线修改时，应改用 CDQ、树套树或动态 K-D Tree。

复杂度判定：排序和所有树状数组操作共 $O((n+q) \log n)$ ，空间 $O(n+q)$ 。

维护的量：point (按值排序的平面点 (a[i], i))；event (每个查询拆成 $< \text{low}$ 与 $\leq \text{high}$ 两个事件)；fenwick (已激活位置的前缀和，维护“当前值阈值以下已激活的位置”)。

警告：这里下标是 0-indexed 闭区间；答案是 $\text{count}(\text{value} \leq \text{high}) - \text{count}(\text{value} < \text{low})$ ；不要用 low-1 当阈值，low=LLONG_MIN 时会溢出；相同阈值时先处理严格 $<$ 事件，再处理 $\leq$ 事件。

约定：l,r 是 0-indexed 闭区间； $\text{range_sum}(l,r)$ 查询闭区间 [l,r]。

【最小完整示例（先抄这一段就能跑）】

题目：a = [3,1,4,2]，问下标 [1,3] 中值落在 [2,4] 的元素个数。

```
vector<i64> a = {3, 1, 4, 2};
vector<ValueRangeQuery> qs = {{1, 3, 2, 4}}; // l,r 是 0-indexed 闭区间
vector<i64> ans = count_values_in_index_and_value_ranges(a, qs);
cout << ans[0] << "\n";
```

样例：ans[0] = 2 (下标 1..3 的值是 1、4、2，其中 4、2 落在 [2,4])。

【传参要求（照这个传不会错）】

- a: const vector<i64>&, 0-indexed 静态数组，全程无修改。
- queries: const vector<ValueRangeQuery>&, 每个查询 {l, r, low, high}: l/r 是 0-indexed 闭区间下标，需满足 $0 \leq l \leq r < n$; low/high 是值域，low > high 时该查询自动跳过。
- 返回值: vector<i64>, 与 queries 同序，第 i 个元素是第 i 个查询的答案。

【API / 入口函数（赛时只认这里列的名字）】

- `add(int index, int delta)` -> 在 `index` 处加 `delta`
- `range_sum(int l, int r)` -> 查询闭区间和 返回 `int`。

【改板时先认这几个量】

- `l/r`: 查询区间, 0-indexed 闭区间。
- `bit`: Fenwick 内部树状数组。

```
struct ValueRangeQuery {
    int l, r;           // 0-indexed 闭区间。
    i64 low, high;
};

struct OfflineFenwick {
    int n;
    vector<int> bit;

    explicit OfflineFenwick(int n_) : n(n_), bit(n_ + 1, 0) {}

    void add(int index, int delta) {
        for (++index; index <= n; index += index & -index) bit[index] += delta;
    }

    int prefix_sum(int index) const { // 查询半开前缀区间 [0, index) 的和。
        int result = 0;
        for (; index > 0; index -= index & -index) result += bit[index];
        return result;
    }

    int range_sum(int l, int r) const { // 查询闭区间 [l, r]。
        return prefix_sum(r + 1) - prefix_sum(l);
    }
};

vector<i64> count_values_in_index_and_value_ranges(
    const vector<i64>& a, const vector<ValueRangeQuery>& queries
) {
    struct Event {
        i64 threshold;
        bool include_equal; // false 表示统计 < threshold, true 表示统计 <= threshold。
        int l, r, id, sign;
    };

    int n = (int)a.size();
    vector<pair<i64, int>> point;
    point.reserve(n);
    for (int i = 0; i < n; ++i) point.push_back({a[i], i});
    sort(point.begin(), point.end());

    vector<Event> event;
    event.reserve(queries.size() * 2);
    for (int id = 0; id < (int)queries.size(); ++id) {
        auto [l, r, low, high] = queries[id];
        assert(0 <= l && l <= r && r < n);
        if (low > high) continue;
        event.push_back({high, true, l, r, id, +1});
        event.push_back({low, false, l, r, id, -1});
    }
    sort(event.begin(), event.end(), [](const Event& x, const Event& y) {
        if (x.threshold != y.threshold) return x.threshold < y.threshold;
        return x.include_equal < y.include_equal;
    });

    OfflineFenwick fenwick(n);
    vector<i64> answer(queries.size(), 0);
    int ptr = 0;
    for (const Event& e : event) {
        while (ptr < n && (point[ptr].first < e.threshold
            || (e.include_equal && point[ptr].first == e.threshold))) {
            fenwick.add(point[ptr].second, 1);
            ++ptr;
        }
        answer[e.id] += (i64)e.sign * fenwick.range_sum(e.l, e.r);
    }
    return answer;
}
```

典题模型：给静态数组与 q 个询问 (l, r, x, y) ，求 $[l, r]$ 中落在值域 $[x, y]$ 的元素数。构造 0-indexed `ValueRangeQuery{1, r, x, y}`，批量调用

`count_values_in_index_and_value_ranges(a, queries)`；二维点带权或需要在线修改时，应改用 CDQ、树套树或动态

K-D Tree。

DFS / 回溯骨架

赛时先看

题目信号： $n \leq 20$ 左右；要求输出方案；状态空间可剪枝。

本质：排列组合枚举、棋盘搜索、小数据暴力、构造方案。

接法：全排列、排列型构造、 $n \leq 10$ 左右的暴力验证。先调用 `enumerate_permutations(n)`；若只需要最优答案，把 `answers.push_back(path)` 换成更新答案即可。N 皇后、数独等题将“当前状态是否合法”的剪枝放在 `if (used[x]) continue`；后面，并保证“压入、递归、弹出”三步完整（压入和弹出成对出现）。

复杂度判定：指数级，取决于分支数与剪枝。

维护的量：`path`（当前已选排列前缀）；`used`（`used[x]=1` 表示 x 已用，1-indexed）；`answers`（收集到的全部排列）。

警告：进入递归前修改状态，返回后必须恢复；剪枝不能剪掉最优解。

【最小完整示例（先抄这一段就能跑）】

题目：枚举 1..3 的全部排列（ n 小、要输出方案或暴力验证时用）。

```
vector<vector<int>> all = enumerate_permutations(3);
for (auto& p : all) { /* 每个 p 是一个长度为 3 的排列 */ }
cout << all.size() << " " << all[0][0] << "\n"; // 排列个数与首个排列首元素
```

样例：`all.size() = 6`，首排列为 `{1,2,3}`。

【传参要求（照这个传不会错）】

- n : `int`，排列元素个数，元素值域 $1..n$ ；结果有 $n!$ 个， $n \leq 10$ 左右用，再大内存会爆。
- 返回值：`vector<vector<int>>`，第 i 个元素是一个长度为 n 的排列；下标 $0..n-1$ ，值域 $1..n$ 。

【API / 入口函数（赛时只认这里列的名字）】

- `enumerate_permutations(int n) ->` 完整的“枚举 $1..n$ 的全部排列”骨架。需要加约束时，在循环里、压入 `path` 之前加入 `continue` 剪枝即可。返回 `vector<vector<int>>`。

```
// 完整的“枚举 1..n 的全部排列”骨架。
// 需要加约束时，在循环里、压入 path 之前加入 continue 剪枝即可。
vector<vector<int>> enumerate_permutations(int n) {
    vector<vector<int>> answers;
    vector<int> path, used(n + 1);
    auto dfs = [&](auto&& self) -> void {
        if ((int)path.size() == n) {
            answers.push_back(path);
            return;
        }
        for (int x = 1; x <= n; ++x) {
            if (used[x]) continue;
            used[x] = 1;
            path.push_back(x);
            self(self);
            path.pop_back();
            used[x] = 0;
        }
    };
    dfs(dfs);
    return answers;
}
```

典题模型：全排列、排列型构造、 $n \leq 10$ 左右的暴力验证。先调用

`enumerate_permutations(n)`；若只需要最优答案，把 `answers.push_back(path)` 换成更新答案即可。N

皇后、数独等题将“当前状态是否合法”的剪枝放在 `if (used[x]) continue`；

后面，并保证“压入、递归、弹出”三步完整（压入和弹出成对出现）。

双向 BFS

赛时先看

题目信号：状态数很大，但最短距离不深；普通 BFS 从一侧会爆。

本质：无权图最短路、状态空间搜索，起点和终点都已知。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；`solve()` 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定：大约从 $O(b^d)$ 降到 $O(b^{(d/2)})$ ， b 为分支数。

维护的量： da/db （起点侧/终点侧到各状态的最短距离）； qa/qb （两侧待扩展队列，每次扩展较短的队）；相遇时 $d1[v]+d2[v]$ 就是答案。

警告：每次扩展状态少的一边；状态哈希要统一；相遇时返回两侧距离和。

【最小完整示例（先抄这一段就能跑）】

题目：从 0 出发，每次 $x \rightarrow x+1$ 或 $x \rightarrow 2x$ ，求到 5 的最少步数（State 用 int）。

```
auto get_next = [](int x) { return vector<int>{x + 1, x * 2}; };
int ans = bidirectional_bfs(0, 5, get_next); // 两端交替扩展，相遇即最短路
cout << ans << "\n";
```

样例：ans = 3 (0 → 2 → 4 → 5)。

【传参要求（照这个传不会错）】

- s / t ：起/终点状态，模板参数 State 需支持 == 并能做 unordered_map 的 key (int、string、vector<int> 等)； $s == t$ 时直接返回 0。
- get_next：可调用对象，输入一个状态返回 vector<State>（所有下一状态）；内部用 $d1/d2$ 判重，允许自环/重边。
- 返回值：int，起点到终点的最短步数；不可达返回 -1。

```
template <class State, class Next>
int bidirectional_bfs(State s, State t, Next get_next) {
    if (s == t) return 0;
    unordered_map<State, int> da, db;
    queue<State> qa, qb;
    da[s] = 0; db[t] = 0;
    qa.push(s); qb.push(t);
    auto expand = [&](queue<State>& q, unordered_map<State, int>& d1,
                    unordered_map<State, int>& d2) -> int {
        int sz = (int)q.size();
        while (sz-- > 0) {
            State u = q.front();
            q.pop();
            for (State v : get_next(u)) {
                if (d1.count(v)) continue;
                d1[v] = d1[u] + 1;
                if (d2.count(v)) return d1[v] + d2[v];
                q.push(v);
            }
        }
        return -1;
    };
    while (!qa.empty() && !qb.empty()) {
        int ans;
        if (qa.size() <= qb.size()) ans = expand(qa, da, db);
        else ans = expand(qb, db, da);
        if (ans != -1) return ans;
    }
    return -1;
}
```

A* 最短路：第 k 短路骨架

赛时先看

题目信号：题面出现第 k 短路；边权非负；需要按路径长度从小到大弹出状态。

本质：求有向图从 s 到 t 的第 k 短路，或带启发函数的最短路搜索。

接法：把本节 struct/class/函数组 整段抄到 solve() 上面；solve()

里先构造对象，再按下方“不会用就看这里”调用公开接口。

复杂度判定：和输出路径数量相关，常见为 $O(k m \log(km))$ 量级。

维护的量： h （反向 Dijkstra 得到的各点到 t 的最短路，作启发值）； cnt （每个点被弹出的次数，第 k 次到 t 即答案）； pq （按 $f = g + h[u]$ 升序的优先队列）。

警告：启发函数必须不超过真实剩余距离；若 $s == t$ ，空路径算一条最短路径，通常要把 k 加一 ($k++$) 再搜。

【最小完整示例（先抄这一段就能跑）】

题目：3 点有向图 $1 \rightarrow 2(1)$ ， $2 \rightarrow 3(1)$ ， $1 \rightarrow 3(4)$ ，求 1 到 3 的第 1、2 短路。

```
int n = 3;
vector<vector<Edge>> g(n + 1), rg(n + 1); // 1-indexed
g[1] = {{2, 1}, {3, 4}}; g[2] = {{3, 1}};
rg[2] = {{1, 1}}; rg[3] = {{2, 1}, {1, 4}}; // rg 是 g 的反图
cout << kth_shortest_path(n, g, rg, 1, 3, 1) << "\n";
```

```
cout << kth_shortest_path(n, g, rg, 1, 3, 2) << "\n";
```

样例：第 1 短路 2，第 2 短路 4。

【传参要求（照这个传不会错）】

- n ：点数，编号 $1..n$ ，邻接表要开 $n+1$ 大小。
- g / rg ：正向/反向邻接表 `vector<vector<Edge>>`，`Edge{to, w}` 边权非负； rg 是 g 每条边反向后的图，只用于求启发值。
- s / t ：起点/终点， $1 \leq s, t \leq n$ ； $s == t$ 时函数内部自动把 k 加 1（空路径算一条最短路径）。
- k ：要第几条短路， $k \geq 1$ 。
- 返回值：`i64`，第 k 短路长度；路径不足 k 条返回 `-1`。

【改板时先认这几个量】

- g ：正向邻接表（ rg 为反向图，用来从终点跑 Dijkstra 求启发值）。
- cur ：刚从优先队列弹出的状态。

```
struct Edge { int to; i64 w; };

vector<i64> dijkstra_rev(int n, const vector<vector<Edge>>& rg, int t) {
    const i64 INF = (1LL << 62);
    vector<i64> h(n + 1, INF);
    priority_queue<pair<i64, int>, vector<pair<i64, int>>, greater<>> pq;
    h[t] = 0;
    pq.push({0, t});
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (d != h[u]) continue;
        for (auto e : rg[u]) {
            if (h[e.to] > d + e.w) {
                h[e.to] = d + e.w;
                pq.push({h[e.to], e.to});
            }
        }
    }
    return h;
}

i64 kth_shortest_path(int n, const vector<vector<Edge>>& g,
                      const vector<vector<Edge>>& rg, int s, int t, int k) {
    const i64 INF = (1LL << 62);
    vector<i64> h = dijkstra_rev(n, rg, t);
    if (h[s] == INF) return -1;
    if (s == t) k++;
    struct Node {
        i64 f, g;
        int u;
        bool operator<(const Node& other) const { return f > other.f; }
    };
    priority_queue<Node> pq;
    vector<int> cnt(n + 1, 0);
    pq.push({h[s], 0, s});
    while (!pq.empty()) {
        auto cur = pq.top();
        pq.pop();
        int u = cur.u;
        cnt[u]++;
        if (u == t && cnt[u] == k) return cur.g;
        if (cnt[u] > k) continue;
        for (auto e : g[u]) {
            if (h[e.to] == INF) continue;
            i64 ng = cur.g + e.w;
            pq.push({ng + h[e.to], ng, e.to});
        }
    }
    return -1;
}
```

IDA* 迭代加深搜索

赛时先看

题目信号：需要最少步数；普通 BFS 内存爆；可以设计乐观估价函数。

本质：状态空间很深但答案步数较小，常见于拼图、魔板、棋盘最少步数。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；`solve()` 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定：指数级，但剪枝强时可过。

维护的量：limit（当前深度上限，逐轮增大）；on_path（当前递归路径上的状态集合，防绕圈）；next_limit（本轮最小的超限 f 值，作为下一轮 limit）。

警告：估价函数 h() 不能高估真实剩余步数；每层深度上限逐步增加。

【最小完整示例（先抄这一段就能跑）】

题目：数字游戏，从 0 出发每次 $x \rightarrow x+1$ 或 $x \rightarrow x+2$ ，求到 5 的最少步数（State 用 int）。

```
struct Hash { size_t operator()(int x) const { return (size_t)x; } };
auto heuristic = [](int x) { return (5 - x + 1) / 2; }; // 可采纳下界：每步最多 +2
auto is_goal = [](int x) { return x == 5; };
auto get_next = [](int x) { return vector<int>{x + 1, x + 2}; };
int steps = ida_star(0, Hash(), heuristic, is_goal, get_next);
cout << steps << "\n";
```

样例：steps = 3 (0 → 2 → 4 → 5)。

【传参要求（照这个传不会错）】

- start: 初始状态，模板参数 State 需支持 ==（供 unordered_set 判重）。
- hash: State 的哈希函数对象，如 struct Hash { size_t operator()(const State&) const; }。
- heuristic: State → int，乐观估价，必须满足 $h(x) \leq$ 真实剩余步数，高估会剪掉最优解。
- is_goal: State → bool，判断是否到达目标状态。
- get_next: State → vector<State>，返回所有合法下一状态。
- 返回值: int，从 start 到目标的最少步数（找到答案时的 limit）；无解返回 -1。

```
// State 需要可比较相等，并由 Hash 提供 unordered_set 所需的哈希。
// heuristic(s) 必须是到目标的可采纳下界；get_next(s) 返回所有合法下一状态。
template <class State, class Hash, class Heuristic, class IsGoal, class Next>
int ida_star(State start, Hash hash, Heuristic heuristic, IsGoal is_goal, Next get_next) {
    constexpr int FOUND = -1;
    unordered_set<State, Hash> on_path(0, hash);
    on_path.insert(start);

    auto dfs = [&](auto&& self, const State& state, int g, int limit) -> int {
        int f = g + heuristic(state);
        if (f > limit) return f;
        if (is_goal(state)) return FOUND;
        int next_limit = INT_MAX;
        for (const State& next : get_next(state)) {
            if (!on_path.insert(next).second) continue; // 防止当前搜索路径绕圈。
            int result = self(self, next, g + 1, limit);
            on_path.erase(next);
            if (result == FOUND) return FOUND;
            next_limit = min(next_limit, result);
        }
        return next_limit;
    };

    int limit = heuristic(start);
    while (true) {
        int result = dfs(dfs, start, 0, limit);
        if (result == FOUND) return limit;
        if (result == INT_MAX) return -1;
        limit = result;
    }
}
```

DLX 精确覆盖

赛时先看

题目信号：题面能转成 0/1 矩阵，每行表示一个选择，每列表示一个约束，每个约束恰好满足一次。

本质：精确覆盖问题，例如数独、重复覆盖、把若干集合选出来使每个元素恰好覆盖一次。

接法：把本节 struct/class/函数组 整段抄到 solve() 上面；外部只调用下面 API

列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定：指数级，实际依赖约束稀疏程度和列选择剪枝。

维护的量：sz[c]（第 c 列含 1 的个数，用于选最少列）；L/R/U/D（十字链表左右上下指针）；row/col（每个节点所在行列）；answer（当前已选行号，回溯时 pop）。

警告：列头、左右上下指针初始化；每次选含 1 最少的列；恢复顺序与删除相反。

【最小完整示例（先抄这一段就能跑）】

题目：3 列 0/1 矩阵，行 1 覆盖 {1,2}、行 2 覆盖 {2,3}、行 3 覆盖 {1,3}，每列恰好被选中行覆盖一次，求选哪些行。

```
DLX dlx(10, 3);           // max_nodes 给大一点，列数 3
dlx.add_row(1, {1, 2});    // 第 1 行覆盖列 1、2
dlx.add_row(2, {2, 3});
dlx.add_row(3, {1, 3});
bool ok = dlx.dance();      // 是否有精确覆盖
if (ok) for (int r : dlx.answer) cout << r << " ";
```

样例：ok = true, answer = {1, 2}（行 1 + 行 2 恰好覆盖全部 3 列）。

【传参要求（照这个传不会错）】

- init(max_nodes, cols): max_nodes 是节点总数上界（行数 × 每行列数 + cols，给大点没事），cols 是列数（约束数）；列编号 1..cols, 0 是表头。
- add_row(r, cols): r 是行号（任意正整数，会原样存进 answer）；cols 是该行覆盖的列编号集合，同一行内不能有重复列。
- dance(): 求解，返回 bool 是否有解；有解时 answer 里按选择顺序存行号（这里示例是 {1,2}，选择顺序可能不同）。
- max_nodes 给太小会越界，不确定时按 行数 * 列数 + 列数 + 5 给。

【API / 入口函数（赛时只认这里列的名字）】

- init(int max_nodes, int cols) -> 初始化/清空结构

```
struct DLX {
    int n_col, idx;
    vector<int> L, R, U, D, row, col, sz;
    vector<int> answer;

    DLX(int max_nodes = 0, int cols = 0) { init(max_nodes, cols); }

    void init(int max_nodes, int cols) {
        n_col = cols;
        L.assign(max_nodes + cols + 5, 0);
        R.assign(max_nodes + cols + 5, 0);
        U.assign(max_nodes + cols + 5, 0);
        D.assign(max_nodes + cols + 5, 0);
        row.assign(max_nodes + cols + 5, 0);
        col.assign(max_nodes + cols + 5, 0);
        sz.assign(cols + 1, 0);
        for (int i = 0; i <= cols; i++) {
            L[i] = i - 1; R[i] = i + 1;
            U[i] = D[i] = i;
        }
        L[0] = cols; R[cols] = 0;
        idx = cols;
        answer.clear();
    }

    void add_row(int r, const vector<int>& cols) {
        int first = 0;
        for (int c : cols) {
            int x = ++idx;
            row[x] = r; col[x] = c; sz[c]++;
            U[x] = U[c]; D[x] = c;
            D[U[c]] = x; U[c] = x;
            if (!first) {
                first = x;
                L[x] = R[x] = x;
            } else {
                L[x] = L[first]; R[x] = first;
                R[L[first]] = x; L[first] = x;
            }
        }
    }

    void remove_col(int c) {
        L[R[c]] = L[c]; R[L[c]] = R[c];
        for (int i = D[c]; i != c; i = D[i]) {
            for (int j = R[i]; j != i; j = R[j]) {
                U[D[j]] = U[j]; D[U[j]] = D[j];
                sz[col[j]]--;
            }
        }
    }

    void restore_col(int c) {
        for (int i = U[c]; i != c; i = U[i]) {

```

```

        for (int j = L[i]; j != i; j = L[j]) {
            sz[col[j]]++;
            U[D[j]] = j; D[U[j]] = j;
        }
        L[R[c]] = c; R[L[c]] = c;
    }

    bool dance() {
        if (R[0] == 0) return true;
        int c = R[0];
        for (int j = R[0]; j != 0; j = R[j]) {
            if (sz[j] < sz[c]) c = j;
        }
        if (sz[c] == 0) return false;
        remove_col(c);
        for (int i = D[c]; i != c; i = D[i]) {
            answer.push_back(row[i]);
            for (int j = R[i]; j != i; j = R[j]) remove_col(col[j]);
            if (dance()) return true;
            for (int j = L[i]; j != i; j = L[j]) restore_col(col[j]);
            answer.pop_back();
        }
        restore_col(c);
        return false;
    }
};

```

de Bruijn 序列：最短覆盖全部长度 n 串的循环串

赛时先看

题目信号：要求最短地枚举/覆盖全部二进制串或 k 进制串；相邻窗口需要重叠 $n-1$ 位；题面可能明确写“每种长度为 n 的密码/状态恰好一次”。

本质：在大小为 k 的字符集上，构造长度恰为 k^n 的循环序列，使每个长度为 n 的串恰好作为循环子串出现一次。

接法：直接调用 `de_bruijn(k, n)` 得到循环序列；题目要普通字符串时，在末尾补上前 $n-1$ 个字符再拼成字符串。

复杂度判定：时间和输出长度都是 $O(k^n)$ ，额外空间 $O(kn)$ 。输出本身就有 k^n 个字符，输出量过大的题目用不了。

维护的量：`a`（长度 $k*n+1$ 的构造数组，存当前候选串）；`sequence`（输出的循环序列）；`dfs` 的 `t/p`（当前位置/最小周期候选）。

警告：返回值是循环序列而非线性字符串，跨结尾的窗口也算。这里 k^n 的长度必须能放进内存；不要把 n 和字母表大小 k 写反。

【最小完整示例（先抄这一段就能跑）】

```

// k=2, n=3: 输出二进制 de Bruijn 循环序列（长度  $2^3=8$ ）。
auto seq = de_bruijn(2, 3);
for (int x : seq) cout << x;
cout << '\n'; // 00010111
string s;
for (int x : seq) s.push_back(char('0' + x));
s += s.substr(0, 2); // 补前 n-1 个字符转成线性串
cout << s << '\n'; // 0001011100

```

- 样例：循环串输出 `00010111`；补 2 个字符后是 `0001011100`。

【传参要求（照这个传不会错）】

- k ：字符集大小， ≥ 1 ；序列元素取值 $[0, k)$ 。
- n ：窗口长度/阶数， ≥ 1 。
- 返回值：`vector<int>`，长度恰为 k^n ，是循环序列（跨结尾的窗口也算一次）；题目要线性字符串时自己补前 $n-1$ 个字符。

【API / 入口函数（赛时只认这里列的名字）】

- `de_bruijn(int k, int n)` $\rightarrow$ 返回 k 元、 n 阶 de Bruijn 循环序列，长度恰为 k^n 。

使用：`de_bruijn(k, n)` 返回一个循环序列，元素在 $[0, k)$ 。若题目要普通字符串，在末尾补上前 $n-1$ 个字符；二进制 CSES De Bruijn Sequence 就是这个做法。

典题模型：CSES De Bruijn Sequence；最短测试串覆盖所有长度固定的状态；基于 de Bruijn 图的欧拉回路构造。

```

// 维护的量：a（长度  $k*n+1$  的构造数组，存当前候选串）；sequence（最终输出的 de Bruijn 循环序列）。
// 不变量：dfs(t, p) 结束时 a[1..n] 是周期为 p 的候选串；仅当 n 是 p 的倍数时输出其最小周期，
// 全部候选按字典序拼接后恰好覆盖每个  $k$  元  $n$  阶串各一次（FKM/Lyndon 词构造）。
vector<int> de_bruijn(int k, int n) {
    assert(k >= 1 && n >= 1);

```

```
vector<int> a(k * n + 1), sequence;
function<void(int, int)> dfs = [&](int t, int p) {
    if (t > n) {
        if (n % p == 0) {
            for (int i = 1; i <= p; ++i) sequence.push_back(a[i]); // 只输出最小周期 p，避免重复
        }
        return;
    }
    a[t] = a[t - p];
    dfs(t + 1, p);
    for (int x = a[t - p] + 1; x < k; ++x) {
        a[t] = x;
        dfs(t + 1, t); // 出现比周期内更大的新字符，最小周期更新为 t
    }
};
dfs(1, 1);
return sequence;
}
```

折半搜索 Meet-in-the-Middle

赛时先看

题目信号： 2^n 太大但 $2^{(n/2)}$ 可接受（如 $n \leq 40$ 的选子集、双数组配对题）。

本质： n 约 40 的子集枚举。

接法：给 $n \leq 40$ 个数，选子集使和不超过 `limit` 且最大，直接 `max_subset_sum_leq(a, limit)`；若题目要“两半各选一个凑目标”，也是同一套“枚举一半 + 排序 + 二分/双指针”骨架。

复杂度判定： $O(2^{(n/2)} \log 2^{(n/2)})$ 。

维护的量：`sums`（当前半边枚举出的所有子集和，各 $2^{(n/2)}$ 个）；`ans`（合并时维护的最大可行和）。

警告：左右两半分别枚举，再排序二分/双指针。

【最小完整示例（先抄这一段就能跑）】

```
// n=6 个数，选一个子集使和不超过 12 且最大。
vector<i64> a = {2, 5, 3, 7, 9, 1};
cout << max_subset_sum_leq(a, 12) << '\n'; // 12 (如 5+7 或 2+3+7)
```

- 样例：输出 12。

【传参要求（照这个传不会错）】

- `subset_sums(a, l, r)`：枚举区间 $[l, r)$ 内元素的全部子集和，返回 `vector<i64>`（内部函数，一般不用直接调）。
- `max_subset_sum_leq(a, limit)`：`a` 为 0-indexed 原数组（`vector<i64>`）；返回不超过 `limit` 的最大子集和；若所有子集和都大于 `limit`，返回 `LLONG_MIN`。
- 元素为负时子集和可能为负，`LLONG_MIN` 的“无解”判断不受影响。

【改板时先认这几个量】

- `sums`：当前半边枚举出的所有子集和。
- `ans`：当前合并后得到的最大可行和。

```
// 维护的量：sums（区间 [l,r) 内元素枚举出的全部子集和）。
// 不变量：返回的 sums 是区间元素所有子集和的完整列表，长度恰为  $2^{(r-l)}$ 。
vector<i64> subset_sums(const vector<i64>& a, int l, int r) {
    vector<i64> sums{0};
    for (int i = l; i < r; ++i) {
        int sz = (int)sums.size();
        for (int j = 0; j < sz; ++j) sums.push_back(sums[j] + a[i]); // 每个旧子集和扩出含 a[i] 的新一半
    }
    return sums;
}

// 维护的量：left/right（两半各自的子集和表）；ans（合并过程中维护的最大可行和）。
// 不变量：枚举 left 的每个 x，用二分找 right 中  $\leq \text{limit} - x$  的最大值，x+它就是一个候选答案。
i64 max_subset_sum_leq(const vector<i64>& a, i64 limit) {
    int n = (int)a.size();
    auto left = subset_sums(a, 0, n / 2);
    auto right = subset_sums(a, n / 2, n);
    sort(right.begin(), right.end());
    i64 ans = LLONG_MIN;
    for (i64 x : left) {
        auto it = upper_bound(right.begin(), right.end(), limit - x); // 右半和必须补足到 limit 内
        if (it != right.begin()) ans = max(ans, x + *prev(it));
    }
    return ans;
}
```

B 数据结构与离线维护

03 基础数据结构与区间维护

从并查集、链表、树状数组、离线区间、线段树到单调结构，按比赛最常翻的维护套路排列。

并查集 DSU

赛时先看

题目信号：题面出现“合并”“连通”“朋友关系”“等价类”“最小生成树 Kruskal”。要动态维护“把两个集合并起来 + 判断两点是否同集合”。

本质：每个集合选一个代表元，其余点通过 `parent` 指针指向它；合并只改两个代表元的指向，路径压缩 + 按大小合并保证树高近似 $O(\log n)$ ，所以均摊接近 $O(1)$ 。

复杂度判定：`find/unite/same` 均摊接近 $O(1)$ ； n 到 $1e6$ 随便用。

维护的量：`parent[x]` (x 的父节点，代表元的 `parent` 指向自己)；`sz[x]` (x 作为根时所在集合大小)。

接法：先 `DSU dsu(n)`；每读到一条“合并/建立关系/同队伍”操作就

`dsu.unite(u,v)`；每读到“是否连通/是否同类”就

`dsu.same(u,v)`。如果题目还要求维护两点距离差、奇偶关系、敌友关系，普通 DSU

不够，要翻“带权并查集”或“关系并查集”。

警告：初始化到 n ；路径压缩和按大小合并一起用。

【最小完整示例（先抄这一段就能跑）】

题目： n 个点， m 条操作。`op=1 u v`：合并 u, v ；`op=2 u v`：判断 u, v 是否连通。

```
DSU dsu(n); // 1. 结构体定义：DSU(点数 n)，编号 1..n
while (m--) {
    int op, u, v;
    cin >> op >> u >> v;
    if (op == 1) dsu.unite(u, v); // 2. 调用：合并
    else cout << (dsu.same(u, v) ? "YES" : "NO") << '\n'; // 3. 调用：判连通
}
```

样例：`unite(1,2)`, `unite(2,3)` 后 `same(1,3)` $\rightarrow$ YES; `same(1,4)` $\rightarrow$ NO。

【传参要求（照这个传不会错）】

- `DSU(n)`：构造；编号范围 $0..n$ 都可用（题目从 0 开始也能直接用）。
- `unite(u, v)`：合并 u, v 所在集合；无返回值（内部自动按大小合并）。
- `same(u, v)`： u, v 是否在同一集合；返回 `bool`。
- `find(x)`： x 所在集合的代表元；`size(x)`： x 所在集合大小。
- 要维护“距离差/奇偶关系/敌友”时普通 DSU 不够，翻“带权并查集”“关系并查集”。

【不会用就照抄】

```
DSU dsu(n); // 点编号 1..n
dsu.unite(u, v); // 合并 u,v
if (dsu.same(u, v)) { } // 是否同一集合
int s = dsu.size(u); // u 所在集合大小
```

- `find(x)` 返回代表元；一般题里优先用 `unite/same/size`。
- 多测时每组重新构造 `DSU dsu(n)`，不要沿用上一组状态。

【API / 入口函数（赛时只认这里列的名字）】

- `DSU dsu(n)` $\rightarrow$ 建一个下标 $0..n$ 都合法的并查集；竞赛一般只用 $1..n$ 。
- `dsu.unite(u,v)` $\rightarrow$ 合并两集合；本来已连通返回 `false`，成功合并返回 `true`。
- `dsu.same(u,v)` $\rightarrow$ 判断两点当前是否在同一集合。
- `dsu.size(u)` $\rightarrow$ 返回 u 所在集合大小。
- `dsu.find(u)` $\rightarrow$ 返回代表元；一般只有需要代表元编号时才直接调用。

【抄板清单（照着做就行）】

1. 抄哪段：整个 `DSU` 结构体，抄到 `solve()` 外面。

- 构造: `DSU dsu(n);`, n 取点编号上限。
- 调用: 合并 `dsu.unite(u,v)`; 判连通 `dsu.same(u,v)`; 查大小 `dsu.size(u)`。
- 取结果: `unite` 返回 `true` 表示这次合并新连上了两个集合, `false` 表示本来就连通; `same` 直接给布尔值。

【改造点（按题目改这几处）】

- 编号: 点从 0 开始也能直接用 (0..n 都是合法下标), 不需要偏移。
- 数连通块个数: 外面维护 `cnt`, 初值 `n`, 每次 `unite` 返回 `true` 就 `--cnt`。
- 要带边权/点关系: 这是朴素并查集, 换“带权并查集”节。

【核心逻辑（改代码时别破坏）】

- `find(x)` 把点映射到集合代表元; 路径压缩负责把树压扁。
- `unite(a,b)` 只合并两个代表元, 按 `sz` 小并大, 避免树变高。

【改板时先认这几个量】

- `sz`: 集合/子树大小。
- `parent`: 并查集父节点。

```
// 维护的量: parent[x] (x 的父节点, 代表元的 parent 指向自己)、sz[x] (x 为根时所在集合大小)。
// 不变量: 每个集合有唯一代表元; find(x) 沿 parent 走到代表元, 路径压缩后树高近似  $O(\log n)$ 。
struct DSU {
    vector<int> parent, sz;

    DSU(int n = 0) { init(n); }

    void init(int n) {
        parent.resize(n + 1);
        sz.assign(n + 1, 1);
        iota(parent.begin(), parent.end(), 0);
    }

    int find(int x) {
        // 路径压缩: 沿途点直接挂到代表元下, 之后 find 都是  $O(1)$  量级
        return parent[x] == x ? x : parent[x] = find(parent[x]);
    }

    bool unite(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b); // 按大小合并: 小的挂到大的树上, 压低树高
        parent[b] = a;
        sz[a] += sz[b];
        return true;
    }

    bool same(int a, int b) {
        return find(a) == find(b);
    }

    int size(int x) {
        return sz[find(x)];
    }
};
```

单链表: 头插、尾插、删除、反转与有序去重

赛时先看

题目信号: 题面强调“在某节点后插入/删除”“只能沿 `next`

向后走”“反转链表”“删除重复结点”; 若频繁按下标随机访问, 应该用 `vector`, 不是链表。

本质: 维护只能从头向后遍历的动态序列; 已知前驱节点时 $O(1)$

插入或删除。适合链式队列、链式哈希桶、按出现顺序动态增删, 或题目明确要求模拟单链表。

接法: 连续操作 `H x` 头插、`T x` 尾插、`I p x` 在编号 `p` 后插入、`D p` 删除 `p` 后一个节点。用 `push_front`、`push_back`、`insert_after`、`erase_after`

直接模拟; 若题目只给值而不给节点编号, 先遍历找到前驱, 复杂度就会变为 $O(n)$ 。

复杂度判定: 头插、尾插、已知前驱后的插删均为 $O(1)$; 按值查找/删除、遍历、反转与去重为 $O(n)$ 。下标池总空间 $O(\text{操作数})$, 本模板删除后不复用编号。

维护的量: `nodes` (节点池, 存 `value/next`); `head/tail` (表头/表尾节点编号, `-1` 表示空表); `length` (当前节点数)。

警告: 单链表删除一个节点需要它的前驱, 头结点要单独处理; 删除后的旧编号不可再作为有效节点使用; 反转后旧 `head` 就是新 `tail`。有序去重只适用于相同值相邻的已排序链表。

【最小完整示例（先抄这一段就能跑）】

题目：n 次操作。H x 头插值 x，T x 尾插值 x，I p x 在编号 p 的节点后插值 x，D p 删除编号 p 后的节点；最后输出链表当前值。

```
StaticSinglyLinkedList list;           // 1. 结构体定义：直接构造，无需参数
list.push_front(3);                    // 2. 调用：头插 3，返回新节点编号
int p = list.push_back(5);             // 3. 调用：尾插 5，返回节点编号
list.insert_after(p, 7);               // 4. 调用：在编号 p 的节点后插入 7
list.erase_after(p);                  // 5. 调用：删除 p 后的节点（7）
for (int v : list.to_vector()) cout << v << ' '; // 6. 遍历输出
```

样例：push_front(3)、push_back(5)、insert_after(p,7)、erase_after(p) 后链表为 [3,5]。

【传参要求（照这个传不会错）】

- StaticSinglyLinkedList(): 构造；无参；空表时 head = tail = -1。
- push_front(value) / push_back(value): 头/尾插入；value 任意 int；返回新节点编号（从 0 起，编号不复用）。
- insert_after(position, value): 在编号 position 的节点后插入；要求 $0 \leq \text{position} < \text{nodes.size()}$ ；返回新节点编号。
- erase_front(): 删除头节点；空表返回 false，否则返回 true。
- erase_after(position): 删除 position 的后继；无后继返回 false；要删头结点需单独处理。
- erase_first_value(value): 删除第一个值等于 value 的节点；找到返回 true，否则 false。
- reverse(): 原地反转链表，无返回值；deduplicate_sorted() 仅适用于已排序链表去重。
- to_vector(): 返回按链表顺序的 vector<int> 值序列。

```
struct StaticSinglyLinkedList {
    struct Node {
        int value;
        int next = -1;
    };

    vector<Node> nodes;
    int head = -1, tail = -1;
    int length = 0;

    bool empty() const { return head == -1; }

    int make_node(int value) {
        nodes.push_back({value, -1});
        return (int)nodes.size() - 1;
    }

    int push_front(int value) {
        int id = make_node(value);
        nodes[id].next = head;
        head = id;
        if (tail == -1) tail = id;
        ++length;
        return id;
    }

    int push_back(int value) {
        int id = make_node(value);
        if (tail == -1) head = tail = id;
        else {
            nodes[tail].next = id;
            tail = id;
        }
        ++length;
        return id;
    }

    int insert_after(int position, int value) {
        assert(0 <= position && position < (int)nodes.size());
        int id = make_node(value);
        nodes[id].next = nodes[position].next;
        nodes[position].next = id;
        if (tail == position) tail = id;
        ++length;
        return id;
    }

    bool erase_front() {
        if (head == -1) return false;
        head = nodes[head].next;
        if (head == -1) tail = -1;
        --length;
    }
};
```

```

    return true;
}

bool erase_after(int position) {
    assert(0 <= position && position < (int)nodes.size());
    int erased = nodes[position].next;
    if (erased == -1) return false;
    nodes[position].next = nodes[erased].next;
    if (tail == erased) tail = position;
    --length;
    return true;
}

bool erase_first_value(int value) {
    if (head == -1) return false;
    if (nodes[head].value == value) return erase_front();
    for (int previous = head; nodes[previous].next != -1; previous = nodes[previous].next) {
        if (nodes[nodes[previous].next].value == value) return erase_after(previous);
    }
    return false;
}

void reverse() {
    int previous = -1, current = head;
    tail = head;
    while (current != -1) {
        int following = nodes[current].next;
        nodes[current].next = previous;
        previous = current;
        current = following;
    }
    head = previous;
}

void deduplicate_sorted() {
    for (int current = head; current != -1 && nodes[current].next != -1; ) {
        int following = nodes[current].next;
        if (nodes[current].value == nodes[following].value) {
            nodes[current].next = nodes[following].next;
            if (tail == following) tail = current;
            --length;
        } else {
            current = following;
        }
    }
}

vector<int> to_vector() const {
    vector<int> result;
    for (int current = head; current != -1; current = nodes[current].next) {
        result.push_back(nodes[current].value);
    }
    return result;
}
};

```

双向循环链表：哨兵、 $O(1)$ 插删、移动与约瑟夫环

赛时先看

题目信号：题面出现“在某元素前/后插入”“删除后仍要得到前后邻居”“把一个元素移到开头/结尾”“环形报数”；若操作只发生在两端，普通 deque 往往更短。

本质：维护循环顺序，并在已知节点编号时 $O(1)$ 插入、删除、移动到首尾。适合 LRU 顺序、桌面牌堆/队列模拟、字符串光标两侧操作、约瑟夫环和“删除当前后继续走”的题。

接法：报数到第 m 人就离开，问最后留下的编号，调用

josephus_survivor(n, m)。若每次删除后还需向左走，删除前同时保存 `previous[current]`；若题目要求按第 k 个位置删除且 n, q 很大，链表不能快速跳第 k 个，应改用树状数组或平衡树。

复杂度判定：已知节点的插入、删除、移动均为 $O(1)$ ；按值寻找节点和完整遍历为 $O(n)$ 。空间 $O(\text{操作数})$ 。

维护的量：value/previous/next（三个平行数组，节点 0 恒为哨兵）；length（链表长度）。

警告：节点 0 是哨兵，不存数据；空表时

`next[0]=prev[0]=0`。删除节点前先保存下一节点，避免用已脱链节点继续走；`move_to_front/back` 只能操作仍在链表中的节点。此模板也不复用已删节点编号。

【最小完整示例（先抄这一段就能跑）】

题目：约瑟夫环： n 个人围一圈报数，报到 m 的人离队，问最后留下的人的编号。

```
int n, m;
```



```
cin >> n >> m;
int survivor = josephus_survivor(n, m); // 1. 调用: n 人, 报数到 m 离队
cout << survivor << '\n';           // 2. 输出: 最后留下的人的编号 (1-based)
```

样例: josephus_survivor(7, 3) -> 输出 4。

【传参要求 (照这个传不会错)】

- josephus_survivor(n, step): n = 人数 (≥ 1), step = 报数到第几离队 (≥ 1); 返回最后留下者的编号 (1-based)。
- push_front(x) / push_back(x): 在表头/表尾插入; 返回新节点编号 (从 1 起, 0 是哨兵, 编号不复用)。
- insert_after(pos, x) / insert_before(pos, x): 在节点 pos 后/前插入; pos 必须已链接 (或为哨兵 0); 返回新节点编号。
- erase(node): 删除节点 node; 要求 node 已链接且不是哨兵 0; 无返回值。
- move_to_front(node) / move_to_back(node): 把节点 node 移到表头/表尾; 要求 node 仍在链表中。
- first() / last(): 返回第一个/最后一个数据节点编号 (空表返回 0)。
- next_alive(node): 返回 node 的后继; 碰到哨兵时绕回第一个节点。

【API / 入口函数 (赛时只认这里列的名字)】

- push_front(x) / push_back(x) -> 在表头/表尾插入, 返回新节点编号。
- insert_after(pos, x) / insert_before(pos, x) -> 在节点 pos 后/前插入, 返回新节点编号。
- erase(int node) -> 删除节点; 节点 0 是哨兵, 不能删。
- move_to_front(node) / move_to_back(node) -> 把节点移到表头/表尾。
- josephus_survivor(n, m) -> 报数到第 m 个人离队, 返回最后留下的编号。

```
struct IndexDoublyCircularList {
    vector<int> value, previous, next;
    int length = 0;

    IndexDoublyCircularList() : value(1), previous(1, 0), next(1, 0) {}

    bool empty() const { return length == 0; }
    int first() const { return next[0]; }
    int last() const { return previous[0]; }
    bool linked(int node) const { return node != 0 && previous[node] != node; }

    int make_node(int x) {
        value.push_back(x);
        int id = (int)value.size() - 1;
        previous.push_back(id);
        next.push_back(id); // 单独节点: 尚未接入循环链表。
        return id;
    }

    void link_between(int left, int node, int right) {
        next[left] = node;
        previous[node] = left;
        next[node] = right;
        previous[right] = node;
    }

    int insert_after(int position, int x) {
        assert(position == 0 || linked(position));
        int node = make_node(x);
        link_between(position, node, next[position]);
        ++length;
        return node;
    }

    int insert_before(int position, int x) {
        assert(position == 0 || linked(position));
        return insert_after(previous[position], x);
    }

    int push_front(int x) { return insert_after(0, x); }
    int push_back(int x) { return insert_before(0, x); }

    void erase(int node) {
        assert(linked(node));
        int left = previous[node], right = next[node];
        next[left] = right;
        previous[right] = left;
        previous[node] = next[node] = node; // 标记为脱链, 防止重复删除。
        --length;
    }
}
```

```

void move_to_front(int node) {
    assert(linked(node));
    int left = previous[node], right = next[node];
    next[left] = right;
    previous[right] = left;
    link_between(0, node, next[0]);
}

void move_to_back(int node) {
    assert(linked(node));
    int left = previous[node], right = next[node];
    next[left] = right;
    previous[right] = left;
    link_between(previous[0], node, 0);
}

int next_alive(int node) const {
    int result = next[node];
    return result == 0 ? next[0] : result;
}

vector<int> to_vector() const {
    vector<int> result;
    for (int node = next[0]; node != 0; node = next[node]) result.push_back(value[node]);
    return result;
}
};

int josephus_survivor(int n, int step) {
    assert(n >= 1 && step >= 1);
    IndexDoublyCircularList list;
    for (int x = 1; x <= n; ++x) list.push_back(x);
    int current = list.first();
    while (list.length > 1) {
        for (int count = 1; count < step; ++count) current = list.next_alive(current);
        int following = list.next_alive(current);
        list.erase(current);
        current = following;
    }
    return list.value[list.first()];
}

```

后继并查集：区间中跳过已处理位置

赛时先看

题目信号：循环里总在“从 1

开始找下一个还没处理的位置”，每个下标最多处理一次；普通逐格扫描会因重叠区间反复访问同一批位置。

本质：初始有位置 $1..n$ 都未处理；一个位置一旦处理就永久删除，要求快速找到不小于 x

的第一个未处理位置。适合区间染色、区间赋值、批量访问未访问节点。

接法：有若干次操作“把区间 $[l, r]$ 中尚未涂色的位置涂成颜色

c ”，每个位置只需第一次涂色。用上面的循环枚举仍存活的位置，写 `color[x]=c`

后删除它；总时间是所有位置数加查询数的近线性，而不是所有区间长度之和。

复杂度判定：初始化 $O(n)$ ；所有 `find` 与删除摊还 $O(\alpha(n))$ ，整轮每个位置只删除一次。

维护的量：`parent[x]`（位置 x 链向的下一个存活位置；`find(x)` 即第一个存活位置）。

警告：只能永久向右删除，不能恢复； $n+1$ 是哨兵，表示不存在可用位置；调用 `erase_and_next(x)` 前必须保证 x 当前未删除。

【最小完整示例（先抄这一段就能跑）】

题目： n 个位置初始都未涂色；每次把区间 $[l, r]$ 中还没涂色的位置涂成颜色 c ，每个位置只涂一次。

```

SuccessorDSU dsu(n); // 1. 结构体定义：n 个位置，编号 1..n
for (int x = dsu.first_alive(1); x <= r; x = dsu.erase_and_next(x)) {
    color[x] = c; // 2. 调用：涂色后删除，x 跳到下一个未处理位置
}

```

样例： $n=6$ ，先涂 $[2,5]$ 再涂 $[3,4]$ $\rightarrow$ 每个位置恰好涂一次，共涂 4 次。

【传参要求（照这个传不会错）】

- `SuccessorDSU(n)`：构造； n = 位置数（编号 $1..n$ ），内部自动开 $n+1$ 哨兵。
- `init(n_)`：重置为 $n_$ 个位置，全部初始存活。
- `find(x)`：返回第一个 $\geq x$ 的存活位置（可能是 $n+1$ ，表示没有）。
- `first_alive(x)`：等价于 `find(x)`，返回第一个存活位置。

- `erase_and_next(x)`: 删除 `x` 并返回下一个存活位置（可能 `n+1`）；要求 `x` 当前未删除。

【API / 入口函数（赛时只认这里列的名字）】

- `find(int x)` -> 查代表元/查找结果 返回 `int`。
- `init(int n_)` -> 初始化/清空结构
- `erase_and_next(int x)` -> 删除仍存活的 `x`，并返回下一个存活位置，可能是 `n+1`。

【改板时先认这几个量】

- `parent`: 并查集父节点。

```
struct SuccessorDSU {
    int n;
    vector<int> parent;

    explicit SuccessorDSU(int n_ = 0) {
        if (n_) init(n_);
    }

    void init(int n_) {
        n = n_;
        parent.resize(n + 2);
        iota(parent.begin(), parent.end(), 0);
    }

    int find(int x) {
        return parent[x] == x ? x : parent[x] = find(parent[x]);
    }

    int first_alive(int x) { return find(x); }

    // 删除仍存活的 x，并返回下一个存活位置，可能是 n+1。
    int erase_and_next(int x) {
        parent[x] = find(x + 1);
        return parent[x];
    }
};

// 把 [1,r] 中仍存活的每个下标恰好处理一次：
// 示例: for (int x = dsu.first_alive(1); x <= r; x = dsu.erase_and_next(x)) {
// 示例: process(x);
// 公式/约定: }
```

可撤销并查集

赛时先看

题目信号: 边会删除；操作可以离线；需要回到某个历史状态。

本质: 离线动态连通性、时间分治、需要撤销合并。

接法: 把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；外部只调用下面 API

列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定: 合并 $O(\log n)$ 以内，回滚按合并次数。

维护的量: `parent`（父节点，不路径压缩）；`sz`（子树大小）；`history`（合并记录栈，回滚时弹栈复原）；`components`（当前连通块数）。

警告: 不能路径压缩，否则无法回滚。

【最小完整示例（先抄这一段就能跑）】

题目: `n` 个点、`m` 条边按顺序加入，过程中要回到某个历史状态再继续加边（离线动态连通性）。

```
RollbackDSU dsu(n);           // 1. 结构体定义: n 个点, 编号 1..n
int snap = dsu.snapshot();     // 2. 调用: 记录当前历史长度, 返回快照编号
dsu.unite(u, v);               // 3. 调用: 合并 u、v 所在集合
bool ok = dsu.same(x, y);      // 4. 调用: 判断 x、y 是否同一集合
dsu.rollback(snap);            // 5. 调用: 撤销 snap 之后的所有合并
```

样例: `n=3`; `snapshot()` 后 `unite(1,2)`、`unite(2,3)`, 再 `rollback(snap)` -> 三个点恢复独立。

【传参要求（照这个传不会错）】

- `RollbackDSU(n)`: 构造; `n` = 点数（编号 `1..n`）。
- `init(n)`: 重置为 `n` 个独立点，并清空历史栈。
- `find(x)`: 返回 `x` 所在集合代表元（不路径压缩，约 $O(\log n)$ ）。
- `unite(a, b)`: 合并 `a`、`b` 所在集合；已同集合返回 `false`，否则合并成功返回 `true`。

- `snapshot()`: 返回当前历史长度 (int), 即“快照编号”。
- `rollback(snap)`: 回滚到快照 `snap` 时的状态; `snap` 必须是之前 `snapshot()` 的返回值。

【API / 入口函数 (赛时只认这里列的名字)】

- `find(int x)` -> 查代表元/查找结果 返回 `int`。
- `init(int n)` -> 初始化/清空结构
- `unite(int a, int b)` -> 合并两个集合 返回 `bool`。
- `snapshot()` -> 记录当前历史长度, 返回快照编号。
- `rollback(int snap)` -> 回滚到快照 `snap` 时的状态。

【改板时先认这几个量】

- `sz`: 集合/子树大小。
- `parent`: 并查集父节点。

```
struct RollbackDSU {
    vector<int> parent, sz;
    vector<pair<int, int>> history;
    int components = 0;

    RollbackDSU(int n = 0) { init(n); }

    void init(int n) {
        parent.resize(n + 1);
        sz.assign(n + 1, 1);
        iota(parent.begin(), parent.end(), 0);
        history.clear();
        components = n;
    }

    int find(int x) const {
        while (parent[x] != x) x = parent[x];
        return x;
    }

    int snapshot() const {
        return (int)history.size();
    }

    bool unite(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) {
            history.push_back({-1, -1});
            return false;
        }
        if (sz[a] < sz[b]) swap(a, b);
        history.push_back({b, sz[a]});
        parent[b] = a;
        sz[a] += sz[b];
        components--;
        return true;
    }

    void rollback(int snap) {
        while ((int)history.size() > snap) {
            auto [b, old_size] = history.back();
            history.pop_back();
            if (b == -1) continue;
            int a = parent[b];
            parent[b] = b;
            sz[a] = old_size;
            components++;
        }
    }
};
```

带权并查集

赛时先看

题目信号: 约束形如 `value[y] - value[x] = w`, 需要判断矛盾。

本质: 维护集合内点之间的相对权值, 如距离差、势能差。

接法: 把本节 `struct/class/函数组` 整段抄到 `solve()` 上面; 外部只调用下面 API 列出的接口, 不要把内部递归参数直接当题面参数。

复杂度判定: 均摊接近 `O(1)`。

维护的量： `parent`（父节点）；`sz`（子树大小）；`diff[x]`（`value[x]-value[parent[x]]`，压缩后即 `value[x]-value[root]`）。

警告： `diff[x]` 表示 `value[x] - value[parent[x]]`；路径压缩时要累加。

【最小完整示例（先抄这一段就能跑）】

题目：n 个变量，q 条约束 `value[y] - value[x] = w`，判断约束是否矛盾。

```
WeightedDSU dsu(n);           // 1. 结构体定义：n 个变量，编号 1..n
bool ok = dsu.unite(x, y, w);  // 2. 调用：加约束 value[y]-value[x]=w；与已知矛盾返回 false
i64 d = dsu.difference(x, y);  // 3. 调用：同一集合时返回 value[y]-value[x]
```

样例：`unite(1,2,3)`、`unite(2,3,4)` → `difference(1,3)=7`；再 `unite(3,1,2)` 会矛盾返回 `false`。

【传参要求（照这个传不会错）】

- `WeightedDSU(n)`：构造；n = 变量数（编号 1..n）。
- `init(n)`：重置为 n 个独立变量，相对值全为 0。
- `find(x)`：路径压缩查找；调用后 `diff[x]` 更新为 `value[x]-value[root]`。
- `weight(x)`：返回 `value[x]-value[root]`（根节点相对值视为 0），i64。
- `unite(x, y, w)`：加约束 `value[y]-value[x]=w`；不同集合则合并返回 `true`；同集合返回约束是否满足（矛盾为 `false`）。
- `difference(x, y)`：要求 x,y 同一集合；返回 `value[y]-value[x]`，i64。

【API / 入口函数（赛时只认这里列的名字）】

- `find(int x)` → 查代表元/查找结果 返回 `int`。
- `init(int n)` → 初始化/清空结构
- `unite(int x, int y, i64 w)` → 合并两个集合 返回 `bool`。

【改板时先认这几个量】

- `sz`：集合/子树大小。
- `diff`：公式：`value[x]-value[parent[x]]`。

```
struct WeightedDSU {
    vector<int> parent, sz;
    vector<i64> diff; // 公式：value[x]-value[parent[x]]。

    WeightedDSU(int n = 0) { init(n); }

    void init(int n) {
        parent.resize(n + 1);
        sz.assign(n + 1, 1);
        diff.assign(n + 1, 0);
        iota(parent.begin(), parent.end(), 0);
    }

    int find(int x) {
        if (parent[x] == x) return x;
        int p = parent[x];
        parent[x] = find(parent[x]);
        diff[x] += diff[p];
        return parent[x];
    }

    i64 weight(int x) {
        find(x);
        return diff[x]; // 公式：value[x]-value[root]。
    }

    bool unite(int x, int y, i64 w) {
        // 加入约束：value[y] - value[x] = w。
        int rx = find(x), ry = find(y);
        i64 wx = weight(x), wy = weight(y);
        if (rx == ry) return wy - wx == w;
        if (sz[rx] < sz[ry]) {
            parent[rx] = ry;
            diff[rx] = wy - wx - w; // 公式：value[rx]-value[ry]。
            sz[ry] += sz[rx];
        } else {
            parent[ry] = rx;
            diff[ry] = wx + w - wy; // 公式：value[ry]-value[rx]。
            sz[rx] += sz[ry];
        }
        return true;
    }
}
```

```
i64 difference(int x, int y) {
    // 要求同一集合: 返回 value[y] - value[x]。
    return weight(y) - weight(x);
}
};
```

带权并查集例题：区间和约束判矛盾

赛时先看

题目信号：区间和、前缀和差值、真假话/矛盾数。

本质：给出若干断言 $\text{sum}[1..r] = s$ ，判断有多少条与之前断言矛盾。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API

列出的接口，不要把内部递归参数直接当题面参数。

> - 复杂度判定： $O(m \alpha(n))$ 。> - 维护的量：`pre[i]`（前缀和节点，编号 `i+1`）；带权并查集的 `weight`（到根的差值）。> - 警告：令 $\text{pre}[i] = \text{sum}[1..i]$ ，约束变成 $\text{pre}[r] - \text{pre}[l-1] = s$ 。> - 约定：`WeightedDSUExample dsu(n + 1)`；前缀节点 `0..n` 都合法；代码把 `pre[i]` 映射到编号 `i+1`。

【最小完整示例（先抄这一段就能跑）】

题目：`n` 个变量，`m` 条约束 $\text{sum}[1..r] = s$ （即 $\text{pre}[r] - \text{pre}[l-1] = s$ ），输出与前面约束矛盾的条数。

```
WeightedDSUExample dsu(n + 1); // 1. 结构体定义：前缀节点 0..n，编号映射为 i+1
int bad = 0;
while (m--) {
    int l, r;
    i64 s;
    cin >> l >> r >> s;
    int x = l; // 2. pre[l-1] 的编号：l-1+1
    int y = r + 1; // 3. pre[r] 的编号：r+1
    if (!dsu.unite(x, y, s)) bad++; // 4. 与已知约束矛盾，计数加 1
}
cout << bad << '\n';
```

样例：`unite(1,3,6)`（ $\text{sum}[1..3]=6$ ）、`unite(2,3,3)`（ $\text{sum}[2..3]=3$ ）都成立；再加 `unite(2,3,5)` 与第二条矛盾返回 `false` $\rightarrow$ `bad = 1`。

【传参要求（照这个传不会错）】

- `WeightedDSUExample(n + 1)`：构造；`n` = 变量个数；前缀节点 `0..n` 都合法，代码把 `pre[i]` 映射到编号 `i+1`。
- `unite(x, y, w)`：加约束 $\text{value}[y] - \text{value}[x] = w$ ；不同集合直接合并返回 `true`，同集合返回约束是否满足（矛盾为 `false`）。
- `find(x)` / `weight(x)`：路径压缩查找与相对权值；一般题里直接 `unite` 判矛盾即可。
- `count_contradictions(n, statements)`：一行求出矛盾条数；`statements` 每个元素是 `{l, r, s}`（1-indexed 闭区间 $\text{sum}[1..r]=s$ ），返回 `int`。

【API / 入口函数（赛时只认这里列的名字）】

- `find(int x)` $\rightarrow$ 查代表元/查找结果 返回 `int`。
- `init(int n)` $\rightarrow$ 初始化/清空结构
- `unite(int x, int y, i64 w)` $\rightarrow$ 合并两个集合 返回 `bool`。

【改板时先认这几个量】

- `sz`：集合/子树大小。
- `parent`：并查集父节点。
- `diff`：公式： $\text{value}[x] - \text{value}[\text{parent}[x]]$ 。

```
struct WeightedDSUExample {
    vector<int> parent, sz;
    vector<i64> diff;

    WeightedDSUExample(int n = 0) { init(n); }
    void init(int n) {
        parent.resize(n + 1);
        sz.assign(n + 1, 1);
        diff.assign(n + 1, 0);
        iota(parent.begin(), parent.end(), 0);
    }
    int find(int x) {
        if (parent[x] == x) return x;
        int p = parent[x];
```

```

    parent[x] = find(parent[x]);
    diff[x] += diff[p];
    return parent[x];
}
i64 weight(int x) { find(x); return diff[x]; }
bool unite(int x, int y, i64 w) {
    int rx = find(x), ry = find(y);
    i64 wx = weight(x), wy = weight(y);
    if (rx == ry) return wy - wx == w;
    parent[rx] = ry;
    diff[rx] = wy - wx - w;
    return true;
}
};

int count_contradictions(int n, const vector<tuple<int, int, i64>>& statements) {
    WeightedDSUExample dsu(n + 1); // 前缀点为 0..n; 若 DSU 使用 1-based, 需要整体 +1 偏移。
    int bad = 0;
    for (auto [l, r, s] : statements) {
        int x = l; // 表示 pre[l-1] 偏移后的编号: l-1+1。
        int y = r + 1; // 表示 pre[r] 偏移后的编号: r+1。
        if (!dsu.unite(x, y, s)) bad++;
    }
    return bad;
}

```

关系并查集：食物链 mod 3

赛时先看

题目信号：关系有模意义，常见 0=同类，1=吃，2=被吃。

本质：维护三类循环关系，如 A 吃 B、同类等。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定：均摊接近 $O(1)$ 。

维护的量：`parent`（父节点）；`rel[x]`（`x` 到 `parent[x]` 的关系，mod 3；0=同类，1=吃，2=被吃）。

警告：这里 `rel[x]` 表示 `x` 到父亲的关系；`relation(x,y)` 返回 `x` 相对 `y`。

【最小完整示例（先抄这一段就能跑）】

题目：食物链：`n` 个动物，`q` 句话。`1 x y` 表示同类，`2 x y` 表示 `x` 吃 `y`；统计与前面已确认信息矛盾的假话数。

```

ModDSU dsu(n); // 1. 结构体定义: n 个动物, 编号 1..n
bool ok = dsu.unite(x, y, r); // 2. 调用: 加约束 x->y 关系为 r (0=同类, 1=吃)
if (!ok) bad++; // 3. 统计: 与已知信息矛盾, 记为假话

```

样例：`unite(1,2,1)`（1 吃 2）、`unite(2,3,1)`（2 吃 3）→ `relation(1,3)=2`（1 被 3 吃）。

【传参要求（照这个传不会错）】

- `ModDSU(n)`：构造；`n` = 元素数（编号 `1..n`）。
- `init(n)`：重置为 `n` 个独立元素。
- `find(x)`：路径压缩查找；压缩后 `rel[x]` 为 `x` 到根的关系（mod 3）。
- `relation(x, y)`：要求 `x,y` 同一集合；返回 `x -> y` 的关系（0=同类，1=吃，2=被吃）。
- `unite(x, y, r)`：加约束 `x -> y` 关系为 `r`（`r` 取 0/1/2）；不同集合则合并返回 `true`；同集合返回已有关系是否等于 `r`。

【API / 入口函数（赛时只认这里列的名字）】

- `find(int x)` → 查代表元/查找结果 返回 `int`。
- `init(int n)` → 初始化/清空结构
- `unite(int x, int y, int r)` → 合并两个集合 返回 `bool`。

【改板时先认这几个量】

- `rel`：`rel[x]` 表示 `x` 到 `parent[x]` 的关系，mod 3。
- `parent`：并查集父节点。

```

struct ModDSU {
    vector<int> parent, rel; // rel[x] 表示 x 到 parent[x] 的关系, mod 3。

    ModDSU(int n = 0) { init(n); }

    void init(int n) {
        parent.resize(n + 1);
    }
}

```



```

    rel.assign(n + 1, 0);
    iota(parent.begin(), parent.end(), 0);
}

int find(int x) {
    if (parent[x] == x) return x;
    int p = parent[x];
    parent[x] = find(parent[x]);
    rel[x] = (rel[x] + rel[p]) % 3;
    return parent[x];
}

int relation(int x, int y) {
    // 确保同一集合后, 返回 x -> y 的关系。
    find(x); find(y);
    return (rel[x] - rel[y] + 3) % 3;
}

bool unite(int x, int y, int r) {
    // 加入约束: x -> y 的关系为 r。
    int rx = find(x), ry = find(y);
    if (rx == ry) return relation(x, y) == r;
    parent[rx] = ry;
    rel[rx] = (r + rel[y] - rel[x] + 3) % 3;
    return true;
}
};

```

树状数组：单点加，区间和

赛时先看

题目信号：数组有单点加/改；反复问 $[1, r]$ 的和；或“前缀和”“逆序对计数”。看到单点改+区间和，先想 BIT。

本质：把前缀和拆成 $O(\log n)$ 段 lowbit 块累加：bit[i] 维护下标 i 往前 lowbit(i)

个位置的和，查询前缀时逐段相加。

复杂度判定：修改/查询 $O(\log n)$ ；n 到 $1e6$ 仍可用（常数极小）；要区间加请换“区间加区间和”双 BIT 版；要区间最值换线段树。

维护的量：n（长度）；bit[i]（第 i 个 lowbit 块的和，1-indexed）。

接法：把题目里的“当前位置贡献多少”变成 add(pos, value)；把“前 r 个的总和”变成 sum_prefix(r)；区间 $[1, r]$ 的和就是 range_sum(1, r)。如果原值很大但只关心相对大小，先离散化成 1..k 再开树状数组；如果要找第 k 个，需要所有频次非负。

警告：下标从 1 开始，add(0, ...) 死循环；range_sum(1, r)=prefix(r)-prefix(1-1)；答案可能到 $1e18$ 用 i64。

【最小完整示例（先抄这一段就能跑）】

题目：n 个数，q 次操作。op=1 x y：把 a[x] 加 y；op=2 l r：输出 a[1..r] 的和。

```

Fenwick<i64> fw(n); // 1. 结构体定义: Fenwick<i64>(长度 n)
for (int i = 1; i <= n; ++i) fw.add(i, a[i]); // 2. 初始化: 把 a[i] 加进下标 i
while (q--) {
    int op, x, y;
    cin >> op >> x >> y;
    if (op == 1) fw.add(x, y); // 3. 调用: a[x] += y
    else cout << fw.range_sum(x, y) << '\n'; // 4. 调用: 输出 a[x..y] 的和
}

```

样例：n=5, a=[1,2,3,4,5]；add(2,10) 后查 [1,5] -> 输出 25。

【传参要求（照这个传不会错）】

- Fenwick<T>(n)：构造；n = 下标上限（数组长度）；内部 1-indexed。
- add(pos, delta)：a[pos] += delta；要求 $1 \leq \text{pos} \leq n$ ，pos=0 死循环。
- sum_prefix(r)：a[1..r] 的和；要求 $0 \leq r \leq n$ (r=0 返回 0)。
- range_sum(l, r)：闭区间 $[l, r]$ 的和；要求 $1 \leq l \leq r \leq n$ 。
- lower_bound(k)：前缀和单调非负时，找最小 pos 使前缀和 $\geq k$ 。
- 返回值：和，类型 = 模板参数 T；数大用 i64。

【不会用就照抄】

```

Fenwick<i64> fw(n);
fw.add(pos, delta); // a[pos] += delta
auto s1 = fw.sum_prefix(r); // a[1..r] 的和
auto s2 = fw.range_sum(l, r); // a[l..r] 的和

```

- 下标必须从 1 开始，add(0, ...) 会死循环。

- 这是“加法版”，不是“把 `a[pos]` 改成 `x`”；赋值要自己先算差值。

【API / 入口函数（赛时只认这里列的名字）】

- `Fenwick<i64> fw(n)` -> 建立 1-indexed 树状数组。
- `fw.add(pos, delta)` -> 执行 `a[pos] += delta`；`pos` 不能为 0。
- `fw.sum_prefix(r)` -> 返回 `a[1..r]` 的和。
- `fw.range_sum(l, r)` -> 返回闭区间 `a[1..r]` 的和。
- `fw.lower_bound(k)` -> 前缀和非负且单调时，找最小 `pos` 使前缀和 $\geq k$ 。

【抄板清单（照着做就行）】

1. 抄哪段：整个 Fenwick 结构体。
2. 构造：`Fenwick<i64> fw(n);`，`n` 为下标上限。
3. 调用：单点加 `fw.add(pos, delta)`；区间和 `fw.range_sum(l, r)`。
4. 取结果：`range_sum` 直接就是答案；要把 `a[pos]` 改成 `x`，先查当前值再 `add(pos, x - 当前值)`。

【改造点（按题目改这几处）】

- 模板参数：答案/权值可能超 `int` 就写 `Fenwick<i64>`。
- 赋值：BIT 只支持“加”，改成 `x` 要自己先算差值 `add(pos, x - 旧值)`。
- 值域大：先离散化成 `1..k` 再开 BIT。
- 当计数器用（逆序对等）：每次 `add(pos, 1)` 统计出现次数。

【核心逻辑（改代码时别破坏）】

- `bit[i]` 管的是长度为 `lowbit(i)` 的后缀块。
- 修改一路 `i += lowbit(i)`；前缀查询一路 `i -= lowbit(i)`；区间和 = 两个前缀相减。

```
// 维护的量：n（长度）；bit[i] = 下标 i 往前 lowbit(i) 个位置的和，1-indexed。
// 不变量：前缀和 sum(x) = bit[x] + bit[x-lowbit(x)] + ...，共 O(log n) 段。
template <class T>
struct Fenwick {
    int n;
    vector<T> bit;

    Fenwick(int n = 0) { init(n); }

    void init(int n_) {
        n = n_;
        bit.assign(n + 1, T{});
    }

    void add(int idx, T val) {
        // 一路向右上走，更新所有覆盖 idx 的 lowbit 块
        for (; idx <= n; idx += idx & -idx) bit[idx] += val;
    }

    T sum_prefix(int idx) const {
        T res{};
        // 从右往左拆成 O(log n) 个 lowbit 块累加
        for (; idx > 0; idx -= idx & -idx) res += bit[idx];
        return res;
    }

    T range_sum(int l, int r) const {
        if (l > r) return T{};
        return sum_prefix(r) - sum_prefix(l - 1); // 区间和 = 两个前缀和相减
    }

    int lower_bound(T target) const {
        assert(n > 0);
        int pos = 0;
        int k = 1;
        while ((k << 1) <= n) k <<= 1; // 最大的不超过 n 的 2 的幂，避免依赖 GCC 的 __lg。
        for (; k >= 1; k >>= 1) {
            int next = pos + k;
            if (next <= n && bit[next] < target) {
                pos = next;
                target -= bit[next];
            }
        }
        return pos + 1;
    }
};
```

`Fenwick::lower_bound(target)` 返回最小下标 `pos` 使 `sum_prefix(pos) >= target`，适合“按频次找第 `k` 小 / 找第 `k` 个未删位置”。前提是所有维护值非负，且 `1 <= target <= sum_prefix(n)`；若有负数，前缀和不再单调，不能二分。

树状数组求逆序对

赛时先看

题目信号：问交换次数、逆序数量、相对顺序混乱程度。

本质：统计 $i < j$ 且 $a[i] > a[j]$ 的对数。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；`solve()` 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O(n \log n)$ 。

维护的量：`Fenwick<int> fw`（值域频次计数，下标 $1..k$ ）；`ans`（已累计的逆序对数）。

警告：值域大时必须离散化。

【最小完整示例（先抄这一段就能跑）】

题目：`n` 个数，求 $i < j$ 且 $a[i] > a[j]$ 的逆序对总数。

```
vector<int> a = {3, 1, 4, 2}; // 1. 数据：原数组（0-indexed，值域任意）
i64 ans = count_inversions(a); // 2. 调用：一行求出逆序对数（内部自动离散化）
cout << ans << '\n'; // 3. 输出
```

样例：`a=[3,1,4,2]` $\rightarrow$ 输出 3（(3,1)、(3,2)、(4,2) 共 3 对）。

【传参要求（照这个传不会错）】

- `count_inversions(a)`：`a` = 原数组（`vector<int>`，0-indexed，值域任意）；函数内自动 `sort+unique+lower_bound` 离散化，无需预处理；返回 `i64` 逆序对数。
- 依赖本文件上方 `Fenwick` 模板，抄板时一起抄。
- 数据是 1-indexed 时，先转成 0-indexed 再传（或复制 `a[1..n]`）。

```
// 约定：本函数为 0-indexed。
i64 count_inversions(vector<int> a) {
    vector<int> xs = a;
    sort(xs.begin(), xs.end());
    xs.erase(unique(xs.begin(), xs.end()), xs.end());

    Fenwick<int> fw((int)xs.size());
    i64 ans = 0;
    for (int i = 0; i < (int)a.size(); ++i) {
        int id = int(lower_bound(xs.begin(), xs.end(), a[i]) - xs.begin()) + 1;
        ans += i - fw.sum_prefix(id);
        fw.add(id, 1);
    }
    return ans;
}
```

树状数组：区间加，区间和

赛时先看

题目信号：操作是 `add l r x` 和 `sum l r`。

本质：区间整体加值，同时查询区间和。

接法：题面操作是“给 $[1, r]$ 每个数都加 x ”就调用 `range_add(1, r, x)`；题面问“ $[1, r]$ 当前总和”就调用 `range_sum(1, r)`。如果题目问的是最大值、最小值或最大子段和，不要硬改这个模板，直接翻线段树。

复杂度判定： $O(\log n)$ 。

维护的量：`n`（长度）；`b1`（差分 BIT）；`b2`（加权差分 BIT，存 `val*(idx-1)`）。

警告：两个 BIT 的公式要配套，不要混用。

【最小完整示例（先抄这一段就能跑）】

题目：`n` 个数，`q` 次操作。`op=1 l r x`：`a[1..r]` 每个数加 x ；`op=2 l r`：输出 `a[1..r]` 的和。

```
RangeFenwick rf(n); // 1. 结构体定义：长度 n，内部两个 Fenwick<i64>
while (q--) {
    int op, l, r;
    i64 x;
    cin >> op >> l >> r;
    if (op == 1) { cin >> x; rf.range_add(l, r, x); } // 2. 调用：区间加
    else cout << rf.range_sum(l, r) << '\n'; // 3. 调用：输出区间和
}
```

}

样例: `n=5, a=[1,2,3,4,5]; range_add(2,4,10)` 后 `range_sum(2,4)` -> 输出 39。

【传参要求（照这个传不会错）】

- `RangeFenwick(n)`: 构造; `n` = 数组长度 (下标 `1..n`)。
- `init(n_)`: 重置长度为 `n_`, 并清空两个 BIT。
- `range_add(l, r, val)`: 闭区间 `[l,r]` 每个位置加 `val`; 要求 `1 <= l <= r <= n`; 无返回值。
- `range_sum(l, r)`: 返回闭区间 `[l,r]` 的和, `i64`; 要求 `1 <= l <= r <= n`。
- 初始值为 0: 数组有初值时逐个 `range_add(i, i, a[i])` 打进去。

【API / 入口函数（赛时只认这里列的名字）】

- `init(int n_)` -> 初始化/清空结构
- `range_add(int l, int r, i64 val)` -> 闭区间加值
- `range_sum(int l, int r)` -> 查询闭区间和 返回 `i64`。

```
struct RangeFenwick {
    int n;
    Fenwick<i64> b1, b2;

    RangeFenwick(int n = 0) { init(n); }

    void init(int n_) {
        n = n_;
        b1.init(n);
        b2.init(n);
    }

    void add_internal(int idx, i64 val) {
        b1.add(idx, val);
        b2.add(idx, val * (idx - 1));
    }

    void range_add(int l, int r, i64 val) {
        add_internal(l, val);
        add_internal(r + 1, -val);
    }

    i64 prefix_sum(int idx) const {
        return b1.sum_prefix(idx) * idx - b2.sum_prefix(idx);
    }

    i64 range_sum(int l, int r) const {
        return prefix_sum(r) - prefix_sum(l - 1);
    }
};
```

二维树状数组

赛时先看

题目信号: 矩阵上动态修改点值, 查询子矩形和。

本质: 二维单点加、矩形求和。

接法: 把本节 `struct/class/函数组` 整段抄到 `solve()` 上面; 外部只调用下面 API 列出的接口, 不要把内部递归参数直接当题面参数。

复杂度判定: 修改/查询 $O(\log n \log m)$ 。

维护的量: `n`、`m` (行列数); `bit[i][j]` (二维 lowbit 块的和, 全 1-indexed)。

警告: 二维下标都从 1 开始; 矩形和用四个前缀相减。

【最小完整示例（先抄这一段就能跑）】

题目: `n*m` 矩阵, `q` 次操作。op=1 `x y v`: `a[x][y] += v`; op=2 `x1 y1 x2 y2`: 输出子矩形 `[x1..x2] × [y1..y2]` 的和。

```
Fenwick2D<i64> fw(n, m); // 1. 结构体定义: n 行 m 列, 行列都从 1 开始
while (q--) {
    int op, x, y, v;
    cin >> op;
    if (op == 1) { cin >> x >> y >> v; fw.add(x, y, v); } // 2. 调用: 单点加
    else {
        int x1, y1, x2, y2;
        cin >> x1 >> y1 >> x2 >> y2;
        cout << fw.rect_sum(x1, y1, x2, y2) << '\n'; // 3. 调用: 输出矩形和
    }
}
```

```
    }
}
```

样例: $n=m=3$; `add(2,2,5)` 后 `rect_sum(1,1,3,3)` $\rightarrow$ 输出 5。

【传参要求（照这个传不会错）】

- `Fenwick2D<T>(n, m)`: 构造; n/m = 行数/列数; 内部下标全 1-based。
- `init(n_, m_)`: 重置为 $n_$ 行 $m_$ 列, 全部清零。
- `add(x, y, val)`: `a[x][y] += val`; 要求 $1 \leq x \leq n, 1 \leq y \leq m$ 。
- `sum_prefix(x, y)`: 子矩形 $[1..x] \times [1..y]$ 的和, 返回 T 。
- `rect_sum(x1,y1,x2,y2)`: 闭矩形 $[x1..x2] \times [y1..y2]$ 的和 (四个前缀相减); 要求 $1 \leq x1 \leq x2 \leq n, 1 \leq y1 \leq y2 \leq m$ 。
- 模板参数 T : 和可能很大就写 `i64`。

【API / 入口函数（赛时只认这里列的名字）】

- `add(int x, int y, T val)` $\rightarrow$ 加入一个元素/贡献
- `init(int n_, int m_)` $\rightarrow$ 初始化/清空结构
- `sum_prefix(int x, int y)` $\rightarrow$ 查询前缀和 返回 T 。
- `rect_sum(x1, y1, x2, y2)` $\rightarrow$ 查询闭矩形 $[x1..x2] \times [y1..y2]$ 的和 返回 T 。

```
template <class T>
struct Fenwick2D {
    int n, m;
    vector<vector<T>> bit;

    Fenwick2D(int n = 0, int m = 0) { init(n, m); }

    void init(int n_, int m_) {
        n = n_;
        m = m_;
        bit.assign(n + 1, vector<T>(m + 1, T{}));
    }

    void add(int x, int y, T val) {
        for (int i = x; i <= n; i += i & -i) {
            for (int j = y; j <= m; j += j & -j) bit[i][j] += val;
        }
    }

    T sum_prefix(int x, int y) const {
        T res{};
        for (int i = x; i > 0; i -= i & -i) {
            for (int j = y; j > 0; j -= j & -j) res += bit[i][j];
        }
        return res;
    }

    T rect_sum(int x1, int y1, int x2, int y2) const {
        return sum_prefix(x2, y2) - sum_prefix(x1 - 1, y2) -
            sum_prefix(x2, y1 - 1) + sum_prefix(x1 - 1, y1 - 1);
    }
};
```

区间颜色种类数：颜色很多且无修改

赛时先看

题目信号: 颜色值很大, 无法用 `bitmask`; 数组不修改。

本质: 静态数组, 多次查询 $[1, r]$ 中不同颜色/数字数量。

接法: 把本节 `struct/class/函数组` 整段抄到 `solve()` 上面; 外部只调用下面 API 列出的接口, 不要把内部递归参数直接当题面参数。

复杂度判定: $O((n+q) \log n)$ 。

维护的量: `fw` (FenwickCount 位置计数 BIT, 当前扫描区间内每个位置贡献 1); `last` (每种颜色最后一次出现的位置); `cur` (已扫描到的右指针)。

警告: 按右端点排序, 遇到重复颜色时把上一次出现位置从 BIT 中删掉。

【最小完整示例（先抄这一段就能跑）】

题目: 静态数组 `a[1..n]`, q 次询问 $[1, r]$ 内不同颜色/数字的个数 (颜色值很大, 不能 `bitmask`)。

```
vector<Query> qs;
for (int i = 0; i < q; ++i) {
```

```
int l, r;
cin >> l >> r;
qs.push_back({l, r, id}); // 1. 收集询问: {l, r, 原始编号 id}
}
vector<int> ans = static_distinct_count(a, qs); // 2. 调用: a 必须 1-indexed
for (int i = 0; i < q; ++i) cout << ans[i] << '\n'; // 3. 按原始编号输出
```

样例: $a=[1,2,1,3,2]$; 问 $[1,3] \rightarrow 2$; $[2,5] \rightarrow 3$ 。

【传参要求（照这个传不会错）】

- a : 原数组, 必须 1-indexed (长度 $n+1$, $a[0]$ 不参与); 值域任意, 用 `unordered_map` 记上次出现位置。
- qs : 询问数组, 元素 `Query{l, r, id}`; l/r 为 1-indexed 闭区间, id 为 0-based 原始编号。
- 返回值: `vector<int>`, $ans[id]$ 就是第 id 个询问的答案。
- 只支持静态无修改的询问; 所有询问先读完 (离线)。

【API / 入口函数（赛时只认这里列的名字）】

- `add(int i, T v) →` 加入一个元素/贡献
- `init(int n_) →` 初始化/清空结构
- `range_sum(int l, int r) →` 查询闭区间和 返回 T 。

【改板时先认这几个量】

- `bit`: FenwickCount 内部树状数组。
- `cur`: 当前指针 (已扫到的位置)。
- `sum`: 区间和/计数和。

```
template <class T>
struct FenwickCount {
    int n;
    vector<T> bit;
    FenwickCount(int n = 0) { init(n); }
    void init(int n_) { n = n_; bit.assign(n + 1, T{}); }
    void add(int i, T v) { for (; i <= n; i += i & -i) bit[i] += v; }
    T sum(int i) const { T r{}; for (; i > 0; i -= i & -i) r += bit[i]; return r; }
    T range_sum(int l, int r) const { return sum(r) - sum(l - 1); }
};

struct Query {
    int l, r, id;
};

vector<int> static_distinct_count(const vector<int>& a, vector<Query> qs) {
    int n = (int)a.size() - 1;
    sort(qs.begin(), qs.end(), [](const Query& x, const Query& y) {
        return x.r < y.r;
    });
    FenwickCount<int> fw(n);
    unordered_map<int, int> last;
    vector<int> ans(qs.size());
    int cur = 0;
    for (auto q : qs) {
        while (cur < q.r) {
            cur++;
            if (last.count(a[cur])) fw.add(last[a[cur]], -1);
            fw.add(cur, 1);
            last[a[cur]] = cur;
        }
        ans[q.id] = fw.range_sum(q.l, q.r);
    }
    return ans;
}
```

离线统计区间不同数字个数

赛时先看

题目信号: 数组静态, 很多区间询问 distinct count。

本质: 多次询问 $[1, r]$ 内不同数的数量。

接法: 把本节 struct/class/函数组 整段抄到 `solve()` 上面; `solve()`

里先构造对象, 再按下方“不会用就看这里”调用公开接口。

复杂度判定: $O((n+q) \log n)$ 。

维护的量: `fw` (FenwickDistinct 位置计数 BIT); `last` (每个值上次出现的位置); `cur` (已扫描到的右指针)。

警告: 按右端点排序, 当前值上一次出现位置要从 BIT 中减掉。与 13 节「区间颜色种类数」同一模型, 二选一即可。

【最小完整示例（先抄这一段就能跑）】

题目：静态数组 $a[1..n]$ ， q 次询问 $[l, r]$ 内不同数字的个数。

```
vector<DistinctQuery> qs;
for (int i = 0; i < q; ++i) {
    int l, r;
    cin >> l >> r;
    qs.push_back({l, r, i}); // 1. 收集询问: {l, r, 原始编号 id}
}
vector<int> ans = distinct_count_queries(a, qs); // 2. 调用: a 必须 1-indexed
for (int i = 0; i < q; ++i) cout << ans[i] << '\n'; // 3. 按原始编号输出
```

样例： $a=[1,2,1,3,2]$ ；问 $[1,3] \rightarrow 2$ ； $[2,5] \rightarrow 3$ 。

【传参要求（照这个传不会错）】

- a ：原数组，必须 1-indexed（长度 $n+1$ ， $a[0]$ 不参与）；值域任意。
- qs ：询问数组，元素 $\text{DistinctQuery}\{l, r, id\}$ ； l/r 为 1-indexed 闭区间， id 为 0-based 原始编号。
- 返回值： $\text{vector}\langle\text{int}\rangle$ ， $\text{ans}[id]$ 就是第 id 个询问的答案。
- 只支持静态无修改；与「区间颜色种类数」同一模型，二选一即可。

```
struct DistinctQuery {
    int l, r, id;
};

// 精简版 Fenwick (int 版本)，本节自带: add / range_sum。
// 与 13 节同一模型，二选一即可；本节的类型名带 Distinct 前缀，与其它节混抄也不会重名。
struct FenwickDistinct {
    int n;
    vector<int> bit;
    FenwickDistinct(int n = 0) { init(n); }
    void init(int n_) { n = n_; bit.assign(n + 1, 0); }
    void add(int i, int v) { for (; i <= n; i += i & -i) bit[i] += v; }
    int sum(int i) const { int r = 0; for (; i > 0; i -= i & -i) r += bit[i]; return r; }
    int range_sum(int l, int r) const { return sum(r) - sum(l - 1); }
};

vector<int> distinct_count_queries(const vector<int>& a, vector<DistinctQuery> qs) {
    int n = (int)a.size() - 1;
    sort(qs.begin(), qs.end(), [](const DistinctQuery& x, const DistinctQuery& y) {
        return x.r < y.r;
    });
    FenwickDistinct fw(n);
    unordered_map<int, int> last;
    vector<int> ans(qs.size());
    int cur = 0;
    for (auto q : qs) {
        while (cur < q.r) {
            cur++;
            if (last.count(a[cur])) fw.add(last[a[cur]], -1);
            fw.add(cur, 1);
            last[a[cur]] = cur;
        }
        ans[q.id] = fw.range_sum(q.l, q.r);
    }
    return ans;
}
```

莫队：区间不同数数量

赛时先看

题目信号：区间询问很多，贡献能在左右端点移动时 $O(1)$ 更新；比如不同数数量、出现次数类。

本质：静态数组离线区间询问，维护一个可增删的窗口。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；`solve()`

里先构造对象，再按下方“不会用就看这里”调用公开接口。

复杂度判定： $O((n+q) \sqrt{n})$ 。

维护的量：`cnt`（当前窗口内各值的出现次数）；`distinct`（窗口内不同数个数）；`cur_l/cur_r`（窗口左右端点）；`block`（块长，取 $\sqrt{n}$ ）。

警告： a 需 1-indexed；离散化在函数内部完成；排序块大小一般取 $\sqrt{n}$ 。

【最小完整示例（先抄这一段就能跑）】

题目：静态数组 $a[1..n]$ ， q 次询问区间 $[l, r]$ 内不同数字的个数。

```
vector<MoQueryBasic> qs;
```



```
for (int i = 0; i < q; ++i) {
    int l, r;
    cin >> l >> r;
    qs.push_back({l, r, i}); // 1. 收集询问: {l, r, 原始编号 id}
}
vector<int> ans = mo_distinct_count(a, qs); // 2. 调用: a 必须 1-indexed
for (int i = 0; i < q; ++i) cout << ans[i] << '\n'; // 3. 按原始编号输出
```

样例: $a=[1,2,1,3,2]$; 问 $[1,3] \rightarrow 2$; $[2,5] \rightarrow 3$ 。

【传参要求（照这个传不会错）】

- a : 必须 1-indexed (长度 $n+1$, $a[0]$ 不参与); 值域不限, 内部自动离散化。
- qs : 询问数组, 每个元素 $\{l, r, id\}$, l/r 是 1-indexed 闭区间, id 是原始询问编号 (0-based)。
- 返回值: vector<int> , $\text{ans}[id]$ 是第 id 个询问的答案。
- 只支持离线 (全部询问先读完再算); 带修改翻“莫队带修改”。

```
struct MoQueryBasic {
    int l, r, id;
};

vector<int> mo_distinct_count(vector<int> a, vector<MoQueryBasic> qs) {
    int n = (int)a.size() - 1;
    vector<int> xs(a.begin() + 1, a.end());
    sort(xs.begin(), xs.end());
    xs.erase(unique(xs.begin(), xs.end()), xs.end());
    for (int i = 1; i <= n; ++i) {
        a[i] = int(lower_bound(xs.begin(), xs.end(), a[i]) - xs.begin());
    }

    int block = max(1, (int)sqrt(n));
    sort(qs.begin(), qs.end(), [&](const MoQueryBasic& x, const MoQueryBasic& y) {
        int bx = x.l / block, by = y.l / block;
        if (bx != by) return bx < by;
        return (bx & 1) ? x.r > y.r : x.r < y.r;
    });

    vector<int> cnt(xs.size(), 0), ans(qs.size());
    int cur_l = 1, cur_r = 0, distinct = 0;

    auto add = [&](int pos) {
        if (++cnt[a[pos]] == 1) distinct++;
    };
    auto remove = [&](int pos) {
        if (--cnt[a[pos]] == 0) distinct--;
    };

    for (auto q : qs) {
        while (cur_l > q.l) add(--cur_l);
        while (cur_r < q.r) add(++cur_r);
        while (cur_l < q.l) remove(cur_l++);
        while (cur_r > q.r) remove(cur_r--);
        ans[q.id] = distinct;
    }
    return ans;
}
```

莫队带修改

赛时先看

题目信号: 允许离线; 查询答案能在左右端点移动和时间前后移动时 $O(1)$ 或 $O(\log n)$ 更新。

本质: 离线处理区间查询, 同时存在单点修改。

接法: 把本节 struct/class/函数组 整段抄到 `solve()` 上面; 外部只调用下面 API 列出的接口, 不要把内部递归参数直接当题面参数。

复杂度判定: 常用块长 $n^{2/3}$, 复杂度约 $O(n^{2/3} q)$ 。

维护的量: a (当前数组, 含已应用的修改); `block_len` (块长 $n^{2/3}$); 窗口统计量由 `add_value/remove_value/current_answer` 维护。

警告: 修改影响当前位置区间时, 要先删旧值再加新值; 记录修改前的值。

【最小完整示例（先抄这一段就能跑）】

题目: n 个数。1 $l\ r$ 询问 $[l, r]$ 内不同数字个数; 2 $p\ x$ 把 $a[p]$ 改成

x 。所有修改和询问先给出 (离线), 按题实现 `add_value/remove_value/current_answer` 后调用。

```
// 先按题目实现: add_value(x) 加一个数、remove_value(x) 删一个数、current_answer() 返回当前答案
vector<MoQueryUpdate> qs; // 收集询问 {l, r, t, id}: t = 之前已发生的修改数
```

```
vector<Modification> mods; // 收集修改 {pos, old_value, new_value} (旧值需记录)
vector<i64> ans = mo_with_updates(a, qs, mods); // 调用: a 必须 1-indexed
for (int i = 0; i < q; ++i) cout << ans[i] << '\n';
```

样例: $a=[1,2,1,3,2]$, 先改 $a[2]=9$; 问 $[1,3] \rightarrow 2$; $[2,5] \rightarrow 4$ 。

【传参要求（照这个传不会错）】

- a : 原数组, 必须 1-indexed (长度 $n+1$, $a[0]$ 不参与)。
- qs : 询问数组, 元素 $MoQueryUpdate\{l, r, t, id\}$; l/r 为 1-indexed 闭区间; t = 本次询问前已应用的修改数 ($0..mods.size()$); id 为 0-based 原始编号。
- $mods$: 修改数组, 按时间顺序存 $\{pos, old_value, new_value\}$; old_value 必须自己记录 (改前先取 $a[pos]$)。
- 返回值: $vector<i64>$, $ans[id]$ 就是第 id 个询问的答案。
- 必须实现 `add_value(int x)` / `remove_value(int x)` / `current_answer()` 三个函数, 签名不能改。

【API / 入口函数（赛时只认这里列的名字）】

- `add_value(int x)` / `remove_value(int x)` / `current_answer()` $\rightarrow$ 三个函数按题目实现: 加数、删数、取当前答案。

```
struct Modification {
    int pos, old_value, new_value;
};

struct MoQueryUpdate {
    int l, r, t, id;
};

int block_len;

bool mo_cmp(const MoQueryUpdate& a, const MoQueryUpdate& b) {
    int al = a.l / block_len, bl = b.l / block_len;
    if (al != bl) return al < bl;
    int ar = a.r / block_len, br = b.r / block_len;
    if (ar != br) return ar < br;
    return a.t < b.t;
}

// add_value/remove_value/current_answer 按题目改。
void add_value(int x) {}
void remove_value(int x) {}
i64 current_answer() { return 0; }

vector<i64> mo_with_updates(vector<int> a, vector<MoQueryUpdate> qs, vector<Modification> mods) {
    int n = (int)a.size() - 1;
    block_len = max(1, (int)pow(n, 2.0 / 3.0));
    sort(qs.begin(), qs.end(), mo_cmp);
    vector<i64> ans(qs.size());
    int l = 1, r = 0, t = 0;
    auto apply = [&](int id) {
        auto& m = mods[id];
        if (l <= m.pos && m.pos <= r) {
            remove_value(a[m.pos]);
            add_value(m.new_value);
        }
        a[m.pos] = m.new_value;
    };
    auto undo = [&](int id) {
        auto& m = mods[id];
        if (l <= m.pos && m.pos <= r) {
            remove_value(a[m.pos]);
            add_value(m.old_value);
        }
        a[m.pos] = m.old_value;
    };
    for (auto q : qs) {
        while (t < q.t) apply(t++);
        while (t > q.t) undo(--t);
        while (l > q.l) add_value(a[--l]);
        while (r < q.r) add_value(a[++r]);
        while (l < q.l) remove_value(a[l++]);
        while (r > q.r) remove_value(a[r--]);
        ans[q.id] = current_answer();
    }
    return ans;
}
```

树上莫队

赛时先看

题目信号：树上多次路径查询，无修改或少修改；答案可通过点的出现奇偶维护。

本质：离线查询树上路径颜色种类、路径众数等可增删维护的信息。

接法：抄下 `struct TreeMo: solver.init(n, LOG)` 初始化，逐条 `solver.add_edge(u,v)`，`solver.build(root)` 后调用 `solver.solve(a, queries)` 返回答案数组（`a` 是 1-indexed 颜色数组）。

> - 复杂度判定： $O((n+q) \sqrt{n})$ 次 toggle；每次 toggle 的代价取决于维护的信息。> - 维护的量：欧拉序 `euler`（长度 $2n$ ）、`first/last`（进出时间戳）、倍增表 `up`、颜色计数 `cnt` 与当前不同数 `distinct`。> - 警告：颜色值需 ≥ 1 或先离散化；LOG 需 $\geq \text{ceil}(\log_2 n)+1$ ；`build` 必须在一棵连通树上执行；路径端点相同的询问答案为 1。

【最小完整示例（先抄这一段就能跑）】

题目：`n` 个点一棵树，每个点有颜色 `a[i]`；`q` 次询问路径 `(u,v)` 上不同颜色的数量。

```
TreeMo solver; // 1. 结构体定义：无参构造
solver.init(n, LOG); // 2. 初始化：LOG >= ceil(log2 n) + 1
for (int i = 1; i < n; ++i) {
    int u, v;
    cin >> u >> v;
    solver.add_edge(u, v); // 3. 加树边
}
solver.build(1); // 4. 建欧拉序：以 1 为根
vector<pair<int, int>> queries; // 5. 收集路径询问 {u, v}
for (int i = 0; i < q; ++i) {
    int u, v;
    cin >> u >> v;
    queries.push_back({u, v});
}
vector<int> ans = solver.solve(a, queries); // 6. 调用：a 是 1-indexed 颜色数组
for (int i = 0; i < q; ++i) cout << ans[i] << '\n'; // 7. 按原始顺序输出
```

样例：`n=3`，边 `1-2、2-3`，颜色 `a=[0,1,2,1]`；问路径 `(2,3) → 2`（颜色 2、1）；问 `(1,3) → 2`（颜色 1、2、1）。

【传参要求（照这个传不会错）】

- `TreeMo solver`：无参构造；然后 `init(n, LOG)`。
- `init(n, LOG)`：`n` = 点数（编号 `1..n`）；LOG 需 $\geq \text{ceil}(\log_2 n) + 1$ 。
- `add_edge(u, v)`：加一条树边；`build(root)`：以 `root` 为根建欧拉序（默认 `root=1`）。
- `solve(a, queries)`：`a` 必须 1-indexed 颜色数组（`a[0]` 不参与，颜色值需 ≥ 1 ）；`queries` 每个元素是 `{u, v}` 路径两端点；返回 `vector<int>`，`ans[i]` 是第 `i` 条路径上不同颜色数，`u == v` 时答案为 1。

【改板时先认这几个量】

- `depth`：深度。
- `up`：倍增祖先表。

```
// 树上莫队：批量回答树上路径的“不同颜色数量”（SP10707 COT2 模型）。
// 用法：TreeMo solver; solver.init(n, LOG); solver.add_edge(u,v); ...; solver.build(root);
// 然后 solver.solve(a, queries) 返回答案数组，a 是 1-indexed 颜色数组。
struct TreeMo {
    int n = 0;
    vector<vector<int>> g;
    vector<int> depth, first, last, euler;
    vector<vector<int>> up;
    int block = 0;

    TreeMo(int n_ = 0, int LOG = 18) { init(n_, LOG); }

    void init(int n_, int LOG = 18) {
        n = n_;
        g.assign(n + 1, {});
        depth.assign(n + 1, 0);
        first.assign(n + 1, 0);
        last.assign(n + 1, 0);
        up.assign(LOG + 1, vector<int>(n + 1, 0));
        euler.clear();
    }

    void add_edge(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void dfs(int u, int p) {
```

```

up[0][u] = p;
for (int j = 1; j < (int)up.size(); ++j) up[j][u] = up[j - 1][up[j - 1][u]];
first[u] = (int)euler.size();
euler.push_back(u);
for (int v : g[u]) {
    if (v == p) continue;
    depth[v] = depth[u] + 1;
    dfs(v, u);
}
last[u] = (int)euler.size();
euler.push_back(u);
}

void build(int root = 1) {
    euler.clear();
    dfs(root, 0);
    block = max(1, (int)(sqrt((long double)euler.size()) + 0.5));
}

int lca(int u, int v) const {
    if (depth[u] < depth[v]) swap(u, v);
    int d = depth[u] - depth[v];
    for (int j = 0; d; ++j, d >>= 1) if (d & 1) u = up[j][u];
    if (u == v) return u;
    for (int j = (int)up.size() - 1; j >= 0; --j) {
        if (up[j][u] != up[j][v]) u = up[j][u], v = up[j][v];
    }
    return up[0][u];
}

// a 是 1-indexed 颜色; 返回第 i 个询问 (u_i, v_i) 路径上不同颜色数量。
vector<int> solve(const vector<int>& a, const vector<pair<int, int>>& queries) const {
    struct MoQuery {
        int l, r, extra, id;
    };
    vector<MoQuery> qs;
    qs.reserve(queries.size());
    for (int id = 0; id < (int)queries.size(); ++id) {
        int u = queries[id].first, v = queries[id].second;
        if (first[u] > first[v]) swap(u, v);
        int w = lca(u, v);
        if (w == u) qs.push_back({first[u], first[v], 0, id});
        else qs.push_back({last[u], first[v], w, id});
    }
    sort(qs.begin(), qs.end(), [&](const MoQuery& x, const MoQuery& y) {
        int bx = x.l / block, by = y.l / block;
        if (bx != by) return bx < by;
        return (bx & 1) ? x.r > y.r : x.r < y.r;
    });

    int max_color = *max_element(a.begin(), a.end());
    vector<int> appear(n + 1, 0), cnt(max_color + 1, 0), ans(queries.size());
    int cur_l = 0, cur_r = -1, distinct = 0;
    auto toggle = [&](int pos) {
        int u = euler[pos], c = a[u];
        if (appear[u]) {
            if (--cnt[c] == 0) --distinct;
        } else {
            if (cnt[c]++ == 0) ++distinct;
        }
        appear[u] ^= 1;
    };
    auto move = [&](int l, int r) {
        while (cur_l > l) toggle(--cur_l);
        while (cur_r < r) toggle(++cur_r);
        while (cur_l < l) toggle(cur_l++);
        while (cur_r > r) toggle(cur_r--);
    };
    for (const MoQuery& q : qs) {
        move(q.l, q.r);
        ans[q.id] = distinct + (q.extra && !appear[q.extra] && cnt[a[q.extra]] == 0 ? 1 : 0);
    }
    return ans;
}
};

```

线段树：单点修改，区间最大值

赛时先看

题目信号：动态修改数组某个位置，并反复问区间最大/最小/和。

本质：最基础线段树，先会这个再学懒标记。

接法：输入数组用 `vector<i64> a(n+1)`，然后 `seg.build(a)`；单点改成新值用 `seg.modify(pos,val)`；区间最大值用 `seg.query(l,r)`。如果要区间和，把 `max` 换成加法并把初始答案改成 `0`；如果要同时维护多个量，另写 `Node` 和 `merge`。

复杂度判定： $O(\log n)$ 。

维护的量： n （长度）；`tr[p]`（线段树第 p 号节点区间的最大值，初始为 `LLONG_MIN/4`）。

警告：线段树下标常用 `1..n`；递归边界要统一。

约定：`a` 必须 1-indexed，且 `a.size() >= n + 1`；`modify(pos,val)` 把第 `pos` 个位置改成 `val`；`query(l,r)` 查询闭区间 `[l,r]` 的最大值。

【最小完整示例（先抄这一段就能跑）】

题目： n 个数， q 次操作。`op=1 pos val`：把 `a[pos]` 改成 `val`；`op=2 l r`：输出闭区间 `[l,r]` 的最大值。

```
SegPointMax seg(n);
seg.build(a); // 建树：a 必须 1-indexed (长度 n+1)
if (op == 1) { int pos; i64 val; cin >> pos >> val; seg.modify(pos, val); } // 单点改值
else { int l, r; cin >> l >> r; cout << seg.query(l, r) << '\n'; } // 区间最大值
```

样例：`a=[1,2,3,4,5]`；`modify(3,-7)` 后 `query(1,5)` -> 输出 4。

【传参要求（照这个传不会错）】

- `SegPointMax seg(n)`： n 为元素个数，下标 `1..n`。
- `build(a)`：`a` 必须 1-indexed，且 `a.size() >= n + 1`；`a[0]` 不参与。
- `modify(pos, val)`：把第 `pos` 个位置改成 `val`；要求 `1 <= pos <= n`；无返回值。
- `query(l, r)`：闭区间 `[l,r]` 的最大值；要求 `1 <= l <= r <= n`；返回 `i64`。

【API / 入口函数（赛时只认这里列的名字）】

- `build(const vector<i64>& a)` -> 完成建树或预处理
- `init(int n_)` -> 初始化/清空结构
- `modify(int pos, i64 val)` -> 单点/指定位置修改
- `query(int l, int r)` -> 查询 返回 `i64`。

```
struct SegPointMax {
    int n;
    vector<i64> tr;

    SegPointMax(int n = 0) { init(n); }

    void init(int n_) {
        n = n_;
        tr.assign(4 * n + 4, LLONG_MIN / 4);
    }

    void build(int p, int l, int r, const vector<i64>& a) {
        if (l == r) {
            tr[p] = a[l];
            return;
        }
        int mid = (l + r) >> 1;
        build(p << 1, l, mid, a);
        build(p << 1 | 1, mid + 1, r, a);
        tr[p] = max(tr[p << 1], tr[p << 1 | 1]);
    }

    void modify(int p, int l, int r, int pos, i64 val) {
        if (l == r) {
            tr[p] = val;
            return;
        }
        int mid = (l + r) >> 1;
        if (pos <= mid) modify(p << 1, l, mid, pos, val);
        else modify(p << 1 | 1, mid + 1, r, pos, val);
        tr[p] = max(tr[p << 1], tr[p << 1 | 1]);
    }

    i64 query(int p, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) return tr[p];
        int mid = (l + r) >> 1;
        i64 ans = LLONG_MIN / 4;
        if (ql <= mid) ans = max(ans, query(p << 1, l, mid, ql, qr));
        if (qr > mid) ans = max(ans, query(p << 1 | 1, mid + 1, r, ql, qr));
        return ans;
    }
};
```

```

void build(const vector<i64>& a) {
    // 外部接口: a 必须是 1-indexed, 且 a.size() >= n + 1。
    assert((int)a.size() > n);
    build(1, 1, n, a);
}

void modify(int pos, i64 val) {
    // 外部接口: 把原数组第 pos 个位置改成 val, pos 为 1-indexed。
    assert(1 <= pos && pos <= n);
    modify(1, 1, n, pos, val);
}

i64 query(int l, int r) {
    // 外部接口: 查询原数组闭区间 [l, r] 的最大值。
    assert(1 <= l && l <= r && r <= n);
    return query(1, 1, n, l, r);
}
};

```

线段树典题：单点修改，区间最大子段和

赛时先看

题目信号：动态单点修改；询问区间内“连续选一段”的最大收益/最大和；元素可为负数，空段通常不允许。

本质：数组某个位置会被赋新值，询问任意区间内非空连续子段的最大和。经典模型为 SPOJ GSS 系列。

接法：把“动态修改某个位置后，问区间内连续一段最大和”直接翻译成这个模板。建树写 `seg.build(a)`，修改写 `seg.point_assign(pos, value)`；询问输出 `seg.max_subarray(l, r)`，或者需要完整信息时看 `seg.query(l, r).best`。如果题目允许空段，最后答案再和 0 取最大。

复杂度判定：建树 $O(n)$ ，单点修改和区间查询 $O(\log n)$ 。

维护的量： n （长度）； $tr[p]$ （节点四元组

$sum/pref/suff/best$ ：区间和、最大前缀和、最大后缀和、最大非空子段和）。

警告： $best$ 是非空子段，叶子四个值都等于元素；查询时不能用全 0 的空节点合并，否则全负区间会被误判为 0；代码只在确实相交的子树之间合并。

约定： a 必须 1-indexed，叶子直接用 $a[1..n]$ 初始化；`query(l, r)` 返回闭区间 $[l, r]$ 的四元组信息。

【最小完整示例（先抄这一段就能跑）】

题目： n 个数， q 次操作。 $op=1$ pos val ：把 $a[pos]$ 赋成 val ； $op=2$ l r ：输出闭区间 $[l, r]$ 内非空连续子段的最大和。

```

SegMaxSubarray seg(n);
seg.build(a); // 建树: a 必须 1-indexed (长度 n+1)
if (op == 1) { int pos; i64 val; cin >> pos >> val; seg.point_assign(pos, val); } // 单点赋值
else { int l, r; cin >> l >> r; cout << seg.max_subarray(l, r) << '\n'; } // 最大子段和

```

样例： $a=[-2,1,-3,4,-1,2,1,-5,4]$ ；`max_subarray(1,9)` $\rightarrow$ 输出 6。

【传参要求（照这个传不会错）】

- `SegMaxSubarray seg(n)`： n 为元素个数，下标 $1..n$ 。
- `build(a)`： a 必须 1-indexed，长度至少 $n+1$ 。
- `point_assign(pos, value)`：把第 pos 个位置赋值为 $value$ ；要求 $1 \leq pos \leq n$ ；无返回值。
- `max_subarray(l, r)`：返回 $[l, r]$ 非空连续子段最大和（ $i64$ ）；要求 $1 \leq l \leq r \leq n$ 。
- `query(l, r)`：需要完整信息时用，返回四元组 `Node`，取 `.best` 即最大子段和；题目允许空段时答案再与 0 取 `max`。

【API / 入口函数（赛时只认这里列的名字）】

- `build(const vector<i64>& a)` $\rightarrow$ 完成建树或预处理
- `init(int n_)` $\rightarrow$ 初始化/清空结构
- `merge(Node left, Node right)` $\rightarrow$ 合并 返回 `Node`。
- `query(int l, int r)` $\rightarrow$ 查询 返回 `Node`。

【改板时先认这几个量】

- tr ：树节点池（节点数组）。
- sum ：区间和/计数和。

维护：每段保存总和 sum 、最大前缀和 $pref$ 、最大后缀和 $suff$ 、最大子段和 $best$ 。合并两段时，跨中点的候选是 $left.suff + right.pref$ 。

```

struct SegMaxSubarray {
    struct Node {
        i64 sum, pref, suff, best;
    };

    int n = 0;
    vector<Node> tr;

    SegMaxSubarray(int n_ = 0) { init(n_); }

    void init(int n_) {
        n = n_;
        tr.assign(4 * n + 4, {0, 0, 0, 0});
    }

    static Node merge(Node left, Node right) {
        return {
            left.sum + right.sum,
            max(left.pref, left.sum + right.pref),
            max(right.suff, right.sum + left.suff),
            max({left.best, right.best, left.suff + right.pref}),
        };
    }

    void build(int p, int l, int r, const vector<i64>& a) {
        if (l == r) {
            tr[p] = {a[l], a[l], a[l], a[l]};
            return;
        }
        int mid = (l + r) >> 1;
        build(p << 1, l, mid, a);
        build(p << 1 | 1, mid + 1, r, a);
        tr[p] = merge(tr[p << 1], tr[p << 1 | 1]);
    }

    void point_assign(int p, int l, int r, int pos, i64 value) {
        if (l == r) {
            tr[p] = {value, value, value, value};
            return;
        }
        int mid = (l + r) >> 1;
        if (pos <= mid) point_assign(p << 1, l, mid, pos, value);
        else point_assign(p << 1 | 1, mid + 1, r, pos, value);
        tr[p] = merge(tr[p << 1], tr[p << 1 | 1]);
    }

    Node query(int p, int l, int r, int ql, int qr) const {
        if (ql <= l && r <= qr) return tr[p];
        int mid = (l + r) >> 1;
        if (qr <= mid) return query(p << 1, l, mid, ql, qr);
        if (ql > mid) return query(p << 1 | 1, mid + 1, r, ql, qr);
        return merge(
            query(p << 1, l, mid, ql, qr),
            query(p << 1 | 1, mid + 1, r, ql, qr)
        );
    }

    void build(const vector<i64>& a) {
        // 外部接口: a 必须是 1-indexed; 叶子直接用 a[1..n] 初始化。
        assert((int)a.size() > n);
        build(1, 1, n, a);
    }

    void point_assign(int pos, i64 value) {
        // 外部接口: 把原数组第 pos 个位置赋值为 value。
        assert(1 <= pos && pos <= n);
        point_assign(1, 1, n, pos, value);
    }

    Node query(int l, int r) const {
        // 外部接口: 返回原数组闭区间 [l, r] 的四元组信息。
        assert(1 <= l && l <= r && r <= n);
        return query(1, 1, n, l, r);
    }

    i64 max_subarray(int l, int r) const {
        // 外部接口: 直接返回 [l, r] 内非空连续子段最大和。
        return query(l, r).best;
    }
};

```

典题模型: 输入数组用 1-indexed `vector<i64> a(n+1)`, `seg.build(a)`; 修改 `seg.point_assign(pos,value)`; 询问 `[l,r]` 输出 `seg.max_subarray(l,r)`。

线段树：区间加，区间和/最大值

赛时先看

题目信号： add 1 r x 后继续查询区间和、区间最大值。

本质： 区间修改和区间查询。

接法： seg.range_add(1,r,x) 表示给一整段加值；seg.query_sum(1,r) 问区间和，seg.query_max(1,r) 问区间最大值。内部每次递归往下走前必须 push，每次子树改完必须 pull；如果题目是区间赋值，不要把加法 lazy 改成赋值 lazy，直接翻下一段区间赋值模板。

复杂度判定： $O(\log n)$ 。

维护的量： n（长度）；tr[p]（节点存 sum 区间和、mx 区间最大值、lazy 懒标记加值）。

警告： 懒标记下传时，区间和要乘长度。

【最小完整示例（先抄这一段就能跑）】

题目：n 个数，q 次操作。op=1 l r x：把 a[l..r] 全部加 x；op=2 l r：输出区间和；op=3 l r：输出区间最大值。

```
SegLazy seg;
seg.build(a);                                // 1. 结构体定义 + 建树：a 必须 1-indexed（长度 n+1）
while (q--) {
    int op, l, r;
    cin >> op >> l >> r;
    if (op == 1) { i64 x; cin >> x; seg.range_add(1, r, x); } // 区间加
    else if (op == 2) cout << seg.query_sum(1, r) << '\n';   // 区间和
    else cout << seg.query_max(1, r) << '\n';               // 区间最大值
}
```

样例：a=[1,2,3,4,5]；add(2,4,+10) 后 query_sum(1,5) -> 输出 45；query_max(1,5) -> 输出 14。

【传参要求（照这个传不会错）】

- SegLazy seg; seg.build(a)：构造后先建树；a 必须 1-indexed（a[0] 不参与，长度 n+1）。
- range_add(1, r, val)：闭区间 [1,r] 全部加 val；要求 $1 \leq l \leq r \leq n$ 。
- query_sum(1, r) / query_max(1, r)：闭区间 [1,r] 的和 / 最大值；要求 $1 \leq l \leq r \leq n$ 。
- 返回 i64；建完树才能查询（查询初值也走 build 后的线段树）。

约定： a 必须 1-indexed，建完后才能查询初值；range_add(1,r,val) 给闭区间 [1,r] 全部加 val；query_sum(1,r) 查询闭区间和。

【API / 入口函数（赛时只认这里列的名字）】

- build(const vector<i64>& a) -> 完成建树或预处理
- init(int n_) -> 初始化/清空结构
- query_max(int l, int r) -> 查询闭区间最大值 返回 i64。
- query_sum(int l, int r) -> 查询闭区间和 返回 i64。
- range_add(int l, int r, i64 val) -> 闭区间加值

【核心逻辑（改代码时别破坏）】

- 节点同时存 sum/mx/lazy；整段命中时只改当前节点和 lazy。
- 下探前 push，子树变化后 pull；区间和加 val 时必须乘区间长度。

【改板时先认这几个量】

- tr：树节点池（节点数组）。
- lazy：懒标记。
- sum：区间和/计数和。
- mx：区间最大值。

```
struct SegLazy {
    struct Node {
        i64 sum = 0;
        i64 mx = LLONG_MIN / 4;
        i64 lazy = 0;
    };

    int n;
    vector<Node> tr;

    SegLazy(int n = 0) { init(n); }
```

```

void init(int n_) {
    n = n_;
    tr.assign(4 * n + 4, {});
}

void pull(int p) {
    tr[p].sum = tr[p << 1].sum + tr[p << 1 | 1].sum;
    tr[p].mx = max(tr[p << 1].mx, tr[p << 1 | 1].mx);
}

void apply(int p, int l, int r, i64 val) {
    tr[p].sum += val * (r - l + 1);
    tr[p].mx += val;
    tr[p].lazy += val;
}

void push(int p, int l, int r) {
    if (tr[p].lazy == 0 || l == r) return;
    int mid = (l + r) >> 1;
    apply(p << 1, l, mid, tr[p].lazy);
    apply(p << 1 | 1, mid + 1, r, tr[p].lazy);
    tr[p].lazy = 0;
}

void build(int p, int l, int r, const vector<i64>& a) {
    if (l == r) {
        tr[p].sum = tr[p].mx = a[l];
        return;
    }
    int mid = (l + r) >> 1;
    build(p << 1, l, mid, a);
    build(p << 1 | 1, mid + 1, r, a);
    pull(p);
}

void range_add(int p, int l, int r, int ql, int qr, i64 val) {
    if (ql <= l && r <= qr) {
        apply(p, l, r, val);
        return;
    }
    push(p, l, r);
    int mid = (l + r) >> 1;
    if (ql <= mid) range_add(p << 1, l, mid, ql, qr, val);
    if (qr > mid) range_add(p << 1 | 1, mid + 1, r, ql, qr, val);
    pull(p);
}

i64 query_sum(int p, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) return tr[p].sum;
    push(p, l, r);
    int mid = (l + r) >> 1;
    i64 ans = 0;
    if (ql <= mid) ans += query_sum(p << 1, l, mid, ql, qr);
    if (qr > mid) ans += query_sum(p << 1 | 1, mid + 1, r, ql, qr);
    return ans;
}

i64 query_max(int p, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) return tr[p].mx;
    push(p, l, r);
    int mid = (l + r) >> 1;
    i64 ans = LLONG_MIN / 4;
    if (ql <= mid) ans = max(ans, query_max(p << 1, l, mid, ql, qr));
    if (qr > mid) ans = max(ans, query_max(p << 1 | 1, mid + 1, r, ql, qr));
    return ans;
}

void build(const vector<i64>& a) {
    // 外部接口: a 必须是 1-indexed, 建完后才能查询初值。
    assert((int)a.size() > n);
    build(1, 1, n, a);
}

void range_add(int l, int r, i64 val) {
    // 外部接口: 给原数组闭区间 [l,r] 全部加 val。
    assert(1 <= l && l <= r && r <= n);
    range_add(1, 1, n, l, r, val);
}

i64 query_sum(int l, int r) {
    // 外部接口: 查询原数组闭区间 [l,r] 的区间和。
    assert(1 <= l && l <= r && r <= n);
    return query_sum(1, 1, n, l, r);
}

```

```

i64 query_max(int l, int r) {
    // 外部接口：查询原数组闭区间 [l,r] 的最大值。
    assert(1 <= l && l <= r && r <= n);
    return query_max(1, 1, n, l, r);
}
};

```

线段树：区间赋值，区间和

赛时先看

题目信号：操作语义是 $a[i]=value$ ($1 \leq i \leq r$)，不是加法；之后问区间总和、平均值或覆盖长度。

本质：反复把 $[1,r]$ 全部改成同一个数，并查询区间和；是“区间染色/覆盖”转为数值统计后的通用骨架。

接法：数列初值 $a[1..n]$ ，操作 $C \ l \ r \ x$ 表示覆盖， $Q \ l \ r$ 问和。建树后分别调用

`seg.range_assign(l,r,x)`、`seg.query_sum(l,r)`；若题面一开始全为同一个值，可不建树，先 `init(n)` 后调用 `seg.range_assign(1,n,initial)`。

复杂度判定：建树 $O(n)$ ，区间赋值和区间和 $O(\log n)$ 。

维护的量： n （长度）； $tr[p]$ （`sum` 区间和、`assign_value` 覆盖值、`has_assign` 是否有覆盖懒标记）。

警告：赋值懒标记不能用 0 表示“无标记”，因为合法赋值可能正好为 0 ；要额外存

`has_assign`。若同时有区间加和赋值，必须规定二者组合顺序，建议直接翻仿射懒标记线段树。

约定： a 必须 1-indexed，初值来自 $a[1..n]$ ；`range_assign(l,r,value)` 把闭区间 $[1,r]$ 全部赋值；`query_sum(l,r)` 查询闭区间和。

【最小完整示例（先抄这一段就能跑）】

题目： n 个数， q 次操作。`op=1 l r x`：把 $a[1..r]$ 全部赋成 x ；`op=2 l r`：输出闭区间和。

```

SegRangeAssignSum seg(n);
seg.build(a); // 建树：a 必须 1-indexed (长度 n+1)
if (op == 1) { i64 x; cin >> x; seg.range_assign(l, r, x); } // 区间赋值
else cout << seg.query_sum(l, r) << '\n'; // 区间和

```

样例： $a=[1,2,3,4,5]$ ；`range_assign(2,4,10)` 后 `query_sum(1,5)` $\rightarrow$ 输出 36。

【传参要求（照这个传不会错）】

- `SegRangeAssignSum seg(n)`： n 为元素个数，下标 $1..n$ 。
- `build(a)`： a 必须 1-indexed，长度至少 $n+1$ ；若整段初值相同，可只 `init(n)` 后 `range_assign(1,n,initial)`，不建树。
- `range_assign(l, r, value)`：闭区间 $[l,r]$ 全部赋成 $value$ ；要求 $1 \leq l \leq r \leq n$ ；无返回值。
- `query_sum(l, r)`：闭区间 $[l,r]$ 的区间和；要求 $1 \leq l \leq r \leq n$ ；返回 `i64`。

【API / 入口函数（赛时只认这里列的名字）】

- `build(const vector<i64>& a)` $\rightarrow$ 完成建树或预处理
- `init(int n_)` $\rightarrow$ 初始化/清空结构
- `query_sum(int l, int r)` $\rightarrow$ 查询闭区间和 返回 `i64`。

【改板时先认这几个量】

- `tr`：树节点池（节点数组）。
- `sum`：区间和/计数和。

```

struct SegRangeAssignSum {
    struct Node {
        i64 sum = 0;
        i64 assign_value = 0;
        bool has_assign = false;
    };

    int n = 0;
    vector<Node> tr;

    SegRangeAssignSum(int n_ = 0) { init(n_); }

    void init(int n_) {
        n = n_;
        tr.assign(4 * n + 4, {});
    }

    void apply_assign(int p, int l, int r, i64 value) {
        tr[p].sum = value * (r - l + 1);
    }
}

```

```

    tr[p].assign_value = value;
    tr[p].has_assign = true;
}

void push(int p, int l, int r) {
    if (!tr[p].has_assign || l == r) return;
    int mid = (l + r) >> 1;
    apply_assign(p << 1, l, mid, tr[p].assign_value);
    apply_assign(p << 1 | 1, mid + 1, r, tr[p].assign_value);
    tr[p].has_assign = false;
}

void build(int p, int l, int r, const vector<i64>& a) {
    if (l == r) {
        tr[p].sum = a[l];
        return;
    }
    int mid = (l + r) >> 1;
    build(p << 1, l, mid, a);
    build(p << 1 | 1, mid + 1, r, a);
    tr[p].sum = tr[p << 1].sum + tr[p << 1 | 1].sum;
}

void range_assign(int p, int l, int r, int ql, int qr, i64 value) {
    if (ql <= l && r <= qr) {
        apply_assign(p, l, r, value);
        return;
    }
    push(p, l, r);
    int mid = (l + r) >> 1;
    if (ql <= mid) range_assign(p << 1, l, mid, ql, qr, value);
    if (qr > mid) range_assign(p << 1 | 1, mid + 1, r, ql, qr, value);
    tr[p].sum = tr[p << 1].sum + tr[p << 1 | 1].sum;
}

i64 query_sum(int p, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) return tr[p].sum;
    push(p, l, r);
    int mid = (l + r) >> 1;
    i64 answer = 0;
    if (ql <= mid) answer += query_sum(p << 1, l, mid, ql, qr);
    if (qr > mid) answer += query_sum(p << 1 | 1, mid + 1, r, ql, qr);
    return answer;
}

void build(const vector<i64>& a) {
    // 外部接口: a 必须是 1-indexed, 初值来自 a[1..n]。
    assert((int)a.size() > n);
    build(1, 1, n, a);
}

void range_assign(int l, int r, i64 value) {
    // 外部接口: 把原数组闭区间 [l,r] 全部赋值为 value。
    assert(1 <= l && l <= r && r <= n);
    range_assign(1, 1, n, l, r, value);
}

i64 query_sum(int l, int r) {
    // 外部接口: 查询原数组闭区间 [l,r] 的区间和。
    assert(1 <= l && l <= r && r <= n);
    return query_sum(1, 1, n, l, r);
}
};

```

仿射懒标记线段树：区间乘、区间加、区间和

赛时先看

题目信号：同一段元素统一乘一个数、加一个数，再问区间和；本质是区间上的仿射函数复合。

本质：维护 $a[i] = a[i] * mul + add \pmod{MOD}$ ，并查询区间和。AtCoder Library Practice Contest 的 K 题就是这一类标准模型。

接法：构造时直接传数组和模数 `AffineSegTree seg(a, MOD)`（自动建树）；区间修改 `seg.range_apply(l,r,mul,add)`；区间和 `seg.range_sum(l,r)`。

复杂度判定：建树 $O(n)$ ，每次修改/查询 $O(\log n)$ 。

维护的量： n （长度）、 mod （模数）； $tr[p]$ （ sum 区间和、 mul/add 仿射懒标记：区间内每个数先乘 mul 再加 add ，均在模意义下）。

警告：新操作在旧操作之后执行，所以懒标记是 $old \rightarrow new: mul = old_mul * new_mul, add = old_add * new_mul + new_add$ 。

约定： $n = (int)a.size() - 1$ ；// a 为 1-indexed

【最小完整示例（先抄这一段就能跑）】

题目：n 个数（模 MOD），q 次操作。op=1 l r mul add: a[l..r] 每个数先乘 mul 再加 add; op=2 l r: 输出闭区间和 mod MOD。

```
AffineSegTree seg(a, MOD);           // 构造即建树: a 必须 1-indexed (长度 n+1)
if (op == 1) { i64 mul, add; cin >> mul >> add; seg.range_apply(l, r, mul, add); } // 区间仿射变换
else cout << seg.range_sum(l, r) << '\n'; // 区间和
```

样例: a=[1,2,3,4,5], MOD=998244353; range_apply(2,4,3,1) 后 range_sum(1,5) -> 输出 36。

【传参要求（照这个传不会错）】

- AffineSegTree seg(a, MOD): 构造时自动建树; a 必须 1-indexed, n = a.size() - 1; MOD 为模数（须为 i64）。
- range_apply(l, r, mul, add): 闭区间 [l,r] 每个数先乘 mul 再加 add（自动取模）; 要求 $1 \leq l \leq r \leq n$; 无返回值。
- range_sum(l, r): 闭区间 [l,r] 的和 mod MOD; 要求 $1 \leq l \leq r \leq n$; 返回 i64。
- 只区间加: 传 range_apply(l, r, 1, x); 只区间乘: 传 range_apply(l, r, x, 0)。

【API / 入口函数（赛时只认这里列的名字）】

- build(const vector<i64>& a) -> 完成建树或预处理
- range_apply(int l, int r, i64 mul, i64 add) -> 区间先乘 mul 再加 add
- range_sum(int l, int r) -> 查询闭区间和 返回 i64。

【改板时先认这几个量】

- tr: 树节点池（节点数组）。
- sum: 区间和/计数和。

典题模型: AtCoder Library Practice Contest K - Range Affine Range Sum。

```
struct AffineSegTree {
    struct Node {
        i64 sum = 0;
        i64 mul = 1, add = 0;
    };
    int n;
    i64 mod;
    vector<Node> tr;

    AffineSegTree(const vector<i64>& a, i64 mod_) : mod(mod_) {
        n = (int)a.size() - 1; // a 为 1-indexed
        tr.assign(4 * n + 4, Node{});
        build_impl(1, 1, n, a);
    }

    void build_impl(int p, int l, int r, const vector<i64>& a) {
        if (l == r) {
            tr[p].sum = (a[l] % mod + mod) % mod;
            return;
        }
        int mid = (l + r) >> 1;
        build_impl(p << 1, l, mid, a);
        build_impl(p << 1 | 1, mid + 1, r, a);
        pull(p);
    }

    void pull(int p) {
        tr[p].sum = (tr[p << 1].sum + tr[p << 1 | 1].sum) % mod;
    }

    void apply(int p, int len, i64 mul, i64 add) {
        tr[p].sum = (tr[p].sum * mul + add * len) % mod;
        tr[p].mul = tr[p].mul * mul % mod;
        tr[p].add = (tr[p].add * mul + add) % mod;
    }

    void push(int p, int l, int r) {
        if (l == r || (tr[p].mul == 1 && tr[p].add == 0)) return;
        int mid = (l + r) >> 1;
        apply(p << 1, mid - l + 1, tr[p].mul, tr[p].add);
        apply(p << 1 | 1, r - mid, tr[p].mul, tr[p].add);
        tr[p].mul = 1;
        tr[p].add = 0;
    }
}
```

```

void range_apply_impl(int p, int l, int r, int ql, int qr, i64 mul, i64 add) {
    if (qr < l || r < ql) return;
    if (ql <= l && r <= qr) {
        apply(p, r - l + 1, (mul % mod + mod) % mod, (add % mod + mod) % mod);
        return;
    }
    push(p, l, r);
    int mid = (l + r) >> 1;
    range_apply_impl(p << 1, l, mid, ql, qr, mul, add);
    range_apply_impl(p << 1 | 1, mid + 1, r, ql, qr, mul, add);
    pull(p);
}

i64 range_sum_impl(int p, int l, int r, int ql, int qr) {
    if (qr < l || r < ql) return 0;
    if (ql <= l && r <= qr) return tr[p].sum;
    push(p, l, r);
    int mid = (l + r) >> 1;
    return (range_sum_impl(p << 1, l, mid, ql, qr)
        + range_sum_impl(p << 1 | 1, mid + 1, r, ql, qr)) % mod;
}

void build(const vector<i64>& a) { build_impl(1, 1, n, a); }
void range_apply(int l, int r, i64 mul, i64 add) { range_apply_impl(1, 1, n, l, r, mul, add); }
i64 range_sum(int l, int r) { return range_sum_impl(1, 1, n, l, r); }
};

// 示例: seg.range_apply(l, r, multiplier, increment);
// 示例: cout << seg.range_sum(l, r) << '\n';

```

区间染色 + 查询区间颜色种类数

赛时先看

题目信号: 颜色数量较少, 通常 `color <= 30/60`; 区间赋值; 查询颜色种类数。经典模型类似 POJ 2777 Count Color。

本质: 维护一条长度为 n 的线段, 操作包括把 $[l, r]$ 全部染成颜色 c , 查询 $[l, r]$ 内出现了多少种颜色。

接法: 把本节 struct/class/函数组 整段抄到 `solve()` 上面; 外部只调用下面 API

列出的接口, 不要把内部递归参数直接当题面参数。

复杂度判定: 每次操作 $O(\log n)$, 统计颜色数 $O(\text{颜色位数})$ 。

维护的量: n (长度); `mask[p]` (p 号节点区间内出现颜色的按位或结果)、`lazy[p]` (整段染色的懒标记位掩码)。

警告: 这是“区间赋值”不是区间加; 颜色从 1 开始时用 `1ULL << (c-1)`; 颜色数超过 60 时不能用 `ui64`, 要换 `bitset` 或其它做法。

【最小完整示例（先抄这一段就能跑）】

题目: n 段初始全为颜色 1, q 次操作。op=1 l r c: 把 $[l, r]$ 全部染成颜色 c ; op=2 l r: 输出 $[l, r]$ 内的颜色种类数。

```

ColorSegTree seg(n); // 初始整段颜色为 1
if (op == 1) { int c; cin >> c; seg.range_assign(1, r, c); } // 区间染色
else cout << seg.query_count(1, r) << '\n'; // 颜色种类数

```

样例: $n=5$ 初始全 1; `range_assign(2,4,3)` 后 `query_count(1,5)` -> 输出 2。

【传参要求（照这个传不会错）】

- `ColorSegTree seg(n)`: n 为长度, 下标 $1..n$; 初始整段颜色为 1。
- `range_assign(1, r, color)`: 闭区间 $[l, r]$ 全部染成颜色 $color$ (颜色编号从 1 开始); 要求 $1 \leq l \leq r \leq n$; 无返回值。
- `query_count(1, r)`: 闭区间 $[l, r]$ 内的颜色种类数; 要求 $1 \leq l \leq r \leq n$; 返回 `int`。
- 颜色总数必须 ≤ 60 (`ui64` 位掩码能装下)。

【不会用就照抄】

```

ColorSegTree seg(n);
seg.range_assign(1, r, c); // 把闭区间 [l,r] 全部染成颜色 c
int kinds = seg.query_count(1, r); // 查询闭区间 [l,r] 内的颜色种类数

```

- `range_assign` / `query_count` 的 l/r 都是 1-based 闭区间; 颜色编号从 1 开始。

【API / 入口函数（赛时只认这里列的名字）】

- `init(int n_)` -> 初始化/清空结构
- `query_count(int l, int r)` -> 查询闭区间 $[l, r]$ 内的颜色种类数 返回 `int`。

- `range_assign(int l, int r, int color)` -> 把闭区间 $[l, r]$ 全部染成颜色 `color`

```
struct ColorSegTree {
    int n;
    vector<ui64> mask, lazy;

    ColorSegTree(int n = 0) { init(n); }

    void init(int n_) {
        n = n_;
        mask.assign(4 * n + 4, 1ULL); // 初始颜色为 1。
        lazy.assign(4 * n + 4, 1ULL);
    }

    ui64 color_bit(int c) {
        return 1ULL << (c - 1);
    }

    void apply(int p, ui64 v) {
        mask[p] = v;
        lazy[p] = v;
    }

    void push(int p) {
        if (lazy[p]) {
            apply(p << 1, lazy[p]);
            apply(p << 1 | 1, lazy[p]);
            lazy[p] = 0;
        }
    }

    void range_assign_impl(int p, int l, int r, int ql, int qr, int color) {
        if (ql <= l && r <= qr) {
            apply(p, color_bit(color));
            return;
        }
        push(p);
        int mid = (l + r) >> 1;
        if (ql <= mid) range_assign_impl(p << 1, l, mid, ql, qr, color);
        if (qr > mid) range_assign_impl(p << 1 | 1, mid + 1, r, ql, qr, color);
        mask[p] = mask[p << 1] | mask[p << 1 | 1];
    }

    ui64 query_mask(int p, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) return mask[p];
        push(p);
        int mid = (l + r) >> 1;
        ui64 res = 0;
        if (ql <= mid) res |= query_mask(p << 1, l, mid, ql, qr);
        if (qr > mid) res |= query_mask(p << 1 | 1, mid + 1, r, ql, qr);
        return res;
    }

    void range_assign(int l, int r, int color) { range_assign_impl(1, 1, n, l, r, color); }

    int query_count(int l, int r) {
        return __builtin_popcountll(query_mask(1, 1, n, l, r));
    }
};
```

线段树 Beats: 区间 chmin + 区间和

赛时先看

题目信号: 操作是区间取 $\min/\max$, 普通懒标记不够。

本质: 支持 $a[i] = \min(a[i], x)$ 和区间和查询。

接法: 把本节 struct/class/函数组 整段抄到 `solve()` 上面; 外部只调用下面 API 列出的接口, 不要把内部递归参数直接当题面参数。

复杂度判定: 均摊 $O(\log n)$ 。

维护的量: n (长度); $tr[p]$ (sum 区间和、 mx1 最大值、 mx2 严格次大值、 cnt_mx 最大值个数)。

警告: 维护最大值 mx1 、严格次大值 mx2 、最大值个数 cnt_mx ; 只有 $\text{mx2} < x < \text{mx1}$ 才能整段打标记。

【最小完整示例 (先抄这一段就能跑)】

题目: n 个数, q 次操作。op=1 l r x: $a[1..r]$ 每个数与 x 取 $\min$; op=2 l r: 输出闭区间和。

```
SegBeatsChminSum seg(n);
seg.build(a); // 建树: a 必须 1-indexed (长度 n+1)
if (op == 1) { i64 x; cin >> x; seg.range_chmin(l, r, x); } // 区间取 min
else cout << seg.query_sum(l, r) << '\n'; // 区间和
```


样例: `a=[1,2,3,4,5]`; `range_chmin(1,5,3)` 后 `query_sum(1,5)` -> 输出 12。

【传参要求（照这个传不会错）】

- `SegBeatsChminSum seg(n)`: `n` 为元素个数, 下标 `1..n`。
- `build(a)`: `a` 必须 1-indexed, 长度至少 `n+1`。
- `range_chmin(l, r, x)`: 闭区间 `[l,r]` 内每个数变成 `min(a[i], x)`; 要求 `1 <= l <= r <= n`; 无返回值。
- `query_sum(l, r)`: 闭区间 `[l,r]` 的和; 要求 `1 <= l <= r <= n`; 返回 `i64`。

【API / 入口函数（赛时只认这里列的名字）】

- `build(const vector<i64>& a)` -> 完成建树或预处理
- `range_chmin(int l, int r, i64 x)` -> 对闭区间 `[l,r]` 取 `min(x, a[i])`
- `query_sum(int l, int r)` -> 查询闭区间和 返回 `i64`。

【改板时先认这几个量】

- `tr`: 树节点池（节点数组）。
- `sum`: 区间和/计数和。

```
struct SegBeatsChminSum {
    struct Node {
        i64 sum = 0;
        i64 mx1 = LLONG_MIN / 4;
        i64 mx2 = LLONG_MIN / 4;
        int cnt_mx = 0;
    };

    int n;
    vector<Node> tr;

    SegBeatsChminSum(int n = 0) { if (n) init(n); }
    void init(int n_) { n = n_; tr.assign(4 * n + 4, {}); }

    Node merge(Node a, Node b) {
        Node c;
        c.sum = a.sum + b.sum;
        if (a.mx1 == b.mx1) {
            c.mx1 = a.mx1;
            c.cnt_mx = a.cnt_mx + b.cnt_mx;
            c.mx2 = max(a.mx2, b.mx2);
        } else if (a.mx1 > b.mx1) {
            c.mx1 = a.mx1;
            c.cnt_mx = a.cnt_mx;
            c.mx2 = max(a.mx2, b.mx1);
        } else {
            c.mx1 = b.mx1;
            c.cnt_mx = b.cnt_mx;
            c.mx2 = max(a.mx1, b.mx2);
        }
        return c;
    }

    void build_impl(int p, int l, int r, const vector<i64>& a) {
        if (l == r) {
            tr[p].sum = tr[p].mx1 = a[l];
            tr[p].mx2 = LLONG_MIN / 4;
            tr[p].cnt_mx = 1;
            return;
        }
        int mid = (l + r) >> 1;
        build_impl(p << 1, l, mid, a);
        build_impl(p << 1 | 1, mid + 1, r, a);
        tr[p] = merge(tr[p << 1], tr[p << 1 | 1]);
    }

    void apply_chmin(int p, i64 x) {
        if (x >= tr[p].mx1) return;
        tr[p].sum -= (tr[p].mx1 - x) * tr[p].cnt_mx;
        tr[p].mx1 = x;
    }

    void push(int p) {
        apply_chmin(p << 1, tr[p].mx1);
        apply_chmin(p << 1 | 1, tr[p].mx1);
    }

    void range_chmin_impl(int p, int l, int r, int ql, int qr, i64 x) {
        if (qr < l || r < ql || x >= tr[p].mx1) return;
```

```

    if (ql <= l && r <= qr && x > tr[p].mx2) {
        apply_chmin(p, x);
        return;
    }
    push(p);
    int mid = (l + r) >> 1;
    range_chmin_impl(p << 1, l, mid, ql, qr, x);
    range_chmin_impl(p << 1 | 1, mid + 1, r, ql, qr, x);
    tr[p] = merge(tr[p << 1], tr[p << 1 | 1]);
}

i64 query_sum_impl(int p, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) return tr[p].sum;
    push(p);
    int mid = (l + r) >> 1;
    i64 ans = 0;
    if (ql <= mid) ans += query_sum_impl(p << 1, l, mid, ql, qr);
    if (qr > mid) ans += query_sum_impl(p << 1 | 1, mid + 1, r, ql, qr);
    return ans;
}

void build(const vector<i64>& a) { build_impl(1, 1, n, a); }
void range_chmin(int l, int r, i64 x) { range_chmin_impl(1, 1, n, l, r, x); }
i64 query_sum(int l, int r) { return query_sum_impl(1, 1, n, l, r); }
};

```

动态开点线段树：巨大值域单点加、区间和

赛时先看

题目信号：动态点权、值域极大、离线压缩不方便或不能压缩；常与主席树、线段树合并、扫描线结合。

本质：坐标范围可到 $1e9$ 、 $1e18$ ，但实际修改点不多时，动态维护频率或权值和。

接法：在线插入整数，查询某个数值区间内已有元素的个数或权值和。

复杂度判定：每次修改/查询 $O(\log V)$ ，空间为实际访问节点数 $O(q \log V)$ 。

维护的量：tr（动态节点池，0 号为空节点）；每节点 lc/rc（左右儿子编号）、sum（该值域区间累计和）。

警告：mid = (l+r)/2 可能溢出，应写成 $l + (r-1)/2$ ；值域和答案常需要 i64。

【最小完整示例（先抄这一段就能跑）】

题目：值域 $[-1e9, 1e9]$ 上在线插入整数，查询某个值域闭区间内已有元素的累计和。

```

DynamicSegTree seg;
int root = 0;
seg.add(root, -1000000000LL, 1000000000LL, x, +1); // 位置 x 上加 delta (可为负)
i64 cnt = seg.query(root, -1000000000LL, 1000000000LL, l, r); // 值域闭区间 [l,r] 的累计和

```

样例：依次对 $x=5$ 、 $x=3$ 各 add +1 后，query(root, -1e9, 1e9, -100, 10) -> 输出 2。

【传参要求（照这个传不会错）】

- DynamicSegTree seg: 无参构造；root 初始为 0，之后始终传同一个 root。
- add(p, l, r, pos, delta): 值域闭区间 $[l, r]$ 内位置 pos 累计加 delta; p 按引用传（root 会被更新）；要求 $l \leq pos \leq r$ ；无返回值。
- query(p, l, r, ql, qr): 值域闭区间 $[ql, qr]$ 的累计和；要求 $l \leq ql \leq qr \leq r$ ；返回 i64。
- 整套 (l,r) 范围每次调用必须一致（同一棵树同一个值域）；值域大时用 i64 并注意 mid 防溢出。

【API / 入口函数（赛时只认这里列的名字）】

- add(int& p, i64 l, i64 r, i64 pos, i64 delta) -> 加入一个元素/贡献
- query(int p, i64 l, i64 r, i64 ql, i64 qr) -> 查询 返回 i64。

【改板时先认这几个量】

- tr: 树节点池（节点数组）。
- root: 当前根节点编号（add/query 的第一个参数）。
- sum: 区间和/计数和。

```

struct DynamicSegTree {
    struct Node {
        int lc = 0, rc = 0;
        i64 sum = 0;
    };
    vector<Node> tr{Node{}}; // 0 号为空节点

    int new_node() {
        tr.push_back(Node{});
    }
};

```

```

    return (int)tr.size() - 1;
}

void add(int& p, i64 l, i64 r, i64 pos, i64 delta) {
    if (!p) p = new_node();
    tr[p].sum += delta;
    if (l == r) return;
    i64 mid = l + (r - l) / 2;
    // 不能把 tr[p].lc / rc 直接按引用传进递归: push_back 扩容会使引用失效。
    if (pos <= mid) {
        int child = tr[p].lc;
        add(child, l, mid, pos, delta);
        tr[p].lc = child;
    } else {
        int child = tr[p].rc;
        add(child, mid + 1, r, pos, delta);
        tr[p].rc = child;
    }
}

i64 query(int p, i64 l, i64 r, i64 ql, i64 qr) const {
    if (!p || qr < l || r < ql) return 0;
    if (ql <= l && r <= qr) return tr[p].sum;
    i64 mid = l + (r - l) / 2;
    return query(tr[p].lc, l, mid, ql, qr)
        + query(tr[p].rc, mid + 1, r, ql, qr);
}

};

// 示例: int root = 0;
// 示例: seg.add(root, -1000000000LL, 1000000000LL, position, +1);

```

线段树合并

赛时先看

题目信号：每个子树/集合都有一棵权值线段树，DFS 中把儿子信息合到父亲。

本质：树上每个节点维护一个值域线段树，合并子树信息，如颜色众数、动态开点频率统计。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API

列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定：总合并复杂度约 $O(\text{总节点数} \log V)$ ，实际开销正比于被合并到的节点数。

维护的量：`tr`（动态节点池，0 号为空节点）；每节点 `lc/rc`（左右儿子编号）、`sum`（该值域区间累计和）。

警告：合并会破坏被合并的树；如果后续还要用旧版本，必须可持久化或复制。

【最小完整示例（先抄这一段就能跑）】

题目：树上每个点一棵权值线段树，DFS 回溯时把儿子树合并进父亲，最后每个点都能拿到子树内的累计信息。

```

MergeSegTree seg;
int root[MAXN] = {}; // root[u]: u 那棵树的根，初始为 0
seg.add(root[u], 1, n, val[u], +1); // 把 u 的权值 val[u] 插进 u 自己的树
root[u] = seg.merge(root[u], root[v]); // 回溯时把儿子 v 的树并进 u，返回新根（v 的树被销毁）
i64 total = seg.tr[root[u]].sum; // 合并后 u 子树内的累计和

```

样例：两棵树各 `add` 一次 `+1` 后 `merge`，`seg.tr[root].sum` $\rightarrow$ 输出 2。

【传参要求（照这个传不会错）】

- `MergeSegTree seg`: 无参构造；每棵树的根初始为 0，值域闭区间 $(1, r)$ 全树统一。
- `add(p, l, r, pos, v)`: 值域闭区间 $[l, r]$ 内位置 `pos` 累计加 `v`；`p` 按引用传（根会变）；要求 $1 \leq \text{pos} \leq r$ ；无返回值。
- `merge(x, y)`: 把树 `y` 合并进树 `x`，返回新根（通常直接 `root[u] = merge(root[u], root[v])`）；要求 `x`、`y` 均为树根编号；合并后 `y` 的内容被破坏，不能再单独使用。

【API / 入口函数（赛时只认这里列的名字）】

- `merge(int x, int y)  $\rightarrow$  合并 返回 int。`
- `add(int& p, int l, int r, int pos, i64 v)  $\rightarrow$  加入一个元素/贡献`

【改板时先认这几个量】

- `tr`: 树节点池（节点数组）。
- `sum`: 区间和/计数和。

```

struct MergeSegTree {
    struct Node { int lc = 0, rc = 0; i64 sum = 0; };

```

```

vector<Node> tr{{{}}};

int new_node() {
    tr.push_back(Node{});
    return (int)tr.size() - 1;
}

void add(int& p, int l, int r, int pos, i64 v) {
    if (!p) p = new_node();
    if (l == r) {
        tr[p].sum += v;
        return;
    }
    int mid = (l + r) >> 1;
    // 不能把 tr[p].lc / tr[p].rc 直接按引用传进递归: new_node() 的 push_back 扩容会使引用悬垂。
    if (pos <= mid) {
        int lc = tr[p].lc;
        add(lc, l, mid, pos, v);
        tr[p].lc = lc;
    } else {
        int rc = tr[p].rc;
        add(rc, mid + 1, r, pos, v);
        tr[p].rc = rc;
    }
    tr[p].sum = (tr[p].lc ? tr[tr[p].lc].sum : 0) + (tr[p].rc ? tr[tr[p].rc].sum : 0);
}

int merge(int x, int y) {
    if (!x || !y) return x | y;
    tr[x].lc = merge(tr[x].lc, tr[y].lc);
    tr[x].rc = merge(tr[x].rc, tr[y].rc);
    tr[x].sum += tr[y].sum;
    return x;
}
};

```

分块：区间加 + 区间和

赛时先看

题目信号： $n, q \leq 2e5$ 左右，操作简单，分块能过。

本质： 懒得写线段树，或操作适合分块维护时。

接法： 把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定： $O(\sqrt{n})$ 。

维护的量： n （长度）、 B （块大小）、 num （块数）； $a[i]$ 原值、 $sum[b]$ 块 b 总和（含 lazy）、 $lazy[b]$ 块 b 的整体加标记。

警告： 散块暴力更新后要重建块和。

约定： $a = \text{init}$; // 1-based, 下标从 1 开始

【最小完整示例（先抄这一段就能跑）】

题目： n 个数， q 次操作。 $op=1 \ l \ r \ x$: $a[1..r]$ 全部加 x ; $op=2 \ l \ r$: 输出闭区间和。

```

SqrtRangeAddSum seg(a); // 构造即建树: a 必须 1-indexed (长度 n+1)
if (op == 1) { i64 x; cin >> x; seg.range_add(1, r, x); } // 区间加
else cout << seg.range_sum(1, r) << '\n'; // 区间和

```

样例: $a=[1,2,3,4,5]$; $\text{range_add}(2,4,10)$ 后 $\text{range_sum}(1,5) \rightarrow$ 输出 45。

【传参要求（照这个传不会错）】

- `SqrtRangeAddSum seg(a)`: a 必须 1-indexed (长度 $n+1$)，构造时自动分块建好。
- `range_add(1, r, v)`: 闭区间 $[1, r]$ 全部加 v ; 要求 $1 \leq l \leq r \leq n$; 无返回值。
- `range_sum(1, r)`: 闭区间 $[1, r]$ 的和; 要求 $1 \leq l \leq r \leq n$; 返回 $i64$ 。

【API / 入口函数（赛时只认这里列的名字）】

- `build(const vector<i64>& init)` $\rightarrow$ 完成建树或预处理
- `range_add(int l, int r, i64 v)` $\rightarrow$ 闭区间加值
- `range_sum(int l, int r)` $\rightarrow$ 查询闭区间和 返回 $i64$ 。

【改板时先认这几个量】

- `lazy`: 懒标记。

- `sum`: 区间和/计数和。

```
struct SqrtRangeAddSum {
    int n, B, num;
    vector<i64> a, sum, lazy;

    SqrtRangeAddSum(const vector<i64>& init = {}) {
        if (!init.empty()) build(init);
    }

    void build(const vector<i64>& init) {
        a = init; // 1-based, 下标从 1 开始。
        n = (int)a.size() - 1;
        B = max(1, (int)sqrt(n));
        num = (n + B - 1) / B;
        sum.assign(num, 0);
        lazy.assign(num, 0);
        for (int i = 1; i <= n; ++i) sum[(i - 1) / B] += a[i];
    }

    void range_add(int l, int r, i64 v) {
        int bl = (l - 1) / B, br = (r - 1) / B;
        if (bl == br) {
            for (int i = l; i <= r; ++i) a[i] += v, sum[bl] += v;
            return;
        }
        int end_l = min(n, (bl + 1) * B);
        for (int i = l; i <= end_l; ++i) a[i] += v, sum[bl] += v;
        for (int b = bl + 1; b <= br - 1; ++b) {
            lazy[b] += v;
            sum[b] += v * B;
        }
        for (int i = br * B + 1; i <= r; ++i) a[i] += v, sum[br] += v;
    }

    i64 range_sum(int l, int r) const {
        int bl = (l - 1) / B, br = (r - 1) / B;
        i64 ans = 0;
        if (bl == br) {
            for (int i = l; i <= r; ++i) ans += a[i] + lazy[bl];
            return ans;
        }
        int end_l = min(n, (bl + 1) * B);
        for (int i = l; i <= end_l; ++i) ans += a[i] + lazy[bl];
        for (int b = bl + 1; b <= br - 1; ++b) ans += sum[b];
        for (int i = br * B + 1; i <= r; ++i) ans += a[i] + lazy[br];
        return ans;
    }
};
```

ST 表

赛时先看

题目信号: 数组不修改, 很多次问区间最大/最小/gcd。

本质: 静态区间最值、区间 gcd。

接法: 把本节 `struct/class/函数组` 整段抄到 `solve()` 上面; 外部只调用下面 API

列出的接口, 不要把内部递归参数直接当题面参数。

复杂度判定: 预处理 $O(n \log n)$, 查询 $O(1)$ 。

维护的量: n (长度)、 LOG (层数)、 $\lg[i]$ (i 的 $\log_2$ 向下取整); `st[k][i]` (从 i 起长度 2^k 的合并答案)。

警告: 只适合可重复覆盖的运算, 如 `min/max/gcd`, 不适合 `sum`。

约定: `a` 是 1-based 数组 (`a[1..n]`), `n = (int)a.size() - 1`。

【最小完整示例 (先抄这一段就能跑)】

题目: 静态数组, q 次询问闭区间 $[l, r]$ 的最大值。

```
auto merge_max = [](i64 x, i64 y) { return max(x, y); };
SparseTable<i64, decltype(merge_max)> st(a, merge_max); // 建表: a 必须 1-indexed (长度 n+1)
while (q--) { int l, r; cin >> l >> r; cout << st.query(l, r) << '\n'; } // O(1) 区间最值
```

样例: `a=[1,3,2,5,4]`; `query(2,5)` $\rightarrow$ 输出 5。

【传参要求 (照这个传不会错)】

- `SparseTable<T, Merge> st(a, merge)`: `a` 必须 1-indexed (`a[1..n]`, 长度 $n+1$); `merge` 必须与查询运算一致 (`max/min/gcd/OR` 等幂等运算), 不能混用。
- `st.query(l, r)`: 闭区间 $[l, r]$ 的答案; 要求 $1 \leq l \leq r \leq n$; 返回 `T`, $O(1)$ 。

- 建表后数组不可修改；只支持幂等运算，不支持求和。

【不会用就照抄】

```
auto merge_max = [](i64 x, i64 y) { return max(x, y); };
SparseTable<i64, decltype(merge_max)> st(a, merge_max); // a[1..n]
auto ans = st.query(l, r); // 闭区间 [l,r] 最大值
```

- 建完以后不能修改数组。
- 先看模板到底维护 min/max/gcd/OR/AND 中哪一种；合并函数不同不能混抄。

【API / 入口函数（赛时只认这里列的名字）】

- `SparseTable<T, Merge> st(a, merge)` -> 用 1-indexed 静态数组建表；merge 必须与查询运算一致。
- `st.query(l, r)` -> $O(1)$ 查询闭区间 $[l, r]$ 。

【核心逻辑（改代码时别破坏）】

- `st[k][i]` 表示从 i 开始、长度 2^k 的答案。
- 查询 $[l, r]$ 用两个长度 2^k 的块覆盖；因此要求运算幂等（min/max/gcd 等）。

```
template <class T, class Merge>
struct SparseTable {
    int n, LOG;
    vector<int> lg;
    vector<vector<T>> st;
    Merge merge;

    SparseTable(const vector<T>& a, Merge merge_) : merge(std::move(merge_)) {
        build(a);
    }

    void build(const vector<T>& a) {
        n = (int)a.size() - 1; // 公式/约定: a is 1-based
        LOG = 1;
        while ((1 << LOG) <= max(1, n)) ++LOG; // 不依赖 GCC 的 __lg。
        lg.assign(n + 1, 0);
        for (int i = 2; i <= n; ++i) lg[i] = lg[i >> 1] + 1;

        st.assign(LOG, vector<T>(n + 1));
        st[0] = a;
        for (int k = 1; k < LOG; ++k) {
            for (int i = 1; i + (1 << k) - 1 <= n; ++i) {
                st[k][i] = merge(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
            }
        }
    }

    T query(int l, int r) const {
        int k = lg[r - l + 1];
        return merge(st[k][l], st[k][r - (1 << k) + 1]);
    }
};
```

单调队列

赛时先看

题目信号：每次只关心最近 k 个元素中的最大/最小值。

本质：滑动窗口最值、DP 优化。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()`

里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O(n)$ 。

维护的量： q （存下标的双端队列，队首是当前窗口最大值下标，队列内下标对应值递减）。

警告：队首先弹过期，再用答案；最大值队列保持递减。

【最小完整示例（先抄这一段就能跑）】

题目： n 个数，窗口大小 k ，输出每个滑动窗口的最大值。

```
vector<int> a; // 0-indexed
vector<int> ans = sliding_window_max(a, k); // ans[i] 是以 a[i+k-1] 为右端点的窗口最大值
for (int x : ans) cout << x << ' '; // 共 a.size()-k+1 个
```

样例： $a=[1,3,-1,-3,5,3,6,7]$ ， $k=3$ -> 输出 3 3 5 5 6 7。

【传参要求（照这个传不会错）】

- `sliding_window_max(a, k)`: `a` 为 0-indexed 数组; `k` 为窗口长度, 要求 `1 <= k <= a.size()`。
- 返回值: `vector<int> ans`, `ans[i]` 是以 `a[i+k-1]` 为右端点的窗口最大值, 长度 `a.size()-k+1`。
- 要窗口最小值: 把第二个 `while` 里 `a[q.back()] <= a[i]` 的 `<=` 改成 `>=` (队列改为递增), 其余不变。

```
// 约定: 本函数为 0-indexed.
vector<int> sliding_window_max(const vector<int>& a, int k) {
    deque<int> q;
    vector<int> ans;
    for (int i = 0; i < (int)a.size(); ++i) {
        while (!q.empty() && q.front() <= i - k) q.pop_front();
        while (!q.empty() && a[q.back()] <= a[i]) q.pop_back();
        q.push_back(i);
        if (i >= k - 1) ans.push_back(a[q.front()]);
    }
    return ans;
}
```

对顶堆：动态中位数、滑动窗口中位数与绝对偏差和

赛时先看

题目信号：题面出现“数据流中位数”“每次加入一个数后输出中位数”“长度为 `k` 的每个窗口中位数”“把窗口内所有数变成同一个数的最小代价”。如果只要静态第 `k` 小，翻排序/主席树；如果要任意排名，不止中位数，翻平衡树/PBDS。

本质：维护一个动态多重集合的中位数。左边用大根堆保存较小的一半，右边用小根堆保存较大的一半；额外维护两边元素和，就能 $O(1)$ 算所有数到中位数的绝对偏差和。带懒删除，适合滑动窗口。

接法：动态中位数直接 `insert(x)` 后输出 `lower_median()`；滑动窗口先插入新值，窗口长度超过 `k` 就 `erase(a[i-k])`，长度达到 `k` 后输出中位数或 `min_abs_deviation_sum()`。若题目要求偶数长度取上中位数，输出 `upper_median()`；若要求两个中位数平均值，用 `median_average()`。

复杂度判定：插入、删除、维护平衡均为 $O(\log n)$ ；查询中位数和绝对偏差和为均摊 $O(1)$ 。空间 $O(n)$ 。

维护的量：`low`（较小一半，大根堆，堆顶即下中位数）、`high`（较大一半，小根堆）；`low_size/high_size`（两半逻辑元素个数）、`low_sum/high_sum`（两半元素和）。

警告：删除用懒删除，调用 `erase(x)` 前必须保证当前集合里确实有一个 `x`；左堆大小保持等于右堆或比右堆多 1，所以 `lower_median()` 是下中位数。偶数个数时，任意位于两个中位数之间的数都能最小化绝对偏差和，本模板用下中位数计算。

【最小完整示例（先抄这一段就能跑）】

题目：依次读入 `n` 个数，每次读入后输出当前已读数的下中位数。

```
DualHeapMedian dh;
for (int i = 1; i <= n; ++i) {
    i64 x; cin >> x;
    dh.insert(x); // 插入一个数，内部自动平衡两堆
    cout << dh.lower_median() << '\n'; // 当前下中位数
}
```

样例：依次插入 1 5 2 4 3，每次输出 -> 1 1 2 2 3。

【传参要求（照这个传不会错）】

- `DualHeapMedian dh`: 无参构造，空集合。
- `insert(x)`: 插入一个数 `x`；无返回值。
- `erase(x)`: 懒删除一个 `x`；调用前必须保证集合里确实存在 `x`；无返回值。
- `lower_median()`: 下中位数（偶数个元素时取较小的那个），返回 `i64`；集合不能为空。
- `upper_median()`: 上中位数，返回 `i64`；`median_average()`: 两个中位数的平均值，返回 `long double`。
- `min_abs_deviation_sum()`: 所有数到下中位数的绝对偏差和，返回 `i64`。
- `size()`: 当前逻辑元素个数，返回 `int`。
- 滑动窗口: 0-indexed 数组 `a` 直接用 `sliding_window_lower_median(a,k)`（中位数）或 `sliding_window_abs_cost_to_median(a,k)`（绝对偏差和）。

【API / 入口函数（赛时只认这里列的名字）】

- `erase(i64 x)` -> 删除元素
- `insert(i64 x)` -> 插入元素/字符串
- `lower_median()` -> 下中位数 返回 `i64`。
- `median_average()` -> 两个中位数的平均值 返回 `long double`。

- `min_abs_deviation_sum()` -> 所有数到下中位数的绝对偏差和 返回 `i64`。
- `size()` -> 查询集合大小 返回 `int`。
- `upper_median()` -> 上中位数 返回 `i64`。

【改板时先认这几个量】

- `low`: 较小一半，大根堆，堆顶是下中位数。
- `high`: 较大一半，小根堆。
- `high_size`: 只统计逻辑存在的元素。

```
struct DualHeapMedian {
    priority_queue<i64> low; // 较小一半，大根堆，堆顶是下中位数。
    priority_queue<i64, vector<i64>, greater<i64>> high; // 较大一半，小根堆。
    unordered_map<i64, int> lazy_low, lazy_high;
    int low_size = 0, high_size = 0; // 只统计逻辑存在的元素。
    i64 low_sum = 0, high_sum = 0;

    void dec(unordered_map<i64, int>& mp, i64 x) {
        auto it = mp.find(x);
        if (--it->second == 0) mp.erase(it);
    }

    void prune_low() {
        while (!low.empty() && lazy_low.count(low.top())) {
            i64 x = low.top();
            low.pop();
            dec(lazy_low, x);
        }
    }

    void prune_high() {
        while (!high.empty() && lazy_high.count(high.top())) {
            i64 x = high.top();
            high.pop();
            dec(lazy_high, x);
        }
    }

    void move_low_to_high() {
        prune_low();
        i64 x = low.top();
        low.pop();
        --low_size;
        low_sum -= x;
        high.push(x);
        ++high_size;
        high_sum += x;
    }

    void move_high_to_low() {
        prune_high();
        i64 x = high.top();
        high.pop();
        --high_size;
        high_sum -= x;
        low.push(x);
        ++low_size;
        low_sum += x;
    }

    void balance() {
        prune_low();
        prune_high();
        while (low_size < high_size) move_high_to_low();
        while (low_size > high_size + 1) move_low_to_high();
        prune_low();
        prune_high();
    }

    int size() const {
        return low_size + high_size;
    }

    bool empty() const {
        return size() == 0;
    }

    void insert(i64 x) {
        prune_low();
        if (low.empty() || x <= low.top()) {
            low.push(x);
            ++low_size;
        }
    }
};
```

```

        low_sum += x;
    } else {
        high.push(x);
        ++high_size;
        high_sum += x;
    }
    balance();
}

void erase(i64 x) {
    assert(!empty());
    prune_low();
    prune_high();
    if (!low.empty() && x <= low.top()) {
        ++lazy_low[x];
        --low_size;
        low_sum -= x;
        if (!low.empty() && low.top() == x) prune_low();
    } else {
        ++lazy_high[x];
        --high_size;
        high_sum -= x;
        if (!high.empty() && high.top() == x) prune_high();
    }
    balance();
}

i64 lower_median() {
    assert(!empty());
    balance();
    return low.top();
}

i64 upper_median() {
    assert(!empty());
    balance();
    if (low_size == high_size) return high.top();
    return low.top();
}

long double median_average() {
    assert(!empty());
    balance();
    if (low_size == high_size) return ((long double)low.top() + high.top()) / 2.0L;
    return low.top();
}

i64 min_abs_deviation_sum() {
    i64 m = lower_median();
    return m * low_size - low_sum + high_sum - m * high_size;
}
};

// 约定: 本函数为 0-indexed.
vector<i64> sliding_window_lower_median(const vector<i64>& a, int k) {
    assert(1 <= k && k <= (int)a.size());
    DualHeapMedian dh;
    vector<i64> answer;
    for (int i = 0; i < (int)a.size(); ++i) {
        dh.insert(a[i]);
        if (i >= k) dh.erase(a[i - k]);
        if (i >= k - 1) answer.push_back(dh.lower_median());
    }
    return answer;
}

// 约定: 本函数为 0-indexed.
vector<i64> sliding_window_abs_cost_to_median(const vector<i64>& a, int k) {
    assert(1 <= k && k <= (int)a.size());
    DualHeapMedian dh;
    vector<i64> answer;
    for (int i = 0; i < (int)a.size(); ++i) {
        dh.insert(a[i]);
        if (i >= k) dh.erase(a[i - k]);
        if (i >= k - 1) answer.push_back(dh.min_abs_deviation_sum());
    }
    return answer;
}

```

典型模型: CSES Sliding Median / Sliding Cost。读入数组 a 和窗口长度 k , 中位数输出 `sliding_window_lower_median(a,k)`; 若问每个窗口把所有数变成同一个数的最小总代价, 输出 `sliding_window_abs_cost_to_median(a,k)`。

单调栈：左右第一个更小元素

赛时先看

题目信号：每个元素作为最小/最大值能控制多大区间。

本质：柱状图、贡献法、区间最值边界。

接法：给 1-indexed 数组 `a` (`a[0]` 占位)，直接 `auto [L, R] = nearest_less(a);`，以 `i` 为最小值的区间就是 `[L[i]+1, R[i]-1]`，长度 `R[i]-L[i]-1`。

复杂度判定： $O(n)$ 。

维护的量：`L[i]/R[i]` (`i` 左/右第一个严格更小元素的下标，无则 $0/n+1$)；`st` (值严格递增的下标栈)。

警告：处理相等元素时要统一用 `<` 还是 `<=`，避免重复计数。

约定：`int n = (int)a.size() - 1;` // 1-based，下标从 1 开始

【最小完整示例（先抄这一段就能跑）】

```
// a 1-indexed: 求每个位置左/右第一个严格更小元素的下标。
vector<int> a = {0, 2, 1, 5, 6, 2, 3}; // a[0] 占位
auto [L, R] = nearest_less(a);
cout << L[2] << ' ' << R[2] << '\n'; // 0 7 (a[2]=1 是全数组最小值)
cout << L[4] << ' ' << R[4] << '\n'; // 3 5 a[4]=6
```

- 样例：输出 0 7 与 3 5。

【传参要求（照这个传不会错）】

- `a`: 1-indexed (长度 `n+1`, `a[0]` 占位, `n = a.size()-1`)。
- 返回值 `{L, R}`: `L[i]` = 左边最近的满足 `a[L[i]] < a[i]` 的下标 (没有则 0); `R[i]` = 右边最近的满足 `a[R[i]] < a[i]` 的下标 (没有则 `n+1`)。
- 以 `i` 为最小值能控制的最大区间: `[L[i]+1, R[i]-1]`，宽度 `R[i]-L[i]-1`。
- 想把“小于等于”也算更小: 把两处 `a[st.back()] >= a[i]` 的 `>=` 改成 `>`。

```
// 维护的量: L[i] / R[i] (i 左/右第一个严格更小的下标, 无则 0 / n+1); st (值单调递增的下标栈)。
// 不变量: 任何时候栈内 a[st[k]] 严格递增; 弹出一个元素时, 它的更小边界就是栈内前一个位置。
pair<vector<int>, vector<int>> nearest_less(const vector<int>& a) {
    int n = (int)a.size() - 1; // 1-based, 下标从 1 开始。
    vector<int> L(n + 1), R(n + 1);
    vector<int> st;
    for (int i = 1; i <= n; ++i) {
        // 弹出 >= a[i] 的, 留下的栈顶就是左边最近的更小值
        while (!st.empty() && a[st.back()] >= a[i]) st.pop_back();
        L[i] = st.empty() ? 0 : st.back();
        st.push_back(i);
    }
    st.clear();
    for (int i = n; i >= 1; --i) {
        while (!st.empty() && a[st.back()] >= a[i]) st.pop_back();
        R[i] = st.empty() ? n + 1 : st.back(); // 倒序扫描对称求右边界
        st.push_back(i);
    }
    return {L, R};
}
```

单调栈例题：柱状图最大矩形

赛时先看

题目信号：每个柱子高度，问连续区间面积最大值。

本质：求直方图中最大矩形面积。

接法：给 1-indexed 柱高 `h` (`h[0]` 占位)，一行 `largest_rectangle_histogram(h)` 出答案；0-indexed 输入记得前面补 0。

复杂度判定： $O(n)$ 。

维护的量：`L/R` (来自 `nearest_less` 的左右更小边界)；`ans` (枚举每根柱时维护的最大面积)。

警告：宽度是 `R[i] - L[i] - 1`。

约定：`int n = (int)h.size() - 1;` // 1-based，下标从 1 开始

【最小完整示例（先抄这一段就能跑）】

```
// h 1-indexed: 柱状图最大矩形面积。
vector<int> h = {0, 2, 1, 5, 6, 2, 3}; // h[0] 占位
cout << largest_rectangle_histogram(h) << '\n'; // 10
```

- 样例：输出 10 (高度 2 覆盖 1..5 号柱，面积 $2*5$ ；高度 5 覆盖 3..4 号柱，面积 $5*2$)。

【传参要求（照这个传不会错）】

- `h`: 1-indexed 柱高（长度 `n+1`, `h[0]` 占位），柱高可为 0。
- 返回值: `i64` 最大矩形面积，即 `max(h[i] * (R[i]-L[i]-1))`。
- 依赖 `nearest_less`，两个函数一起抄。

```
// 维护的量: L/R (nearest_less 算出的左右更小边界); ans (扫描过程中维护的最大面积)。
// 不变量: 以第 i 根柱为最小高度的最大矩形宽度是 R[i]-L[i]-1, 面积 = h[i] * 宽度。
i64 largest_rectangle_histogram(const vector<int>& h) {
    int n = (int)h.size() - 1; // 1-based, 下标从 1 开始。
    auto [L, R] = nearest_less(h);
    i64 ans = 0;
    for (int i = 1; i <= n; ++i) {
        ans = max(ans, 1LL * h[i] * (R[i] - L[i] - 1)); // 左右都不小于 h[i] 的区间宽度
    }
    return ans;
}
```

区间合并

赛时先看

题目信号: 很多 `[l,r]`，问合并后段数/总长度。

本质: 合并重叠区间、求覆盖长度。

接法: 把区间全部放进 `vector<pair<i64,i64>> segs`，调用 `merge_intervals(segs)`；返回的 `merged` 就是合并结果，`merged.size()` 是段数，长度自己按闭/半开口径累加。

复杂度判定: $O(n \log n)$ 。

维护的量: `segs` (按左端点排序后的区间表)；`res` (合并结果栈，`res.back().second` 是当前覆盖到的最右端)。

警告: 闭区间长度是 `r-l+1`，半开区间长度是 `r-l`。

【最小完整示例（先抄这一段就能跑）】

```
// 合并重叠/相接区间，输出合并后的段数与闭区间总覆盖长度。
vector<pair<i64, i64>> segs = {{1, 3}, {2, 6}, {8, 10}, {15, 18}};
auto merged = merge_intervals(segs);
cout << merged.size() << '\n'; // 3
i64 cover = 0;
for (auto [l, r] : merged) cover += r - l + 1; // 闭区间长度
cout << cover << '\n'; // 13
```

- 样例: 输出 3 与 13。

【传参要求（照这个传不会错）】

- `segs`: `vector<pair<i64,i64>>`，闭区间 `{l, r}`；顺序任意（内部 `sort`），`i64` 到 `1e18` 安全。
- 返回值: 合并后的区间（`vector<pair<i64,i64>>`）；重叠或相接（`l <= res.back().second`）都会并成一段。
- 半开区间口径: 长度按 `r-l` 算，合并条件不变。

```
// 维护的量: segs (按左端点排序后的区间); res (合并结果栈, res.back().second 是当前覆盖到的最右端)。
// 不变量: res 中相邻区间严格不相交 (后一个 l 大于前一个 r)。
vector<pair<i64,i64>> merge_intervals(vector<pair<i64,i64>> segs) {
    sort(segs.begin(), segs.end());
    vector<pair<i64,i64>> res;
    for (auto [l, r] : segs) {
        if (res.empty() || l > res.back().second) res.push_back({l, r}); // 与当前段断开, 新开一段
        else res.back().second = max(res.back().second, r); // 重叠/相接则并进当前段
    }
    return res;
}
```

启发式合并：小集合并入大集合

赛时先看

题目信号: 很多集合需要两两合并（并集）；合并后还要维护集合内信息（元素集合、计数、和、最小值、去重数）；或树上每个点一个集合、自底向上合并到父亲（如“每个子树内不同颜色数”）。

本质: 把“小集合”的每个元素逐个搬进“大集合”再丢弃小集合。每个元素每被搬一次，所在集合大小至少翻倍，所以总搬移次数 $O(n \log n)$ ——这是它快的原因。

复杂度判定: 每个元素最多搬 $O(\log n)$ 次；用 `std::set` 时每次插入还要 $O(\log n)$ ，总 $O(n \log^2 n)$ ($n \leq 2e5$ 可用)；用哈希集合则可到均摊 $O(n \log n)$ 。

若合并是“可撤销的”，翻同章可撤销并查集；若只要子树颜色数且要求更快，翻 C 章 DSU on Tree。

维护的量: 每个集合的元素表（`set<int>`）与伴随统计（如 `sum`）；合并后大集合的统计量就是答案。

接法: `merge_bags(big, small)` 一行合并，先保证大集合在左边；树上回溯时把儿子的包并进父亲。

警告：先 `swap` 保证大集合在左边再搬；搬完必须清空小集合；伴随信息要跟着集合一起 `swap`。

【不会用就照抄】

```
BagInfo bag(n + 1);           // 每个点一个包
bag[u].s.insert(x); bag[u].sum += x; // 往包里塞元素
merge_bags(bag[u], bag[v]); // 把 v 的包并入 u 的包
int ans = (int)bag[u].s.size(); // 合并后的集合大小/和就是答案
```

【最小完整示例（先抄这一段就能跑）】

题目：n 个集合初始各含一个元素（集合 i 含元素 i）；m 次操作。op=1 x y：合并集合 x、y（取并集，重复元素只留一份）；op=2 x：输出集合 x 的元素个数与元素和。

```
vector<BagInfo> bag(n + 1);           // 1. 结构体定义：每个集合一个包，编号 1..n
for (int i = 1; i <= n; ++i) {
    bag[i].s.insert(i);               // 2. 初始化：塞入初始元素
    bag[i].sum += i;
}
while (m--) {
    int op, x, y;
    cin >> op;
    if (op == 1) {
        cin >> x >> y;
        merge_bags(bag[x], bag[y]); // 3. 调用：把 y 的包并入 x 的包（内部自动小入大）
    } else {
        cin >> x;
        cout << bag[x].s.size() << ' ' << bag[x].sum << '\n'; // 4. 输出：元素个数 + 元素和
    }
}
```

样例：初始 `bag[1].s={1}`、`bag[2].s={2}`；`merge_bags(bag[1], bag[2])` 后 `->`
`bag[1].s.size()=2`，`bag[1].sum=3`。

【传参要求（照这个传不会错）】

- `BagInfo bag(n + 1)`：每个集合一个包；s 存集合元素（`set<int>`），sum 是伴随统计（元素和，示例）。
- `merge_bags(big, small)`：把 `small` 的包并入 `big`；内部自动把小集合并入大集合（必要时整体交换），伴随信息跟着一起交换；并入后 `small` 被清空，不能再用。
- `merge_into(big, small)`：无伴随信息版，直接对两个 `set<int>` 合并，规则同上。
- `dfs_merge_example(u, parent, bag, g)`：树形合并骨架；在回溯处自动把儿子并入父亲，合并后 `bag[u].s.size()` 即子树内不同元素数。
- 换元素类型：`set<int>` 改成 `set<i64>` / `set<pair<int,int>>` 等即可，合并函数不用改。

【抄板清单（照着做就行）】

1. 抄哪段：`merge_into` 或 `merge_bags`（带伴随信息用后者）。
2. 构造：给每个“起点”建一个空 `BagInfo`，把初始元素 `insert` 进去。
3. 操作：需要合并时调用 `merge_bags(big, small)`；函数内部自动把小的并入大的。
4. 取结果：`big.s.size()` / `big.sum` 就是合并后的集合信息。

【改造点（按题目改这几处）】

- 集合元素类型：`set<int>` 换成 `set<i64>` / `set<pair<int,int>>` / `unordered_set`。
- 伴随信息：`sum` 换成题目要的（个数、最小值、最大值、和值），跟着 `swap` 一起走。
- 树形合并：把 `dfs_merge_example` 的合并点放到回溯处，答案在回溯后取。
- 要按颜色统计频次而非去重：`set` 换成 `map<int,int>` 时“小入大”原则不变。

【核心逻辑（改代码时别破坏）】

- `size` 小的必须在右边，搬完清空小集合，否则元素会被重复计入。
- 伴随统计量必须与集合一起 `swap`，否则统计值跟错集合。

【改板时先认这几个量】

- `s`：集合元素表。
- `sum`：集合内元素总和（伴随信息示例）。

```
// 维护的量：big.s = 合并后的大集合；small.s = 待并入的小集合（并入后清空）。
// 不变量：每个元素在任何时刻只存在于一个集合；集合大小每搬一次至少翻倍，
//          所以每个元素最多被搬 O(log n) 次。
// 把 small 的所有元素并入 big，并保证总是把小的并入大的。
```

```
// 元素去重语义：相同元素只保留一份。
void merge_into(set<int>& big, set<int> small) {
    if (big.size() < small.size()) big.swap(small); // 保证 big 是较大的集合。
    for (int x : small) big.insert(x);
    small.clear(); // 必须清空，否则元素被重复计入。
}

// 带伴随信息（元素和）的版本：信息跟着集合一起走。
struct BagInfo {
    set<int> s;
    i64 sum = 0;
};

void merge_bags(BagInfo& big, BagInfo& small) {
    if (big.s.size() < small.s.size()) {
        big.s.swap(small.s);
        swap(big.sum, small.sum); // 伴随信息必须和集合同步交换。
    }
    for (int x : small.s) {
        if (big.s.insert(x).second) big.sum += x; // 只有新元素才计入和。
    }
    small.s.clear();
    small.sum = 0;
}

// 典型骨架：树上每个点一个集合，自底向上把儿子的集合合并进父亲，
// 最后每个点的集合大小/元素和就是该子树内的答案。
// 用法：先 dfs 处理完所有儿子（递归），再在回溯时调用 merge_bags。
void dfs_merge_example(int u, int parent, vector<BagInfo>& bag, const vector<vector<int>>& g) {
    for (int v : g[u]) {
        if (v == parent) continue;
        dfs_merge_example(v, u, bag, g);
        merge_bags(bag[u], bag[v]); // 回溯时把小儿子并入父亲。
    }
    // 此时 bag[u].s 包含 u 子树内全部元素；bag[u].s.size() 即子树不同元素数。
}
```

典型模型：树上每个点有一种颜色，问每个子树内有多少种不同颜色（CF 600E 的弱化版直接数 size；带频次统计请翻 C 章「DSU on Tree」）。合并方向固定的树形合并、按大小合并的并查集也是同一个「小入大」思想。

04 高级数据结构与离线分治

可持久化、平衡树、可并堆、动态树、线性基、PBDS 以及 CDQ/整体二分/离线动态连通性放在这里。

笛卡尔树

赛时先看

题目信号：需要按区间最小/最大值递归分治；数组下标顺序和堆序都重要。

本质：把数组转成同时满足中序为原序列、堆性质的树。常用于 RMQ、区间最值分治、最大矩形变种。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；优先调用下面 API 列出的入口，其余 helper 默认不要从 `solve()` 直接调。

复杂度判定： $O(n)$ 。

维护的量：`left_child/right_child`（下标 $1..n$ 每个位置的左右儿子，0 表示空）；返回根节点编号。

警告：最小笛卡尔树用 `>` 弹栈，最大笛卡尔树用 `<` 弹栈；根是最后留在栈底的元素。

【最小完整示例（先抄这一段就能跑）】

题目：给 `a[1..n]` 建最小笛卡尔树（RMQ / 区间最值分治），拿到树根和每个位置的左右儿子。

```
vector<int> lc(n + 1), rc(n + 1);
int root = build_cartesian_tree(a, lc, rc); // 1. 建树：a 必须 1-indexed, lc/rc 被填成左右儿子
// 2. 中序遍历 lc/rc 即原序列；根是最小值位置，左右子树分别对应左右区间
cout << root << '\n'; // 3. root 即全局最小值所在下标
```

样例：`a=[3,1,2] -> root=2 (a[2]=1 最小)`，`lc[2]=1, rc[2]=3`。

【传参要求（照这个传不会错）】

- `a`: 1-indexed，只用 `a[1..n]`，`a[0]` 不参与建树。
- `left_child/right_child`: 输出参数，传入空 vector 即可，函数会填好每个下标 $1..n$ 的左右儿子，0 表示空。
- 返回值：根节点下标（全局最小元素所在位置）， $n \geq 1$ 时不为 0。
- 默认建最小笛卡尔树；要最大笛卡尔树，把栈内比较 `a[st.back()] > a[i]` 的 `>` 改成 `<`。

【API / 入口函数（赛时只认这里列的名字）】

- `build_cartesian_tree(const vector<int>& a, vector<int>& left_child, vector<int>& right_child)` -> 构建最小笛卡尔树，返回根。下标 `1..n`。

```
// 构建最小笛卡尔树，返回根。下标 1..n。
int build_cartesian_tree(const vector<int>& a, vector<int>& left_child, vector<int>& right_child) {
    int n = (int)a.size() - 1;
    left_child.assign(n + 1, 0);
    right_child.assign(n + 1, 0);
    vector<int> st;
    for (int i = 1; i <= n; i++) {
        int last = 0;
        while (!st.empty() && a[st.back()] > a[i]) {
            last = st.back();
            st.pop_back();
        }
        if (!st.empty()) right_child[st.back()] = i;
        left_child[i] = last;
        st.push_back(i);
    }
    return st.empty() ? 0 : st.front();
}
```

主席树：静态区间第 k 小 / 排名 / 前驱后继

赛时先看

题目信号：数组不修改，多次询问 $[L,R]$ 的第 k 小、中位数、排名、前驱或后继。静态区间顺序统计，直接上主席树。

本质：对值域建权值线段树，`root[i]` 存前缀 `a[1..i]` 的版本；`root[R] - root[L-1]` 两版本相减就是区间 $[L,R]$ 的频次分布，第 k 小沿“左右儿子计数之差”决定往左还是往右走。

复杂度判定：建树 $O(n \log M)$ ；单次查询 $O(\log M)$ ；空间 $O(n \log M)$ ， M 为离散化后值域大小； n, m 到 $2e5$ 很稳，节点池约 $n*22$ 个，vector 动态增长即可。

维护的量：`root` (`root[i]`：前缀 `a[1..i]` 的版本根)；`tr` (节点池，0 号为空节点)；`xs` (排序去重后的原值，离散编号 `1..xs.size()`)。

接法：数组必须写成 `a[1..n]`。先 `build_from_array(a)`；之后直接调用 `kth_value / count_lt / count_le / rank_of / predecessor / successor`。

警告： k 从 1 开始；`kth_value()` 返回原值；`*_impl()`

都是内部函数；建树后不支持单点修改，动态修改请用下一节“动态区间顺序统计（树套树）”。

【最小完整示例（先抄这一段就能跑）】

题目：静态数组 `a[1..n]`， m 次询问 $[L,R]$ 的第 k 小。

```
PersistentKth pt;
pt.build_from_array(a);           // 1. 结构体定义 + 建树：a 必须 1-indexed
while (m--) {
    int L, R, k;
    cin >> L >> R >> k;
    cout << pt.kth_value(L, R, k) << '\n'; // 2. 调用：返回第 k 小的原值
}
```

样例：`a=[1,5,2,4,3]`；问 $[2,5]$ 第 2 小 -> 输出 3 (区间内是 5,2,4,3，排序后第 2 小是 3)。

【传参要求（照这个传不会错）】

- `build_from_array(a)`：`a` 必须 1-indexed (`a[0]` 不参与)；内部会离散化，不需你处理。
- `kth_value(L, R, k)`： k 从 1 开始 (第 1 小传 1)； $1 \leq L \leq R \leq n$ ；返回原数组里的实际值。
- 中位数：奇数长度区间传 $k = (R-L+2)/2$ 。
- `rank_of(L,R,x)` 返回排名；`count_lt/count_le` 返回严格小于/小于等于的个数；`predecessor/successor` 找不到时返回 `nullopt`。
- 只支持静态数组；带修改用下一节树套树。

【不会用就照抄（最常用：静态区间第 k 小 / 中位数）】

```
vector<i64> a(n + 1);
for (int i = 1; i <= n; ++i) cin >> a[i];

PersistentKth pt;
pt.build_from_array(a);

// [l,r] 第 k 小: k 从 1 开始
cout << pt.kth_value(l, r, k) << '\n';
```



```
// 奇数长度区间中位数
```

```
cout << pt.kth_value(l, r, (r - l + 2) / 2) << '\n';
```

- 你在 `solve()` 里主要只碰: `build_from_array` / `kth_value` / `count_lt` / `count_le` / `rank_of` / `predecessor` / `successor`。
- 不要直接调 `kth_impl()`、`count_prefix_impl()`: 它们的 `l/r` 是离散值域, 不是题目询问区间。
- `kth_value()` 返回的是原数组实际值, 不是离散化编号。
- 建树后数组不能修改: 有 `a[pos]=x` 就换下一节动态模板。

【API / 入口函数 (赛时只认这里列的名字)】

- `pt.build_from_array(a)` $\rightarrow$ `a` 必须 1-indexed; 建立静态前缀版本。
- `pt.kth_value(L,R,k)` $\rightarrow$ 闭区间第 `k` 小 实际值; `k` 从 1 开始。
- `pt.count_lt(L,R,x)` $\rightarrow$ 统计 `<x` 的元素个数。
- `pt.count_le(L,R,x)` $\rightarrow$ 统计 `<=x` 的元素个数。
- `pt.rank_of(L,R,x)` $\rightarrow$ 排名 = `<x` 的个数 + 1。
- `pt.predecessor/successor(...)` $\rightarrow$ 返回 `optional<i64>`; 不存在为 `nullopt`。

【抄板清单 (照着做就行)】

1. 抄哪段: 整个 `PersistentKth` 结构体, 抄到 `solve()` 外面。
2. 构造: `PersistentKth pt`; (数据量大时可 `PersistentKth pt(n * 22)`; 预留节点空间)。
3. 建树: `pt.build_from_array(a)`; `a` 必须 1-indexed, `a[0]` 不参与。
4. 调用: 第 `k` 小 `pt.kth_value(L, R, k)`; 中位数 = `k = (R-L+2)/2` 的第 `k` 小; 排名 `pt.rank_of(L,R,x)`。
5. 取结果: `kth_value` 返回原数组里的实际值, 直接输出。

【改造点 (按题目改这几处)】

- 只问第 `k` 小/中位数: 建树 + `kth_value` 两行搞定, 其他函数不碰。
- 问排名/前驱/后继: 用 `count_lt` / `count_le` / `rank_of` / `predecessor` / `successor`, 输入 `x` 用原值。
- 区间 `[L,R]`: 题目闭区间, 1-indexed 直接传。
- 前驱/后继返回 `nullopt`: 表示不存在, 题目要输出什么哨兵 (如 `-1`) 自己判。

【核心逻辑 (改代码时别破坏)】

- `root[i]` 是前缀 `a[1..i]` 的频次版本; 区间 `[L,R]` 的频次 = `root[R] - root[L-1]`。
- 查第 `k` 小时比较“两个版本左儿子计数之差”, 决定向左还是向右。

【改板时先认这几个量】

- `root`: `root[i]`: 前缀 `a[1..i]` 的版本根。
- `tr`: 树节点池 (节点数组)。
- `xs`: 排序去重后的值域。
- `sum`: 区间和/计数和。

接口含义 (看不懂参数时看这里)

- `build_from_array(a)`: `a` 必须是 1-indexed, `a[0]` 不参与建树。
- `kth_value(L,R,k)`: 查询闭区间 `[L,R]` 的第 `k` 小, `1 <= k <= R-L+1`。
- `count_lt(L,R,x)` / `count_le(L,R,x)`: 统计 `< x` / `<= x` 的元素数量。
- `rank_of(L,R,x)`: 返回题目常见定义的排名, 即 `< x` 的个数 +1。
- `predecessor/successor`: 返回 `optional<i64>`; 不存在时为 `nullopt`, 由题目决定输出什么哨兵值。

```
// 维护的量: root (root[i]: 前缀 a[1..i] 的版本根)、tr (节点池, 0 号为空节点)、xs (排序去重后的原值, 离散编号 1..xs.size())。
// 不变量: root[R] 与 root[L-1] 对应节点相减, 即区间 [L,R] 的频次分布。
```

```
struct PersistentKth {
    struct Node {
        int l = 0, r = 0, sum = 0;
    };

    vector<Node> tr{Node{}}; // 0 号节点为空节点。
    vector<int> root;        // root[i]: 前缀 a[1..i] 的版本根。
    vector<i64> xs;          // 排序去重后的原值; 离散编号为 1..xs.size()。

    explicit PersistentKth(int reserve_nodes = 0) {
        if (reserve_nodes > 0) tr.reserve(reserve_nodes);
    }
};
```

```

}

// [内部] 复制旧节点, 返回新节点编号。
int clone_node(int p) {
    tr.push_back(tr[p]);
    return (int)tr.size() - 1;
}

// [内部] 在旧版本 p 上把离散位置 pos 的出现次数 +1, 返回新版本根。
int update_impl(int p, int l, int r, int pos) {
    int q = clone_node(p);
    ++tr[q].sum;
    if (l == r) return q;
    int mid = (l + r) >> 1;
    if (pos <= mid) tr[q].l = update_impl(tr[p].l, l, mid, pos);
    else tr[q].r = update_impl(tr[p].r, mid + 1, r, pos);
    return q;
}

// [内部] u=root[L-1], v=root[R]; l/r 是离散值域, 不是原数组区间。
// 返回第 k 小的“离散编号”, 外部通常不要直接调用。
int kth_impl(int u, int v, int l, int r, int k) const {
    if (l == r) return l;
    int left_count = tr[tr[v].l].sum - tr[tr[u].l].sum; // 两版本左儿子频次之差 = 区间内落在左半值域的个数
    int mid = (l + r) >> 1;
    if (k <= left_count) return kth_impl(tr[u].l, tr[v].l, l, mid, k);
    return kth_impl(tr[u].r, tr[v].r, mid + 1, r, k - left_count);
}

// [内部] 统计两个版本之差中, 离散编号 <= qr 的元素数量。
int count_prefix_impl(int u, int v, int l, int r, int qr) const {
    if (qr <= 0) return 0;
    if (r <= qr) return tr[v].sum - tr[u].sum;
    int mid = (l + r) >> 1;
    int ans = count_prefix_impl(tr[u].l, tr[v].l, l, mid, qr);
    if (qr > mid) ans += count_prefix_impl(tr[u].r, tr[v].r, mid + 1, r, qr);
    return ans;
}

// 外部建树接口: a 必须为 1-indexed, 真实数据是 a[1..n]。
void build_from_array(const vector<i64>& a) {
    assert(a.size() >= 2);
    xs.assign(a.begin() + 1, a.end());
    sort(xs.begin(), xs.end());
    xs.erase(unique(xs.begin(), xs.end()), xs.end());

    root.assign(a.size(), 0);
    tr.clear();
    tr.push_back(Node{});
    tr.reserve(max<size_t>(tr.capacity(), a.size() * 22));

    for (int i = 1; i < (int)a.size(); ++i) {
        int pos = int(lower_bound(xs.begin(), xs.end(), a[i]) - xs.begin()) + 1;
        root[i] = update_impl(root[i - 1], 1, (int)xs.size(), pos);
    }
}

// 外部查询: [L,R] 第 k 小的“实际值”; L/R/k 全部按题面常见的 1-indexed 使用。
i64 kth_value(int L, int R, int k) const {
    assert(1 <= L && L <= R && R < (int)root.size());
    assert(1 <= k && k <= R - L + 1);
    int id = kth_impl(root[L - 1], root[R], 1, (int)xs.size(), k);
    return xs[id - 1];
}

int count_le(int L, int R, i64 x) const {
    int id = int(upper_bound(xs.begin(), xs.end(), x) - xs.begin()); // <=x 的离散值个数
    if (id == 0) return 0;
    return count_prefix_impl(root[L - 1], root[R], 1, (int)xs.size(), id);
}

int count_lt(int L, int R, i64 x) const {
    int id = int(lower_bound(xs.begin(), xs.end(), x) - xs.begin()); // <x 的离散值个数
    if (id == 0) return 0;
    return count_prefix_impl(root[L - 1], root[R], 1, (int)xs.size(), id);
}

int rank_of(int L, int R, i64 x) const {
    return count_lt(L, R, x) + 1;
}

optional<i64> predecessor(int L, int R, i64 x) const {
    int c = count_lt(L, R, x);
    if (c == 0) return nullopt;
    return kth_value(L, R, c);
}

```

```

    }

    optional<i64> successor(int L, int R, i64 x) const {
        int c = count_le(L, R, x);
        if (c == R - L + 1) return nullopt;
        return kth_value(L, R, c + 1);
    }
};

```

能力边界：这是静态结构。若存在 `a[pos] = x`，不要尝试修改 `root[pos..n]`；直接换下一节动态模板。

动态区间顺序统计：值域线段树套 Fenwick (P3380 五操作)

赛时先看

题目信号：数组有单点修改，同时查询任意 $[L,R]$ 的排名、第 k 小、前驱、后继。

本质：外层按“值域”建线段树；每个值域节点内部用 Fenwick

统计这些值当前出现在哪些“位置”。修改沿值域路径更新，查询第 k 小直接按左右值域下降。

接法：必须先把所有操作读入，因为模板会预收集“某位置未来可能出现的值”来压缩各内部 Fenwick。然后 `build(a, ops)`，再按原顺序调用 `modify / rank_of / kth_value / predecessor / successor`。

复杂度判定：预处理、修改、排名、第 k 小均为 $O(\log M \log n)$ ；空间约 $O((n + \text{修改次数}) \log M)$ 。

维护的量：`value`（当前数组，1-indexed）；`xs`（所有可能出现的值，排序去重）；`coord/bit`（每个值域节点：可出现的位置集合 + 该集合上的 Fenwick）。

警告：本模板是“离线预处理坐标、在线执行操作”：答案仍按输入顺序实时输出，但 `build()`

前必须已经读完所有操作； k 从 1 开始；原数组和位置都是 1-indexed。

【最小完整示例（先抄这一段就能跑）】

题目：P3380 五操作： n 个数， m 次操作 = 区间排名 / 区间第 k 小 / 单点改值 / 前驱 / 后继。

```

using Op = DynamicRangeOrderStatistic::Operation;
vector<Op> ops(m);
for (auto &op : ops) {
    // 1. 先读完全部操作：build 要预收集所有 type=3 的新值
    cin >> op.type;
    if (op.type == 3) cin >> op.a >> op.b; // 单点改：pos, newValue
    else cin >> op.a >> op.b >> op.c; // 查询：L, R, x/k
}
DynamicRangeOrderStatistic ds;
ds.build(a, ops); // 2. 建树：a 必须 a[1..n], 1-indexed
for (auto op : ops) {
    if (op.type == 1) cout << ds.rank_of(op.a, op.b, op.c) << '\n';
    if (op.type == 2) cout << ds.kth_value(op.a, op.b, op.c) << '\n';
    if (op.type == 3) ds.modify(op.a, op.b); // 3. 单点赋值 a[pos] = x
    if (op.type == 4) cout << ds.predecessor(op.a, op.b, op.c) << '\n';
    if (op.type == 5) cout << ds.successor(op.a, op.b, op.c) << '\n';
}

```

样例：`a=[2,1,3]`；操作 `2 1 3 2` $\rightarrow$ 输出 `2`（区间 $[1,3]$ 第 2 小）。

【传参要求（照这个传不会错）】

- `build(a, ops)`：`a` 必须 1-indexed；`ops` 必须已读完全部操作（特别是 `type=3` 的未来新值）。
- `modify(pos, x)`：`pos` 是 1-indexed 位置；`x` 必须曾作为某个 `type=3` 操作传给 `build()`。
- `rank_of(L,R,x)`：`x` 用原值，返回 1-based 排名（ $<x$ 的个数 + 1）。
- `kth_value(L,R,k)`： k 从 1 开始（第 1 小传 1）； $1 \leq L \leq R \leq n$ ；返回原值。
- `predecessor/successor(L,R,x)`：返回原值；不存在时返回哨兵（默认 `-2147483647 / 2147483647`，与 P3380 一致）。
- 执行时在线输出答案，但结构必须离线预读，不能边读操作边 `build`。

【不会用就照抄（P3380 五操作）】

```

using Op = DynamicRangeOrderStatistic::Operation;
vector<Op> ops(m);

// 先把全部操作读完：build 必须提前知道未来所有 type=3 的新值
for (auto &op : ops) {
    cin >> op.type;
    if (op.type == 3) cin >> op.a >> op.b; // pos, newValue
    else cin >> op.a >> op.b >> op.c; // L, R, x/k
}

DynamicRangeOrderStatistic ds;
ds.build(a, ops); // a 必须是 a[1..n]

```

```
for (auto op : ops) {
    if (op.type == 1) cout << ds.rank_of(op.a, op.b, op.c) << '\n';
    if (op.type == 2) cout << ds.kth_value(op.a, op.b, op.c) << '\n';
    if (op.type == 3) ds.modify(op.a, op.b);
    if (op.type == 4) cout << ds.predecessor(op.a, op.b, op.c) << '\n';
    if (op.type == 5) cout << ds.successor(op.a, op.b, op.c) << '\n';
}
```

- 这份板子虽然“执行时在线”，但 build 前必须先读完全部操作。
- 位置和 [L,R] 都是 1-indexed; kth_value 的 k 也是 1-indexed。

【API / 入口函数（赛时只认这里列的名字）】

- ds.build(a,ops) -> 先读完全部操作再建结构; a 1-indexed。
- ds.modify(pos,x) -> 单点赋值 a[pos]=x。
- ds.rank_of(L,R,x) -> 区间内 x 的排名。
- ds.kth_value(L,R,k) -> 区间第 k 小, k 从 1 开始。
- ds.predecessor/successor(L,R,x) -> 严格前驱/后继; 无解时返回 P3380 哨兵。

【核心逻辑（改代码时别破坏）】

- 外层按值域二分; 每个值域节点内部 Fenwick 统计“哪些位置目前落在这个值域”。
- 单点修改只沿 $O(\log M)$ 个值域节点更新; 区间第 k 小在值域树上按 [L,R] 左儿子数量下降。

【改板时先认这几个量】

- value: 当前数组, 1-indexed。
- xs: 所有“可能真正出现在数组里”的值, 排序去重。
- bit: 每个值域节点: 可出现的位置 + 该位置集合上的 Fenwick。

为什么它比静态主席树多这一层: 静态主席树的 root[i] 是前缀版本, 改 a[pos] 会影响所有 root[pos..n]。这里改为按值域分层, 并在每个值域节点维护位置频次, 所以一次修改只动 $O(\log M)$ 个值域节点, 每个节点内部再做一次 $O(\log n)$ Fenwick 修改。

接口约定

- Operation{type,a,b,c}: 只用于预读; type=3 时 a=pos, b=newValue, 其余操作 a=L, b=R, c=x/k。
- build(a, ops): a[1..n] 为初始数组; ops 必须包含全部未来修改, 用来预收集坐标。
- modify(pos,newValue): 单点赋值。
- count_lt/count_le(L,R,x): 统计 $<x$ / $\leq x$ 。
- rank_of(L,R,x): 严格小于 x 的数量 + 1。
- kth_value(L,R,k): 第 k 小实际值, k 1-indexed。
- predecessor/successor: 直接返回值; 不存在时默认返回 P3380 要求的两个哨兵。

```
struct DynamicRangeOrderStatistic {
    struct Operation {
        int type;
        int a, b, c;
        // type=3: a=pos, b=newValue, c 未使用
        // 其他 : a=L, b=R, c=x 或 k
    };

    int n = 0, M = 0;
    vector<int> value; // 当前数组, 1-indexed。
    vector<int> xs; // 所有“可能真正出现在数组里”的值, 排序去重。
    vector<vector<int>> coord, bit; // 每个值域节点: 可出现的位置 + 该位置集合上的 Fenwick。

    static int lowbit(int x) { return x & -x; }

    // [内部] 在某个值域节点的局部 Fenwick 中, 把位置 pos 的计数 += delta。
    void local_add(int o, int pos, int delta) {
        auto &c = coord[o];
        auto &b = bit[o];
        int k = int(lower_bound(c.begin(), c.end(), pos) - c.begin()) + 1;
        for (int i = k; i < (int)b.size(); i += lowbit(i)) b[i] += delta;
    }

    // [内部] 某值域节点中, 位置 <= pos 的当前元素个数。
    int local_prefix(int o, int pos) const {
        if (o == 0 || coord[o].empty()) return 0;
        const auto &c = coord[o];
        const auto &b = bit[o];
        int k = int(upper_bound(c.begin(), c.end(), pos) - c.begin());
    }
};
```

```

int res = 0;
for (int i = k; i > 0; i -= lowbit(i)) res += b[i];
return res;
}

// [内部] 某值域节点中, 原数组位置落在 [L, R] 的元素个数。
int local_range(int o, int L, int R) const {
    if (L > R || o == 0) return 0;
    return local_prefix(o, R) - local_prefix(o, L - 1);
}

// [内部] 预处理: 声明 “pos 将来可能以 vid 这个值出现”。
void reserve_candidate(int o, int l, int r, int vid, int pos) {
    coord[o].push_back(pos);
    if (l == r) return;
    int mid = (l + r) >> 1;
    if (vid <= mid) reserve_candidate(o << 1, l, mid, vid, pos);
    else reserve_candidate(o << 1 | 1, mid + 1, r, vid, pos);
}

// [内部] 当前数组中, 在 (pos, vid) 这一个点上加/减一次出现次数。
void point_add(int o, int l, int r, int vid, int pos, int delta) {
    local_add(o, pos, delta);
    if (l == r) return;
    int mid = (l + r) >> 1;
    if (vid <= mid) point_add(o << 1, l, mid, vid, pos, delta);
    else point_add(o << 1 | 1, mid + 1, r, vid, pos, delta);
}

int id_exact(int x) const {
    return int(lower_bound(xs.begin(), xs.end(), x) - xs.begin()) + 1;
}

// 外部建树接口。
// a 必须 1-indexed; ops 必须已经读入全部操作, 至少要包含所有 type=3 的未来新值。
void build(const vector<int>& a, const vector<Operation>& ops) {
    n = (int)a.size() - 1;
    value = a;

    xs.assign(a.begin() + 1, a.end());
    for (const auto &op : ops) if (op.type == 3) xs.push_back(op.b);
    sort(xs.begin(), xs.end());
    xs.erase(unique(xs.begin(), xs.end()), xs.end());
    M = (int)xs.size();

    coord.assign(4 * M + 8, {});
    bit.assign(4 * M + 8, {});

    // 初值和未来修改值都要预先登记 “这个位置可能走过哪些值域节点”。
    for (int pos = 1; pos <= n; ++pos)
        reserve_candidate(1, 1, M, id_exact(a[pos]), pos);
    for (const auto &op : ops)
        if (op.type == 3)
            reserve_candidate(1, 1, M, id_exact(op.b), op.a);

    for (int o = 1; o < (int)coord.size(); ++o) {
        if (coord[o].empty()) continue;
        auto &c = coord[o];
        sort(c.begin(), c.end());
        c.erase(unique(c.begin(), c.end()), c.end());
        bit[o].assign(c.size() + 1, 0);
    }

    for (int pos = 1; pos <= n; ++pos)
        point_add(1, 1, M, id_exact(value[pos]), pos, +1);
}

// 外部: a[pos] = new_value。
// new_value 必须已作为某个 type=3 操作传给 build() 预处理过。
void modify(int pos, int new_value) {
    point_add(1, 1, M, id_exact(value[pos]), pos, -1);
    value[pos] = new_value;
    point_add(1, 1, M, id_exact(value[pos]), pos, +1);
}

// 外部: 统计 a[L..R] 中 < x 的个数。
int count_lt(int L, int R, int x) const {
    int t = int(lower_bound(xs.begin(), xs.end(), x) - xs.begin());
    if (t <= 0) return 0;
    if (t >= M) return R - L + 1;

    int o = 1, l = 1, r = M, ans = 0;
    while (l < r) {
        int mid = (l + r) >> 1;
        if (t <= mid) {

```

```

        o <= 1;
        r = mid;
    } else {
        ans += local_range(o < 1, L, R);
        o = o < 1 | 1;
        l = mid + 1;
    }
}
ans += local_range(o, L, R);
return ans;
}

// 外部: 统计 a[L..R] 中 <= x 的个数。
int count_le(int L, int R, int x) const {
    int t = int(upper_bound(xs.begin(), xs.end(), x) - xs.begin());
    if (t <= 0) return 0;
    if (t >= M) return R - L + 1;

    int o = 1, l = 1, r = M, ans = 0;
    while (l < r) {
        int mid = (l + r) >> 1;
        if (t <= mid) {
            o <= 1;
            r = mid;
        } else {
            ans += local_range(o < 1, L, R);
            o = o < 1 | 1;
            l = mid + 1;
        }
    }
    ans += local_range(o, L, R);
    return ans;
}

int rank_of(int L, int R, int x) const {
    return count_lt(L, R, x) + 1;
}

// 外部: a[L..R] 的第 k 小实际值; k 从 1 开始。
int kth_value(int L, int R, int k) const {
    assert(1 <= k && k <= R - L + 1);
    int o = 1, l = 1, r = M;
    while (l < r) {
        int mid = (l + r) >> 1;
        int left_count = local_range(o < 1, L, R);
        if (k <= left_count) {
            o <= 1;
            r = mid;
        } else {
            k -= left_count;
            o = o < 1 | 1;
            l = mid + 1;
        }
    }
    return xs[l - 1];
}

int predecessor(int L, int R, int x, int none_value = -2147483647) const {
    int c = count_lt(L, R, x);
    return c ? kth_value(L, R, c) : none_value;
}

int successor(int L, int R, int x, int none_value = 2147483647) const {
    int c = count_le(L, R, x);
    return c < R - L + 1 ? kth_value(L, R, c + 1) : none_value;
}
};

```

【P3380 最小接法（这一段可以直接照抄）】

```

vector<DynamicRangeOrderStatistic::Operation> ops;
// 先把 m 个操作全部读进 ops。

DynamicRangeOrderStatistic ds;
ds.build(a, ops);

for (auto op : ops) {
    if (op.type == 1) cout << ds.rank_of(op.a, op.b, op.c) << '\n';
    if (op.type == 2) cout << ds.kth_value(op.a, op.b, op.c) << '\n';
    if (op.type == 3) ds.modify(op.a, op.b);
    if (op.type == 4) cout << ds.predecessor(op.a, op.b, op.c) << '\n';
    if (op.type == 5) cout << ds.successor(op.a, op.b, op.c) << '\n';
}

```

这版的接口刻意和上一节静态主席树保持一致：遇到“静态/动态”的变化，只需要换数据结构，`rank_of / kth_value / predecessor / successor` 的调用习惯基本不变。

普通 01 Trie：最大异或对与最大子段异或

赛时先看

题目信号：题面出现 `xor / ^`、最大异或值、最大子数组异或值，数值范围在 32 位整数内。

本质：维护一批非负整数，查询和给定数异或后的最大值；前缀异或配合它可求最大子段异或和。

接法：给长度 `n` 的数组，求 `max(a[1] xor ... xor a[r])`。

复杂度判定：插入、查询都是 $O(\log V)$ ，这里为 31 层。

维护的量：`tr` (Trie 节点池，0 号为根)；每个节点 `ch[0]/ch[1]` (0/1 儿子编号) 与 `cnt` (子树内插入次数)。

警告：最大子段异或要先插入前缀异或 0；若数据可能使用第 31 位，改成 `unsigned` 并把 `LOG` 改为 31。

【最小完整示例（先抄这一段就能跑）】

题目：`n` 个数，求两两异或的最大值（最大异或对）。

```
BinaryTrie trie;
for (int x : a) trie.insert(x);           // 1. 先把所有数插进去
int ans = 0;
for (int x : a) ans = max(ans, trie.max_xor(x)); // 2. 对每个数查与集合的最大异或
```

样例：`a=[3,10,5,25,2,8]` -> 输出 28 (`5 xor 25`)。

【传参要求（照这个传不会错）】

- `insert(x)`：插入非负整数，可重复插；默认 `LOG=30`，覆盖 $0..2^{31}-1$ 。
- `max_xor(x)`：返回 `x` 与已插入集合中某个数的最大异或值；调用前先插入至少一个数。
- 最大子段异或：直接调用同节 `max_subarray_xor(a)`，`a` 是 0-indexed 的 `vector<int>`，内部已先插入前缀异或 0。
- 数据可能用到第 31 位：把 `LOG` 改成 31 并换 `unsigned` 存储。

【API / 入口函数（赛时只认这里列的名字）】

- `insert(int x)` -> 插入元素/字符串

```
struct BinaryTrie {
    static const int LOG = 30;
    struct Node {
        int ch[2] = {0, 0};
        int cnt = 0;
    };
    vector<Node> tr{Node{}};

    void insert(int x) {
        int p = 0;
        tr[p].cnt++;
        for (int b = LOG; b >= 0; --b) {
            int c = (x >> b) & 1;
            if (!tr[p].ch[c]) {
                tr[p].ch[c] = (int)tr.size();
                tr.push_back(Node{});
            }
            p = tr[p].ch[c];
            tr[p].cnt++;
        }
    }

    int max_xor(int x) const {
        int p = 0, ans = 0;
        for (int b = LOG; b >= 0; --b) {
            int c = (x >> b) & 1;
            int want = c ^ 1;
            if (tr[p].ch[want] && tr[tr[p].ch[want]].cnt) {
                ans |= 1 << b;
                p = tr[p].ch[want];
            } else {
                p = tr[p].ch[c];
            }
        }
        return ans;
    }
};

int max_subarray_xor(const vector<int>& a) {
    BinaryTrie trie;
    trie.insert(0);
```



```

int pre = 0, ans = 0;
for (int x : a) {
    pre ^= x;
    ans = max(ans, trie.max_xor(pre));
    trie.insert(pre);
}
return ans;
}

```

可持久化 01 Trie：区间最大异或

赛时先看

题目信号：区间最大 xor；数组静态；可转成前缀异或 `pre[i]`。

本质：静态数组前缀异或，查询区间 `[l,r]` 内某个数与 `x` 的最大异或。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定：插入/查询 $O(\log V)$ 。

维护的量：`tr`（节点池：`ch[0]/ch[1]` 儿子、`cnt` 出现次数）；`root`（版本根数组，`root[i]` 是前缀 `pre[1..i]` 的版本）。

警告：查询 `[l,r]` 的数时用两个版本相减；最大子段异或常查询前缀范围 `[l-1,r-1]`。

【最小完整示例（先抄这一段就能跑）】

题目：静态数组 `a[1..n]`，`m` 次询问 `[l,r]` 内某个数与 `x` 的最大异或。

```

PersistentBinaryTrie pt; // 1. 构造: root[0] 是空版本
for (int i = 1; i <= n; ++i) pt.root.push_back(pt.insert(pt.root.back(), a[i]));
int L, R, x;
cin >> L >> R >> x;
cout << pt.query_max_xor(pt.root[L - 1], pt.root[R], x) << '\n'; // 2. 两版本相减查区间

```

样例：`a=[3,10,5]`；问 `[1,3]` 与 `8` 的最大异或 $\rightarrow$ 输出 `13` (`5 xor 8`)。

【传参要求（照这个传不会错）】

- `PersistentBinaryTrie pt`：直接构造；`root` 初始为 `{0}`（空版本），节点数多可传 `PersistentBinaryTrie(n * 31)` 预留。
- `insert(old_root, x)`：在旧版本上插 `x`，返回新版本根；按前缀逐个插入后 `pt.root.push_back(返回值)`。
- `query_max_xor(left_root, right_root, x)`：区间 `[l,r]` 内的数与 `x` 的最大异或；`left_root = root[l-1]`、`right_root = root[r]`（版本相减），返回最大异或值。
- 下标 1-based：`root[i]` 是前缀 `pre[1..i]` 的版本。
- 最大子段异或：版本建在前缀数组 `pre[0..n]` 上，枚举右端点 `j` 时查前缀范围 `[l-1, j-1]`（原区间 `[l,r]` 对应下标范围 `[l-1,r-1]`）。

【API / 入口函数（赛时只认这里列的名字）】

- `insert(int old_root, int x)  $\rightarrow$  插入元素/字符串 返回 int。`

【改板时先认这几个量】

- `tr`：树节点池（节点数组）。
- `root`：版本根/树根数组。

```

struct PersistentBinaryTrie {
    static const int LOG = 30;
    struct Node { int ch[2] = {0, 0}; int cnt = 0; };
    vector<Node> tr;
    vector<int> root;

    PersistentBinaryTrie(int reserve_nodes = 1) {
        tr.reserve(reserve_nodes);
        tr.push_back(Node{});
        root.push_back(0);
    }

    int clone(int p) {
        tr.push_back(tr[p]);
        return (int)tr.size() - 1;
    }

    int insert(int old_root, int x) {
        int nr = clone(old_root);
        int p = nr;
        tr[p].cnt++;
    }
}

```

```

    for (int b = LOG; b >= 0; b--) {
        int c = (x >> b) & 1;
        int np = clone(tr[p].ch[c]);
        tr[p].ch[c] = np;
        p = np;
        tr[p].cnt++;
    }
    return nr;
}

int query_max_xor(int left_root, int right_root, int x) const {
    int ans = 0;
    int a = left_root, b = right_root;
    for (int bit = LOG; bit >= 0; bit--) {
        int c = (x >> bit) & 1;
        int want = c ^ 1;
        int cnt = tr[tr[b].ch[want]].cnt - tr[tr[a].ch[want]].cnt;
        if (cnt > 0) {
            ans |= 1 << bit;
            a = tr[a].ch[want];
            b = tr[b].ch[want];
        } else {
            a = tr[a].ch[c];
            b = tr[b].ch[c];
        }
    }
    return ans;
}
};

```

Treap 平衡树

赛时先看

题目信号：插入删除后还要查询排名/第 k 小，PBDS 不方便处理重复。

本质：动态维护排名、第 k 小、前驱后继，可处理重复值。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定：期望 $O(\log n)$ 。

维护的量：root（树根指针，insert/erase 用引用自更新）；节点

val（键值）、pri（随机优先级）、cnt（重复次数）、sz（子树总个数）。

警告：重复值用 cnt；旋转后要 pull。

【最小完整示例（先抄这一段就能跑）】

题目：动态集合：插入/删除 x ，查询 x 的排名与第 k 小（允许重复值）。

```

Treap tp;
tp.insert(tp.root, 5);           // 1. 插入 5（重复值自动累加 cnt）
tp.insert(tp.root, 3);
cout << tp.rank(tp.root, 5) << '\n'; // 2. 排名：返回 <= 5 的个数
cout << tp.kth(tp.root, 2) << '\n';  // 3. 第 k 小：k 从 1 开始
tp.erase(tp.root, 5);           // 4. 删除一个 5

```

样例：插 5、3 后 `rank(root,5)=2`, `kth(root,2)=5`；删 5 后集合只剩 3。

【传参要求（照这个传不会错）】

- `tp.insert(tp.root, v)`: v 是要插入的值；第一个参数固定传 `tp.root`（引用自更新）。
- `tp.erase(tp.root, v)`: 删一个 v （重复值 `cnt-1`）；值不存在则无事发生。
- `tp.rank(tp.root, v)`: 返回 $\leq v$ 的元素个数（含相等）；要“严格小于个数+1”式排名自己减/加调整。
- `tp.kth(tp.root, k)`: 第 k 小， k 从 1 开始； k 超出集合总大小返回 `INT_MIN`。
- `tp.size(tp.root)`: 集合总元素个数（含重复值）。

【API / 入口函数（赛时只认这里列的名字）】

- `erase(Node*& t, int v)` -> 删除元素
- `insert(Node*& t, int v)` -> 插入元素/字符串
- `size(Node* t)` -> 查询集合大小 返回 `int`。

【改板时先认这几个量】

- `sz`: 平衡树子树大小。
- `root`: 树根指针（insert/erase 用引用更新）。

```

struct Treap {
    struct Node {
        int val, pri, cnt = 1, sz = 1;
        Node *l = nullptr, *r = nullptr;
        Node(int v, int p) : val(v), pri(p) {}
    };

    mt19937 rng{71236721};
    Node* root = nullptr;

    int size(Node* t) const { return t ? t->sz : 0; }
    void pull(Node* t) { if (t) t->sz = t->cnt + size(t->l) + size(t->r); }

    void rotate_left(Node*& t) {
        Node* x = t->r;
        t->r = x->l;
        x->l = t;
        pull(t); pull(x);
        t = x;
    }

    void rotate_right(Node*& t) {
        Node* x = t->l;
        t->l = x->r;
        x->r = t;
        pull(t); pull(x);
        t = x;
    }

    void insert(Node*& t, int v) {
        if (!t) {
            t = new Node(v, (int)rng());
            return;
        }
        if (v == t->val) t->cnt++;
        else if (v < t->val) {
            insert(t->l, v);
            if (t->l->pri > t->pri) rotate_right(t);
        } else {
            insert(t->r, v);
            if (t->r->pri > t->pri) rotate_left(t);
        }
        pull(t);
    }

    void erase(Node*& t, int v) {
        if (!t) return;
        if (v == t->val) {
            if (t->cnt > 1) t->cnt--;
            else if (!t->l || !t->r) {
                Node* old = t;
                t = t->l ? t->l : t->r;
                delete old;
            } else if (t->l->pri > t->r->pri) {
                rotate_right(t);
                erase(t->r, v);
            } else {
                rotate_left(t);
                erase(t->l, v);
            }
        } else if (v < t->val) erase(t->l, v);
        else erase(t->r, v);
        pull(t);
    }

    // rank(t, v) 返回的是 <= v 的元素个数（含相等）；
    // 需要“严格小于的个数+1”式排名请自行减/加调整。
    int rank(Node* t, int v) const {
        if (!t) return 0;
        if (v <= t->val) return rank(t->l, v);
        return size(t->l) + t->cnt + rank(t->r, v);
    }

    int kth(Node* t, int k) const {
        if (!t) return INT_MIN;
        if (k <= size(t->l)) return kth(t->l, k);
        if (k <= size(t->l) + t->cnt) return t->val;
        return kth(t->r, k - size(t->l) - t->cnt);
    }
};

```

隐式 FHQ Treap: 序列插入与第 k 个

赛时先看

题目信号：题面有“在第 x 行/列后插入”“动态序列下标查询”； q 可到 $5e5$ ，数组中间插入不能直接 `vector`。

本质：维护动态序列，支持在第 pos 个元素后插入、删除、查询第 k 个元素。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定：期望 $O(\log n)$ 。

维护的量：`root`（当前序列根编号）；`tr`（节点池：1/r 儿子、`sz` 子树大小、`pri` 随机优先级、`val` 元素值）。

警告：`split(root, k, a, b)` 表示前 k 个进 `a`；插入到第 pos 个后就是按 `pos` 切开。

【最小完整示例（先抄这一段就能跑）】

题目：动态序列：在第 pos 个元素后插入 v 、删除第 pos 个、查询当前第 k 个元素。

```
ImplicitTreap it;
for (int i = 1; i <= n; ++i) it.insert_after(it.size(it.root), a[i]); // 1. 建初始序列 (0-based 位置)
it.insert_after(2, 9); // 2. 在第 2 个元素后插入 9
it.erase_at(3); // 3. 删除当前第 3 个元素 (1-based)
cout << it.kth(1) << '\n'; // 4. 查询当前第 1 个元素
```

样例：`a=[1,2,3]`；第 2 个后插 9 $\rightarrow$ `[1,2,9,3]`；删第 3 个 $\rightarrow$ `[1,2,3]`；`kth(1)` $\rightarrow$ 输出 1。

【传参要求（照这个传不会错）】

- `insert_after(pos, v)`: `pos` 是 0-based 计数（前 `pos` 个元素之后），可取 $0 \leq pos \leq$ 当前长度；末尾追加就传当前长度。
- `erase_at(pos)`: `pos` 是 1-based（当前序列第 `pos` 个元素）， $1 \leq pos \leq$ 当前长度。
- `kth(k)`: k 从 1 开始， $1 \leq k \leq$ 当前长度；越界返回 -1。
- `split(root, k, a, b)`: 前 k 个进 `a`，其余进 `b`；`merge(a, b)` 要求 `a` 序列整体在 `b` 之前。

【API / 入口函数（赛时只认这里列的名字）】

- `merge(int a, int b)` $\rightarrow$ 合并 返回 `int`。
- `size(int p)` $\rightarrow$ 查询集合大小 返回 `int`。

【改板时先认这几个量】

- `sz`: 平衡树子树大小。
- `tr`: 树节点池（节点数组）。
- `root`: 序列根指针（`split/merge` 用引用更新）。

```
struct ImplicitTreap {
    struct Node {
        int l = 0, r = 0, sz = 1, pri = 0;
        i64 val = 0;
    };
    vector<Node> tr[{}];
    mt19937 rng{(uint32_t)chrono::steady_clock::now().time_since_epoch().count()};
    int root = 0;

    int new_node(i64 v) {
        tr.push_back(Node{0, 0, 1, (int)rng(), v});
        return (int)tr.size() - 1;
    }

    int size(int p) const { return p ? tr[p].sz : 0; }

    void pull(int p) {
        if (p) tr[p].sz = size(tr[p].l) + size(tr[p].r) + 1;
    }

    void split(int p, int k, int& a, int& b) {
        if (!p) {
            a = b = 0;
            return;
        }
        if (size(tr[p].l) >= k) {
            b = p;
            split(tr[p].l, k, a, tr[p].l);
            pull(b);
        } else {
            a = p;
            split(tr[p].r, k - size(tr[p].l) - 1, tr[p].r, b);
            pull(a);
        }
    }

    int merge(int a, int b) {
        if (!a || !b) return a | b;
    }
};
```

```

    if (tr[a].pri < tr[b].pri) {
        tr[a].r = merge(tr[a].r, b);
        pull(a);
        return a;
    }
    tr[b].l = merge(a, tr[b].l);
    pull(b);
    return b;
}

// insert_after 的 pos 是“前 pos 个元素之后”（0-based 计数，pos 可取 0..当前长度）。
void insert_after(int pos, i64 v) {
    int a, b;
    split(root, pos, a, b);
    root = merge(merge(a, new_node(v)), b);
}

// erase_at 的 pos 是 1-based 位置（删除当前序列第 pos 个元素）。
void erase_at(int pos) {
    int a, b, c;
    split(root, pos - 1, a, b);
    split(b, 1, b, c);
    root = merge(a, c);
}

i64 kth(int k) const {
    int p = root;
    while (p) {
        int left_size = size(tr[p].l);
        if (k == left_size + 1) return tr[p].val;
        if (k <= left_size) p = tr[p].l;
        else {
            k -= left_size + 1;
            p = tr[p].r;
        }
    }
    return -1;
}
};

```

左偏树：可并堆

赛时先看

题目信号：多个集合各有堆，操作会合并堆并查询集合最小/最大。

本质：维护可合并优先队列，支持合并两个堆、取最小值、删除最小值。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定：合并/删除 $O(\log n)$ 均摊。

维护的量：`tr`（节点池：`l/r` 儿子、`val` 值、`dist` 到最近空节点的距离）；堆根编号即堆顶。

警告：删除节点后要合并它的左右儿子；如果节点已被删除，要先找到堆根。

【最小完整示例（先抄这一段就能跑）】

题目：n 个小根堆（每点自成一堆），支持合并两堆、取堆顶、删堆顶。

```

LeftistHeap h(n);
for (int i = 1; i <= n; ++i) cin >> h.tr[i].val; // 1. 每个点自成一堆（小根堆）
int r = 1;
r = h.merge(r, 2); // 2. 合并两堆，返回新堆根
cout << h.tr[r].val << '\n'; // 3. 堆顶即最小值
r = h.pop(r); // 4. 删堆顶，r 更新为新堆根

```

样例：`val(1)=5, val(2)=3` -> 合并后堆顶 `3`；`pop` 后堆顶 `5`。

【传参要求（照这个传不会错）】

- `LeftistHeap h(n)`：n 为节点上限；节点从 1 开始编号，直接给 `h.tr[i].val` 赋值（0 号节点为空）。
- `merge(x, y)`：合并两个堆，返回新堆根；x/y 传堆根编号，0 表示空堆。
- `pop(root)`：删堆顶（根），返回合并左右儿子后的新根；删某个节点 x 时先 `merge(h.tr[x].l, h.tr[x].r)` 再和所在堆合并，已删过的节点要先找到堆根。
- 小根堆按 `val` 小的在上；要大根堆把 `merge` 里的 `>` 改成 `<`。

【API / 入口函数（赛时只认这里列的名字）】

- `merge(int x, int y)` -> 合并 返回 `int`。

【改板时先认这几个量】

- **tr**: 堆节点池（节点数组）。
- **root**: 当前堆根节点编号（`pop(root)` 会更新它）。
- **dist**: 距离。

```
struct LeftistHeap {
    struct Node {
        int l = 0, r = 0, dist = 0;
        i64 val = 0;
    };
    vector<Node> tr;

    LeftistHeap(int n = 0) { tr.resize(n + 1); }

    int merge(int x, int y) {
        if (!x || !y) return x | y;
        if (tr[x].val > tr[y].val) swap(x, y); // 小根堆。
        tr[x].r = merge(tr[x].r, y);
        if (tr[tr[x].l].dist < tr[tr[x].r].dist) swap(tr[x].l, tr[x].r);
        tr[x].dist = tr[tr[x].r].dist + 1;
        return x;
    }

    int pop(int root) {
        return merge(tr[root].l, tr[root].r);
    }
};
```

Link-Cut Tree: 动态森林

赛时先看

题目信号: 树边动态加删; 查询两点路径 xor/sum/max; 要求在线。

本质: 维护动态森林路径信息, 支持 link、cut、路径查询/修改。

接法: 把本节 struct/class/函数组 整段抄到 `solve()` 上面; 外部只调用下面 API 列出的接口, 不要把内部递归参数直接当题面参数。

复杂度判定: 单次 $O(\log n)$ 均摊。

维护的量: **tr** (节点池: `ch[0]/ch[1]` 左右儿子、**fa** 父指针、**val** 点权、**xr** 当前 splay 内路径异或和、**rev** 翻转标记)。

警告: 所有访问前先 `makeroot`; `cut` 要确认确实有边; `pushdown` 顺序从祖先到当前。

【最小完整示例（先抄这一段就能跑）】

题目: n 个点的森林, 动态加边/删边, 随时询问两点路径上点权的异或和（在线）。

```
LinkCutTree lct(n); // 1. 构造即 init(n)
for (int i = 1; i <= n; ++i) cin >> lct.tr[i].val; // 2. 赋点权
lct.link(u, v); // 3. 加边 (u, v)
lct.cut(u, v); // 4. 删边 (u, v)
cout << lct.query_xor(x, y) << '\n'; // 5. 路径 x-y 的点权异或和
```

样例: 三个点 `val=1, 2, 3`, `link(1, 2)`、`link(2, 3)` 后 `query_xor(1, 3)` $\rightarrow$ 输出 `0` ($1^2^3=0$)。

【传参要求（照这个传不会错）】

- `LinkCutTree lct(n)`: 构造时自动 `init(n)`; 点编号 `1..n`。
- 改点权: 先 `lct.makeroot(x)`; `lct.access(x)`; `lct.splay(x)`; 再赋 `tr[x].val`, 最后 `lct.push_up(x)`。
- `link(x, y)`: 加边; 两点已连通则内部跳过。
- `cut(x, y)`: 删边; 内部判断确认 (x, y) 间确实有边才删。
- `query_xor(x, y)`: 返回路径 $x-y$ 上所有点权的异或和, 点编号 1-based 直接传。
- `makeroot(x)`: 把 x 变成所在树的根; `findroot(x)` 返回 x 所在树根, 判连通用 `findroot(a)==findroot(b)`。

【API / 入口函数（赛时只认这里列的名字）】

- `cut(int x, int y)` $\rightarrow$ 删除边 (x, y)
- `findroot(int x)` $\rightarrow$ 查询 x 所在树的根
- `init(int n)` $\rightarrow$ 初始化/清空结构
- `link(int x, int y)` $\rightarrow$ 连接边 (x, y)
- `makeroot(int x)` $\rightarrow$ 把 x 变为所在树的根
- `query_xor(int x, int y)` $\rightarrow$ 查询路径 $x-y$ 的异或和 返回 `int`。

【改板时先认这几个量】

- **tr**: 树节点池（节点数组）。
- **fa**: 父节点/并查集父亲。

```

struct LinkCutTree {
    struct Node {
        int ch[2] = {0, 0}, fa = 0;
        int val = 0, xr = 0;
        bool rev = false;
    };
    vector<Node> tr;

    LinkCutTree(int n = 0) { init(n); }
    void init(int n) { tr.assign(n + 1, Node{}); }

    bool is_root(int x) {
        int f = tr[x].fa;
        return !f || (tr[f].ch[0] != x && tr[f].ch[1] != x);
    }

    void push_up(int x) {
        tr[x].xr = tr[tr[x].ch[0]].xr ^ tr[x].val ^ tr[tr[x].ch[1]].xr;
    }

    void apply_rev(int x) {
        if (!x) return;
        swap(tr[x].ch[0], tr[x].ch[1]);
        tr[x].rev ^= 1;
    }

    void push_down(int x) {
        if (tr[x].rev) {
            apply_rev(tr[x].ch[0]);
            apply_rev(tr[x].ch[1]);
            tr[x].rev = false;
        }
    }

    void rotate(int x) {
        int y = tr[x].fa, z = tr[y].fa;
        int k = (tr[y].ch[1] == x), w = tr[x].ch[k ^ 1];
        if (!is_root(y)) tr[z].ch[tr[z].ch[1] == y] = x;
        tr[x].fa = z;
        tr[x].ch[k ^ 1] = y; tr[y].fa = x;
        tr[y].ch[k] = w; if (w) tr[w].fa = y;
        push_up(y); push_up(x);
    }

    void splay(int x) {
        static vector<int> st;
        st.clear();
        int y = x;
        st.push_back(y);
        while (!is_root(y)) {
            y = tr[y].fa;
            st.push_back(y);
        }
        while (!st.empty()) {
            push_down(st.back());
            st.pop_back();
        }
        while (!is_root(x)) {
            int y = tr[x].fa, z = tr[y].fa;
            if (!is_root(y)) {
                bool zigzig = (tr[y].ch[0] == x) == (tr[z].ch[0] == y);
                rotate(zigzig ? y : x);
            }
            rotate(x);
        }
    }

    void access(int x) {
        for (int y = 0; x; y = x, x = tr[x].fa) {
            splay(x);
            tr[x].ch[1] = y;
            push_up(x);
        }
    }

    void makeroot(int x) {
        access(x);
        splay(x);
        apply_rev(x);
    }
}

```



```

    }

    int findroot(int x) {
        access(x);
        splay(x);
        while (tr[x].ch[0]) {
            push_down(x);
            x = tr[x].ch[0];
        }
        splay(x);
        return x;
    }

    void link(int x, int y) {
        makeroot(x);
        if (findroot(y) != x) tr[x].fa = y;
    }

    void cut(int x, int y) {
        makeroot(x);
        access(y);
        splay(y);
        if (tr[y].ch[0] == x && !tr[x].ch[1]) {
            tr[y].ch[0] = tr[x].fa = 0;
            push_up(y);
        }
    }

    int query_xor(int x, int y) {
        makeroot(x);
        access(y);
        splay(y);
        return tr[y].xr;
    }
};

```

线性基

赛时先看

题目信号：题面出现 xor，要求选若干数使异或值最大/可达。

本质：异或最大值、判断某个数能否由子集异或得到。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定：插入/查询 $O(\log V)$ 。

维护的量：`b[i]`（最高位为 `i` 的基向量，0 表示该位为空）。

警告：LOG 要覆盖数据范围；i64 通常用 62。

【最小完整示例（先抄这一段就能跑）】

题目：给 `n` 个数，求任意子集异或的最大值，并判断某个数能否被子集异或表示。

```

LinearBasis lb;
for (int i = 0; i < n; ++i) lb.insert(a[i]); // 1. 全部插入（重复/线性相关会自动跳过）
cout << lb.max_xor() << '\n'; // 2. 子集异或最大值（空集视为 0）
cout << (lb.can_make(x) ? "YES" : "NO") << '\n'; // 3. 判断 x 可否由子集异或得到

```

样例：`a=[1,2,4]` $\rightarrow$ `max_xor()=7` (1^2^4)，`can_make(3)=true` (1^2)。

【传参要求（照这个传不会错）】

- `insert(x)`：插入一个数，返回 `bool` (`true`=真正放进基里，`false`=线性相关)；重复插入没关系。
- `max_xor(start=0)`：从 `start` 出发异或基向量能得到的最大值；直接 `max_xor()` 即全集合的最大子集异或。
- `can_make(x)`：判断 `x` 能否由已插入的子集异或得到（空集不算，除非 `x=0`）。
- 默认 `LOG=62` 覆盖 i64 全部 63 位；数据在 `int` 范围内可不动；超过 i64 本节不适用。

【API / 入口函数（赛时只认这里列的名字）】

- `insert(i64 x)` $\rightarrow$ 插入元素/字符串 返回 `bool`。

```

struct LinearBasis {
    static const int LOG = 62;
    i64 b[LOG + 1]{};

    bool insert(i64 x) {
        for (int i = LOG; i >= 0; --i) {
            if (((x >> i) & 1) == 0) continue;
            if (!b[i]) {

```

```

        b[i] = x;
        return true;
    }
    x ^= b[i];
}
return false;
}

i64 max_xor(i64 start = 0) const {
    i64 ans = start;
    for (int i = LOG; i >= 0; --i) ans = max(ans, ans ^ b[i]);
    return ans;
}

bool can_make(i64 x) const {
    for (int i = LOG; i >= 0; --i) {
        if (((x >> i) & 1) && b[i]) x ^= b[i];
    }
    return x == 0;
}
};

```

PBDS 有序树

赛时先看

题目信号：需要插入删除后仍查询排名/第 k 小。

本质：动态第 k 小、排名。

复杂度判定： $O(\log n)$ 。

维护的量：os（红黑树容器）；`tree_order_statistics_node_update` 自动维护子树大小，`order_of_key/find_by_order` 都依赖它。

警告：`tree` 不支持重复元素，重复可存 `{value, unique_id}`。

约定：`os.find_by_order(k)`：0-based 第 k 小的迭代器

【最小完整示例（先抄这一段就能跑）】

题目：动态插入/删除数字，问 x 的排名（严格小于的个数）与第 k 小（0-based）。

```

OrderedSet<int> os;
os.insert(5); os.insert(3); os.insert(8); // 1. 插入（重复键只存一份）
cout << os.order_of_key(5) << '\n'; // 2. 严格小于 5 的个数 = 1
auto it = os.find_by_order(2); // 3. 0-based 第 2 小 = 第 3 小
cout << (it == os.end() ? -1 : *it) << '\n'; // 4. 输出 8

```

样例：插 `{5,3,8}` → `order_of_key(5)=1`, `find_by_order(2)=8`。

【传参要求（照这个传不会错）】

- `os.insert(x)` / `os.erase(x)`：直接传值；重复键只存一份，要存重复值改用 `pair<value, unique_id>`（如 `{x, ++cnt}`）。
- `os.order_of_key(x)`：返回严格小于 x 的元素个数，即 0-based 排名；要 1-based 排名自己加 1。
- `os.find_by_order(k)`：0-based 第 k 小（第 1 小传 0）；越界返回 `os.end()`，取值前先判 `!= os.end()`。
- `*it` 取键值；迭代器可直接 `++/--` 找前后继。

【不会用就照抄】

```

OrderedSet<int> os;
os.insert(x);
int rank0 = os.order_of_key(x); // 严格小于 x 的数量
auto it = os.find_by_order(k); // 0-based 第 k 小；第 1 小传 k=0

```

- PBDS 原生 `tree<int,...>` 不保存重复值；要存重复值，用 `pair<value, unique_id>`。

【API / 入口函数（赛时只认这里列的名字）】

- `OrderedSet<T> os` → 建立不允许重复键的有序树。
- `os.order_of_key(x)` → 严格小于 x 的元素个数，也就是 0-based rank。
- `os.find_by_order(k)` → 返回 0-based 第 k 小的迭代器；第 1 小传 $k=0$ 。

```

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

template <class T>
using OrderedSet = tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update>;

```

```
// os.find_by_order(k): 0-based 第 k 小的迭代器
// os.order_of_key(x): 小于 x 的元素个数
```

CDQ 分治：三维偏序计数骨架

赛时先看

题目信号：三维偏序、动态逆序对、按时间分治处理贡献。

本质：统计每个点前面有多少点满足 $x \leq X, y \leq Y, z \leq Z$ 。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定： $O(n \log^2 n)$ （或 $O(n \log n)$ ，取决于实现）。

维护的量：`a[i]`（点： $x/y/z$ 三维 + id）；`ans[id]`（每个点的累计贡献）；`fw`（FenwickCompact，维护 z 维计数）。

警告：先按第一维排序；CDQ 中按第二维归并，用 BIT 维护第三维。

【最小完整示例（先抄这一段就能跑）】

题目： n 个三维点，统计每个点前面满足 $x \leq X, y \leq Y, z \leq Z$ 的点数（三维偏序）。

```
vector<Point3D> a(n); // 1. 读入 x,y,z 并赋 id=i
sort(a.begin(), a.end(), [](auto& p, auto& q) { return p.x < q.x; }); // 2. 按 x 排序
vector<i64> ans(n);
FenwickCompact<int> fw(m); // 3. z 压缩成 1..m 后当 BIT 下标
cdq_3d(a, 0, n, fw, ans); // 4. CDQ: 按 y 归并、BIT 维护 z
for (int i = 0; i < n; ++i) cout << ans[i] << '\n'; // 5. ans[id] 即该点的答案
```

样例：三点 $(1,1,1), (2,2,2), (1,2,2)$ (id 0,1,2) $\rightarrow$ `ans=[0,1,2]`。

【传参要求（照这个传不会错）】

- 调用前先把点按 x 升序排进 `a` (`a[0..n-1]`，区间 `[l,r)` 左闭右开)； z 先离散化成 `1..m`。
- `cdq_3d(a, 0, n, fw, ans)`：对全数组调用一次即可；`ans` 用 `a[i].id` 索引，需先 `resize(n)` 并清零。
- `fw` 用 `FenwickCompact<int>`，大小 `m` (z 的最大编号)，调用前 `fw.init(m)` 或新开一个。
- 骨架计数语义是 $x \leq X$ （同 x 也会计入）；题目要求“严格小于”时排序键用 (x,y,z) 并按同 x 分组处理。

【API / 入口函数（赛时只认这里列的名字）】

- `cdq_3d(a, l, r, fw, ans)  $\rightarrow$` 对 `a[l..r)` 做 CDQ 分治，贡献累加进 `ans`。
- 警告： z 坐标需要先离散化压缩成 `1..m` 才能直接作为 BIT 下标； x 维要先按第一维排序。

【改板时先认这几个量】

- `bit`: FenwickCompact 内部树状数组。
- `sum`: 区间和/计数和。

```
struct Point3D {
    int x, y, z, id;
    i64 ans = 0;
};

template <class T>
struct FenwickCompact {
    int n;
    vector<T> bit;
    FenwickCompact(int n = 0) { init(n); }
    void init(int n_) { n = n_; bit.assign(n + 1, T{}); }
    void add(int i, T v) { for (; i <= n; i += i & -i) bit[i] += v; }
    T sum(int i) const { T r{}; for (; i > 0; i -= i & -i) r += bit[i]; return r; }
};

void cdq_3d(vector<Point3D>& a, int l, int r, FenwickCompact<int>& fw, vector<i64>& ans) {
    if (r - l <= 1) return;
    int m = (l + r) >> 1;
    cdq_3d(a, l, m, fw, ans);
    cdq_3d(a, m, r, fw, ans);
    vector<Point3D> left(a.begin() + l, a.begin() + m);
    vector<Point3D> right(a.begin() + m, a.begin() + r);
    sort(left.begin(), left.end(), [](auto& p, auto& q) { return p.y < q.y; });
    sort(right.begin(), right.end(), [](auto& p, auto& q) { return p.y < q.y; });
    int i = 0;
    for (auto& p : right) {
        while (i < (int)left.size() && left[i].y <= p.y) {
            fw.add(left[i].z, 1);
            i++;
        }
        ans[p.id] += fw.sum(p.z);
    }
}
```

```
for (int j = 0; j < i; ++j) fw.add(left[j].z, -1);
}
```

整体二分 Parallel Binary Search

赛时先看

题目信号：第 k 小、最早满足条件时间、离线可加不可删。

本质：多询问，每个询问答案可二分，修改按时间/权值逐步加入。

复杂度判定： $O((n+q) \log V * \text{单次操作})$ 。

维护的量：updates（候选修改，按 value 决定加入）；queries（带 id 的询问）；ans（答案数组）；外加 apply/undo/check 三个回调与每层的 applied（本层已加修改，用于撤销）。

警告：每层递归要撤销当前加入的修改，或用可重置数据结构。

【最小完整示例（先抄这一段就能跑）】

题目：静态数组 $a[1..n]$ ， m 个询问 $[l, r]$ 内第 k 小（答案值域 $1..V$ ，整体二分）。

```
struct Update { int pos, value; }; // 每个元素 = 一个修改（在 pos 处加入一个 value）
struct Query { int l, r, k, id; };
FenwickCompact<int> fw(n); // 计数 BIT：可借本节 CDQ 的 FenwickCompact
auto apply = [&](Update u) { fw.add(u.pos, 1); };
auto undo = [&](Update u) { fw.add(u.pos, -1); };
auto check = [&](Query q) { return fw.sum(q.r) - fw.sum(q.l - 1) >= q.k; };
parallel_bs(1, V, updates, queries, ans, apply, undo, check); // ans[q.id] 即第 k 小
```

样例： $a=[1,5,2,4,3]$ ；询问 $[2,5]$ 第 2 小 $\rightarrow ans[0]=2$ （区间 $5, 2, 4, 3$ 中第 2 小是 2）。

【传参要求（照这个传不会错）】

- updates: 每个候选“修改”必须有 value 字段（代码按 $u.value \leq mid$ 判定是否加入），其余字段在 apply/undo 里用。
- queries: 每个询问必须有 id 字段（答案写进 $ans[q.id]$ ）；check(q) 返回“当前已加入的修改下，答案 $\leq mid$ 是否成立”。
- apply/undo: 必须成对且完全可逆；每层结束会撤销全部 applied，递归两半都拿完整 updates 重来。
- parallel_bs(l, r, updates, queries, ans, apply, undo, check): $[l, r]$ 为答案值域（闭区间），先按此调用一次；ans 需 resize 成询问个数。
- 无解询问（如区间元素不足 k 个）最后会落到 $ans=r$ ，题目要输出 -1 时自行判。

```
// 骨架：把询问按 mid 判定分到左/右。
// Update 和 Query 需要按题目自定义。
template <class Update, class Query, class Apply, class Undo, class Check>
void parallel_bs(int l, int r,
                vector<Update>& updates,
                vector<Query>& queries,
                vector<int>& ans,
                Apply apply,
                Undo undo,
                Check check) {
    if (queries.empty()) return;
    if (l == r) {
        for (auto& q : queries) ans[q.id] = l;
        return;
    }
    int mid = (l + r) >> 1;
    vector<Query> left, right;
    vector<Update> applied;
    for (auto& u : updates) {
        if (u.value <= mid) {
            apply(u);
            applied.push_back(u);
        }
    }
    for (auto& q : queries) {
        if (check(q)) left.push_back(q);
        else right.push_back(q);
    }
    for (auto it = applied.rbegin(); it != applied.rend(); ++it) undo(*it);
    parallel_bs(l, mid, updates, left, ans, apply, undo, check);
    parallel_bs(mid + 1, r, updates, right, ans, apply, undo, check);
}
```

离线动态连通性：线段树分治 + 可撤销并查集

赛时先看

题目信号：动态删边；在线做很难，但所有操作可提前读入。

本质：边会加入/删除，离线回答每个时刻连通性。

接法：先构造 `OfflineDynamicConnectivity odc(n, q)`；每条边按存在时间段 `odc.add_interval(1, r, u, v)`；最后 `odc.solve(1, 1, q, lambda)`，在 `lambda` 里用 `dsu.find(x) == dsu.find(y)` 回答该时刻的询问。

复杂度判定： $O((m \log q) \alpha(n))$ 。

维护的量：`seg`（时间线段树，每个节点存覆盖该时间段的边表）；`dsu`（可撤销并查集

`HistoryDSU: p/sz/hist/comps`）。

警告：把每条边的存在时间段插入时间线段树；递归退出时回滚 `DSU`。

【最小完整示例（先抄这一段就能跑）】

```
// n=3 个点, q=4 个时刻: 边 (1,2) 在时刻 1..3 存在, (2,3) 在时刻 2..4 存在。
OfflineDynamicConnectivity odc(3, 4);
odc.add_interval(1, 3, 1, 2);
odc.add_interval(2, 4, 2, 3);
vector<int> ans(5);
odc.solve(1, 1, 4, [&](int t, const HistoryDSU& dsu) {
    ans[t] = dsu.find(1) == dsu.find(3); // 时刻 t 时 1、3 是否连通
});
cout << ans[1] << ' ' << ans[2] << ' ' << ans[3] << ' ' << ans[4] << '\n'; // 0 1 1 0
```

- 样例：输出 `0 1 1 0`（时刻 2、3 时 1 和 3 连通）。

【传参要求（照这个传不会错）】

- `OfflineDynamicConnectivity(n, q)`：`n` = 点数（编号 `1..n`），`q` = 时刻总数（时间轴 `1..q`）。
- `add_interval(1, r, u, v)`：边 `(u,v)` 在时刻闭区间 `[1,r]` 内存在；调用顺序任意。
- `solve(1, 1, q, answer_leaf)`：必须从根 1、区间 `[1, q]` 开始；每个叶子时刻 `t` 会回调 `answer_leaf(t, dsu)`，在回调里回答询问（只读 `find`，不要改 `dsu`）。
- `HistoryDSU: find/snapshot/unite/rollback`：供 `solve` 内部使用，一般不在外面直接调。

【API / 入口函数（赛时只认这里列的名字）】

- `add_interval(int l, int r, int u, int v)` → 边 `(u,v)` 在时刻闭区间 `[1,r]` 内存在，加入时间线段树。
- `solve(1, 1, q, answer_leaf)` → 分治遍历时刻，在叶子时刻调用 `answer_leaf(t, dsu)` 回答该时刻的询问。

【改板时先认这几个量】

- `sz`：并查集按大小合并的大小数组。
- `dsu`：可撤销并查集实例（`HistoryDSU`）。

```
// 维护的量: p (父节点, 不路径压缩以支持回滚); sz (按大小合并的子树大小);
// hist (合并记录栈, rollback 时按逆序复原); comps (当前连通块数)。
// 不变量: hist.size() 就是当前“版本号”; rollback(snap) 后状态与 snapshot() 时完全一致。
struct HistoryDSU {
    vector<int> p, sz;
    vector<pair<int, int>> hist;
    int comps;
    HistoryDSU(int n = 0) { init(n); }
    void init(int n) { p.resize(n + 1); sz.assign(n + 1, 1); iota(p.begin(), p.end(), 0); hist.clear(); comps = n; }
    int find(int x) const { while (p[x] != x) x = p[x]; return x; }
    int snapshot() const { return (int)hist.size(); }
    bool unite(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) { hist.push_back({-1, -1}); return false; }
        if (sz[a] < sz[b]) swap(a, b);
        hist.push_back({b, sz[a]}); // 记录被挂到 a 下的 b 与 a 的原大小, 供回滚复原
        p[b] = a; sz[a] += sz[b]; comps--;
        return true;
    }
    void rollback(int snap) {
        while ((int)hist.size() > snap) {
            auto [b, old] = hist.back(); hist.pop_back();
            if (b == -1) continue;
            int a = p[b];
            p[b] = b; sz[a] = old; comps++; // 逆序撤销一次合并
        }
    }
};
```

```
// 维护的量: seg (时间线段树, seg[p] 存覆盖节点 p 时间段的全部边); dsu (可撤销并查集)。
// 不变量: solve 递归到节点 p 时, dsu 中恰好装着根到 p 路径上所有时间段内的边;
// 离开节点时 rollback, 保证兄弟子树互不影响。
struct OfflineDynamicConnectivity {
    int q;
    vector<vector<pair<int, int>>> seg;
```

```

HistoryDSU dsu;

OfflineDynamicConnectivity(int n, int q_) : q(q_), seg(4 * q_ + 4), dsu(n) {}

void add_interval_impl(int p, int l, int r, int ql, int qr, pair<int, int> e) {
    if (ql <= l && r <= qr) {
        seg[p].push_back(e); // 当前段完全被 [ql,qr] 覆盖：边挂在这个节点上
        return;
    }
    int mid = (l + r) >> 1;
    if (ql <= mid) add_interval_impl(p << 1, l, mid, ql, qr, e);
    if (qr > mid) add_interval_impl(p << 1 | 1, mid + 1, r, ql, qr, e);
}

// 外部入口：边 (u,v) 在时刻闭区间 [l,r] 内存在。
void add_interval(int l, int r, int u, int v) {
    add_interval_impl(1, 1, q, l, r, {u, v});
}

template <class AnswerLeaf>
void solve(int p, int l, int r, AnswerLeaf answer_leaf) {
    int snap = dsu.snapshot();
    for (auto [u, v] : seg[p]) dsu.unite(u, v); // 本节点覆盖时间段的边全部生效
    if (l == r) {
        answer_leaf(l, dsu); // 叶子时刻：回调回答询问
    } else {
        int mid = (l + r) >> 1;
        solve(p << 1, l, mid, answer_leaf);
        solve(p << 1 | 1, mid + 1, r, answer_leaf);
    }
    dsu.rollback(snap); // 递归退出时回滚，回到进入本节点前的状态
}
};

```

C 树、图论与网络流

05 树上问题

树题通常是图论、数据结构和 DP 的混合题。本章按直径、LCA、剖分、欧拉序、树 DP、点分治、虚树的比赛翻阅顺序整理。

树的直径

赛时先看

题目信号：树上最长路径、最远距离、通信延迟最大值。

本质：求树上最远两点距离。

接法：树上最长路径直接 `tree_diameter(g)`。无权树把每条边权设为

1；带权树用真实边权。若题目还要输出直径端点，保留两次 `farthest` 的返回点即可。

复杂度判定： $O(n)$ 。

维护的量：`g`（邻接表，边权 $\{v, w\}$ ）、`dist`（起点到各点距离）。

警告：带权树要累加边权；两次 DFS/BFS。

约定：顶点需从 1 开始编号；否则把起点改成实际第一个点。

【最小完整示例（先抄这一段就能跑）】

题目：`n` 个点的带权树（顶点从 1 开始编号），求直径长度。

```

vector<vector<pair<int, i64>>> g(n + 1); // 1. 邻接表：顶点从 1 开始编号
for (int i = 1; i < n; ++i) { int u, v; i64 w; cin >> u >> v >> w; g[u].push_back({v, w}); g[v].push_back({u, w}); } // 2
. 加无向带权边
cout << tree_diameter(g) << '\n'; // 3. 直接调用，返回直径长度

```

样例：链 1-2-3，边权 1, 2；直径 $\rightarrow$ 3。

【传参要求（照这个传不会错）】

- `g`：邻接表，`g[u]` 存 $\{v, w\}$ ；顶点从 1 开始编号（函数用 `g.size()-1` 当点数）。
- `tree_diameter(g)`：返回 `i64` 直径长度；内部自动做两次 `farthest`，无需 `build`。
- `farthest(s, g)`：返回 `pair<int, i64>`：first 是离 `s` 最远的点，second 是最远距离；要输出直径端点就保留这两次返回值。

- 无权树把所有边权传 `1` 即可。

【改板时先认这几个量】

- `g`: 邻接表。
- `dist`: 距离。

```
pair<int, i64> farthest(int s, const vector<vector<pair<int, i64>>>& g) {
    int n = (int)g.size() - 1;
    vector<i64> dist(n + 1, -1);
    queue<int> q;
    q.push(s);
    dist[s] = 0;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (auto [v, w] : g[u]) {
            if (dist[v] != -1) continue;
            dist[v] = dist[u] + w;
            q.push(v);
        }
    }
    int id = s;
    for (int i = 1; i <= n; ++i) if (dist[i] > dist[id]) id = i;
    return {id, dist[id]};
}

i64 tree_diameter(const vector<vector<pair<int, i64>>>& g) {
    auto [a, _] = farthest(1, g);
    auto [b, diam] = farthest(a, g);
    return diam;
}
```

LCA: 倍增

赛时先看

题目信号: 树不修改，反复问“两点路径/最近公共祖先/距离/向上跳 k 步”。看到“多次树上路径、祖先、距离询问”，先想 LCA 倍增。

本质: 用倍增表把“沿树向上走”压缩成 $O(\log n)$ 次跳跃: `up[k][u]` 预存 u 的 2^k 级祖先，任意祖先关系与路径询问按二进制位逐段跳拼出来。

复杂度判定: 预处理 $O(n \log n)$ ，查询 $O(\log n)$ ； n, q 到 $2e5$ 轻松过。若树静态且询问可全量离线，Tarjan 离线 LCA 更省一个 $\log$ ；树会改则换树链剖分。

维护的量: `depth`（深度）、`up`（倍增祖先表）、`dist`（根到点的带权距离）。

接法: 先 `LCA solver(n)`，每条无向边 `add_edge(u,v,w)`，所有边加完 `build(root)` 一次。问最近公共祖先调用 `lca(u,v)`；问距离调用 `distance(u,v)`；问向上跳调用 `jump(u,k)`。如果边无权， w 默认用 `1`，距离就是边数。

警告: 根的父亲设成根 (`dfs(root, root)`)，防止倍增跳出树外；`LOG` 要覆盖 n （模板已自动算好，别手改）；`build` 前不要查询。

【最小完整示例（先抄这一段就能跑）】

题目: n 个点的树， q 次询问两点最近公共祖先（LCA）和距离。

```
LCA solver(n); // 1. 结构体定义: LCA(点数 n)
for (int i = 1; i < n; ++i) {
    int u, v;
    cin >> u >> v;
    solver.add_edge(u, v); // 2. 加边 (无向边加一次即可)
}
solver.build(1); // 3. 建树: 根从 1 开始; 必须先 build 再查询
while (q--) {
    int u, v;
    cin >> u >> v;
    cout << solver.lca(u, v) << '\n'; // 最近公共祖先
    cout << solver.distance(u, v) << '\n'; // 树上距离 (边数)
}
```

样例: 链 `1-2-3`; `lca(1,3) -> 1`; `distance(1,3) -> 2`。

【传参要求（照这个传不会错）】

- `LCA(n)`: 构造: 点编号 `1..n`。
- `add_edge(u, v, w=1)`: 加无向边; 边带权时传 w (`distance` 按带权距离算)。
- `build(root)`: 所有边加完后调用一次; `root` 是树根; `build` 前查询是错的。
- `lca(u, v)`: 返回最近公共祖先 (`int`)。

- `distance(u, v)`: 返回 `u` 到 `v` 的带权距离 (i64)。
- `jump(u, k)`: 从 `u` 向上跳 `k` 步, 越界返回 0。
- 边数很多时 LOG 模板已自动算好, 别手改。

【不会用就照抄】

```
LCA solver(n);
solver.add_edge(u, v); // 每条树边都加
solver.build(root); // 所有边加完以后只 build 一次
cout << solver.lca(u, v) << '\n';
```

- 顺序固定: 构造 $\rightarrow$ add_edge $\rightarrow$ build(root) $\rightarrow$ lca()。
- 节点通常是 1..n; build 前不要查询。

【API / 入口函数 (赛时只认这里列的名字)】

- `LCA solver(n)` $\rightarrow$ 初始化 1..n 的树。
- `solver.add_edge(u,v,w)` $\rightarrow$ 加无向边; 无权树可省略 `w`。
- `solver.build(root)` $\rightarrow$ 所有边加入后预处理; 查询前必须调用。
- `solver.lca(u,v)` $\rightarrow$ 最近公共祖先。
- `solver.jump(u,k)` $\rightarrow$ 从 `u` 向上跳 `k` 条边。
- `solver.distance(u,v)` $\rightarrow$ 按边权求两点距离。

【抄板清单 (照着做就行)】

1. 抄哪段: 整个 `struct LCA` (含 `dfs/build/jump/lca/distance`)。
2. 构造: `LCA solver(n);`, `n` 是树的点数。
3. 加边: 每条树边 `solver.add_edge(u, v, w);`, 无权树可省略 `w` (默认 1)。
4. 预处理: 所有边加完后 `solver.build(root);`, 只调用一次。
5. 查询: `solver.lca(u, v)` / `solver.distance(u, v)` / `solver.jump(u, k)`。
6. 取结果: 返回值直接用; 距离用 `i64` 接收。

【改造点 (按题目改这几处)】

- 根的选择: 不指定时默认 `root = 1`; 题面要求“以 `x` 为根”就把 `x` 传进 `build`。
- LOG 大小: 模板按 `n` 自动算好, 不用改; 只有 `n` 极大 ($1e6+$) 时留意 ($1 << \text{LOG}$) 别溢出。
- 边权 `w`: 无权树省略 (默认 1, 距离即边数); 带权树传真实边权, `distance` 自动返回带权和。
- 编号: 模板按 1..n; 题给 0..n-1 就全部下标 +1 再用。
- 只问“`u` 是否为 `v` 的祖先”: 判断 `lca(u,v) == u` 即可。

【核心逻辑 (改代码时别破坏)】

- `up[k][u]` 是 `u` 的 2^k 级祖先。
- 先把两点拉到同深度, 再从大到小尝试一起跳, 最后父亲就是 LCA。

【改板时先认这几个量】

- `g`: 邻接表。
- `up`: 倍增祖先表。
- `depth`: 深度。
- `dist`: 距离。

// 维护的量: `depth[u]` = `u` 的深度; `up[k][u]` = `u` 向上 2^k 步的祖先; `dist[u]` = 根到 `u` 的带权距离。
 // 不变量: `depth + up` 支撑 `lca` 的“拉平再齐跳”; `dist` 供 `distance` 用前缀差求路径长。

```
struct LCA {
    int n, LOG;
    vector<vector<pair<int, i64>>> g;
    vector<vector<int>> up;
    vector<int> depth;
    vector<i64> dist;

    LCA(int n = 0) { if (n) init(n); }

    void init(int n_) {
        n = n_;
        LOG = 1;
        while ((1 << LOG) <= n) LOG++; // LOG 满足  $2^{\text{LOG}} > n$ , 覆盖所有跳跃步长
        g.assign(n + 1, {});
        up.assign(LOG, vector<int>(n + 1));
```

```

    depth.assign(n + 1, 0);
    dist.assign(n + 1, 0);
}

void add_edge(int u, int v, i64 w = 1) {
    g[u].push_back({v, w});
    g[v].push_back({u, w});
}

void dfs(int u, int p) {
    up[0][u] = p;
    for (int k = 1; k < LOG; ++k) up[k][u] = up[k - 1][up[k - 1][u]]; // 2^k 祖先 = 2^(k-1) 祖先的 2^(k-1) 祖先
    for (auto [v, w] : g[u]) {
        if (v == p) continue;
        depth[v] = depth[u] + 1;
        dist[v] = dist[u] + w; // 累加根到点的带权距离, 供 distance 查询
        dfs(v, u);
    }
}

void build(int root = 1) {
    dfs(root, root); // 根的父亲设成根自己, 防止倍增跳出树外
}

int jump(int u, int steps) const {
    for (int k = 0; k < LOG; ++k) {
        if ((steps >> k) & 1) u = up[k][u]; // steps 按二进制位拆成若干次 2^k 跳
    }
    return u;
}

int lca(int a, int b) const {
    if (depth[a] < depth[b]) swap(a, b);
    a = jump(a, depth[a] - depth[b]); // 先把深的拉到与浅的同一深度
    if (a == b) return a;
    for (int k = LOG - 1; k >= 0; --k) {
        if (up[k][a] != up[k][b]) { // 从大到小试跳, 跳后仍不同才跳, 保证不越过 LCA
            a = up[k][a];
            b = up[k][b];
        }
    }
    return up[0][a]; // 两指针的父亲就是 LCA
}

i64 distance(int a, int b) const {
    int c = lca(a, b);
    return dist[a] + dist[b] - 2 * dist[c]; // 路径长 = 两段根距离之和减两倍 LCA 距离
}
};

```

Tarjan 离线 LCA：并查集批量求最近公共祖先

赛时先看

题目信号：树不修改、询问可以全部先读入、只需要最终一次性输出； n, q 很大，想避免倍增的 $O(q \log n)$ ，或题目本身就是 DFS 回溯场景。

本质：一棵静态树（或森林）上的所有 LCA 询问都已提前读入时，以并查集离线求每一对点的最近公共祖先。

接法：给一棵 $n \leq 5e5$ 的静态树和 q 个“求 u, v 的 LCA”询问，所有询问先输入、后统一输出。对边调用 `add_edge`，每个询问保存 `id = add_query(u, v)`，`auto ans = solver.solve(root)` 后输出 `ans[id]`。如果题目混有在线改根、修改边或动态查询，则回到倍增、树链剖分或 LCT，不能用这份离线板子。

复杂度判定： $O((n + q) \alpha(n))$ ，其中 α 是反阿克曼函数；空间 $O(n + q)$ 。

维护的量：`g`（邻接表）、`query[u]`（挂在 u 上的询问 {另一端, 编号}）、`answer`（答案数组）、`dsu/ancestor`（并查集与代表元祖先）。

警告：这是离线算法，必须先 `add_query` 再 `solve`；并查集的合并发生在“孩子子树 DFS 完成”之后；无根森林里不同连通块的询问答案保持 `-1`；深链很深时要留意递归栈。

【最小完整示例（先抄这一段就能跑）】

题目： n 个点的树， q 个“求 u, v 的 LCA”询问全部先读入、最后统一输出。

```

TarjanOfflineLCA solver(n); // 1. 结构体定义: TarjanOfflineLCA(点数 n)
for (int i = 1; i < n; ++i) { int u, v; cin >> u >> v; solver.add_edge(u, v); } // 2. 加无向树边
vector<int> id(q);
for (int i = 0; i < q; ++i) { int u, v; cin >> u >> v; id[i] = solver.add_query(u, v); } // 3. 询问先登记
vector<int> ans = solver.solve(1); // 4. 登记完再 solve, 一次性出全部答案
for (int i = 0; i < q; ++i) cout << ans[id[i]] << '\n'; // 5. 按编号取答案

```

样例：链 1-2-3；询问 $(1, 3) \rightarrow 1$ ； $(2, 3) \rightarrow 2$ 。

【传参要求（照这个传不会错）】

- `TarjanOfflineLCA(n)`: 构造; 点编号 `1..n`。
- `add_edge(u, v)`: 加无向边。
- `add_query(u, v)`: 登记一条询问, 返回编号 `id` (从 0 开始); 必须先 `add_query` 再 `solve`。
- `solve(root=1)`: 全部 `add_query` 后调用一次; 剩余点自动当森林; 返回 `vector<int>`, `ans[id]` 是该询问的 LCA, 不同连通块为 `-1`。
- 在线查询不能用这份板子 (倍增 LCA 才支持在线)。

【API / 入口函数（赛时只认这里列的名字）】

- `add_edge(int u, int v)` -> 加入一条边
- `find(int x)` -> 查代表元/查找结果 返回 `int`。
- `solve(int root = 1)` -> 优先选 `root`; 剩余点也允许组成森林。 返回 `vector<int>`。

【改板时先认这几个量】

- `g`: 邻接表。
- `query`: {另一个端点, 询问编号}。

```
struct TarjanOfflineLCA {
    int n;
    vector<vector<int>>> g;
    vector<vector<pair<int, int>>>> query; // {另一个端点, 询问编号}。
    vector<int> dsu, ancestor, component, answer;
    vector<char> done;

    explicit TarjanOfflineLCA(int n_)
        : n(n_), g(n_ + 1), query(n_ + 1) {}

    void add_edge(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    int add_query(int u, int v) {
        int id = (int)answer.size();
        answer.push_back(-1);
        query[u].push_back({v, id});
        query[v].push_back({u, id});
        return id;
    }

    int find(int x) { return dsu[x] == x ? x : dsu[x] = find(dsu[x]); }

    void dfs(int u, int parent, int component_id) {
        dsu[u] = u;
        ancestor[u] = u;
        component[u] = component_id;
        for (int v : g[u]) {
            if (v == parent) continue;
            dfs(v, u, component_id);
            dsu[find(v)] = u; // 把已经处理完的儿子集合合并到 u 的集合中。
            ancestor[find(u)] = u; // u 所在集合的代表元对应 u 的答案祖先。
        }
        done[u] = 1;
        for (auto [v, id] : query[u]) {
            if (done[v] && component[v] == component[u]) {
                answer[id] = ancestor[find(v)];
            }
        }
    }

    // 优先选 root; 剩余点也允许组成森林。
    vector<int> solve(int root = 1) {
        dsu.resize(n + 1);
        iota(dsu.begin(), dsu.end(), 0);
        ancestor.assign(n + 1, 0);
        component.assign(n + 1, -1);
        done.assign(n + 1, 0);
        fill(answer.begin(), answer.end(), -1);

        int component_id = 0;
        if (1 <= root && root <= n) dfs(root, 0, component_id++);
        for (int u = 1; u <= n; ++u) {
            if (!done[u]) dfs(u, 0, component_id++);
        }
        return answer;
    }
}
```

};

典题模型：给一棵 $n \leq 5e5$ 的静态树和 q 个“求 u, v 的 LCA”询问，所有询问先输入、后统一输出。对边调用 `add_edge`，每个询问保存 `id = add_query(u, v)`，`auto ans = solver.solve(root)` 后输出 `ans[id]`。如果题目混有在线改根、修改边或动态查询，则回到倍增、树链剖分或 LCT，不能用这份离线板子。

长链剖分： $O(1)$ 第 k 级祖先

赛时先看

题目信号：题目专门大量问“第 k 个祖先/向上跳 k 步”， q 很大，树不修改；或需要在复杂树上 DP 中频繁取祖先。普通 LCA、只问少量祖先时优先使用更直观的倍增。

本质：在静态有根树上反复询问节点 u 向父亲走 k 条边后的祖先。相比倍增的 $O(\log n)$ ，预处理后单次查询 $O(1)$ 。

接法：给定根为 1 的树， $q \leq 1e6$ 次询问 (u, k) ，要求第 k 级祖先。建边后 `solver.build(1)`，每次输出 `solver.kth_ancestor(u, k)`；如果题目把根编号看作自己的祖先，遇到返回 0 时按题意改成根或 -1。

复杂度判定：预处理 $O(n \log n)$ ，空间 $O(n \log n)$ （倍增表），每次合法查询 $O(1)$ 。

维护的量：`parent/depth`（父亲与深度）、`up`（倍增祖先表）、`heavy/top`（长链重儿子与链顶）、`down/above`（长链向下节点与链顶上方祖先）。

警告： $k=0$ 答案就是自身； $k > \text{depth}[u]$ 时本模板返回

0；长链按“向下最长”儿子选择，不是重链剖分按子树大小选择；根的父亲为 0。

【最小完整示例（先抄这一段就能跑）】

题目：根为 1 的树， q 次询问 (u, k) 求第 k 级祖先。

```
LongChainKthAncestor solver(n);           // 1. 结构体定义: LongChainKthAncestor(点数 n)
for (int i = 1; i < n; ++i) { int u, v; cin >> u >> v; solver.add_edge(u, v); } // 2. 加无向边
solver.build(1);                           // 3. 建树: 根从 1 开始; 必须先 build 再查询
while (q--) { int u, k; cin >> u >> k; cout << solver.kth_ancestor(u, k) << '\n'; } // 4. 第 k 级祖先
```

样例：链 1-2-3-4; `kth_ancestor(4, 2) -> 2`; `kth_ancestor(2, 5) -> 0`。

【传参要求（照这个传不会错）】

- `LongChainKthAncestor(n)`：构造；点编号 $1..n$ 。
- `add_edge(u, v)`：加无向边。
- `build(root=1)`：所有边加完后调用一次；`root` 是树根；`build` 前查询是错的。
- `kth_ancestor(u, k)`：返回 u 的第 k 级祖先（int）； $k=0$ 返回自身，越过根返回 0（越界情况按题意自行改）。
- 只问少量祖先且 q 不大时，直接用更直观的倍增 LCA 的 `jump` 也行。

【API / 入口函数（赛时只认这里列的名字）】

- `add_edge(int u, int v)` -> 加入一条边
- `build(int root = 1)` -> 完成建树或预处理
- `init(int n_)` -> 初始化/清空结构
- `kth_ancestor(int u, int k)` -> 如果第 k 级祖先越过根节点，返回 0。

【改板时先认这几个量】

- `up`：倍增祖先表。
- `parent`：树上父节点。
- `heavy`：长链重儿子。
- `top`：长链链顶。
- `g`：邻接表。

```
struct LongChainKthAncestor {
    int n, LOG;
    vector<vector<int>> g, up;
    vector<int> parent, depth, longest, heavy, top;
    vector<vector<int>> down, above;

    explicit LongChainKthAncestor(int n_ = 0) {
        if (n_) init(n_);
    }

    void init(int n_) {
        n = n_;
        LOG = 1;
        while ((1 << LOG) <= n) ++LOG;
        g.assign(n + 1, {});
    }
};
```

```

        up.assign(LOG, vector<int>(n + 1, 0));
        parent.assign(n + 1, 0);
        depth.assign(n + 1, 0);
        longest.assign(n + 1, 0);
        heavy.assign(n + 1, 0);
        top.assign(n + 1, 0);
        down.assign(n + 1, {});
        above.assign(n + 1, {});
    }

    void add_edge(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void dfs1(int u, int p) {
        parent[u] = p;
        up[0][u] = p;
        for (int j = 1; j < LOG; ++j) up[j][u] = up[j - 1][up[j - 1][u]];
        longest[u] = 1;
        heavy[u] = 0;
        for (int v : g[u]) {
            if (v == p) continue;
            depth[v] = depth[u] + 1;
            dfs1(v, u);
            if (longest[v] > longest[heavy[u]]) heavy[u] = v;
        }
        longest[u] += longest[heavy[u]];
    }

    void dfs2(int u, int chain_top) {
        top[u] = chain_top;
        if (u == chain_top) {
            // 长链上的节点, 按从浅到深排列。
            for (int x = u; x; x = heavy[x]) down[chain_top].push_back(x);
            // O(1) 查询公式最多只会用到这么多级祖先。
            for (int x = parent[u]; x && (int)above[chain_top].size() < longest[u]; x = parent[x]) {
                above[chain_top].push_back(x);
            }
        }
        if (heavy[u]) dfs2(heavy[u], chain_top);
        for (int v : g[u]) {
            if (v != parent[u] && v != heavy[u]) dfs2(v, v);
        }
    }

    void build(int root = 1) {
        depth[root] = 0;
        dfs1(root, 0);
        dfs2(root, root);
    }

    // 如果第 k 级祖先越过根节点, 返回 0。
    int kth_ancestor(int u, int k) const {
        if (k < 0 || k > depth[u]) return 0;
        if (k == 0) return u;
        int bit = 31 - __builtin_clz(k);
        u = up[bit][u];
        k -= 1 << bit;
        if (k == 0) return u;

        int chain_top = top[u];
        int inside = depth[u] - depth[chain_top];
        if (k <= inside) return down[chain_top][inside - k];
        return above[chain_top][k - inside - 1];
    }
};

```

典题模型：给定根为 1 的树， $q \leq 1e6$ 次询问 (u, k) ，要求第 k 级祖先。建边后 `solver.build(1)`，每次输出 `solver.kth_ancestor(u, k)`；如果题目把根编号看作自己的祖先，遇到返回 0 时按题意改成根或 -1。

树链剖分 HLD

赛时先看

题目信号：多次修改或查询树上两点路径，不只是求 LCA。

本质：树上路径查询/修改，配合线段树。

接法：HLD 本身只负责把树上路径拆成若干段，真正维护和/最大值/赋值的是外面的线段树。通常先按 `dfn[u]` 把点权放进数组建线段树；路径查询时循环跳重链，每段查询 `[dfn[top], dfn[u]]`；子树查询直接查 `[dfn[u], dfn[u] + sz[u] - 1]`。

复杂度判定：预处理 $O(n)$ ，每条路径拆成 $O(\log n)$ 段。

维护的量: `parent/depth` (父亲与深度)、`sz` (子树大小)、`heavy` (重儿子)、`top` (链顶)、`dfn/rid` (DFS 序与逆映射)。

警告: `dfn` 后子树是连续区间; 路径段交给线段树处理。

【最小完整示例 (先抄这一段就能跑)】

题目: n 个点的树, 若干次“路径上每个点加 x ”, 最后问点 u 的值 (区间加/单点查)。

```
HLD hld(n); // 1. 结构体定义: HLD(点数 n)
for (int i = 1; i < n; ++i) { int u, v; cin >> u >> v; hld.add_edge(u, v); } // 2. 加无向边
hld.build(1); // 3. 建树: 根从 1 开始; 必须先 build 再操作
vector<i64> diff(n + 2); // 4. 点权按 dfn 存的差分数组 (区间加/单点查)
auto add_path = [&](int u, int v, i64 x) { // 5. 路径每个点 +x
    hld.for_path(u, v, [&](int l, int r) { diff[l] += x; diff[r + 1] -= x; }); // 6. 路径拆成 O(log n) 段 dfn 区间
};
for (int i = 1; i <= n; ++i) diff[i] += diff[i - 1]; // 7. 前缀和还原点权
cout << diff[hld.dfn[u]] << '\n'; // 8. 点 u 当前值
```

样例: 链 1-2-3; 路径 (1,3) 加 5 后, 点 2 的值 $\rightarrow 5$ 。

【传参要求 (照这个传不会错)】

- `HLD(n)`: 构造; 点编号 `1..n`。
- `add_edge(u, v)`: 加无向边。
- `build(root=1)`: 所有边加完后调用一次; 必须先 `build`, `dfn/sz` 才有值。
- `for_path(a, b, visit)`: 把路径 `a-b` (含两 endpoints) 拆成 $O(\log n)$ 段, 每段回调 `visit(l, r)`, `[l, r]` 是 `dfn` 区间。
- `subtree_range(u)`: 返回 `{dfn[u], dfn[u]+sz[u]-1}`, 子树是连续区间, 直接交给线段树。
- `lca(a, b)`: 返回最近公共祖先 (`int`)。
- 点权按 `dfn[u]` 放进线段树数组; `dfn/rid` 是公开成员。

【API / 入口函数 (赛时只认这里列的名字)】

- `add_edge(int u, int v)` $\rightarrow$ 加入一条边
- `build(int root = 1)` $\rightarrow$ 完成建树或预处理
- `init(int n_)` $\rightarrow$ 初始化/清空结构
- `lca(int a, int b)` $\rightarrow$ 最近公共祖先 返回 `int`。

【改板时先认这几个量】

- `g`: 邻接表。
- `parent`: 树上父节点。
- `sz`: 子树大小。
- `dfn`: DFS 序时间戳。
- `depth`: 深度。

```
struct HLD {
    int n, timer = 0;
    vector<vector<int>> g;
    vector<int> parent, depth, heavy, sz, top, dfn, rid;

    HLD(int n = 0) { init(n); }

    void init(int n_) {
        n = n_;
        g.assign(n + 1, {});
        parent.assign(n + 1, 0);
        depth.assign(n + 1, 0);
        heavy.assign(n + 1, 0);
        sz.assign(n + 1, 0);
        top.assign(n + 1, 0);
        dfn.assign(n + 1, 0);
        rid.assign(n + 1, 0);
        timer = 0;
    }

    void add_edge(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void dfs1(int u, int p) {
        parent[u] = p;
```

```

    sz[u] = 1;
    heavy[u] = 0;
    for (int v : g[u]) {
        if (v == p) continue;
        depth[v] = depth[u] + 1;
        dfs1(v, u);
        sz[u] += sz[v];
        if (!heavy[u] || sz[v] > sz[heavy[u]]) heavy[u] = v;
    }
}

void dfs2(int u, int tp) {
    top[u] = tp;
    dfn[u] = ++timer;
    rid[timer] = u;
    if (heavy[u]) dfs2(heavy[u], tp);
    for (int v : g[u]) {
        if (v != parent[u] && v != heavy[u]) dfs2(v, v);
    }
}

void build(int root = 1) {
    depth[root] = 0;
    dfs1(root, 0);
    dfs2(root, root);
}

int lca(int a, int b) const {
    while (top[a] != top[b]) {
        if (depth[top[a]] < depth[top[b]]) b = parent[top[b]];
        else a = parent[top[a]];
    }
    return depth[a] < depth[b] ? a : b;
}

template <class F>
void for_path(int a, int b, F visit) const {
    while (top[a] != top[b]) {
        if (depth[top[a]] < depth[top[b]]) swap(a, b);
        visit(dfn[top[a]], dfn[a]);
        a = parent[top[a]];
    }
    if (depth[a] > depth[b]) swap(a, b);
    visit(dfn[a], dfn[b]);
}

pair<int, int> subtree_range(int u) const {
    return {dfn[u], dfn[u] + sz[u] - 1};
}
};

```

树上主席树：路径点权第 k 小

赛时先看

题目信号：树上多次问“两点路径中第 k 小/有多少值不超过 x ”，点权或边权固定， n, q 常到 $2e5$ 。经典模型是 SPOJ COT - Count on a tree。

本质：点权静态、不修改的树上路径顺序统计。询问 $u-v$ 路径上所有节点权值的第 k 小；边权问题把一条边的权放在较深端节点即可。

接法：每个节点有整数权值，不修改； q 次询问 (u, v, k) 求路径上的第 k 小点权。读取权值到 1-indexed 数组 a ，调用 `solver.set_values(a)`、`solver.build(1)`，答案是 `solver.kth_value(u, v, k)`。要问“路径上不超过 x 的点数”时，可在同样四个版本的线段树上做前缀计数。

复杂度判定：离散化后预处理 $O(n \log n)$ ，每次路径第 k 小 $O(\log n)$ ，空间 $O(n \log n)$ 。

维护的量：`values/pos`（点权与离散化下标）、`root[u]`（根到 u 路径的主席树版本）、`up/depth`（倍增祖先与深度）、`tr`（线段树节点池）。

警告：本模板统计的是路径两端都包含的节点；组合版本时是 `root[u] + root[v] - root[lca] - root[parent(lca)]`，不能减两次 `root[lca]`； k 从 1 开始；权值相同会被离散到同一位置，返回原始权值。

约定： a 使用 1-indexed， $a[1]..a[n]$ 是各点点权

【最小完整示例（先抄这一段就能跑）】

题目： n 个点、点权固定不修改； q 次询问 (u, v, k) 求路径上点权第 k 小。

```

TreePathKth solver(n); // 1. 结构体定义：TreePathKth(点数 n)
for (int i = 1; i < n; ++i) { int u, v; cin >> u >> v; solver.add_edge(u, v); } // 2. 加无向边
solver.set_values(a); // 3. 传 1-indexed 点权数组 a[1..n]
solver.build(1); // 4. 建树：根从 1 开始；必须先 build 再查询
while (q--) { int u, v, k; cin >> u >> v >> k; cout << solver.kth_value(u, v, k) << '\n'; } // 5. 路径第 k 小

```


样例：链 1-2-3，点权 {3,1,2}；`kth_value(1,3,2)` $\rightarrow$ 2。

【传参要求（照这个传不会错）】

- `TreePathKth(n)`：构造；点编号 `1..n`。
- `add_edge(u, v)`：加无向边。
- `set_values(a)`：传 1-indexed 点权数组 `a[1..n]`；内部自动离散化；必须在 build 之前调用。
- `build(tree_root=1)`：`set_values` 且加完边后调用一次；build 前查询是错的。
- `kth_value(u, v, k)`：返回 `u-v` 路径上第 `k` 小的原始权值（`i64`）；`k` 从 1 开始；路径含两 endpoint。
- 边权题：把每条边权放到较深端节点即可。

【API / 入口函数（赛时只认这里列的名字）】

- `set_values(const vector<i64>& a)` $\rightarrow$ 读入 1-indexed 点权 `a[1..n]`。
- `build(int tree_root = 1)` $\rightarrow$ 建树/预处理；查询前必须调用。
- `kth_value(int u, int v, int k)` $\rightarrow$ 查询 `u-v` 路径第 `k` 小的实际值，`k` 从 1 开始。返回 `i64`。

【改板时先认这几个量】

- `up`：倍增祖先表。
- `tr`：树节点池（节点数组）。

```
struct TreePathKth {
    struct Node {
        int left = 0, right = 0, sum = 0;
    };

    int n, LOG, value_count;
    vector<vector<int>> g, up;
    vector<int> depth, root, pos;
    vector<i64> values, coordinate;
    vector<Node> tr;

    explicit TreePathKth(int n_ = 0) {
        if (n_) init(n_);
    }

    void init(int n_) {
        n = n_;
        LOG = 1;
        while ((1 << LOG) <= n) ++LOG;
        g.assign(n + 1, {});
        up.assign(LOG, vector<int>(n + 1, 0));
        depth.assign(n + 1, 0);
        root.assign(n + 1, 0);
        pos.assign(n + 1, 0);
        values.assign(n + 1, 0);
        coordinate.clear();
        tr.assign(1, {}); // 0 号版本表示空版本。
    }

    void add_edge(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    // a 使用 1-indexed, a[1]..a[n] 是各点点权。
    void set_values(const vector<i64>& a) {
        values = a;
        coordinate.assign(a.begin() + 1, a.end());
        sort(coordinate.begin(), coordinate.end());
        coordinate.erase(unique(coordinate.begin(), coordinate.end()), coordinate.end());
        value_count = (int)coordinate.size();
        for (int u = 1; u <= n; ++u) {
            pos[u] = (int)(lower_bound(coordinate.begin(), coordinate.end(), values[u]) - coordinate.begin()) + 1;
        }
        tr.assign(1, {});
        tr.reserve((n + 5) * (LOG + 1));
    }

    int insert(int previous, int l, int r, int x) {
        int current = (int)tr.size();
        tr.push_back(tr[previous]);
        ++tr[current].sum;
        if (l == r) return current;
        int mid = (l + r) >> 1;
        if (x <= mid) tr[current].left = insert(tr[previous].left, l, mid, x);
        else tr[current].right = insert(tr[previous].right, mid + 1, r, x);
    }
};
```

```

    return current;
}

void dfs(int u, int p) {
    up[0][u] = p;
    for (int j = 1; j < LOG; ++j) up[j][u] = up[j - 1][up[j - 1][u]];
    root[u] = insert(root[p], 1, value_count, pos[u]);
    for (int v : g[u]) {
        if (v == p) continue;
        depth[v] = depth[u] + 1;
        dfs(v, u);
    }
}

void build(int tree_root = 1) {
    assert(value_count > 0);
    depth[tree_root] = 0;
    dfs(tree_root, 0);
}

int lca(int u, int v) const {
    if (depth[u] < depth[v]) swap(u, v);
    int gap = depth[u] - depth[v];
    for (int j = 0; j < LOG; ++j) if (gap >> j & 1) u = up[j][u];
    if (u == v) return u;
    for (int j = LOG - 1; j >= 0; --j) {
        if (up[j][u] != up[j][v]) u = up[j][u], v = up[j][v];
    }
    return up[0][u];
}

int kth_index(int ru, int rv, int rlca, int rparent, int k) const {
    int l = 1, r = value_count;
    while (l < r) {
        int left_count = tr[tr[ru].left].sum + tr[tr[rv].left].sum
            - tr[tr[rlca].left].sum - tr[tr[rparent].left].sum;
        int mid = (l + r) >> 1;
        if (k <= left_count) {
            ru = tr[ru].left;
            rv = tr[rv].left;
            rlca = tr[rlca].left;
            rparent = tr[rparent].left;
            r = mid;
        } else {
            k -= left_count;
            ru = tr[ru].right;
            rv = tr[rv].right;
            rlca = tr[rlca].right;
            rparent = tr[rparent].right;
            l = mid + 1;
        }
    }
    return l;
}

i64 kth_value(int u, int v, int k) const {
    int w = lca(u, v);
    int pw = up[0][w];
    int count = tr[root[u]].sum + tr[root[v]].sum - tr[root[w]].sum - tr[root[pw]].sum;
    assert(1 <= k && k <= count);
    return coordinate[kth_index(root[u], root[v], root[w], root[pw], k) - 1];
}
};

```

典题模型：每个节点有整数权值，不修改； q 次询问 (u, v, k) 求路径上的第 k 小点权。读取权值到 1-indexed 数组 a ，调用 `solver.set_values(a)`、`solver.build(1)`，答案是 `solver.kth_value(u, v, k)`。要问“路径上不超过 x 的点数”时，可在同样四个版本的线段树上做前缀计数。

Euler 序 + 树状数组：子树加、单点查

赛时先看

题目信号：操作只针对“某个节点的整个子树”，不涉及任意两点路径。

本质：树上子树整体加值，查询某个点当前值。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；外部只调用下面 API

列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定：DFS $O(n)$ ，每次操作 $O(\log n)$ 。

维护的量：`tin/tout`（DFS 进出时间戳）、`fw`（差分树状数组，下标为时间戳）。

警告：DFS 序中一个子树是连续区间 $[tin[u], tout[u]]$ ；必须先 dfs 再操作，否则 `tin/tout` 全 0。

【最小完整示例（先抄这一段就能跑）】

题目： n 个点的树， q 次操作：子树整体加 x / 问点 u 当前值。

```
SubtreeAddPointQuery solver(n);           // 1. 结构体定义：SubtreeAddPointQuery(点数 n)
for (int i = 1; i < n; ++i) { int u, v; cin >> u >> v; solver.add_edge(u, v); } // 2. 加无向边
solver.dfs(1, 0);                          // 3. 必须先 dfs 出 tin/tout 再操作
solver.subtree_add(2, 5);                  // 4. 点 2 的整个子树 +5
cout << solver.point_query(3) << '\n';    // 5. 点 3 当前值
```

样例：链 1-2-3; subtree_add(2,5) 后 point_query(3) $\rightarrow$ 5。

【传参要求（照这个传不会错）】

- SubtreeAddPointQuery(n): 构造；点编号 $1..n$ 。
- add_edge(u, v): 加无向边。
- dfs($root, 0$): 必须最先调用一次（如 dfs(1, 0)），否则 tin/tout 全 0。
- subtree_add(u, val): 给点 u 的整个子树加 val （内部对 $[tin[u], tout[u]]$ 区间加）。
- point_query(u): 返回点 u 当前值（i64），即 $tin[u]$ 处前缀和。

【API / 入口函数（赛时只认这里列的名字）】

- dfs($root$) $\rightarrow$ 必须先调用，否则 tin/tout 全 0。
- subtree_add(u, val) $\rightarrow$ 给 u 的子树整体加 val 。
- point_query(x) $\rightarrow$ 查询点 x 的当前值。

【改板时先认这几个量】

- bit: Fenwick 内部树状数组。
- g: 邻接表。

```
template <class T>
struct Fenwick {
    int n;
    vector<T> bit;
    Fenwick(int n = 0) { init(n); }
    void init(int n_) { n = n_; bit.assign(n + 1, T{}); }
    void add(int i, T v) { for (; i <= n; i += i & -i) bit[i] += v; }
    T sum(int i) const { T r{}; for (; i > 0; i -= i & -i) r += bit[i]; return r; }
};

struct SubtreeAddPointQuery {
    int n, timer = 0;
    vector<vector<int>> g;
    vector<int> tin, tout;
    Fenwick<i64> fw;

    SubtreeAddPointQuery(int n = 0) { init(n); }

    void init(int n_) {
        n = n_;
        timer = 0;
        g.assign(n + 1, {});
        tin.assign(n + 1, 0);
        tout.assign(n + 1, 0);
        fw.init(n + 2);
    }

    void add_edge(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void dfs(int u, int p) {
        tin[u] = ++timer;
        for (int v : g[u]) if (v != p) dfs(v, u);
        tout[u] = timer;
    }

    void subtree_add(int u, i64 val) {
        fw.add(tin[u], val);
        fw.add(tout[u] + 1, -val);
    }

    i64 point_query(int u) const {
        return fw.sum(tin[u]);
    }
};
```

Euler 序 + 树状数组：子树和查询

赛时先看

题目信号：树上“子树和”，单点权值会变化。

本质：点权修改，查询某个子树的权值和。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定：DFS $O(n)$ ，每次操作 $O(\log n)$ 。

维护的量：`tin/tout`（DFS 进出时间戳）、`val[u]`（点 `u` 当前权值）、`fw`（按时间戳存点权的树状数组）。

警告：把点 `u` 的权值放在 `tin[u]` 位置，子树就是连续区间；必须先 dfs 再操作，否则 `tin/tout` 全 0。

【最小完整示例（先抄这一段就能跑）】

题目：n 个点带权的树，q 次操作：改点权 / 问子树点权和。

```
SubtreeSum solver(n); // 1. 结构体定义：SubtreeSum(点数 n)
for (int i = 1; i < n; ++i) { int u, v; cin >> u >> v; solver.add_edge(u, v); } // 2. 加无向边
solver.dfs(1, 0); // 3. 必须先 dfs (dfs 时把初始点权放进 BIT)
solver.set_value(2, 10); // 4. 点 2 权值改为 10 (必须在 dfs 之后)
solver.set_value(3, 4);
cout << solver.subtree_sum(1) << '\n'; // 5. 点 1 子树点权和
```

样例：链 1-2-3，初始全 0；改点权为 10, 4 后 `subtree_sum(1)` -> 14。

【传参要求（照这个传不会错）】

- `SubtreeSum(n)`：构造；点编号 1..n。
- `add_edge(u, v)`：加无向边。
- `dfs(root, 0)`：必须最先调用（如 `dfs(1, 0)`），dfs 时按当前 `val` 初始化 BIT；之后才能改权/查询。
- `set_value(u, v)`：把点 `u` 权值改为 `v`；必须在 dfs 之后调用。
- `subtree_sum(u)`：返回点 `u` 子树所有点权和（i64）。

【API / 入口函数（赛时只认这里列的名字）】

- `dfs(root)` -> 必须先调用，否则 `tin/tout` 全 0。
- `set_value(x, v)` -> 把点 `x` 的权值设为 `v`；必须在 `dfs()` 之后调用，因为树状数组在 `dfs` 中初始化。
- `subtree_sum(x)` -> 查询点 `x` 的子树的点权和。

【改板时先认这几个量】

- `bit`：Fenwick 内部树状数组。
- `g`：邻接表。

// 与上一节的 Fenwick 同名功能重复，这里改名避免跨节照抄冲突。

```
template <class T>
struct FenwickRangeSum {
    int n;
    vector<T> bit;
    FenwickRangeSum(int n = 0) { init(n); }
    void init(int n_) { n = n_; bit.assign(n + 1, T{}); }
    void add(int i, T v) { for (; i <= n; i += i & -i) bit[i] += v; }
    T sum(int i) const { T r{}; for (; i > 0; i -= i & -i) r += bit[i]; return r; }
    T range_sum(int l, int r) const { return sum(r) - sum(l - 1); }
};

struct SubtreeSum {
    int n, timer = 0;
    vector<vector<int>> g;
    vector<int> tin, tout;
    FenwickRangeSum<i64> fw;
    vector<i64> val;

    SubtreeSum(int n = 0) { init(n); }

    void init(int n_) {
        n = n_;
        timer = 0;
        g.assign(n + 1, {});
        tin.assign(n + 1, 0);
        tout.assign(n + 1, 0);
        val.assign(n + 1, 0);
        fw.init(n + 2);
    }
};
```

```

void add_edge(int u, int v) {
    g[u].push_back(v);
    g[v].push_back(u);
}

void dfs(int u, int p) {
    tin[u] = ++timer;
    fw.add(tin[u], val[u]);
    for (int v : g[u]) if (v != p) dfs(v, u);
    tout[u] = timer;
}

void set_value(int u, i64 new_val) {
    fw.add(tin[u], new_val - val[u]);
    val[u] = new_val;
}

i64 subtree_sum(int u) const {
    return fw.range_sum(tin[u], tout[u]);
}
};

```

树上差分：路径加，点统计

赛时先看

题目信号：很多路径 $[u, v]$ ，最后统一问点/边贡献。

本质：多次给树上路径加 1，最后统计每个点被多少条路径经过。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；优先调用下面 API 列出的入口，其余 helper 默认不要从 `solve()` 直接调。

复杂度判定： $O((n+q) \log n)$ ，最后 DFS 汇总。

维护的量：diff（差分数组，1-indexed）、g（邻接表）、up（倍增祖先表，借 LCA 的公开成员）。

警告：点差分：`cnt[u]++ cnt[v]++ cnt[lca]-- cnt[parent[lca]]--`。

【最小完整示例（先抄这一段就能跑）】

题目：n 个点的树，q 条路径，每条路径上的点计数 +1，最后输出每个点的覆盖次数。

```

LCA lca_solver(n); // 1. 建 LCA 倍增（提供 lca 与 up）
vector<vector<int>> g(n + 1); // 2. 另存一份邻接表供 DFS 汇总
for (int i = 1; i < n; ++i) { int u, v; cin >> u >> v; lca_solver.add_edge(u, v); g[u].push_back(v); g[v].push_back(u); }
lca_solver.build(1); // 3. 必须先 build 再差分
vector<i64> diff(n + 1);
for (int i = 0; i < q; ++i) { int u, v; cin >> u >> v; add_path_point(u, v, diff, lca_solver); } // 4. 每条路径一次点差分
collect_tree_diff(1, 0, g, diff); // 5. 自底向上汇总，diff[u] = u 的覆盖次数

```

样例：链 1-2-3；路径 (1,3) 加一次 $\rightarrow$ `diff[2]` $\rightarrow$ 1。

【传参要求（照这个传不会错）】

- `add_path_point(u, v, diff, solver)`：对路径 $u-v$ 做点差分（内部用 `solver.lca(u,v)` 与 `solver.up[0][lca]`）；diff 是 1-indexed 的 `vector<i64>`。
- `collect_tree_diff(u, p, g, diff)`：从根 DFS 汇总子节点差分到父亲；g 必须是 `vector<vector<int>>` 邻接表；汇总完 `diff[u]` 就是点 u 被覆盖次数。
- 必须先把 LCA build 好再调用 `add_path_point`；up 是 LCA 的公开成员。
- 边统计版：`diff[u]++ diff[v]++ diff[lca] -= 2`，最后汇总边贡献。

【API / 入口函数（赛时只认这里列的名字）】

- `add_path_point(int u, int v, vector<i64>& diff, const LCA& solver)` $\rightarrow$ 需要 LCA 的 `lca(u,v)` 和 `parent[0][x]`。

【改板时先认这几个量】

- diff：差分数组（最后 DFS 汇总）。
- g：邻接表。
- up：倍增祖先表。

```

// 需要 LCA 的 lca(u,v) 和 parent[0][x]。
void add_path_point(int u, int v, vector<i64>& diff, const LCA& solver) {
    int w = solver.lca(u, v);
    diff[u]++;
    diff[v]++;
    diff[w]--;
}

```

```

    int pw = solver.up[0][w];
    if (pw != w) diff[pw]--;
}

void collect_tree_diff(int u, int p, const vector<vector<int>>& g, vector<i64>& diff) {
    for (int v : g[u]) {
        if (v == p) continue;
        collect_tree_diff(v, u, g, diff);
        diff[u] += diff[v];
    }
}

```

树上路径交

赛时先看

题目信号：多条树上路径之间的公共部分。

本质：判断两条树上路径是否相交，或求路径交的端点辅助。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；优先调用下面 API 列出的入口，其余 helper 默认不要从 `solve()` 直接调。

复杂度判定：使用 LCA 后 $O(\log n)$ 。

维护的量：无自有成员；只用 LCA 的 `lca` 与 `distance`。

警告：需要一个 `is_on_path(x,a,b)` 判断点是否在路径上。

【最小完整示例（先抄这一段就能跑）】

题目：n 个点的树，q 组询问：路径 a-b 与路径 c-d 是否相交。

```

LCA solver(n); // 1. 建 LCA 倍增：构造 + 加边
for (int i = 1; i < n; ++i) { int u, v; cin >> u >> v; solver.add_edge(u, v); } // 2. 加无向边
solver.build(1); // 3. 必须先 build 再查询
while (q--) { int a, b, c, d; cin >> a >> b >> c >> d;
    cout << (path_intersect(solver, a, b, c, d) ? "YES" : "NO") << '\n'; } // 4. 两路径是否相交

```

样例：链 1-2-3-4；(1,2) 与 (3,4) → 不相交；(1,3) 与 (2,4) → 相交。

【传参要求（照这个传不会错）】

- `path_intersect(solver, a, b, c, d)`：判断路径 a-b 与 c-d 是否相交，返回 `bool`；内部先取两条路径的 LCA 再判断。
- `is_on_path(solver, x, a, b)`：判断点 x 是否在路径 a-b 上（距离等式 $\text{dist}(a,x)+\text{dist}(x,b)=\text{dist}(a,b)$ ），返回 `bool`。
- `solver` 必须先 build；无权树 `distance` 就是边数。

【API / 入口函数（赛时只认这里列的名字）】

- `is_on_path(const LCA& solver, int x, int a, int b)` → 依赖 LCA 结构中的 `lca(a,b)` 和 `distance(a,b)`。返回 `bool`。

```

// 依赖 LCA 结构中的 lca(a,b) 和 distance(a,b)。
bool is_on_path(const LCA& solver, int x, int a, int b) {
    return solver.distance(a, x) + solver.distance(x, b) == solver.distance(a, b);
}

bool path_intersect(const LCA& solver, int a, int b, int c, int d) {
    int x = solver.lca(a, b);
    int y = solver.lca(c, d);
    return is_on_path(solver, x, c, d) || is_on_path(solver, y, a, b);
}

```

树形 DP：最大独立集

赛时先看

题目信号：父子不能同时选；答案由子树合并。

本质：树上选择问题的基础模型。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定： $O(n)$ 。

维护的量：`w[u]`（点权，默认 1）、`dp[u][0/1]`（不选/选 u 的子树最大值）、`g`（邻接表）。

警告：先定义清楚 `dp[u][0/1]`，再写转移。

读答案：答案在公开成员 `dp[u][0/1]` 中。

【最小完整示例（先抄这一段就能跑）】

题目：n 个点带权树，父子不能同时选，求最大点权和（点权默认 1 时即最大独立集大小）。

```
TreeIndependentSet solver(n); // 1. 结构体定义：TreeIndependentSet(点数 n)
for (int i = 1; i < n; ++i) { int u, v; cin >> u >> v; solver.add_edge(u, v); } // 2. 加无向边
for (int i = 1; i <= n; ++i) solver.w[i] = w[i]; // 3. 赋点权（不赋默认全 1）
solver.dfs(1, 0); // 4. 根为 1 跑一遍树形 DP
cout << max(solver.dp[1][0], solver.dp[1][1]) << '\n'; // 5. 答案：根选/不选取 max
```

样例：链 1-2-3，点权 {5,1,4} -> 选 1 和 3，答案 9。

【传参要求（照这个传不会错）】

- `TreeIndependentSet(n)`：构造；点编号 1..n。
- `add_edge(u, v)`：加无向边。
- `w[u]`：公开成员点权，默认 1；要在 `dfs` 之前赋好。
- `dfs(root, 0)`：从根调用一次即可（如 `dfs(1, 0)`）。
- `dp[u][0/1]`：公开成员，`dp[u][0]` 不选 u、`dp[u][1]` 选 u 的子树最大值；答案 `max(dp[root][0], dp[root][1])`。

【API / 入口函数（赛时只认这里列的名字）】

- `add_edge(int u, int v)` -> 加入一条边
- `init(int n_)` -> 初始化/清空结构

【改板时先认这几个量】

- `g`：邻接表。
- `dp`：DP 状态。

```
struct TreeIndependentSet {
    int n;
    vector<vector<int>>> g;
    vector<array<i64, 2>>> dp;
    vector<i64> w;

    TreeIndependentSet(int n = 0) { init(n); }

    void init(int n_) {
        n = n_;
        g.assign(n + 1, {});
        dp.assign(n + 1, {0, 0});
        w.assign(n + 1, 1);
    }

    void add_edge(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void dfs(int u, int p) {
        dp[u][0] = 0;
        dp[u][1] = w[u];
        for (int v : g[u]) {
            if (v == p) continue;
            dfs(v, u);
            dp[u][0] += max(dp[v][0], dp[v][1]);
            dp[u][1] += dp[v][0];
        }
    }
};
```

树上最大权匹配：每个点至多选一条邻边

赛时先看

题目信号：配对只能发生在树的相邻点之间；每个点最多参与一次配对；问最多配对数、最大收益的边集合或树上的不相交边。

本质：从树边中选若干条两两不共端点的边，使总边权最大，并恢复选边。所有边权设为 1 就是最大匹配边数。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；`solve()`

里先构造对象，再按下方“不会用就看这里”调用公开接口。

复杂度判定： $O(n)$ 时间、 $O(n)$ 空间；实现用迭代后序，链状树也不会递归爆栈。

维护的量：`best[u]`（u 自由时的最优值）、`blocked[u]`（u 已和父亲匹配时的最优值）、`chosen_child[u]`（恢复选边用）、`edges`（0-based 选边结果）。

警告：选边 (u,v) 时， v 不能再和自己的儿子配对，所以增益是 `edge_weight + blocked[v] - best[v]`。负权边可以不选；若题目强制恰好选若干条边，需要额外加维度，不能直接套。

约定：`vector<pair<int, int>> edges;` // 0-based 顶点编号

【最小完整示例（先抄这一段就能跑）】

题目： n 个点的带权树（顶点从 0 开始编号），每个点至多选一条邻边，求最大匹配权值和选边。

```
vector<vector<pair<int, i64>>> g(n); // 1. 邻接表: 顶点从 0 开始编号
for (int i = 0; i < n - 1; ++i) { int u, v; i64 w; cin >> u >> v >> w; g[u].push_back({v, w}); g[v].push_back({u, w}); }
// 2. 加无向带权边
TreeMatchingResult res = tree_maximum_weight_matching(g); // 3. 一次调用出答案
cout << res.max_weight << '\n'; // 4. 最大匹配权值和
for (auto [u, v] : res.edges) cout << u << ' ' << v << '\n'; // 5. 选中的边 (0-based)
```

样例：链 0-1-2，边权 {5,3} → 选 (0,1)，权值 5。

【传参要求（照这个传不会错）】

- `g`: 邻接表 `g[u] = {v, w}`；顶点是 0-based（从 0 开始，不是 1），点数 = `g.size()`。
- `tree_maximum_weight_matching(g)`: 一次调用返回 `TreeMatchingResult`；无需 build。
- `res.max_weight`: 最大匹配权值和（i64）。
- `res.edges`: 选中的边列表（`pair<int,int>`，0-based）；所有边权设为 1 就是最大匹配边数。
- 负权边自动不选；链状树也安全（迭代后序，不爆栈）。

【改板时先认这几个量】

- `edges`: 0-based 顶点编号。
- `best`: u 自由时的最优值。
- `blocked`: u 已和父亲匹配时的最优值。

```
struct TreeMatchingResult {
    i64 max_weight = 0;
    vector<pair<int, int>> edges; // 0-based 顶点编号。
};

TreeMatchingResult tree_maximum_weight_matching(
    const vector<vector<pair<int, i64>>>& graph
) {
    int n = (int)graph.size();
    if (n == 0) return {};
    vector<int> parent(n, -1), order = {0};
    for (int i = 0; i < (int)order.size(); ++i) {
        int u = order[i];
        for (auto [v, weight] : graph[u]) {
            if (v == parent[u]) continue;
            parent[v] = u;
            order.push_back(v);
        }
    }

    // blocked[u]: u 已经和父亲匹配，不能再选子边；best[u]: u 自由时最优。
    vector<i64> blocked(n), best(n);
    vector<int> chosen_child(n, -1);
    for (int index = n - 1; index >= 0; --index) {
        int u = order[index];
        i64 best_gain = 0;
        for (auto [v, weight] : graph[u]) {
            if (parent[v] != u) continue;
            blocked[u] += best[v];
            best_gain = max(best_gain, weight + blocked[v] - best[v]);
        }
        best[u] = blocked[u] + best_gain;
        if (best_gain > 0) {
            for (auto [v, weight] : graph[u]) {
                if (parent[v] == u && weight + blocked[v] - best[v] == best_gain) {
                    chosen_child[u] = v;
                    break;
                }
            }
        }
    }

    TreeMatchingResult result;
    result.max_weight = best[0];
    vector<pair<int, bool>> stack = {{0, false}}; // bool: 本点是否已经与父亲匹配。
    while (!stack.empty()) {
        auto [u, matched_to_parent] = stack.back();
```

```

stack.pop_back();
int picked = matched_to_parent ? -1 : chosen_child[u];
for (auto [v, weight] : graph[u]) {
    if (parent[v] != u) continue;
    if (v == picked) {
        result.edges.push_back({u, v});
        stack.push_back({v, true});
    } else {
        stack.push_back({v, false});
    }
}
}
return result;
}

```

基环树最大点权独立集：剥叶 + 树 DP + 环 DP

赛时先看

题目信号：题目给 n 点 n 边，或每个点都连向/依赖一个点；图不是树但每个连通块只有一个环；出现“不能同时选相邻点、上司与下属不能同时参加、骑士互相冲突”等约束。

本质：选一些点使任意相邻两点不能同时选，最大化点权和。输入可以有多个连通块，但要求每个连通块都恰好有一个环。

复杂度判定： $O(n)$ 时间、 $O(n)$ 空间。

维护的量：`weight[u]`（点权）、`in_cycle`（是否在环上）、`take/skip`（选/不选该点的树部分 DP 值）。

警告：只支持“每个连通块恰好一个环”；自环不能当环处理；点权可为负，答案允许一个点也不选，初值别设成负无穷。

【最小完整示例（先抄这一段就能跑）】

题目： n 个点 n 条边的无向图（每个连通块恰有一个环），点带权，相邻点不能同时选，求最大点权和。

```

vector<i64> weight(n); // 1. 点权：顶点从 0 开始编号
for (int i = 0; i < n; ++i) cin >> weight[i];
vector<pair<int, int>> edges(n); // 2. 边数 = 点数（每个连通块恰一个环）
for (int i = 0; i < n; ++i) { int u, v; cin >> u >> v; edges[i] = {u, v}; } // 3. 无向冲突边，0-based
cout << maximum_weight_independent_set_unicyclic_forest(weight, edges) << '\n'; // 4. 最大点权和

```

样例：三角形 0-1-2（3 条边），点权 {1,2,3} → 只能选一个点，答案 3。

【传参要求（照这个传不会错）】

- `weight`：点权数组，0-based (`weight[0..n-1]`)；题面从 1 编号就减一。
- `edges`：无向边列表，长度必须等于 n （每个连通块恰有一个环）；不支持自环，平行边长度 2 的环支持。
- `maximum_weight_independent_set_unicyclic_forest(weight, edges)`：返回 `i64` 最大点权和；多连通块自动累加；无需 build。
- 点权可为负，全负时答案 0（允许一个点不选）。
- 若原题是有向依赖边，先确认能否当作无向冲突边。

【改板时先认这几个量】

- `in_cycle`：点是否在环上。
- `take/skip`：选/不选该点的 DP 值。
- `parent`：树边父亲。

状态：

- 剥掉度数不超过 1 的点，剩余点就是环。
- 对每个环点向外的树，`take[u]` 表示选 u ，`skip[u]` 表示不选 u 。有 `take[u] = w[u] + sum(skip[v])`，`skip[u] = sum(max(take[v], skip[v]))`。
- 环上首尾相邻，分别强制首点不选/选，在线性链 DP 中转移，取两种情况最大值。
- 这份函数处理的是每个连通块各有一个环，不要求整图连通；因此总边数等于总点数。
- 点权可以是负数，答案允许一个点也不选；不要把初值设成最小负数。
- 平行边形成的长度 2 环也能处理；自环会改变独立集定义，模板明确不支持。
- 若原题是有向函数图，先确认冲突关系是否应看成无向边；若是“沿有向边选点”的约束，转移含义可能不同。

```

// 每个连通块恰有一个环的无向图上，求最大点权独立集。
// 支持多个连通块与长度为 2 的平行边环；不支持自环。
i64 maximum_weight_independent_set_unicyclic_forest(
    const vector<i64>& weight, const vector<pair<int, int>>& edges
) {
    int n = (int)weight.size();

```

```

if (n == 0) return 0;
assert((int)edges.size() == n); // 每个连通块恰有一个环时，边数等于点数。

vector<vector<pair<int, int>>> graph(n);
vector<int> degree(n);
for (int id = 0; id < n; ++id) {
    auto [u, v] = edges[id];
    assert(0 <= u && u < n && 0 <= v && v < n && u != v);
    graph[u].push_back({v, id});
    graph[v].push_back({u, id});
    ++degree[u];
    ++degree[v];
}

// 剥掉所有叶子，剩下的节点恰好是每个连通块的环。
queue<int> que;
vector<char> removed(n);
for (int u = 0; u < n; ++u) if (degree[u] <= 1) que.push(u);
while (!que.empty()) {
    int u = que.front();
    que.pop();
    if (removed[u]) continue;
    removed[u] = true;
    for (auto [v, id] : graph[u]) {
        if (removed[v]) continue;
        if (--degree[v] == 1) que.push(v);
    }
}
vector<char> in_cycle(n);
for (int u = 0; u < n; ++u) in_cycle[u] = !removed[u];

// 从所有环点向外建根，后序计算环外森林的 take / skip DP。
vector<int> parent(n, -1), order;
for (int root = 0; root < n; ++root) if (in_cycle[root]) {
    for (auto [v, id] : graph[root]) {
        if (!in_cycle[v]) {
            assert(parent[v] == -1);
            parent[v] = root;
            order.push_back(v);
        }
    }
}
for (int index = 0; index < (int)order.size(); ++index) {
    int u = order[index];
    for (auto [v, id] : graph[u]) {
        if (v == parent[u] || in_cycle[v]) continue;
        assert(parent[v] == -1);
        parent[v] = u;
        order.push_back(v);
    }
}

vector<i64> skip(n), take(n);
auto calculate_tree_state = [&](int u) {
    take[u] = weight[u];
    skip[u] = 0;
    for (auto [v, id] : graph[u]) if (parent[v] == u) {
        take[u] += skip[v];
        skip[u] += max(skip[v], take[v]);
    }
};
for (int index = (int)order.size() - 1; index >= 0; --index) {
    calculate_tree_state(order[index]);
}
for (int u = 0; u < n; ++u) if (in_cycle[u]) calculate_tree_state(u);

// 把每个环拆为链：首点不选 / 首点选两种情况，避免首尾同时选。
const i64 NEG = -(1LL << 60);
vector<char> cycle_seen(n);
i64 answer = 0;
for (int start = 0; start < n; ++start) {
    if (!in_cycle[start] || cycle_seen[start]) continue;
    vector<int> cycle;
    int u = start, previous_edge = -1;
    do {
        cycle.push_back(u);
        cycle_seen[u] = true;
        int next = -1, next_edge = -1;
        for (auto [v, edge_id] : graph[u]) {
            if (in_cycle[v] && edge_id != previous_edge) {
                next = v;
                next_edge = edge_id;
                break;
            }
        }
    } while (next != start);
    if (cycle.size() > 1) {
        // 环的大小为 2 的情况
        // ... (code for cycle size 2) ...
    }
    // ... (code for cycle size > 2) ...
}

```

```

    assert(next != -1);
    previous_edge = next_edge;
    u = next;
} while (u != start);

auto solve_chain = [&](bool first_is_taken) {
    i64 previous_skip = first_is_taken ? NEG : skip[cycle[0]];
    i64 previous_take = first_is_taken ? take[cycle[0]] : NEG;
    for (int i = 1; i < (int)cycle.size(); ++i) {
        int v = cycle[i];
        i64 current_skip = max(previous_skip, previous_take) + skip[v];
        i64 current_take = previous_skip + take[v];
        previous_skip = current_skip;
        previous_take = current_take;
    }
    return first_is_taken ? previous_skip : max(previous_skip, previous_take);
};
answer += max(solve_chain(false), solve_chain(true));
}
return answer;
}

```

典题：骑士互斥 / 环外树挂在一个环上

赛时先看

题目信号：每个点有收益、冲突双方不能同时选；关系图每个连通块恰有一个环，或题面给出一个环和若干向外树枝。

本质：把“角色互斥、上司和下属不能同时选、骑士互相冲突”等题面，映射成基环树上的最大点权独立集。

复杂度判定：调用上节函数后为 $O(n)$ 时间、 $O(n)$ 空间。

维护的量：weight（点权，下标 $0..n-1$ ）；edges（无向冲突边，长度 n ）；返回值 i64。

警告：必须先确认关系是无向“互斥边”；有向依赖边不能直接当作无向冲突。若输入有多个连通块，所有连通块的答案都要累加。

【最小完整示例（先抄这一段就能跑）】

题目：n 个角色各有收益，n 条无向互斥边（每个连通块恰有一个环），求互不冲突的最大总收益。

```

vector<i64> weight(n);
for (int i = 0; i < n; ++i) cin >> weight[i]; // weight[i] = 角色 i 的收益
vector<pair<int, int>> edges(n);
for (int i = 0; i < n; ++i) {
    int u, v; cin >> u >> v;
    edges[i] = {u - 1, v - 1}; // 1-based 读入，存成 0-based
}
i64 ans = maximum_weight_independent_set_unicyclic_forest(weight, edges);
cout << ans << '\n'; // 最大总收益

```

样例：3 点环，收益 {5, 2, 8} → ans = 8。

【传参要求（照这个传不会错）】

- weight: vector<i64> 点权，下标 $0..n-1$ ；点权可为负，全负时答案 0（可以一个都不选）。
- edges: vector<pair<int,int>> 无向冲突边，长度必须等于 n（边数等于点数）；不支持自环，平行边（长度 2 的环）支持。
- 返回值: i64 最大点权独立集权值和；多连通块自动累加。

模型：每个角色有收益，若两人存在冲突边则不能同时选。每个连通块恰有一个环，求最大总收益。

输入改写：点权放入 weight[0..n-1]，每条无向冲突边放入 edges；调用

maximum_weight_independent_set_unicyclic_forest(weight, edges)。若题目从 1 开始编号，读入后减一。

手算检查：

- 纯 3 环的答案是三个点权中最大的一个，不能误选两个端点。
- 环点挂一棵树时，先把该树的 take/skip 合到环点，不能把环外点再拿去做一遍环 DP。
- 点权全负时答案应为 0，因为可以不选任何点。

经典练习：洛谷 P2607《骑士》。它是“环 +

挂树”模型的代表；做题时先确认题目给出的关系转成无向冲突边后是否允许平行边，再按题面处理。

换根 DP：所有点到其他点距离和

赛时先看

题目信号：对树上每个点求全局值；从父亲换到儿子可快速更新。

本质：每个点作为根都要答案。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定： $O(n)$ 。

维护的量：`sz[u]`（子树大小）；`dp[u]`（以 `u` 为根时子树内各点到 `u` 的距离和）；`ans[u]`（以 `u` 为根的全局距离和，即最终答案）。

警告：换根前保存现场，递归回来恢复。

【最小完整示例（先抄这一段就能跑）】

题目：`n` 点树，对每个点 `u` 求所有点到 `u` 的距离和。

```
RerootDistanceSum sol;
sol.init(n);           // 1. 初始化，点编号 1..n
for (int i = 1; i < n; ++i) {
    int u, v; cin >> u >> v;
    sol.add_edge(u, v); // 2. 加无向边
}
vector<i64> ans = sol.solve(); // 3. ans[u] = 以 u 为根的距离和
```

样例：链 1-2-3 $\rightarrow$ `ans = {3, 2, 3}`。

【传参要求（照这个传不会错）】

- 下标：点编号 1..n，`init(n_)` 自动开 `n+1` 的数组。
- `init(int n_)`：先调用；`n_` 为点数，重复调用即清空重来。
- `add_edge(int u, int v)`：每条无向边调一次，内部自动补双向。
- `solve()`：加完全部边后调用，无需参数；返回 `vector<i64>`，`ans[u]` 直接输出。

【API / 入口函数（赛时只认这里列的名字）】

- `add_edge(int u, int v)` $\rightarrow$ 加入一条边
- `init(int n_)` $\rightarrow$ 初始化/清空结构
- `solve()` $\rightarrow$ 执行主算法并返回答案

【改板时先认这几个量】

- `g`：邻接表。
- `sz`：集合/子树大小。

```
struct RerootDistanceSum {
    int n;
    vector<vector<int>> g;
    vector<int> sz;
    vector<i64> dp, ans;

    RerootDistanceSum(int n = 0) { init(n); }

    void init(int n_) {
        n = n_;
        g.assign(n + 1, {});
        sz.assign(n + 1, 0);
        dp.assign(n + 1, 0);
        ans.assign(n + 1, 0);
    }

    void add_edge(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void dfs1(int u, int p) {
        sz[u] = 1;
        for (int v : g[u]) {
            if (v == p) continue;
            dfs1(v, u);
            sz[u] += sz[v];
            dp[u] += dp[v] + sz[v];
        }
    }

    void dfs2(int u, int p) {
        ans[u] = dp[u];
        for (int v : g[u]) {
```

```

        if (v == p) continue;
        i64 du = dp[u], dv = dp[v];
        int su = sz[u], sv = sz[v];

        dp[u] -= dp[v] + sz[v];
        sz[u] -= sz[v];
        dp[v] += dp[u] + sz[u];
        sz[v] += sz[u];

        dfs2(v, u);

        dp[u] = du; dp[v] = dv;
        sz[u] = su; sz[v] = sv;
    }
}

vector<i64> solve() {
    dfs1(1, 0);
    dfs2(1, 0);
    return ans;
}
};

```

DSU on Tree

赛时先看

题目信号：每个节点都问“它的子树里……”；需要保留重儿子信息。

本质：统计每个子树内颜色出现次数、众数、满足条件数量。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定： $O(n \log n)$ 或均摊 $O(n)$ 级别。

维护的量：`color[u]`（点 `u` 的颜色，手动赋值）；`cnt[c]`（当前桶内颜色 `c` 的出现次数）；`best`（当前众数出现次数）；`ans[u]`（`u` 子树内众数出现次数）。

警告：轻儿子算完清空，重儿子保留；`add_subtree` 要跳过重儿子。

读答案：答案在公开成员 `ans[u]` 中。

【最小完整示例（先抄这一段就能跑）】

题目：`n` 点树，每个点有颜色，对每个点求其子树内出现次数最多的颜色出现了几次。

```

DsuOnTree d(5, 10);           // 1. 初始化：点数、颜色值上界
d.add_edge(1, 2); d.add_edge(2, 3);
d.add_edge(3, 4); d.add_edge(3, 5); // 2. 加无向边
d.color[1] = 1; d.color[2] = 2; d.color[3] = 2;
d.color[4] = 2; d.color[5] = 3;    // 3. 赋颜色（范围 1..max_color）
d.dfs_size(1, 0);                 // 4. 先算子树大小/重儿子
d.dfs(1, 0, true);                 // 5. 跑 DSU on Tree，答案进 d.ans

```

样例：上例 $\rightarrow$ `d.ans[1]=3`（颜色 2 全树出现 3 次），`d.ans[3]=2`。

【传参要求（照这个传不会错）】

- 下标：点编号 `1..n`；颜色值 `1..max_color`。
- `init(int n_, int max_color)`：先调用；`max_color` 是颜色最大值（`cnt` 开到 `max_color+1`）。
- `add_edge(int u, int v)`：每条无向边调一次。
- `color[u]`：建完图后逐个赋值颜色，必须在 `dfs` 之前。
- `dfs_size(1, 0)` 与 `dfs(1, 0, true)`：两步都要从根调用一次；返回值为 `void`，结果读公开成员 `ans[u]`。

【API / 入口函数（赛时只认这里列的名字）】

- `add_edge(int u, int v)` $\rightarrow$ 加入一条边
- `init(int n_, int max_color)` $\rightarrow$ 初始化/清空结构

【改板时先认这几个量】

- `g`：邻接表。
- `sz`：集合/子树大小。

```

struct DsuOnTree {
    int n;
    vector<vector<int>> g;
    vector<int> color, sz, heavy, ans;
    vector<int> cnt;
    int best = 0;
}

```

```

DsuOnTree(int n = 0, int max_color = 0) { init(n, max_color); }

void init(int n_, int max_color) {
    n = n_;
    g.assign(n + 1, {});
    color.assign(n + 1, 0);
    sz.assign(n + 1, 0);
    heavy.assign(n + 1, 0);
    ans.assign(n + 1, 0);
    cnt.assign(max_color + 1, 0);
}

void add_edge(int u, int v) {
    g[u].push_back(v);
    g[v].push_back(u);
}

void dfs_size(int u, int p) {
    sz[u] = 1;
    heavy[u] = 0;
    for (int v : g[u]) {
        if (v == p) continue;
        dfs_size(v, u);
        sz[u] += sz[v];
        if (!heavy[u] || sz[v] > sz[heavy[u]]) heavy[u] = v;
    }
}

void add_node(int u) {
    best = max(best, ++cnt[color[u]]);
}

void add_subtree(int u, int p, int banned) {
    add_node(u);
    for (int v : g[u]) {
        if (v != p && v != banned) add_subtree(v, u, banned);
    }
}

void clear_subtree(int u, int p) {
    cnt[color[u]]--;
    for (int v : g[u]) {
        if (v != p) clear_subtree(v, u);
    }
}

void dfs(int u, int p, bool keep) {
    for (int v : g[u]) {
        if (v != p && v != heavy[u]) dfs(v, u, false);
    }
    if (heavy[u]) dfs(heavy[u], u, true);
    add_subtree(u, p, heavy[u]);
    ans[u] = best;
    if (!keep) {
        clear_subtree(u, p);
        best = 0;
    }
}
};

```

树的重心

赛时先看

题目信号：需要把树尽量均匀地分开；树上分治。

本质：找删除后最大连通块最小的点，点分治前置。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定： $O(n)$ 。

维护的量：`sz[u]`（子树大小）；`mx`（删 `u` 后最大连通块大小，dfs 内局部变量）；`centroids`（重心集合，1 或 2 个）；`best`（当前最小的最大块）。

警告：树可能有两个重心。

【最小完整示例（先抄这一段就能跑）】

题目：`n` 点树，求所有重心。

```

TreeCentroid tc;
tc.init(4); // 1. 初始化，点编号 1..n

```



```
tc.add_edge(1, 2);           // 2. 加无向边
tc.add_edge(2, 3);
tc.add_edge(2, 4);
vector<int> c = tc.solve();    // 3. 重心集合
```

样例：1-2, 2-3, 2-4（星形）→ $c = \{2\}$ ；链 1-2-3-4 有两个重心 $c = \{2, 3\}$ 。

【传参要求（照这个传不会错）】

- 下标：点编号 1..n, DFS 从 1 出发（树须连通）。
- `init(int n_)`：先调用，清空结构。
- `add_edge(int u, int v)`：每条无向边调一次。
- `solve()`：加完全部边后调用；返回 `vector<int>`，含 1 或 2 个重心编号。

【API / 入口函数（赛时只认这里列的名字）】

- `add_edge(int u, int v)` → 加入一条边
- `init(int n_)` → 初始化/清空结构
- `solve()` → 执行主算法并返回答案

【改板时先认这几个量】

- `g`：邻接表。
- `sz`：集合/子树大小。
- `mx`：区间最大值。

```
struct TreeCentroid {
    int n;
    vector<vector<int>>> g;
    vector<int> sz, centroids;
    int best;

    TreeCentroid(int n = 0) { init(n); }

    void init(int n_) {
        n = n_;
        g.assign(n + 1, {});
        sz.assign(n + 1, 0);
        centroids.clear();
        best = n + 1;
    }

    void add_edge(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void dfs(int u, int p) {
        sz[u] = 1;
        int mx = 0;
        for (int v : g[u]) {
            if (v == p) continue;
            dfs(v, u);
            sz[u] += sz[v];
            mx = max(mx, sz[v]);
        }
        mx = max(mx, n - sz[u]);
        if (mx < best) {
            best = mx;
            centroids = {u};
        } else if (mx == best) {
            centroids.push_back(u);
        }
    }

    vector<int> solve() {
        dfs(1, 0);
        return centroids;
    }
};
```

点分治骨架

赛时先看

题目信号：树上路径统计，直接从每个点 DFS 会超时；路径跨过某个分治重心。

本质：统计树上路径类问题，如距离不超过 K 的点对数。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定：常见 $O(n \log n)$ 。

维护的量：`sz[u]`（当前连通块的子树大小）；`dead[u]`（该点已被当作重心删除的标记）；`g` 为带权邻接表，存（终点，边权）。

警告：每层分治要标记删除重心；统计时先减去同一子树内部贡献，再加全局贡献。

【最小完整示例（先抄这一段就能跑）】

题目：统计距离不超过 `k` 的点对数（统计逻辑写在 `solve_at_centroid`）。

```
CentroidDecomposition cd;
cd.init(5); // 1. 初始化，点编号 1..n
cd.add_edge(1, 2, 1); // 2. 加无向带权边 (u, v, w)
cd.add_edge(2, 3, 1);
cd.add_edge(3, 4, 1);
cd.add_edge(3, 5, 1);
cd.decompose(1); // 3. 分治入口，答案在 solve_at_centroid 里统计
```

样例：链 1-2-3：点对距离 `{(1,2):1, (2,3):1, (1,3):2}`，在 `solve_at_centroid(c)` 中收集距离后计数。

【传参要求（照这个传不会错）】

- 下标：点编号 `1..n`。
- `init(int n_)`：先调用。
- `add_edge(int u, int v, int w = 1)`：无向带权边；不传 `w` 时默认边权 1。
- `decompose(int entry)`：加完后从任意点（一般 1）调用启动分治；每层自动把重心 `c` 传进 `solve_at_centroid(c)`，统计逻辑写在这里。
- 返回值：无；结果在你写的统计逻辑里记录。

【API / 入口函数（赛时只认这里列的名字）】

- `add_edge(int u, int v, int w = 1)` -> 加入一条边
- `init(int n_)` -> 初始化/清空结构

【改板时先认这几个量】

- `g`：邻接表。
- `sz`：集合/子树大小。

```
struct CentroidDecomposition {
    int n;
    vector<vector<pair<int, int>>> g;
    vector<int> sz, dead;

    CentroidDecomposition(int n = 0) { init(n); }

    void init(int n_) {
        n = n_;
        g.assign(n + 1, {});
        sz.assign(n + 1, 0);
        dead.assign(n + 1, 0);
    }

    void add_edge(int u, int v, int w = 1) {
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }

    int calc_size(int u, int p) {
        sz[u] = 1;
        for (auto [v, w] : g[u]) {
            if (v != p && !dead[v]) sz[u] += calc_size(v, u);
        }
        return sz[u];
    }

    int find_centroid(int u, int p, int total) {
        for (auto [v, w] : g[u]) {
            if (v != p && !dead[v] && sz[v] * 2 > total) return find_centroid(v, u, total);
        }
        return u;
    }

    void collect_dist(int u, int p, int d, vector<int>& ds) {
        ds.push_back(d);
    }
};
```

```

    for (auto [v, w] : g[u]) {
        if (v != p && !dead[v]) collect_dist(v, u, d + w, ds);
    }

    void solve_at_centroid(int c) {
        // 在这里写“经过 c 的路径”的统计逻辑。
        // 常见做法：对每个子树 collect_dist，先查询全局桶，再把孩子树距离加入桶。
    }

    void decompose(int entry) {
        int total = calc_size(entry, 0);
        int c = find_centroid(entry, 0, total);
        dead[c] = 1;
        solve_at_centroid(c);
        for (auto [v, w] : g[c]) {
            if (!dead[v]) decompose(v);
        }
    }
};

```

树上路径计数：点分治统计恰好 K 条边

赛时先看

题目信号：树上所有点对/路径计数，朴素枚举两端点是

$O(n^2)$ ；条件只与路径长度或长度可聚合属性有关；题目给出一个 K ，问有多少对距离刚好为 K 。

本质：统计无权树上距离（边数）恰好为 K 的无序点对数。可作为“路径长度不超过 K ”“路径权值等于 K ”的点分治入门母版。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API

列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定： $O(n \log n)$ 时间、 $O(n)$ 空间。下面是 0-indexed 无权树版本，组件扫描迭代完成，只有深度 $O(\log n)$ 的分治递归。

维护的量：`subtree_size[u]`（组件子树大小）；`frequency[d]`（当前分治层已处理子树中深度 d 的节点数桶）；`blocked[u]`（重心删除标记）；`answer`（累计点对数）。

警告： $K = 0$ 时答案为 0（每点对距离为正，不把“同一个点”计入）；图必须 0-indexed；点对计数上限是 $n*(n-1)/2$ ，用 `i64`。

约定：0-indexed 无权树：统计距离（边数）恰为 K 的无序点对数量

【最小完整示例（先抄这一段就能跑）】

题目： n 点 0-indexed 无权树，统计距离（边数）恰为 K 的无序点对数。

```

CentroidExactLengthPathCounter counter(4); // 1. 0-indexed, 4 个点
counter.add_edge(0, 1); // 2. 加无向边
counter.add_edge(1, 2);
counter.add_edge(1, 3);
i64 ans = counter.count_paths_exactly(2); // 3. 距离恰为 2 的点对数

```

样例：星形（中心 1，叶子 0/2/3） $\rightarrow$ `ans = 3`（三个叶子两两配对）。

【传参要求（照这个传不会错）】

- 下标：0-indexed，点 $0..n-1$ ；`graph` 开 n 个。
- `init(int n_)`（或构造时直接传）：先调用；重复调用即清空。
- `add_edge(int u, int v)`：无向边，要求 $0 \leq u, v < n$ 且 $u \neq v$ 。
- `count_paths_exactly(int k)`：加完边后调用一次； $k \leq 0$ 或 $k \geq n$ 直接返回 0；返回 `i64` 点对数（ $K=0$ 返回 0，端点相同的对不计）。

【API / 入口函数（赛时只认这里列的名字）】

- `add_edge(int u, int v)` $\rightarrow$ 加入一条边
- `init(int n_)` $\rightarrow$ 初始化/清空结构

【改板时先认这几个量】

- `graph`：邻接表。
- `subtree_size`：子树大小。
- `frequency`：每个深度的出现次数桶。
- `blocked`：分治中被删除的重心标记。

核心：对每层分治重心 c ，维护已处理子树中每个深度出现次数。遍历一棵新子树的深度 d 时，累加已经出现的 $k-d$ ，这样只统计“路径经过 c 且两端来自不同子树”的点对。子树内部点对留给递归层处理。

- `frequency[0]=1` 表示重心本身，因此自动统计“一个端点是重心”的路径；不要额外加一次。
- 一定要先查询当前子树、再把当前子树深度加入桶，否则会把同一子树内部路径提前计入，之后重复计算。
- 本块统计的是不同端点的无序对， $K=0$ 返回 0 。若题目允许端点相同，需要单独加 n 。
- 带权边不能直接以深度做数组下标；小权值可改桶，大权值常用“收集距离 + 排序双指针”统计不超过某阈值。

```
// 0-indexed 无权树：统计距离（边数）恰为 K 的无序点对数量。
// 仅分治过程递归，深度  $O(\log n)$ ；组件遍历和距离收集均为迭代。
struct CentroidExactLengthPathCounter {
    int n = 0, target = 0;
    vector<vector<int>> graph;
    vector<int> parent, subtree_size, frequency;
    vector<char> blocked;
    i64 answer = 0;

    explicit CentroidExactLengthPathCounter(int n_ = 0) { init(n_); }

    void init(int n_) {
        n = n_;
        graph.assign(n, {});
        parent.assign(n, -1);
        subtree_size.assign(n, 0);
        blocked.assign(n, false);
    }

    void add_edge(int u, int v) {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v);
        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    i64 count_paths_exactly(int k) {
        if (n == 0 || k <= 0 || k >= n) return 0;
        target = k;
        answer = 0;
        fill(blocked.begin(), blocked.end(), false);
        frequency.assign(target + 1, 0);
        decompose(0);
        return answer;
    }

private:
    int find_centroid(int entry) {
        vector<int> order = {entry};
        parent[entry] = -1;
        for (int index = 0; index < (int)order.size(); ++index) {
            int u = order[index];
            for (int v : graph[u]) {
                if (blocked[v] || v == parent[u]) continue;
                parent[v] = u;
                order.push_back(v);
            }
        }

        for (int index = (int)order.size() - 1; index >= 0; --index) {
            int u = order[index];
            subtree_size[u] = 1;
            for (int v : graph[u]) {
                if (!blocked[v] && parent[v] == u) subtree_size[u] += subtree_size[v];
            }
        }

        int total = (int)order.size();
        int centroid = entry, largest_part = total;
        for (int u : order) {
            int largest = total - subtree_size[u];
            for (int v : graph[u]) {
                if (!blocked[v] && parent[v] == u) largest = max(largest, subtree_size[v]);
            }
            if (largest < largest_part) {
                largest_part = largest;
                centroid = u;
            }
        }
        return centroid;
    }

    vector<int> collect_depths(int start, int par) const {
        struct Node { int u, parent, depth; };
        vector<Node> stack = {{start, par, 1}};

```

```

vector<int> depths;
while (!stack.empty()) {
    Node current = stack.back();
    stack.pop_back();
    if (current.depth > target) continue;
    depths.push_back(current.depth);
    for (int v : graph[current.u]) {
        if (!blocked[v] && v != current.parent) {
            stack.push_back({v, current.u, current.depth + 1});
        }
    }
}
return depths;
}

void decompose(int entry) {
    int centroid = find_centroid(entry);
    blocked[centroid] = true;

    vector<int> touched = {0};
    frequency[0] = 1; // 重心本身。
    for (int v : graph[centroid]) {
        if (blocked[v]) continue;
        vector<int> depths = collect_depths(v, centroid);
        for (int depth : depths) answer += frequency[target - depth];
        for (int depth : depths) {
            if (frequency[depth]++ == 0) touched.push_back(depth);
        }
    }
    for (int depth : touched) frequency[depth] = 0;

    for (int v : graph[centroid]) if (!blocked[v]) decompose(v);
}
};

```

典题：CSES Fixed-Length Paths I；将树读成 0-indexed 后，输出 `counter.count_paths_exactly(K)`。CSES Fixed-Length Paths II 把条件改为路径长度在 $[K1, K2]$ ，需要把本块的单深度桶换成前缀和/Fenwick 查询，分治“跨子树统计”的结构不变。

动态点分治：点开关，查询最近标记点

赛时先看

题目信号：树不变；操作是点颜色翻转、点加入/删除集合；查询离某点最近的特殊点。经典模型类似 QTREE5。

本质：树上动态维护一批“被标记/变黑”的点，支持单点开关，查询某个点到最近标记点的距离。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API

列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定：建树 $O(n \log n)$ ，每次修改/查询 $O(\log^2 n)$ ，如果用堆加懒删除可做到 $O(\log n \log n)$ 量级。

维护的量：`up[u]`（点 u 到每个点分祖先重心的（重心，距离）列表）；`bag[c]`（重心 c 处所有已激活点到 c 的距离 multiset）；`active[u]`（ u 当前是否点亮）。

警告：每个点要记录它到所有点分树祖先重心的距离；删除时用 `multiset.find` 删除一个距离；没有标记点时返回 `-1`。

【最小完整示例（先抄这一段就能跑）】

题目： n 点树， q 次操作：开关点 / 查询某点到最近亮点距离。

```

DynamicCentroidDecomposition dc;
dc.init(5); // 1. 初始化，点编号 1..n
dc.add_edge(1, 2); // 2. 加无向边（可带权）
dc.add_edge(2, 3);
dc.add_edge(3, 4); dc.add_edge(3, 5);
dc.build(1); // 3. 先建点分树，之后才能开关/查询
dc.toggle(2); // 4. 点亮 2
i64 d = dc.query_nearest(4); // 5. 4 到最近亮点的距离

```

样例：上述 5 点树，点亮 2 后 $\rightarrow$ `query_nearest(4) = 2`；无亮点时返回 `-1`。

【传参要求（照这个传不会错）】

- 下标：点编号 $1..n$ 。
- 顺序：`init(n)` $\rightarrow$ `add_edge(u, v, w = 1)` $\rightarrow$ `build(root = 1)`；必须先 build 再开关/查询。
- `toggle(int u)` / `turn_on(int u)` / `turn_off(int u)`：开关点，重复调用安全。
- `query_nearest(int u)`：返回 `i64` 最近亮点距离；无亮点返回 `-1`。

【API / 入口函数（赛时只认这里列的名字）】

- `add_edge(int u, int v, i64 w = 1)` -> 加入一条边
- `build(int root = 1)` -> 完成建树或预处理
- `init(int n_)` -> 初始化/清空结构

【改板时先认这几个量】

- `g`: 邻接表。
- `up`: `up[u]` 存所有祖先点分中心及 `u` 到它的距离。
- `bag`: `bag[c]` 存所有已激活点到点分中心 `c` 的距离。

```
struct DynamicCentroidDecomposition {
    int n;
    vector<vector<pair<int, i64>>> g;
    vector<int> sz, dead, active;
    vector<vector<pair<int, i64>>> up; // up[u] 存所有祖先点分中心及 u 到它的距离。
    vector<multiset<i64>> bag; // bag[c] 存所有已激活点到点分中心 c 的距离。

    DynamicCentroidDecomposition(int n = 0) { init(n); }

    void init(int n_) {
        n = n_;
        g.assign(n + 1, {});
        sz.assign(n + 1, 0);
        dead.assign(n + 1, 0);
        active.assign(n + 1, 0);
        up.assign(n + 1, {});
        bag.assign(n + 1, {});
    }

    void add_edge(int u, int v, i64 w = 1) {
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }

    int calc_size(int u, int p) {
        sz[u] = 1;
        for (auto [v, w] : g[u]) {
            if (v != p && !dead[v]) sz[u] += calc_size(v, u);
        }
        return sz[u];
    }

    int find_centroid(int u, int p, int total) {
        for (auto [v, w] : g[u]) {
            if (v != p && !dead[v] && sz[v] * 2 > total) {
                return find_centroid(v, u, total);
            }
        }
        return u;
    }

    void collect(int u, int p, i64 d, int c) {
        up[u].push_back({c, d});
        for (auto [v, w] : g[u]) {
            if (v != p && !dead[v]) collect(v, u, d + w, c);
        }
    }

    void decompose(int entry) {
        int total = calc_size(entry, 0);
        int c = find_centroid(entry, 0, total);
        dead[c] = 1;
        collect(c, 0, 0, c);
        for (auto [v, w] : g[c]) {
            if (!dead[v]) decompose(v);
        }
    }

    void build(int root = 1) {
        decompose(root);
    }

    void turn_on(int u) {
        if (active[u]) return;
        active[u] = 1;
        for (auto [c, d] : up[u]) bag[c].insert(d);
    }

    void turn_off(int u) {
        if (!active[u]) return;
        active[u] = 0;
        for (auto [c, d] : up[u]) {
```

```

        auto it = bag[c].find(d);
        if (it != bag[c].end()) bag[c].erase(it);
    }
}

void toggle(int u) {
    if (active[u]) turn_off(u);
    else turn_on(u);
}

i64 query_nearest(int u) const {
    const i64 INF = (1LL << 60);
    i64 ans = INF;
    for (auto [c, d] : up[u]) {
        if (!bag[c].empty()) ans = min(ans, d + *bag[c].begin());
    }
    return ans == INF ? -1 : ans;
}
};

```

虚树 Virtual Tree

赛时先看

题目信号：多次询问给出若干关键点，要在树上按路径关系 DP。

本质：给一小批关键点，在原树上构造只包含关键点和 LCA 的压缩树。

接法：把本节 struct/class/函数组 整段抄到 solve() 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定： $O(k \log k)$ 。

维护的量：tin[u]、depth[u]（自备 DFS 序与深度，需先赋值）；lca(a,b)（自备 LCA 回调）；build 返回的 vt（虚树邻接表，节点按排序去重后的 nodes 顺序编号）。

警告：关键点按 DFS 序排序；相邻点的 LCA 加入集合；用栈连边。

依赖：需自备倍增/HLD 的 tin、depth、lca；本模板的 LCA 结构没有成员 tin，可改用 HLD 的 dfn 或自行记录。

【最小完整示例（先抄这一段就能跑）】

题目：给 k 个关键点，建出只含关键点与两两 LCA 的虚树。

```

VirtualTree vt;
vt.tin = tin; vt.depth = depth; // 1. 来自自备 LCA/HLD
vt.lca = [&](int a, int b) { return solver.lca(a, b); }; // 2. 绑定 LCA 回调
vector<vector<int>> vt_edges = vt.build({2, 4, 5}); // 3. 传关键点列表

```

样例：链 1-2-3-4-5（tin 即编号）-> build({2,4,5}) 得节点 {2,4,5}，边 2-4、4-5。

【传参要求（照这个传不会错）】

- 下标：与自备 LCA 的点编号一致（0 或 1 开头均可）。
- 先准备：赋值 tin、depth 两个成员，并把 lca 绑定成 function<int(int,int)>；本模板不含 LCA 实现。
- build(vector<int> nodes)：传关键点编号，可乱序、可重复；返回 vt，其中下标 i 对应 nodes（排序去重后）第 i 个点，vt 里存的邻接也是虚树内下标。

【API / 入口函数（赛时只认这里列的名字）】

- build(vector<int> nodes) -> 完成建树或预处理 返回 vector<vector<int>>。

```

struct VirtualTree {
    vector<int> tin, depth;
    function<int(int,int)> lca;

    bool by_dfn(int a, int b) const { return tin[a] < tin[b]; }

    vector<vector<int>> build(vector<int> nodes) {
        sort(nodes.begin(), nodes.end(), [&](int a, int b) { return tin[a] < tin[b]; });
        int k = (int)nodes.size();
        for (int i = 0; i + 1 < k; ++i) nodes.push_back(lca(nodes[i], nodes[i + 1]));
        sort(nodes.begin(), nodes.end(), [&](int a, int b) { return tin[a] < tin[b]; });
        nodes.erase(unique(nodes.begin(), nodes.end()), nodes.end());

        vector<vector<int>> vt(nodes.size());
        unordered_map<int, int> id;
        for (int i = 0; i < (int)nodes.size(); ++i) id[nodes[i]] = i;

        vector<int> st;
        for (int u : nodes) {
            if (st.empty()) {
                st.push_back(u);
            }
        }
    }
};

```



```

        continue;
    }
    while (!st.empty() && lca(st.back(), u) != st.back()) st.pop_back();
    vt[id[st.back()]].push_back(id[u]);
    st.push_back(u);
}
return vt;
};

```

虚树例题：关键点两两路径总长度

赛时先看

题目信号：每次询问给 k 个点， $\sum k$ 较大但不能每对处理。

本质：给一批关键点，求它们在树上诱导路径的总边长。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()`

里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O(k \log k)$ 。

维护的量：无结构体；只维护局部 `sum`（关键点按 DFS 序排成环后相邻点距离和，除以 2 即答案）。

警告：把关键点按 DFS 序排成环，答案是相邻距离和的一半。

依赖：需自备倍增/HLD 的 `tin`、`depth`、`lca`；本模板的 LCA 结构没有成员 `tin`，可改用 HLD 的 `dfn` 或自行记录。

【最小完整示例（先抄这一段就能跑）】

题目：每次给 k 个关键点，求覆盖它们的最小 Steiner 树边长（关键点两两路径并的总长）。

```

LCA solver; // 自备 LCA, 需有 distance(a,b)
solver.build(n, edges); // 预处理
vector<int> tin = dfn; // DFS 序数组
i64 len = marked_steiner_tree_length({2, 4, 5}, solver, tin);

```

样例：链 1-2-3-4-5 选 {2,4,5} $\rightarrow$ `len = 3`（路径并 2-3-4 与 4-5）。

【传参要求（照这个传不会错）】

- `nodes`：关键点编号列表，可乱序、可重复（内部先排序去重）。
- `solver`：自备 LCA 结构，必须实现 `distance(a, b)`（返回树上两点距离）。
- `tin`：DFS 序数组，`tin[u]` 与 `solver` 的点编号一致。
- 返回值：`i64` 最小 Steiner 树总边长； $k = 1$ 或空列表时返回 0。

```

i64 marked_steiner_tree_length(vector<int> nodes, const LCA& solver, const vector<int>& tin) {
    sort(nodes.begin(), nodes.end(), [&](int a, int b) { return tin[a] < tin[b]; });
    nodes.erase(unique(nodes.begin(), nodes.end()), nodes.end());
    i64 sum = 0;
    int k = (int)nodes.size();
    for (int i = 0; i < k; ++i) {
        int a = nodes[i], b = nodes[(i + 1) % k];
        sum += solver.distance(a, b);
    }
    return sum / 2;
}

```

函数图倍增

赛时先看

题目信号：状态转移固定 `next[x]`，问走很多步后到哪里。

本质：每个点出度为 1，快速跳 k 步。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API

列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定：预处理 $O(n \log K)$ ，查询 $O(\log K)$ 。

维护的量：`up[k][i]`（点 i 跳 2^k 步到达的点）； n （点数）；`LOG`（倍增层数）；`max_k`（步数上界）。

警告：`up[0][i] = f[i]`；查询的 `steps` 不能超过构造时传入的 `max_k`。

【最小完整示例（先抄这一段就能跑）】

题目：每个点出度为 1（`nxt[i]` 是下一步），问从 x 走 `steps` 步到哪个点。

```

vector<int> nxt = {0, 2, 3, 1}; // 1-based nxt[1]=2, nxt[2]=3, nxt[3]=1
FunctionalGraphJump fg(nxt, 1e18); // 2. 建表, 步数上界 1e18
int to = fg.jump(1, 5); // 3. 从 1 走 5 步

```

样例：环 1->2->3->1 -> `jump(1, 5) = 3`。

【传参要求（照这个传不会错）】

- `nxt`: 长度 `n+1` 的 `vector<int>`, `nxt[i]` 为点 `i` 的下一步（点编号 `1..n`, `nxt[0]` 不参与）。
- `max_k`: 查询步数最大值，只影响建表层数；`jump` 的 `steps` 不能超过它（否则 `assert` 失败）。
- `jump(int x, i64 steps)`: `x` 为起点 (`1..n`)，`steps` 为步数；返回 `int` 走完后的点编号。

【API / 入口函数（赛时只认这里列的名字）】

- `jump(int x, i64 steps)` -> 向上跳若干级祖先 返回 `int`。

【改板时先认这几个量】

- `up`: 倍增祖先表。
- `nxt`: 转移/子节点。

```
struct FunctionalGraphJump {
    int n, LOG;
    i64 max_k;
    vector<vector<int>> up;

    FunctionalGraphJump(const vector<int>& nxt, i64 max_k) : max_k(max_k) {
        n = (int)nxt.size() - 1;
        LOG = 1;
        while ((1LL << LOG) <= max_k) LOG++;
        up.assign(LOG, vector<int>(n + 1));
        for (int i = 1; i <= n; ++i) up[0][i] = nxt[i];
        for (int k = 1; k < LOG; ++k) {
            for (int i = 1; i <= n; ++i) up[k][i] = up[k - 1][up[k - 1][i]];
        }
    }

    int jump(int x, i64 steps) const {
        assert(steps <= max_k); // steps 不能超过构造时的 max_k。
        for (int k = 0; steps; ++k, steps >>= 1) {
            if (steps & 1) x = up[k][x];
        }
        return x;
    }
};
```

树哈希：无根树同构判定

赛时先看

题目信号：节点没有固定编号含义；只关心树的形状；题目询问两棵树能否重标号后相同。

本质：快速判断两棵无根树是否同构，或对大量树形结构分组。先找树中心，再把中心作为根求子树哈希。

接法：给出很多棵树，统计互不同构的树形数量。

复杂度判定：找中心和计算哈希都为 $O(n \log n)$ ， $\log n$ 来自对子哈希排序。

维护的量：无结构体；`TreeHash = pair<ui64, ui64>`（一棵子树/一棵无根树的哈希对），用 `==` 判同构。

警告：单哈希可能碰撞；这里用两组 `uint64_t`，对需要完全确定性的题可改成 AHU 括号规范表示或离散化子树类型。

【最小完整示例（先抄这一段就能跑）】

题目：判断两棵无根树是否同构（可重标号）。

```
vector<vector<int>> g1(n), g2(n); // 0-indexed 无向邻接表
TreeHash h1 = unrooted_tree_hash(g1); // 1. 整棵树的哈希
TreeHash h2 = unrooted_tree_hash(g2);
bool same = (h1 == h2); // 2. 哈希相等即同构
```

样例：4 点链 0-1-2-3 与 4 点星形（中心 0）-> 哈希不同（不同构）。

【传参要求（照这个传不会错）】

- 下标：0-indexed, `g[u]` 存 `u` 的邻居（无向，每边存两次）。
- `unrooted_tree_hash(g)`: 直接传整棵树的邻接表，内部自动找树中心再求哈希；返回 `TreeHash`（两个 `ui64`）。
- `rooted_hash(u, p, g)` / `combine_hash(child)` / `splitmix64(x)`: 内部辅助函数，一般无需外部直接调用。
- 返回值：两个 `TreeHash` 相等即同构；要分组统计时直接用 `TreeHash` 作 `map/set` 的 key。

典题模型：给出很多棵树，统计互不同构的树形数量。

```
using TreeHash = pair<ui64, ui64>;
```

```

ui64 splitmix64(ui64 x) {
    x += 0x9e3779b97f4a7c15ULL;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
    x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
    return x ^ (x >> 31);
}

TreeHash combine_hash(vector<TreeHash> child) {
    sort(child.begin(), child.end());
    ui64 a = 0x123456789abcdef0ULL;
    ui64 b = 0xfedcba9876543210ULL;
    for (auto [x, y] : child) {
        a = splitmix64(a ^ (x + 0x9e3779b97f4a7c15ULL));
        b = splitmix64(b ^ (y + 0xbf58476d1ce4e5b9ULL));
    }
    return {splitmix64(a + child.size()), splitmix64(b + child.size())};
}

TreeHash rooted_hash(int u, int p, const vector<vector<int>>& g) {
    vector<TreeHash> child;
    for (int v : g[u]) {
        if (v != p) child.push_back(rooted_hash(v, u, g));
    }
    return combine_hash(child);
}

vector<int> tree_centers(const vector<vector<int>>& g) {
    int n = (int)g.size();
    if (n == 1) return {0};
    vector<int> deg(n);
    queue<int> q;
    for (int i = 0; i < n; ++i) {
        deg[i] = (int)g[i].size();
        if (deg[i] <= 1) q.push(i);
    }
    int remain = n;
    while (remain > 2) {
        int layer = (int)q.size();
        remain -= layer;
        while (layer-- > 0) {
            int u = q.front();
            q.pop();
            for (int v : g[u]) if (--deg[v] == 1) q.push(v);
        }
    }
    vector<int> center;
    while (!q.empty()) {
        center.push_back(q.front());
        q.pop();
    }
    return center;
}

TreeHash unrooted_tree_hash(const vector<vector<int>>& g) {
    auto c = tree_centers(g);
    if (c.size() == 1) return rooted_hash(c[0], -1, g);
    return combine_hash({rooted_hash(c[0], c[1], g), rooted_hash(c[1], c[0], g)});
}

```

06 图论基础：遍历、最短路与生成树

先放邻接表、BFS、最短路、差分约束、生成树、Kruskal 重构树、树形图和生成树计数。

图的邻接表

赛时先看

题目信号： $n, m \leq 1e5/1e6$ ，边比较稀疏。

本质： 大多数图论题的存图方式。

接法： 把本节 struct/class/函数组 整段抄到 `solve()` 上面；优先调用下面 API 列出的入口，其余 helper 默认不要从 `solve()` 直接调。

复杂度判定： 建图 $O(m)$ 。

维护的量： `g[u]`（点 u 的所有出边终点，无权图）；`wg[u]`（点 u 的（终点，边权）出边，有权图）。

警告： 无向图要加两次边；有权图用 `pair<int, i64>`。

【最小完整示例（先抄这一段就能跑）】

题目： n 点 m 边，读边建好 `g` / `wg` 后，所有图论模板直接拿来用。

```
int n, m;
cin >> n >> m;
vector<vector<int>> g(n + 1);           // 1. 无权图邻接表
vector<vector<pair<int, i64>>> wg(n + 1); // 2. 带权图邻接表 (终点, 边权)
for (int i = 0; i < m; ++i) {
    int u, v;
    i64 w;
    cin >> u >> v >> w;
    g[u].push_back(v); wg[u].push_back({v, w}); // 3. 有向边
    g[v].push_back(u); wg[v].push_back({u, w}); // 无向图补反向边
}
```

样例: $n=3, m=2$ 边 $1-2(5), 2-3(5) \rightarrow g[1]=\{2\}, wg[1]=\{(2,5)\}$ 。

【传参要求（照这个传不会错）】

- 下标: 点编号 $1..n$, 数组开 $n+1$; 题给 $0..n-1$ 就把数组开 n 个、编号直接用。
- `g[u].push_back(v)`: 无权有向边 $u \rightarrow v$; 无向图再 `g[v].push_back(u)`。
- `wg[u].push_back({v, w})`: 带权有向边 $u \rightarrow v$, 权 w 用 `i64`; 无向图再 `wg[v].push_back({u, w})`。
- 返回值: 无; 建好的 `g` / `wg` 直接传给 BFS / Dijkstra 等模板。

【API / 入口函数（赛时只认这里列的名字）】

- `push_back(v) \rightarrow` 无权图。 返回 `g[u].`。

```
int n, m;
vector<vector<int>> g(n + 1);
vector<vector<pair<int, i64>>> wg(n + 1);

// 无权图。
g[u].push_back(v);
g[v].push_back(u);

// 带权图。
wg[u].push_back({v, w});
wg[v].push_back({u, w});
```

BFS 最短步数

赛时先看

题目信号: 每条边代价相同; 问最少走几步。

本质: 无权图最短路、网格最短步数、多源扩散。

接法: 如果题面问“最少走几步/最少操作次数”，并且每一步代价都一样，就把每个状态当点、每次合法操作当边。起点放进 `starts`; 返回的 `dist[x]` 是最少步数，`-1` 表示到不了。网格题把 (x,y) 编号或直接开二维 `dist`。

复杂度判定: $O(n+m)$ 。

维护的量: `dist[v]` (起点到 v 的最少步数, `-1` 表示还没到过); `starts` (全部起点, 先入队并设距离 `0`)。

警告: 多源 BFS 把所有起点先入队并设距离 `0`。

【最小完整示例（先抄这一段就能跑）】

题目: n 点 m 条无权边, 多起点, 问每个点最少几步可达。

```
int n, m;
cin >> n >> m;
vector<vector<int>> g(n + 1);           // 1. 建无权邻接表, 点编号 1..n
for (int i = 0; i < m; ++i) {
    int u, v;
    cin >> u >> v;
    g[u].push_back(v);
    g[v].push_back(u);                 // 无向图补反向边
}
vector<int> starts = {1};              // 2. 起点列表, 多源就多塞几个
auto dist = bfs(n, g, starts);        // 3. 调用: dist[v] = 最少步数
for (int v = 1; v <= n; ++v) cout << dist[v] << " \n"[v == n];
```

样例: $1-2, 2-3, 1-3$, 起点 $\{1\} \rightarrow$ 输出 `0 1 1`。

【传参要求（照这个传不会错）】

- `bfs(n, g, starts)`: n = 点数; `g` = 无权邻接表 (1-indexed, 与 n 配套); `starts` = 起点编号列表 (至少 1 个)。
- 返回值: `vector<int> dist`, `dist[v]` = 离最近起点的最少步数; `-1` = 不可达。
- 只适用于每条边代价都是 1; 边带权换 Dijkstra / 0-1 BFS。

【改板时先认这几个量】

- `g`: 邻接表。
- `dist`: 距离。

```
vector<int> bfs(int n, const vector<vector<int>>& g, vector<int> starts) {
    vector<int> dist(n + 1, -1);
    queue<int> q;
    for (int s : starts) {
        dist[s] = 0;
        q.push(s);
    }
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : g[u]) {
            if (dist[v] != -1) continue;
            dist[v] = dist[u] + 1;
            q.push(v);
        }
    }
    return dist;
}
```

网格 BFS

赛时先看

题目信号: $n*m$ 网格，每步上下左右移动且代价相同。

本质: 迷宫、网格最短步数、连通块。

接法: 把本节 `struct/class/函数组` 整段抄到 `solve()` 上面; `solve()` 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定: $O(nm)$ 。

维护的量: `dist[x][y]` (最近起点到 (x,y) 的最少步数, `-1` 表示没到过); `grid[x][y] == '#'` 视为墙不可走。

警告: 多源 BFS 将所有起点先入队。

【最小完整示例（先抄这一段就能跑）】

题目: $n*m$ 网格 ('.' 可走、'#' 墙)，多起点，问终点最少几步可达。

```
int n, m;
cin >> n >> m;
vector<string> grid(n); // 1. 读网格，下标 (行 0..n-1, 列 0..m-1)
for (auto& s : grid) cin >> s;
vector<pair<int, int>> starts = {{0, 0}}; // 2. 起点 (行, 列)，多源就多塞几个
auto dist = grid_bfs(grid, starts); // 3. 调用: dist[x][y] = 最少步数
cout << dist[n - 1][m - 1] << '\n'; // 4. -1 = 到不了
```

样例: 3×3 网格 `... / .#. / ...`, 起点 $(0,0)$ 终点 $(2,2) \rightarrow$ 输出 4。

【传参要求（照这个传不会错）】

- `grid_bfs(grid, starts)`: `grid` = 行字符串数组 (0-indexed); `starts` = 起点坐标 (行, 列) 列表。
- 返回值: `vector<vector<int>> dist`, `dist[x][y]` = 最近起点到 (x,y) 的最少步数; `-1` = 墙或不可达。
- 只认 '#' 为墙，其余字符一律可走；障碍字符不同时改 `grid[nx][ny] == '#'` 这一处。

```
vector<vector<int>> grid_bfs(const vector<string>& grid, vector<pair<int, int>> starts) {
    int n = (int)grid.size(), m = (int)grid[0].size();
    vector<vector<int>> dist(n, vector<int>(m, -1));
    queue<pair<int, int>> q;
    for (auto [x, y] : starts) {
        dist[x][y] = 0;
        q.push({x, y});
    }
    int dx[4] = {1, -1, 0, 0};
    int dy[4] = {0, 0, 1, -1};
    while (!q.empty()) {
        auto [x, y] = q.front();
        q.pop();
        for (int d = 0; d < 4; ++d) {
            int nx = x + dx[d], ny = y + dy[d];
            if (nx < 0 || nx >= n || ny < 0 || ny >= m) continue;
            if (grid[nx][ny] == '#') continue;
            if (dist[nx][ny] != -1) continue;
            dist[nx][ny] = dist[x][y] + 1;
            q.push({nx, ny});
        }
    }
    return dist;
}
```

0-1 BFS

赛时先看

题目信号：代价只有两种，通常是“不改变方向代价 0，改变方向代价 1”。

本质：边权只有 0 和 1 的最短路。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O(n+m)$ 。

维护的量：`dist[v]`（源点 s 到 v 的最短路）；双端队列 `dq`（0 权边从前面推、1 权边从后面推，保证出队距离单调）。

警告：权 0 边推到 deque 前面，权 1 边推到后面。

【最小完整示例（先抄这一段就能跑）】

题目： n 点 m 边，边权只有 0 或 1，求 s 到各点最短路。

```
vector<vector<pair<int, int>>> g(n + 1); // 1. 建图 (终点, 边权), 边权只能 0/1, 点编号 1..n
for (int i = 0; i < m; ++i) {
    int u, v, w;
    cin >> u >> v >> w;
    g[u].push_back({v, w});
    g[v].push_back({u, w}); // 无向图补反向边
}
auto dist = zero_one_bfs(n, g, 1); // 2. 调用: dist[v] = 1 到 v 的最短路
cout << (dist[t] >= 1e9 ? -1 : dist[t]) << '\n'; // 3. 1e9(INF) = 不可达
```

样例：1-2(0), 2-3(1), 1-3(1), $s=1$, $t=3$ -> 输出 1。

【传参要求（照这个传不会错）】

- `zero_one_bfs(n, g, s)`: n = 点数; g = 邻接表 `vector<vector<pair<int,int>>>`, 边权是 `int` 且只能 0/1 (1-indexed, 与 n 配套); s = 源点。
- 返回值: `vector<int> dist`, `dist[v]` = s 到 v 的最短路; `1e9` (INF) = 不可达。
- 边权出现 >1 的值不能用（正确性只在 0/1 成立）；网格转向问题用本目录“0-1 BFS 网格”版。

【改板时先认这几个量】

- `g`: 邻接表。
- `dist`: 距离。

```
vector<int> zero_one_bfs(int n, const vector<vector<pair<int, int>>>& g, int s) {
    const int INF = 1e9;
    vector<int> dist(n + 1, INF);
    deque<int> dq;
    dist[s] = 0;
    dq.push_front(s);
    while (!dq.empty()) {
        int u = dq.front();
        dq.pop_front();
        for (auto [v, w] : g[u]) {
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                if (w == 0) dq.push_front(v);
                else dq.push_back(v);
            }
        }
    }
    return dist;
}
```

0-1 BFS 网格：转向次数最少

赛时先看

题目信号：问最少转弯次数、镜子数、方向状态。

本质：网格中移动，直走代价 0，转向代价 1。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O(nm * 4)$ 。

维护的量：`dist[x][y][d]`（以方向 d 进入 (x,y) 的最少转弯数）；答案取终点四个方向的最小值。

警告：状态是 (x,y,dir) ，起点四个方向距离都可设 0。

【最小完整示例（先抄这一段就能跑）】

题目：网格从 s 走到 t ，直走代价 0、转向代价 1，问最少转向次数。

```
int n, m;
cin >> n >> m;
vector<string> grid(n); // 1. 读网格, 0-indexed, '#' 为墙
for (auto& s : grid) cin >> s;
pair<int, int> s = {0, 0}; // 2. 起点 (行, 列)
pair<int, int> t = {n - 1, m - 1}; // 2. 终点 (行, 列)
int ans = min_turns_grid(grid, s, t); // 3. 调用: 直接得到最少转向次数
cout << (ans >= 1e9 ? -1 : ans) << "\n"; // 4. 1e9 (INF) = 到不了
```

样例：3x3 全可走网格， $s=(0,0)$ ， $t=(2,2)$ $\rightarrow$ 输出 1（先直走到边再转向一次）。

【传参要求（照这个传不会错）】

- `min_turns_grid(grid, s, t)`: `grid` = 行字符串数组 (0-indexed); `s/t` = 起点/终点 (行, 列)。
- 返回值: `int`, `s` 到 `t` 的最少转弯次数 (直走 0、转向 1); `1e9` (INF) = 不可达。
- 起点四个方向都当初始方向 (距离 0)，不用特判起步方向。
- 若起点就是终点，返回 0，不用单独判。

【改板时先认这几个量】

- `dist`: 三维距离 `dist[x][y][dir]`。

```
int min_turns_grid(const vector<string>& grid, pair<int,int> s, pair<int,int> t) {
    int n = grid.size(), m = grid[0].size();
    const int INF = 1e9;
    int dx[4] = {1, -1, 0, 0};
    int dy[4] = {0, 0, 1, -1};
    vector<dist>(n, vector(m, array<int,4>{INF, INF, INF, INF}));
    deque<tuple<int,int,int>> dq;
    for (int d = 0; d < 4; ++d) {
        dist[s.first][s.second][d] = 0;
        dq.push_back({s.first, s.second, d});
    }
    while (!dq.empty()) {
        auto [x, y, dir] = dq.front();
        dq.pop_front();
        int cur = dist[x][y][dir];
        for (int nd = 0; nd < 4; ++nd) {
            int nx = x + dx[nd], ny = y + dy[nd];
            if (nx < 0 || nx >= n || ny < 0 || ny >= m || grid[nx][ny] == '#') continue;
            int w = (nd == dir ? 0 : 1);
            if (dist[nx][ny][nd] > cur + w) {
                dist[nx][ny][nd] = cur + w;
                if (w == 0) dq.push_front({nx, ny, nd});
                else dq.push_back({nx, ny, nd});
            }
        }
    }
    return *min_element(dist[t.first][t.second].begin(), dist[t.first][t.second].end());
}
```

Dijkstra

赛时先看

题目信号：从单个源点出发求到各点（或到 t ）的最短距离，边权非负；看到“单源最短路 + 边权非负”，无脑 Dijkstra。有负边（或负环）立刻停用这个模板。

本质：维护“已定最短路的点集”：每次取出 `dist` 最小者，非负边权保证它一旦出堆就定型，再用它松弛邻边；重复 n 轮即得全部最短路。

复杂度判定： $O((n+m) \log n)$ ； n, m 到 $2e5$ 轻松过，稀疏图 n 到 $1e6$ 也可；边权全为 1 用 BFS 更短；稠密图可换朴素 $O(n^2)$ 版。

维护的量：`dist`（源点到各点当前最短路）、小根堆 `priority_queue`（存候选（距离，点））。

接法：先建 `vector<vector<pair<int,i64>>> g(n+1)`，每条有向边

`g[u].push_back({v,w})`，无向边再反向加一次；调用 `auto dist = dijkstra(n,g,s)`。问 `s` 到 `t` 就输出 `dist[t]`，若仍为 `LINF` 说明不可达。

警告：有负边不能用（贪心定型失效）；堆里旧状态用 `if (d != dist[u]) continue` 跳过，否则会重复扩展。

【最小完整示例（先抄这一段就能跑）】

题目： n 点 m 条有向边（边权非负），从 s 出发到 t 的最短路。


```
vector<vector<pair<int, i64>>> g(n + 1); // 1. 建图: 邻接表 (终点, 边权)
for (int i = 0; i < m; ++i) {
    int u, v;
    i64 w;
    cin >> u >> v >> w;
    g[u].push_back({v, w}); // 无向边就再 push 一次 {u, w}
}
auto dist = dijkstra(n, g, s); // 2. 调用: 返回 dist[1..n]
if (dist[t] >= LINF) cout << -1 << '\n'; // 3. 不可达
else cout << dist[t] << '\n';
```

样例: 1->2(2), 2->3(3), 1->3(5); s=1, t=3 -> 输出 5。

【传参要求 (照这个传不会错)】

- `dijkstra(n, g, s)`: `n` = 点数; `g` = 邻接表 `vector<vector<pair<int, i64>>>` (0/1-indexed 均可, 但要和 `n` 配套); `s` = 源点。
- 返回值: `vector<i64> dist`, `dist[v]` 是 `s` 到 `v` 的最短路; 仍为 `LINF` 表示不可达。
- 边权必须非负; 有负边换 Bellman-Ford/SPFA。
- 需要输出路径: 另开 `pre[v]` 记录每个点从哪来, 跑完沿 `pre` 倒推。

【不会用就照抄】

```
vector<vector<pair<int, i64>>> g(n + 1);
g[u].push_back({v, w}); // 无向图再补 g[v].push_back({u, w})
auto dist = dijkstra(n, g, s);
cout << dist[t] << '\n';
```

- 只能保证非负边权。有负边不要硬套。
- 距离用 `i64`; 不可达通常是 `LINF`。

【API / 入口函数 (赛时只认这里列的名字)】

- `dijkstra(n, g, s)` -> `g[u]` 存 `{v, w}`; 返回 `dist[1..n]`, 源点为 `s`。

【抄板清单 (照着做就行)】

1. 抄哪段: 整个 `dijkstra` 函数。
2. 构造: 建 `vector<vector<pair<int, i64>>> g(n + 1);`。
3. 加边: 有向边 `g[u].push_back({v, w});`; 无向边再补 `g[v].push_back({u, w});`。
4. 调用: `auto dist = dijkstra(n, g, s);`。
5. 取结果: 输出 `dist[t]`; 仍为 `LINF` 即不可达。

【改造点 (按题目改这几处)】

- 0/1-indexed: 模板按 `1..n` 建数组 (大小 `n+1`); 题给 `0..n-1` 就把数组建 `n` 个 (大小 `n`), 编号 `0..n-1` 直接用。
- 图规模: 点/边极大时保持 `vector` 邻接表 (内存 `O(n+m)`); 边数接近 n^2 的稠密图换朴素 $O(n^2)$ 版。
- 需要路径恢复: 加 `vector<int> pre(n+1)`, 松弛成功时记 `pre[v] = u`, 最后从 `t` 沿 `pre` 倒着走回 `s` 即最短路。
- 多源: 建虚拟源点连 0 边, 或对每个起点各跑一次取 `min`。

【核心逻辑 (改代码时别破坏)】

- 堆里存 (当前距离, 点): 每次只扩展还等于 `dist[u]` 的最新状态。
- 只有非负边权时, “当前最小 `dist` 出堆即可定型”的贪心才成立。

【改板时先认这几个量】

- `g`: 邻接表。
- `dist`: 距离。

```
// 维护的量: dist[u] = 源点 s 到 u 的最短路; 小根堆 pq 存候选 (距离, 点)。
// 不变量: 出堆时 d == dist[u] 的点立即定型; 非负边权保证它之后不会再被改小。
vector<i64> dijkstra(int n, const vector<vector<pair<int, i64>>>& g, int s) {
    vector<i64> dist(n + 1, LINF);
    priority_queue<pair<i64, int>, vector<pair<i64, int>>, greater<pair<i64, int>>> pq;
    dist[s] = 0;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (d != dist[u]) continue; // 过期状态 (已被更短路径覆盖), 跳过
        for (auto [v, w] : g[u]) {
            if (dist[v] > d + w) {
                dist[v] = d + w; // 松弛成功才入堆, 维护 dist 最小性
            }
        }
    }
}
```

```

        pq.push({dist[v], v});
    }
}
}
return dist;
}

```

线段树优化建图：点到区间 / 区间到点最短路

赛时先看

题目信号：题面不断给“一个点能到一整段编号连续的点”或“一整段点能到一个点”，直接逐点连边会达到 $O(nq)$ ；通常要求最后跑 Dijkstra。经典题是 Codeforces 786B Legacy。

本质：有 n 个原始点，边除了 $u \rightarrow v$ 外，还可能是 $u \rightarrow [l, r]$ 或 $[l, r] \rightarrow v$ ；边权非负，求单源最短路。

接法： $n, q \leq 1e5$ ，操作是加普通有向边、从点 u 到编号区间 $[l, r]$ 的边、从 $[l, r]$ 到点 v 的边，最后问源点 s 到所有点。创建 SegmentTreeGraph graph(n)；普通边调用 add_edge(u, v, w)，其余两类分别调用 add_point_to_range、add_range_to_point，最后只输出 graph.dijkstra(s)[1..n]。

复杂度判定：建两棵线段树产生 $O(n)$ 个辅助点和零边；每条区间边拆成 $O(\log n)$ 条边。总图规模 $O(n + q \log n)$ ，之后 Dijkstra 为 $O((n + q \log n) \log(n + q))$ 。

维护的量：out_node/in_node（线段树节点对应的辅助点编号）；g（原始点 + 辅助点的邻接表）；tot（当前总点数）；辅助点不可见，只影响内部图规模。

警告：点到区间与区间到点的辅助边方向相反，必须用两棵树；只适用于所有真实边权非负；返回的距离数组包含辅助点，答案只读取原始点编号 1..n；调用 add_point_to_range / add_range_to_point 前保证 $1 \leq ql \leq qr \leq n$ 。

【最小完整示例（先抄这一段就能跑）】

题目： $n, q \leq 1e5$ ，三类操作（普通边 / 点连区间 / 区间连点），边权非负，问源点 s 到各点最短路。

```

SegmentTreeGraph graph(n);          // 1. 建树，原图点 1..n
for (int i = 0; i < q; ++i) {
    int op;
    cin >> op;
    if (op == 1) { int u, v; i64 w; cin >> u >> v >> w; graph.add_edge(u, v, w); } // 2. 普通边
    if (op == 2) { int u, l, r; i64 w; cin >> u >> l >> r >> w; graph.add_point_to_range(u, l, r, w); } // 3. u -> [l, r]
    if (op == 3) { int l, r, v; i64 w; cin >> l >> r >> v >> w; graph.add_range_to_point(l, r, v, w); } // 4. [l, r] -> v
}
auto dist = graph.dijkstra(s);        // 5. 调用：只读原始点 1..n
cout << (dist[t] >= LINF ? -1 : dist[t]) << '\n';

```

样例： $n=3$ ，操作 2: $1 \rightarrow [2, 3]$ 权 4， $s=1$ ， $t=3 \rightarrow$ 输出 4。

【传参要求（照这个传不会错）】

- SegmentTreeGraph graph(n): n = 原始点数，原始点编号 1..n (1-indexed)。
- add_edge(u, v, w): 普通有向边 $u \rightarrow v$ ，权 w (i64)。
- add_point_to_range(u, ql, qr, w): 点 u 到区间内每个点，权 w ；要求 $1 \leq ql \leq qr \leq n$ 。
- add_range_to_point(ql, qr, v, w): 区间内每个点到 v ，权 w ；要求 $1 \leq ql \leq qr \leq n$ 。
- dijkstra(s): 返回 vector<i64>（含辅助点），答案只读下标 1..n；LINF = 不可达。
- 无向边 = 正反各调一次 add_edge；区间边没有“无向”版本，分两次调。

【API / 入口函数（赛时只认这里列的名字）】

- add_edge(int u, int v, i64 w) $\rightarrow$ 加入一条边
- init(int n_) $\rightarrow$ 初始化/清空结构

【改板时先认这几个量】

- g: 邻接表。
- dist: 距离。

```

struct SegmentTreeGraph {
    struct Edge {
        int to;
        i64 w;
    };

    int n, tot;
    vector<int> out_node, in_node;
    vector<vector<Edge>> g;

    explicit SegmentTreeGraph(int n_) { init(n_); }

    void init(int n_) {

```

```

n = n_;
tot = n; // 原图点编号为 1..n。
out_node.assign(4 * n + 5, 0);
in_node.assign(4 * n + 5, 0);
g.assign(9 * n + 5, {});
build(1, 1, n);
}

int new_node() { return ++tot; }

void add_edge(int u, int v, i64 w) { g[u].push_back({v, w}); }

void build(int p, int l, int r) {
    out_node[p] = new_node(); // 从区间节点连向区间内每个原图点。
    in_node[p] = new_node(); // 从区间内每个原图点连向区间节点。
    if (l == r) {
        add_edge(out_node[p], l, 0);
        add_edge(l, in_node[p], 0);
        return;
    }
    int mid = (l + r) >> 1;
    build(p << 1, l, mid);
    build(p << 1 | 1, mid + 1, r);

    add_edge(out_node[p], out_node[p << 1], 0);
    add_edge(out_node[p], out_node[p << 1 | 1], 0);
    add_edge(in_node[p << 1], in_node[p], 0);
    add_edge(in_node[p << 1 | 1], in_node[p], 0);
}

void add_point_to_range(int u, int ql, int qr, i64 w) {
    add_point_to_range(1, 1, n, u, ql, qr, w);
}

void add_point_to_range(int p, int l, int r, int u, int ql, int qr, i64 w) {
    if (ql <= l && r <= qr) {
        add_edge(u, out_node[p], w);
        return;
    }
    int mid = (l + r) >> 1;
    if (ql <= mid) add_point_to_range(p << 1, l, mid, u, ql, qr, w);
    if (qr > mid) add_point_to_range(p << 1 | 1, mid + 1, r, u, ql, qr, w);
}

void add_range_to_point(int ql, int qr, int v, i64 w) {
    add_range_to_point(1, 1, n, ql, qr, v, w);
}

void add_range_to_point(int p, int l, int r, int ql, int qr, int v, i64 w) {
    if (ql <= l && r <= qr) {
        add_edge(in_node[p], v, w);
        return;
    }
    int mid = (l + r) >> 1;
    if (ql <= mid) add_range_to_point(p << 1, l, mid, ql, qr, v, w);
    if (qr > mid) add_range_to_point(p << 1 | 1, mid + 1, r, ql, qr, v, w);
}

vector<i64> dijkstra(int source) const {
    vector<i64> dist(tot + 1, LINF);
    priority_queue<pair<i64, int>, vector<pair<i64, int>>, greater<pair<i64, int>>> pq;
    dist[source] = 0;
    pq.push({0, source});
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (d != dist[u]) continue;
        for (const auto& e : g[u]) {
            if (d <= LINF - e.w && d + e.w < dist[e.to]) {
                dist[e.to] = d + e.w;
                pq.push({dist[e.to], e.to});
            }
        }
    }
    return dist;
}
};

```

典题模型： $n, q \leq 1e5$ ，操作是加普通有向边、从点 u 到编号区间 $[l, r]$ 的边、从 $[l, r]$ 到点 v 的边，最后问源点 s 到所有点。创建 `SegmentTreeGraph graph(n)`；普通边调用 `add_edge(u, v, w)`，其余两类分别调用 `add_point_to_range`、`add_range_to_point`，最后只输出 `graph.dijkstra(s)[1..n]`。

Bellman-Ford

赛时先看

题目信号：边权可能为负； n, m 不大；差分约束。

本质：有负边的单源最短路、判断源点可达负环。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里先构造对象，再按下方“不会用就看这里”调用公开接口。

复杂度判定： $O(nm)$ 。

维护的量：`dist[v]`（源点 s 到 v 的当前最短路）；`edges`（边列表，每轮全量扫一遍松弛）。

警告：第 n 轮还能松弛，说明存在可达负环。

【最小完整示例（先抄这一段就能跑）】

题目： n 点 m 边（边权可为负），从 s 求最短路并判负环。

```
vector<Edge> edges; // 1. 只存边列表，不用建邻接表，点编号 1..n
for (int i = 0; i < m; ++i) {
    int u, v;
    i64 w;
    cin >> u >> v >> w;
    edges.push_back({u, v, w});
}
vector<i64> dist;
if (!bellman_ford(n, edges, s, dist)) cout << "neg cycle\n"; // 2. false = 有可达负环
else cout << (dist[t] == (1LL << 62) ? -1 : dist[t]) << '\n'; // 3. INF = 不可达
```

样例：1-2(-1), 2-3(2), 1-3(10), $s=1$, $t=3$ → 输出 1。

【传参要求（照这个传不会错）】

- `bellman_ford(n, edges, s, dist)`: n = 点数；`edges` = 边列表 `Edge{u,v,w}`（1-indexed，权可为负）； s = 源点；`dist` 传一个空 `vector<i64>`，函数负责赋值。
- 返回值：`true` = 无负环，`dist[v]` 即最短路；`false` = 有从 s 可达的负环（此时 `dist` 无效）。
- `dist[v] == (1LL << 62)` (INF) = 从 s 不可达。
- 无向负权边等价于两条反向有向边 = 负环，不要建。

```
struct Edge {
    int u, v;
    i64 w;
};

bool bellman_ford(int n, const vector<Edge>& edges, int s, vector<i64>& dist) {
    const i64 INF = (1LL << 62);
    dist.assign(n + 1, INF);
    dist[s] = 0;
    for (int i = 1; i <= n - 1; ++i) {
        bool changed = false;
        for (auto [u, v, w] : edges) {
            if (dist[u] == INF) continue;
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                changed = true;
            }
        }
        if (!changed) break;
    }
    for (auto [u, v, w] : edges) {
        if (dist[u] != INF && dist[v] > dist[u] + w) return false;
    }
    return true;
}
```

Floyd

赛时先看

题目信号： $n \leq 500$ ，多次询问任意两点最短路。

本质：全源最短路。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O(n^3)$ 。

维护的量：`d[i][j]`（ i 到 j 的当前最短路，跑完即全源最短路）；每轮枚举中转点 k 做松弛。

警告：初始化 `d[i][i]=0`，无边设为 `INF`，跳过 `INF` 相加。

【最小完整示例（先抄这一段就能跑）】

题目： `n ≤ 500`，多次询问任意两点最短路，边权可为负（无负环）。

```
const i64 INF = (1LL << 62);
vector<vector<i64>> d(n + 1, vector<i64>(n + 1, INF)); // 1. 邻接矩阵, 1-indexed
for (int i = 1; i <= n; ++i) d[i][i] = 0;
for (int i = 0; i < m; ++i) {
    int u, v;
    i64 w;
    cin >> u >> v >> w;
    d[u][v] = min(d[u][v], w); // 2. 有向边; 重边取最小; 无向边再补 d[v][u]
}
floyd(d, n); // 3. 调用: 原地更新 d
cout << (d[s][t] >= INF / 2 ? -1 : d[s][t]) << '\n'; // 4. INF/2 以上 = 不可达
```

样例： `1->2(2)`, `2->3(3)`, `1->3(10)`, `s=1, t=3` -> 输出 `5`。

【传参要求（照这个传不会错）】

- `floyd(d, n)`: `d = n+1` 行 `n+1` 列矩阵（1-indexed），`d[i][i]=0`、无边为 `INF`；`n` = 点数。
- 函数原地修改 `d`：跑完 `d[i][j]` = `i` 到 `j` 的最短路；`d[i][j] >= INF/2` = 不可达。
- 有负环时跑完 `d[i][i] < 0`；要判断负环就检查这一条。
- `n` 到 1000 都勉强 ($O(n^3)$ 约 $1e9$)，更大换 Johnson / 多次 Dijkstra。

```
void floyd(vector<vector<i64>>& d, int n) {
    const i64 INF = (1LL << 62);
    for (int k = 1; k <= n; ++k) {
        for (int i = 1; i <= n; ++i) {
            if (d[i][k] == INF) continue;
            for (int j = 1; j <= n; ++j) {
                if (d[k][j] == INF) continue;
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
            }
        }
    }
}
```

min-plus 矩阵快速幂：恰好 / 至多 k 条边最短路

赛时先看

题目信号：题面强调“恰好走 k 次转移/航班/边”， k 极大（如 $1e18$ ），但点数较小，常见 `n ≤ 60~150`；也常和自动机、有限状态 DP 联动。

本质：在加权有向图中，求每一对点之间恰好经过 k

条边的最短路。把普通矩阵乘法的“乘法+加法”替换为“加法+取最小值”，再二进制快速幂。

接法：城市之间可乘坐航班，必须恰好乘 k 班，问 `s` 到 `t` 的最低票价， $k ≤ 1e18$ 而城市数只有几十。初始化 `MinPlusMatrix trans(n)`，将每条航班写入 `trans.a[u][v]`，`min_plus_power(trans, k).a[s][t]` 即为答案；若值仍至少为 `LINF/2` 则不可达。

复杂度判定：朴素 min-plus 乘法 $O(n^3)$ ，快速幂总计 $O(n^3 \log k)$ ，空间 $O(n^2)$ 。

维护的量：`a[i][j]` (`i` 到 `j` 恰好走“当前步数”的最短路)；快速幂中 `base` 每轮自乘翻倍、`ans` 按二进制位累计。

警告：这不是普通最短路， $k=0$ 时只有 `i -> i` 的答案为 `0`；平行边只保留最小权；“至多 k 条边”是在每个点加一条权值 `0` 自环后，仍然求恰好 k 条边；要保证任何合法路径的绝对权值和远小于 `LINF`。

【最小完整示例（先抄这一段就能跑）】

题目： `n` 城航班，恰好乘 k 班 ($k ≤ 1e18$)，问 `s` 到 `t` 最低票价。

```
MinPlusMatrix trans(n); // 1. n x n 全 LINF, 下标 0..n-1 (0-indexed)
for (int i = 0; i < m; ++i) {
    int u, v;
    i64 w;
    cin >> u >> v >> w;
    trans.a[u][v] = min(trans.a[u][v], w); // 2. 有向边; 平行边只留最小权
}
MinPlusMatrix res = min_plus_power(trans, k); // 3. 恰好 k 条边的转移矩阵
i64 ans = res.a[s][t];
cout << (ans >= LINF / 2 ? -1 : ans) << '\n'; // 4. 不可达
```

样例： `n=3`，边 `0->1(1)`, `1->2(2)`, `0->2(5)`, `k=2, s=0, t=2` -> 输出 `3` (`0->1->2`)。

【传参要求（照这个传不会错）】

- `MinPlusMatrix trans(n)`: `n` = 点数, 下标 `0..n-1`; 初值全 `LINF`。
- `trans.a[u][v] = w`: 有向边 `u -> v` 权 `w`; 平行边手动 `min`; 无向边再写 `trans.a[v][u] = w`。
- `min_plus_power(trans, k)`: 返回“恰好 `k` 条边”的矩阵, 答案取 `.a[s][t]`; $\geq \text{LINF}/2$ = 不可达。
- 要“至多 `k` 条边”: 先 `trans.a[i][i] = min(trans.a[i][i], 0LL)` (`0` 权自环) 再跑同样的代码。
- `k=0`: 答案 `0` 当且仅当 `s == t`, 否则不可达。

```
struct MinPlusMatrix {
    int n;
    vector<vector<i64>> a;

    explicit MinPlusMatrix(int n_, i64 init = LINF) : n(n_), a(n_, vector<i64>(n_, init)) {}

    static MinPlusMatrix identity(int n) {
        MinPlusMatrix res(n);
        for (int i = 0; i < n; ++i) res.a[i][i] = 0;
        return res;
    }
};

MinPlusMatrix min_plus_multiply(const MinPlusMatrix& x, const MinPlusMatrix& y) {
    int n = x.n;
    MinPlusMatrix z(n);
    for (int i = 0; i < n; ++i) {
        for (int k = 0; k < n; ++k) {
            if (x.a[i][k] >= LINF / 2) continue;
            for (int j = 0; j < n; ++j) {
                if (y.a[k][j] >= LINF / 2) continue;
                z.a[i][j] = min(z.a[i][j], x.a[i][k] + y.a[k][j]);
            }
        }
    }
    return z;
}

MinPlusMatrix min_plus_power(MinPlusMatrix base, i64 exp) {
    MinPlusMatrix ans = MinPlusMatrix::identity(base.n);
    while (exp > 0) {
        if (exp & 1) ans = min_plus_multiply(ans, base);
        base = min_plus_multiply(base, base);
        exp >>= 1;
    }
    return ans;
}

// 示例: 对每条费用为 w 的有向边 u -> v:
// 示例: trans.a[u][v] = min(trans.a[u][v], w);
// 示例: exactly_k = min_plus_power(trans, k);
// 若要求至多 k 条边, 先把 trans.a[i][i] 与 0 取 min。
```

典题模型: 城市之间可乘坐航班, 必须恰好乘 `k` 班, 问 `s` 到 `t` 的最低票价, $k \leq 1e18$ 而城市数只有几十。初始化 `MinPlusMatrix trans(n)`, 将每条航班写入 `trans.a[u][v]`, `min_plus_power(trans, k).a[s][t]` 即为答案; 若值仍至少为 `LINF/2` 则不可达。

Johnson 全源最短路

赛时先看

题目信号: `n` 较大、`m` 较稀疏, Floyd 过不了; 有负边。

本质: 稀疏有向图全源最短路, 允许负边但不能有负环。

接法: 把本节 `struct/class/函数组` 整段抄到 `solve()` 上面; `solve()`

里先构造对象, 再按下方“不会用就看这里”调用公开接口。

复杂度判定: $O(nm \log n)$ 。

维护的量: `h[v]` (势能, 把负权边重标成非负用); `dist[s][v]` (`s` 到 `v` 的真实最短路, 跑完要减回 `h[v]-h[s]`)。

警告: 先用 SPFA/Bellman-Ford 求势能 `h`; 重标边权 `w+h[u]-h[v]` 必须非负。

【最小完整示例 (先抄这一段就能跑)】

题目: `n` 较大、`m` 稀疏、边权可为负 (无负环), 求所有点对最短路。

```
vector<JEdge> edges; // 1. 边列表 {u, v, w}, 点编号 1..n
for (int i = 0; i < m; ++i) {
    int u, v;
    i64 w;
    cin >> u >> v >> w;
    edges.push_back({u, v, w});
}
```

```
auto dist = johnson(n, edges); // 2. 调用: dist[s][v] = s 到 v 的真实最短路
if (dist.empty()) { cout << "neg cycle\n"; return; } // 3. 空数组 = 有负环
cout << (dist[s][t] == (1LL << 60) ? -1 : dist[s][t]) << '\n'; // 4. INF = 不可达
```

样例: 1-2(-1), 2-3(2), 1-3(10), s=1, t=3 -> 输出 1。

【传参要求（照这个传不会错）】

- johnson(n, edges): n = 点数; edges = 边列表 JEdge{u,v,w} (1-indexed, 权可为负)。
- 返回值: vector<vector<i64>> (n+1 x n+1), dist[s][v] = s 到 v 的真实最短路; (1LL << 60) (INF) = 不可达。
- 返回空数组 = 图中存在负环, 无最短路定义。
- 无向带负权边 = 两条反向有向边 = 负环, 直接判无解。

【改板时先认这几个量】

- g: 邻接表。
- dist: 距离。

```
struct JEdge { int u, v; i64 w; };

vector<vector<i64>> johnson(int n, vector<JEdge> edges) {
    const i64 INF = (1LL << 60);
    vector<i64> h(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        bool changed = false;
        for (auto e : edges) {
            if (h[e.v] > h[e.u] + e.w) {
                h[e.v] = h[e.u] + e.w;
                changed = true;
            }
        }
        if (!changed) break;
        if (i == n) return {}; // 负环。
    }
    vector<vector<pair<int, i64>>> g(n + 1);
    for (auto e : edges) {
        g[e.u].push_back({e.v, e.w + h[e.u] - h[e.v]});
    }
    vector<vector<i64>> dist(n + 1, vector<i64>(n + 1, INF));
    for (int s = 1; s <= n; s++) {
        priority_queue<pair<i64, int>, vector<pair<i64, int>>, greater<>> pq;
        dist[s][s] = 0;
        pq.push({0, s});
        while (!pq.empty()) {
            auto [d, u] = pq.top(); pq.pop();
            if (d != dist[s][u]) continue;
            for (auto [v, w] : g[u]) {
                if (dist[s][v] > d + w) {
                    dist[s][v] = d + w;
                    pq.push({dist[s][v], v});
                }
            }
        }
        for (int v = 1; v <= n; v++) {
            if (dist[s][v] < INF) dist[s][v] += h[v] - h[s];
        }
    }
    return dist;
}
```

SPFA 与负环判定

赛时先看

题目信号: 边权可能为负; 需要判断是否存在负环。

本质: 含负边图的最短路、判断负环、差分约束。

复杂度判定: 平均较快, 最坏 $O(nm)$ 。

维护的量: dist[v] (s 到 v 的当前最短路); cnt[v] (v 入队次数, $\geq n$ 即有负环); inq[v] (v 是否在队列里)。

警告: SPFA 会被卡; 能用 Dijkstra/Bellman-Ford 时优先用更稳的。

【最小完整示例（先抄这一段就能跑）】

题目: 边权可为负, 求 s 到各点最短路并判断 s 可达的负环。

```
vector<vector<pair<int, i64>>> g(n + 1); // 1. 邻接表 (终点, 边权), 点编号 1..n
for (int i = 0; i < m; ++i) {
    int u, v;
    i64 w;
```



```

    cin >> u >> v >> w;
    g[u].push_back({v, w});
    g[v].push_back({u, w});           // 无向图补反向边
}
vector<i64> dist;
if (!spfa(n, g, s, dist)) cout << "neg cycle\n"; // 2. false = 有可达负环
else cout << (dist[t] == (1LL << 62)) ? -1 : dist[t] << '\n'; // 3. INF = 不可达

```

样例：1-2(-1), 2-3(2), 1-3(10), s=1, t=3 -> 输出 1。

【传参要求（照这个传不会错）】

- spfa(n, g, s, dist): n = 点数; g = 邻接表 `vector<vector<pair<int,i64>>>` (1-indexed, 权可为负); s = 源点; dist 传空 `vector<i64>`, 函数负责赋值。
- 返回值: true = 无负环, dist[v] 即最短路; false = 有从 s 可达的负环。
- dist[v] == (1LL << 62) (INF) = 从 s 不可达。
- 只判"s 可达"的负环; 要判全图负环先加超级源连所有点 (见差分约束)。

【改板时先认这几个量】

- g: 邻接表。
- dist: 距离。

```

bool spfa(int n, const vector<vector<pair<int, i64>>>& g, int s,
          vector<i64>& dist) {
    const i64 INF = (1LL << 62);
    dist.assign(n + 1, INF);
    vector<int> inq(n + 1, 0), cnt(n + 1, 0);
    queue<int> q;
    dist[s] = 0;
    q.push(s);
    inq[s] = 1;

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        inq[u] = 0;
        for (auto [v, w] : g[u]) {
            if (dist[u] != INF && dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                if (!inq[v]) {
                    q.push(v);
                    inq[v] = 1;
                    if (++cnt[v] >= n) return false; // 存在从源点可达的负环。
                }
            }
        }
    }
    return true;
}

```

差分约束

赛时先看

题目信号: 变量之间是差值限制, 如 $x_i - x_j \leq c$ 。

本质: 求一组不等式 $x[v] \leq x[u] + w$ 是否有解, 并给出一组解。

接法: 把本节 struct/class/函数组 整段抄到 solve() 上面; 外部只调用下面 API 列出的接口, 不要把内部递归参数直接当题面参数。

复杂度判定: 建图后 Bellman-Ford/SPFA。

维护的量: g (差分约束图: 边 $u \rightarrow v$ 权 w 表示 $x[v] \leq x[u] + w$); n (变量个数)。

警告: 约束 $x[v] \leq x[u] + w$ 建边 $u \rightarrow v$ 权 w; 超级源连所有点权 0。

【最小完整示例（先抄这一段就能跑）】

题目: 给 m 条约束 $x[v] - x[u] \leq w$, 求一组可行解或判定无解。

```

DiffConstraint dc(n);           // 1. 建空图, 变量编号 1..n
for (int i = 0; i < m; ++i) {
    int u, v;
    i64 w;
    cin >> u >> v >> w;
    dc.add_constraint(u, v, w); // 2. 约束 x[v] - x[u] <= w
}
vector<i64> x(n + 1);
if (!dc.solve(x)) cout << "no solution\n"; // 3. false = 无解
else for (int v = 1; v <= n; ++v) cout << x[v] << " \n"[v == n]; // 4. 一组可行解

```

样例：约束 $x_2 - x_1 \leq 3, x_3 - x_2 \leq 1 \rightarrow$ 输出一组解 0 3 4。

【传参要求（照这个传不会错）】

- `DiffConstraint dc(n)`: n = 变量个数，编号 1.. n (1-indexed)。
- `add_constraint(u, v, w)`: 约束 $x[v] - x[u] \leq w$ (建边 $u \rightarrow v$ 权 w)； $\geq w$ 的约束改写为 `add_constraint(v, u, -w)`。
- `solve(x)`: x 传长度为 $n+1$ 的 `vector<i64>`，函数填一组解；返回 `true` = 有解，`false` = 无解（负环）。
- $x[v] - x[u] == w$: 拆成 $\leq w$ 和 $\geq w$ 两条约束。

【API / 入口函数（赛时只认这里列的名字）】

- `init(int n_)` $\rightarrow$ 初始化/清空结构
- `solve(vector<i64>& x)` $\rightarrow$ 执行主算法并返回答案

```
struct DiffConstraint {
    int n;
    vector<vector<pair<int, i64>>> g;

    DiffConstraint(int n = 0) { init(n); }

    void init(int n_) {
        n = n_;
        g.assign(n + 1, {});
    }

    void add_constraint(int u, int v, i64 w) {
        // 差分约束: x[v] <= x[u] + w。
        g[u].push_back({v, w});
    }

    bool solve(vector<i64>& x) {
        vector<vector<pair<int, i64>>> gg = g;
        gg.push_back({});
        int s = n + 1;
        gg.resize(n + 2);
        for (int i = 1; i <= n; ++i) gg[s].push_back({i, 0});
        return spfa(n + 1, gg, s, x);
    }
};
```

最短路与次短路

赛时先看

题目信号：题目问“第二短路径”“严格比最短长的最短路”。

本质：求从源点到每个点的最短和严格次短距离。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；`solve()` 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O((n+m) \log n)$ 。

维护的量：`dist[v][0]`（最短路）、`dist[v][1]`（严格次短路），1-indexed，不可达为 `INF`。

警告：次短必须严格大于最短；相等路径计数是另一类问题。

【最小完整示例（先抄这一段就能跑）】

```
// n=4 个点 (1..4)，s=1；边：1-2 权1，2-3 权2，1-3 权4
vector<vector<pair<int, i64>>> g(5);
g[1].push_back({2, 1}); g[2].push_back({1, 1});
g[2].push_back({3, 2}); g[3].push_back({2, 2});
g[1].push_back({3, 4}); g[3].push_back({1, 4});
auto dist = dijkstra_second(4, g, 1);
// 到 3：最短 dist[3][0]=3 (1-2-3)，次短 dist[3][1]=4 (1-3)
```

【传参要求（照这个传不会错）】

- n : 点数，顶点编号 1.. n 。
- g : 邻接表， $g[u]$ 存 $\{v, w\}$ 表示边 $u \rightarrow v$ 权 w ，1-indexed，边权非负。
- s : 源点，范围 $[1, n]$ 。
- 返回 `vector<array<i64, 2>>`: `dist[v][0]` 最短路、`dist[v][1]` 严格次短路；不可达为 `INF = (1LL<<62)`。

【改板时先认这几个量】

- g : 邻接表。

- `dist`: 距离。

```
vector<array<i64, 2>> dijkstra_second(int n, const vector<vector<pair<int, i64>>>& g, int s) {
    const i64 INF = (1LL << 62);
    vector<array<i64, 2>> dist(n + 1, {INF, INF});
    priority_queue<pair<i64, int>, vector<pair<i64, int>>, greater<pair<i64, int>>> pq;
    dist[s][0] = 0;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (d > dist[u][1]) continue;
        for (auto [v, w] : g[u]) {
            i64 nd = d + w;
            if (nd < dist[v][0]) {
                dist[v][1] = dist[v][0];
                dist[v][0] = nd;
                pq.push({nd, v});
            } else if (dist[v][0] < nd && nd < dist[v][1]) {
                dist[v][1] = nd;
                pq.push({nd, v});
            }
        }
    }
    return dist;
}
```

所有最短路条数

赛时先看

题目信号: 问“最短路径有多少条”，边权非负。

本质: 求最短路长度同时统计最短路数量。

复杂度判定: $O((n+m) \log n)$ 。

维护的量: `dist[v]` (最短路长度)、`cnt[v]` (最短路条数, 对 `mod` 取模), 1-indexed。

警告: 相同最短距离时累加方案数。

【最小完整示例（先抄这一段就能跑）】

```
// n=4 个点 (1..4), s=1; 边: 1-2 权1, 2-3 权1, 1-3 权2
vector<vector<pair<int, i64>>> g(5);
g[1].push_back({2, 1}); g[2].push_back({1, 1});
g[2].push_back({3, 1}); g[3].push_back({2, 1});
g[1].push_back({3, 2}); g[3].push_back({1, 2});
auto [dist, cnt] = dijkstra_count_paths(4, g, 1, (i64)1e9 + 7);
// 到 3: dist[3]=2, cnt[3]=2 (1-2-3 与 1-3 两条)
```

【传参要求（照这个传不会错）】

- `n`: 点数, 顶点编号 1..n。
- `g`: 邻接表, `g[u]` 存 `{v, w}`, 1-indexed, 边权非负。
- `s`: 源点, 范围 `[1, n]`。
- `mod`: 方案数取模的模数。
- 返回 `{dist, cnt}`: `dist[v]` 最短路长 (不可达为 `INF`), `cnt[v]` 最短路条数 % `mod`。

【改板时先认这几个量】

- `g`: 邻接表。
- `dist`: 距离。

```
pair<vector<i64>, vector<i64>> dijkstra_count_paths(
    int n,
    const vector<vector<pair<int, i64>>>& g,
    int s,
    i64 mod
) {
    const i64 INF = (1LL << 62);
    vector<i64> dist(n + 1, INF), cnt(n + 1, 0);
    priority_queue<pair<i64, int>, vector<pair<i64, int>>, greater<pair<i64, int>>> pq;
    dist[s] = 0;
    cnt[s] = 1;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (d != dist[u]) continue;
        for (auto [v, w] : g[u]) {
            i64 nd = d + w;
```

```

        if (nd < dist[v]) {
            dist[v] = nd;
            cnt[v] = cnt[u];
            pq.push({nd, v});
        } else if (nd == dist[v]) {
            cnt[v] = (cnt[v] + cnt[u]) % mod;
        }
    }
}
return {dist, cnt};
}

```

Karp: 最小平均权环

赛时先看

题目信号: 长期平均成本、重复执行一段操作、找平均收益最高/平均代价最低的循环；题面会暗示“循环一次的平均值”。

本质: 求有向图中平均边权最小的环，即最小化 环总权 / 环边数。

接法: 任务可反复转移，求长期每步最小平均耗时；或在收益图中找最大平均收益环（将边权取负）。

复杂度判定: $O(nm)$ ，空间 $O(n^2)$ ；适合 n, m 在几千量级或更小的图。

维护的量: $dp[len][v]$ （恰好走 len 条边到达 v 的最小总权，0-based），所有答案只由环决定。

警告: 它和最短路不同，答案只由环决定；初始化 $dp[0][v]=0$ 才能枚举图中任意连通部分的环。无环图返回 INF 。

【最小完整示例（先抄这一段就能跑）】

```

// n=3 个点 (0..2): 0->1 权1, 1->2 权1, 2->0 权2
vector<MeanCycleEdge> edges = {{0, 1, 1}, {1, 2, 1}, {2, 0, 2}};
long double ans = minimum_mean_cycle(3, edges);
// 唯一环 0->1->2->0, 平均权 (1+1+2)/3 = 4/3

```

【传参要求（照这个传不会错）】

- n : 顶点数，编号 $0..n-1$ (0-based)。
- $edges$: 每条 $\{u, v, w\}$, $u, v \in [0, n)$, w 为有向边权。
- 返回最小平均环权 ($long\ double$)；无环返回 $INF = 1e100L$ 。找最大平均收益环时把 w 全部取负再调用。

约定: 顶点 0-based。

典题模型：任务可反复转移，求长期每步最小平均耗时；或在收益图中找最大平均收益环（将边权取负）。

```

struct MeanCycleEdge {
    int u, v;
    long double w;
};

long double minimum_mean_cycle(int n, const vector<MeanCycleEdge>& edges) {
    const long double INF = 1e100L;
    vector<vector<long double>>> dp(n + 1, vector<long double>(n, INF));
    for (int v = 0; v < n; ++v) dp[0][v] = 0; // 超级源连向所有点，边权 0
    for (int len = 1; len <= n; ++len) {
        for (const auto& e : edges) {
            if (dp[len - 1][e.u] < INF / 2) {
                dp[len][e.v] = min(dp[len][e.v], dp[len - 1][e.u] + e.w);
            }
        }
    }
    long double ans = INF;
    for (int v = 0; v < n; ++v) if (dp[n][v] < INF / 2) {
        long double worst = -INF;
        for (int len = 0; len < n; ++len) if (dp[len][v] < INF / 2) {
            worst = max(worst, (dp[n][v] - dp[len][v]) / (n - len));
        }
        ans = min(ans, worst);
    }
    return ans; // INF 表示没有环
}

```

最小生成树: Kruskal

赛时先看

题目信号: 题面问“铺路/建网/连通所有点/最小费用”；边较少。

本质: 无向图连接所有点的最小总代价。

接法: 把所有候选连接方案都放进 $edges$ ，每条是 $\{u, v, cost\}$ ；调用 $auto [cost, used] = kruskal(n, edges)$ 。只有 $used == n - 1$ 时所有点才被连通，否则按题意输出 `impossible` 或特殊值。MST

解决的是“选边把所有点连起来”，不是“从 A 到 B 的最短路”。

复杂度判定: $O(m \log m)$ 。

维护的量: `cost` (已选边总权)、`used` (已选边数); 内部 `dsu` 判两点是否已连通。

警告: 用并查集; 最终选边数必须是 $n-1$, 否则图不连通。

【最小完整示例 (先抄这一段就能跑)】

```
// n=4 个点 (1..4), 边: 1-2 权1, 2-3 权2, 3-4 权3, 1-4 权10
vector<MstEdge> edges = {{1, 2, 1}, {2, 3, 2}, {3, 4, 3}, {1, 4, 10}};
auto [cost, used] = kruskal(4, edges);
// cost = 6, used = 3 == n-1, 图连通
```

【传参要求 (照这个传不会错)】

- `n`: 点数, 顶点编号 $1..n$ 。
- `edges`: 每条 $\{u, v, w\}$ 表示 $u-v$ 无向边权 w , $u, v \in [1, n]$ 。
- 返回 `pair<i64, int>`: `cost` 为 MST 总权, `used` 为选边数; `used == n-1` 才连通, 否则按题意输出失败标记。

【API / 入口函数 (赛时只认这里列的名字)】

- `kruskal(n, edges)` -> 主入口; 返回 `{cost, used}`, `used == n-1` 表示连通。

【改板时先认这几个量】

- `sz`: 集合/子树大小。
- `dsu`: 内部并查集实例 (按大小合并, 用于生成树判定)。

```
struct DSU {
    vector<int> p, sz;
    DSU(int n = 0) { init(n); }
    void init(int n) { p.resize(n + 1); sz.assign(n + 1, 1); iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool unite(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a; sz[a] += sz[b];
        return true;
    }
};

struct MstEdge {
    int u, v;
    i64 w;
};

pair<i64, int> kruskal(int n, vector<MstEdge> edges) {
    sort(edges.begin(), edges.end(), [](const MstEdge& a, const MstEdge& b) {
        return a.w < b.w;
    });
    DSU dsu(n);
    i64 cost = 0;
    int used = 0;
    for (auto e : edges) {
        if (dsu.unite(e.u, e.v)) {
            cost += e.w;
            used++;
        }
    }
    return {cost, used}; // used == n-1 表示最小生成树已经连通。
}
```

最小生成树: Prim

赛时先看

题目信号: n 较小、边很多、给的是矩阵。

本质: 稠密图 MST, 或用邻接矩阵更方便时。

接法: 如果题目给的是完整代价矩阵 `w[i][j]`, 直接用这个矩阵版 Prim; 不存在的边设为 `INF`, `w[i][i]=0`。返回 `-1` 说明无法连通。边很少时 Kruskal 更短更稳。

复杂度判定: 矩阵版 $O(n^2)$ 。

维护的量: `dist[v]` (v 到已选集合的最小边权)、`used[v]` (是否已加入 MST), 1-indexed。

警告: 每轮找未加入集合且距离最小的点; 找不到说明不连通。

【最小完整示例 (先抄这一段就能跑)】

```
// n=3 个点 (1..3), 代价矩阵: 1-2 权5, 2-3 权2, 1-3 权7
vector<vector<i64>> w(4, vector<i64>(4, (1LL << 62))));
```

```
for (int i = 1; i <= 3; i++) w[i][i] = 0;
w[1][2] = w[2][1] = 5; w[2][3] = w[3][2] = 2; w[1][3] = w[3][1] = 7;
i64 ans = prim_dense(w, 3);
// ans = 7 (选 1-2 与 2-3)
```

【传参要求（照这个传不会错）】

- w : $(n+1) \times (n+1)$ 对称代价矩阵 $w[i][j]$, 1-indexed; 不存在的边设为 INF , $w[i][i]=0$ 。
- n : 点数。
- 返回 MST 总权 $i64$; 图不连通返回 -1 。

【改板时先认这几个量】

- $dist$: 到已选集合的最小边权。
- $used$: 是否已加入 MST。

```
i64 prim_dense(const vector<vector<i64>>& w, int n) {
    const i64 INF = (1LL << 62);
    vector<i64> dist(n + 1, INF);
    vector<int> used(n + 1, 0);
    dist[1] = 0;
    i64 ans = 0;
    for (int it = 1; it <= n; ++it) {
        int u = -1;
        for (int i = 1; i <= n; ++i) {
            if (!used[i] && (u == -1 || dist[i] < dist[u])) u = i;
        }
        if (u == -1 || dist[u] == INF) return -1;
        used[u] = 1;
        ans += dist[u];
        for (int v = 1; v <= n; ++v) {
            if (!used[v]) dist[v] = min(dist[v], w[u][v]);
        }
    }
    return ans;
}
```

最小 Steiner 树：终端集状压 DP + 多源 Dijkstra

赛时先看

题目信号: 图上只有很少的关键城市/机器/宝石必须连通, 题目允许经过其他点; 终端数 k 很小 (常见 $k \leq 11 \sim 13$), 但总点数 n 可以较大。

本质: 在非负边权无向图中, 连接所有指定终端点, 允许额外经过非终端点, 求最小总边权。额外经过的点就是 Steiner 点。

接法: $k \leq 12$ 个指定城市必须接入同一条管网, 其他城市允许作为中转且每条道路只能付一次钱。建无向图后, 把这 k 个城市传给 `minimum_cost` 即可; 若某个点集不连通, 返回值会保持为 INF , 要按题意输出 -1 或其他失败标记。

复杂度判定: $O(3^k n + 2^k (n + m) \log n)$, 空间 $O(2^k n)$ 。边权必须非负; k 是决定能否使用的关键, 而不是 n 。

维护的量: g (无向邻接表); $dp[mask][v]$ (连通终端集 $mask$ 且树根在 v 的最小代价)。

警告: 这不是“终端之间跑 MST”, 因为最优解可能使用额外 Steiner

点; 所有终端可以重复时应先去重; 模板只返回代价, 若题目要求具体边集, 需要额外记录“子集划分”和 Dijkstra 前驱。

【最小完整示例（先抄这一段就能跑）】

```
// n=4 个点 {0..3}, 终端 {0,2}; 边: 0-1 权1, 1-2 权1, 2-3 权10, 0-3 权1
SteinerTree st(4);
st.add_edge(0, 1, 1); st.add_edge(1, 2, 1);
st.add_edge(2, 3, 10); st.add_edge(0, 3, 1);
i64 ans = st.minimum_cost({0, 2});
// ans = 2 (走 0-1-2, 中间点 1 是 Steiner 点)
```

【传参要求（照这个传不会错）】

- `SteinerTree st(n)`: n 为点数, 顶点编号 $0..n-1$ (0-based)。
- `add_edge(u, v, w)`: 加一条 $u-v$ 无向边, 权 w 非负。
- `minimum_cost(terminals)`: 终端编号 $\in [0, n)$ 且互不相同; 返回最小总代价 $i64$, 不连通时保持 INF , 输出前转 -1 。

【API / 入口函数（赛时只认这里列的名字）】

- `add_edge(int u, int v, i64 w) ->` 加入一条边

- `minimum_cost(const vector<int>& terminals)` -> `terminals` 中的终端点必须互不相同，且编号范围为 $[0, n)$ 。返回 `i64`。

【改板时先认这几个量】

- `g`: 邻接表。
- `dist`: 距离。
- `dp`: DP 状态。

```
struct SteinerTree {
    static constexpr i64 INF = (1LL << 62);

    int n;
    vector<vector<pair<int, i64>>> g;

    explicit SteinerTree(int n_) : n(n_), g(n_) {}

    void add_edge(int u, int v, i64 w) {
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }

    void multi_source_dijkstra(vector<i64>& dist) const {
        priority_queue<pair<i64, int>, vector<pair<i64, int>>, greater<pair<i64, int>>> pq;
        for (int v = 0; v < n; ++v) {
            if (dist[v] < INF) pq.push({dist[v], v});
        }
        while (!pq.empty()) {
            auto [d, u] = pq.top();
            pq.pop();
            if (d != dist[u]) continue;
            for (auto [v, w] : g[u]) {
                if (d > INF - w) continue;
                if (d + w < dist[v]) {
                    dist[v] = d + w;
                    pq.push({dist[v], v});
                }
            }
        }
    }

    // terminals 中的终端点必须互不相同，且编号范围为 [0, n)。
    i64 minimum_cost(const vector<int>& terminals) const {
        int k = (int)terminals.size();
        if (k <= 1) return 0;
        int full = 1 << k;
        vector<vector<i64>> dp(full, vector<i64>(n, INF));
        for (int i = 0; i < k; ++i) dp[1 << i][terminals[i]] = 0;

        for (int mask = 1; mask < full; ++mask) {
            if ((mask & (mask - 1)) != 0) {
                for (int sub = (mask - 1) & mask; sub; sub = (sub - 1) & mask) {
                    int other = mask ^ sub;
                    if (sub > other) continue; // 同一种集合划分不需要重复合并两次。
                    for (int v = 0; v < n; ++v) {
                        if (dp[sub][v] >= INF || dp[other][v] >= INF) continue;
                        dp[mask][v] = min(dp[mask][v], dp[sub][v] + dp[other][v]);
                    }
                }
            }
            // 把合并后的树根沿任意一条最短路转移。
            multi_source_dijkstra(dp[mask]);
        }
        return *min_element(dp.back().begin(), dp.back().end());
    }
};
```

典题模型: `k <= 12`

个指定城市必须接入同一条管网，其他城市允许作为中转且每条道路只能付一次钱。建无向图后，把这 `k` 个城市传给 `minimum_cost` 即可；若某个点集不连通，返回值会保持为 `INF`，要按题意输出 `-1` 或其他失败标记。

次小生成树

赛时先看

题目信号: 最小生成树之外，还问第二小、是否唯一。

本质: 求严格次小生成树或判断 MST 是否唯一。

接法: 把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定: Kruskal $O(m \log m)$, 再枚举非树边 $O(m \log n)$ 。

维护的量: `mst` (MST 总权)、`second` (严格次小总权); `helper.mx[k][u]` (倍增表中路径最大边权)。

警告: 严格次小要求替换后权值必须大于 MST; 非严格次小可以允许相等。

【最小完整示例 (先抄这一段就能跑)】

```
// n=4 个点 (1..4), 边: 1-2 权1, 2-3 权2, 3-4 权3, 1-4 权4
vector<Edge> edges = {{1, 2, 1}, {2, 3, 2}, {3, 4, 3}, {1, 4, 4}};
auto [mst, second] = strict_second_mst(4, edges);
// mst = 6, second = 7 (MST 中 1-2 换 1-4: 2+3+4)
```

【传参要求 (照这个传不会错)】

- `n`: 点数, 顶点编号 $1..n$ 。
- `edges`: 每条 $\{u, v, w\}$ 表示 $u-v$ 无向边权 w , $u, v \in [1, n]$ 。
- 返回 `pair<i64, i64>`: `{mst, second}`; 不存在严格次小 (或图不连通) 时 `second = INF = (1LL<<62)`。

【API / 入口函数 (赛时只认这里列的名字)】

- `find(int x) ->` 查代表元/查找结果 返回 `int`。
- `init(int n) ->` 初始化/清空结构
- `init(int n_) ->` 初始化/清空结构
- `unite(int a, int b) ->` 合并两个集合 返回 `bool`。
- `strict_second_mst(int n, vector<Edge> edges) ->` 返回 `{mst_weight, strict_second_mst_weight}`; 不存在严格次小则 `second = INF`。

【改板时先认这几个量】

- `sz`: 集合/子树大小。
- `depth`: 深度。
- `up`: 倍增祖先表。
- `mx`: 区间最大值。

```
struct Edge {
    int u, v;
    i64 w;
    bool in_mst = false;
};

struct DSU {
    vector<int> p, sz;
    DSU(int n = 0) { init(n); }
    void init(int n) { p.resize(n + 1); sz.assign(n + 1, 1); iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool unite(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a; sz[a] += sz[b];
        return true;
    }
};

struct SecondMST {
    int n, LOG;
    vector<vector<pair<int, i64>>> tree;
    vector<int> depth;
    vector<vector<int>> up;
    vector<vector<i64>> mx;

    SecondMST(int n = 0) { init(n); }
    void init(int n_) {
        n = n_;
        LOG = 1;
        while ((1 << LOG) <= n) LOG++;
        tree.assign(n + 1, {});
        depth.assign(n + 1, 0);
        up.assign(LOG, vector<int>(n + 1, 1));
        mx.assign(LOG, vector<i64>(n + 1, 0));
    }

    void dfs(int u, int p) {
        up[0][u] = p;
        for (int k = 1; k < LOG; ++k) {
            up[k][u] = up[k - 1][up[k - 1][u]];
            mx[k][u] = max(mx[k - 1][u], mx[k - 1][up[k - 1][u]]);
        }
    }
};
```

```

    }
    for (auto [v, w] : tree[u]) {
        if (v == p) continue;
        depth[v] = depth[u] + 1;
        mx[0][v] = w;
        dfs(v, u);
    }
}

i64 max_on_path(int a, int b) {
    i64 ans = 0;
    if (depth[a] < depth[b]) swap(a, b);
    int diff = depth[a] - depth[b];
    for (int k = 0; k < LOG; ++k) if (diff >= k & 1) {
        ans = max(ans, mx[k][a]);
        a = up[k][a];
    }
    if (a == b) return ans;
    for (int k = LOG - 1; k >= 0; --k) {
        if (up[k][a] != up[k][b]) {
            ans = max(ans, mx[k][a]);
            ans = max(ans, mx[k][b]);
            a = up[k][a];
            b = up[k][b];
        }
    }
    ans = max(ans, mx[0][a]);
    ans = max(ans, mx[0][b]);
    return ans;
}

};

// 返回 {mst_weight, strict_second_mst_weight}; 不存在严格次小则 second = INF。
pair<i64, i64> strict_second_mst(int n, vector<Edge> edges) {
    const i64 INF = (1LL << 62);
    sort(edges.begin(), edges.end(), [](const Edge& a, const Edge& b) {
        return a.w < b.w;
    });
    DSU dsu(n);
    SecondMST helper(n);
    i64 mst = 0;
    int used = 0;
    for (auto& e : edges) {
        if (dsu.unite(e.u, e.v)) {
            e.in_mst = true;
            mst += e.w;
            used++;
            helper.tree[e.u].push_back({e.v, e.w});
            helper.tree[e.v].push_back({e.u, e.w});
        }
    }
    if (used != n - 1) return {INF, INF};
    helper.dfs(1, 1);

    i64 second = INF;
    for (auto e : edges) {
        if (e.in_mst) continue;
        i64 mx = helper.max_on_path(e.u, e.v);
        i64 candidate = mst + e.w - mx;
        if (candidate > mst) second = min(second, candidate);
    }
    return {mst, second};
}

```

Kruskal 重构树

赛时先看

题目信号：多次询问两点在边权不超过 x 时是否连通，或 MST 路径最大边。

本质：MST 合并过程建树，回答路径最大边权阈值、按边权连通的查询。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定：建树 $O(m \log m)$ ，查询配合 LCA。

维护的量：`parent`（并查集父）、`val[x]`（节点权值，新点取合并边权）、`tree`（重构树儿子列表，新点编号 $n+1..tot$ ）。

警告：新建节点权值为当前合并边权；原点是叶子。

【最小完整示例（先抄这一段就能跑）】

```
// 原点点 1..3; 边: 1-2 权2, 2-3 权5
```

```
struct E { int u, v; i64 w; };
KruskalTree kt(3);
kt.build(vector<E>{{1, 2, 2}, {2, 3, 5}});
// 新点 4 (权2)、5 (权5)；查询两点在边权<=t 下是否连通：找 LCA 权值<=t
```

【传参要求（照这个传不会错）】

- `KruskalTree kt(n)`: `n` 为原点数，编号 `1..n`。
- `build(edges)`: `edges` 元素需含 `u/v/w` 字段（`w` 为边权），按边权升序自动建树。
- `find(x)`: 并查集代表元查询。
- 建完后原点（`1..n`）是叶子，新点 `val[x]` = 合并边权，`tot` 为节点总数。

【API / 入口函数（赛时只认这里列的名字）】

- `build(vector<Edge> edges)` -> 完成建树或预处理
- `find(int x)` -> 查代表元/查找结果 返回 `int`。
- `init(int n_)` -> 初始化/清空结构

```
struct KruskalTree {
    int n, tot;
    vector<int> parent;
    vector<i64> val;
    vector<vector<int>> tree;

    KruskalTree(int n = 0) { init(n); }
    void init(int n_) {
        n = n_;
        tot = n;
        parent.resize(2 * n + 5);
        val.assign(2 * n + 5, 0);
        tree.assign(2 * n + 5, {});
        iota(parent.begin(), parent.end(), 0);
    }

    int find(int x) { return parent[x] == x ? x : parent[x] = find(parent[x]); }

    template <class Edge>
    void build(vector<Edge> edges) {
        sort(edges.begin(), edges.end(), [](const Edge& a, const Edge& b) { return a.w < b.w; });
        for (auto e : edges) {
            int ru = find(e.u), rv = find(e.v);
            if (ru == rv) continue;
            int x = ++tot;
            val[x] = e.w;
            tree[x].push_back(ru);
            tree[x].push_back(rv);
            parent[ru] = parent[rv] = parent[x] = x;
        }
    }
};
```

隐式 Kruskal 重构树：增量网格连通块吞并阈值

赛时先看

题目信号：题目按权值从小到大加入元素；元素会吞并相邻连通块；询问“从某个点出发要变多大才能吞掉整个可达块”；普通 DSU 只能知道连通性，但还需要知道向上合并时的最大门槛。

本质：在线按权值非降序激活点，每次新点与相邻已激活连通块合并。合并时不显式建重构树，只维护并查集父亲、连通块大小，以及每个儿子往父亲走需要的阈值，支持查询单点到当前根路径上的最大阈值。

接法：新点权值为 `w`，新建一个真实节点；每遇到相邻连通块根 `r`，创建虚点 `rt`，令两个根指向 `rt`，并给每个旧根记录 `need = w - size[root] + 1`。查询某真实点 `u` 时先 `find(u)` 压缩，然后答案是 `max(need[u] - value[u], 0)`。

复杂度判定：每次激活检查四邻域，均摊 $O(\alpha(q))$ ；空间 $O(q)$ ，最多 q 个原点和 $q-1$ 个合并虚点。

维护的量：`parent`（并查集父）、`value[x]`（节点权值）、`need[x]`（儿子到父亲要跨过的阈值）、`comp_size[x]`（连通块大小）。

警告：`find(x)` 路径压缩时要把 `need[x]` 更新成到新根路径上的最大阈值；合并产生的新虚点才是父亲；老根的 `size` 清零后不能再拿来当当前连通块大小。该模型要求激活权值非降，否则阈值语义会坏。

【最小完整示例（先抄这一段就能跑）】

```
// 3x3 网格，最多 9 次激活：激活权值必须非降
ImplicitKruskalGridEat g(3, 3, 9);
g.activate(1, 1, 1);
g.activate(1, 2, 2);
int rest = g.activate(2, 1, 2); // 吞并后该连通块还能再吞 rest 格
int need = g.extra_needed_to_eat_component(2, 1); // 该格还要变大 need 才能吃满
```

```
// 本样例: rest = 3 (还可吞 3 格), need = 0
```

【传参要求（照这个传不会错）】

- `init(n_, m_, max_ops)`: 网格 `n_` 行 `m_` 列, 坐标 1-indexed; `max_ops` 为最大激活次数 (内部按 `2*max_ops+5` 预留空间)。
- `activate(x, y, w)`: 按非降权值 `w` 激活格子 `(x, y)`; 返回当前连通块还能吞并的其他格子数量。
- `extra_needed_to_eat_component(x, y)`: 查询该格要吃满所在连通块还需变大的最小增量, 格子必须已激活。
- 警告: 激活权值必须非降, 否则阈值语义会坏。

【API / 入口函数（赛时只认这里列的名字）】

- `find(int x)` -> 查代表元/查找结果 返回 `int`。
- `init(int n_, int m_, int max_ops)` -> 初始化/清空结构
- `activate(int x, int y, int w)` -> 按非降权值 `w` 激活格子 `(x, y)`。 返回当前连通块中还能吞并的其他格子数量。

【改板时先认这几个量】

- `parent`: 并查集父节点。
- `find`: 带上限的路径压缩查询 (`find(x, need)`)。

```
struct ImplicitKruskalGridEat {
    int n = 0, m = 0, total = 0;
    vector<vector<int>> id;
    vector<int> parent, comp_size, value, need;
    const int dx[4] = {1, 0, -1, 0};
    const int dy[4] = {0, 1, 0, -1};

    ImplicitKruskalGridEat() = default;
    ImplicitKruskalGridEat(int n_, int m_, int max_ops) { init(n_, m_, max_ops); }

    void init(int n_, int m_, int max_ops) {
        n = n_;
        m = m_;
        total = 0;
        id.assign(n + 1, vector<int>(m + 1, 0));
        int cap = 2 * max_ops + 5;
        parent.assign(cap, 0);
        comp_size.assign(cap, 0);
        value.assign(cap, 0);
        need.assign(cap, 0);
    }

    int find(int x) {
        if (parent[x] == x) return x;
        int p = parent[x];
        int root = find(p);
        need[x] = max(need[x], need[p]);
        return parent[x] = root;
    }

    void merge_roots(int a, int b, int current_weight) {
        a = find(a);
        b = find(b);
        if (a == b) return;
        int root = ++total;
        parent[root] = root;
        comp_size[root] = comp_size[a] + comp_size[b];
        value[root] = current_weight;
        need[root] = 0;

        parent[a] = parent[b] = root;
        need[a] = current_weight - comp_size[a] + 1;
        need[b] = current_weight - comp_size[b] + 1;
        comp_size[a] = comp_size[b] = 0;
    }

    // 按非降权值 w 激活格子 (x, y)。
    // 返回当前连通块中还能吞并的其他格子数量。
    int activate(int x, int y, int w) {
        int u = ++total;
        id[x][y] = u;
        parent[u] = u;
        comp_size[u] = 1;
        value[u] = w;
        need[u] = 0;
        for (int dir = 0; dir < 4; ++dir) {
            int nx = x + dx[dir], ny = y + dy[dir];
            if (nx < 1 || nx > n || ny < 1 || ny > m || id[nx][ny] == 0) continue;
            merge_roots(u, id[nx][ny], w);
        }
    }
};
```

```

    }
    return comp_size[find(u)] - 1;
}

int extra_needed_to_eat_component(int x, int y) {
    int u = id[x][y];
    assert(u != 0);
    find(u);
    return max(need[u] - value[u], 0);
}
};

```

典题：本场 C 《Fish Eating》。activate(x,y,w)

对应放入一条鱼后能吃掉的数量；extra_needed_to_eat_component(x,y)

对应把该格鱼最少变大多少才能达到最大可吃数量。

朱刘算法：有向图最小树形图

赛时先看

题目信号：有向图版本 MST；每个非根点要选一条入边。

本质：求以 root 为根，到达所有点的有向最小生成树。

接法：把本节 struct/class/函数组 整段抄到 solve() 上面；solve() 里先构造对象，再按下方“不会用就看这里”调用公开接口。

复杂度判定：朴素实现 $O(nm)$ 。

维护的量：in[v]（每轮 v 的最小入边权）、pre[v]（该入边起点）、id[v]（缩环后的新编号）。

警告：必须检查所有点可达；缩环时边权减去入边权。

【最小完整示例（先抄这一段就能跑）】

```

// n=3, 根 1; 有向边 1->2 权4, 1->3 权2, 3->2 权1
vector<DirectedEdge> edges = {{1, 2, 4}, {1, 3, 2}, {3, 2, 1}};
i64 ans = directed_mst(3, 1, edges);
// ans = 3 (选 1->3 与 3->2)

```

【传参要求（照这个传不会错）】

- n: 顶点数，编号 1..n。
- root: 根，范围 [1, n]。
- edges: 每条 {u, v, w} 表示有向边 $u \rightarrow v$ 权 w, $u, v \in [1, n]$ ；自环会被忽略。
- 返回最小树形图总权 i64；有点不可达返回 -1。

```

struct DirectedEdge { int u, v; i64 w; };

i64 directed_mst(int n, int root, vector<DirectedEdge> edges) {
    const i64 INF = (1LL << 60);
    i64 ans = 0;
    while (true) {
        vector<i64> in(n + 1, INF);
        vector<int> pre(n + 1, -1);
        for (auto e : edges) {
            if (e.u != e.v && e.w < in[e.v]) {
                in[e.v] = e.w;
                pre[e.v] = e.u;
            }
        }
        in[root] = 0;
        for (int i = 1; i <= n; i++) if (in[i] == INF) return -1;
        int cnt = 0;
        vector<int> id(n + 1, 0), vis(n + 1, 0);
        for (int i = 1; i <= n; i++) {
            ans += in[i];
            int v = i;
            while (vis[v] != i && !id[v] && v != root) {
                vis[v] = i;
                v = pre[v];
            }
            if (v != root && !id[v]) {
                id[v] = ++cnt;
                for (int u = pre[v]; u != v; u = pre[u]) id[u] = cnt;
            }
        }
        if (!cnt) break;
        for (int i = 1; i <= n; i++) if (!id[i]) id[i] = ++cnt;
        vector<DirectedEdge> ne;
        for (auto e : edges) {
            int u = id[e.u], v = id[e.v];

```

```

        i64 w = e.w;
        if (u != v) w -= in[e.v];
        ne.push_back({u, v, w});
    }
    root = id[root];
    n = cnt;
    edges.swap(ne);
}
return ans;
}

```

Stoer-Wagner 全局最小割

赛时先看

题目信号：不是指定 $s-t$ 最小割，而是全局最小割；点数通常几百以内。

本质：无向带权图求任意划分两侧割边权最小值。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O(n^3)$ 。

维护的量：`dis[j]`（ j 到当前已选集合的总边权）、`v`（剩余点集）、`w`（每轮合并后收缩的权矩阵）。

警告：图必须是无向图；每轮把最后加入的两个点合并；图不连通时返回 0（最小割为 0），按题意自行判断。

【最小完整示例（先抄这一段就能跑）】

```

// 3 个点 (0..2)，边：0-1 权2，1-2 权3，0-2 权5
vector<vector<i64>> w(3, vector<i64>(3, 0));
w[0][1] = w[1][0] = 2; w[1][2] = w[2][1] = 3; w[0][2] = w[2][0] = 5;
i64 ans = stoer_wagner(w);
// ans = 5 (割 {0,2} | {1}: 2+3)

```

【传参要求（照这个传不会错）】

- `w`: $n \times n$ 对称权矩阵（0-based 编号），`w[i][j]` 为 $i-j$ 边权，无边为 0，重边直接累加。
- 返回全局最小割 `i64`；图不连通时返回 0。

```

i64 stoer_wagner(vector<vector<i64>> w) {
    int n = (int)w.size();
    vector<int> v(n);
    iota(v.begin(), v.end(), 0);
    const i64 INF = (1LL << 62);
    i64 ans = INF;
    while (n > 1) {
        vector<i64> dis(n, 0);
        vector<int> used(n, 0);
        int prev = -1, sel = -1;
        for (int i = 0; i < n; i++) {
            sel = -1;
            for (int j = 0; j < n; j++) {
                if (!used[j] && (sel == -1 || dis[j] > dis[sel])) sel = j;
            }
            if (i == n - 1) {
                ans = min(ans, dis[sel]);
                for (int j = 0; j < n; j++) {
                    w[prev][j] += w[sel][j];
                    w[j][prev] += w[j][sel];
                }
                v.erase(v.begin() + sel);
                for (int j = 0; j < n; j++) w[j].erase(w[j].begin() + sel);
                w.erase(w.begin() + sel);
                n--;
                break;
            }
            used[sel] = 1;
            prev = sel;
            for (int j = 0; j < n; j++) if (!used[j]) dis[j] += w[sel][j];
        }
    }
    return ans;
}

```

Matrix-Tree 定理：生成树计数

赛时先看

题目信号：题面问生成树个数； n 不大但边多；答案取模。

本质：求无向图生成树数量，或带权生成树总权。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O(n^3)$ 。

维护的量：`lap` (1-indexed 拉普拉斯矩阵)、`a` (删第 1 行第 1 列后的 $n-1$ 阶子式)。

警告：构造拉普拉斯矩阵后删掉任意一行一列求行列式；重边要累加；下面 `det_mod` 版本要求 `mod` 为质数。

【最小完整示例（先抄这一段就能跑）】

```
// n=3 个点 (1..3) : 1-2 两条重边, 2-3 一条
vector<pair<int, int>> edges = {{1, 2}, {1, 2}, {2, 3}};
i64 ans = count_spanning_tree(3, edges, (i64)1e9 + 7);
// ans = 2 (必含 2-3, 再在两条 1-2 重边里任选一条)
```

【传参要求（照这个传不会错）】

- `count_spanning_tree(n, edges, mod)`: n 个点编号 $1..n$; `edges` 为 $\{u, v\}$ 无向边，重边算多条；返回生成树个数 $\% \text{mod}$ 。
- `det_mod(a, mod)`: `a` 为 $n-1$ 阶模矩阵，`mod` 必须是质数；返回 $\det \% \text{mod}$ 。
- 用法：1-indexed 建拉普拉斯矩阵，删第 1 行第 1 列，子式行列式即答案。

约定：拉普拉斯矩阵用 1-indexed，删第 1 行第 1 列。

```
i64 det_mod(vector<vector<i64>> a, i64 mod) {
    int n = (int)a.size();
    i64 ans = 1;
    for (int i = 0; i < n; i++) {
        int pivot = i;
        while (pivot < n && a[pivot][i] % mod == 0) pivot++;
        if (pivot == n) return 0;
        if (pivot != i) {
            swap(a[pivot], a[i]);
            ans = (mod - ans) % mod;
        }
        i64 inv = 1, base = (a[i][i] % mod + mod) % mod, e = mod - 2;
        while (e) {
            if (e & 1) inv = inv * base % mod;
            base = base * base % mod;
            e >>= 1;
        }
        ans = ans * ((a[i][i] % mod + mod) % mod) % mod;
        for (int j = i + 1; j < n; j++) {
            i64 factor = (a[j][i] % mod + mod) % mod * inv % mod;
            for (int k = i; k < n; k++) {
                a[j][k] = (a[j][k] - factor * a[i][k]) % mod;
            }
        }
    }
    return (ans % mod + mod) % mod;
}

i64 count_spanning_tree(int n, const vector<pair<int, int>>& edges, i64 mod) {
    vector lap(n + 1, vector<i64>(n + 1, 0));
    for (auto [u, v] : edges) {
        lap[u][u]++; lap[v][v]++;
        lap[u][v]--; lap[v][u]--;
    }
    vector a(n - 1, vector<i64>(n - 1, 0));
    for (int i = 1; i < n; i++) {
        for (int j = 1; j < n; j++) a[i - 1][j - 1] = lap[i][j];
    }
    return det_mod(a, mod);
}
```

07 图论进阶：连通性、拓扑与结构

拓扑、DAG、支配树、强连通/双连通、仙人掌、2-SAT、欧拉路、传递闭包和最大团放在这里。

拓扑排序

赛时先看

题目信号：有向边表示先后顺序；问是否存在合法顺序。

本质：DAG 顺序、依赖关系、课程先修、DAG DP。

接法：边 $u \rightarrow v$ 表示 u 必须在 v 前完成；把所有入度为 0 的点入队。返回的 `order` 是一种合法顺序；如果 `order.size() < n`，说明依赖里有环，DAG DP 不能直接做。

复杂度判定： $O(n+m)$ 。

维护的量: `indeg` (每个点的入度) 与 `order` (已排出的拓扑序)。

警告: 结果长度小于 `n` 说明有环。

【最小完整示例 (先抄这一段就能跑)】

题目: `n` 门课有先后依赖, 边 `u->v` 表示先修 `u` 再修 `v`, 求一种合法学习顺序。

```
vector<int> order = topo_sort(n, g);           // 1-based 邻接表
if ((int)order.size() < n) cout << "有环, 无合法顺序\n";
else for (int u : order) cout << u << ' ';
```

样例: `n=4`, 边 `1->2 2->3 3->4`, 输出 `1 2 3 4`。

【传参要求 (照这个传不会错)】

- `n`: 顶点数; 下标 `1..n`。
- `g`: 邻接表, `g[u]` 存从 `u` 出发的边终点 `v`, 范围 `1..n`。
- 返回值 `order`: 一种合法拓扑序; `order.size() < n` 说明有环 (无合法顺序)。

【改板时先认这几个量】

- `g`: 邻接表。
- `indeg`: 入度。

```
vector<int> topo_sort(int n, const vector<vector<int>>& g) {
    vector<int> indeg(n + 1), order;
    for (int u = 1; u <= n; ++u) {
        for (int v : g[u]) indeg[v]++;
    }
    queue<int> q;
    for (int i = 1; i <= n; ++i) if (indeg[i] == 0) q.push(i);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        order.push_back(u);
        for (int v : g[u]) {
            if (--indeg[v] == 0) q.push(v);
        }
    }
    return order;
}
```

DAG 最长路

赛时先看

题目信号: 有依赖顺序, 边可以有权, 图无环。

本质: 有向无环图上的最长路径/最大收益。

接法: 把本节 `struct/class/函数组` 整段抄到 `solve()` 上面; `solve()` 里直接按下方“不会用就看这里”调用, 不需要自己猜内部参数。

复杂度判定: $O(n+m)$ 。

维护的量: `dp[v]` (到 `v` 的最长距离, `NEG` 表示不可达)、`indeg` (入度)。

警告: 普通图最长路很难; 只有 DAG 可以这样做。

【最小完整示例 (先抄这一段就能跑)】

题目: DAG 上每条边有权, 求从某个入度为 0 的点出发到各点的最长距离。

```
vector<i64> dp = dag_longest_path(n, g); // g[u] 存 {v, w}, w 为 i64 权
cout << dp[v] << '\n';                // v 的最长路; NEG 表示不可达
```

样例: `n=3`, 边 `1->2(5) 2->3(4)`, `dp[3]=9`。

【传参要求 (照这个传不会错)】

- `n`: 顶点数; 下标 `1..n`。
- `g`: 邻接表, `g[u]` 存 `{v, w}` (终点、边权), 编号 `1..n`, 权用 `i64`。
- 返回值 `dp`: `dp[v]` 为到 `v` 的最长距离; 入度为 0 的点为 0; 不可达为 `NEG = -(1LL<<60)`。

【改板时先认这几个量】

- `g`: 邻接表。
- `dp`: DP 状态。

- indeg: 入度。

```
vector<i64> dag_longest_path(int n, const vector<vector<pair<int, i64>>>& g) {
    vector<int> indeg(n + 1);
    for (int u = 1; u <= n; ++u) {
        for (auto [v, w] : g[u]) indeg[v]++;
    }
    queue<int> q;
    for (int i = 1; i <= n; ++i) if (indeg[i] == 0) q.push(i);

    const i64 NEG = -(1LL << 60);
    vector<i64> dp(n + 1, NEG);
    for (int i = 1; i <= n; ++i) if (indeg[i] == 0) dp[i] = 0;

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (auto [v, w] : g[u]) {
            if (dp[u] != NEG) dp[v] = max(dp[v], dp[u] + w);
            if (--indeg[v] == 0) q.push(v);
        }
    }
    return dp;
}
```

非树边造好环 + BFS 定向构造

赛时先看

题目信号：题目要求输出每条无向边的方向；只要每个弱连通块/基环森林里的环满足某个性质即可；存在一条非树边能制造关键环；其它边只需保证能流向这个环。

本质：在无向连通图中先找一条满足性质的非树边，让最终有向图至少包含一个“好环”，再把其余边按 BFS 深度定向成指向汇点的 DAG。

接法：把“不坏的边”先用于建生成结构；遇到一条端点满足好性质且已经在同一 DSU 内的边，就把它留作特殊非树边 (s, t) ；从 s BFS，其余边按深度从大到小指向小深度，特殊边强制 $s \leftarrow t$ 或按题意反向。

复杂度判定：并查集找边 $O(m \alpha(n))$ ，BFS 定向 $O(n+m)$ 。

维护的量：dsu（挑“不坏的边”建生成结构、找出特殊非树边 (s, t) ）； $depth[u]$ （从 s BFS 的深度，定向依据：深指向浅）；特殊边 (s, t) 单独记下、最后强制反向。

警告：先用 DSU 选择“可作为额外环边”的边；BFS 定向时深度小的点应成为汇点方向；最后要对特殊非树边反向，确保环里包含关键差异。若没有合格非树边，应输出无解。

典题：本场 I 《Combination of Two Nice

Problems》。先用相等坐标边尽量连通，找到一条端点坐标不同且会成环的非树边；从其中一端 BFS，对其他边按层次定向，特殊边反向，使唯一环含有不同坐标点。

支配树：Lengauer-Tarjan

赛时先看

题目信号：控制流图、必须经过的检查点、删去一个点会让哪些点从起点不可达、所有路径公共关键节点。

本质：在有向图从起点 s 出发时，求每个点的直接支配者。若到达 v 的每条路径都必须经过 u ，则 u 支配 v 。

接法：给出有向图和起点，询问“必须经过某点才能到达”的支配关系，或对支配树做子树统计。

复杂度判定：接近线性，常写为 $O((n+m) \alpha(n))$ 。

维护的量： $idom[v]$ (v 的直接支配者)、 $sdom/dom$ （半支配者/支配者，按 DFS 序坐标维护）。

警告：只对从根可达的节点有定义；返回 $idom[v] = -1$ 表示不可达，根自己的直接支配者置为根。

约定：图 0-based。

【最小完整示例（先抄这一段就能跑）】

题目：有向图问“删掉某点后哪些点从起点不可达”，即求每个点的直接支配者。

```
DominatorTree dt(g); // g 为 0-based 邻接表
vector<int> idom = dt.build(root); // root 为源点 (0-based)
for (int v = 0; v < n; ++v) cout << idom[v] << ' '; // 直接支配者, -1=不可达
```

样例： $n=3$ ，边 $0 \rightarrow 1$ $0 \rightarrow 2$ $1 \rightarrow 2$ ， $idom = \{0, 0, 0\}$ ($idom[0]=0$ 表示根支配自己)。

【传参要求（照这个传不会错）】

- 构造 `DominatorTree(graph)`：graph 是 0-based 邻接表，大小 n 。
- `build(int root)`：root 为源点，0-based，范围 $0..n-1$ 。

- 返回值 `idom` (长度 `n`) : `idom[v]` 是 `v` 的直接支配者; `idom[root] = root`; `idom[v] = -1` 表示 `v` 从 `root` 不可达。

【API / 入口函数 (赛时只认这里列的名字)】

- `build(int root)` -> 完成建树或预处理 返回 `vector<int>`。

【改板时先认这几个量】

- `dsu`: Lengauer-Tarjan 的 DSU 辅助数组 (路径压缩 + 半支配者合并)。
- `root`: `build(root)` 的源点。
- `g`: 邻接表。
- `depth`: 深度。

典题模型: 给出有向图和起点, 询问“必须经过某点才能到达”的支配关系, 或对支配树做子树统计。

```
struct DominatorTree {
    int n, timer = 0;
    vector<vector<int>> g, rg, bucket;
    vector<int> arr, rev, par, sdom, dom, dsu, label;

    DominatorTree(const vector<vector<int>>& graph) : n((int)graph.size()), g(graph) {}

    void dfs(int u) {
        arr[u] = timer;
        rev[timer] = u;
        label[timer] = sdom[timer] = dsu[timer] = timer;
        ++timer;
        for (int v : g[u]) {
            if (arr[v] != -1) continue;
            dfs(v);
            par[arr[v]] = arr[u];
        }
    }

    int find_set(int u, int depth = 0) {
        if (u == dsu[u]) return depth ? -1 : u;
        int v = find_set(dsu[u], depth + 1);
        if (v < 0) return u;
        if (sdom[label[dsu[u]]] < sdom[label[u]]) label[u] = label[dsu[u]];
        dsu[u] = v;
        return depth ? v : label[u];
    }

    vector<int> build(int root) {
        arr.assign(n, -1);
        rev.assign(n, -1);
        par.assign(n, -1);
        sdom.assign(n, 0);
        dom.assign(n, 0);
        dsu.assign(n, 0);
        label.assign(n, 0);
        timer = 0;
        dfs(root);

        int T = timer;
        rg.assign(T, {});
        bucket.assign(T, {});
        for (int u = 0; u < n; ++u) if (arr[u] != -1) {
            for (int v : g[u]) if (arr[v] != -1) rg[arr[v]].push_back(arr[u]);
        }

        for (int i = T - 1; i >= 0; --i) {
            for (int v : rg[i]) sdom[i] = min(sdom[i], sdom[find_set(v)]);
            if (i) bucket[sdom[i]].push_back(i);
            for (int v : bucket[i]) {
                int y = find_set(v);
                dom[v] = (sdom[y] == sdom[v] ? sdom[v] : y);
            }
            if (i) dsu[i] = par[i];
        }
        for (int i = 1; i < T; ++i) {
            if (dom[i] != sdom[i]) dom[i] = dom[dom[i]];
        }

        vector<int> idom(n, -1);
        idom[root] = root;
        for (int i = 1; i < T; ++i) idom[rev[i]] = rev[dom[i]];
        return idom;
    }
};
```

DAG 最小路径覆盖

赛时先看

题目信号：有向无环图，问最少链/路径覆盖全部点。

本质：用最少数路径覆盖 DAG 上所有点。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定：建二分图后最大匹配，答案 $n - \text{matching}$ 。

维护的量：无持久状态；内部维护二分图最大匹配数，答案即 $n - \text{matching}$ 。

警告：只适用于 DAG；每个原点拆成左点和右点。依赖第 08 章第 66 节 HopcroftKarp，抄板时须一并抄上。

【最小完整示例（先抄这一段就能跑）】

题目：DAG 上选尽量少的路径覆盖所有点，求最少路径条数。

```
int ans = minimum_path_cover_dag(n, dag); // dag 为 1-based 邻接表
cout << ans << '\n';                  // 最少路径条数
```

样例： $n=3$ ，边 $1 \rightarrow 2$ ，答案 2（路径 $1 \rightarrow 2$ 与 3）。

【传参要求（照这个传不会错）】

- n ：顶点数；下标 $1..n$ 。
- dag ：邻接表， $\text{dag}[u]$ 存 u 可直接到达的后继 v ，范围 $1..n$ 。
- 返回值：`int`，最小路径覆盖数（ $= n - \text{最大匹配}$ ）。
- 依赖：须一并抄第 08 章 HopcroftKarp，否则无法编译。

```
int minimum_path_cover_dag(int n, const vector<vector<int>>& dag) {
    HopcroftKarp hk(n, n);
    for (int u = 1; u <= n; ++u) {
        for (int v : dag[u]) hk.add_edge(u, v);
    }
    return n - hk.max_matching();
}
```

Tarjan 强连通分量 SCC

赛时先看

题目信号：有向图中问“互相到达”“环依赖”“缩点后 DAG”。

本质：有向图缩点、互相可达关系。

接法：先 SCC `scc(n)`，所有有向边调用 `add_edge(u,v)`，最后 `scc.run()`。如果题目问两个点是否互相可达，看 `scc.comp[u] == scc.comp[v]`；如果要缩点建 DAG，就把原边 (u,v) 中 `comp[u] != comp[v]` 的部分连到新图。

复杂度判定： $O(n+m)$ 。

维护的量：`dfn/low`（时间戳/lowlink）、`comp[u]`（ u 所属 SCC 编号）、`comp_cnt`。

警告：`comp[u]` 相同表示同一强连通分量。

【最小完整示例（先抄这一段就能跑）】

题目：有向图问 u 、 v 是否互相可达（同一 SCC），并统计缩点个数。

```
SCC scc(n); // n 为点数，编号 1..n
scc.add_edge(u, v); // 每条有向边都加
int cnt = scc.run(); // 返回强连通分量个数
if (scc.comp[u] == scc.comp[v]) cout << "互相可达\n";
```

样例： $n=2$ ，边 $1 \rightarrow 2$ $2 \rightarrow 1$ ，`run()` 返回 1，`comp[1]==comp[2]`。

【传参要求（照这个传不会错）】

- `SCC(n) / init(n_)`： n 为点数，编号 $1..n$ 。
- `add_edge(int u, int v)`：加一条有向边， u 、 v 范围 $1..n$ 。
- `run()`：返回 `comp_cnt`（SCC 个数），并填好 `comp`。
- `comp[u]`： u 所属 SCC 编号，范围 $1..comp_cnt$ ；相等即互相可达。

【API / 入口函数（赛时只认这里列的名字）】

- `add_edge(int u, int v)` -> 加入一条边

- `init(int n_)` -> 初始化/清空结构

【改板时先认这几个量】

- `g`: 邻接表。
- `dfn`: DFS 序时间戳。
- `low`: `lowlink` / 最早可达时间戳。

```
struct SCC {
    int n, timer = 0, comp_cnt = 0;
    vector<vector<int>> g;
    vector<int> dfn, low, st, in_st, comp;

    SCC(int n = 0) { init(n); }
    void init(int n_) {
        n = n_;
        g.assign(n + 1, {});
        dfn.assign(n + 1, 0);
        low.assign(n + 1, 0);
        in_st.assign(n + 1, 0);
        comp.assign(n + 1, 0);
        st.clear();
        timer = comp_cnt = 0;
    }
    void add_edge(int u, int v) { g[u].push_back(v); }

    void dfs(int u) {
        dfn[u] = low[u] = ++timer;
        st.push_back(u);
        in_st[u] = 1;
        for (int v : g[u]) {
            if (!dfn[v]) {
                dfs(v);
                low[u] = min(low[u], low[v]);
            } else if (in_st[v]) {
                low[u] = min(low[u], dfn[v]);
            }
        }
        if (low[u] == dfn[u]) {
            comp_cnt++;
            while (true) {
                int x = st.back();
                st.pop_back();
                in_st[x] = 0;
                comp[x] = comp_cnt;
                if (x == u) break;
            }
        }
    }

    int run() {
        for (int i = 1; i <= n; ++i) if (!dfn[i]) dfs(i);
        return comp_cnt;
    }
};
```

割点与桥

赛时先看

题目信号: 网络脆弱性、关键道路、删点/删边后是否断开。

本质: 无向图中找删掉后影响连通性的点/边。

接法: 把本节 `struct/class/函数组` 整段抄到 `solve()` 上面; 外部只调用下面 API 列出的接口, 不要把内部递归参数直接当题面参数。

复杂度判定: $O(n+m)$ 。

维护的量: `dfn/low` (时间戳/`lowlink`)、`is_cut[u]` (割点标记)、`bridges` (所有桥的端点对)。

警告: 无向边要带边编号, 避免把反向边当父边跳过。

读答案: 答案在公开成员 `is_cut[u]` 与 `bridges` 中。

【最小完整示例 (先抄这一段就能跑)】

题目: 无向图问删掉哪些点 (或边) 会使图不连通, 输出所有割点与桥。

```
CutBridge cb(n);           // n 为点数, 编号 1..n
cb.add_edge(u, v);         // 无向边加一次即可
cb.run();                  // 跑完答案直接可读
if (cb.is_cut[u]) cout << "u 是割点\n";
for (auto [a, b] : cb.bridges) cout << a << ' ' << b << '\n';
```

样例：三角形 1-2-3-1，无割点、bridges 为空。

【传参要求（照这个传不会错）】

- CutBridge(n) / init(n): n 为点数，编号 1..n。
- add_edge(int u, int v): 加一条无向边，u、v 范围 1..n，重边也安全。
- run(): 无返回值；执行后结果就绪。
- is_cut[u]: 1 表示 u 是割点；bridges: 所有桥 {u,v} 的列表。

【API / 入口函数（赛时只认这里列的名字）】

- add_edge(int u, int v) -> 加入一条边
- init(int n) -> 初始化/清空结构

【改板时先认这几个量】

- g: 邻接表。
- dfn: DFS 序时间戳。
- low: lowlink / 最早可达时间戳。

```
struct CutBridge {
    int n, timer = 0, edge_id = 0;
    vector<vector<pair<int, int>>> g;
    vector<int> dfn, low, is_cut;
    vector<pair<int, int>> bridges;

    CutBridge(int n = 0) { init(n); }
    void init(int n_) {
        n = n_;
        g.assign(n + 1, {});
        dfn.assign(n + 1, 0);
        low.assign(n + 1, 0);
        is_cut.assign(n + 1, 0);
        bridges.clear();
        timer = edge_id = 0;
    }
    void add_edge(int u, int v) {
        g[u].push_back({v, edge_id});
        g[v].push_back({u, edge_id++});
    }
    void dfs(int u, int parent_edge) {
        dfn[u] = low[u] = ++timer;
        int child = 0;
        for (auto [v, id] : g[u]) {
            if (id == parent_edge) continue;
            if (!dfn[v]) {
                child++;
                dfs(v, id);
                low[u] = min(low[u], low[v]);
                if (low[v] > dfn[u]) bridges.push_back({u, v});
                if (parent_edge != -1 && low[v] >= dfn[u]) is_cut[u] = 1;
            } else {
                low[u] = min(low[u], dfn[v]);
            }
        }
        if (parent_edge == -1 && child > 1) is_cut[u] = 1;
    }
    void run() {
        for (int i = 1; i <= n; ++i) if (!dfn[i]) dfs(i, -1);
    }
};
```

边双连通分量 e-DCC

赛时先看

题目信号：询问桥、删一条边后的连通性、桥树直径、至少加多少边使边双连通。

本质：无向图删掉桥后得到边双连通分量，可缩点成桥树。

接法：把本节 struct/class/函数组 整段抄到 solve() 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定： $O(n+m)$ 。

维护的量：is_bridge[id]（边是否为桥）、comp[u]（u 所属边双编号）、comp_cnt。

警告：无向边要用边编号排除反向边；重边不是桥。

【最小完整示例（先抄这一段就能跑）】

题目：无向图删掉所有桥后分成几个连通块，输出每个点所属边双。

```
EdgeBCC eb(n);           // n 为点数，编号 1..n
eb.add_edge(u, v);       // 无向边加一次即可
int cnt = eb.build();     // 返回边双个数
for (int u = 1; u <= n; ++u) cout << eb.comp[u] << ' ';
```

样例：n=4，边 1-2 2-3 3-1 3-4，build() 返回 2（{1,2,3} 与 {4}，3-4 是桥）。

【传参要求（照这个传不会错）】

- EdgeBCC(n) / init(n_): n 为点数，编号 1..n。
- add_edge(int u, int v): 加一条无向边，u、v 范围 1..n，重边自动处理。
- build(): 返回 comp_cnt（边双个数），并填好 comp。
- comp[u]: u 所属边双编号，范围 1..comp_cnt。

【API / 入口函数（赛时只认这里列的名字）】

- add_edge(int u, int v) -> 加入一条边
- build() -> 完成建树或预处理 返回 int。
- init(int n_) -> 初始化/清空结构

【改板时先认这几个量】

- g: 邻接表。
- dfn: DFS 序时间戳。
- low: lowlink / 最早可达时间戳。

```
struct EdgeBCC {
    int n, timer = 0, comp_cnt = 0;
    vector<vector<pair<int, int>>> g;
    vector<int> dfn, low, is_bridge, comp;
    vector<pair<int, int>> edges;

    EdgeBCC(int n = 0) { init(n); }
    void init(int n_) {
        n = n_;
        g.assign(n + 1, {});
        dfn.assign(n + 1, 0);
        low.assign(n + 1, 0);
        comp.assign(n + 1, 0);
        is_bridge.clear();
        edges.clear();
        timer = comp_cnt = 0;
    }

    void add_edge(int u, int v) {
        int id = (int)edges.size();
        edges.push_back({u, v});
        is_bridge.push_back(0);
        g[u].push_back({v, id});
        g[v].push_back({u, id});
    }

    void tarjan(int u, int in_edge) {
        dfn[u] = low[u] = ++timer;
        for (auto [v, id] : g[u]) {
            if (!dfn[v]) {
                tarjan(v, id);
                low[u] = min(low[u], low[v]);
                if (low[v] > dfn[u]) is_bridge[id] = 1;
            } else if (id != in_edge) {
                low[u] = min(low[u], dfn[v]);
            }
        }
    }

    void dfs_comp(int u) {
        comp[u] = comp_cnt;
        for (auto [v, id] : g[u]) {
            if (comp[v] || is_bridge[id]) continue;
            dfs_comp(v);
        }
    }

    int build() {
        for (int i = 1; i <= n; i++) if (!dfn[i]) tarjan(i, -1);
        for (int i = 1; i <= n; i++) if (!comp[i]) {
            comp_cnt++;
            dfs_comp(i);
        }
    }
};
```



```

        dfs_comp(i);
    }
    return comp_cnt;
}
};

```

点双连通分量 v-DCC 与圆方树

赛时先看

题目信号：询问割点、点双、任意两点路径是否必须经过某些割点；建圆方树做树上问题。

本质：无向图按割点分解点双连通分量，常用于经过割点的路径统计。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定： $O(n+m)$ 。

维护的量：`bcc`（所有点双的顶点集合）、`cut[u]`（割点标记）、`bcc_cnt`。

警告：点双分量会共享割点；根节点割点判定是子树数大于 1。

【最小完整示例（先抄这一段就能跑）】

题目：无向图列出所有点双连通分量，并标记割点（后续可建圆方树）。

```

VertexBCC vb(n);           // n 为点数，编号 1..n
vb.add_edge(u, v);         // 无向边加一次即可
vb.build();                // 预处理完成
if (vb.cut[u]) cout << "u 是割点\n";
for (auto& c : vb.bcc) { /* c 是一个点双，割点会出现在多个点双里 */ }

```

样例： $n=3$ ，边 1-2 2-3，`cut[2]=1`，`bcc = {{2,1},{2,3}}`。

【传参要求（照这个传不会错）】

- `VertexBCC(n) / init(n_)`：n 为点数，编号 1..n。
- `add_edge(int u, int v)`：加一条无向边，u、v 范围 1..n。
- `build()`：无返回值；执行后 `bcc`、`cut` 就绪。
- `cut[u]`：1 表示 u 是割点；`bcc`：每个元素是一个点双（割点会在多个点双中出现）；`bcc_cnt = bcc.size()`。

【API / 入口函数（赛时只认这里列的名字）】

- `add_edge(int u, int v)` -> 加入一条边
- `build()` -> 完成建树或预处理
- `init(int n_)` -> 初始化/清空结构

【改板时先认这几个量】

- `g`：邻接表。
- `bcc`：所有点双分量。
- `cut`：割点标记。
- `dfn`：DFS 序时间戳。
- `low`：lowlink / 最早可达时间戳。

```

struct VertexBCC {
    int n, timer = 0, bcc_cnt = 0;
    vector<vector<int>> g, bcc;
    vector<int> dfn, low, cut, st;

    VertexBCC(int n = 0) { init(n); }
    void init(int n_) {
        n = n_;
        g.assign(n + 1, {});
        dfn.assign(n + 1, 0);
        low.assign(n + 1, 0);
        cut.assign(n + 1, 0);
        bcc.clear(); st.clear();
        timer = bcc_cnt = 0;
    }

    void add_edge(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void tarjan(int u, int root) {
        dfn[u] = low[u] = ++timer;
    }
};

```

```

    st.push_back(u);
    int child = 0;
    for (int v : g[u]) {
        if (!dfn[v]) {
            child++;
            tarjan(v, root);
            low[u] = min(low[u], low[v]);
            if (low[v] >= dfn[u]) {
                if (u != root || child > 1) cut[u] = 1;
                vector<int> cur;
                while (true) {
                    int x = st.back();
                    st.pop_back();
                    cur.push_back(x);
                    if (x == v) break;
                }
                cur.push_back(u);
                bcc.push_back(cur);
            }
        } else {
            low[u] = min(low[u], dfn[v]);
        }
    }
}

void build() {
    for (int i = 1; i <= n; i++) {
        if (!dfn[i]) {
            st.clear();
            tarjan(i, i);
            if (g[i].empty()) bcc.push_back({i});
        }
    }
    bcc_cnt = (int)bcc.size();
}
};

```

仙人掌图找环

赛时先看

题目信号：题面保证 “each edge appears in at most one cycle”； m 可能大于 $n-1$ 但结构接近树。

本质：无向连通图中每条边最多属于一个简单环，提取所有环，后续可建圆方树、做树形 DP 或距离压缩。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API

列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定： $O(n+m+环长总和)$ ；仙人掌中环长总和 $O(m)$ 。

维护的量：`dep`（深度/访问标记）、`parent/parent_edge`（父点与连向父的边 `id`）、`cycles`（每个环的边 `id` 列表）。

警告：无向边要用边编号跳过父边；只在遇到返祖边 `dep[v] < dep[u]` 时收集一次环。

【最小完整示例（先抄这一段就能跑）】

题目：仙人掌图（每条边至多在一个环中），提取所有环，按环长处理。

```

CactusCycles cc(n);           // n 为点数，编号 1..n
cc.add_edge(u, v);            // 无向边加一次即可
vector<vector<int>> cyc = cc.find_cycles(); // 每个环是边 id 列表
for (auto& c : cyc) cout << c.size() << '\n'; // 环长（边数）

```

样例： $n=3$ ，边 $0:1-2$ $1:2-3$ $2:3-1$ ，`find_cycles()` 返回 `{{0,1,2}}`。

【传参要求（照这个传不会错）】

- 构造 `CactusCycles(n)`： n 为点数，编号 $1..n$ 。
- `add_edge(int u, int v)`：加一条无向边， u, v 范围 $1..n$ ；边 `id` 从 0 起自动分配。
- `find_cycles()`：返回 `cycles`，每个元素是一个环的边 `id` 列表（`id` 0-based，按环上顺序）。

【API / 入口函数（赛时只认这里列的名字）】

- `add_edge(int u, int v)` -> 加入一条边

【改板时先认这几个量】

- `g`：邻接表。
- `cycles`：每个环用边编号列表表示。

```

struct CactusCycles {
    struct Edge {
        int to;

```

```

    int id;
};

int n = 0, edge_count = 0;
vector<vector<Edge>> g;
vector<int> dep, parent, parent_edge;
vector<vector<int>> cycles; // 每个环用边编号列表表示。

CactusCycles(int n = 0) : n(n), g(n + 1), dep(n + 1), parent(n + 1), parent_edge(n + 1, -1) {}

void add_edge(int u, int v) {
    int id = edge_count++;
    g[u].push_back({v, id});
    g[v].push_back({u, id});
}

void dfs(int u, int pe) {
    for (auto [v, id] : g[u]) {
        if (id == pe) continue;
        if (!dep[v]) {
            dep[v] = dep[u] + 1;
            parent[v] = u;
            parent_edge[v] = id;
            dfs(v, id);
        } else if (dep[v] < dep[u]) {
            vector<int> cyc{id};
            int x = u;
            while (x != v) {
                cyc.push_back(parent_edge[x]);
                x = parent[x];
            }
            cycles.push_back(cyc);
        }
    }
}

vector<vector<int>> find_cycles() {
    cycles.clear();
    for (int s = 1; s <= n; s++) {
        if (!dep[s]) {
            dep[s] = 1;
            dfs(s, -1);
        }
    }
    return cycles;
}
};

```

2-SAT

赛时先看

题目信号：每个对象二选一；条件形如“选 A 就必须选 B”“A 和 B 至少一个”。

本质：布尔变量约束可行性。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定： $O(n+m)$ 。

维护的量：`g`（蕴含图）、`comp`（SCC 编号）、`assignment`（求得的 0/1 赋值）。

警告：`x` 和 `not x` 在同一 SCC 中则无解。

约定：`return 2 * x + (val ? 0 : 1);` // `x` 是 0-based 下标

【最小完整示例（先抄这一段就能跑）】

题目：`n` 个布尔变量，条件形如“`a` 为真则 `b` 为真”“`a` 或 `b` 至少一个”，求一组可行赋值。

```

TwoSAT ts(n); // n 个变量，下标 0..n-1
ts.imply(a, true, b, false); // a=true => b=false
ts.add_or(a, false, b, true); // (a=false) 或 (b=true)，即 a 假则 b 真
vector<int> assign;
if (ts.solve(assign)) for (int x : assign) cout << x; // 输出 0/1 赋值
else cout << "无解\n";

```

样例：`n=1, add_or(0,true,0,true)`（即 `x x`），输出 `1`。

【传参要求（照这个传不会错）】

- `TwoSAT(n)` / `init(n)`: `n` 为变量个数；变量 `x` 用 0-based 下标 `0..n-1`。
- `id(int x, bool val)`: `x` 0-based; `val=true` 得 `2x`, `val=false` 得 `2x+1`。

- `imply(a, va, b, vb)`: 加蕴含边 $(a=va) \Rightarrow (b=vb)$ 。
- `add_or(a, va, b, vb)`: 加两个蕴含边, 表示 $(a=va)$ 或 $(b=vb)$ 至少一个成立。
- `solve(assignment)`: 有解返回 `true` 并写入 `assignment[x]` (0/1); 无解返回 `false`。

【API / 入口函数（赛时只认这里列的名字）】

- `init(int n_)` -> 初始化/清空结构
- `solve(vector<int>& assignment)` -> 执行主算法并返回答案

【改板时先认这几个量】

- `g`: 邻接表。
- `dfn`: DFS 序时间戳。
- `low`: `lowlink` / 最早可达时间戳。

```
struct TwoSAT {
    int n;
    vector<vector<int>> g;
    vector<int> dfn, low, comp, st, in_st;
    int timer = 0, comp_cnt = 0;

    TwoSAT(int n = 0) { init(n); }
    void init(int n_) {
        n = n_;
        g.assign(2 * n, {});
        dfn.assign(2 * n, 0);
        low.assign(2 * n, 0);
        comp.assign(2 * n, 0);
        in_st.assign(2 * n, 0);
        st.clear();
        timer = comp_cnt = 0;
    }

    int id(int x, bool val) const {
        return 2 * x + (val ? 0 : 1); // x 是 0-based 下标。
    }

    void imply(int a, bool va, int b, bool vb) {
        g[id(a, va)].push_back(id(b, vb));
    }

    void add_or(int a, bool va, int b, bool vb) {
        imply(a, !va, b, vb);
        imply(b, !vb, a, va);
    }

    void dfs(int u) {
        dfn[u] = low[u] = ++timer;
        st.push_back(u);
        in_st[u] = 1;
        for (int v : g[u]) {
            if (!dfn[v]) dfs(v), low[u] = min(low[u], low[v]);
            else if (in_st[v]) low[u] = min(low[u], dfn[v]);
        }
        if (low[u] == dfn[u]) {
            comp_cnt++;
            while (true) {
                int x = st.back();
                st.pop_back();
                in_st[x] = 0;
                comp[x] = comp_cnt;
                if (x == u) break;
            }
        }
    }

    bool solve(vector<int>& assignment) {
        for (int i = 0; i < 2 * n; ++i) if (!dfn[i]) dfs(i);
        assignment.assign(n, 0);
        for (int i = 0; i < n; ++i) {
            if (comp[2 * i] == comp[2 * i + 1]) return false;
            assignment[i] = comp[2 * i] > comp[2 * i + 1];
        }
        return true;
    }
};
```

欧拉路径/回路: Hierholzer

赛时先看

题目信号：题面要求“一笔画”“每条边恰好走一次”。

本质：输出经过每条边一次的路径。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定： $O(n+m)$ 。

维护的量：`used[id]`（边是否已走）、`path`（走出的顶点序列，倒序存储）。

警告：本模板仅实现无向图版本；有向图请自行改边表。无向图要标记边 `id`。

【最小完整示例（先抄这一段就能跑）】

题目：无向图求一笔画回路（每条边恰好走一次），输出顶点顺序。

```
EulerUndirected eu(n);           // n 为点数，编号 1..n
eu.add_edge(u, v);               // 无向边加一次即可
eu.dfs(1);                       // 从起点开始走（先自行验证度数条件）
reverse(eu.path.begin(), eu.path.end()); // path 是倒序，反转后输出
for (int x : eu.path) cout << x << ' ';
```

样例：三角形 1-2-3-1，输出 1 2 3 1。

【传参要求（照这个传不会错）】

- `EulerUndirected(n) / init(n_)`：n 为点数，编号 1..n。
- `add_edge(int u, int v)`：加一条无向边，u、v 范围 1..n；边 `id` 自动分配。
- `dfs(int start)`：从 `start` 开始；调用前需自行保证存在欧拉回路/路径（度数条件）。
- `path`：顶点序列，倒序存储，输出前要 `reverse`；无解时不会走完全部边，需自行判长度。

【API / 入口函数（赛时只认这里列的名字）】

- `add_edge(int u, int v)` -> 加入一条边
- `init(int n_)` -> 初始化/清空结构

```
struct EulerUndirected {
    int n, edge_id = 0;
    vector<vector<pair<int, int>>> g;
    vector<int> used, path;

    EulerUndirected(int n = 0) { init(n); }
    void init(int n_) {
        n = n_;
        g.assign(n + 1, {});
        used.clear();
        path.clear();
        edge_id = 0;
    }

    void add_edge(int u, int v) {
        g[u].push_back({v, edge_id});
        g[v].push_back({u, edge_id});
        used.push_back(0);
        edge_id++;
    }

    void dfs(int u) {
        while (!g[u].empty()) {
            auto [v, id] = g[u].back();
            g[u].pop_back();
            if (used[id]) continue;
            used[id] = 1;
            dfs(v);
        }
        path.push_back(u);
    }
};
```

bitset 传递闭包

赛时先看

题目信号： $n \leq 3000/5000$ ，问很多可达性。

本质：有向图可达性闭包，比 Floyd 快很多。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O(n^3 / \text{word_size})$ 。

维护的量: `reach[u]` (bitset: `u` 能到达的所有点)。

警告: 先按拓扑序/编号逆序处理 DAG; 一般图可先 SCC 缩点。

【最小完整示例 (先抄这一段就能跑)】

题目: DAG 上问所有点对可达性 (`u` 能否到达 `v`), 多次查询。

```
auto reach = dag_transitive_closure<5005>(n, dag, topo); // MAXN 需 >= n+1
if (reach[u][v]) cout << "u 可达 v\n";
```

样例: `n=3`, 边 `1->2 2->3`, `reach[1]={1,2,3}`、`reach[2]={2,3}`、`reach[3]={3}`。

【传参要求 (照这个传不会错)】

- `MAXN` (模板参数): bitset 长度, 必须 `>= n+1` (如 5005)。
- `n`: 顶点数, 编号 `1..n`。
- `dag`: 邻接表, `dag[u]` 存 `u` 的后继 `v`, 范围 `1..n`。
- `topo`: 长度 `n` 的合法拓扑序 (含全部顶点); 一般图要先 SCC 缩点成 DAG 再传。
- 返回值 `reach`: `reach[u]` 是 `bitset<MAXN>`, `reach[u][v]=1` 表示 `u` 可达 `v`。

```
template <int MAXN>
vector<bitset<MAXN>> dag_transitive_closure(int n, const vector<vector<int>>& dag, const vector<int>& topo) {
    vector<bitset<MAXN>> reach(n + 1);
    for (int idx = n - 1; idx >= 0; --idx) {
        int u = topo[idx];
        reach[u][u] = 1;
        for (int v : dag[u]) reach[u] |= reach[v];
    }
    return reach;
}
```

最大团: Bron-Kerbosch bitset

赛时先看

题目信号: `n <= 60/100`, 问最大互相相连集合。

本质: 小图最大团/最大独立集。

接法: 把本节 `struct/class/函数组` 整段抄到 `solve()` 上面; `solve()`

里直接按下方“不会用就看这里”调用, 不需要自己猜内部参数。

复杂度判定: 指数级, 但剪枝强。

维护的量: `adj_clique[i]` (点 `i` 的邻接 bitset, 0-indexed)、`best_clique` (全局答案: 当前最大团大小)。

警告: 最大独立集可在补图上求最大团; `n` 必须 `<= MAXC`, 更大请改大 `MAXC`。

【最小完整示例 (先抄这一段就能跑)】

题目: `n` 点无向图, 求最大团大小 (两两相连的最大点集)。

```
int n, m;
cin >> n >> m;
for (int i = 0; i < n; ++i) adj_clique[i].reset(); // 1. 邻接 bitset, 下标 0..n-1
for (int i = 0; i < m; ++i) {
    int u, v;
    cin >> u >> v;
    adj_clique[u][v] = adj_clique[v][u] = 1;
}
best_clique = 0;
bitset<MAXC> P; // 2. 候选集: 全部点
for (int i = 0; i < n; ++i) P[i] = 1;
bron_kerbosch(bitset<MAXC>(), P, n); // 3. R 空集、P 全选
cout << best_clique << '\n'; // 4. 读全局答案
```

样例: `n=4`, 边 `(0,1)(1,2)(2,3)(0,2)` -> 输出 `3` (最大团 `{0,1,2}`)。

【传参要求 (照这个传不会错)】

- `MAXC`: 顶点数上限常量, 必须 `n <= MAXC` (默认 100, 更大就改大它)。
- `adj_clique[i]`: 点 `i` 的邻接 bitset (`i` 属于 `[0, n)`), 先建好再调用, 函数只读它。
- `bron_kerbosch(R, P, n)`: `R` 当前已选点 (初始传空集 `bitset<MAXC>()`)、`P` 候选点 (初始全部 1)、`n` 顶点数; 递归过程中会修改局部的 `R/P`, 按值传递即可。
- 返回值: 无; 答案写在全局 `best_clique`, 调用完直接输出。

```
const int MAXC = 100;
int best_clique;
```

```

bitset<MAXC> adj_clique[MAXC];

void bron_kerbosch(bitset<MAXC> R, bitset<MAXC> P, int n) {
    if (P.none()) {
        best_clique = max(best_clique, (int)R.count());
        return;
    }
    if ((int)R.count() + (int)P.count() <= best_clique) return;
    int pivot = -1, mx = -1;
    for (int i = 0; i < n; ++i) if (P[i]) {
        int c = (int)(P & adj_clique[i]).count();
        if (c > mx) mx = c, pivot = i;
    }
    bitset<MAXC> cand = P & ~adj_clique[pivot];
    for (int v = 0; v < n; ++v) if (cand[v]) {
        R[v] = 1;
        bron_kerbosch(R, P & adj_clique[v], n);
        R[v] = 0;
        P[v] = 0;
        if ((int)R.count() + (int)P.count() <= best_clique) return;
    }
}

```

三元环计数：按度定向枚举

赛时先看

题目信号：题面问三角形、三个点两两相连、长度为 3 的环； m 较大，不能 $O(n^3)$ 。

本质：统计简单无向图中的三元环数量，也可以在三角形上累加权值贡献。

接法：先按 $(\text{degree}, \text{id})$ 从小到大给每条边定向；枚举 $u \rightarrow v \rightarrow w$ ，检查 $u \rightarrow w$ 是否存在。每个三角形只会被最小的那个点统计一次。

复杂度判定：按度定向后 $O(m \sqrt{m})$ 级别，实际常数较小。

维护的量： $\text{degree}[u]$ （原图度数，定向比较依据）； $g[u]$ （按 $(\text{degree}, \text{id})$ 定向后的有向邻接表）； mark （标记当前枚举点的邻居）； answer （三元环计数）。

警告：重边和自环会破坏计数，读入时应去重或确认题目是简单图；定向规则必须全局一致，否则会重复或漏算。

【最小完整示例（先抄这一段就能跑）】

题目： n 点 m 条无向边，统计三元环个数。

```

vector<pair<int, int>> edges;
for (int i = 0; i < m; ++i) {
    int u, v; cin >> u >> v;
    edges.push_back({u, v}); // 顶点编号 1..n
}
i64 ans = count_triangles(n, edges); // 无向图三元环个数
cout << ans << '\n';

```

样例：边 1-2 2-3 1-3 $\rightarrow$ $\text{ans} = 1$ 。

【传参要求（照这个传不会错）】

- n ：顶点数，编号 1..n。
- edges ：每条无向边一个 $\text{pair}<\text{int}, \text{int}>$ ，编号 1..n；按值传参，原数组不会被改。
- 返回值： i64 三元环个数。
- 注意：重边会被重复计数，读入前去重或确认是简单图；自环代码自动跳过。

```

i64 count_triangles(int n, vector<pair<int, int>> edges) {
    vector<int> degree(n + 1);
    for (auto [u, v] : edges) {
        if (u == v) continue; // 简单图题一般没有自环，这里防一下。
        ++degree[u];
        ++degree[v];
    }

    vector<vector<int>> g(n + 1);
    auto less_order = [&](int a, int b) {
        if (degree[a] != degree[b]) return degree[a] < degree[b];
        return a < b;
    };

    for (auto [u, v] : edges) {
        if (u == v) continue;
        if (less_order(v, u)) swap(u, v);
        g[u].push_back(v); // 只保留从低优先级到高优先级的有向边。
    }

    vector<int> mark(n + 1, 0);

```



```

i64 answer = 0;
for (int u = 1; u <= n; ++u) {
    for (int v : g[u]) mark[v] = 1;
    for (int v : g[u]) {
        for (int w : g[v]) {
            if (mark[w]) ++answer;
        }
    }
    for (int v : g[u]) mark[v] = 0;
}
return answer;
}

```

08 网络流、匹配与割建模

二分图、一般图匹配、稳定匹配、最大流、费用流、上下界流、闭合图和全局/点对最小割集中放在本章。

二分图染色判定

赛时先看

题目信号：题面问“能否分给 A/B 两方”“能否染成两种颜色”“同组内任意两点不能满足某种冲突关系”；一旦两个对象不能放同一组，就在它们之间连一条无向边。

本质：判断一堆对象能否分成两组，使所有“不能同组”的冲突对都被分到不同组。

接法：先把题面条件翻译成“哪些点不能同组”。例如“同一方距离必须大于 d ”，那么距离 $\leq d$ 的两点就是冲突边。建好冲突图后调用 `is_bipartite(n, g, color)`。

复杂度判定： $O(n+m)$ 。

维护的量：`color[u]`：点 u 的分组 (0/1)，-1 表示还没处理。

警告：图可能不连通，要从每个未染色点开始 BFS/DFS；`color[u]` 只是第 u 个对象被分到第 0/1 组，不代表输入里天然的左右部；冲突边必须双向加入。

参数怎么传：

- **n**：对象数量，默认对象编号为 $1..n$ 。
- **g**：冲突图邻接表，`g[u]` 里放所有“不能和 u 同组”的对象编号 v 。如果 u 和 v 冲突，要同时写 `g[u].push_back(v)` 和 `g[v].push_back(u)`。
- **color**：输出数组。函数会把它改成长度 $n+1$ ，`color[u]=0/1` 表示对象 u 分到哪一组，-1 表示还没处理。
- 返回值：**true** 表示可以分成两组；**false** 表示某条边两端被迫同色，也就是出现了奇环，无法满足。

```

bool is_bipartite(int n, const vector<vector<int>>& g, vector<int>& color) {
    // color[u] 的含义：
    // -1: 点 u 还没有分组；
    // 0: 点 u 分到第一组；
    // 1: 点 u 分到第二组。
    // 说明：
    // 注意：这两个组的名字随便叫，可以对应题面里的“武松/施恩”“黑/白”“A/B”。
    color.assign(n + 1, -1);

    // 图可能不连通，所以不能只从 1 号点 BFS。
    // 每遇到一个还没染色的点，就把它当成一个新连通块的起点。
    for (int s = 1; s <= n; ++s) {
        if (color[s] != -1) continue;

        queue<int> q;
        color[s] = 0; // 新连通块第一个点放哪组都可以。
        q.push(s);

        while (!q.empty()) {
            int u = q.front();
            q.pop();

            // g[u] 存的是所有和 u 有冲突、不能同组的点。
            // 所以每条边 u-v 都要求 color[v] = color[u] ^ 1。
            for (int v : g[u]) {
                if (color[v] == -1) {
                    // v 还没分组，就把它放到 u 的另一组。
                    color[v] = color[u] ^ 1;
                    q.push(v);
                } else if (color[v] == color[u]) {
                    // v 已经分组，但它和 u 同组；而 u-v 是冲突边。
                    // 说明条件互相矛盾，不存在合法分法。
                    return false;
                }
            }
        }
    }
}

```

```

    }
    return true;
}

```

典型建图：如果题面说“同组内任意两点距离必须大于 d ”，那么距离 $\leq d$ 的两点不能同组，要连冲突边。

```

struct Point {
    i64 x, y;
};

bool can_split_points_by_distance(const vector<Point>& p, i64 d, vector<int>& color) {
    int n = (int)p.size();
    vector<vector<int>> g(n + 1);
    i128 limit = (i128)d * d;

    for (int i = 0; i < n; ++i) {
        for (int j = i + 1; j < n; ++j) {
            i128 dx = (i128)p[i].x - p[j].x;
            i128 dy = (i128)p[i].y - p[j].y;
            i128 dist2 = dx * dx + dy * dy;

            // 同组要求距离 > d.
            // 因此距离 <= d 的两点是“冲突对”，必须分到不同组。
            if (dist2 <= limit) {
                int u = i + 1;
                int v = j + 1;
                g[u].push_back(v);
                g[v].push_back(u);
            }
        }
    }

    return is_bipartite(n, g, color);
}

```

这类题的完整使用方式：

```

void solve() {
    int n;
    i64 d;
    cin >> n >> d;

    vector<Point> p(n);
    for (auto& point : p) cin >> point.x >> point.y;

    vector<int> color;
    cout << (can_split_points_by_distance(p, d, color) ? "YES" : "NO") << '\n';
}

```

典题：两组分配哨岗；同一方任意两哨岗欧几里得距离必须大于 d 。先连出所有距离 $\leq d$ 的冲突边，再判二分图。

出处：码蹄杯 2025 官方题单 MC0421 分配哨岗。

二分图最大匹配 Hopcroft-Karp

赛时先看

题目信号：左集合与右集合配对，求最多配多少对。

本质：大二分图匹配，每个点最多匹配一次。

接法：把对象分成左、右两类，能配对就 `add_edge(left_id, right_id)`；调用 `max_matching()` 得到最多配对数。匹配结果在 `ml[left]` 和 `mr[right]` 里，值为 $\emptyset$

表示未匹配。若左右不是天然两类，先做二分图染色，不是二分图就不能用这个模板。

复杂度判定： $O(E \sqrt{V})$ 。

维护的量：`ml[l]`（左点 l 配到的右点）、`mr[r]`（右点 r 配到的左点， $\emptyset$ =未匹配）、`dist`（增广路分层）。

警告：左侧编号 $1..nL$ ，右侧编号 $1..nR$ 。

【最小完整示例（先抄这一段就能跑）】

题目：左右各 nL 、 nR 个点， m 条可配对边，求最多配对数。

```

HopcroftKarp hk(nL, nR);          // 1. 构造：左侧 nL 个、右侧 nR 个点
for (int i = 0; i < m; ++i) {
    int l, r;
    cin >> l >> r;
    hk.add_edge(l, r);             // 2. 加边：左点 l 可与右点 r 配对
}
cout << hk.max_matching() << '\n'; // 3. 返回最大匹配数

```

样例： $nL=3, nR=3$ ，边 $(1,1)(1,2)(2,2)(3,3) \rightarrow$ 输出 3。

【传参要求（照这个传不会错）】

- `init(L, R)` / 构造: `L` 左部点数、`R` 右部点数; 左右都从 1 编号 (0 是“未匹配”空位)。
- `add_edge(l, r)`: `l` 属于 `[1, nL]`, `r` 属于 `[1, nR]`; 表示左点 `l` 可与右点 `r` 配对, 加重复边无害。
- `max_matching()`: 返回 `int` 最大匹配数; 跑完后 `ml[l]` 是左点 `l` 配到的右点、`mr[r]` 是右点 `r` 配到的左点, 值为 0 表示未匹配。

【API / 入口函数（赛时只认这里列的名字）】

- `add_edge(int l, int r)` -> 加入一条边
- `init(int L, int R)` -> 初始化/清空结构

【改板时先认这几个量】

- `g`: 邻接表。
- `dist`: 距离。

```
struct HopcroftKarp {
    int nL, nR;
    vector<vector<int>> g;
    vector<int> dist, ml, mr;

    HopcroftKarp(int L = 0, int R = 0) { init(L, R); }

    void init(int L, int R) {
        nL = L;
        nR = R;
        g.assign(nL + 1, {});
        ml.assign(nL + 1, 0);
        mr.assign(nR + 1, 0);
        dist.resize(nL + 1);
    }

    void add_edge(int l, int r) {
        g[l].push_back(r);
    }

    bool bfs() {
        queue<int> q;
        for (int i = 1; i <= nL; ++i) {
            if (!ml[i]) dist[i] = 0, q.push(i);
            else dist[i] = -1;
        }
        bool found = false;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int v : g[u]) {
                int u2 = mr[v];
                if (!u2) found = true;
                else if (dist[u2] == -1) {
                    dist[u2] = dist[u] + 1;
                    q.push(u2);
                }
            }
        }
        return found;
    }

    bool dfs(int u) {
        for (int v : g[u]) {
            int u2 = mr[v];
            if (!u2 || (dist[u2] == dist[u] + 1 && dfs(u2))) {
                ml[u] = v;
                mr[v] = u;
                return true;
            }
        }
        dist[u] = -1;
        return false;
    }

    int max_matching() {
        int ans = 0;
        while (bfs()) {
            for (int i = 1; i <= nL; ++i) {
                if (!ml[i] && dfs(i)) ans++;
            }
        }
        return ans;
    }
};
```

};

二分图最小点覆盖

赛时先看

题目信号：二分图 + 覆盖所有边 + 最少点。根据 Konig 定理，大小等于最大匹配。

本质：在二分图中选最少点覆盖所有边。

复杂度判定：匹配后 $O(n+m)$ 。

维护的量：visL/visR（交替路访问标记）、coverL/coverR（选入最小点覆盖的左右点编号）。

警告：从左侧未匹配点出发，沿“非匹配边左到右、匹配边右到左”走；答案是“未访问左点 + 已访问右点”。必须先运行第 66 节 Hopcroft-Karp 得到 ml/mr 再调用。

【最小完整示例（先抄这一段就能跑）】

题目：二分图已跑完 Hopcroft-Karp（得到 ml/mr），求最小点覆盖（覆盖点数 = 最大匹配数）。

```
auto [coverL, coverR] =
    min_vertex_cover_bipartite(nL, nR, g, ml, mr); // 1. 传左右点数、左部邻接表、匹配结果
cout << coverL.size() + coverR.size() << '\n'; // 2. 覆盖点数，等于最大匹配数
for (int x : coverL) cout << x << ' '; // 3. 左侧被选点（编号 1..nL）
for (int y : coverR) cout << y << ' '; // 4. 右侧被选点（编号 1..nR）
```

样例：nL=2, nR=2, 边 (1,1)(1,2)(2,1), 最大匹配 2 → 覆盖点数 2。

【传参要求（照这个传不会错）】

- nL / nR: 左右部点数，左点编号 1..nL、右点编号 1..nR。
- g: 左部邻接表 g[l] 存左点 l 连到的右点 r，必须和跑 Hopcroft-Karp 时用同一个图。
- ml / mr: Hopcroft-Karp 跑完的匹配数组，长度 nL+1 / nR+1，值 0 表示未匹配。
- 返回值: pair<vector<int>, vector<int>>: first 是左侧选入覆盖的点（coverL）、second 是右侧选入覆盖的点（coverR），两边加起来是一组最小点覆盖。

```
// 已有二分图 g[1..nL] → right 1..nR，以及最大匹配 ml, mr。
pair<vector<int>, vector<int>> min_vertex_cover_bipartite(
    int nL, int nR,
    const vector<vector<int>>& g,
    const vector<int>& ml,
    const vector<int>& mr
) {
    vector<int> visL(nL + 1, 0), visR(nR + 1, 0);
    queue<int> q;
    for (int i = 1; i <= nL; ++i) {
        if (!ml[i]) {
            visL[i] = 1;
            q.push(i);
        }
    }
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : g[u]) {
            if (ml[u] == v) continue; // 只沿非匹配边从左部走到右部。
            if (visR[v]) continue;
            visR[v] = 1;
            if (mr[v] && !visL[mr[v]]) {
                visL[mr[v]] = 1;
                q.push(mr[v]);
            }
        }
    }
    vector<int> coverL, coverR;
    for (int i = 1; i <= nL; ++i) if (!visL[i]) coverL.push_back(i);
    for (int i = 1; i <= nR; ++i) if (visR[i]) coverR.push_back(i);
    return {coverL, coverR};
}
```

Gale-Shapley: 稳定匹配 / 稳定婚姻

赛时先看

题目信号：不是求最大匹配或最大权匹配；题面强调志愿、偏好、录取、配对后不能存在一对人更愿意彼此选择；每个人都有候选排序。

本质：两侧个体各自给出严格偏好，求一个没有“互相都想换对象”的稳定匹配。左侧提出申请时，得到的是对左侧最优的稳定匹配。

接法：医院-住院医师、学校志愿、双边录取、偏好约束下的稳定配对；SPOJ STABLEMP 是直接模板题。

复杂度判定： $O(P + nL \cdot nR)$ ， P 是左侧偏好表总长度；这里额外建右侧排名表，空间

$O(nL \cdot nR)$ 。支持不完整偏好表与两侧人数不同。

维护的量： $match_left[x]$ （左侧 x 最终匹配的右侧）、 $next[x]$ （左侧 x 下次要提亲的位置）、 $rank[y][x]$ （右侧 y 对左侧 x 的偏好排名，越小越喜欢）。

警告：右侧排名越小代表越喜欢；左侧列出而右侧没有列出的对象视为不可接受。想让右侧最优，交换两侧输入后再把答案映回。它解决的是稳定性，不是“匹配数最大”或“总权值最大”。

【最小完整示例（先抄这一段就能跑）】

题目： nL 个男生、 nR 个女生各给偏好排序，求稳定匹配（返回对男生最优的）。

```
int nL, nR;
cin >> nL >> nR;
vector<vector<int>>> pref_left(nL), pref_right(nR); // 1. 下标 0..nL-1 / 0..nR-1
// 2. 读偏好：每人一个 vector，按“更喜欢在前”写入对方编号
auto match = gale_shapley(pref_left, pref_right); // 3. 返回 match_left
for (int x = 0; x < nL; ++x) cout << match[x] << ' '; // 4. 男生 x 配到的女生，-1 未匹配
```

样例： $pref_left=[[0,1],[1,0]]$ 、 $pref_right=[[1,0],[0,1]]$ → 输出 0 1。

【传参要求（照这个传不会错）】

- $pref_left[x]$ ：左侧 x （下标 $[0, nL)$ ）的偏好表，按“更喜欢在前”列右侧编号 $[0, nR)$ 。
- $pref_right[y]$ ：右侧 y （下标 $[0, nR)$ ）的偏好表，按“更喜欢在前”列左侧编号 $[0, nL)$ ；没列出的对象视为不可接受，可为空表。
- 返回值： $vector<int>$ 即 $match_left[x]$ ：左侧 x 配到的右侧编号， -1 表示未匹配；这是对左侧最优的稳定匹配。

使用： $pref_left[x]$ 与 $pref_right[y]$ 都按“更喜欢在前”列出可接受对象，编号分别为 $[0, nL)$ 与 $[0, nR)$ 。返回 $match_left[x]$ ， -1 表示未匹配。

典题模型：医院-住院医师、学校志愿、双边录取、偏好约束下的稳定配对；SPOJ STABLEMP 是直接模板题。

```
// 返回左侧到右侧的匹配；两侧偏好表可以不完整。
vector<int> gale_shapley(
    const vector<vector<int>>& pref_left,
    const vector<vector<int>>& pref_right
) {
    const int nL = (int)pref_left.size();
    const int nR = (int)pref_right.size();
    const int INF = 1e9;

    // rank[y][x] 越小，说明右侧 y 越喜欢左侧 x。
    vector<vector<int>>> rank(nR, vector<int>(nL, INF));
    for (int y = 0; y < nR; ++y) {
        for (int i = 0; i < (int)pref_right[y].size(); ++i) {
            rank[y][pref_right[y][i]] = i;
        }
    }

    vector<int> next(nL, match_left(nL, -1), match_right(nR, -1));
    queue<int> q;
    for (int x = 0; x < nL; ++x) q.push(x);

    auto retry = [&](int x) {
        if (next[x] < (int)pref_left[x].size()) q.push(x);
    };

    while (!q.empty()) {
        int x = q.front(); q.pop();
        if (next[x] == (int)pref_left[x].size()) continue;
        int y = pref_left[x][next[x]++];

        // 右侧 y 没把 x 列为可接受对象，直接拒绝。
        if (rank[y][x] == INF) {
            retry(x);
            continue;
        }
        int old = match_right[y];
        if (old == -1 || rank[y][x] < rank[y][old]) {
            if (old != -1) {
                match_left[old] = -1;
                retry(old);
            }
            match_left[x] = y;
            match_right[y] = x;
        } else {
            retry(x);
        }
    }
}
```

```

        retry(x);
    }
}
return match_left;
}

```

KM：二分图最大权完美匹配

赛时先看

题目信号：指派问题、每个人分配一个任务、要求总权最大且一一匹配。

本质：左右各 n 个点，求完美匹配最大权。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O(n^3)$ 。

维护的量：`lx/ly`（左右顶标）、`slack`（松弛量）、`match_y[y]`（右部 y 配到的左点）、`pre`（交替树前驱）。

警告：权值可以为负时初始化要小；若求最小权，把权值取反。题目保证有完美匹配时才可用，否则返回 -1 。

约定： w 是 1-indexed 的 $n+1$ 方阵。

【最小完整示例（先抄这一段就能跑）】

题目： n 人分 n 个任务，`w[i][j]` 是第 i 人做第 j 事的收益，求总收益最大的完美指派。

```

int n;
cin >> n;
vector<vector<i64>> w(n + 1, vector<i64>(n + 1)); // 1. 1-indexed 的 n+1 方阵
for (int i = 1; i <= n; ++i)
    for (int j = 1; j <= n; ++j) cin >> w[i][j]; // 2. 读权值
cout << km_max_weight_matching(w) << '\n'; // 3. 返回最大总收益；无完美匹配返回 -1

```

样例： $n=2$ ，`w[1][1]=5`，`w[1][2]=1`，`w[2][1]=2`，`w[2][2]=6` $\rightarrow$ 输出 11。

【传参要求（照这个传不会错）】

- w ： $(n+1) \times (n+1)$ 的 `i64` 方阵，1-indexed：`w[i][j]` 是左点 i （人）配右点 j （任务）的权， i/j 属于 $[1, n]$ ，`w[0]` 那一行/列不用。
- 返回值：`i64` 完美匹配的最大总权；若不存在完美匹配返回 -1 。
- 求最小权：把所有权值取反传入，答案取负即可。

【改板时先认这几个量】

- `lx/ly`：左右顶标。
- `slack`：松弛量。
- `match_y`：右部点的匹配左点。

```

// 题目保证有完美匹配时才可用；否则返回 -1。
i64 km_max_weight_matching(const vector<vector<i64>>& w) {
    int n = (int)w.size() - 1;
    const i64 INF = (1LL << 60);
    vector<i64> lx(n + 1), ly(n + 1), slack(n + 1);
    vector<int> match_y(n + 1), pre(n + 1);
    for (int i = 1; i <= n; i++) {
        lx[i] = w[i][1];
        for (int j = 2; j <= n; j++) lx[i] = max(lx[i], w[i][j]);
    }
    for (int root = 1; root <= n; root++) {
        vector<int> vx(n + 1), vy(n + 1);
        fill(slack.begin(), slack.end(), INF);
        int py = 0;
        match_y[0] = root;
        do {
            vy[py] = 1;
            int x = match_y[py], yy = 0;
            i64 delta = INF;
            for (int y = 1; y <= n; y++) if (!vy[y]) {
                i64 cur = lx[x] + ly[y] - w[x][y];
                if (cur < slack[y]) {
                    slack[y] = cur;
                    pre[y] = py;
                }
                if (slack[y] < delta) {
                    delta = slack[y];
                    yy = y;
                }
            }
            py = yy;
        } while (py != 0);
        for (int y = 0; y <= n; y++) {

```

```

        if (vy[y]) {
            lx[match_y[y]] -= delta;
            ly[y] += delta;
        } else {
            slack[y] -= delta;
        }
    }
    py = yy;
} while (match_y[py]);
while (py) {
    int last = pre[py];
    match_y[py] = match_y[last];
    py = last;
}
}
i64 ans = 0;
for (int y = 1; y <= n; y++) {
    if (match_y[y] == 0) return -1; // 不存在完美匹配。
    ans += w[match_y[y]][y];
}
return ans;
}

```

一般图最大匹配 Blossom

赛时先看

题目信号：图不是二分图，但要求最多匹配边数； n 通常几百到几千。

本质：非二分图最大匹配。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定：常见实现 $O(n^3)$ 。

维护的量：`match[u]` (u 的配对点， 0 =未匹配)、`base[u]` (花基/所在花)、`p[u]` (交替树父亲)、`used` (访问标记)。

警告：缩花时 LCA 与 blossom 标记最容易写错；点编号从 1 开始。

【最小完整示例（先抄这一段就能跑）】

题目： n 点 m 条无向边的非二分图，求最大匹配边数。

```

Blossom b(n); // 1. 构造：顶点编号 1..n
for (int i = 0; i < m; ++i) {
    int u, v;
    cin >> u >> v;
    b.add_edge(u, v); // 2. 加无向边，反向自动存好
}
cout << b.solve() << '\n'; // 3. 返回最大匹配边数

```

样例： $n=4$ ，边 $(1,2)(2,3)(3,4)(4,1)$ $\rightarrow$ 输出 2。

【传参要求（照这个传不会错）】

- `init(n_)` / 构造： $n_$ 顶点数，顶点编号 $1..n$ (0 是空位)。
- `add_edge(u, v)`：无向边， u/v 属于 $[1, n]$ ；内部自动存双向，不需要自己加两条。
- `solve()`：返回 `int` 最大匹配边数；跑完后 `match[u]` 是与 u 配对的点， 0 表示未匹配。

【API / 入口函数（赛时只认这里列的名字）】

- `add_edge(int u, int v)` $\rightarrow$ 加入一条边
- `init(int n_)` $\rightarrow$ 初始化/清空结构
- `lca(int a, int b)` $\rightarrow$ 最近公共祖先 返回 `int`。
- `solve()` $\rightarrow$ 执行主算法并返回答案

【改板时先认这几个量】

- `g`：邻接表。
- `match`：匹配关系。
- `base`：花基/所在花。
- `p`：交替树父亲。

```

struct Blossom {
    int n;
    vector<vector<int>> g;
    vector<int> match, p, base, q;
    vector<int> used, blossom;
}

```

```

Blossom(int n = 0) { init(n); }
void init(int n_) {
    n = n_;
    g.assign(n + 1, {});
    match.assign(n + 1, 0);
    p.resize(n + 1); base.resize(n + 1); q.resize(n + 1);
    used.resize(n + 1); blossom.resize(n + 1);
}
void add_edge(int u, int v) { g[u].push_back(v); g[v].push_back(u); }

int lca(int a, int b) {
    vector<int> used_lca(n + 1, 0);
    while (true) {
        a = base[a];
        used_lca[a] = 1;
        if (!match[a]) break;
        a = p[match[a]];
    }
    while (true) {
        b = base[b];
        if (used_lca[b]) return b;
        b = p[match[b]];
    }
}

void mark_path(int v, int b, int children) {
    while (base[v] != b) {
        blossom[base[v]] = blossom[base[match[v]]] = 1;
        p[v] = children;
        children = match[v];
        v = p[match[v]];
    }
}

int find_path(int root) {
    fill(used.begin(), used.end(), 0);
    fill(p.begin(), p.end(), 0);
    iota(base.begin(), base.end(), 0);
    int qh = 0, qt = 0;
    q[qt++] = root;
    used[root] = 1;
    while (qh < qt) {
        int v = q[qh++];
        for (int u : g[v]) {
            if (base[v] == base[u] || match[v] == u) continue;
            if (u == root || (match[u] && p[match[u]])) {
                int cur = lca(v, u);
                fill(blossom.begin(), blossom.end(), 0);
                mark_path(v, cur, u);
                mark_path(u, cur, v);
                for (int i = 1; i <= n; i++) {
                    if (blossom[base[i]]) {
                        base[i] = cur;
                        if (!used[i]) {
                            used[i] = 1;
                            q[qt++] = i;
                        }
                    }
                }
            }
            else if (!p[u]) {
                p[u] = v;
                if (!match[u]) return u;
                u = match[u];
                used[u] = 1;
                q[qt++] = u;
            }
        }
    }
    return 0;
}

int solve() {
    int matching = 0;
    for (int i = 1; i <= n; i++) if (!match[i]) {
        int v = find_path(i);
        if (!v) continue;
        matching++;
        while (v) {
            int pv = p[v], nv = match[pv];
            match[v] = pv;
            match[pv] = v;
            v = nv;
        }
    }
}

```



```

        return matching;
    }
};

```

Dinic 最大流

赛时先看

题目信号：源点 s 到汇点 t 、边带容量，问“最多能运/选/匹配多少”；或最小割类题（最大流 = 最小割）；或二分图匹配（左部连源、右部连汇、匹配边容量 1）。

本质：在残量网络上反复做“BFS 分层 + DFS 找阻塞流”：每次沿 $level+1$

的残量边一次推多条增广路，直到残量网络够不到汇点；当前弧 it 跳过本轮已推不动的边，保证每层只扫一遍。

复杂度判定：理论 $O(V^2 E)$ ，实际在比赛图（单位容量、分层少）上远快于理论界： n, m 到 $1e5$

的常规建模放心用；只有稠密大图 + 高容量分层多且常数卡紧时才换 HLPP。

维护的量：残量网络 g （每条边带剩余容量 cap ，反向边初始 0）、 $level$ （BFS 分层）、 it （当前弧指针）。

接法：先确定源点 s 和汇点 t ，`Dinic dinic(n)`，每条限制写成一条有容量的边 `add_edge(u,v,cap)`，所有边加完后 `max_flow(s,t)` 返回答案。如果题目是二分图匹配，也可以左边连源、右边连汇、匹配边容量为 1，但专门的 Hopcroft-Karp 更短。求最小割时跑完最大流后，从源点沿残量边能到的点集就是割的一侧。

警告：`add_edge` 自动建残量反边，不要自己再手动加 0 容量反边；无向容量边按题意加两条有向边；容量建议用 `i64`；`max_flow` 会修改残量网络，不能对同一对象换源汇重跑。

【最小完整示例（先抄这一段就能跑）】

题目： n 点 m 条有向边（每条容量 c ），求 s 到 t 的最大流。

```

Dinic dinic(n); // 1. 结构体定义：Dinic(点数 n)
for (int i = 0; i < m; ++i) {
    int u, v;
    i64 c;
    cin >> u >> v >> c;
    dinic.add_edge(u, v, c); // 2. 加边：正向容量 c；反向 0 容量自动建好
}
cout << dinic.max_flow(s, t) << '\n'; // 3. 调用：返回最大流

```

样例： $1 \rightarrow 2(3)$ ， $2 \rightarrow 3(2)$ ， $1 \rightarrow 3(2)$ ； $s=1$ ， $t=3 \rightarrow$ 输出 4。

【传参要求（照这个传不会错）】

- `Dinic(n)`：构造；点编号 $1..n$ （`Dinic` 用 $n+1$ 个槽，从 1 用）。
- `add_edge(u, v, cap)`：加一条容量 cap 的有向边；反向边自动建好，别再手动加。
- `max_flow(s, t)`：返回 `i64` 最大流；会修改残量网络，不能换源汇重跑同一对象。
- 无向容量边：`add_edge(u,v,c)`；`add_edge(v,u,c)`；两条。
- 最小割：跑完最大流后从 s 沿残量边可达的点集就是割的一侧；最大流值 = 最小割值。

【不会用就照抄】

```

Dinic dinic(n);
dinic.add_edge(u, v, cap); // 有向容量边 u -> v
// ... 所有边加完
cout << dinic.max_flow(s, t) << '\n';

```

- `add_edge` 会在内部创建残量反边；不要自己再手动加一条 0 容量反边，除非模板明确要求。
- 无向容量边要按题意加两条有向容量边。

【API / 入口函数（赛时只认这里列的名字）】

- `Dinic flow(n)` $\rightarrow$ 初始化 $1..n$ 网络。
- `flow.add_edge(u,v,cap)` $\rightarrow$ 加入有向容量边，同时自动建残量反边。
- `flow.max_flow(s,t)` $\rightarrow$ 所有边加完后求最大流；会修改残量网络。

【抄板清单（照着做就行）】

1. 抄哪段：整个 `struct Dinic`（含 `init/add_edge/bfs/dfs/max_flow`）。
2. 构造：`Dinic dinic(n);`， n 是点编号上限。
3. 加边：每条有向容量边 `dinic.add_edge(u, v, cap);`，内部自动补残量反边。
4. 调用：`i64 ans = dinic.max_flow(s, t);`， s/t 从题面取。
5. 取结果：`ans` 即最大流；要最小割就再从 s 沿残量边跑一遍可达性。

【改造点（按题目改这几处）】

- 0/1-indexed: 模板数组大小 $n+1$, 编号 $0..n$ 都可用, 只要最大编号 $\leq n$ 。
- 容量类型: 模板默认 `i64`; 容量小也别改成 `int`, 防相加溢出。
- 拆点建模: 点有容量 (每个点最多经过一次) 时把点 u 拆成入点 u 与出点 $u+n$, 中间连容量 = 点容量的边, 原图边从出点连向入点。
- 二分图匹配: 左部连源 s 、右部连汇 t 、匹配边容量 1, `max_flow(s,t)` 即最大匹配数。
- 多组数据: 每组重新 `Dinic dinic(n)` 或先 `init(n)` 清空。

【核心逻辑 (改代码时别破坏)】

- BFS 建只沿残量正边的分层图; DFS 只走 `level+1` 的边。
- `it` 是当前弧优化: 一条已经证明确实推不动的边, 本轮 BFS 不再重试。

【改板时先认这几个量】

- `g`: 邻接表。
- `it`: Dinic 当前弧位置。

// 维护的量: `g` = 残量网络 (每条边带 `to/rev/cap`, `cap` 是剩余容量); `level` = BFS 分层; `it` = 当前弧指针。
 // 不变量: 正向边与反向边容量互补增减 (反向边初始 0), 残量网络守恒。

```
struct Dinic {
    struct Edge {
        int to, rev;
        i64 cap;
    };

    int n;
    vector<vector<Edge>> g;
    vector<int> level, it;

    Dinic(int n = 0) { init(n); }

    void init(int n_) {
        n = n_;
        g.assign(n + 1, {});
        level.resize(n + 1);
        it.resize(n + 1);
    }

    void add_edge(int u, int v, i64 cap) {
        Edge a{v, (int)g[v].size(), cap};
        Edge b{u, (int)g[u].size(), 0}; // 反向残量边初始 0, 供撤销流量
        g[u].push_back(a);
        g[v].push_back(b);
    }

    bool bfs(int s, int t) {
        fill(level.begin(), level.end(), -1);
        queue<int> q;
        level[s] = 0;
        q.push(s);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (const auto& e : g[u]) {
                if (e.cap > 0 && level[e.to] == -1) { // 只沿残量 > 0 的边分层
                    level[e.to] = level[u] + 1;
                    q.push(e.to);
                }
            }
        }
        return level[t] != -1; // 汇点不可达即无增广路, 最大流结束
    }

    i64 dfs(int u, int t, i64 f) {
        if (u == t) return f;
        for (int& i = it[u]; i < (int)g[u].size(); ++i) { // 当前弧: 本轮推不动的边不再重试
            Edge& e = g[u][i];
            if (e.cap <= 0 || level[e.to] != level[u] + 1) continue;
            i64 ret = dfs(e.to, t, min(f, e.cap));
            if (ret) {
                e.cap -= ret;
                g[e.to][e.rev].cap += ret; // 正向减、反向加, 残量守恒
                return ret;
            }
        }
        return 0;
    }

    i64 max_flow(int s, int t) {
        i64 flow = 0;
        while (bfs(s, t)) {
            for (int u = s; u < t; ++u) it[u] = g[u].begin();
            flow += dfs(s, t, 1e18);
        }
        return flow;
    }
};
```

```

while (bfs(s, t)) {
    fill(it.begin(), it.end(), 0); // 每轮 BFS 后重置当前弧
    while (i64 f = dfs(s, t, (1LL << 62))) flow += f;
}
return flow;
};

```

HLPP: 最高标号预流推进最大流

赛时先看

题目信号: 标准最大流模型已经建好; n 、 m 较大或图较稠密; Dinic

可能卡在高容量/复杂残量网络上; 题目不要求最小费用。

本质: 求一般有向图最大流。当 Dinic 在稠密图、容量很大、分层次数很多时卡常, HLPP 往往更稳。

接法: 大规模二分图匹配以外的容量分配、最小割建模、网格网络流、项目选择的最大流子过程; 洛谷 P4722 “最大流加强版”是直接模板题。

复杂度判定: 最高标号选择 + GAP 优化的理论复杂度为 $O(V^2 \sqrt{E})$, 实际常数通常很强。空间 $O(V+E)$ 。

维护的量: `excess[u]` (点 u 的超额流)、`height[u]` (高度标号)、`bucket` (按高度分桶的待推进点)、`cur[u]` (当前弧指针)。

警告: 这是有向加边; 无向容量边通常需要分别调用 `add_edge(u,v,c)` 和 `add_edge(v,u,c)`。容量和总流量用 `i64`。本实现会修改残量网络, 不能直接对同一个对象换源汇再跑一次。

【最小完整示例 (先抄这一段就能跑)】

题目: n 点 m 条有向容量边, 求 s 到 t 的最大流 (顶点从 0 编号)。

```

HLPP flow(n); // 1. 构造: n 个点, 编号 0..n-1
for (int i = 0; i < m; ++i) {
    int u, v;
    i64 c;
    cin >> u >> v >> c;
    flow.add_edge(u, v, c); // 2. 有向容量边; 无向边再 add_edge(v, u, c)
}
cout << flow.maxflow(s, t) << '\n'; // 3. 返回 i64 最大流

```

样例: 边 $0 \rightarrow 1(3)$, $1 \rightarrow 2(2)$, $0 \rightarrow 2(2)$; $s=0$, $t=2 \rightarrow$ 输出 4 。

【传参要求 (照这个传不会错)】

- `HLPP(n)`: 构造; 顶点编号 $0..n-1$ (和 Dinic 的 1-indexed 不一样, 别混用)。
- `add_edge(u, v, cap, rev_cap = 0)`: 有向边 $u \rightarrow v$ 容量 `cap`; `rev_cap` 是反向残量初值 (默认 0 , 双向容量边可传 `rev_cap = cap` 或加两次)。
- `maxflow(s, t)`: 所有边加完后调用, 返回 `i64` 最大流; 会修改残量网络, 之后不能再 `add_edge`, 也不能换源汇重跑。

【API / 入口函数 (赛时只认这里列的名字)】

- `add_edge(int u, int v, i64 cap, i64 rev_cap = 0) ->` 加入一条边
- `maxflow(int s, int t) ->` 计算最大流 返回 `i64`。
- `add_flow(Edge& e, i64 f) ->` 沿残量边 e 推 f ; 若终点第一次获得超额流, 就放进对应高度桶。

【改板时先认这几个量】

- `g`: 邻接表。
- `excess`: 预流推进中的超额流。
- `cur`: 当前弧/当前指针。

使用: `HLPP flow(n); flow.add_edge(u, v, cap);`, 最后调用 `flow.maxflow(s, t)`。顶点从 0 编号。所有边必须在调用 `maxflow` 之前加入。

典型模型: 大规模二分图匹配以外的容量分配、最小割建模、网格网络流、项目选择的最大流子过程; 洛谷 P4722 “最大流加强版”是直接模板题。

```

struct HLPP {
    struct Edge {
        int to, rev;
        i64 flow, cap; // flow 是当前流量, cap 是剩余容量
    };

    vector<vector<Edge>> g;
    vector<i64> excess;
    vector<Edge*> cur;
};

```

```

vector<vector<int>> bucket;
vector<int> height;

explicit HLPP(int n)
: g(n), excess(n), cur(n), bucket(2 * n), height(n) {}

void add_edge(int u, int v, i64 cap, i64 rev_cap = 0) {
    if (u == v) return;
    g[u].push_back({v, (int)g[v].size(), 0, cap});
    g[v].push_back({u, (int)g[u].size() - 1, 0, rev_cap});
}

// 沿残量边 e 推 f; 若终点第一次获得超额流, 就放进对应高度桶。
void add_flow(Edge& e, i64 f) {
    Edge& back = g[e.to][e.rev];
    if (excess[e.to] == 0 && f != 0) bucket[height[e.to]].push_back(e.to);
    e.flow += f;
    e.cap -= f;
    excess[e.to] += f;
    back.flow -= f;
    back.cap += f;
    excess[back.to] -= f;
}

i64 maxflow(int s, int t) {
    const int n = (int)g.size();
    height.assign(n, 0);
    excess.assign(n, 0);
    bucket.assign(2 * n, {});

    // 所有边已加完后才保存指针; 之后不能再 add_edge。
    for (int i = 0; i < n; ++i) cur[i] = g[i].data();
    vector<int> count(2 * n, 0);
    count[0] = n - 1;
    height[s] = n;
    excess[t] = 1; // 防止汇点进入活动桶

    // 源点所有出边先饱和, 形成预流。
    for (Edge& e : g[s]) add_flow(e, e.cap);

    for (int highest = 0;;) {
        while (bucket[highest].empty()) {
            if (highest == 0) return -excess[s];
            --highest;
        }
        int u = bucket[highest].back();
        bucket[highest].pop_back();

        while (excess[u] > 0) {
            Edge* end = g[u].data() + g[u].size();
            if (cur[u] == end) {
                // 没有可推边: 重贴标签为所有残量邻居高度的最小值 + 1。
                height[u] = 2 * n - 1;
                for (Edge& e : g[u]) {
                    if (e.cap > 0 && height[u] > height[e.to] + 1) {
                        height[u] = height[e.to] + 1;
                        cur[u] = &e;
                    }
                }
                ++count[height[u]];

                // GAP: 某一层空了, 夹在它与源点之间的点不可能再到汇点。
                if (--count[highest] == 0 && highest < n) {
                    for (int v = 0; v < n; ++v) {
                        if (highest < height[v] && height[v] < n) {
                            --count[height[v]];
                            height[v] = n + 1;
                        }
                    }
                }
                highest = height[u];
            } else if (cur[u]->cap > 0 &&
                height[u] == height[cur[u]->to] + 1) {
                add_flow(*cur[u], min(excess[u], cur[u]->cap));
            } else {
                ++cur[u];
            }
        }
    }
}
};

```

最小费用最大流

赛时先看

题目信号：既有容量限制，又要费用最小或收益最大。

本质：带费用的匹配、分配、运输问题。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定：SPFA 版适合中小图。

维护的量：`g`（含容量与单位费用的残量网络）、`dist`（SPFA 最短费用距离）、`pv/pe`（最短路前驱点/边）。

警告：最大收益可以把费用取负；确认是否必须跑满指定流量。

【最小完整示例（先抄这一段就能跑）】

题目：`n` 点 `m` 条有向边（容量 `c`、单位费用 `w`），求 `s` 到 `t` 的最小费用流。

```
MinCostMaxFlow mcmf(n); // 1. 构造：顶点编号 1..n
for (int i = 0; i < m; ++i) {
    int u, v, c;
    i64 w;
    cin >> u >> v >> c >> w;
    mcmf.add_edge(u, v, c, w); // 2. 有向边：容量 c、单位费用 w
}
auto [flow, cost] = mcmf.min_cost_flow(s, t); // 3. {实际流量, 最小总费用}
cout << flow << ' ' << cost << '\n';
```

样例：边 `1->2(2,1)`，`2->3(2,2)`，`1->3(1,3)`；`s=1`，`t=3` -> 输出 `3 9`。

【传参要求（照这个传不会错）】

- `init(n_)` / 构造：`n_` 顶点数，编号 `1..n`。
- `add_edge(u, v, cap, cost)`：有向边 `u->v`：容量 `cap`（int）、单位费用 `cost`（i64，可为负）；反向残量边自动建好，别再手动加。
- `min_cost_flow(s, t, need = INT_MAX)`：`s/t` 属于 `[1, n]`；`need` 是要送满的目标流量（默认不加限制，跑到不可增广为止）。返回 `pair<int, i64>`：`first` 实际流量、`second` 最小总费用。
- 求最大收益：费用取负传入，总费用取负即为最大收益。

【API / 入口函数（赛时只认这里列的名字）】

- `add_edge(int u, int v, int cap, i64 cost)` -> 加入一条边
- `init(int n_)` -> 初始化/清空结构

【改板时先认这几个量】

- `g`：邻接表。
- `dist`：距离。

```
struct MinCostMaxFlow {
    struct Edge {
        int to, rev, cap;
        i64 cost;
    };

    int n;
    vector<vector<Edge>> g;

    MinCostMaxFlow(int n = 0) { init(n); }

    void init(int n_) {
        n = n_;
        g.assign(n + 1, {});
    }

    void add_edge(int u, int v, int cap, i64 cost) {
        Edge a{v, (int)g[v].size(), cap, cost};
        Edge b{u, (int)g[u].size(), 0, -cost};
        g[u].push_back(a);
        g[v].push_back(b);
    }

    pair<int, i64> min_cost_flow(int s, int t, int need = INT_MAX) {
        const i64 INF = (1LL << 62);
        int flow = 0;
        i64 cost = 0;
        vector<i64> dist(n + 1);
        vector<int> inq(n + 1), pv(n + 1), pe(n + 1);
```

```

while (flow < need) {
    fill(dist.begin(), dist.end(), INF);
    fill(inq.begin(), inq.end(), 0);
    queue<int> q;
    dist[s] = 0;
    q.push(s);
    inq[s] = 1;

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        inq[u] = 0;
        for (int i = 0; i < (int)g[u].size(); ++i) {
            Edge& e = g[u][i];
            if (e.cap > 0 && dist[e.to] > dist[u] + e.cost) {
                dist[e.to] = dist[u] + e.cost;
                pv[e.to] = u;
                pe[e.to] = i;
                if (!inq[e.to]) q.push(e.to), inq[e.to] = 1;
            }
        }
    }

    if (dist[t] == INF) break;
    int add = need - flow;
    for (int v = t; v != s; v = pv[v]) add = min(add, g[pv[v]][pe[v]].cap);
    for (int v = t; v != s; v = pv[v]) {
        Edge& e = g[pv[v]][pe[v]];
        e.cap -= add;
        g[v][e.rev].cap += add;
    }
    flow += add;
    cost += 1LL * add * dist[t];
}
return {flow, cost};
};

```

最小割建模提醒

赛时先看

题目信号：每个对象二选一；有收益/代价；选 A 不选 B 有惩罚。

本质：把“选或不选”的代价问题转成最小割。

复杂度判定：建图后跑最大流。

警告：最大权闭合子图常用“正权源连点，负权点连汇，依赖边 INF”。

```

// 最大权闭合子图建模：
// 点权 w[i] > 0: S -> i, capacity w[i], total += w[i]
// 点权 w[i] < 0: i -> T, capacity -w[i]
// 依赖：选 u 必须选 v，则 u -> v, capacity INF
// 答案 = 正权总和 - 最小割。

```

最大权闭合图：最小割建模与选点恢复

赛时先看

题目信号：出现“选 A 必须选

B”“做课程/项目必须先完成前置项”“得到收益但要承担依赖成本”；选择关系是有向蕴含，目标是最大化点权和。

本质：每个项目/点有正负收益；一旦选择 u 就必须同时选择它依赖的

v，求合法选择集合的最大总收益，并恢复一组选择方案。

接法：每门课有收益（可以是负数），选一门课要先选所有先修课。对每条“选 u 必选 v”调用 `require(u,v)`，对每个点调用 `set_weight`。`solve().first` 是最大净收益，`solve().second` 是一组可直接输出的选课/项目编号。

复杂度判定：建图 $O(n + d)$ ，之后一次 Dinic 最大流；d 为依赖条数。

维护的量：`weight[u]`（点权）、`dependency`（“选 u 必选 v”的依赖边）、`reach`（残量图从源可达的点，即被选中的点）。

警告：依赖 $u \rightarrow v$ 应连容量 INF 的边 $u \rightarrow v$ ，不是反向；INF 必须严格大于所有点权绝对值总和；本实现要求权值绝对值总和小于 2^{62} ；代码中的“选中点”是最终残量图从源点可达的原点。

【最小完整示例（先抄这一段就能跑）】

题目：n 门课各有收益（可负），选 u 必须先选 v，求最大净收益和选课方案。

```

MaximumWeightClosure mwc(n);           // 1. 构造: n 个点, 编号 0..n-1
for (int i = 0; i < n; ++i) {
    i64 w;
    cin >> w;
    mwc.set_weight(i, w);               // 2. 点权: 正=收益, 负=成本
}
mwc.require(u, v);                     // 3. 依赖: 选 u 必须选 v (0-indexed)
auto [best, chosen] = mwc.solve();      // 4. {最大净收益, 被选点编号}
cout << best << '\n';

```

样例: $n=3$, 权 $[5, -2, -3]$, 依赖 $1 \rightarrow 0$ 、 $2 \rightarrow 0 \rightarrow$ 输出 5 (选点 $\{0\}$)。

【传参要求（照这个传不会错）】

- `MaximumWeightClosure(n_)`: $n_$ 点数, 点编号 $0..n-1$ 。
- `set_weight(u, w)`: 点 u 的权值 w (i64, 正负均可)。
- `require(u, v)`: 依赖边“选 u 必须选 v ”, u/v 属于 $[0, n)$; 内部建成容量 `INF` 的边 $u \rightarrow v$ 。
- `solve()`: 返回 `pair<i64, vector<int>>`: `first` 最大净收益、`second` 是 0-indexed 的被选点编号列表（直接输出即可）。

【API / 入口函数（赛时只认这里列的名字）】

- `add_edge(int u, int v, i64 cap)` $\rightarrow$ 加入一条边
- `max_flow(int s, int t)` $\rightarrow$ 计算最大流 返回 `i64`。
- `solve()` $\rightarrow$ 返回 {最大总权值, 一组最优选择点集}。

【改板时先认这几个量】

- `g`: 内部 Dinic 残量网络（含反向边）。
- `it`: Dinic 当前弧。
- `dependency`: 选择 u 时必须同时选择 v 。

```

struct ClosureDinic {
    struct Edge { int to, rev; i64 cap; };

    int n;
    vector<vector<Edge>> g;
    vector<int> level, it;

    explicit ClosureDinic(int n_) : n(n_), g(n_), level(n_), it(n_) {}

    void add_edge(int u, int v, i64 cap) {
        Edge a{v, (int)g[v].size(), cap};
        Edge b{u, (int)g[u].size(), 0};
        g[u].push_back(a);
        g[v].push_back(b);
    }

    bool bfs(int s, int t) {
        fill(level.begin(), level.end(), -1);
        queue<int> q;
        level[s] = 0;
        q.push(s);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (const auto& e : g[u]) {
                if (e.cap > 0 && level[e.to] == -1) {
                    level[e.to] = level[u] + 1;
                    q.push(e.to);
                }
            }
        }
        return level[t] != -1;
    }

    i64 dfs(int u, int t, i64 f) {
        if (u == t) return f;
        for (int& i = it[u]; i < (int)g[u].size(); ++i) {
            Edge& e = g[u][i];
            if (e.cap == 0 || level[e.to] != level[u] + 1) continue;
            i64 got = dfs(e.to, t, min(f, e.cap));
            if (!got) continue;
            e.cap -= got;
            g[e.to][e.rev].cap += got;
            return got;
        }
        return 0;
    }
}

```

```

i64 max_flow(int s, int t) {
    i64 ans = 0;
    const i64 INF = (1LL << 62);
    while (bfs(s, t)) {
        fill(it.begin(), it.end(), 0);
        while (i64 f = dfs(s, t, INF)) ans += f;
    }
    return ans;
}

vector<int> reachable_from_source(int s) const {
    vector<int> vis(n);
    queue<int> q;
    vis[s] = 1;
    q.push(s);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (const auto& e : g[u]) {
            if (e.cap > 0 && !vis[e.to]) vis[e.to] = 1, q.push(e.to);
        }
    }
    return vis;
}
};

struct MaximumWeightClosure {
    int n;
    vector<i64> weight;
    vector<pair<int, int>> dependency; // 选择 u 时必须同时选择 v。

    explicit MaximumWeightClosure(int n_) : n(n_), weight(n_, 0) {}

    void set_weight(int u, i64 w) { weight[u] = w; }
    void require(int u, int v) { dependency.push_back({u, v}); }

    // 返回 {最大总权值, 一组最优选择点集}。
    pair<i64, vector<int>> solve() const {
        int s = n, t = n + 1;
        ClosureDinic dinic(n + 2);
        i64 positive_sum = 0;
        i128 abs_sum = 0;
        for (int u = 0; u < n; ++u) {
            positive_sum += max(0LL, weight[u]);
            abs_sum += llabs(weight[u]);
        }
        assert(abs_sum < (1LL << 62));
        i64 INF = (i64)abs_sum + 1;
        for (int u = 0; u < n; ++u) {
            if (weight[u] > 0) dinic.add_edge(s, u, weight[u]);
            if (weight[u] < 0) dinic.add_edge(u, t, -weight[u]);
        }
        for (auto [u, v] : dependency) dinic.add_edge(u, v, INF);

        i64 best = positive_sum - dinic.max_flow(s, t);
        vector<int> reach = dinic.reachable_from_source(s), chosen;
        for (int u = 0; u < n; ++u) if (reach[u]) chosen.push_back(u);
        return {best, chosen};
    }
};

```

典题模型：每门课有收益（可以是负数），选一门课要先选所有先修课。对每条“选 u 必选 v ”调用 `require(u,v)`，对每个点调用 `set_weight`。`solve().first` 是最大净收益，`solve().second` 是一组可直接输出的选课/项目编号。

上下界可行流 / 最大流 / 最小流

赛时先看

题目信号：题面有“每条边至少/至多流多少”；点有供需平衡；要求可行方案。

本质：每条边有流量下界 `low` 和上界 `cap`，判断是否存在可行流，或求有源汇上下界最大/最小流。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；外部只调用下面 API

列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定：一次或两次最大流。

维护的量：`demand[i]`（点 i 的需求量：出边下界减、入边下界加）、`original`（所有原边记录 $\{u,v,low, cap\}$ ）。

警告：原边容量改成 `cap-low`；对点记录需求 `d[u]-=low, d[v]+=low`；超级源汇连需求边。

【最小完整示例（先抄这一段就能跑）】

题目：n 点 m 条边，每条流量范围 [low, cap]，判断是否存在无源汇可行流。

```
LowerBoundFlowBuilder lb(n); // 1. 构造：原图点编号 1..n (SS=n+1, TT=n+2)
for (int i = 0; i < m; ++i) {
    int u, v;
    i64 low, cap;
    cin >> u >> v >> low >> cap;
    lb.add_edge(u, v, low, cap); // 2. 记录边并累计 demand
}
Dinic dinic(n + 2); // 3. 复用 Dinic
for (const auto& e : lb.original) dinic.add_edge(e.u, e.v, e.cap - e.low);
for (int i = 1; i <= n; ++i) {
    if (lb.demand[i] > 0) dinic.add_edge(lb.S, i, lb.demand[i]);
    if (lb.demand[i] < 0) dinic.add_edge(i, lb.T, -lb.demand[i]);
}
i64 f = dinic.max_flow(lb.S, lb.T); // 4. SS 出边全满 => 存在可行流
```

样例：边 1->2(2,3)、2->1(1,2) -> 可行；只有 1->2(3,4) -> 不可行。

【传参要求（照这个传不会错）】

- `init(n_)` / 构造：n_ 原图点数，编号 1..n；S = n+1、T = n+2 是自动预留的超级源/汇。
- `add_edge(u, v, low, cap)`：原图一条边：u/v 属于 [1, n]，下界 low、上界 cap (i64, $0 \leq \text{low} \leq \text{cap}$)；只更新 demand 与 original，不真正建图。
- 结果：本结构只负责建模；可行性/最大/最小流按代码内注释用 Dinic 补边后跑 `maxflow(S, T)`（无源汇）/ `maxflow(s,t)`（有源汇）得到。

【API / 入口函数（赛时只认这里列的名字）】

- `add_edge(int u, int v, i64 low, i64 cap)` -> 加入一条边
- `init(int n_)` -> 初始化/清空结构

```
// 需要已有 Dinic，点编号 1..n。这里给建模流程。
struct LowerBoundFlowBuilder {
    struct Edge { int u, v; i64 low, cap; };
    int n, S, T;
    vector<i64> demand;
    vector<Edge> original;

    LowerBoundFlowBuilder(int n = 0) { init(n); }
    void init(int n_) {
        n = n_;
        S = n + 1;
        T = n + 2;
        demand.assign(n + 3, 0);
        original.clear();
    }

    void add_edge(int u, int v, i64 low, i64 cap) {
        original.push_back({u, v, low, cap});
        demand[u] -= low;
        demand[v] += low;
        // 示例：Dinic.add_edge(u, v, cap - low);
    }

    // 无源汇可行流：
    // 1. 加所有原边 cap-low。
    // 公式/约定：2. demand[i] > 0: SS -> i, capacity demand[i]。
    // 公式/约定：3. demand[i] < 0: i -> TT, capacity -demand[i]。
    // 4. 跑 maxflow(SS, TT)，所有 SS 出边满流则可行。
    // 说明：
    // 有源汇 s,t 可行流：
    // 在上述基础上额外加 t -> s, INF，再判可行。
    // 说明：
    // 有源汇最大流：
    // 先求可行流，然后删除超级源汇，保留残量网络，跑 maxflow(s,t)。
    // 说明：
    // 有源汇最小流：
    // 先求可行流。若 t -> s 这条边流量为 base，则答案为
    // base - maxflow(t,s)（在删除超级源汇和该辅助边后）。
};
```

Gomory-Hu Tree：无向图全点对最小割

赛时先看

题目信号：无向图；很多次询问不同点对的最小割/最大流；直接对每个询问跑最大流太慢。

本质：把一个无向带容量图的所有点对最小割压缩成一棵树。任意两点的最小割值，等于树上两点路径边权的最小值。

接法：给无向通信网络，多次询问两座城市之间最多能同时通过多少容量；或统计最小割至少为某个阈值的点对数。

复杂度判定：建树需要 $n-1$ 次最大流；这里每次用 Dinic，因此总体为 $O(n * \text{MaxFlow})$ 。建好后下面的朴素查询为 $O(n)$ ，大量查询可再加 LCA 二进制提升到 $O(\log n)$ 。

维护的量：edges（无向容量边集合）、tree（Gomory-Hu 树邻接表，边权为割值）、parent/cut（建树中间量）。

警告：只适用于无向图；一条无向容量边要拆成两个相反方向、容量均为 c 的有向边。最大流结束后，必须从残量网络中找 s 可达侧来更新父亲。图不连通时最小割为 0，本模板查询返回 -1，按题意自行判断。

约定：图 0-based。

【最小完整示例（先抄这一段就能跑）】

题目： n 点无向带权图， q 次询问任意两点的最小割。

```
GomoryHu gh(n); // 1. 构造：顶点编号 0..n-1
for (int i = 0; i < m; ++i) {
    int u, v;
    i64 c;
    cin >> u >> v >> c;
    gh.add_undirected_edge(u, v, c); // 2. 无向容量边（内部自动拆两条有向边）
}
gh.build(); // 3. 建 GH 树：内部跑 n-1 次最大流
cout << gh.mincut_query(s, t) << '\n'; // 4. 查询 s, t 的最小割（0-indexed）
```

样例： $n=4$ ，边 $(0,1,3)(1,2,2)(2,3,4)(3,0,5)$ ；`mincut_query(0,2)` -> 输出 5。

【传参要求（照这个传不会错）】

- `GomoryHu(n)`： n 顶点数，编号 $0..n-1$ 。
- `add_undirected_edge(u, v, cap)`：无向容量边， u/v 属于 $[0, n)$ ；内部自动拆成 $u \rightarrow v$ 、 $v \rightarrow u$ 两条容量 cap 的边。
- `build()`：所有边加完后调用一次；建树需要 $n-1$ 次最大流。
- `mincut_query(s, t)`：返回 `i64`： s 与 t 的最小割值； s/t 属于 $[0, n)$ ；不连通返回 -1。

【API / 入口函数（赛时只认这里列的名字）】

- `add_edge(int u, int v, i64 cap)` -> 加入一条边
- `build()` -> 完成建树或预处理
- `maxflow(int s, int t)` -> 计算最大流 返回 `i64`。

【改板时先认这几个量】

- `g`：内部 Dinic 残量网络（含反向边）。
- `it`：Dinic 当前弧。

典题模型：给无向通信网络，多次询问两座城市之间最多能同时通过多少容量；或统计最小割至少为某个阈值的点对数。

```
struct GH_Dinic {
    struct Edge { int to, rev; i64 cap; };
    int n;
    vector<vector<Edge>> g;
    vector<int> level, it;

    explicit GH_Dinic(int n) : n(n), g(n), level(n), it(n) {}

    void add_edge(int u, int v, i64 cap) {
        Edge a{v, (int)g[v].size(), cap};
        Edge b{u, (int)g[u].size(), 0};
        g[u].push_back(a);
        g[v].push_back(b);
    }

    bool bfs(int s, int t) {
        fill(level.begin(), level.end(), -1);
        queue<int> q;
        level[s] = 0;
        q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (const Edge& e : g[u]) if (e.cap > 0 && level[e.to] == -1) {
                level[e.to] = level[u] + 1;
                q.push(e.to);
            }
        }
    }
};
```

```

    return level[t] != -1;
}

i64 dfs(int u, int t, i64 f) {
    if (u == t) return f;
    for (int& i = it[u]; i < (int)g[u].size(); ++i) {
        Edge& e = g[u][i];
        if (e.cap <= 0 || level[e.to] != level[u] + 1) continue;
        i64 got = dfs(e.to, t, min(f, e.cap));
        if (!got) continue;
        e.cap -= got;
        g[e.to][e.rev].cap += got;
        return got;
    }
    return 0;
}

i64 maxflow(int s, int t) {
    i64 ans = 0, pushed;
    while (bfs(s, t)) {
        fill(it.begin(), it.end(), 0);
        while ((pushed = dfs(s, t, LLONG_MAX / 4))) ans += pushed;
    }
    return ans;
}

vector<int> residual_reachable(int s) const {
    vector<int> vis(n);
    queue<int> q;
    q.push(s);
    vis[s] = 1;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (const Edge& e : g[u]) if (e.cap > 0 && !vis[e.to]) {
            vis[e.to] = 1;
            q.push(e.to);
        }
    }
    return vis;
}
};

struct GomoryHu {
    struct UEdge { int u, v; i64 cap; };
    int n;
    vector<UEdge> edges;
    vector<vector<pair<int, i64>>> tree;

    explicit GomoryHu(int n) : n(n) {}
    void add_undirected_edge(int u, int v, i64 cap) { edges.push_back({u, v, cap}); }

    void build() {
        vector<int> parent(n, 0);
        vector<i64> cut(n);
        for (int s = 1; s < n; ++s) {
            int t = parent[s];
            GH_Dinic flow(n);
            for (const auto& e : edges) {
                flow.add_edge(e.u, e.v, e.cap);
                flow.add_edge(e.v, e.u, e.cap);
            }
            i64 value = flow.maxflow(s, t);
            vector<int> side = flow.residual_reachable(s);
            for (int i = s + 1; i < n; ++i) {
                if (parent[i] == t && side[i]) parent[i] = s;
            }
            if (side[parent[t]]) {
                parent[s] = parent[t];
                parent[t] = s;
                cut[s] = cut[t];
                cut[t] = value;
            } else {
                cut[s] = value;
            }
        }
        tree.assign(n, {});
        for (int v = 1; v < n; ++v) {
            tree[v].push_back({parent[v], cut[v]});
            tree[parent[v]].push_back({v, cut[v]});
        }
    }

    i64 mincut_query(int s, int t) const {
        vector<int> vis(n);
        queue<pair<int, i64>> q;

```

```

q.push({s, LLONG_MAX / 4});
vis[s] = 1;
while (!q.empty()) {
    auto [u, best] = q.front(); q.pop();
    if (u == t) return best;
    for (auto [v, w] : tree[u]) if (!vis[v]) {
        vis[v] = 1;
        q.push({v, min(best, w)});
    }
}
return -1;
}
};

```

D 字符串与序列

09 字符串与序列匹配

字符串按单模式、多模式、哈希、回文、后缀结构和经典序列 DP 排列。出现位置、覆盖贡献这类典题紧跟 AC 自动机。

KMP 前缀函数

赛时先看

题目信号：题面问“模式串在文本中出现位置/次数”“前缀等于后缀（border）”“最小循环节”。这类“单模式匹配 + border”问题直接上 KMP。

本质：预处理模式串每个前缀的最长真前后缀长度 `pi`；匹配时文本指针只前进，失配靠 `pi` 回跳模式串指针，把暴力的回退省掉。

复杂度判定： $O(n+m)$ ； $n+m$ 到 $1e6$ 常规操作， $1e7$ 也能过；要多模式串同时匹配请翻 AC 自动机，不要对每个模式各跑一遍 KMP。

维护的量：`pi[i]` (`s[0..i]` 的最长真前后缀长度)；`j` (当前文本后缀与模式串前缀的最长匹配长度)。

接法：“在 `text` 里找 `pat` 的所有出现位置”直接调用 `kmp_find(text, pat)`，返回 0-based 起点；题目输出从 1 开始时记得 +1。“前缀等于后缀/最小循环节”对单个串跑 `prefix_function(s)`，看最后一个值 `pi[n-1]`。

警告：`pi[i]` 是 `s[0..i]` 的最长真前后缀长度（不含整串本身）；循环节长度为 $n - pi[n-1]$ ，且仅当 $n \% (n - pi[n-1]) == 0$ 时 `s` 才是完全周期串。

【最小完整示例（先抄这一段就能跑）】

题目：在文本 `text` 里找模式串 `pat` 的所有出现位置（起点 0-based）。

```

string text, pat;
cin >> text >> pat;
vector<int> pos = kmp_find(text, pat); // 1. 调用：返回所有 0-based 起点
cout << pos.size() << '\n';
for (int p : pos) cout << p << ' '; // 2. 输出：起点下标；题目从 1 开始就 +1
cout << '\n';

```

样例：`text = "abababa"`，`pat = "aba"` -> 起点 0 2 4，共 3 个。

【传参要求（照这个传不会错）】

- `kmp_find(text, pat)`：`text` 文本、`pat` 模式串；返回 `vector<int>`，元素是 0-based 起点（按出现顺序）。
- `pat` 为空：返回所有字符间隙位置 (`0..text.size()`)，一般题不会传空串。
- `prefix_function(s)`：返回 `pi[i] = s[0..i]` 的最长真前后缀长度。
- 最小循环节： $n - pi[n-1]$ ；仅当 $n \% (n - pi[n-1]) == 0$ 时才是完整周期串。
- 多模式串匹配请翻 AC 自动机，别对每个模式各跑一遍 KMP。

【不会用就照抄】

```
auto pi = prefix_function(s); // pi[i]: s[0..i] 的最长真前后缀长度
```

【API / 入口函数（赛时只认这里列的名字）】

- `prefix_function(s)` -> 返回 `pi`；`pi[i]` 是 `s[0..i]` 的最长真前后缀长度。
- `kmp_find(text, pat)` -> 返回模式 `pat` 在 `text` 中所有 0-indexed 起点。

【抄板清单（照着做就行）】

1. 抄哪段: `prefix_function` 和 `kmp_find` 两个函数, 整段抄到 `solve()` 上面。
2. 构造: 不用构造对象, 直接当普通函数调。
3. 调用: `auto pi = prefix_function(s);` 或 `auto pos = kmp_find(text, pat);`
4. 取结果: `pi[n-1]` 是 `s` 的最长真前后缀长度; `pos` 是 0-based 起点列表, 原样输出或逐个 +1。

【改造点（按题目改这几处）】

- 求最小循环节: 循环节长度 = `n - pi[n-1]`; 若 `n % (n - pi[n-1]) == 0` 则 `s` 由循环节重复构成 (如 `ababab`), 否则不存在小于 `n` 的循环节 (如 `ababa`)。
- 只要出现次数不要位置: 输出 `pos.size()`, 或删掉 `pos` 只维护计数器。
- 输出从 1 开始计数: 输出 `pos[i] + 1`。
- 问每个前缀的出现次数/每个 border 长度: 对 `pi` 数组做桶计数 + 后缀累加。

【核心逻辑（改代码时别破坏）】

- `pi[i]` 是 `s[0..i]` 的最长真前后缀长度。
- 失配时把当前匹配长度 `j` 跳到 `pi[j-1]`, 不用回退文本指针。

需要模式匹配时, 优先用本节给出的完整匹配函数; 不要把不同教材里的 `next[]` 定义和这份 `pi[]` 混用。

```
// 维护的量: pi[i] = s[0..i] 的最长真前后缀长度 (不含整串本身)。只读参数 s, 不改。
vector<int> prefix_function(const string& s) {
    int n = (int)s.size();
    vector<int> pi(n);
    for (int i = 1; i < n; ++i) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) j = pi[j - 1]; // 失配沿 border 链回跳
        if (s[i] == s[j]) j++; // 匹配上则长度 +1
        pi[i] = j;
    }
    return pi;
}

// 维护的量: j = 当前已匹配的模式串长度; pos = 所有 0-based 匹配起点。
// 不变量: 失配时 j 回退到 pi[j-1], 文本指针不回退, 保证 O(n+m)。
vector<int> kmp_find(const string& text, const string& pat) {
    if (pat.empty()) {
        vector<int> boundaries(text.size() + 1);
        iota(boundaries.begin(), boundaries.end(), 0);
        return boundaries; // 空串在每个字符间隙 (含两端) 匹配一次。
    }
    vector<int> pi = prefix_function(pat), pos;
    int j = 0;
    for (int i = 0; i < (int)text.size(); ++i) {
        while (j > 0 && text[i] != pat[j]) j = pi[j - 1];
        if (text[i] == pat[j]) j++;
        if (j == (int)pat.size()) {
            pos.push_back(i - (int)pat.size() + 1); // 完整匹配, 记录起点
            j = pi[j - 1]; // 回退以允许重叠匹配
        }
    }
    return pos;
}
```

KMP 自动机: 单个禁串计数 DP

赛时先看

题目信号: 题面要求“不能出现某个连续模式”“每加入一个字符要知道当前匹配了模式串多长前缀”; 模式串只有一个, 且字母表较小。

本质: 构造或计数长度为 `len` 的字符串, 要求不包含某个模式串; 也可把自动机状态接到数位 DP、状压 DP 或矩阵快速幂中。

接法: 看到“长度为 `n` 的字符串不能出现某个禁串”就用这个模板。 `state` 表示当前后缀已经匹配了禁串前多少个字符, 转移到 `m` 就说明禁串出现了, DP 时跳过。字符集不是小写前若干个字母时, 先把字符压缩或修改 `SIG` 和字符映射。

复杂度判定: 自动机 $O(\text{字符集} * |\text{pattern}|)$, 长度为 `len` 的普通 DP 为 $O(\text{len} * \text{字符集} * |\text{pattern}|)$ 。

维护的量: `state` (当前后缀已匹配禁串前缀的长度, 范围 `[0, m-1]`); `nxt[state][c]` (加字符 `c` 后的转移); `pi` (KMP 前缀函数)。

警告: 完整匹配后的下一转移必须从 `pi[m-1]`

回退, 才能识别重叠匹配; 空模式串会匹配所有字符串, 本模板直接禁止; 大长度时把转移矩阵快速幂。

【最小完整示例（先抄这一段就能跑）】

题目：长度 `len` 的字符串只用 `'a'..'a'+sigma-1` 组成，求不含模式串 `pattern` 的个数，答案模 `998244353`。

```
string pattern;
int len, sigma;
cin >> pattern >> len >> sigma;
i64 ans = count_strings_avoiding_one_pattern(pattern, len, sigma);
cout << ans << '\n'; // 调用：返回模 998244353 的合法串个数
```

样例：`pattern = "ab"`, `len = 2`, `sigma = 2` $\rightarrow$ 3 (`aa`, `ba`, `bb`)。

【传参要求（照这个传不会错）】

- `count_strings_avoiding_one_pattern(pattern, len, alphabet_size, modulus = modn)`: `pattern` 禁串（非空、全小写，长度 `m`）；`len` 目标长度（ ≥ 0 ）；`alphabet_size` 字符集大小（`1..26`，取前 `alphabet_size` 个小写字母）；`modulus` 取模值（默认 `modn`）；返回 `i64`，模意义下的合法串数量。
- 也可手动建 `KMPAutomaton automaton(pattern)`，`automaton.nxt[state][c]` 是加字符 `c` 后的新状态（等于 `m` 表示命中禁串）；要接矩阵快速幂/数位 DP 时自己迭代这个转移。

【API / 入口函数（赛时只认这里列的名字）】

- `build(const string& s)` $\rightarrow$ 完成建树或预处理

【改板时先认这几个量】

- `pi`: KMP 前缀函数。
- `nxt`: 转移/子节点。

状态 `state` 是当前字符串后缀与模式串前缀的最长匹配长度；转移后到 `m` 代表刚刚完整匹配模式串。计数禁串时只保留状态 `[0,m-1]`。

```
struct KMPAutomaton {
    static constexpr int SIG = 26;
    string pattern;
    vector<int> pi;
    vector<array<int, SIG>> nxt;

    KMPAutomaton() = default;
    explicit KMPAutomaton(const string& s) { build(s); }

    void build(const string& s) {
        assert(!s.empty());
        pattern = s;
        int m = (int)pattern.size();
        pi.assign(m, 0);
        for (int i = 1; i < m; ++i) {
            int j = pi[i - 1];
            while (j && pattern[i] != pattern[j]) j = pi[j - 1];
            if (pattern[i] == pattern[j]) ++j;
            pi[i] = j;
        }

        nxt.assign(m + 1, {});
        for (int state = 0; state <= m; ++state) {
            for (int c = 0; c < SIG; ++c) {
                char ch = char('a' + c);
                int j = (state == m ? pi[m - 1] : state);
                while (j && pattern[j] != ch) j = pi[j - 1];
                if (pattern[j] == ch) ++j;
                nxt[state][c] = j;
            }
        }
    }
};

i64 count_strings_avoiding_one_pattern(
    const string& pattern, int len, int alphabet_size, i64 modulus = modn
) {
    assert(0 <= len && 1 <= alphabet_size && alphabet_size <= 26 && modulus > 0);
    KMPAutomaton automaton(pattern);
    int m = (int)pattern.size();
    vector<i64> dp(m);
    dp[0] = 1 % modulus;
    for (int pos = 0; pos < len; ++pos) {
        vector<i64> next_dp(m);
        for (int state = 0; state < m; ++state) {
            for (int c = 0; c < alphabet_size; ++c) {
                int to = automaton.nxt[state][c];
                if (to == m) continue; // 形成禁串，丢弃。
            }
        }
    }
}
```

```

        next_dp[to] += dp[state];
        if (next_dp[to] >= modulus) next_dp[to] -= modulus;
    }
    dp.swap(next_dp);
}
i64 answer = 0;
for (i64 ways : dp) {
    answer += ways;
    if (answer >= modulus) answer -= modulus;
}
return answer;
}

```

典型模型：小写前 sigma 个字符组成长度 n 的字符串，要求不出现 pattern ，答案模 998244353。调用 `count_strings_avoiding_one_pattern(pattern, n, sigma)`；若要求恰好出现次数，需额外加“已经匹配几次”的维度。

KMP 前缀数组计数字符串

赛时先看

题目信号：题面给 `next[1..n]` / LPS / failure function，反问字符串数量或合法性。

本质：给定 KMP `pi/next` 数组和字符集大小，计算有多少个字符串恰好产生该数组，并可同时构造一个代表串。

复杂度判定：通常 $O(n + \text{失败链总访问})$ ，常见数据可过；只要构造合法性时非常好用。

维护的量：`id[i]`（构造出的代表串第 i 个字符的编号）；`used`（已使用的字符种类数）；`ans`（合法字符串计数，模 `KMP_COUNT_MOD`）。

警告：`pi[i]` 最多比 `pi[i-1]` 多 1；当 `pi[i]=0` 时，当前字符必须避开失败链上会导致匹配成功的字符。

【最小完整示例（先抄这一段就能跑）】

题目：给定 KMP 前缀数组 `pi` 和字符集大小，求有多少个字符串恰好产生该数组（模 998244353）。

```

vector<int> pi = {0, 0}; // 题目给的 pi (0-based, pi[0] 必须为 0)
i64 cnt = count_strings_with_prefix_function(pi, 2);
cout << cnt << '\n'; // 调用：返回模 998244353 的计数

```

样例：`pi = {0, 0}`，`alphabet_size = 2` $\rightarrow$ 2 (`ab`、`ba`)。

【传参要求（照这个传不会错）】

- `pi`：给定前缀数组，0-based，长度 n ；要求 `pi[0] == 0`、 $0 \leq pi[i] \leq i$ 、`pi[i] <= pi[i-1] + 1`，不满足返回 0。
- `alphabet_size`：字符集大小（正整数），字符用编号 `0..alphabet_size-1` 表示。
- `canonical`：可选输出参数，传 `vector<int>*` 时把构造出的一个代表串写进去（`id[i]` 为第 i 个字符编号）；不需要传 `nullptr`。
- 返回值：`i64`，模 998244353 的合法字符串数量；`pi` 非法时返回 0。

```

const i64 KMP_COUNT_MOD = 998244353;

i64 count_strings_with_prefix_function(
    const vector<int>& pi,
    i64 alphabet_size,
    vector<int>* canonical = nullptr
) {
    int n = (int)pi.size();
    if (n == 0) return 1;
    if (pi[0] != 0 || alphabet_size <= 0) return 0;

    vector<int> id(n, 0);
    int used = 1;
    i64 ans = alphabet_size % KMP_COUNT_MOD;

    auto next_state = [&](int prev, int c) {
        int j = prev;
        while (j > 0 && c != id[j]) j = pi[j - 1];
        if (c == id[j]) j++;
        return j;
    };

    for (int i = 1; i < n; i++) {
        if (pi[i] < 0 || pi[i] > i || pi[i] > pi[i - 1] + 1) return 0;
        if (pi[i] > 0) {
            int c = id[pi[i] - 1];
            if (next_state(pi[i - 1], c) != pi[i]) return 0;
            id[i] = c;
        } else {
            vector<char> banned(max(used, 1), 0);

```



```

int banned_count = 0;
auto ban = [&](int c) {
    if (c >= (int)banned.size()) banned.resize(c + 1, 0);
    if (!banned[c]) {
        banned[c] = 1;
        banned_count++;
    }
};
for (int j = pi[i - 1]; j > 0; j = pi[j - 1]) ban(id[j]);
ban(id[0]);

i64 choices = alphabet_size - banned_count;
if (choices <= 0) return 0;
ans = ans * (choices % KMP_COUNT_MOD) % KMP_COUNT_MOD;

int c = -1;
for (int x = 0; x < used; x++) {
    if (x >= (int)banned.size() || !banned[x]) {
        c = x;
        break;
    }
}
if (c == -1) c = used++;
id[i] = c;
if (next_state(pi[i - 1], c) != 0) return 0;
}
}

if (canonical) *canonical = id;
return ans;
}

```

Z 函数

赛时先看

题目信号：所有后缀与前缀比较；统计每个前缀出现次数。

本质：每个位置和全串前缀的 LCP。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O(n)$ 。

维护的量：`z[i]` (`s` 与后缀 `s[i..]` 的 LCP)；匹配盒 `[l, r]` (当前最右已探索区间)。

警告：`z[0]` 通常设为 0，按题意可改成 `n`。

【最小完整示例（先抄这一段就能跑）】

题目：求字符串 `s` 每个后缀与全串的最长公共前缀长度。

```

string s;
cin >> s;
vector<int> z = z_function(s); // 调用: z[i] = LCP(s, s[i..]), 0-based
cout << z[1] << '\n';

```

样例：`s = "aaaaa" → z[1] = 4`；`s = "ababa" → z[1] = 0`。

【传参要求（照这个传不会错）】

- `z_function(s)`: `s` 原串 (0-based)；返回 `vector<int> z`，长度 `|s|`。
- `z[i] = s[0..]` 与 `s[i..]` 的最长公共前缀长度 (`i` 为 0-based 起点)。
- `z[0]` 约定为 0；有的题按定义应为 `n`，用时按题意自行处理。

```

vector<int> z_function(const string& s) {
    int n = (int)s.size();
    vector<int> z(n);
    for (int i = 1, l = 0, r = 0; i < n; ++i) {
        if (i <= r) z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
    }
    return z;
}

```

序列自动机：多模式子序列查询与最短缺失子序列

赛时先看

题目信号：题面说“可删除若干字符但不能改变相对顺序”“很多模式串是否能嵌入一个固定文本”“最短不出现的子序列”；注意子序列不要求连续，和 KMP/SAM 的子串问题不同。

本质：对一个固定小写文本串，快速判断很多模式串是否为它的子序列，并输出一组最早匹配位置；也可在给定字符集内构造一个最短、且不是文本子序列的字符串。

接法：文本 S 固定， q 个模式串询问是否可由 S 删除字符得到。先 `SubsequenceAutomaton sam(S)`，每次调用 `sam.contains_subsequence(T)`；若需要输出选中的下标，使用 `sam.earliest_embedding(T)`。若问由 `abc` 组成、不是 S 子序列的最短串，调用 `shortest_missing_subsequence(sam, "abc")`。

复杂度判定：预处理 $O(26n)$ ，每次判断/还原 $O(|pattern|)$ ；最短缺失子序列构造 $O(26n)$ 。字符集不是小写字母时，把 26 换成压缩后的字符数。

维护的量： n （文本长度）；`nxt[i][c]`（从位置 i 起第一个字符 c 的下标，找不到为 n ）。

警告：`nxt[i][c]` 表示从位置 i 起第一个 c 的位置，找不到时为 n ；匹配到位置 p 后下一个状态应是 $p+1$ ；最短缺失串是“不是子序列”，不是“不是连续子串”。

【最小完整示例（先抄这一段就能跑）】

题目：固定文本 S ，询问模式串 T 是否为 S 的子序列（可跳字符）。

```
string S, T;
cin >> S >> T;
SubsequenceAutomaton sam(S);           // 1. 构造：对固定文本 S 预处理
if (sam.contains_subsequence(T))       // 2. 调用：T 是否为 S 的子序列
    cout << "yes\n";
else
    cout << "no\n";
```

样例： $S = \text{"abcde"}$ ， $T = \text{"ace"}$ $\rightarrow$ yes； $T = \text{"adc"}$ $\rightarrow$ no。

【传参要求（照这个传不会错）】

- `SubsequenceAutomaton sam(S)`： S 固定文本（0-based，小写）；`sam.nxt[i][c]` 是从位置 i 起第一个 c 的下标，找不到为 n 。
- `sam.contains_subsequence(T)`：返回 `bool`， T 是否为 S 的子序列。
- `sam.earliest_embedding(T)`：返回 `optional<vector<int>>`， T 的最早匹配下标（0-based，升序）；不存在时返回 `nullopt`。
- `shortest_missing_subsequence(sam, alphabet)`：`alphabet` 是允许使用的小写字符集（可乱序，内部排序去重）；返回字典序最小的最短非空“不是子序列”串（`optional<string>`）；字符集为空时返回 `nullopt`。

【API / 入口函数（赛时只认这里列的名字）】

- `build(const string& text)` $\rightarrow$ 完成建树或预处理

【改板时先认这几个量】

- `nxt`：转移/子节点。
- `dp`：DP 状态。

```
struct SubsequenceAutomaton {
    static constexpr int SIG = 26;
    int n = 0;
    vector<array<int, SIG>> nxt;

    SubsequenceAutomaton() = default;
    explicit SubsequenceAutomaton(const string& text) { build(text); }

    void build(const string& text) {
        n = (int)text.size();
        nxt.assign(n + 1, {});
        nxt[n].fill(n);
        for (int i = n - 1; i >= 0; --i) {
            nxt[i] = nxt[i + 1];
            int c = text[i] - 'a';
            assert(0 <= c && c < SIG);
            nxt[i][c] = i;
        }
    }

    optional<vector<int>> earliest_embedding(const string& pattern) const {
        vector<int> position;
        position.reserve(pattern.size());
        int state = 0;
        for (char ch : pattern) {
            int c = ch - 'a';
            if (c < 0 || c >= SIG || nxt[state][c] == n) return nullopt;
            position.push_back(nxt[state][c]);
            state = nxt[state][c] + 1;
        }
        return position;
    }
};
```

```

        int at = nxt[state][c];
        position.push_back(at);
        state = at + 1;
    }
    return position;
}

bool contains_subsequence(const string& pattern) const {
    return earliest_embedding(pattern).has_value();
}
};

// 返回字典序最小的最短非空字符串，字符来自给定字符集；
// 该字符串不是自动机原串的子序列。
optional<string> shortest_missing_subsequence(
    const SubsequenceAutomaton& automaton, string alphabet
) {
    sort(alphabet.begin(), alphabet.end());
    alphabet.erase(unique(alphabet.begin(), alphabet.end()), alphabet.end());
    if (alphabet.empty()) return nullopt;
    for (char ch : alphabet) assert('a' <= ch && ch <= 'z');

    const int INF_LEN = 1e9;
    int n = automaton.n;
    vector<int> dp(n + 1, INF_LEN);
    vector<char> choice(n + 1, '?');
    for (int state = n; state >= 0; --state) {
        for (char ch : alphabet) {
            int next_pos = automaton.nxt[state][ch - 'a'];
            int candidate = next_pos == n ? 1 : 1 + dp[next_pos + 1];
            if (candidate < dp[state]
                || (candidate == dp[state] && ch < choice[state])) {
                dp[state] = candidate;
                choice[state] = ch;
            }
        }
    }

    string answer;
    int state = 0;
    while (true) {
        char ch = choice[state];
        answer.push_back(ch);
        int next_pos = automaton.nxt[state][ch - 'a'];
        if (next_pos == n) return answer;
        state = next_pos + 1;
    }
}

```

典题模型：文本 S 固定， q 个模式串询问是否可由 S 删除字符得到。先 `SubsequenceAutomaton sam(S)`，每次调用 `sam.contains_subsequence(T)`；若需要输出选中的下标，使用 `sam.earliest_embedding(T)`。若问由 `abc` 组成、不是 S 子序列的最短串，调用 `shortest_missing_subsequence(sam, "abc")`。

Trie 字典树

赛时先看

题目信号：大量字符串插入，问某前缀是否存在/出现几次。

本质：前缀查询、字符串集合、01 Trie 基础。

接法：先把所有字符串 `insert` 进 `trie`；问某个前缀出现过多少次就 `count_prefix(prefix)`。如果题目问完整单词出现次数，读到末尾节点后返回 `end`；如果题目是二进制最大异或，不用这个字符版，翻 01 Trie。

复杂度判定：总字符数。

维护的量：`tr`（节点池，`tr[0]` 是根）；每个节点

`pass`（经过该节点的字符串数）、`end`（恰在该节点结束的字符串数）。

警告：字符集不是小写字母时要改数组大小或用 `map`。

【最小完整示例（先抄这一段就能跑）】

题目：插入若干字符串，询问某个前缀出现过多少次、某个完整单词出现过几次。

```

Trie trie;
trie.insert("apple");           // 1. 插入字符串（可重复）
trie.insert("apply");
int pref = trie.count_prefix("app"); // 2. 以 "app" 为前缀的字符串数
int exact = trie.count_word("apple"); // 3. 完整单词 "apple" 出现次数
cout << pref << ' ' << exact << '\n';

```

样例：插入 `apple`、`apply` 后 $\rightarrow$ `count_prefix("app") = 2`, `count_word("apple") = 1`。

【传参要求（照这个传不会错）】

- `insert(s)`: 插入一个小写字符串；可重复插入，重复时 `end/pass` 各 +1。
- `count_prefix(p)`: 返回以 `p` 为前缀的已插入字符串个数（`p` 本身也计入）。
- `count_word(s)`: 返回完整字符串 `s` 被插入的次数（读到末尾节点取 `end`）。
- `contains(s)`: 返回 `bool`, `s` 是否至少被完整插入过一次。
- 字符映射固定 `'a'..'z'` (`ch - 'a'`)，节点 `0` 是根；字符集不同时改 `nxt[26]` 或换 `map`。

【不会用就照抄】

```
Trie trie;
trie.insert(s);
int pref = trie.count_prefix(prefix); // 有多少已插入字符串以 prefix 开头
int exact = trie.count_word(s);      // 完整字符串 s 出现次数
if (trie.contains(s)) { }           // 是否至少插入过一次 s
```

- 先确认字符集映射（通常 `'a'..'z'`）。
- `insert` 之后再查；节点 `0` 通常是根。

【API / 入口函数（赛时只认这里列的名字）】

- `trie.insert(s)` $\rightarrow$ 插入一个小写字符串，可重复插入。
- `trie.count_prefix(p)` $\rightarrow$ 有多少已插入字符串以 `p` 为前缀。
- `trie.count_word(s)` $\rightarrow$ 完整字符串 `s` 被插入了多少次。
- `trie.contains(s)` $\rightarrow$ 是否至少插入过一次完整字符串 `s`。

【核心逻辑（改代码时别破坏）】

- 一条根到节点的路径就是一个前缀；`pass` 统计经过该节点的字符串数，`end` 统计恰好在此结束的字符串数。

【改板时先认这几个量】

- `tr`: 树节点池/自动机节点池。
- `nxt`: 转移/子节点。

```
struct Trie {
    struct Node {
        int nxt[26]{};
        int pass = 0, end = 0;
    };
    vector<Node> tr{Node{}};

    void insert(const string& s) {
        int u = 0;
        tr[u].pass++;
        for (char ch : s) {
            int c = ch - 'a';
            if (!tr[u].nxt[c]) {
                tr[u].nxt[c] = (int)tr.size();
                tr.push_back(Node{});
            }
            u = tr[u].nxt[c];
            tr[u].pass++;
        }
        tr[u].end++;
    }

    int count_prefix(const string& s) const {
        int u = 0;
        for (char ch : s) {
            int c = ch - 'a';
            if (!tr[u].nxt[c]) return 0;
            u = tr[u].nxt[c];
        }
        return tr[u].pass;
    }

    int count_word(const string& s) const {
        int u = 0;
        for (char ch : s) {
            int c = ch - 'a';
            if (!tr[u].nxt[c]) return 0;
            u = tr[u].nxt[c];
        }
        return tr[u].end;
    }
};
```

```

    }

    bool contains(const string& s) const {
        return count_word(s) > 0;
    }
};

```

AC 自动机

赛时先看

题目信号：给很多关键词，问它们在长文本里出现次数或是否出现。

本质：多模式串匹配。

接法：多个模式串一次性插入，编号从 0 到 `pattern_count-1`；`build()` 后扫文本，`match_count(text, pattern_count)[id]` 就是第 `id` 个模式出现次数。若模式很多且互为后缀，这个简单版会把输出列表复制到节点上，极端情况下可能大；只要最终次数时优先翻下一段 fail 树累计版。

复杂度判定：建树 $O(\text{总模式长度} * \text{字符集})$ ，匹配 $O(\text{文本长度} + \text{命中数})$ 。

维护的量：`tr` (Trie 节点池，`tr[0]` 是根)；`fail` (失配指针)；每个节点 `out` (在该节点结束的模式编号，`build` 后并入 fail 链上的编号)。

警告：`build` 后缺失转移被补到 fail 转移。

【最小完整示例（先抄这一段就能跑）】

题目：给 `m` 个关键词，统计每个关键词在长文本 `text` 中的出现次数（允许重叠）。

```

ACAutomaton ac;
for (int i = 0; i < m; ++i) {
    string pat;
    cin >> pat;
    ac.insert(pat, i); // 1. 插入模式串，绑定 0-based 编号 i
}
ac.build(); // 2. 所有模式插完后 build 一次
vector<int> cnt = ac.match_count(text, m); // 3. 调用：统计每个模式出现次数
for (int i = 0; i < m; ++i) cout << cnt[i] << '\n'; // 4. 按编号输出

```

样例：模式 `he, she`，文本 `"shehe"` -> `cnt = {2, 1}`。

【传参要求（照这个传不会错）】

- `insert(pattern, id)`: `pattern` 小写模式串；`id` 0-based 编号（范围 `0..m-1`），查询结果按它对齐。
- `build()`: 所有模式插完后调用一次；没 `build` 就查询是错的。
- `match_count(text, m)`: `text` 长文本；`m` 模式总数；返回 `vector<int>`，`cnt[id] =` 第 `id` 个模式在 `text` 中的重叠出现次数（含互为后缀时 fail 链上的命中）。
- 模式很多且互为后缀时，本版把输出列表复制到节点上可能很慢；只要最终次数请翻下一节 fail 树累计版。

【不会用就照抄】

```

ACAutomaton ac;
for (auto &pat : patterns) ac.insert(pat, id);
ac.build(); // 所有模式串插完后只 build 一次
// 然后再拿文本串 query / walk

```

- 调用顺序固定：`insert -> build -> query`。
- 没 `build` 就查询是错的；`build` 会补 fail 和自动机转移。

【API / 入口函数（赛时只认这里列的名字）】

- `ac.insert(pattern, id)` -> 插入模式串并绑定编号 `id`。
- `ac.build()` -> 所有模式插完后构建 fail；查询前必须执行一次。
- `ac.match_count(text, m)` -> 统计编号 `0..m-1` 各模式在文本中出现次数。

【核心逻辑（改代码时别破坏）】

- Trie 负责模式前缀；fail 把失配状态跳到最长可用后缀。
- `build()` 后把缺失转移补成 fail 转移，因此匹配文本时每个字符只走一次状态转移。

【改板时先认这几个量】

- `tr`: 树节点池/自动机节点池。
- `fail`: AC 失配指针。
- `nxt`: 转移/子节点。

```

struct ACAutomaton {
    static const int SIG = 26;
    struct Node {
        int nxt[SIG]{};
        int fail = 0;
        vector<int> out;
    };
    vector<Node> tr{Node{}};

    void insert(const string& s, int id) {
        int u = 0;
        for (char ch : s) {
            int c = ch - 'a';
            if (!tr[u].nxt[c]) {
                tr[u].nxt[c] = (int)tr.size();
                tr.push_back(Node{});
            }
            u = tr[u].nxt[c];
        }
        tr[u].out.push_back(id);
    }

    void build() {
        queue<int> q;
        for (int c = 0; c < SIG; ++c) if (tr[0].nxt[c]) q.push(tr[0].nxt[c]);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int c = 0; c < SIG; ++c) {
                int v = tr[u].nxt[c];
                if (v) {
                    tr[v].fail = tr[tr[u].fail].nxt[c];
                    auto& a = tr[v].out;
                    auto& b = tr[tr[v].fail].out;
                    a.insert(a.end(), b.begin(), b.end());
                    q.push(v);
                } else {
                    tr[u].nxt[c] = tr[tr[u].fail].nxt[c];
                }
            }
        }
    }

    vector<int> match_count(const string& text, int pattern_count) const {
        vector<int> cnt(pattern_count);
        int u = 0;
        for (char ch : text) {
            int c = ch - 'a';
            u = tr[u].nxt[c];
            for (int id : tr[u].out) cnt[id]++;
        }
        return cnt;
    }
};

```

AC 自动机：fail 树累计所有模式出现次数

赛时先看

题目信号：模式串可能互为前后缀（如

a、aa、aaa），模式总长度和文本都很大，题目要求“每个关键词出现了多少次”。

本质：给很多模式串和一段文本，分别统计每个模式串在文本中的重叠出现次数；适合只要最终次数、不需要逐个枚举匹配位置的题。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定：建树 $O(\text{总模式长度} * \text{字符集})$ ，匹配和反向累计 $O(\text{文本长度} + \text{节点数} + \text{模式数})$ 。

维护的量：`visits[u]`（自动机状态 `u` 在文本中的访问次数，扫描时只加 1、不枚举 fail 链）；`terminal[id]`（第 `id` 个模式串的终止节点）；`bfs_order`（BFS 顺序，逆序即 fail 树的拓扑序，用它把次数推给 fail 父亲）。

警告：必须先 `insert` 完所有模式串再 `build`；`terminal`

按插入顺序保存，重复模式串会自然得到相同答案；匹配文本可以重叠，不能匹配后把状态清零。

【最小完整示例（先抄这一段就能跑）】

题目：给 `m` 个模式串，统计每个模式串在长文本 `text` 中出现了多少次（允许重叠）。

```

ACAutomatonFailTree ac;
for (int i = 0; i < m; ++i) {
    string pat;
    cin >> pat;
}

```

```

    ac.insert(pat);                // 1. 插入所有模式串（按题目输入顺序）
}
ac.build();                      // 2. 所有模式插完后 build 一次
vector<i64> cnt = ac.match_count(text); // 3. 调用：统计每个模式的出现次数
for (int i = 0; i < m; ++i) cout << cnt[i] << '\n'; // 4. 按插入顺序输出

```

样例：模式 a, aa, aaa, 文本 "aaaa" -> 出现次数 4, 3, 2。

【传参要求（照这个传不会错）】

- `insert(pattern)`: 插入一个模式串；返回它的编号（按插入顺序从 0 开始）。
- `build()`: 所有模式插完后调用一次；没 `build` 就 `match` 是错的。
- `match_count(text)`: 返回 `vector<i64>`, `cnt[id]` = 第 `id` 个模式在文本中的重叠出现次数。
- 只统计次数、不返回位置；要“出现位置”翻下一节 `PatternOccurrenceAC`。

【API / 入口函数（赛时只认这里列的名字）】

- `build()` -> 完成建树或预处理
- `insert(const string& s)` -> 插入元素/字符串 返回 `int`。
- `match_count(const string& text)` -> 统计每个模式串在文本中的出现次数 返回 `vector<i64>`。

【改板时先认这几个量】

- `terminal`: 第 `id` 个模式串的终止节点，允许重复模式串。
- `tr`: 树节点池/自动机节点池。
- `fail`: AC 失配指针。
- `nxt`: 转移/子节点。

核心：扫描文本时只给当前自动机状态加 1；BFS 建立的 `fail` 树按逆序把访问次数推给 `fail` 父亲。一个模式串结尾节点收到的总次数，就是该模式串的答案。不会复制整条 `fail` 链的输出列表。

```

struct ACAutomatonFailTree {
    static constexpr int SIG = 26;
    struct Node {
        array<int, SIG> nxt{};
        int fail = 0;
    };

    vector<Node> tr{Node{}};
    vector<int> terminal; // 第 id 个模式串的终止节点，允许重复模式串
    vector<int> bfs_order;

    int insert(const string& s) {
        assert(!s.empty());
        int u = 0;
        for (char ch : s) {
            int c = ch - 'a';
            assert(0 <= c && c < SIG);
            if (!tr[u].nxt[c]) {
                tr[u].nxt[c] = (int)tr.size();
                tr.push_back(Node{});
            }
            u = tr[u].nxt[c];
        }
        terminal.push_back(u);
        return (int)terminal.size() - 1;
    }

    void build() {
        queue<int> q;
        bfs_order = {0};
        for (int c = 0; c < SIG; ++c) {
            int v = tr[0].nxt[c];
            if (v) {
                q.push(v);
                bfs_order.push_back(v);
            }
        }
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int c = 0; c < SIG; ++c) {
                int v = tr[u].nxt[c];
                if (v) {
                    tr[v].fail = tr[tr[u].fail].nxt[c];
                    q.push(v);
                    bfs_order.push_back(v);
                } else {

```

```

        tr[u].nxt[c] = tr[tr[u].fail].nxt[c];
    }
}
}
}

vector<i64> match_count(const string& text) const {
    vector<i64> visits(tr.size());
    int u = 0;
    for (char ch : text) {
        int c = ch - 'a';
        assert(0 <= c && c < SIG);
        u = tr[u].nxt[c];
        visits[u]++;
    }
    for (int i = (int)bfs_order.size() - 1; i > 0; --i) {
        int node = bfs_order[i];
        visits[tr[node].fail] += visits[node];
    }
    vector<i64> answer(tr.size());
    for (int id = 0; id < (int)terminal.size(); ++id) {
        answer[id] = visits[terminal[id]];
    }
    return answer;
}
};

// 典型调用: 每个 pattern 的答案按插入顺序返回。
// 示例: ACAutomatonFailTree ac;
// 示例: for (const string& pattern : patterns) ac.insert(pattern);
// 示例: ac.build();
// 示例: vector<i64> count = ac.match_count(text);

```

AC 自动机：批量收集所有模式串出现位置

赛时先看

题目信号：固定文本

S，很多模式串询问；不仅要出现次数，还要每个模式的出现位置；模式串总长较大，不能每个串单独 KMP。

本质：对所有模式串建 AC 自动机，扫描文本时沿 `up` 指针访问当前状态 `fail` 链上的模式串终点，收集每个模式串的所有出现位置。

接法：所有询问先 `insert`，再 `build`，然后

`collect_occurrences(text)`。如果后续只与每个串出现位置有关，同一模式串重复询问时直接复用结果。

复杂度判定：建树 $O(\text{总模式长度} * \text{字符集})$ ；扫描复杂度为 $O(|S| + \text{输出出现次数})$ 。若每个位置会命中很多模式，要确认总输出量是否可承受。

维护的量：`trie` (Trie 节点池)；`up[u]` (fail 链上最近的有模式结束的节点，-1 表示没有)；`terminal` (各模式终点节点)、`length` (各模式长度)。

警告：重复模式串要么去重，要么让同一终点保存多个 `id`；`up[u]` 指向 fail 链上最近的“有模式结束”的节点，不是普通 fail；返回位置为 0-based 起点。

【最小完整示例（先抄这一段就能跑）】

题目：给 `m` 个模式串，收集每个模式串在文本 `text` 中的所有出现起点。

```

PatternOccurrenceAC ac;
for (int i = 0; i < m; ++i) {
    string pat;
    cin >> pat;
    ac.insert(pat);
}
ac.build();
auto occ = ac.collect_occurrences(text);
for (int p : occ[0]) cout << p << ' ';

```

// 1. 插入模式串（返回 0-based 编号）
// 2. 所有模式插完后 build 一次
// 3. 调用：收集所有出现位置
// 4. 第 0 个模式的所有起点（0-based）

样例：模式 `aba`, `a`，文本 `"ababa"` -> `occ[0] = {0, 2}`, `occ[1] = {0, 2, 4}`。

【传参要求（照这个传不会错）】

- `insert(pattern)`：插入一个模式串，返回 `int` 编号（按插入顺序从 0 开始）；重复模式串会让同一终点保存多个 `id`。
- `build()`：所有模式插完后调用一次；没 `build` 就 `collect` 是错的。
- `collect_occurrences(text)`：返回 `vector<vector<int>>`，`occ[id]` = 第 `id` 个模式在 `text` 中的所有出现起点（0-based，升序）。
- 命中模式很多时输出总量可能巨大，先估量再收集。

【API / 入口函数（赛时只认这里列的名字）】

- `build()` -> 完成建树或预处理
- `insert(const string& pattern)` -> 插入元素/字符串 返回 `int`。

【改板时先认这几个量】

- `up`: fail 链上最近的有模式结束的节点（枚举命中时沿它跳）。
- `fail`: AC 失配指针。

```
struct PatternOccurrenceAC {
    static constexpr int SIG = 26;
    struct Node {
        array<int, SIG> next{};
        int fail = 0;
        int up = -1;
        vector<int> out;
    };

    vector<Node> trie{Node{}};
    vector<int> terminal, length;

    int insert(const string& pattern) {
        assert(!pattern.empty());
        int u = 0;
        for (char ch : pattern) {
            int c = ch - 'a';
            assert(0 <= c && c < SIG);
            if (!trie[u].next[c]) {
                trie[u].next[c] = (int)trie.size();
                trie.push_back(Node{});
            }
            u = trie[u].next[c];
        }
        int id = (int)length.size();
        length.push_back((int)pattern.size());
        terminal.push_back(u);
        trie[u].out.push_back(id);
        return id;
    }

    void build() {
        queue<int> q;
        for (int c = 0; c < SIG; ++c) {
            int v = trie[0].next[c];
            if (v) q.push(v);
        }
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            int f = trie[u].fail;
            trie[u].up = trie[f].out.empty() ? trie[f].up : f;
            for (int c = 0; c < SIG; ++c) {
                int v = trie[u].next[c];
                if (v) {
                    trie[v].fail = trie[f].next[c];
                    q.push(v);
                } else {
                    trie[u].next[c] = trie[f].next[c];
                }
            }
        }
    }

    vector<vector<int>> collect_occurrences(const string& text) const {
        vector<vector<int>> occ(length.size());
        int u = 0;
        for (int i = 0; i < (int)text.size(); ++i) {
            int c = text[i] - 'a';
            assert(0 <= c && c < SIG);
            u = trie[u].next[c];
            for (int v = trie[u].out.empty() ? trie[u].up : u; v != -1; v = trie[v].up) {
                for (int id : trie[v].out) occ[id].push_back(i - length[id] + 1);
            }
        }
        return occ;
    }
};
```

典题：本场 L 《Substrings of Substrings》。先用本模板收集每个询问串在 `s` 中的所有出现起点，再把出现起点交给下一节的区间贡献公式。

典题：子串出现覆盖的所有区间最大权值和与总权值和

赛时先看

题目信号：题面说区间是好的当且仅当模式串是该区间的子串；需要统计所有好区间的最大子段权值/总权值；模式串出现位置已经能求出来。

本质：给定数组权值和某模式串在文本中的所有出现起点，统计所有“包含至少一次该模式”的区间的最大权值和，以及全部区间权值和。

接法：先对原权值做前缀和 P 、 P 的前缀最小值、后缀最大值，以及 P 的前缀和。对于第 k 个出现起点 p ，左端点只允许在 $(prev, p]$ 之间，右端点在 $[p+len, n]$ 的前缀下标之间。

复杂度判定：对一个模式串，若出现次数为 m ，计算为 $O(m + n)$ 预处理后 $O(m)$ ；所有模式合计取决于出现位置总量。

维护的量：`prefix`（权值前缀和）、`prefix_min`（前缀最小前缀和）、`suffix_max`（后缀最大前缀和）、`sum_prefix_values`（前缀和数组的前缀和，用于快速求区间权和）。

警告：要按出现起点分段，避免同一个好区间被多个出现位置重复统计。位置统一用 0-based 起点，权值数组用 0-based。

【最小完整示例（先抄这一段就能跑）】

题目：给权值数组与模式串的所有出现起点，统计所有“包含至少一次该模式”的区间的最大权值和与总权值和（模 mod ）。

```
vector<i64> a = {1, 2, 3};           // 0-based 权值数组
vector<int> occ = {1};              // 模式出现起点 (0-based)
auto res = covered_intervals_by_occurrences(a, 1, occ);
cout << res.maximum << ' ' << res.sum_mod << '\n'; // 调用：最大权和 总权和(模)
```

样例： $a = \{1, 2, 3\}$ 、 $pattern_length = 1$ 、起点 $\{1\}$ $\rightarrow$ $maximum = 6$ ， $sum_mod = 16$ 。

【传参要求（照这个传不会错）】

- `weight`: `vector<i64>`，0-based 权值数组，长度 n 。
- `pattern_length`: 模式串长度 ($1..n$)。
- `occurrence`: `vector<int>`，所有出现起点 (0-based；顺序任意，内部会排序去重)。
- `mod`: 总权值和取模值（默认 $modn$ ）。
- 返回值: `CoveredIntervalStats{maximum, sum_mod}`——`maximum` 是所有好区间中的最大权值和；`sum_mod` 是全部好区间权值和模 mod 。
- 配套用法：出现起点先用上一节 `PatternOccurrenceAC::collect_occurrences` 拿到。

```
struct CoveredIntervalStats {
    i64 maximum = -(1LL << 62);
    i64 sum_mod = 0;
};

i64 normalize_mod(i128 x, i64 mod) {
    x %= mod;
    if (x < 0) x += mod;
    return (i64)x;
}

CoveredIntervalStats covered_intervals_by_occurrences(
    const vector<i64>& weight, int pattern_length, vector<int> occurrence,
    i64 mod = modn
) {
    int n = (int)weight.size();
    assert(1 <= pattern_length && pattern_length <= n);
    sort(occurrence.begin(), occurrence.end());
    occurrence.erase(unique(occurrence.begin(), occurrence.end()), occurrence.end());

    vector<i64> prefix(n + 1), prefix_min(n + 1), suffix_max(n + 1);
    for (int i = 0; i < n; ++i) prefix[i + 1] = prefix[i] + weight[i];
    prefix_min[0] = prefix[0];
    for (int i = 1; i <= n; ++i) prefix_min[i] = min(prefix_min[i - 1], prefix[i]);
    suffix_max[n] = prefix[n];
    for (int i = n - 1; i >= 0; --i) suffix_max[i] = max(suffix_max[i + 1], prefix[i]);

    vector<i64> sum_prefix_values(n + 2);
    for (int i = 0; i <= n; ++i) sum_prefix_values[i + 1] = sum_prefix_values[i] + prefix[i];
    auto range_sum_prefix_values = [&](int l, int r) -> i64 {
        if (l > r) return 0;
        return sum_prefix_values[r + 1] - sum_prefix_values[l];
    };

    CoveredIntervalStats result;
```

```

i128 total = 0;
int previous = -1;
for (int p : occurrence) {
    assert(0 <= p && p + pattern_length <= n);
    result.maximum = max(result.maximum, suffix_max[p + pattern_length] - prefix_min[p]);

    i64 count_left = p - previous;
    i64 count_right = n - (p + pattern_length) + 1;
    i64 sum_right = range_sum_prefix_values(p + pattern_length, n);
    i64 sum_left = range_sum_prefix_values(previous + 1, p);
    total += (i128)count_left * sum_right - (i128)count_right * sum_left;
    previous = p;
}
result.sum_mod = normalize_mod(total, mod);
return result;
}

```

典题：本场 L。对每个询问串拿到出现起点 `occ` 后，调用 `covered_intervals_by_occurrences(a, |t|, occ)`，输出 `maximum` 和 `sum_mod`。

AC 自动机 DP：计数不含任一禁串的字符串

赛时先看

题目信号：给很多敏感词/禁止子串，问长度为 `len` 的合法串数量；或要求在构造/数位 DP 中维护“到目前为止没有出现禁串”。

本质：有多个禁用模式串，计数/构造不出现任何模式串的字符串。它是单模式 KMP 自动机的多模式推广。

接法：长度 `len` 的字符串只由 `'a'..'a'+sigma-1` 组成，给 `patterns`，求不含任意模式串的数量。`ACAvoidPatterns ac; for (auto& s : patterns) ac.insert(s); ac.build(); cout << ac.count_avoiding(len,sigma);`。如果题目要构造字典序最小合法串，贪心试字符并用“剩余长度是否仍存在合法状态”的 DP 判定。

复杂度判定：建树 $O(\text{总模式长度} * \text{字符集})$ ，普通长度 DP $O(\text{len} * \text{节点数} * \text{字符集})$ ；`len`

极大时，将合法状态之间的转移做矩阵快速幂。

维护的量：`tr` (Trie 节点池)；每个节点 `forbidden` (该节点或其 fail 祖先上是否有禁串结尾)；`dp[u]` (长度为当前已填部分的合法状态计数)。

警告：`forbidden` 要从 `fail`

父亲向下传递；模式串可以互为后缀；空模式串意味着所有字符串非法，本模板禁止插入空串；先全部 `insert` 后再 `build`。

【最小完整示例（先抄这一段就能跑）】

题目：长度 `len` 的字符串只用 `'a'..'a'+sigma-1` 组成，求不含任何禁串的字符串数量（模 `modn`）。

```

ACAvoidPatterns ac;
for (auto& s : patterns) ac.insert(s);    // 1. 插入所有禁串（非空小写串）
ac.build();                               // 2. 全部插入后 build 一次
i64 ans = ac.count_avoiding(len, sigma);  // 3. 调用：长度 len、字符集大小 sigma
cout << ans << '\n';

```

样例：禁串 `"ab"`，`len = 2`，`sigma = 2` $\rightarrow$ 3 (aa、ba、bb)。

【传参要求（照这个传不会错）】

- `insert(pattern)`：插入一个禁串（非空、小写）；模式串可互为后缀，`build` 时自动传递 `forbidden`。
- `build()`：所有禁串插入完再调用一次；没 `build` 就 `count` 是错的。
- `count_avoiding(len, alphabet_size, modulus = modn)`：`len` 目标长度 (≥ 0)；`alphabet_size` 字符集大小 (1..26，取前 `alphabet_size` 个小写字母)；`modulus` 取模值 (默认 `modn`)；返回 `i64` 模意义下的合法串数量。
- `len` 极大 ($1e9$ 量级) 时，把状态间转移写成矩阵快速幂。

【API / 入口函数（赛时只认这里列的名字）】

- `build()` $\rightarrow$ 完成建树或预处理
- `insert(const string& pattern)` $\rightarrow$ 插入元素/字符串

【改板时先认这几个量】

- `tr`：树节点池/自动机节点池。
- `fail`：AC 失配指针。
- `nxt`：转移/子节点。
- `dp`：DP 状态。

状态：AC 节点表示当前后缀匹配到的 trie 节点；若节点自身或其 fail 链上有一个模式串结尾，则该状态 forbidden，DP 不得进入。

```
struct ACAvoidPatterns {
    static constexpr int SIG = 26;
    struct Node {
        array<int, SIG> nxt{};
        int fail = 0;
        bool forbidden = false;
    };

    vector<Node> tr{Node{}};

    void insert(const string& pattern) {
        assert(!pattern.empty());
        int u = 0;
        for (char ch : pattern) {
            int c = ch - 'a';
            assert(0 <= c && c < SIG);
            if (!tr[u].nxt[c]) {
                tr[u].nxt[c] = (int)tr.size();
                tr.push_back(Node{});
            }
            u = tr[u].nxt[c];
        }
        tr[u].forbidden = true;
    }

    void build() {
        queue<int> q;
        for (int c = 0; c < SIG; ++c) {
            int v = tr[0].nxt[c];
            if (v) q.push(v);
        }
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            tr[u].forbidden = tr[u].forbidden || tr[tr[u].fail].forbidden;
            for (int c = 0; c < SIG; ++c) {
                int v = tr[u].nxt[c];
                if (v) {
                    tr[v].fail = tr[tr[u].fail].nxt[c];
                    q.push(v);
                } else {
                    tr[u].nxt[c] = tr[tr[u].fail].nxt[c];
                }
            }
        }
    }

    i64 count_avoiding(int len, int alphabet_size, i64 modulus = modn) const {
        assert(0 <= len && 1 <= alphabet_size && alphabet_size <= SIG && modulus > 0);
        vector<i64> dp(tr.size());
        dp[0] = 1 % modulus;
        for (int pos = 0; pos < len; ++pos) {
            vector<i64> next_dp(tr.size());
            for (int u = 0; u < (int)tr.size(); ++u) {
                if (tr[u].forbidden || dp[u] == 0) continue;
                for (int c = 0; c < alphabet_size; ++c) {
                    int v = tr[u].nxt[c];
                    if (tr[v].forbidden) continue;
                    next_dp[v] += dp[u];
                    if (next_dp[v] >= modulus) next_dp[v] -= modulus;
                }
            }
            dp.swap(next_dp);
        }
        i64 answer = 0;
        for (int u = 0; u < (int)tr.size(); ++u) {
            if (tr[u].forbidden) continue;
            answer += dp[u];
            if (answer >= modulus) answer -= modulus;
        }
        return answer;
    }
};
```

典题模型：长度 `len` 的字符串只由 `'a'..'a'+sigma-1` 组成，给

`patterns`，求不含任意模式串的数量。ACAvoidPatterns ac; for (auto& s: patterns) ac.insert(s); ac.build();
cout << ac.count_avoiding(len,sigma);。如果题目要构造字典序最小合法串，贪心试字符并用“剩余长度是否仍存在合法状态”的 DP 判定。

双哈希

赛时先看

题目信号：大量子串相等判断，不能每次逐字符比较。

本质： $O(1)$ 比较子串、子串去重、回文辅助。

接法：先 `DoubleHash h(s)`；比较 `s[l1..r1]` 和 `s[l2..r2]` 时先判断长度相等，再比较 `h.get(l1,r1) == h.get(l2,r2)`。本模板区间是 0-based 且两端闭区间。若题目卡哈希，命中答案后逐字符复核。

复杂度判定：预处理 $O(n)$ ，查询 $O(1)$ 。

维护的量： $h1/h2$ （前缀哈希， $h[i]$ 是 `s[0..i-1]` 的哈希）； $p1/p2$ （底数幂， $p[i] = \text{BASE}^i$ ，用于 `get` 时把区间左端对齐）。

警告：哈希有冲突，正式赛用双模更稳。

【最小完整示例（先抄这一段就能跑）】

题目：多次判断字符串 `s` 的两个子串是否相等。

```
string s;
cin >> s;
DoubleHash h(s);           // 1. 结构体定义：DoubleHash(字符串)
if (h.get(l1, r1) == h.get(l2, r2)) // 2. 调用：比较两个子串
    cout << "equal" << '\n';
```

样例：`s = "abcabc"`；`get(0,2) == get(3,5) -> equal`（都是 `abc`）。

【传参要求（照这个传不会错）】

- `DoubleHash(s)`：构造时预处理；`s` 是普通字符串（0-based）。
- `get(l, r)`：返回子串 `s[l..r]` 的双模哈希；0-based 且两端闭区间。
- 比较前必须先确认两段长度相等（长度不同的哈希没有可比性）。
- 返回值是 `pair<i64, i64>`，直接 `==` 比较。
- 哈希有极小碰撞概率；对抗数据建议命中后逐字符复核。

【API / 入口函数（赛时只认这里列的名字）】

- `build(const string& s) ->` 完成建树或预处理

```
struct DoubleHash {
    static const i64 MOD1 = 1000000007LL;
    static const i64 MOD2 = 1000000009LL;
    static const i64 BASE = 911382323LL;
    vector<i64> h1, h2, p1, p2;

    DoubleHash() = default;
    DoubleHash(const string& s) { build(s); }

    void build(const string& s) {
        int n = (int)s.size();
        h1.assign(n + 1, 0); h2.assign(n + 1, 0);
        p1.assign(n + 1, 1); p2.assign(n + 1, 1);
        for (int i = 0; i < n; ++i) {
            int x = (unsigned char)s[i] + 1;
            h1[i + 1] = (h1[i] * BASE + x) % MOD1;
            h2[i + 1] = (h2[i] * BASE + x) % MOD2;
            p1[i + 1] = p1[i] * BASE % MOD1;
            p2[i + 1] = p2[i] * BASE % MOD2;
        }
    }

    pair<i64, i64> get(int l, int r) const {
        i64 a = (h1[r + 1] - h1[l] * p1[r - l + 1]) % MOD1;
        i64 b = (h2[r + 1] - h2[l] * p2[r - l + 1]) % MOD2;
        if (a < 0) a += MOD1;
        if (b < 0) b += MOD2;
        return {a, b};
    }
};
```

双模二维滚动哈希： $O(1)$ 取任意子矩阵

赛时先看

题目信号：二维字符/颜色/数字网格；多次问“两个子矩阵是否完全相同”“某个矩形是否等于给定图案”；或需要枚举许多候选位置而逐格比较会到 $O(n^2m^2)$ 。

本质：预处理一个字符矩阵后，快速判断两块同尺寸矩形是否相等。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定：预处理 $O(nm)$ ，单个子矩阵哈希 $O(1)$ ，空间 $O(nm)$ ；双模可显著降低随机碰撞概率。

维护的量：hash1/hash2（两个模数下的二维前缀多项式哈希， $H[i][j]$ 是 $[0,i) \times [0,j)$ 的哈希）；row_pow/col_pow（行、列底数幂，查询时按位置归一化）；n/m（网格行数/列数）。

警告：哈希相等不是绝对相等，对抗性数据建议命中后逐格复核；不同尺寸矩形的哈希不可比，查询前必须确认高宽相同。

约定：下方实现统一为 0-indexed 半开区间 $[x1,x2) \times [y1,y2)$ ，不要混用闭区间；双模二维多项式哈希。所有坐标均为 0-indexed、右下角开区间 $[x1,x2) \times [y1,y2)$

【最小完整示例（先抄这一段就能跑）】

题目：n x m 字符网格，多次判断两个同尺寸子矩形是否完全相同。

```
DoubleHash2D h(grid);           // 1. 结构体定义：DoubleHash2D(字符网格)
if (h.get(x1, y1, x2, y2) == h.get(a1, b1, a2, b2)) // 2. 调用：比较两块矩形
    cout << "same" << '\n';
```

样例：grid = {"ab","ab"}; get(0,0,1,1) == get(0,1,1,2) -> same（都是 b）。

【传参要求（照这个传不会错）】

- DoubleHash2D(grid)：构造即预处理；grid 是 vector<string>，所有行必须等长。
- get(x1, y1, x2, y2)：半开区间 $[x1,x2) \times [y1,y2)$ ，全部 0-indexed。
- 比较前必须确认两块矩形高、宽相同（不同尺寸不可比）。
- 返回 Hash2DValue，直接 == 比较；对抗数据建议命中后逐格复核。

【API / 入口函数（赛时只认这里列的名字）】

- get(int x1, int y1, int x2, int y2) -> 返回矩形 $[x1,x2) \times [y1,y2)$ 的归一化哈希；比较前必须确认两个矩形尺寸相同。

状态：令 $H[i][j]$ 是左上角到 $[0,i) \times [0,j)$ 的二维多项式哈希。行、列使用不同底数；查询时像二维前缀和一样容斥，并乘对应幂把位置归一化。（add/sub/mul 是内部取模助手，不要直接调。）

- 哈希相等不是数学上的绝对相等。对抗性强或必须零误判时，哈希命中后再逐格确认；不要只用单模数。
- 比较前必须确认两个矩形的高、宽都相同；不同尺寸的多项式哈希没有可比性。
- 下方实现统一为 0-indexed 半开区间 $[x1,x2) \times [y1,y2)$ ，不要混用闭区间。
- char 可能是有符号类型，映射字符值时转为 unsigned char；网格行长度必须相等。

```
struct Hash2DValue {
    int first = 0;
    int second = 0;

    bool operator==(const Hash2DValue& other) const {
        return first == other.first && second == other.second;
    }
    bool operator!=(const Hash2DValue& other) const {
        return !(*this == other);
    }
};

// 双模二维多项式哈希。所有坐标均为 0-indexed、右下角开区间 [x1,x2) x [y1,y2)。
struct DoubleHash2D {
    static constexpr int MOD1 = 1'000'000'007;
    static constexpr int MOD2 = 1'000'000'009;
    static constexpr int ROW_BASE1 = 911382323;
    static constexpr int COL_BASE1 = 972663749;
    static constexpr int ROW_BASE2 = 972663749;
    static constexpr int COL_BASE2 = 911382323;

    int n = 0, m = 0;
    vector<int> row_pow1, col_pow1, row_pow2, col_pow2;
    vector<vector<int>> hash1, hash2;

    static int add(int a, int b, int mod) {
        a += b;
        if (a >= mod) a -= mod;
        return a;
    }

    static int sub(int a, int b, int mod) {
```

```

    a -= b;
    if (a < 0) a += mod;
    return a;
}

static int mul(int a, int b, int mod) {
    return (i64)a * b % mod;
}

DoubleHash2D() = default;

explicit DoubleHash2D(const vector<string>& grid) {
    n = (int)grid.size();
    m = n == 0 ? 0 : (int)grid[0].size();
    for (const string& row : grid) assert((int)row.size() == m);

    row_pow1.assign(n + 1, 1);
    row_pow2.assign(n + 1, 1);
    for (int i = 1; i <= n; ++i) {
        row_pow1[i] = mul(row_pow1[i - 1], ROW_BASE1, MOD1);
        row_pow2[i] = mul(row_pow2[i - 1], ROW_BASE2, MOD2);
    }
    col_pow1.assign(m + 1, 1);
    col_pow2.assign(m + 1, 1);
    for (int j = 1; j <= m; ++j) {
        col_pow1[j] = mul(col_pow1[j - 1], COL_BASE1, MOD1);
        col_pow2[j] = mul(col_pow2[j - 1], COL_BASE2, MOD2);
    }

    hash1.assign(n + 1, vector<int>(m + 1));
    hash2.assign(n + 1, vector<int>(m + 1));
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) {
            int value = (unsigned char)grid[i][j] + 1;
            hash1[i + 1][j + 1] = extend(
                hash1, i, j, value, MOD1, ROW_BASE1, COL_BASE1
            );
            hash2[i + 1][j + 1] = extend(
                hash2, i, j, value, MOD2, ROW_BASE2, COL_BASE2
            );
        }
    }
}

// 返回矩形 [x1, x2) x [y1, y2) 的归一化哈希；比较前必须确认两个矩形尺寸相同。
Hash2DValue get(int x1, int y1, int x2, int y2) const {
    assert(0 <= x1 && x1 <= x2 && x2 <= n);
    assert(0 <= y1 && y1 <= y2 && y2 <= m);
    return {
        rectangle_hash(hash1, x1, y1, x2, y2, row_pow1, col_pow1, MOD1),
        rectangle_hash(hash2, x1, y1, x2, y2, row_pow2, col_pow2, MOD2),
    };
}

private:
    static int extend(const vector<vector<int>>& hash, int i, int j, int value,
                     int mod, int row_base, int col_base) {
        int answer = value;
        answer = add(answer, mul(hash[i][j + 1], row_base, mod), mod);
        answer = add(answer, mul(hash[i + 1][j], col_base, mod), mod);
        answer = sub(answer, mul(mul(hash[i][j], row_base, mod), col_base, mod), mod);
        return answer;
    }

    static int rectangle_hash(const vector<vector<int>>& hash,
                              int x1, int y1, int x2, int y2,
                              const vector<int>& row_power,
                              const vector<int>& col_power, int mod) {
        int rows = x2 - x1, cols = y2 - y1;
        int answer = hash[x2][y2];
        answer = sub(answer, mul(hash[x1][y2], row_power[rows], mod), mod);
        answer = sub(answer, mul(hash[x2][y1], col_power[cols], mod), mod);
        answer = add(answer, mul(mul(hash[x1][y1], row_power[rows], mod),
                                   col_power[cols], mod), mod);
        return answer;
    }
};

```

典题 std：在大网格中找出全部图案出现位置

赛时先看

题目信号：题目是“找二维字符串/像素块/棋盘局面出现的位置”，模式大小固定但候选位置很多；允许用哈希，或可以在命中后验证。

本质：文本矩阵 `text` 给定，模式矩阵 `pattern`

给定，返回所有左上角位置。可以直接改成只问是否存在、第一次出现位置或出现次数。

接法：给 $n \times m$ 字符网格与 $h \times w$ 图案，调用 `find_2d_pattern(text, pattern)` 后输出 `positions.size()`。若网格中有一次修改、随后多次查询，不要每次重建二维哈希，改用二维 Fenwick/线段树或离线处理；普通滚动哈希只适合静态矩阵。

复杂度判定：建哈希 $O(nm + hw)$ ，扫描 $O((n-h+1)(m-w+1))$ ；每个候选位置的哈希比较 $O(1)$ 。

维护的量：`text_hash/pattern_hash`（两个 `DoubleHash2D`）；`positions`（所有匹配的左上角坐标，0-based）。

警告：模式比文本大时答案为空；比赛若严格防碰撞，在 `if` 命中后循环 `i, j` 比较每个字符再加入答案。

【最小完整示例（先抄这一段就能跑）】

题目：在 $n \times m$ 字符网格中找出 $h \times w$ 图案的全部出现位置（左上角）。

```
vector<string> grid = {"ab", "ab", "cd"}; // 文本网格（每行等长）
vector<string> pat = {"ab"};           // 图案（h x w）
auto pos = find_2d_pattern(grid, pat); // 1. 调用：返回所有左上角
for (auto [x, y] : pos) cout << x << ' ' << y << '\n'; // 2. 0-based 坐标
```

样例：`grid = {"ab","ab","cd"}、pat = {"ab"} → 左上角 (0,0)、(1,0)。`

【传参要求（照这个传不会错）】

- `text`: `vector<string>`， $n \times m$ 字符网格，每行等长（0-based）。
- `pattern`: `vector<string>`， $h \times w$ 图案，非空且每行等长。
- 返回值: `vector<pair<int,int>>`，每个元素是一个匹配的左上角坐标（0-based，行序递增）。
- $h > n$ 或 $w > m$ （图案比文本大）时返回空；对抗数据可对命中位置逐格复核。

```
vector<pair<int, int>> find_2d_pattern(const vector<string>& text,
                                     const vector<string>& pattern) {
    assert(!pattern.empty() && !pattern[0].empty());
    int pattern_width = (int)pattern[0].size();
    for (const string& row : pattern) assert((int)row.size() == pattern_width);
    if (text.empty()) return {};

    DoubleHash2D text_hash(text), pattern_hash(pattern);
    int n = text_hash.n, m = text_hash.m;
    int height = pattern_hash.n, width = pattern_hash.m;
    if (height > n || width > m) return {};

    Hash2DValue target = pattern_hash.get(0, 0, height, width);
    vector<pair<int, int>> positions;
    for (int x = 0; x + height <= n; ++x) {
        for (int y = 0; y + width <= m; ++y) {
            if (text_hash.get(x, y, x + height, y + width) == target) {
                positions.push_back({x, y});
            }
        }
    }
    return positions;
}
```

典型模型：给 $n \times m$ 字符网格与 $h \times w$ 图案，调用 `find_2d_pattern(text, pattern)` 后输出 `positions.size()`。若网格中有一次修改、随后多次查询，不要每次重建二维哈希，改用二维 Fenwick/线段树或离线处理；普通滚动哈希只适合静态矩阵。

Manacher

赛时先看

题目信号：题面出现“回文子串/最长回文/回文半径”，或要求对每个中心求回文长度。连续回文子串问题先翻 Manacher。

本质：回文有镜像对称性质：当前最右回文 `[l, r]`

内部的中心，其半径初值可借镜像中心提供，再向外暴力扩展；摊下来每个字符只被扩 $O(1)$ 次。

复杂度判定： $O(n)$ ； n 到 $1e7$ 也稳（常数极小、无 $\log$ ）；求最长回文子序列或“删字符变回文”不要用这里，要翻 DP。

维护的量：`d1[i]`（以 `i` 为中心的奇回文半径）、`d2[i]`（中心在 `i-1/i` 之间的偶回文半径）、当前最右回文 `[l, r]`。

接法：最长回文子串翻 Manacher；最长回文子序列不要翻这里，要翻 DP。`d1[i]` 表示以 `i` 为中心的奇数回文半径，实际长度 $2*d1[i]-1$ ；`d2[i]` 表示中心在 `i-1` 和 `i` 之间的偶数回文半径，实际长度 $2*d2[i]$ 。

警告：`d1` 是奇数回文半径，`d2` 是偶数回文半径；半径与实际长度的换算 $(2*d1-1 / 2*d2)$ 是本节最容易错的地方。

【最小完整示例（先抄这一段就能跑）】

题目：求字符串 `s` 的最长回文子串长度。

```
string s;
cin >> s;
auto d1 = manacher_odd(s); // 1. 调用：奇数回文半径数组（长度 = 2*d1[i]-1）
auto d2 = manacher_even(s); // 2. 调用：偶数回文半径数组（长度 = 2*d2[i]）
int ans = 1;
for (int i = 0; i < (int)s.size(); ++i) {
    ans = max(ans, 2 * d1[i] - 1); // 以 i 为中心的奇回文
    ans = max(ans, 2 * d2[i]); // 中心在 i-1 与 i 之间的偶回文
}
cout << ans << '\n';
```

样例：`s = "abba" -> ans = 4`；`s = "ababc" -> ans = 3`。

【传参要求（照这个传不会错）】

- `manacher_odd(s)`：返回 `d1[i]` = 以 `i` 为中心的奇回文半径；实际长度 `2*d1[i]-1`。
- `manacher_even(s)`：返回 `d2[i]` = 中心在 `i-1/i` 之间的偶回文半径；实际长度 `2*d2[i]`。
- 输入输出都是 0-based；两个数组长度都等于 `|s|`。
- 求“最长回文子序列”（允许跳字符）不要用这里，翻 D 章 DP。

【不会用就照抄】

```
auto d1 = manacher_odd(s); // 奇数回文：长度 = 2*d1[i]-1
auto d2 = manacher_even(s); // 偶数回文：长度 = 2*d2[i]
```

- 这类板子最容易错在“半径到底含不含中心”和“原串下标如何映射”；比赛时直接沿用本节返回约定。

【API / 入口函数（赛时只认这里列的名字）】

- `manacher_odd(s)` -> 返回 `d1`；奇回文实际长度 `2*d1[i]-1`。
- `manacher_even(s)` -> 返回 `d2`；中心在 `i-1/i` 间，实际长度 `2*d2[i]`。

【抄板清单（照着做就行）】

1. 抄哪段：`manacher_odd` 和 `manacher_even` 两个函数，整段抄到 `solve()` 上面。
2. 构造：不用构造对象，直接当普通函数调。
3. 调用：`auto d1 = manacher_odd(s); auto d2 = manacher_even(s);`
4. 取结果：以 `i` 为中心的奇回文长度 `2*d1[i]-1`；中心在 `i-1/i` 间的偶回文长度 `2*d2[i]`。

【改造点（按题目改这几处）】

- 求最长回文子串：扫一遍 `d1/d2` 取最大长度；奇数最长候选起点 `i - d1[i] + 1`，偶数候选起点 `i - d2[i]`，按长度优先映射回原串下标输出。
- 只关心最长长度：`max_len = max(max_i(2*d1[i]-1), max_i(2*d2[i]))`。
- 统计回文子串数量：`sum(d1) + sum(d2)`（每个中心半径之和就是回文子串个数）。
- 题目以“某个已知回文 + 扩展”的形式问：直接 `2*d1[i]-1 / 2*d2[i]` 换算，别自己数。

【核心逻辑（改代码时别破坏）】

- 维护当前最右回文 `[l,r]`，镜像半径提供初值，再向外暴力扩。
- `d1[i]` 含中心，奇回文长度 `2*d1[i]-1`；`d2[i]` 是缝中心，偶回文长度 `2*d2[i]`。

```
// 维护的量：d[i] = 以 i 为中心（含中心）的奇回文半径；l/r = 当前最右回文 [l,r]。
// 不变量：i <= r 时初值 k = min(镜像半径, r-i+1)，保证不落后于已发现的最长回文。
vector<int> manacher_odd(const string& s) {
    int n = (int)s.size();
    vector<int> d(n);
    for (int i = 0, l = 0, r = -1; i < n; ++i) {
        int k = (i > r) ? 1 : min(d[l + r - i], r - i + 1); // 镜像半径给初值
        while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) k++;
        d[i] = k--;
        if (i + k > r) l = i - k, r = i + k; // 更新最右回文边界
    }
    return d;
}

// 维护的量：d[i] = 中心在 i-1 与 i 之间的偶回文半径（单位为半个字符）；l/r = 当前最右回文。
// 不变量：半径 k 对应回文区间 [i-k, i+k-1]，长度恒为 2*k。
vector<int> manacher_even(const string& s) {
    int n = (int)s.size();
    vector<int> d(n);
```



```

for (int i = 0, l = 0, r = -1; i < n; ++i) {
    int k = (i > r) ? 0 : min(d[l + r - i + 1], r - i + 1);
    while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) k++;
    d[i] = k--;
    if (i + k > r) l = i - k - 1, r = i + k;
}
return d;
}

```

回文自动机 PAM

赛时先看

题目信号：动态加入字符；询问不同回文串数量、每个回文出现次数。

本质：维护一个字符串的所有不同回文子串，统计数量、出现次数、最长回文后缀。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定： $O(n * \text{alphabet})$ 或用 map 后 $O(n \log \text{alphabet})$ 。

维护的量：tr（回文树节点池，tr[0]/tr[1] 是长度 0/-1

的两个根）；last（当前字符的最长回文后缀节点）；cnt（节点回文出现次数，count_occurrences 后才是真实值）。

警告：有两个根，长度分别为 0 和 -1；fail 链用于找最长可扩展回文后缀。

【最小完整示例（先抄这一段就能跑）】

题目：逐个加入字符，求不同回文子串数量与每个回文的出现次数。

```

string s;
cin >> s;
PalindromicTree pt((int)s.size()); // 1. 构造：参数 n 是总字符数上界
for (char ch : s) pt.add_char(ch); // 2. 逐个加入字符
cout << (int)pt.tr.size() - 2 << '\n'; // 3. 不同回文子串数（节点数减两个根）

```

样例：s = "ababa" -> 不同回文子串 5（a、b、aba、bab、ababa）。

【传参要求（照这个传不会错）】

- PalindromicTree pt(n) / init(n): n 为将要加入的字符总数上界（预留节点）。
- add_char(ch): 追加一个小写字符；所有字符加完后可用 count_occurrences() 按 fail 链自底向上累加，之后 tr[i].cnt 是第 i 个节点回文串的出现次数。
- 节点下标：tr[0] 是长度 0 的根（fail = 1），tr[1] 是长度 -1 的根，tr[2..] 才是真回文节点。
- 不同回文子串数量 = tr.size() - 2（去掉两个根）。

【API / 入口函数（赛时只认这里列的名字）】

- init(int n) -> 初始化/清空结构

【改板时先认这几个量】

- tr: 树节点池/自动机节点池。
- cur: 当前节点（get_fail 找到的最长可扩展回文后缀）。
- fail: 回文后缀指针。

```

struct PalindromicTree {
    struct Node {
        int next[26] = {};
        int len = 0, fail = 0, cnt = 0;
    };
    vector<Node> tr;
    string s;
    int last;

    PalindromicTree(int n = 0) { init(n); }
    void init(int n) {
        tr.assign(2, Node{});
        tr.reserve(n + 3);
        tr[0].len = 0; tr[0].fail = 1;
        tr[1].len = -1; tr[1].fail = 1;
        s = "#";
        last = 0;
    }

    int get_fail(int x) {
        int pos = (int)s.size() - 1;
        while (s[pos - tr[x].len - 1] != s[pos]) x = tr[x].fail;
        return x;
    }
}

```

```

    }

    void add_char(char ch) {
        s.push_back(ch);
        int c = ch - 'a';
        int cur = get_fail(last);
        if (!tr[cur].next[c]) {
            Node node;
            node.len = tr[cur].len + 2;
            int fail_to = get_fail(tr[cur].fail);
            node.fail = tr[fail_to].next[c];
            if (!node.fail) node.fail = 0;
            tr[cur].next[c] = (int)tr.size();
            tr.push_back(node);
        }
        last = tr[cur].next[c];
        tr[last].cnt++;
    }

    void count_occurrences() {
        for (int i = (int)tr.size() - 1; i >= 2; i--) {
            tr[tr[i].fail].cnt += tr[i].cnt;
        }
    }
};

```

后缀数组

赛时先看

题目信号：所有子串/后缀按字典序处理。

本质：后缀排序、最长公共子串、不同子串数量。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定： $O(n \log n)$ 。

维护的量：`sa`（后缀按字典序排序后的起点）、`rk`（各起点的排名）、`height`（相邻后缀 LCP）。

警告：`height[i] = LCP(sa[i], sa[i-1])`。

【最小完整示例（先抄这一段就能跑）】

题目：对字符串 `s` 求后缀数组，输出所有后缀按字典序排列后的起点。

```

string s;
cin >> s;
SuffixArray sa(s); // 1. 构造：内部自动 build
for (int i = 0; i < (int)s.size(); ++i)
    cout << sa.sa[i] << ' '; // 2. 字典序第 i 小后缀的起点 (0-based)

```

样例：`s = "banana" -> sa = {5, 3, 1, 0, 4, 2}`。

【传参要求（照这个传不会错）】

- `SuffixArray(string s_)` / `build(s_)`: `s_` 原串（0-based；字符按 `unsigned char` 值排序）。
- `sa[i]`: 字典序第 `i` 小后缀的起点下标（0-based）。
- `rk[i]`: 起点 `i` 的排名（恒有 `rk[sa[i]] = i`）。
- `height[i]`: `LCP(sa[i], sa[i-1])`（字典序相邻两后缀的 LCP），`height[0] = 0`。
- 不同子串数量 = $n(n+1)/2 - \text{sum}(\text{height})$ 。

【API / 入口函数（赛时只认这里列的名字）】

- `build(string s_)` -> 完成建树或预处理

```

struct SuffixArray {
    string s;
    vector<int> sa, rk, height;

    SuffixArray() = default;
    SuffixArray(string s_) { build(std::move(s_)); }

    void build(string s_) {
        s = std::move(s_);
        int n = (int)s.size();
        sa.resize(n);
        rk.resize(n);
        height.assign(n, 0);
        iota(sa.begin(), sa.end(), 0);
        for (int i = 0; i < n; ++i) rk[i] = (unsigned char)s[i];
    }
};

```

```

vector<int> tmp(n);
for (int k = 1; k <= 1) {
    auto cmp = [&](int a, int b) {
        if (rk[a] != rk[b]) return rk[a] < rk[b];
        int ra = a + k < n ? rk[a + k] : -1;
        int rb = b + k < n ? rk[b + k] : -1;
        return ra < rb;
    };
    sort(sa.begin(), sa.end(), cmp);
    tmp[sa[0]] = 0;
    for (int i = 1; i < n; ++i) tmp[sa[i]] = tmp[sa[i - 1]] + cmp(sa[i - 1], sa[i]);
    rk = tmp;
    if (rk[sa[n - 1]] == n - 1) break;
}

for (int i = 0, k = 0; i < n; ++i) {
    if (rk[i] == 0) continue;
    int j = sa[rk[i] - 1];
    while (i + k < n && j + k < n && s[i + k] == s[j + k]) k++;
    height[rk[i]] = k;
    if (k) k--;
}
};

```

后缀数组 LCP RMQ 查询

赛时先看

题目信号：大量询问两个后缀最长公共前缀；需要比较任意子串字典序。

本质：已建好 SA 与 height 后， $O(1)$ 查询任意两个后缀的 LCP。

接法：把本节 struct/class/函数组 整段抄到 solve() 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定：预处理 $O(n \log n)$ ，查询 $O(1)$ 。

维护的量：st[k][i] (height 从 i 起长 2^k 区间的最小值)；lg[i] (floor(log2(i)))。

警告：height[i] = lcp(sa[i], sa[i-1])，查询 rank 间 (l+1..r) 的最小值。

【最小完整示例（先抄这一段就能跑）】

题目：对字符串 s 建后缀数组后，任意询问两个后缀的最长公共前缀。

```

string s;
cin >> s;
SuffixArray sa(s);           // 1. 建 SA: 内部自动填 sa/rk/height
LcpRMQ rmq(sa.height);      // 2. 用 height 建稀疏表
cout << rmq.lcp_suffix(0, 5, sa.rk) << '\n'; // 3. 后缀 s[0..] 与 s[5..] 的 LCP

```

样例：s = "banana", lcp_suffix(0, 5, sa.rk) -> 1。

【传参要求（照这个传不会错）】

- LcpRMQ(const vector<int>& height = {}) / init(height): height 传上节 SuffixArray::height (height[0] = 0, 其余为相邻后缀 LCP)；0-based。
- range_min(l, r): 闭区间 [l, r]，返回该段 height 最小值。
- lcp_suffix(i, j, rk): i, j 是后缀起点下标 (0-based)，rk 传同一次建出的 sa.rk；返回两后缀最长公共前缀长度；i == j 时返回后缀长度。

【API / 入口函数（赛时只认这里列的名字）】

- init(const vector<int>& height) -> 初始化/清空结构

```

struct LcpRMQ {
    int n;
    vector<int> lg;
    vector<vector<int>> st;

    LcpRMQ(const vector<int>& height = {}) { if (!height.empty()) init(height); }

    void init(const vector<int>& height) {
        n = (int)height.size() - 1;
        lg.assign(n + 1, 0);
        for (int i = 2; i <= n; i++) lg[i] = lg[i >> 1] + 1;
        st.assign(lg[n] + 1, vector<int>(n + 1));
        st[0] = height;
        for (int k = 1; k <= lg[n]; k++) {
            for (int i = 1; i + (1 << k) - 1 <= n; i++) {

```

```

        st[k][i] = min(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
    }
}

int range_min(int l, int r) const {
    int k = lg[r - l + 1];
    return min(st[k][l], st[k][r - (1 << k) + 1]);
}

int lcp_suffix(int i, int j, const vector<int>& rk) const {
    if (i == j) return n - i + 1;
    int a = rk[i], b = rk[j];
    if (a > b) swap(a, b);
    return range_min(a + 1, b);
}
};

```

后缀自动机 SAM

赛时先看

题目信号：对子串集合做统计，后缀数组不够方便。

本质：统计不同子串数量、子串出现次数、最长公共子串。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()`

里先构造对象，再按下方“不会用就看这里”调用公开接口。

复杂度判定： $O(n * \text{字符集转移成本})$ 。

维护的量：`st`（状态数组；`link` 后缀链接、`len` 最长串长、`occurrence` 前缀结束位置数、`next` 转移）；`last` 为整个当前串对应的状态。

警告：`last` 表示整个当前串对应的状态；clone 状态复制转移。

【最小完整示例（先抄这一段就能跑）】

题目：统计字符串 `s` 的本质不同子串数量。

```

string s;
cin >> s;
SuffixAutomaton sam((int)s.size()); // 1. 构造：max_len 传字符串长度即可
for (char c : s) sam.extend(c); // 2. 逐个字符插入
cout << sam.count_distinct_substrings() << '\n'; // 3. 本质不同子串数

```

样例：`s = "abab" → 7`。

【传参要求（照这个传不会错）】

- `SuffixAutomaton(int max_len = 0)`：构造；`max_len` 传字符串长度（内部预留 $2 * \text{max_len}$ 个状态）。
- `extend(char c)`：追加一个字符 `c`（字符集任意，内部 map 转移）；无返回值；按原串顺序调用。
- `count_distinct_substrings()`：返回本质不同子串数量（i64）。

```

struct SuffixAutomaton {
    struct State {
        int link = -1, len = 0;
        i64 occurrence = 0; // suffix-link 汇总前的前缀结束位置数量。
        map<char, int> next;
    };
    vector<State> st;
    int last;

    SuffixAutomaton(int max_len = 0) {
        st.reserve(2 * max_len);
        st.push_back(State{});
        last = 0;
    }

    void extend(char c) {
        int cur = (int)st.size();
        st.push_back(State{});
        st[cur].len = st[last].len + 1;
        st[cur].occurrence = 1;

        int p = last;
        while (p != -1 && !st[p].next.count(c)) {
            st[p].next[c] = cur;
            p = st[p].link;
        }
        if (p == -1) {
            st[cur].link = 0;
        } else {

```

```

    int q = st[p].next[c];
    if (st[p].len + 1 == st[q].len) {
        st[cur].link = q;
    } else {
        int clone = (int)st.size();
        st.push_back(st[q]);
        st[clone].len = st[p].len + 1;
        st[clone].occurrence = 0; // clone 状态不是新的结束位置。
        while (p != -1 && st[p].next[c] == q) {
            st[p].next[c] = clone;
            p = st[p].link;
        }
        st[q].link = st[cur].link = clone;
    }
}
last = cur;
}

i64 count_distinct_substrings() const {
    i64 ans = 0;
    for (int i = 1; i < (int)st.size(); ++i) {
        ans += st[i].len - st[st[i].link].len;
    }
    return ans;
}
};

```

SAM：子串出现次数与两串最长公共子串

赛时先看

题目信号：问一个模式串在文本中作为连续子串出现几次；或问两个串的最长公共连续片段。注意这不是 LCS（最长公共子序列），不能跳过中间字符。

本质：对固定文本串预处理多个模式串的出现次数，或在线性扫描第二个串时求两串最长公共子串长度。两者都直接复用上节构造好的 SuffixAutomaton。

接法：文本串 `text` 固定，有很多模式串询问。先对 `text` 的每个字符 `sam.extend(c)`，再做一次 `auto occ = sam_endpos_sizes(sam)`；每个模式输出 `sam_count_occurrences(sam, occ, pattern)`。若只问 `a, b` 的最长公共子串长度，对 `a` 建 SAM 后调用 `sam_longest_common_substring_length(sam, b)`。

复杂度判定：用 `map` 转移时，预处理出现次数 $O(|S| \log |S|)$ ，单个模式/第二串扫描 $O(|P| \log |S|)$ ；若字符集固定，`map` 换成数组后可视为线性。

维护的量：`order`（按 `len` 从大到小排的状态序，用于自底向上累加）；`occurrence`（各状态 `endpos` 大小，统计时沿 `link` 加到父状态）。

警告：所有非 `clone` 新状态的 `occurrence` 初值为 1，`clone` 必须为 0；统计前按 `len` 从大到小把出现次数累加到 `link`；模式串若沿转移失败，答案就是 0。

【最小完整示例（先抄这一段就能跑）】

题目：文本串 `text` 固定，问模式串 `pattern` 在 `text` 中出现几次，以及 `text` 与串 `other` 的最长公共子串长度。

```

string text, pattern, other;
cin >> text >> pattern >> other;
SuffixAutomaton sam((int)text.size()); // 1. 对文本建 SAM
for (char c : text) sam.extend(c);
auto occ = sam_endpos_sizes(sam); // 2. 所有状态的出现次数
cout << sam_count_occurrences(sam, occ, pattern) << '\n'; // 3. 子串出现次数
cout << sam_longest_common_substring_length(sam, other) << '\n'; // 4. 最长公共子串

```

样例：`text = "ababa", pattern = "aba", other = "bab" -> 2 和 3。`

【传参要求（照这个传不会错）】

- `sam_endpos_sizes(sam)`：在文本全部 `extend` 完之后调用；返回 `vector<i64>`，下标为状态号，`occ[u]` 是状态 `u` 对应子串的出现次数。
- `sam_count_occurrences(sam, occ, pattern)`：`pattern` 非空；返回其在文本中的出现次数，未出现返回 0。
- `sam_longest_common_substring_length(sam, other)`：返回 `other` 与建 SAM 的串的最长公共连续子串长度（无公共字符时为 0）。

【API / 入口函数（赛时只认这里列的名字）】

- `sam_endpos_sizes(const SuffixAutomaton& sam)` -> 必须在文本串所有字符都 `extend` 完之后再调用。返回 `vector<i64>`。

【改板时先认这几个量】

- **order**: 按 **len** 从大到小排序后的状态编号, 用于自底向上累加出现次数。
- **occurrence**: 每个状态的 **endpos** 大小 (出现次数), 统计时沿 **link** 累加到父状态。

// 必须在文本串所有字符都 **extend** 完之后再调用。

```
vector<i64> sam_endpos_sizes(const SuffixAutomaton& sam) {
    int n = (int)sam.st.size();
    vector<int> order(n);
    iota(order.begin(), order.end(), 0);
    sort(order.begin(), order.end(), [&](int x, int y) {
        return sam.st[x].len > sam.st[y].len;
    });

    vector<i64> occurrence(n);
    for (int u = 0; u < n; ++u) occurrence[u] = sam.st[u].occurrence;
    for (int u : order) {
        int parent = sam.st[u].link;
        if (parent != -1) occurrence[parent] += occurrence[u];
    }
    return occurrence;
}

int sam_state_of(const SuffixAutomaton& sam, const string& pattern) {
    int u = 0;
    for (char c : pattern) {
        auto it = sam.st[u].next.find(c);
        if (it == sam.st[u].next.end()) return -1;
        u = it->second;
    }
    return u;
}

i64 sam_count_occurrences(
    const SuffixAutomaton& sam, const vector<i64>& endpos_size, const string& pattern
) {
    assert(!pattern.empty()); // 空串出现次数的定义因题而异, 不能直接默认。
    int u = sam_state_of(sam, pattern);
    return u == -1 ? 0 : endpos_size[u];
}

int sam_longest_common_substring_length(const SuffixAutomaton& sam, const string& other) {
    int u = 0, matched = 0, answer = 0;
    for (char c : other) {
        while (u && !sam.st[u].next.count(c)) {
            u = sam.st[u].link;
            matched = sam.st[u].len;
        }
        auto it = sam.st[u].next.find(c);
        if (it != sam.st[u].next.end()) {
            u = it->second;
            ++matched;
        } else {
            u = 0;
            matched = 0;
        }
        answer = max(answer, matched);
    }
    return answer;
}
```

典型模型: 文本串 **text** 固定, 有很多模式串询问。先对 **text** 的每个字符 **sam.extend(c)**, 再做一次 **auto occ = sam_endpos_sizes(sam)**; 每个模式输出 **sam_count_occurrences(sam, occ, pattern)**。若只问 **a, b** 的最长公共子串长度, 对 **a** 建 SAM 后调用 **sam_longest_common_substring_length(sam, b)**。

多串最长公共子串: SAM 最小匹配长度

赛时先看

题目信号: 题面明确是多个字符串的“共同连续片段”; 字符串数量多、总长度大; 只建立一个串的 SAM, 再逐个串扫描更新匹配长度。

本质: 给多个字符串, 求同时作为连续子串出现的最长字符串, 并恢复一份答案。比逐对求 LCS 更准确, 适用于多个日志、DNA 片段、字符串集合的共同连续片段。

接法: 读入 **m** 个字符串后调用 **MultiStringLongestCommonSubstring solver; auto ans = solver.solve(strings);**; 输出 **ans.length** 或 **ans.substring**。一个字符也没有共同出现时答案长度为 0、字符串为空。

复杂度判定: 设总长度为 **L**, 状态数为 **S**, 这里使用 **map<char, int>**, 复杂度约 $O(L \log \text{alphabet} + S \log S)$; 固定小字符集可将 **map** 改数组降常数。

维护的量: `state` (SAM 状态池, 含 `link/len/first_end/next`); `best[u]` (状态 `u` 在全部串中的最小匹配长度); `matched[u]` (当前扫描串在状态 `u` 的最大匹配长度)。

警告: 求的是最长公共子串 (连续), 不是 LCS; 状态向 `link` 推时必须截断为 `min(match[v], len[link[v]])`; “所有串的不同子串并集”类问题要换广义 SAM, 本模板不适用。

【最小完整示例 (先抄这一段就能跑)】

题目: 给 `m` 个字符串, 求它们共同的最长公共连续子串, 输出长度和一个答案串。

```
int m;
cin >> m;
vector<string> strings(m);
for (int i = 0; i < m; ++i) cin >> strings[i];
MultiStringLongestCommonSubstring solver; // 1. 建求解器 (自动选最短串建 SAM)
auto ans = solver.solve(strings);          // 2. 调用: 返回最长公共子串结果
cout << ans.length << ' ' << ans.substring << '\n'; // 3. 输出长度 + 一个答案串
```

样例: `strings = {"ababc", "babca", "abcb"}` -> `ans.length = 3`, `ans.substring = "abc"`。

【传参要求 (照这个传不会错)】

- `MultiStringLongestCommonSubstring solver`: 先建对象, 构造不需要参数。
- `solver.solve(strings)`: `strings` 为全部待求字符串 (`vector<string>`, 0-based, 任意 `char` 字符集); 返回 `MultiStringLCSResult{length, substring}`。
- `ans.length`: 最长公共子串长度; 没有任何共同字符时为 0。
- `ans.substring`: 一个具体的公共子串; 长度为 0 时空串。
- 求的是公共子串 (连续), 不能跳字符; 跳字符的公共子序列翻 LCS 模板。

【API / 入口函数 (赛时只认这里列的名字)】

- `solve(const vector<string>& strings)` -> 执行主算法并返回答案

核心: 选最短串建 SAM。对每一条其余字符串, 扫描时记录每个 SAM 状态的最大匹配长度, 再按 `len` 从大到小把信息沿 suffix link 向上推。每个状态最终取所有字符串中的最小匹配长度, 最大值就是答案。

- 是最长公共子串, 不能跳过字符; 允许跳字符的是 LCS 模板。
- 状态 `v` 往 `link[v]` 推时必须截断为 `min(match[v], len[link[v]])`, 否则长度会超出父状态表示范围。
- 多串 “不同子串总数并集” 等题需要真正的广义 SAM/Trie-SAM; 本块只解决多串共同子串, 写得更短也更稳。

```
struct MultiStringLCSResult {
    int length = 0;
    string substring;
};

// 任意 char 字符集的多串最长公共子串。选最短串建 SAM, 减少状态数。
struct MultiStringLongestCommonSubstring {
    struct State {
        int link = -1;
        int len = 0;
        int first_end = -1;
        map<char, int> next;
    };

    vector<State> state;
    int last = 0;

    void reset() {
        state.assign(1, {});
        last = 0;
    }

    void extend(char c, int position) {
        int current = (int)state.size();
        state.push_back({-1, state[last].len + 1, position, {}});
        int p = last;
        while (p != -1 && !state[p].next.count(c)) {
            state[p].next[c] = current;
            p = state[p].link;
        }
        if (p == -1) {
            state[current].link = 0;
        } else {
            int q = state[p].next[c];
            if (state[p].len + 1 == state[q].len) {
                state[current].link = q;
            } else {

```



```

        int clone = (int)state.size();
        state.push_back(state[q]);
        state[clone].len = state[p].len + 1;
        while (p != -1 && state[p].next[c] == q) {
            state[p].next[c] = clone;
            p = state[p].link;
        }
        state[q].link = state[current].link = clone;
    }
}
last = current;
}

MultiStringLCSResult solve(const vector<string>& strings) {
    if (strings.empty()) return {};
    int base_index = 0;
    for (int i = 1; i < (int)strings.size(); ++i) {
        if (strings[i].size() < strings[base_index].size()) base_index = i;
    }
    const string& base = strings[base_index];
    if (base.empty()) return {};

    reset();
    for (int i = 0; i < (int)base.size(); ++i) extend(base[i], i);

    int states = (int)state.size();
    vector<int> order(states);
    iota(order.begin(), order.end(), 0);
    sort(order.begin(), order.end(), [&](int a, int b) {
        return state[a].len > state[b].len;
    });

    vector<int> best(states);
    for (int u = 0; u < states; ++u) best[u] = state[u].len;
    for (int index = 0; index < (int)strings.size(); ++index) {
        if (index == base_index) continue;
        vector<int> matched(states);
        int u = 0, length = 0;
        for (char c : strings[index]) {
            while (u && !state[u].next.count(c)) {
                u = state[u].link;
                length = state[u].len;
            }
            auto it = state[u].next.find(c);
            if (it == state[u].next.end()) {
                u = 0;
                length = 0;
            } else {
                u = it->second;
                ++length;
            }
            matched[u] = max(matched[u], length);
        }
        for (int v : order) {
            int p = state[v].link;
            if (p != -1) {
                matched[p] = max(matched[p], min(matched[v], state[p].len));
            }
        }
        for (int state_id = 0; state_id < states; ++state_id) {
            best[state_id] = min(best[state_id], matched[state_id]);
        }
    }

    int chosen = 0;
    for (int u = 1; u < states; ++u) {
        if (best[u] > best[chosen]) chosen = u;
    }
    int length = best[chosen];
    if (length == 0) return {};
    return {length, base.substr(state[chosen].first_end - length + 1, length)};
}
};

```

典题模型：读入 m 个字符串后调用 `MultiStringLongestCommonSubstring solver; auto ans = solver.solve(strings);`；输出 `ans.length` 或 `ans.substring`。一个字符也没有共同出现时答案长度为 0、字符串为空。

SAM：字典序第 k 小本质不同子串

赛时先看

题目信号：题面要求“第 k 小不同子串”“本质不同子串按字典序”“输出第 k 个子串”； k 通常从 1 开始，且不把重复出现的相同字符串重复计数。

本质：按字典序访问所有不同非空子串，求第 k 小；同一条 SAM

路径代表一个不同子串，状态转移按字符有序枚举即可。

接法：给字符串 s 与多次 k 询问，要求输出字典序第 k 小的不同子串。建立 SAM 后只需预处理一次 `auto dp = sam_distinct_substring_path_counts(sam)`，每次查询调用 `sam_kth_distinct_substring(sam, dp, k)`；返回 `nullopt` 时按题意输出 -1。

复杂度判定：从每个状态汇总一次后 $O(|S| \log |S|)$ （这里 `map` 已按字符有序），单次第 k 小构造 O （答案长度 + 经过的转移数）；固定字符集数组转移可降常数。

维护的量：`dp[u]`（从状态 u 出发的非空本质不同子串路径数，上限 `CAP = 4e18` 防溢出）。

警告：本节计的是不同子串，不是按出现次数展开后的第 k 个；每条边的块大小是 $1 + dp[child]$ ，其中 1 对应刚走到该边形成的字符串； k 超过 `dp[root]` 时没有答案。

【最小完整示例（先抄这一段就能跑）】

题目：对字符串 s 输出字典序第 k 小的本质不同子串（ k 从 1 开始）。

```
string s;
i64 k;
cin >> s >> k;
SuffixAutomaton sam((int)s.size()); // 1. 对 s 建 SAM
for (char c : s) sam.extend(c);
auto dp = sam_distinct_substring_path_counts(sam); // 2. 一次预处理
auto ans = sam_kth_distinct_substring(sam, dp, k); // 3. 第 k 小
cout << (ans ? *ans : "-1") << '\n'; // 4. 无解 (k 超范围) 输出 -1
```

样例：`s = "aab", k = 3` -> `"aab"`。

【传参要求（照这个传不会错）】

- `sam_distinct_substring_path_counts(sam)`：在字符全部插入后调用一次；返回 `vector<i64>`，`dp[u]` 为从状态 u 出发的非空不同子串数。
- `sam_kth_distinct_substring(sam, dp, k)`： k 从 1 开始；返回 `optional<string>`，无解（ $k \leq 0$ 或 $k > dp[0]$ ）时返回 `nullopt`（按题意输出 -1）。

```
vector<i64> sam_distinct_substring_path_counts(const SuffixAutomaton& sam) {
    const i64 CAP = 4000000000000000LL; // 只用于约束很大时保护加法不溢出。
    int n = (int)sam.st.size();
    vector<int> order(n);
    iota(order.begin(), order.end(), 0);
    sort(order.begin(), order.end(), [&](int x, int y) {
        return sam.st[x].len > sam.st[y].len;
    });

    vector<i64> dp(n, 0); // 从每个状态出发的非空路径字符串数量。
    for (int u : order) {
        for (auto [c, v] : sam.st[u].next) {
            dp[u] = min(CAP, dp[u] + 1 + dp[v]);
        }
    }
    return dp;
}

optional<string> sam_kth_distinct_substring(
    const SuffixAutomaton& sam, const vector<i64>& dp, i64 k
) {
    if (k <= 0 || k > dp[0]) return nullopt;
    string answer;
    int u = 0;
    while (true) {
        bool moved = false;
        for (auto [c, v] : sam.st[u].next) { // std::map 会保持字典序遍历。
            i64 block_size = 1 + dp[v];
            if (k > block_size) {
                k -= block_size;
                continue;
            }
            answer.push_back(c);
            if (k == 1) return answer;
            --k;
            u = v;
            moved = true;
            break;
        }
        assert(moved); // 前面已经用 dp[0] 检查过 k，因此这里必然能找到转移。
    }
}
```

}

典题模型：给字符串 `s` 与多次 `k` 询问，要求输出字典序第 `k` 小的不同子串。建立 SAM 后只需预处理一次 `auto dp = sam_distinct_substring_path_counts(sam)`，每次查询调用 `sam_kth_distinct_substring(sam, dp, k)`；返回 `nullopt` 时按题意输出 `-1`。

LCS 最长公共子序列

赛时先看

题目信号：两个序列中按原顺序选字符，不要求连续。

本质：求两个字符串/序列的最长公共子序列长度。

接法：如果题面写“删除一些字符后仍保持相对顺序”，就是子序列，可以用 LCS；如果写“连续片段/子串”，不要用这个模板。`n*m` 太大时不能硬跑二维 DP，改翻 bit-parallel LCS、SAM 或哈希模型。

复杂度判定： $O(nm)$ 。

维护的量：`dp[i][j]`（`a` 前 `i` 个与 `b` 前 `j` 个字符的最长公共子序列长度，`i` 行 `j` 列）。

警告：LCS 是子序列，不是子串；子串要连续。

【最小完整示例（先抄这一段就能跑）】

题目：求字符串 `a` 与 `b` 的最长公共子序列长度（可跳字符）。

```
string a, b;
cin >> a >> b;
cout << lcs_length(a, b) << '\n'; // 直接输出长度
```

样例：`a = "abcde", b = "ace" → 3`。

【传参要求（照这个传不会错）】

- `lcs_length(a, b)`：两个参数都按原串传（0-based，内部按字符比较）；返回最长公共子序列长度；任一为空串时返回 0。

```
int lcs_length(const string& a, const string& b) {
    int n = (int)a.size(), m = (int)b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            if (a[i - 1] == b[j - 1]) dp[i][j] = dp[i - 1][j - 1] + 1;
            else dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
        }
    }
    return dp[n][m];
}
```

LCS 输出方案

赛时先看

题目信号：题目要求构造公共子序列。

本质：不仅要长度，还要输出一个最长公共子序列。

接法：先和普通 LCS 一样填表，再从右下角倒着走。字符相同就选这个字符并同时左上走；不相同就走向 `dp` 值更大的邻格。最后反转得到答案。

复杂度判定： $O(nm)$ 。

维护的量：`dp[i][j]`（长度表，填法与普通 LCS 相同）；回溯指针 `i/j` 从右下角倒走。

警告：回溯时从 `dp[n][m]` 往回走。

【最小完整示例（先抄这一段就能跑）】

题目：输出 `a` 与 `b` 的一个最长公共子序列字符串。

```
string a, b;
cin >> a >> b;
cout << lcs_string(a, b) << '\n'; // 输出任意一条最长公共子序列
```

样例：`a = "abcde", b = "ace" → "ace"`。

【传参要求（照这个传不会错）】

- `lcs_string(a, b)`：按原串传（0-based）；返回 `string`，是 `a`、`b` 共同的一个最长公共子序列（有多个时返回回溯得到的其一）；无公共字符时返回空串。

```

string lcs_string(const string& a, const string& b) {
    int n = (int)a.size(), m = (int)b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            if (a[i - 1] == b[j - 1]) dp[i][j] = dp[i - 1][j - 1] + 1;
            else dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
        }
    }
    string res;
    int i = n, j = m;
    while (i > 0 && j > 0) {
        if (a[i - 1] == b[j - 1]) {
            res.push_back(a[i - 1]);
            i--, j--;
        } else if (dp[i - 1][j] >= dp[i][j - 1]) {
            i--;
        } else {
            j--;
        }
    }
    reverse(res.begin(), res.end());
    return res;
}

```

最短公共超序列 SCS：合并两条保持顺序的序列

赛时先看

题目信号：同时满足两条操作/事件序列的相对顺序；允许插入，但不允许打乱任一原序列。若只要求长度，答案是 $|a| + |b| - \text{LCS}(a, b)$ ；若要求构造，直接回溯 DP 表。

本质：构造一个最短字符串，使两个原串都是它的子序列。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定：时间和空间都是 $O(|a||b|)$ 。

维护的量：`dp[i][j]`（a 前 i 个与 b 前 j 个的最短公共超序列长度，边界 `dp[i][0]=i`、`dp[0][j]=j`）。

警告：SCS

是“子序列”而非“公共子串”。两个字符相等时只追加一次；回溯到某串耗尽后，要把另一串剩余部分全部接上。

【最小完整示例（先抄这一段就能跑）】

题目：求最短字符串使 a、b 都是它的子序列（顺序不变，允许插入）。

```

string a, b;
cin >> a >> b;
cout << shortest_common_supersequence(a, b) << '\n'; // 输出最短超序列

```

样例：a = "abac", b = "cab" -> "cabac"（长度 5）。

【传参要求（照这个传不会错）】

- `shortest_common_supersequence(a, b)`：按原串传（0-based）；返回 `string`，是包含 a、b 的最短公共超序列；长度恒为 $|a| + |b| - \text{LCS}(a, b)$ 。

```

string shortest_common_supersequence(const string& a, const string& b) {
    int n = (int)a.size(), m = (int)b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1));
    for (int i = 0; i <= n; ++i) dp[i][0] = i;
    for (int j = 0; j <= m; ++j) dp[0][j] = j;

    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            if (a[i - 1] == b[j - 1]) dp[i][j] = dp[i - 1][j - 1] + 1;
            else dp[i][j] = min(dp[i - 1][j], dp[i][j - 1]) + 1;
        }
    }

    string answer;
    for (int i = n, j = m; i > 0 || j > 0;) {
        if (i == 0) answer.push_back(b[--j]);
        else if (j == 0) answer.push_back(a[--i]);
        else if (a[i - 1] == b[j - 1]) {
            answer.push_back(a[--i]);
            --j;
        } else if (dp[i - 1][j] <= dp[i][j - 1]) {
            answer.push_back(a[--i]);
        } else {
            answer.push_back(b[--j]);
        }
    }
    reverse(answer.begin(), answer.end());
    return answer;
}

```

```

    }
}
reverse(answer.begin(), answer.end());
return answer;
}

```

Bit-parallel LCS: 只求长度

赛时先看

题目信号: 明确问 LCS 长度; 字符集小或可压成字节; 长度达到数万到十万级, 二维 DP 内存或时间都不能接受。

本质: 两串很长, 普通 $O(nm)$ LCS DP 超时, 但只需要最长公共子序列长度, 不需要还原具体方案。

接法: 给两条 DNA/日志/字符串, $|A|, |B|$ 都到 $5e4$, 只问最长公共子序列长度。普通 DP 约需 $2.5e9$ 次转移, 而此模板约为它的 $1/64$; 若题目要求输出方案, 回到本册已有的普通 LCS/Hirschberg 类做法。

复杂度判定: $O(|A||B| / 64 + 256 * \min(|A|, |B|) / 64)$, 空间 $O(256 * \min(|A|, |B|) / 64)$ 。实现通过 64 位字并行模拟一整列 DP。

维护的量: `match[ch]` (字符 `ch` 在短串 `a` 中出现位置的位向量, 按 64 位字存); `state` (当前列 DP 值的位向量数组)。

警告: 只能求长度, 不能直接恢复 LCS; `unsigned char` 索引避免非 ASCII 字符变负数; 若字符是整数数组, 可把 `array<..., 256>` 换成 `unordered_map<值, bit-vector>`。

【最小完整示例 (先抄这一段就能跑)】

题目: 两条长串 `a, b` (长度可到 $5e4$), 只求最长公共子序列长度。

```

string a, b;
cin >> a >> b;
cout << lcs_bitparallel_length(a, b) << '\n'; // 约  $O(|a||b|/64)$ 

```

样例: `a = "abcdbdab", b = "bdcaba" -> 4`。

【传参要求 (照这个传不会错)】

- `lcs_bitparallel_length(a, b)`: 按原串传值 (内部自动把较短串换到 `a` 当位下标); 返回 LCS 长度; 字符按 `unsigned char` 索引, 非 ASCII 也安全; 只求长度, 不能恢复方案。

```

int lcs_bitparallel_length(string a, string b) {
    if (a.size() > b.size()) swap(a, b); // 把较短字符串作为 bitset 的位下标, 节省空间。
    int m = (int)a.size();
    if (m == 0) return 0;
    int words = (m + 63) >> 6;

    array<vector<ui64>, 256> match;
    for (auto& bits : match) bits.assign(words, 0);
    for (int i = 0; i < m; ++i) {
        match[(unsigned char)a[i]][i >> 6] |= 1ULL << (i & 63);
    }

    vector<ui64> state(words, 0);
    for (unsigned char ch : b) {
        ui64 shift_carry = 1; // 状态转移写法: (state << 1) | 1。
        ui64 borrow = 0;
        for (int w = 0; w < words; ++w) {
            ui64 x = state[w] | match[ch][w];
            ui64 y = (state[w] << 1) | shift_carry;
            shift_carry = state[w] >> 63;

            ui64 y_with_borrow = y + borrow;
            bool overflow = y_with_borrow < y;
            ui64 diff = x - y_with_borrow;
            borrow = overflow || x < y_with_borrow;
            state[w] = x & ~diff;
        }
    }

    int answer = 0;
    for (auto bits : state) answer += __builtin_popcountll(bits);
    return answer;
}

```

典型模型: 给两条 DNA/日志/字符串, $|A|, |B|$ 都到 $5e4$, 只问最长公共子序列长度。普通 DP 约需 $2.5e9$ 次转移, 而此模板约为它的 $1/64$; 若题目要求输出方案, 回到本册已有的普通 LCS/Hirschberg 类做法。

编辑距离 Levenshtein: 最少插入、删除、替换

赛时先看

题目信号：两串相互转换、拼写纠错、最少修改字符使两串相等；三个操作的代价均为 1。

本质：把字符串 a 变成 b 的最少操作次数，每步可插入一个字符、删除一个字符或替换一个字符。

接法：问把 s 改为 t 的最少编辑次数，直接输出 `levenshtein_distance(s,t)`。若只允许插入删除，答案才是 $|s|+|t|-2*LCS(s,t)$ 。

复杂度判定： $O(|a||b|)$ 时间， $O(\min(|a|,|b|))$ 空间。

维护的量： $dp[j]$ （当前行： a 前缀变到 $b[0..j]$ 的最少代价）；`diagonal`（滚动数组中左上角 $dp[j-1]$ 的旧值）。

警告：编辑距离允许替换，不能用 $|a|+|b|-2*LCS$ 代替（后者只允许插删）；若三种操作代价不同，应改成带权转移；本版本只返回最小代价，恢复操作序列需要保留完整二维 dp 。

【最小完整示例（先抄这一段就能跑）】

题目：把串 s 改成 t 的最少编辑次数（插入、删除、替换各算 1 次）。

```
string s, t;
cin >> s >> t;
cout << levenshtein_distance(s, t) << '\n'; // 最少编辑次数
```

样例： $s = \text{"horse"}, t = \text{"ros"} \rightarrow 3$ 。

【传参要求（照这个传不会错）】

- `levenshtein_distance(a, b)`：按原串传值（内部可能交换 a/b 以压缩空间）；返回把 a 变成 b 的最少插入/删除/替换次数，三种操作代价均为 1。

【改板时先认这几个量】

- up ：上一行的 $dp[j]$ （滚动数组里左上对角的值）。
- dp ：DP 状态。

状态： $dp[j]$ 是处理完 a 的当前前缀后，变成 $b[0..j]$ 的最少代价。转移来自删除、插入、替换/保留三个方向。

```
int levenshtein_distance(string a, string b) {
    if (a.size() < b.size()) swap(a, b); // b 是较短串，压缩空间。
    int n = (int)a.size(), m = (int)b.size();
    vector<int> dp(m + 1);
    iota(dp.begin(), dp.end(), 0);
    for (int i = 1; i <= n; ++i) {
        int diagonal = dp[0]; // 上一行的 dp[j-1]
        dp[0] = i;
        for (int j = 1; j <= m; ++j) {
            int up = dp[j];
            if (a[i - 1] == b[j - 1]) {
                dp[j] = diagonal;
            } else {
                dp[j] = 1 + min({diagonal, up, dp[j - 1]});
            }
            diagonal = up;
        }
    }
    return dp[m];
}
```

典题模型：问把 s 改为 t 的最少编辑次数，直接输出 `levenshtein_distance(s,t)`。若只允许插入删除，答案才是 $|s|+|t|-2*LCS(s,t)$ 。

最长公共上升子序列 LCIS

赛时先看

题目信号：两个序列、元素值需要相同、相对顺序要保留，并且选出的公共序列还要严格递增；通常 $n, m \leq 3000$ 左右。

本质：求两个整数序列共同拥有、且严格递增的最长子序列；比“分别求 LIS 再取交”严格得多。

接法：给数组 a, b ，输出共同严格递增子序列最大长度或一条方案。调用 `auto`

`ans=longest_common_increasing_subsequence(a,b);`，长度为 `ans.length`，方案为 `ans.sequence`。

复杂度判定： $O(nm)$ 时间， $O(m)$ 空间。

维护的量： $dp[j]$ （以 $b[j]$ 结尾的 LCIS

最大长度）；`last_node[j]`（该长度对应的回溯链节点）；`trace`（回溯链，记录取值与前驱）。

警告：`current` 只能由 $b[j] < a[i]$ 更新，等于时用于接上；循环内不要把同一轮刚更新的相等状态拿来继续转移；重复数值时返回任意一条最优严格递增方案即可。

【最小完整示例（先抄这一段就能跑）】

题目：两个整数数组 a, b ，求共同且严格递增的最长子序列的长度与一条方案。

```
vector<int> a, b;
// 读入 a、b ...
auto ans = longest_common_increasing_subsequence(a, b);
cout << ans.length << '\n'; // 最大长度
for (int x : ans.sequence) cout << x << ' '; // 一条方案 (严格递增)
```

样例: $a = \{2, 3, 1, 4, 5\}$, $b = \{1, 2, 3, 4\}$ $\rightarrow$ 长度 3, 方案 2 3 4。

【传参要求 (照这个传不会错)】

- `longest_common_increasing_subsequence(a, b)`: 两个 `vector<int>` 按 0-based 原顺序传入; 返回 `LCISResult`: `length` 为最大长度, `sequence` 为一条最优严格递增公共子序列 (长度为 0 时 `sequence` 为空)。

状态: 扫描 $a[i]$ 时, $dp[j]$ 表示以 $b[j]$ 结尾的 LCIS 最大长度。遍历 b , 维护所有 $b[j] < a[i]$ 的最佳长度与位置; 遇到 $a[i] == b[j]$ 时接上它。

```
struct LCISResult {
    int length = 0;
    vector<int> sequence;
};

LCISResult longest_common_increasing_subsequence(
    const vector<int>& a, const vector<int>& b
) {
    int m = (int)b.size();
    vector<int> dp(m, last_node(m, -1));
    struct TraceNode { int value, previous; };
    vector<TraceNode> trace;
    for (int x : a) {
        int best_length = 0, best_node = -1;
        for (int j = 0; j < m; ++j) {
            if (x == b[j] && best_length + 1 > dp[j]) {
                dp[j] = best_length + 1;
                trace.push_back({x, best_node});
                last_node[j] = (int)trace.size() - 1;
            }
            if (b[j] < x && dp[j] > best_length) {
                best_length = dp[j];
                best_node = last_node[j];
            }
        }
        int end_pos = -1;
        for (int j = 0; j < m; ++j) {
            if (end_pos == -1 || dp[j] > dp[end_pos]) end_pos = j;
        }
        LCISResult result;
        result.length = (end_pos == -1 ? 0 : dp[end_pos]);
        for (int node = (end_pos == -1 ? -1 : last_node[end_pos]);
            node != -1; node = trace[node].previous) {
            result.sequence.push_back(trace[node].value);
        }
        reverse(result.sequence.begin(), result.sequence.end());
        return result;
    }
}
```

典型模型: 给数组 a, b , 输出共同严格递增子序列最大长度或一条方案。调用 `auto ans = longest_common_increasing_subsequence(a, b);`, 长度为 `ans.length`, 方案为 `ans.sequence`。

最长回文子序列与最少插入成回文

赛时先看

题目信号: 不要求连续地选字符, 要求前后对称; 题目问“最多保留多少形成回文”或“最少再插入多少”。注意它不同于 Manacher 的连续回文子串。

本质: 允许删除字符后的最长回文长度, 或求最少插入多少字符让整个字符串变成回文。

接法: 字符串长度 $n \leq 5000$, 可删除任意字符, 求最长回文子序列; 调用第一函数。若每次可以在任意位置插入字符, 最少插入数直接调用第二函数。

复杂度判定: $O(n^2)$ 时间, $O(n)$ 空间。

维护的量: $dp[j]$ (当前外层 i 下区间 $s[i..j]$ 的最长回文子序列长度); `previous_diagonal` (旧的 $dp[j-1]$, 对角线值)。

警告: `previous_diagonal` 必须保存旧的 $dp[j-1]$; 空串答案是 0; 最少插入数恰好是 $n - \text{LPS}$, 但“最少删除成回文”也是同一个数。

【最小完整示例 (先抄这一段就能跑)】

题目：求串 `s` 的最长回文子序列长度，以及最少插入多少字符使 `s` 成为回文。

```
string s;
cin >> s;
cout << longest_palindromic_subsequence_length(s) << '\n'; // 最长回文子序列
cout << minimum_insertions_to_palindrome(s) << '\n'; // 最少插入数
```

样例：`s = "bbbab"` -> 4 和 1。

【传参要求（照这个传不会错）】

- `longest_palindromic_subsequence_length(s)`：按原串传（0-based）；返回最长回文子序列长度；空串返回 0。
- `minimum_insertions_to_palindrome(s)`：按原串传；返回最少插入字符数，恒等于 `n - 最长回文子序列长度`。

状态：区间 `s[i..j]` 的最长回文子序列。相等时两端都取，否则丢掉一端。压缩后 `dp[j]` 对应当前 `i` 下的区间答案。

```
int longest_palindromic_subsequence_length(const string& s) {
    int n = (int)s.size();
    vector<int> dp(n);
    for (int i = n - 1; i >= 0; --i) {
        dp[i] = 1;
        int previous_diagonal = 0;
        for (int j = i + 1; j < n; ++j) {
            int old_dp_j = dp[j];
            if (s[i] == s[j]) dp[j] = previous_diagonal + 2;
            else dp[j] = max(dp[j], dp[j - 1]);
            previous_diagonal = old_dp_j;
        }
    }
    return n == 0 ? 0 : dp[n - 1];
}

int minimum_insertions_to_palindrome(const string& s) {
    return (int)s.size() - longest_palindromic_subsequence_length(s);
}
```

典型模型：字符串长度 `n <= 5000`，可删除任意字符，求最长回文子序列；调用第一函数。若每次可以在任意位置插入字符，最少插入数直接调用第二函数。

通配符匹配：? 匹配一个，* 匹配任意串

赛时先看

题目信号：题面显式给 `?`、`*` 或“星号可代替任意多个字符”；要求整串匹配，不是只找一个子串。

本质：判断文本能否被含通配符的模式串完整匹配，`?` 匹配任意一个字符，`*` 匹配任意长度（含空）字符串。

接法：文件名匹配、DNA 模板匹配、含 `*` 和 `?` 的模式能否匹配全文。直接调用

`wildcard_full_match(text, pattern)`；若 `pattern` 很长且只含少量 `*`，可按星号分段后用 KMP/哈希进一步优化。

复杂度判定： $O(|text| |pattern|)$ 时间， $O(|pattern|)$ 空间。

维护的量：`dp[j]`（当前文本前缀能否匹配模式前 `j` 个字符）；`next_dp`（下一行滚动数组）。

警告：初始化时只有连续前导 `*` 可以匹配空串；`*`

既可空也可多字符，两个转移都不能漏；本模板是全串匹配，若题目问子串匹配要在模式两端补 `*` 或改状态定义。

【最小完整示例（先抄这一段就能跑）】

题目：判断含 `?`、`*` 的模式串 `pattern` 能否完整匹配文本 `text`。

```
string text, pattern;
cin >> text >> pattern;
cout << (wildcard_full_match(text, pattern) ? "YES" : "NO") << '\n'; // 整串匹配
```

样例：`text = "adceb"`，`pattern = "*a*b"` -> YES。

【传参要求（照这个传不会错）】

- `wildcard_full_match(text, pattern)`：按原串传（0-based）；要求整串匹配；`?` 匹配任意一个字符、`*` 匹配任意长度（含空）字符串；返回 `bool`，完全匹配为 `true`。

状态：`dp[j]` 表示当前处理的文本前缀，能否匹配模式前 `j` 个字符。遇到 `*`

可选择匹配空串（`next[j-1]`）或吞掉当前文本字符（旧 `dp[j]`）。

```
bool wildcard_full_match(const string& text, const string& pattern) {
    int m = (int)pattern.size();
    vector<char> dp(m + 1);
    dp[0] = true;
```

```

for (int j = 1; j <= m && pattern[j - 1] == '*'; ++j) dp[j] = true;

for (char ch : text) {
    vector<char> next_dp(m + 1);
    for (int j = 1; j <= m; ++j) {
        if (pattern[j - 1] == '*') {
            next_dp[j] = next_dp[j - 1] || dp[j];
        } else if (pattern[j - 1] == '?' || pattern[j - 1] == ch) {
            next_dp[j] = dp[j - 1];
        }
    }
    dp.swap(next_dp);
}
return dp[m];
}

```

典型模型：文件名匹配、DNA 模板匹配、含 * 和 ? 的模式能否匹配全文。直接调用

wildcard_full_match(text, pattern); 若 pattern 很长且只含少量 *, 可按星号分段后用 KMP/哈希进一步优化。

最小表示法 Booth

赛时先看

题目信号：环形字符串、旋转同构、最小循环表示。

本质：求一个循环字符串中字典序最小的旋转。

接法：把本节 struct/class/函数组 整段抄到 solve() 上面；solve()

里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O(n)$ 。

维护的量：t (双倍串 s+s)；i/j (两个候选起点)；k (当前已比较相等的长度)；答案 min(i, j) 即最小旋转起点。

警告：返回的是起始下标，最小旋转串为 s.substr(pos)+s.substr(0,pos)。

【最小完整示例（先抄这一段就能跑）】

题目：求字符串 s 的所有循环旋转中字典序最小的那个，输出其 0-based 起点（可选还原旋转串）。

```

string s;
cin >> s;
int pos = min_rotation(s); // 1. 调用：最小旋转的起始下标 (0-based)
string best = s.substr(pos) + s.substr(0, pos); // 2. 还原最小旋转串 (只要下标就省掉这行)
cout << pos << '\n'; // 3. 输出起点：要串就输出 best

```

样例：s = "bcab" -> 起点 2，最小旋转串 "abbc"。

【传参要求（照这个传不会错）】

- min_rotation(s): s 任意字符串 (0-based)；返回 int，字典序最小旋转的起始下标 (0-based)。
- 最小旋转串 = s.substr(pos) + s.substr(0, pos)；并列最小时返回最小的那个下标。
- 空串或全相同字符时返回 0。

```

int min_rotation(const string& s) {
    string t = s + s;
    int n = (int)s.size();
    int i = 0, j = 1, k = 0;
    while (i < n && j < n && k < n) {
        if (t[i + k] == t[j + k]) {
            k++;
        } else {
            if (t[i + k] > t[j + k]) i = i + k + 1;
            else j = j + k + 1;
            if (i == j) j++;
            k = 0;
        }
    }
    return min(i, j);
}

```

整数序列最小循环表示 Booth

赛时先看

题目信号：循环移位、环状序列、需要最小表示；元素不是字符而是整数/结构体。

本质：求一个整数序列所有循环移位中字典序最小的起点。

接法：把本节 struct/class/函数组 整段抄到 solve() 上面；solve()

里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O(n)$ 。

维护的量： i/j （两个候选循环起点）； k （当前已比较相等的长度）；比较用取模下标 $(i+k) \% n$ ；答案 $\min(i, j) \% n$ 为最小起点。

警告：如果所有元素相同，返回 0 ；比较时取模下标。

【最小完整示例（先抄这一段就能跑）】

题目：求整数序列所有循环移位中字典序最小的表示，输出起点，可选拿回整个最小序列。

```
vector<int> a = {3, 1, 2, 1};
int p = min_rotation_index(a);           // 1. 调用：最小循环表示的起点 (0-based)
auto seq = min_rotation_sequence(a);     // 2. 可选：直接得到最小循环表示序列
cout << p << '\n';
for (int x : seq) cout << x << ' ';     // 3. 输出：起点 / 最小序列
```

样例： $a = \{3, 1, 2, 1\} \rightarrow$ 起点 1 ，最小序列 $1\ 2\ 1\ 3$ 。

【传参要求（照这个传不会错）】

- `min_rotation_index(s)`： s 元素需支持 `==` 与 `>`（`int/long long/char` 均可）；0-based；返回 `int` 最小循环表示起点（0-based）；空向量或全相同元素返回 0 。
- `min_rotation_sequence(s)`：返回 `vector<T>`，从最小起点开始的整个环，长度与 s 相同；只输出起点时不调它。
- 比较用 $(i + k) \% n$ 取模下标，等价于环上逐位比较。

```
template <class T>
int min_rotation_index(const vector<T>& s) {
    int n = (int)s.size();
    if (n == 0) return 0;
    int i = 0, j = 1, k = 0;
    while (i < n && j < n && k < n) {
        const T& a = s[(i + k) % n];
        const T& b = s[(j + k) % n];
        if (a == b) {
            k++;
        } else if (a > b) {
            i = i + k + 1;
            if (i <= j) i = j + 1;
            k = 0;
        } else {
            j = j + k + 1;
            if (j <= i) j = i + 1;
            k = 0;
        }
    }
    return min(i, j) % n;
}

template <class T>
vector<T> min_rotation_sequence(const vector<T>& s) {
    int p = min_rotation_index(s);
    vector<T> res;
    for (int i = 0; i < (int)s.size(); i++) res.push_back(s[(p + i) % s.size()]);
    return res;
}
```

Lyndon 分解 Duval

赛时先看

题目信号：题面出现 Lyndon、字典序分解、最小循环位移的扩展。

本质：把字符串唯一分解成非增 Lyndon 串，常用于最小表示、字符串周期分析。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；`solve()` 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O(n)$ 。

维护的量： i （当前段起点）； j （段探测终点）； k （段内周期比较位置）；`res`（Lyndon 段列表，每段为半开区间 $[l, r)$ ）。

警告：返回的是每段 $[l, r)$ ；比较时下标推进规则不要写反。

【最小完整示例（先抄这一段就能跑）】

题目：把字符串 s 唯一分解成若干 Lyndon 串，输出每段内容。

```
string s;
cin >> s;
auto parts = duval(s);           // 1. 调用：Lyndon 分解的段区间列表
for (auto [l, r] : parts)
    cout << s.substr(l, r - l) << ' '; // 2. 输出每段（区间是半开的 [l, r)）
```

```
cout << '\n';
```

样例: `s = "banana"` → 分段 `[0,1)` `[1,3)` `[3,5)` `[5,6)`, 即 `b an an a`。

【传参要求（照这个传不会错）】

- `duval(s)`: `s` 原串 (0-based); 返回 `vector<pair<int,int>>`, 每段是 半开区间 `[l, r)` (0-based, 右端点取不到)。
- 各段按原串顺序排列、互不重叠, 拼接起来恰好是整个 `s`; 每段本身是 Lyndon 串且段间字典序不增。
- 取段内容用 `s.substr(l, r - l)`。

```
vector<pair<int, int>> duval(const string& s) {
    int n = (int)s.size();
    vector<pair<int, int>> res;
    int i = 0;
    while (i < n) {
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j]) k = i;
            else k++;
            j++;
        }
        while (i <= k) {
            res.push_back({i, i + j - k});
            i += j - k;
        }
    }
    return res;
}
```

不重叠子序列匹配次数

赛时先看

题目信号: 关键词按顺序出现但不要求连续; 可以选择多个且位置不能重叠; 模式固定且较短。

本质: 给定模式序列, 统计文本中最多能选出多少个互不重叠、等于模式的子序列。

复杂度判定: $O(n)$ 。

维护的量: `p` (模式串已匹配到第几个字符); `ans` (已完成的不重叠匹配次数); 匹配完一个模式立即从头开始。

警告: 每次尽早完成一个模式一定最优, 这是区间调度的“最早结束”贪心。

【最小完整示例（先抄这一段就能跑）】

题目: 在文本序列中统计最多能选出多少个互不重叠的、等于模式 `pattern` 的子序列。

```
vector<int> text = {1, 2, 1, 3, 2, 3};
vector<int> pattern = {1, 2, 3};
i64 cnt = count_disjoint_subsequence_matches(text, pattern); // 1. 调用: 最多不重叠次数
cout << cnt << '\n'; // 2. 输出次数
```

样例: `text = {1,2,1,3,2,3}`, `pattern = {1,2,3}` → 2 (分别用下标 0,1,3 和 2,4,5)。

【传参要求（照这个传不会错）】

- `text`: 文本序列 (0-based; 元素 `T` 需支持 `==`, `int/long long/char` 均可)。
- `pattern`: 模式序列 (0-based); `pattern` 为空时返回 0。
- 返回值: `i64`, 最多能选出的互不重叠的完整模式子序列个数; 每次匹配完从头再找。
- 贪心正确性: 每次尽早匹配完一个模式, 剩余文本最长, 一定不劣。

```
template <class T>
i64 count_disjoint_subsequence_matches(
    const vector<T>& text,
    const vector<T>& pattern
) {
    if (pattern.empty()) return 0;
    int p = 0;
    i64 ans = 0;
    for (const T& x : text) {
        if (x == pattern[p]) {
            p++;
            if (p == (int)pattern.size()) {
                ans++;
                p = 0;
            }
        }
    }
    return ans;
}
```

字典序最小受限子序列

赛时先看

题目信号：删掉若干字符得到车牌/编号；有若干规则禁止 `ab` 相邻；要求字典序最小可行解。

本质：从原串中选长度为 `need` 的子序列，同时禁止某些相邻字符对，要求字典序最小。

复杂度判定： $O(26 * n + 26 * need)$ 。

维护的量：`bad[a][b]`（禁止 `a` 后紧跟 `b`）；`next_pos[i][c]`（位置 `i` 起第一个字符 `c` 的下标）；`dp[i][last]`（位置 `i` 起、上一位为 `last` 时能延长的最长合法子序列长度，`last=26` 表示还没选）。

警告：贪心选当前字符时，要用后缀 DP 判断是否还能补齐；`last=26` 表示还没选字符。

【最小完整示例（先抄这一段就能跑）】

题目：从 `s` 中选长度为 `need` 的子序列，选中序列里不允许出现若干指定相邻字符对，求字典序最小者。

```
string s = "abcabc";
int need = 3;
vector<pair<char, char>> forb = {{'b', 'a'}}; // 禁止 "ba" 相邻
string ans = min_lex_subsequence_with_forbidden_pairs(s, need, forb); // 1. 调用
cout << ans << '\n'; // 2. 输出答案；-1 表示凑不够
```

样例：`s = "abcabc"`，`need = 3`，禁对 `{'b', 'a'}` → `"aab"`。

【传参要求（照这个传不会错）】

- `s`：原串（0-based，小写字母）。
- `need`：要求的子序列长度（正整数）；凑不够时返回 `"-1"`。
- `forbidden_pairs`：禁止相邻对列表，每对 `{a, b}` 表示选中的子序列中不允许 `a` 后紧跟 `b`（`a`、`b` 均为小写字母，内部按 `bad[a-'a'][b-'a']` 标记，重复传无影响）。
- 返回值：`string`，字典序最小的长度为 `need` 的合法子序列；不存在返回 `"-1"`。

```
string min_lex_subsequence_with_forbidden_pairs(
    const string& s,
    int need,
    const vector<pair<char, char>>& forbidden_pairs
) {
    int n = (int)s.size();
    const int NONE = 26;
    bool bad[27][26] = {};
    for (auto [a, b] : forbidden_pairs) bad[a - 'a'][b - 'a'] = true;

    vector next_pos(n + 2, array<int, 26>{});
    for (int c = 0; c < 26; c++) next_pos[n][c] = next_pos[n + 1][c] = n;
    for (int i = n - 1; i >= 0; i--) {
        next_pos[i] = next_pos[i + 1];
        next_pos[i][s[i] - 'a'] = i;
    }

    vector dp(n + 1, array<int, 27>{});
    for (int last = 0; last <= 26; last++) dp[n][last] = 0;
    for (int i = n - 1; i >= 0; i--) {
        int c = s[i] - 'a';
        for (int last = 0; last <= 26; last++) {
            dp[i][last] = dp[i + 1][last];
            if (last == NONE || !bad[last][c]) {
                dp[i][last] = max(dp[i][last], 1 + dp[i + 1][c]);
            }
        }
    }
    if (dp[0][NONE] < need) return "-1";

    string ans;
    int pos = 0, last = NONE, rem = need;
    while (rem > 0) {
        bool picked = false;
        for (int c = 0; c < 26; c++) {
            if (last != NONE && bad[last][c]) continue;
            int j = next_pos[pos][c];
            if (j < n && dp[j + 1][c] >= rem - 1) {
                ans.push_back(char('a' + c));
                pos = j + 1;
                last = c;
                rem--;
                picked = true;
                break;
            }
        }
    }
}
```

```

    if (!picked) return "-1";
}
return ans;
}

```

括号序列合法性与最长合法括号

赛时先看

题目信号：只含 (和)，问是否合法或最长合法段。

本质：括号匹配、最长合法括号子串。

接法：把本节 struct/class/函数组 整段抄到 solve() 上面；solve() 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O(n)$ 。

维护的量：bal（当前未匹配的 (数）；st（下标栈，栈底始终是最后一个非法位置，初始 -1）；ans（最长合法段长度）。

警告：最长合法括号用栈存下标，栈底放最后一个非法位置。

【最小完整示例（先抄这一段就能跑）】

题目：判断括号串是否合法，并求最长合法连续子串长度。

```

string s;
cin >> s;
bool ok = valid_parentheses(s); // 1. 调用：是否为合法括号序列
int best = longest_valid_parentheses(s); // 2. 调用：最长合法连续子串长度
cout << (ok ? "yes" : "no") << ' ' << best << '\n';

```

样例：s = ")()())" -> no，最长合法段长度 4。

【传参要求（照这个传不会错）】

- valid_parentheses(s)：s 只含 (与)（有其他字符需先剥掉）；返回 bool，任意前缀中) 不超过 (且总数相等即合法。
- longest_valid_parentheses(s)：返回 int，最长连续合法括号子串长度；没有合法段时返回 0。
- 实现约定：栈存下标，栈底 -1 为虚拟位置；右括号弹栈后若栈空，把当前位置入栈作为新的“最后一个非法位置”。

```

bool valid_parentheses(const string& s) {
    int bal = 0;
    for (char c : s) {
        if (c == '(') bal++;
        else bal--;
        if (bal < 0) return false;
    }
    return bal == 0;
}

int longest_valid_parentheses(const string& s) {
    vector<int> st;
    st.push_back(-1);
    int ans = 0;
    for (int i = 0; i < (int)s.size(); ++i) {
        if (s[i] == '(') st.push_back(i);
        else {
            st.pop_back();
            if (st.empty()) st.push_back(i);
            else ans = max(ans, i - st.back());
        }
    }
    return ans;
}

```

E 数学、数论、几何与多项式

10 数论、组合计数与同余

模数工具、质数筛、因数分解、反演、组合计数、CRT、离散对数、原根、二次剩余和随机分解集中放在这里。

本册通用辅助函数

赛时先看

题目信号：复制本册数学模板时，先带上这一小块。

本质：供本册原根、exBSGS、Lagrange、FWHT、BM 使用。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接调用本节列出的函数即可。

复杂度判定：快速幂 $O(\log \text{mod})$ ，扩欧 $O(\log \text{mod})$ 。

维护的量：无状态纯函数：`mod_pow`（快速幂）、`exgcd`（扩展欧几里得）、`inv_general`（任意模数逆元），只依赖入参。

警告：`inv_general` 在逆元不存在时返回 `-1`。

【最小完整示例（先抄这一段就能跑）】

题目：求 3 在模 7 下的逆元，并算 $2^{10} \bmod 1000$ 。

```
i64 x, y;
i64 g = exgcd(30, 12, x, y); // 1. 求 30x+12y=gcd(30,12)=6 的一组解 (x,y 引用输出)
i64 inv = inv_general(3, 7); // 2. 求 3 在模 7 下的逆元
i64 pw = mod_pow(2, 10, 1000); // 3. 快速幂: 2^10 % 1000
cout << g << ' ' << x << ' ' << y << ' ' << inv << ' ' << pw << '\n';
```

样例：`g=6, x=1, y=-2; inv=5` ($3*5 \equiv 1 \pmod{7}$)；`pw=24` ($2^{10} = 1024 \pmod{1000}$)。

【传参要求（照这个传不会错）】

- `mod_pow(a, e, mod)`: `a` 底数、`e` 指数（i64，可到 $1e18$ ）、`mod` 模数；返回 $a^e \bmod \text{mod}$ ，范围 $[0, \text{mod})$ ；`mod=1` 时返回 0。
- `exgcd(a, b, x, y)`: `a/b` 为任意 i64；引用参数 `x/y` 输出一组解，满足 $ax+by=g$ ；返回 `gcd(a,b)`（非负）。
- `inv_general(a, mod)`: 返回 `a` 在模 `mod` 下的逆元（范围 $[0, \text{mod})$ ）；要求 $\text{gcd}(a, \text{mod})=1$ ，否则返回 `-1`。

```
i64 mod_pow(i64 a, i64 e, i64 mod) {
    i64 r = 1 % mod;
    a %= mod;
    while (e) {
        if (e & 1) r = (i128)r * a % mod;
        a = (i128)a * a % mod;
        e >>= 1;
    }
    return r;
}

i64 exgcd(i64 a, i64 b, i64& x, i64& y) {
    if (b == 0) {
        x = (a >= 0 ? 1 : -1);
        y = 0;
        return llabs(a);
    }
    i64 x1, y1;
    i64 g = exgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - a / b * y1;
    return g;
}

i64 inv_general(i64 a, i64 mod) {
    i64 x, y;
    i64 g = exgcd(a, mod, x, y);
    if (g != 1) return -1;
    x %= mod;
    if (x < 0) x += mod;
    return x;
}
```

ModInt

赛时先看

题目信号：整题都在固定模数下计算。

本质：封装模意义加减乘除。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定： $O(1)$ ；除法需要快速幂。

维护的量：`v`（模意义下的当前值，int，恒在 $[0, \text{MOD})$ ）。

警告：模数非质数时不能用费马逆元。

【最小完整示例（先抄这一段就能跑）】

题目：模 $1e9+7$ 下计算 $a+b$ $a*b$ a/b 并输出 ($a=3, b=5$)。

```
using mint = ModInt<1000000007>; // 1. 模板参数直接给模数（编译期常量）
mint a = 3, b = 5;
mint c = a + b, d = a * b, e = a / b; // 2. 加减乘除全自动取模
cout << c.v << ' ' << d.v << ' ' << e.v << '\n'; // 3. 输出用成员 .v
```

样例： $a+b \rightarrow 8$; $a*b \rightarrow 15$; $a/b \rightarrow 200000002$ ($3 * \text{inv}(5) \bmod 1e9+7$)。

【传参要求（照这个传不会错）】

- `ModInt<MOD>`：模板参数 `MOD` 是编译期常量模数（`int` 范围，建议质数）；`ModInt(x)` 传 `i64`，构造时自动取模并把负数修正到 $[0, \text{MOD})$ 。
- 运算符 `+` `-` `*` `/`：返回 `ModInt`；除法内部走 `a * b.inv()`，要求模数是质数（费马逆元），非质数时 `b` 可能不可逆。
- `pow(i64 e)`：返回 a^e 的 `ModInt`；`inv()`：返回模逆元 `ModInt`。
- 输出用成员 `v` (`int`，已取模后的值)。

【不会用就照抄】

```
using mint = ModInt<998244353>;
mint a = x, b = y;
mint c = a + b;
mint d = a * b;
mint e = a / b; // 只有 b 可逆时才合法
```

- 除法依赖逆元；若模板用费马逆元，模数通常需要是质数。
- 输出成员名（如 `.v/.val`）以本节代码为准。

【API / 入口函数（赛时只认这里列的名字）】

- `inv()` $\rightarrow$ 模逆元 返回 `ModInt`。
- `pow(i64 e)` $\rightarrow$ 快速幂 返回 `ModInt`。

```
template <int MOD>
struct ModInt {
    int v;
    ModInt(i64 x = 0) {
        x %= MOD;
        if (x < 0) x += MOD;
        v = (int)x;
    }
    ModInt operator+(const ModInt& o) const { int x = v + o.v; if (x >= MOD) x -= MOD; return x; }
    ModInt operator-(const ModInt& o) const { int x = v - o.v; if (x < 0) x += MOD; return x; }
    ModInt operator*(const ModInt& o) const { return 1LL * v * o.v % MOD; }
    ModInt pow(i64 e) const {
        ModInt a = *this, r = 1;
        while (e) {
            if (e & 1) r = r * a;
            a = a * a;
            e >>= 1;
        }
        return r;
    }
    ModInt inv() const { return pow(MOD - 2); }
    ModInt operator/(const ModInt& o) const { return *this * o.inv(); }
};
```

线性求逆元与阶乘逆元

赛时先看

题目信号：大量组合数、除法取模、同一模数多次询问。

本质：模数为质数时批量求 $1..n$ 的逆元，以及组合数预处理。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定： $O(n)$ 。

维护的量：`inv[i]` ($1..n$ 的逆元数组)；`fac/ifac` (阶乘与阶乘逆元)、`mod` (模数)。

警告：模数必须为质数或保证每个数与模数互质。

【最小完整示例（先抄这一段就能跑）】

题目：模 $1e9+7$ 下求 $1..10$ 的逆元，并回答 $C(5,2)$ 。

```
vector<i64> inv = linear_inv(10, 1000000007); // 1. 批量求 1..10 的逆元
CombFast comb(100000, 1000000007); // 2. 预处理阶乘/阶乘逆元到  $1e6$ 
```

```
cout << inv[3] << '\n';           // 3. 3 的逆元
cout << comb.C(5, 2) << '\n';     // 4. 组合数 C(5, 2)
```

样例: `inv[3] -> 333333336` ($3 \times 333333336 \equiv 1$); `C(5, 2) -> 10`。

【传参要求（照这个传不会错）】

- `linear_inv(n, mod)`: `n` 为逆元上限, `mod` 为模数（必须质数或保证所有 $1..n$ 与 `mod` 互质）; 返回 1-indexed 数组, `inv[i] = i^{-1} \bmod \text{mod}`; `n <= 0` 返回空。
- `CombFast(max_n, mod)`: 构造即预处理到 `max_n`; `mod` 必须是质数（逆元走 `qpow(fac[n], mod-2)`）。
- `comb.C(n, k)`: 返回 `C(n, k) % mod` (`i64`); `k < 0 || k > n` 返回 0; 要求 `n <= max_n`。
- 组合数查询要求 `n < mod`, 否则阶乘含 `mod` 因子结果恒 0, 换 Lucas。

【API / 入口函数（赛时只认这里列的名字）】

- `C(int n, int k) -> 组合数 C(N, K) 返回 i64`。
- `init(int n, i64 mod_) -> 初始化/清空结构`

```
vector<i64> linear_inv(int n, i64 mod) {
    if (n <= 0) return {};
    vector<i64> inv(n + 1);
    inv[1] = 1;
    for (int i = 2; i <= n; i++) {
        inv[i] = (mod - mod / i) * inv[mod % i] % mod;
    }
    return inv;
}

struct CombFast {
    i64 mod;
    vector<i64> fac, ifac;
    CombFast(int n = 0, i64 mod_ = 1) { if (n) init(n, mod_); }
    i64 qpow(i64 a, i64 e) {
        i64 r = 1;
        while (e) {
            if (e & 1) r = r * a % mod;
            a = a * a % mod;
            e >>= 1;
        }
        return r;
    }
    void init(int n, i64 mod_) {
        mod = mod_;
        fac.assign(n + 1, 1);
        ifac.assign(n + 1, 1);
        for (int i = 1; i <= n; i++) fac[i] = fac[i - 1] * i % mod;
        ifac[n] = qpow(fac[n], mod - 2);
        for (int i = n; i >= 1; i--) ifac[i - 1] = ifac[i] * i % mod;
    }
    i64 C(int n, int k) const {
        if (k < 0 || k > n) return 0;
        return fac[n] * ifac[k] % mod * ifac[n - k] % mod;
    }
};
```

扩展欧几里得

赛时先看

题目信号: 线性同余、CRT、模数不一定是质数。

本质: 求 $ax + by = \gcd(a, b)$ 的一组解, 求非质数模逆元。

接法: 求 `a` 在任意模数 `mod` 下的逆元, 调用 `inv_general_ext(a, mod)`, 返回 `-1` 表示不存在。看到 $a \cdot x \equiv b \pmod{m}$, 先用 $g = \gcd(a, m)$ 判断 `b` 是否能被 `g` 整除, 再把方程除以 `g` 后求逆元。

复杂度判定: $O(\log \min(a, b))$ 。

维护的量: 无状态; 输出参数 `x/y` 为 $ax+by=\gcd$ 的一组解。

警告: 返回 `gcd` 要非负; 逆元存在条件是 $\gcd(a, \text{mod})=1$ 。本节的 `exgcd_ext` / `inv_general_ext` 与本册通用辅助函数里的 `exgcd` / `inv_general` 功能相同, 名字不同避免重名; 与其它节混抄时二选一即可。

【最小完整示例（先抄这一段就能跑）】

题目: 求 $\gcd(30, 12)$ 的一组整系数解, 并求 3 在模 7 下的逆元。

```
i64 x, y;
i64 g = exgcd_ext(30, 12, x, y); // 1. 求 30x+12y=g 的一组解 (x, y 引用返回)
i64 inv = inv_general_ext(3, 7); // 2. 任意模数逆元: 3^{-1} \bmod 7
i64 bad = inv_general_ext(2, 6); // 3. gcd(2, 6)=2 != 1, 逆元不存在
```

```
cout << g << ' ' << x << ' ' << y << ' ' << inv << ' ' << bad << '\n';
```

样例: $g=6, x=1, y=-2; inv=5 (3*5 \equiv 1 \pmod{7}); bad=-1$ 。

【传参要求（照这个传不会错）】

- `exgcd_ext(a, b, x, y)`: a/b 为任意 i64; x/y 引用参数输出满足 $ax+by=g$ 的一组解; 返回 $\gcd(a,b)$ (非负)。
- `inv_general_ext(a, mod)`: 要求 $\gcd(a,mod)=1$; 返回 $a^{-1} \pmod{mod}$ (范围 $[0, mod)$); 互质不成立时返回 -1 。
- 本节函数与本册通用辅助函数的 `exgcd / inv_general` 功能相同, 混抄时二选一避免重名。

```
i64 exgcd_ext(i64 a, i64 b, i64& x, i64& y) {
    if (b == 0) {
        x = (a >= 0 ? 1 : -1);
        y = 0;
        return llabs(a);
    }
    i64 x1, y1;
    i64 g = exgcd_ext(b, a % b, x1, y1);
    x = y1;
    y = x1 - a / b * y1;
    return g;
}

i64 inv_general_ext(i64 a, i64 mod) {
    i64 x, y;
    i64 g = exgcd_ext(a, mod, x, y);
    if (g != 1) return -1;
    x %= mod;
    if (x < 0) x += mod;
    return x;
}
```

线性筛

赛时先看

题目信号: 题面出现“大量质数/最小质因子/phi/mu 查询”且 $n \leq$

$1e7$, 比如反复判断质数、质因子分解、莫比乌斯反演的批量前缀; 看到“预处理一张 $1..n$ 的质数/积性函数表”就先想线性筛。

本质: 每个合数只被它的最小质因子筛掉一次; 外层枚举 i , 内层只用质数 $p \leq \minp[i]$ 生成 $i*p$, 保证每个合数唯一标记, 所以是 $O(n)$ 而不是埃氏筛的 $O(n \log \log n)$ 。

复杂度判定: $O(n)$ 时间、 $O(n)$ 内存; $n \leq 1e7$ 时四个 `int` 数组约 160MB 以内, 稳; n 到 $1e8$

以上内存不够, 改用分解质因数或高级筛 (`min_25 / 杜教筛`)。

维护的量: `primes` (质数表)、`minp[x]` (x 的最小质因子)、`phi[x]` (欧拉函数)、`mu[x]` (莫比乌斯函数), 全部 1-indexed。

接法: 如果后面要反复判断质数、求最小质因子、`phi` 或 `mu`, 先 `LinearSieve sieve(maxn)`。`sieve.primes` 是质数表, `sieve.minp[x]` 是最小质因子, `phi[x]` 和 `mu[x]` 可直接查。`maxn` 太大时不要硬筛, 改用分解或高级筛。

警告: 1 不是质数; 数组大小按最大值开; `phi[1]=mu[1]=1` 是特判; 需要约数个数/约数和等积性函数时, 本节没有, 换 `MultiplicativeSieve`。

【最小完整示例（先抄这一段就能跑）】

题目: 筛到 $1e6$, 判 97 是否质数, 查 `phi(97)` 与 `mu(6)`。

```
LinearSieve sieve(1000000); // 1. 构造即筛到 1e6 (只筛一次即可)
bool isp = sieve.minp[97] == 97; // 2. minp[x]==x 等价于 x 是质数
int phi97 = sieve.phi[97]; // 3. 欧拉函数直接查表
int mu6 = sieve.mu[6]; // 4. 莫比乌斯函数查表
cout << isp << ' ' << phi97 << ' ' << mu6 << '\n';
```

样例: $isp=1; phi(97)=96; mu(6)=1 (6=2*3, \text{两个不同质因子})$ 。

【传参要求（照这个传不会错）】

- `LinearSieve(maxn)`: 构造即筛到 `maxn`; `maxn` 取题面最大查询上界, 一次筛到最大即可, 别重复 `init`。
- `sieve.primes`: 质数表 (升序); `sieve.minp[x]`: x 的最小质因子, `minp[x]==x` 即 x 为质数。
- `sieve.phi[x]`: 欧拉函数; `sieve.mu[x]`: 莫比乌斯函数 (有平方因子时为 0); 全部 1-indexed, `phi[1]=mu[1]=1` 已特判。
- 需要约数个数/约数和时本节没有, 换「线性筛: `phi / mu / 约数个数`」的 `MultiplicativeSieve`。

【抄板清单（照着做就行）】

1. 抄哪段: 整个 `LinearSieve` 结构体。

- 构造: `LinearSieve sieve(maxn);`, `maxn` 取题面最大的查询上界 (一次筛到最大即可)。
- 调用: 判质数看 `sieve.minp[x] == x`; 最小质因子直接 `sieve.minp[x]`; `sieve.phi[x]`、`sieve.mu[x]` 查表即得。
- 取结果: 查表值直接输出, 无需额外计算。

【改造点 (按题目改这几处)】

- `n` 上限: `maxn` 从题面取, 只筛到需要的最大值; 多次询问不同上界时取最大的一次筛, 别重复 `init`。
- 需要约数个数/约数和: 本节只筛 `phi/mu`, 换「线性筛: `phi / mu / 约数个数`」的 `MultiplicativeSieve`。
- 只判质数用不到 `phi/mu`: 仍可直接用; 若内存紧张, 可以删掉 `phi`、`mu` 两个数组只留 `minp`。
- 质数计数 `pi(n)`: 对 `primes` 二分或做前缀和。

【核心逻辑 (改代码时别破坏)】

- 每个合数只被最小质因子标记一次, 总复杂度 $O(n)$ 。
- `p > minp[i]` 提前 `break`: 防止用非最小质因子的质数重复筛同一个合数。

【改板时先认这几个量】

- `minp[x]`: `x` 的最小质因子, `minp[x] == x` 等价于 `x` 是质数。
- `phi[x]`: 与 `x` 互质且不超过 `x` 的正整数个数; `mu[x]`: 莫比乌斯函数 (有平方因子时为 0)。

// 维护的量: `primes` 质数表; `minp[x] = x` 的最小质因子; `phi[x]` 欧拉函数; `mu[x]` 莫比乌斯函数 (均 1-indexed)。
 // 不变量: 每个合数只被其最小质因子筛一次, 总复杂度 $O(n)$ 。

```
struct LinearSieve {
    vector<int> primes, minp, phi, mu;

    LinearSieve(int n = 0) { if (n) init(n); }

    void init(int n) {
        primes.clear();
        minp.assign(n + 1, 0);
        phi.assign(n + 1, 0);
        mu.assign(n + 1, 0);
        if (n >= 1) phi[1] = 1, mu[1] = 1; // 特判 1: phi(1)=mu(1)=1
        for (int i = 2; i <= n; ++i) {
            if (!minp[i]) {
                // i 是质数: minp 记自身, phi=i-1, mu=-1
                minp[i] = i;
                primes.push_back(i);
                phi[i] = i - 1;
                mu[i] = -1;
            }
            for (int p : primes) {
                // 只用不超过 minp[i] 的质数生成 i*p, 保证每个合数只被筛一次
                if (p > minp[i] || 1LL * i * p > n) break;
                minp[i * p] = p;
                if (i % p == 0) {
                    // p | i: i*p 与 i 质因子集合相同, phi 乘 p; mu 记 0 并 break
                    phi[i * p] = phi[i] * p;
                    mu[i * p] = 0;
                    break;
                } else {
                    // p & i: 两者互质, 积性函数值相乘
                    phi[i * p] = phi[i] * (p - 1);
                    mu[i * p] = -mu[i];
                }
            }
        }
    }
};
```

线性筛: `phi / mu / 约数个数`

赛时先看

题目信号: 多次询问积性函数; 需要莫比乌斯反演; `n` 可到 $1e7$ 左右。

本质: 一次筛出素数、欧拉函数、莫比乌斯函数、约数个数。

接法: 把本节 `struct/class/函数组` 整段抄到 `solve()` 上面; 外部只调用下面 API 列出的接口, 不要把内部递归参数直接当题面参数。

复杂度判定: $O(n)$ 。

维护的量: `primes/is_comp` (质数表/合数标记)、`phi/mu/d` (欧拉/莫比乌斯/约数个数)、`cnt` (最小质因子指数)。

警告: `p | i` 时要 `break`; 约数个数需要额外维护最小质因子的指数。

【最小完整示例 (先抄这一段就能跑)】

题目：筛到 1000，查 $d(12)$ 、 $\phi(9)$ 、 $\mu(8)$ 。

```
MultiplicativeSieve s(1000);           // 1. 构造即筛出全部积性函数
int d12 = s.d[12];                      // 2. 约数个数 d(12)
int phi9 = s.phi[9];                   // 3. 欧拉函数 phi(9)
int mu8 = s.mu[8];                     // 4. 莫比乌斯函数 mu(8)
cout << d12 << ' ' << phi9 << ' ' << mu8 << '\n';
```

样例： $d(12)=6$ （约数 1, 2, 3, 4, 6, 12）； $\phi(9)=6$ ； $\mu(8)=0$ （含平方因子）。

【传参要求（照这个传不会错）】

- `MultiplicativeSieve(maxn)`：构造即预处理到 `maxn`（ $1e7$ 左右仍可行）；所有数组 1-indexed，`phi[1]=mu[1]=d[1]=1` 已特判。
- `s.primes`：质数表；`s.is_comp[x]` 为 1 表示合数；`s.phi[x]`、`s.mu[x]`、`s.d[x]` 直接查表。
- `s.cnt[x]`：x 的最小质因子指数（维护约数个数的中间量，一般不直接用）。
- 只需要判质数/phi/mu、不需要约数个数时，用「线性筛」的 `LinearSieve` 更省内存。

【API / 入口函数（赛时只认这里列的名字）】

- `init(int n_)` → 初始化/清空结构

```
struct MultiplicativeSieve {
    int n;
    vector<int> primes, is_comp, phi, mu, d, cnt;

    MultiplicativeSieve(int n = 0) { if (n) init(n); }

    void init(int n_) {
        n = n_;
        primes.clear();
        is_comp.assign(n + 1, 0);
        phi.assign(n + 1, 0);
        mu.assign(n + 1, 0);
        d.assign(n + 1, 1);
        cnt.assign(n + 1, 0);
        phi[1] = 1; mu[1] = 1; d[1] = 1;
        for (int i = 2; i <= n; i++) {
            if (!is_comp[i]) {
                primes.push_back(i);
                phi[i] = i - 1;
                mu[i] = -1;
                d[i] = 2;
                cnt[i] = 1;
            }
            for (int p : primes) {
                i64 v = 1LL * i * p;
                if (v > n) break;
                is_comp[v] = 1;
                if (i % p == 0) {
                    phi[v] = phi[i] * p;
                    mu[v] = 0;
                    cnt[v] = cnt[i] + 1;
                    d[v] = d[i] / (cnt[i] + 1) * (cnt[v] + 1);
                    break;
                } else {
                    phi[v] = phi[i] * (p - 1);
                    mu[v] = -mu[i];
                    cnt[v] = 1;
                    d[v] = d[i] * 2;
                }
            }
        }
    }
};
```

试除分解质因数

赛时先看

题目信号：n <= $1e12$ 或单次分解不多。

本质：单个数分解、求约数个数、phi 单点。

接法：单个 n 不太大时直接 `factorize_trial(n)`；返回（质因子，次数）。可以据此算约数个数、欧拉函数、枚举约数。若 n 是 64 位大整数且很多次分解，翻 Pollard-Rho。

复杂度判定： $O(\sqrt{n})$ 。

维护的量：无状态；返回 f（质因子，次数）列表）。

警告：循环结束后若 $n > 1$ ，剩下的 n 就是最后一个质因子。

【最小完整示例（先抄这一段就能跑）】

题目：分解 360 并输出所有质因子及次数。

```
auto f = factorize_trial(360); // 1. 返回（质因子，次数）列表
for (auto [p, c] : f)
    cout << p << '^' << c << ' '; // 2. 逐个输出
```

样例：2^3 3^2 5^1 (360 = 2^3 * 3^2 * 5)。

【传参要求（照这个传不会错）】

- factorize_trial(n): n 为 i64; n ≤ 1e12 时 O(sqrt n) 单次可行，更大或需多次分解时翻 Pollard-Rho。
- 返回 vector<pair<i64,int>>: 每个元素（质因子，次数），按质因子升序；n 本身是质数时返回 {(n,1)}；n=1 返回空列表。
- 可基于返回值算约数个数 $\prod(c+1)$ 、欧拉函数，或用 divisors_from_factorization 枚举约数。

```
vector<pair<i64, int>> factorize_trial(i64 n) {
    vector<pair<i64, int>> f;
    for (i64 p = 2; p * p <= n; ++p) {
        if (n % p) continue;
        int c = 0;
        while (n % p == 0) n /= p, c++;
        f.push_back({p, c});
    }
    if (n > 1) f.push_back({n, 1});
    return f;
}
```

单个数欧拉函数

赛时先看

题目信号：只问少量几个数的欧拉函数，不值得筛。

本质：求小范围外单个 $\phi(n)$ 。

接法：把本节 struct/class/函数组 整段抄到 solve() 上面；solve() 里直接调用本节列出的函数即可。

复杂度判定：O(sqrt n) 试除。

维护的量：ans (phi 值，每遇质因子 p 乘 (p-1)/p)；无其它状态。

警告：每遇到一个质因子 p，答案变成 ans / p * (p - 1)。

【最小完整示例（先抄这一段就能跑）】

题目：求 $\phi(36)$ 。

```
i64 ans = phi_single(36); // 1. 一次试除求单点欧拉函数
cout << ans << '\n';
```

样例： $\phi(36)=12$ (36=2^2*3^2, 36*(1/2)*(2/3)=12)。

【传参要求（照这个传不会错）】

- phi_single(n): n 为 i64 (n ≤ 1e12 时 O(sqrt n) 可行)；返回 $\phi(n)$ (i64)。
- n=1 时返回 1；n 为质数时返回 n-1。
- 只求少量几个数的 phi 时用它；批量查询用「线性筛」或杜教筛。

```
i64 phi_single(i64 n) {
    i64 ans = n;
    for (i64 p = 2; p * p <= n; ++p) {
        if (n % p) continue;
        ans = ans / p * (p - 1);
        while (n % p == 0) n /= p;
    }
    if (n > 1) ans = ans / n * (n - 1);
    return ans;
}
```

枚举约数

赛时先看

题目信号：题目需要遍历所有因子、判断约数贡献。

本质：得到一个数的所有正约数。

接法：把本节 struct/class/函数组 整段抄到 solve() 上面；solve() 里直接调用本节列出的函数即可。

复杂度判定：约数个数通常远小于 n。

维护的量: `divs` (结果列表, 升序); DFS 参数 `idx/cur` 为中间状态。

警告: 先分解质因数, 再 DFS 乘出约数。

【最小完整示例 (先抄这一段就能跑)】

题目: 枚举 36 的所有正约数。

```
auto fac = factorize_trial(36);           // 1. 先分解质因数
auto divs = divisors_from_factorization(fac); // 2. DFS 乘出全部约数 (已排序)
for (i64 d : divs) cout << d << ' ';
```

样例: 1 2 3 4 6 9 12 18 36。

【传参要求 (照这个传不会错)】

- `divisors_from_factorization(fac)`: `fac` 直接传 `factorize_trial` 的返回 ((质因子, 次数) 升序列表); 返回所有正约数 (升序, 含 1 与 `n` 本身)。
- `fac` 为空 (`n=1`) 时返回 `{1}`。
- 约数个数远小于 `n`, 枚举安全; 必须先分解, 不要对 `n` 直接枚举到 `sqrt`。

```
void gen_divisors_dfs(int idx, i64 cur,
                    const vector<pair<i64, int>>& fac,
                    vector<i64>& divs) {
    if (idx == (int)fac.size()) {
        divs.push_back(cur);
        return;
    }
    auto [p, c] = fac[idx];
    i64 x = 1;
    for (int i = 0; i <= c; ++i) {
        gen_divisors_dfs(idx + 1, cur * x, fac, divs);
        x *= p;
    }
}

vector<i64> divisors_from_factorization(vector<pair<i64, int>> fac) {
    vector<i64> divs;
    gen_divisors_dfs(0, 1, fac, divs);
    sort(divs.begin(), divs.end());
    return divs;
}
```

整除分块

赛时先看

题目信号: 求和里有 n / i , n 很大, 不能逐个枚举。

本质: 快速枚举 $\text{floor}(n / i)$ 相同的一段。

接法: 把本节 struct/class/函数组 整段抄到 `solve()` 上面; `solve()` 里直接调用本节列出的函数即可。

复杂度判定: $O(\sqrt{n})$ 段。

维护的量: l/r (当前段端点)、 $q = \text{floor}(n/l)$ (该段取值); 无其它状态。

警告: 右端点 $r = n / (n / l)$ 。

【最小完整示例 (先抄这一段就能跑)】

题目: 求 $\sum_{i=1}^n \text{floor}(n/i)$, $n=10$ 。

```
i64 sum = 0;
divisor_blocks(10, [&](i64 l, i64 r, i64 q) {
    sum += q * (r - l + 1); // 2. 这段 [l,r] 内 floor(n/i) 都等于 q
});                          // 1. 自动按取值相同分段
cout << sum << '\n';
```

样例: $n=10$ 分 5 段: (1,1,10) (2,2,5) (3,3,3) (4,5,2) (6,10,1), 和 =27。

【传参要求 (照这个传不会错)】

- `divisor_blocks(n, visit)`: `n` 为 `i64` (可到 $1e12+$); `visit(l, r, q)` 对每段调用一次, $[l,r]$ 为段内 i 的范围, $q = \text{floor}(n/l)$ 为该段取值。
- 段数 $O(\sqrt{n})$ ($n=1e12$ 时约 $2e6$ 段), 每段 $O(1)$ 处理即可。
- 求和必须用段长 $(r-l+1)$ 乘 q , 不要逐个枚举 i 。

```
template <class F>
void divisor_blocks(i64 n, F visit) {
    for (i64 l = 1, r; l <= n; l = r + 1) {
```



```

    i64 q = n / l;
    r = n / q;
    visit(l, r, q); // i 在 [l,r] 内时 floor(n/i)=q。
}
}

```

杜教筛：大范围前缀 ϕ / μ 和

赛时先看

题目信号： ϕ/μ 前缀和、互质对计数、整除分块，且 n 远大于可筛上界。

本质： 求 $\sum_{i \leq n} \phi(i)$ 或 $\sum_{i \leq n} \mu(i)$ ，其中 n 可到 $1e10$ 甚至更大，不能直接筛到 n 。

接法： 求 $\sum_{i=1}^n \phi(i)$ 、 $\sum_{i=1}^n \mu(i)$ ，或把 gcd 计数中的莫比乌斯前缀和替换成大范围查询。

复杂度判定： 预筛 $O(N)$ ；记忆化后单次大询问常见 $O(n^{2/3})$ 量级。

维护的量： $\text{pre_}\phi/\text{pre_}\mu$ （预筛到 N 的前缀和表， $n \leq N$ 直接查）； $\text{memo_}\phi/\text{memo_}\mu$ （大 n 的答案缓存）。

警告： 这里返回的是前缀和，不是单点函数值； ϕ 前缀和可达 $\Theta(n^2)$ 量级， $\text{sum_}\phi$ 在 n 超过约 $4e9$ 时 $i64$ 会溢出， n 较大时改用 $i128$ 或按题目取模。

【最小完整示例（先抄这一段就能跑）】

题目：求 $\sum_{i \leq 10} \phi(i)$ 与 $\sum_{i \leq 10} \mu(i)$ 。

```

DuJiaoPhiMu d(100);           // 1. 结构体定义：DuJiaoPhiMu(预筛上限)，默认 5e6
i64 p = d.sum_phi(10);         // 2. 调用：phi 前缀和，n 可远超预筛上界
i64 m = d.sum_mu(10);          // 3. 调用：mu 前缀和，内部整除分块 + 记忆化
cout << p << ' ' << m << '\n';

```

样例： $\text{sum_}\phi(10) \rightarrow 32$ ； $\text{sum_}\mu(10) \rightarrow -1$ （本板无取模，原样输出）。

【传参要求（照这个传不会错）】

- $\text{DuJiaoPhiMu } d(\text{limit})$ ： limit = 预筛上界（int，默认 5000000）； $n \leq \text{limit}$ 直接查 $\text{pre_}\phi/\text{pre_}\mu$ 表， $n > \text{limit}$ 走记忆化递归。
- $d.\text{sum_}\phi(n) / d.\text{sum_}\mu(n)$ ： n 为 $i64$ ，可到 $1e10$ 甚至更大；返回对应前缀和（ $i64$ ）。
- 返回的是前缀和不是单点函数值； $\text{sum_}\phi$ 在 n 约 $4e9$ 时开始溢出 $i64$ ，需要时改用 $i128$ 或按题目取模。

【API / 入口函数（赛时只认这里列的名字）】

- $\text{init}(\text{int } \text{limit}) \rightarrow$ 初始化/清空结构

典型模型：求 $\sum_{i=1}^n \phi(i)$ 、 $\sum_{i=1}^n \mu(i)$ ，或把 gcd 计数中的莫比乌斯前缀和替换成大范围查询。

```

struct DuJiaoPhiMu {
    int N;
    vector<int> primes, is_comp, phi, mu;
    vector<i64> pre_phi, pre_mu;
    unordered_map<i64, i64> memo_phi, memo_mu;

    explicit DuJiaoPhiMu(int limit = 5000000) { init(limit); }

    void init(int limit) {
        N = limit;
        is_comp.assign(N + 1, 0);
        phi.assign(N + 1, 0);
        mu.assign(N + 1, 0);
        primes.clear();
        phi[1] = 1;
        mu[1] = 1;
        for (int i = 2; i <= N; ++i) {
            if (!is_comp[i]) {
                primes.push_back(i);
                phi[i] = i - 1;
                mu[i] = -1;
            }
            for (int p : primes) {
                i64 v = 1LL * i * p;
                if (v > N) break;
                is_comp[(int)v] = 1;
                if (i % p == 0) {
                    phi[(int)v] = phi[i] * p;
                    mu[(int)v] = 0;
                    break;
                }
                phi[(int)v] = phi[i] * (p - 1);
                mu[(int)v] = -mu[i];
            }
        }
        pre_phi.assign(N + 1, 0);
    }
}

```

```

    pre_mu.assign(N + 1, 0);
    for (int i = 1; i <= N; ++i) {
        pre_phi[i] = pre_phi[i - 1] + phi[i];
        pre_mu[i] = pre_mu[i - 1] + mu[i];
    }
    memo_phi.clear();
    memo_mu.clear();
}

i64 sum_mu(i64 n) {
    if (n <= N) return pre_mu[(int)n];
    if (auto it = memo_mu.find(n); it != memo_mu.end()) return it->second;
    i64 ans = 1;
    for (i64 l = 2, r; l <= n; l = r + 1) {
        r = n / (n / l);
        ans -= (r - l + 1) * sum_mu(n / l);
    }
    return memo_mu[n] = ans;
}

i64 sum_phi(i64 n) {
    if (n <= N) return pre_phi[(int)n];
    if (auto it = memo_phi.find(n); it != memo_phi.end()) return it->second;
    i64 ans = n * (n + 1) / 2;
    for (i64 l = 2, r; l <= n; l = r + 1) {
        r = n / (n / l);
        ans -= (r - l + 1) * sum_phi(n / l);
    }
    return memo_phi[n] = ans;
}
};

```

Min_25 筛核心： $\pi(n)$ 与质数和

赛时先看

题目信号： n 大到不能直接筛，例如 $1e10$ ；答案依赖“所有不超过 n 的质数”的数量或质数和；更复杂的积性函数题可在此基础上枚举最小质因子和幂次递归。

本质：在远大于筛上界的 n 上，求质数个数 $\pi(n)$ 与 $\sum_{p \leq n} p$ 。这是 Min_25 筛处理积性函数前缀和的第一层核心。

接法：求 $\pi(n)$ 、 $\sum_{p \leq n} p$ ；或先计算 $f(p)$ 的前缀和，再结合素数幂递归求完全积性函数相关前缀和。

复杂度判定： $O(n^{3/4} / \log n)$ 量级，空间 $O(\sqrt{n})$ 。

维护的量：`values`（整除分块点集）；`prime_count/prime_sum`（去掉合数贡献后的质数计数/质数和表，`prime_sum` 为 `i128`）。

警告：这份是最稳的“质数计数/质数和核心版”，不是任意积性函数的一键模板；`sum_primes` 用 `i128` 保存，输出时自行写转换函数。

【最小完整示例（先抄这一段就能跑）】

题目： $n=1e10$ 时求 $\pi(100)$ 与 $\sum_{p \leq 100} p$ （ n 大到不能直接筛）。

```

Min25PrimePrefix s(10000000000LL); // 1. 结构体定义：Min25PrimePrefix(n)，构造即 build
i64 cnt = s.pi(100); // 2. 调用：pi(x)，x 可以远小于 n
i128 sum = s.sum_primes(100); // 3. 调用：质数和，返回 i128
cout << cnt << '\n'; // 4. i128 输出需自己写转换

```

样例： $\pi(100) \rightarrow 25$ ； $\text{sum_primes}(100) \rightarrow 1060$ （ $2+3+\dots+97$ ）。

【传参要求（照这个传不会错）】

- `Min25PrimePrefix s(n)`： n = 题目上界（i64，如 $1e10$ ）；构造时完成全部预处理，之后只查表。
- `s.pi(x)`： x 为 i64， $1 \leq x \leq n$ ；返回 $[1, x]$ 内质数个数（i64）。
- `s.sum_primes(x)`：同样要求 $x \leq n$ ；返回 $\sum_{p \leq x} p$ ，类型 `i128`，输出前自行转字符串或拆成两个 `long long`。
- 只负责质数计数/质数和；任意积性函数前缀和要按注释改 `prime_count/prime_sum` 的转移。

【API / 入口函数（赛时只认这里列的名字）】

- `build()` $\rightarrow$ 完成建树或预处理

【改板时先认这几个量】

- `cur`：整除分块中当前值 x 在 `values` 里的下标。
- `pi(x)`：质数个数函数，查 `prime_count` 表。

典题模型：求 $\pi(n)$ 、 $\sum_{p \leq n} p$ ；或先计算 $f(p)$ 的前缀和，再结合素数幂递归求完全积性函数相关前缀和。

```

struct Min25PrimePrefix {
    i64 n;
    int sq;
    vector<int> primes;
    vector<i64> values, prime_count;
    vector<i128> prime_sum;
    vector<int> id_small, id_large;

    explicit Min25PrimePrefix(i64 n_) : n(n_) { build(); }

    int id(i64 x) const {
        return x <= sq ? id_small[(int)x] : id_large[(int)(n / x)];
    }

    void build() {
        sq = sqrtl(n);
        while (1LL * (sq + 1) * (sq + 1) <= n) ++sq;
        while (1LL * sq * sq > n) --sq;

        vector<int> is_comp(sq + 1);
        for (int i = 2; i <= sq; ++i) {
            if (!is_comp[i]) primes.push_back(i);
            for (int p : primes) {
                if (1LL * i * p > sq) break;
                is_comp[i * p] = 1;
                if (i % p == 0) break;
            }
        }
        vector<i128> pre_prime_sum(primes.size() + 1);
        for (int i = 0; i < (int)primes.size(); ++i) pre_prime_sum[i + 1] = pre_prime_sum[i] + primes[i];

        id_small.assign(sq + 1, -1);
        id_large.assign(sq + 1, -1);
        for (i64 l = 1, r; l <= n; l = r + 1) {
            i64 x = n / l;
            r = n / x;
            int cur = (int)values.size();
            values.push_back(x);
            prime_count.push_back(x - 1); // 2..x 的整数个数
            prime_sum.push_back((i128)x * (x + 1) / 2 - 1); // 2..x 的整数和
            if (x <= sq) id_small[(int)x] = cur;
            else id_large[(int)(n / x)] = cur;
        }

        for (int i = 0; i < (int)primes.size(); ++i) {
            i64 p = primes[i];
            if (p * p > n) break;
            for (int j = 0; j < (int)values.size() && values[j] >= p * p; ++j) {
                int k = id(values[j] / p);
                prime_count[j] -= prime_count[k] - i;
                prime_sum[j] -= (i128)p * (prime_sum[k] - pre_prime_sum[i]);
            }
        }
    }

    i64 pi(i64 x) const { return prime_count[id(x)]; }
    i128 sum_primes(i64 x) const { return prime_sum[id(x)]; }
};

```

Meissel-Lehmer：超大范围质数个数 $\pi(n)$

赛时先看

题目信号：题目直接问 $\pi(n)$ ，或统计中需要大量知道“ $\leq x$ 的质数数量”； n 可到 $1e10$ 、 $1e12$ 甚至更高，但查询次数不多。

本质：求不大于 n 的质数个数 $\pi(n)$ 。相比直接筛到 n ，它用一个中等大小的预处理筛和递归计数处理远大于内存上界的 n 。

接法： n 极大时求 $[1, n]$ 里质数个数；多组大 x 的质数计数；数论分块里反复出现 $\pi(n / i)$ 的变种（查询很多时要再做记忆化策略评估）。

复杂度判定：单次是亚线性；对本实现的常见 $n \leq 1e12$ 场景很实用。预处理时间 $O(\text{SIEVE_N} \log \log \text{SIEVE_N})$ ，内存约 65 MB（ $\text{SIEVE_N}=5e6$ 、 $\text{PHI_N}=1e5$ ）。

维护的量： π （ SIEVE_N 内质数个数前缀表）； primes （质数表）； phi_small （小范围 phi 查表）； memo （大 x 的 π 结果缓存）。

警告：它只求“质数个数”，不是完整的 Min_25 任意积性函数筛。浮点开根会偏一，代码中的整数根修正不能删。 n 必须非负， $i64$ 范围内乘方比较使用 $i128$ 。

【最小完整示例（先抄这一段就能跑）】

题目：求 $\text{pi}(100)$ 与 $\text{pi}(1e10)$ （超大范围质数个数，构造一次后反复查询）。

```
LehmerPrimeCounter counter;           // 1. 结构体定义：构造即预筛（内存约 65 MB）
i64 a = counter.pi_count(100);        // 2. 调用：小 x 直接查 pi 表
i64 b = counter.pi_count(10000000000LL); // 3. 调用：大 x 走递归计数 + memo 缓存
cout << a << ' ' << b << '\n';
```

样例： $\text{pi_count}(100) \rightarrow 25$ ； $\text{pi_count}(1e10) \rightarrow 455052511$ 。

【传参要求（照这个传不会错）】

- `LehmerPrimeCounter counter;`：无参构造，内部完成 `SIEVE_N=5e6` 预筛 + `PHI_N=1e5` 表格；多次查询复用同一个对象。
- `counter.pi_count(x)`：x 为 i64（非负）；返回 $[1, x]$ 内质数个数（i64）。
- 内存低于 128 MB 时改小 `SIEVE_N/PHI_N` 并压测，或换 `Min_25` 核心版。

【API / 入口函数（赛时只认这里列的名字）】

- `build()` $\rightarrow$ 完成建树或预处理

【改板时先认这几个量】

- `pi`：前缀表，`pi[i]` 为 $[1, i]$ 内的质数个数。
- `memo`：大 x 的 `pi_count` 结果缓存。

使用：构造一次 `LehmerPrimeCounter counter;`，调用 `counter.pi_count(n)`。先评估内存限制；内存低于 128 MB 时，应改小参数并充分压测，或改用 `Min_25` 核心版。

典题模型：n 极大时求 $[1, n]$ 里质数个数；多组大 x 的质数计数；数论分块里反复出现 $\text{pi}(n / i)$ 的变种（查询很多时要再做记忆化策略评估）。

```
struct LehmerPrimeCounter {
    static constexpr int SIEVE_N = 5'000'000;
    static constexpr int PHI_N = 100'000;
    static constexpr int PHI_S = 100;

    vector<int> primes, pi;
    vector<array<int, PHI_S>> phi_small;
    unordered_map<i64, i64> memo;

    LehmerPrimeCounter() { build(); }

    static i64 isqrt(i64 x) {
        i64 r = sqrtl((long double)x);
        while ((r + 1) <= x / (r + 1)) ++r;
        while (r > x / r) --r;
        return r;
    }

    static i64 icbrt(i64 x) {
        i64 r = cbrtl((long double)x);
        while (((i128)(r + 1) * (r + 1) * (r + 1) <= x) ++r;
        while ((i128)r * r * r > x) --r;
        return r;
    }

    void build() {
        vector<bool> composite(SIEVE_N + 1);
        pi.assign(SIEVE_N + 1, 0);
        for (int i = 2; i <= SIEVE_N; ++i) {
            if (!composite[i]) primes.push_back(i);
            for (int p : primes) {
                if (1LL * i * p > SIEVE_N) break;
                composite[i * p] = true;
                if (i % p == 0) break;
            }
            pi[i] = pi[i - 1] + (!composite[i]);
        }

        phi_small.resize(PHI_N);
        for (int x = 0; x < PHI_N; ++x) phi_small[x][0] = x;
        for (int s = 1; s < PHI_S; ++s) {
            for (int x = 0; x < PHI_N; ++x) {
                phi_small[x][s] = phi_small[x][s - 1]
                    - phi_small[x / primes[s - 1]][s - 1];
            }
        }
    }

    i64 phi(i64 x, int s) const {
```

```

    if (s == 0) return x;
    if (s < PHI_S && x < PHI_N) return phi_small[(int)x][s];
    return phi(x, s - 1) - phi(x / primes[s - 1], s - 1);
}

i64 pi_count(i64 x) {
    if (x <= SIEVE_N) return pi[(int)x];
    if (auto it = memo.find(x); it != memo.end()) return it->second;

    i64 a = pi_count(isqrt(isqrt(x))); // 公式:  $\pi(x^{1/4})$ 。
    i64 b = pi_count(isqrt(x));       // 公式:  $\pi(\sqrt{x})$ 。
    i64 c = pi_count(icbrt(x));       // 公式:  $\pi(x^{1/3})$ 。
    i64 ans = phi(x, (int)a) + (b + a - 2) * (b - a + 1) / 2;

    for (i64 i = a; i < b; ++i) {
        i64 w = x / primes[(int)i];
        ans -= pi_count(w);
        if (i < c) {
            i64 limit = pi_count(isqrt(w));
            for (i64 j = i; j < limit; ++j) {
                ans -= pi_count(w / primes[(int)j]) - j;
            }
        }
    }
    return memo[x] = ans;
}
};

```

Mobius 反演常见求和

赛时先看

题目信号: 题面出现 $\gcd(i, j)=1$ 、 $\gcd(i, j)=d$ 、 $\text{floor}(n/i)$ 分块。

本质: 求互质对数、 $\gcd$ 相关计数、整除关系计数。

接法: 把本节 struct/class/函数组 整段抄到 `solve()` 上面; 优先调用下面 API 列出的入口, 其余 helper 默认不要从 `solve()` 直接调。

复杂度判定: 预处理 $O(n)$, 单次求和 $O(\sqrt{n})$ 或 $O(n)$ 。

维护的量: `prefix_mu` (`mu` 前缀和, 分块差分用); n/m (求和上界); $1/r$ (整除分块端点)。

警告: 先预处理 `mu` 前缀和; $\gcd(i, j)=d$ 转成互质对 $(i/d, j/d)$ 。

【最小完整示例 (先抄这一段就能跑)】

题目: 统计 $1 \leq i, j \leq 10$ 中 $\gcd(i, j)=1$ 的互质数对个数。

```

MultiplicativeSieve s(10); // 1. 先线性筛 mu (存在 s.mu)
vector<i64> prefix_mu(11, 0);
for (int i = 1; i <= 10; ++i) prefix_mu[i] = prefix_mu[i - 1] + s.mu[i];
i64 ans = coprime_pairs(10, 10, prefix_mu); // 2. 调用: 统计 1<=i, j<=10 的互质对数
cout << ans << '\n'; // 3. 取结果

```

样例: `coprime_pairs(10, 10, prefix_mu) -> 63`; `gcd_equal_pairs(10, 10, 2, prefix_mu) -> 19` (即 5×5 互质对数)。

【传参要求 (照这个传不会错)】

- `coprime_pairs(n, m, prefix_mu)`: $n/m =$ 上界 (`i64`, ≥ 1); `prefix_mu` 长度需 $\geq \min(n, m)+1$, 且 `prefix_mu[i] = sum_{j=1..i} mu(j)`。
- 返回 $1 \leq i \leq n$ 、 $1 \leq j \leq m$ 中 $\gcd(i, j)=1$ 的数对个数 (`i64`)。
- `gcd_equal_pairs(n, m, g, prefix_mu)`: 返回 $\gcd(i, j)=g$ 的数对个数, 内部即 `coprime_pairs(n/g, m/g, prefix_mu)`, `g` 需 ≥ 1 。
- 前提: `prefix_mu` 必须先用线性筛构造好; `mu(1)=1`, 含平方因子的下标不会增加前缀值。

【API / 入口函数 (赛时只认这里列的名字)】

- `coprime_pairs(i64 n, i64 m, const vector<i64>& prefix_mu)` -> 统计满足 $1 \leq i \leq n$ 、 $1 \leq j \leq m$ 且 $\gcd(i, j)=1$ 的数对。返回 `i64`。

```

// 统计满足 1<=i<=n, 1<=j<=m 且 gcd(i, j)=1 的数对。
i64 coprime_pairs(i64 n, i64 m, const vector<i64>& prefix_mu) {
    i64 ans = 0;
    i64 lim = min(n, m);
    for (i64 l = 1, r; l <= lim; l = r + 1) {
        r = min(n / (n / l), m / (m / l));
        ans += (prefix_mu[r] - prefix_mu[l - 1]) * (n / l) * (m / l);
    }
    return ans;
}

```

```

}

// 统计 gcd(i,j)=g 的数对。
i64 gcd_equal_pairs(i64 n, i64 m, i64 g,
                    const vector<i64>& prefix_mu) {
    return coprime_pairs(n / g, m / g, prefix_mu);
}

```

组合数预处理

赛时先看

题目信号：题面出现“方案数”“选若干个”“组合计数”且模数是质数、 $n <$

mod ，比如网格路径数、至少/至多选择类计数、二项式求和；看到“大量 $C(n,k)$ 查询 mod 质数”就先想这节。

本质：一次预处理 $\text{fac}[i]=i!$ 与 $\text{ifac}[i]=(i!)^{-1}$ （费马小定理求最大阶乘逆元后回推），查询

$C(N,K)=\text{fac}[N]*\text{ifac}[K]*\text{ifac}[N-K]$ ，把每次 $O(K)$ 的阶乘除法压成 $O(1)$ 查表。

复杂度判定：预处理 $O(n)$ ，查询 $O(1)$ ； n 到 $1e6-1e7$ 都稳；若 $n \geq \text{mod}$ ，阶乘会含 mod 因子而出现 0，换 Lucas；模数不是质数换 exLucas。

维护的量： n （预处理上限）、 mod （质数模数）、 fac （阶乘）、 ifac （阶乘逆元）、 inv （单点逆元，备用）。

接法：模数是质数且 $N \leq \text{max_n}$ 时先 $\text{Comb comb}(\text{max_n}, \text{mod})$ ，再 $\text{comb.C}(n,k)$ 。网格从 $(1,1)$ 到 (n,m) 只向右下走通常是 $C(n+m-2, n-1)$ 。若模数不是质数，不能用费马逆元预处理，翻 exLucas 或其他计数方法。

警告： mod 必须是质数（逆元走 $a^{(\text{mod}-2)}$ ）； $n \geq \text{mod}$ 时阶乘出现 0，不能直接套这版；非法 K 越界返回 0。

【最小完整示例（先抄这一段就能跑）】

题目： m 次询问 $C(n,k) \bmod 1e9+7$ ， n 最大 $1e6$ 。

```

Comb comb(1000000, 1000000007); // 1. 结构体定义：Comb(预处理上限, 质数模数)
while (m--) {
    int n, k;
    cin >> n >> k;
    cout << comb.C(n, k) << '\n'; // 2. 调用：O(1) 返回 C(n,k) mod
}

```

样例： $C(5,2) \rightarrow 10$ ； $C(10,3) \rightarrow 120$ 。

【传参要求（照这个传不会错）】

- $\text{Comb}(\text{max_n}, \text{mod})$ ：构造即预处理； max_n = 可能出现的最大 n ； mod 必须是质数。
- $C(n, k)$ ：返回 int （取模后）； $k < 0 \parallel k > n$ 返回 0。
- 要求 $n < \text{mod}$ ； $n \geq \text{mod}$ 时阶乘会含 mod 因子（结果 0），换 Lucas。
- 网格路径数（只向右下走） $(1,1) \rightarrow (n,m) = C(n+m-2, n-1)$ 。
- 模数不是质数时换 exLucas 或别的计数方法。

【不会用就照抄】

```

Comb C(MAXN);
cout << C.C(n, k) << '\n';

```

- MAXN 必须覆盖题目最大 n ；构造 $\text{Comb C}(\text{MAXN}, \text{mod})$ 后再查询。
- 这版逆元用 $a^{(\text{mod}-2)}$ ，因此 mod 必须是质数； $n \geq \text{mod}$ 时阶乘会出现 0，也不能直接套这版。

【API / 入口函数（赛时只认这里列的名字）】

- $\text{Comb C}(\text{max_n}, \text{mod}) \rightarrow$ 预处理到 max_n ；这版要求 mod 为质数。
- $C.C(N,K) \rightarrow$ 返回 $C(N,K) \bmod \text{mod}$ ；非法 K 返回 0。

【抄板清单（照着做就行）】

1. 抄哪段：整个 Comb 结构体。
2. 构造： $\text{Comb C}(\text{MAXN}, \text{mod})$ ； MAXN 必须 $\geq$ 题面最大 n ， mod 填题面模数（默认 $1e9+7$ ）。
3. 调用： $C.C(n, k)$ 。
4. 取结果：返回的就是 $C(n,k) \bmod \text{mod}$ ，直接输出。

【改造点（按题目改这几处）】

- 模数：构造时第二个参数传题面模数；必须是质数，否则翻 exLucas。
- $n \geq \text{mod}$ ：本板阶乘会含 mod 因子，直接失效，换同章 lucas。
- 需要可重复组合（从 n 种里可重复选 k 个）：用 $C(n+k-1, k)$ 。

- 大 n ($1e18$) 小模数：翻同章 Lucas 定理；模数极小（如 2）时注意 Lucas 递归到 $k==0$ 的基例。

【核心逻辑（改代码时别破坏）】

- 先预处理 $\text{fac}[i]=i!$ 与 $\text{ifac}[i]=(i!)^{-1}$ 。
- 查询 $C(N,K)=\text{fac}[N]*\text{ifac}[K]*\text{ifac}[N-K]$ ，所以单次 $O(1)$ 。

【改板时先认这几个量】

- $\text{fac}[i]$: $i! \bmod \text{mod}$; $\text{ifac}[i]$: $(i!)^{-1} \bmod \text{mod}$ 。
- 查询公式 $C(N,K)=\text{fac}[N]*\text{ifac}[K]*\text{ifac}[N-K]$ 。

```
// 维护的量: fac[i] = i! mod mod; ifac[i] = (i!)^{-1} mod mod; inv[i] 单点逆元（备用）。
// 不变量: fac[n] * ifac[n] ≡ 1 (mod mod), 查询 C(N,K)=fac[N]*ifac[K]*ifac[N-K]。
// 前提: mod 必须为质数（逆元走费马小定理 a^{(mod-2)}），且 max_n < mod。
struct Comb {
    int n;
    i64 mod;
    vector<i64> fac, ifac, inv;

    i64 mod_pow(i64 a, i64 b) const {
        i64 res = 1 % mod;
        while (b) {
            if (b & 1) res = (i128)res * a % mod;
            a = (i128)a * a % mod;
            b >>= 1;
        }
        return res;
    }

    Comb(int max_n = 0, i64 mod_ = 1000000007LL) { init(max_n, mod_); }

    void init(int max_n, i64 mod_) {
        n = max_n;
        mod = mod_;
        fac.assign(n + 1, 1);
        ifac.assign(n + 1, 1);
        inv.assign(n + 1, 1);
        for (int i = 1; i <= n; ++i) fac[i] = fac[i - 1] * i % mod; // 递推阶乘
        ifac[n] = mod_pow(fac[n], mod - 2); // 费马小定理求最大阶乘的逆元
        for (int i = n; i >= 1; --i) ifac[i - 1] = ifac[i] * i % mod; // 回推所有阶乘逆元
    }

    i64 C(int N, int K) const {
        if (K < 0 || K > N || N < 0 || N > n) return 0;
        return fac[N] * ifac[K] % mod * ifac[N - K] % mod; // 组合数公式, O(1) 查表
    }
};
```

Lucas 定理

赛时先看

题目信号: n 可到 $1e18$ ，但 p 较小且为质数。

本质: n, k 很大，模数是小质数。

接法: n, k 很大但模数 p 是小质数时，先 `Comb comb(p-1, p)`，再 `lucas(n, k, comb)`。如果 p 不是质数，Lucas 不成立，翻 `exLucas`。

复杂度判定: $O(\log_p n)$ 次组合数查询。

维护的量: 无状态；递归参数 n/k 每层除以 `comb.mod`，直至 $k==0$ 。

警告: 组合数预处理到 $p-1$ 。

【最小完整示例（先抄这一段就能跑）】

题目: $p=7$ 下求 $C(10,3) \bmod 7$ 。

```
Comb comb(6, 7); // 1. 预处理组合数到 p-1=6
i64 ans = lucas(10, 3, comb); // 2. 递归 Lucas: n 大、p 为小质数
cout << ans << '\n';
```

样例: `lucas(10,3) -> 1` ($C(10,3)=120 \equiv 1 \bmod 7$)。

【传参要求（照这个传不会错）】

- `lucas(n, k, comb)`: n/k 为 $i64$ （可到 $1e18$ ）；返回 $C(n,k) \% \text{comb.mod}$ ($i64$)； $k < 0 \mid k > n$ 返回 0。
- `comb` 用「组合数预处理」的 `Comb(p-1, p)` 构造：预处理上限必须 $\geq p-1$ ，模数 p 必须是质数，否则 Lucas 不成立（换 `exLucas`）。

- 复杂度 $O(\log_p n)$ 次组合数查询，单次查询 $O(1)$ 。

```
i64 lucas(i64 n, i64 k, const Comb& comb) {
    if (k < 0 || k > n) return 0;
    if (k == 0) return 1;
    return lucas(n / comb.mod, k / comb.mod, comb) *
        comb.C(n % comb.mod, k % comb.mod) % comb.mod;
}
```

exLucas: 组合数模任意合数

赛时先看

题目信号: 题面明确给出合数模数；要求组合数模 $6 / 8 / 10^6 / 998244352$ 等非质数；逆元法、普通 Lucas 不能直接使用； n 很大但模数及其各素数幂不大。

本质: 计算 $C(n, k) \bmod m$ ，其中 m 是合数。代码把 m 分解为互素的素数幂，在每个素数幂下去掉 p 因子计算阶乘，最后用 CRT 合并。

接法: 大 n 小合数模的二项式计数；周期计数后对合数取模；容斥中的组合数模 10^k ；CRT 综合题。

复杂度判定: 预处理每个素数幂 p^a 为 $O(p^a)$ ，单次组合数为 $O(\log_p n)$ ；分解模数为 $O(\sqrt{m})$ 。本实现要求所有素数幂因子不超过 $2e6$ ，这是用空间换稳定性的版本。

维护的量: $p/\text{exponent}/pk$ （当前素数幂及其模数）； $\text{prefix}[i]$ （ $1..i$ 去掉 p 因子的乘积模 pk ）； $\text{remain}/\text{merged_mod}/\text{ans}$ （CRT 合并中间量）。

警告: 普通 Lucas 只适用于质数模数。不要对含 p 因子的阶乘直接求逆；先记录 p 的指数，剩余部分才互素。模数为 1 的答案恒为 0 。

【最小完整示例（先抄这一段就能跑）】

题目：求 $C(20, 10) \bmod 1000000$ （模数是合数）。

```
i64 ans = binom_mod_composite(20, 10, 1000000); // 1. 调用: n、k、合数模数
cout << ans << '\n'; // 2. 取结果: C(20, 10) mod 1e6
```

样例： $\text{binom_mod_composite}(20, 10, 1000000) \rightarrow 184756$ （ $C(20, 10)=184756$ 本身 $< 1e6$ ）； $\text{binom_mod_composite}(8, 4, 6) \rightarrow 4$ （ $70 \bmod 6 = 4$ ）。

【传参要求（照这个传不会错）】

- $\text{binom_mod_composite}(n, k, \text{mod})$: n = 总数（ $\text{exlucas_ll} = i64$ ，可很大）； k = 选取数（要求 $0 \leq k \leq n$ ）； mod = 模数（ exlucas_ll ， ≥ 1 ，合数也行）。
- 返回 $C(n, k) \bmod \text{mod}$ ； $\text{mod} == 1$ 或 $k < 0$ 或 $k > n$ 时返回 0 。
- 前提： mod 的每个质数幂因子 p^e 都 $\leq \text{EXLUCAS_MAX_PRIME_POWER}$ （2000000），否则建表 `assert` 失败。
- `factorial_without_p`、`ExLucasPrimePower` 是内部 helper，不要从 `solve()` 直接调。

【API / 入口函数（赛时只认这里列的名字）】

- $\text{binom_mod_composite}(\text{exlucas_ll } n, \text{exlucas_ll } k, \text{exlucas_ll } \text{mod}) \rightarrow$ 要求 mod 的每个质数幂因子都不超过 $\text{EXLUCAS_MAX_PRIME_POWER}$ 。返回 exlucas_ll 。
- $\text{factorial_without_p}(\text{exlucas_ll } n) \rightarrow$ 把 $n!$ 中所有 p 因子剔除后，对 p^{exponent} 取模。返回 exlucas_ll 。

【改板时先认这几个量】

- prefix : $\text{prefix}[i]$ 表示 $1..i$ 中去掉 p 因子的乘积，模 pk 。
- pk : 当前素数幂模数 p^{exponent} ，本节运算都在该模下进行。

典型模型：大 n 小合数模的二项式计数；周期计数后对合数取模；容斥中的组合数模 10^k ；CRT 综合题。

```
using exlucas_ll = i64;
constexpr exlucas_ll EXLUCAS_MAX_PRIME_POWER = 2000000;

exlucas_ll exlucas_exgcd(exlucas_ll a, exlucas_ll b,
                        exlucas_ll& x, exlucas_ll& y) {
    if (b == 0) { x = 1; y = 0; return a; }
    exlucas_ll x1, y1;
    exlucas_ll g = exlucas_exgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

exlucas_ll exlucas_inv(exlucas_ll a, exlucas_ll mod) {
    exlucas_ll x, y;
    exlucas_ll g = exlucas_exgcd(a, mod, x, y);
```

```

    assert(g == 1);
    x %= mod;
    if (x < 0) x += mod;
    return x;
}

exlucas_ll exlucas_pow(exlucas_ll a, exlucas_ll e, exlucas_ll mod) {
    exlucas_ll r = 1 % mod;
    while (e) {
        if (e & 1) r = (exlucas_ll)((i128)r * a % mod);
        a = (exlucas_ll)((i128)a * a % mod);
        e >>= 1;
    }
    return r;
}

struct ExLucasPrimePower {
    exlucas_ll p, pk;
    int exponent;
    vector<exlucas_ll> prefix; // prefix[i] 表示 1..i 中去掉 p 因子的乘积, 模 pk。

    ExLucasPrimePower(exlucas_ll prime, int power) : p(prime), exponent(power) {
        pk = 1;
        for (int i = 0; i < exponent; ++i) pk *= p;
        // 表长为 pk+1; 只有确认内存足够后才能调大上界。
        assert(pk <= EXLUCAS_MAX_PRIME_POWER);
        prefix.assign((int)pk + 1, 1);
        for (int i = 1; i <= pk; ++i) {
            prefix[i] = prefix[i - 1];
            if (i % p != 0) prefix[i] = prefix[i] * i % pk;
        }
    }

    exlucas_ll valuation(exlucas_ll n) const {
        exlucas_ll ans = 0;
        while (n /= p, ans += 1) {}
    }

    // 把 n! 中所有 p 因子剔除后, 对 p^exponent 取模。
    exlucas_ll factorial_without_p(exlucas_ll n) const {
        if (n == 0) return 1;
        exlucas_ll ans = exlucas_pow(prefix[pk], n / pk, pk);
        ans = ans * prefix[n % pk] % pk;
        return ans * factorial_without_p(n / p) % pk;
    }

    exlucas_ll binom(exlucas_ll n, exlucas_ll k) const {
        if (k < 0 || k > n) return 0;
        exlucas_ll e = valuation(n) - valuation(k) - valuation(n - k);
        if (e >= exponent) return 0;
        exlucas_ll a = factorial_without_p(n);
        exlucas_ll b = factorial_without_p(k);
        exlucas_ll c = factorial_without_p(n - k);
        exlucas_ll ans = a * exlucas_inv(b, pk) % pk * exlucas_inv(c, pk) % pk;
        return ans * exlucas_pow(p, e, pk) % pk;
    }
};

// 要求 mod 的每个质数幂因子都不超过 EXLUCAS_MAX_PRIME_POWER。
exlucas_ll binom_mod_composite(exlucas_ll n, exlucas_ll k, exlucas_ll mod) {
    assert(mod >= 1);
    if (mod == 1 || k < 0 || k > n) return 0;
    exlucas_ll remain = mod, ans = 0, merged_mod = 1;
    for (exlucas_ll p = 2; p <= remain / p; ++p) {
        if (remain % p != 0) continue;
        int e = 0;
        exlucas_ll pk = 1;
        while (remain % p == 0) remain /= p, pk *= p, ++e;
        ExLucasPrimePower solver(p, e);
        exlucas_ll residue = solver.binom(n, k);
        exlucas_ll delta = (residue - ans) % pk;
        if (delta < 0) delta += pk;
        exlucas_ll t = delta * exlucas_inv(merged_mod % pk, pk) % pk;
        ans = (exlucas_ll)((i128)ans + (i128)merged_mod * t);
        merged_mod *= pk;
        ans %= merged_mod;
    }
    if (remain > 1) {
        ExLucasPrimePower solver(remain, 1);
        exlucas_ll pk = remain, residue = solver.binom(n, k);
        exlucas_ll delta = (residue - ans) % pk;
        if (delta < 0) delta += pk;
        exlucas_ll t = delta * exlucas_inv(merged_mod % pk, pk) % pk;
        ans = (exlucas_ll)((i128)ans + (i128)merged_mod * t);
    }
}

```

```
merged_mod *= pk;
ans %= merged_mod;
}
return ans;
}
```

Catalan 数

赛时先看

题目信号：方案数满足“不越过对角线”“任意前缀某类数量不少于另一类”。

本质：合法括号序列、栈出栈序列、凸多边形三角剖分、二叉树计数。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；优先调用下面 API 列出的入口，其余 helper 默认不要从 `solve()` 直接调。

复杂度判定：预处理组合数后 $O(1)$ 查询。

维护的量：无状态；只依赖 `comb.C` 的两次查询 $C(2n, n)$ 与 $C(2n, n+1)$ 。

警告：常见公式 $C(2n, n)/(n+1)$ ，模数非质数时不能直接除。

【最小完整示例（先抄这一段就能跑）】

题目：模 $1e9+7$ 下求第 5 个 Catalan 数（如 5 对括号的合法序列数）。

```
CombFast comb(10, 1000000007); // 1. 预处理到 2n=10 (组合数 O(1) 查询)
i64 ans = catalan(5, comb); // 2. 直接返回 Catalan(5)
cout << ans << '\n';
```

样例：`catalan(5) -> 42`。

【传参要求（照这个传不会错）】

- `catalan(n, comb)`: n 为 int（非负）；返回 $C(2n, n) - C(2n, n+1) \bmod \text{comb.mod}$ (i64)。
- `comb` 用「线性求逆元与阶乘逆元」的 `CombFast(2*n, mod)` 构造，`mod` 必须是质数；预处理上限要 $\geq 2n$ 。
- 等价公式 $C(2n, n)/(n+1)$ 需要模意义除法，非质数模数下本板也不能用，换其它计数方式。

【API / 入口函数（赛时只认这里列的名字）】

- `catalan(int n, const CombFast& comb) -> mod` 是质数，已有组合数 C 。返回 i64。

```
// mod 是质数，已有组合数 C。
i64 catalan(int n, const CombFast& comb) {
    return (comb.C(2 * n, n) - comb.C(2 * n, n + 1) + comb.mod) % comb.mod;
}
```

Stirling 数

赛时先看

题目信号：把 n 个不同元素分成 k 个非空无标号集合；或排列恰有 k 个环。

本质：集合划分、排列环数、容斥计数。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接调用本节列出的函数即可。

复杂度判定： $O(nk)$ 。

维护的量：`dp[i][j]` (i 个元素分成 j 块的方案数，二维表)；答案取 `dp[n][k]`。

警告：第二类 $S(n, k) = S(n-1, k-1) + k \cdot S(n-1, k)$ ；第一类无符号 $s(n, k) = s(n-1, k-1) + (n-1) \cdot s(n-1, k)$ 。

【最小完整示例（先抄这一段就能跑）】

题目：模 $1e9+7$ 下求第二类 Stirling 数 $S(5, 3)$ 与第一类无符号 $s(5, 3)$ 。

```
auto s2 = stirling2(5, 5, 1000000007LL); // 1. 调用：返回 (n+1) x (k+1) 的 dp 表
auto s1 = stirling1_unsigned(5, 5, 1000000007LL);
cout << s2[5][3] << ' ' << s1[5][3] << '\n'; // 2. 取结果：答案在 dp[n][k]
```

样例：`S(5, 3) -> 25`；`s(5, 3) -> 35`（均 mod $1e9+7$ ，原值小于模数）。

【传参要求（照这个传不会错）】

- `stirling2(n, k, mod) / stirling1_unsigned(n, k, mod)`: n 、 k 为 int（非负）；`mod` 为 i64 模数（本板只用加法乘法，合数模也能用）。
- 返回 `vector<vector<i64>>`，大小 $(n+1) \times (k+1)$ ；答案取 `dp[n][k]`（下标从 1 开始数，`dp[0][0]=1`）。
- $j > i$ 的位置恒为 0；复杂度 $O(nk)$ ， $n \cdot k$ 到 $1e7$ 内稳妥。

```
vector<vector<i64>> stirling2(int n, int k, i64 mod) {
```

```

vector dp(n + 1, vector<i64>(k + 1, 0));
dp[0][0] = 1;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= min(i, k); j++) {
        dp[i][j] = (dp[i - 1][j - 1] + dp[i - 1][j] * j) % mod;
    }
}
return dp;
}

vector<vector<i64>> stirling1_unsigned(int n, int k, i64 mod) {
    vector dp(n + 1, vector<i64>(k + 1, 0));
    dp[0][0] = 1;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= min(i, k); j++) {
            dp[i][j] = (dp[i - 1][j - 1] + dp[i - 1][j] * (i - 1)) % mod;
        }
    }
    return dp;
}

```

Burnside / Polya 枚举

赛时先看

题目信号：问“旋转相同算一种”“翻转相同算一种”“本质不同”。

本质：群作用下本质不同方案计数，如项链、手链、旋转翻转等价染色。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；优先调用下面 API 列出的入口，其余 helper 默认不要从 `solve()` 直接调。

复杂度判定：枚举群元素，通常 $O(|G| \log n)$ 。

维护的量：sum（各置换固定方案数累加）；无额外结构，跑一遍群元素即可。

警告：Burnside 是所有置换固定方案数的平均值；颜色数为 `c` 时，一个置换固定方案为 $c^{(\text{cycle_count})}$ 。

【最小完整示例（先抄这一段就能跑）】

题目：`n=4` 颗珠子、`c=2` 种颜色，旋转等价的不同项链数，模 `1e9+7`。

```

i64 ans = necklace_rotation_count(4, 2, 1000000007LL); // 1. 调用：旋转等价项链数
cout << ans << '\n'; // 2. 取结果：直接输出

```

样例：`necklace_rotation_count(4, 2, 1e9+7) -> 6` ($(16+2+4+2)/4=6$)。

【传参要求（照这个传不会错）】

- `necklace_rotation_count(n, c, mod)`: `n` = 珠子数 (int, ≥ 1)；`c` = 颜色数 (i64)；`mod` 必须是质数（末尾要乘 `n` 的逆元，走 $n^{(\text{mod}-2)}$ ）。
- 返回旋转等价的本质不同项链数（已对 `mod` 取模，i64）。
- 只处理旋转等价；要算翻转（二面体群）需自行枚举镜像置换，公式同理 $c^{(\text{cycle_count})}$ 。

【API / 入口函数（赛时只认这里列的名字）】

- `necklace_rotation_count(int n, i64 c, i64 mod)` -> `n` 个珠子，`c` 种颜色，旋转等价的项链数量。`mod` 为质数。返回 i64。

```

// 与通用辅助函数中的 mod_pow 功能相同，改名避免重名。
i64 mod_pow_burnside(i64 a, i64 e, i64 mod) {
    i64 r = 1 % mod;
    while (e) {
        if (e & 1) r = r * a % mod;
        a = a * a % mod;
        e >>= 1;
    }
    return r;
}

// n 个珠子，c 种颜色，旋转等价的项链数量。mod 为质数。
i64 necklace_rotation_count(int n, i64 c, i64 mod) {
    i64 sum = 0;
    for (int shift = 0; shift < n; shift++) {
        sum = (sum + mod_pow_burnside(c, std::gcd(n, shift), mod)) % mod;
    }
    return sum * mod_pow_burnside(n, mod - 2, mod) % mod;
}

```

Pruefer 序列：带标号树编码、解码与凯莱公式

赛时先看

题目信号：题面是带标号树计数；每次删编号最小叶子并记录其邻点；给一串长度 $n-2$ 的数要求还原树；或给森林各连通块大小，问添加 $k-1$ 条边后连通的方案数。

本质：把 n 个带标号点的树双射为一个长度 $n-2$

的整数序列。除了树编码/还原，还可直接推出凯莱公式和“把多个连通块加边连成一棵树”的计数。

接法：Luogu P6086 Pruefer 序列；凯莱公式；森林连通方案计数；随机生成带标号树。

复杂度判定：这里使用小根堆，编码与解码都是 $O(n \log n)$ ，空间 $O(n)$ 。点从 0 编号。

维护的量：degree（各点当前度数）；leaves（编号最小的叶子小根堆）；code/edges（Pruefer 序列/边集）。

警告：编码与解码都是“每次取编号最小的叶子”；Pruefer 序列长度是 $n-2$ ，不是 $n-1$ 。完全图 K_n 的生成树数为 $n^{(n-2)}$ ；有 $k \geq 2$ 个大小为 s_i 的连通块时，加 $k-1$ 条边恰好连通的方案数是 $n^{(k-2)} * \text{product}(s_i)$ 。

【最小完整示例（先抄这一段就能跑）】

题目：给 4 点星形树（中心 0 连 1、2、3）求 Pruefer 序列，再把序列还原成边集。

```
vector<vector<int>> g = {{1, 2, 3}, {0}, {0}, {0}}; // 邻接表，点从 0 编号
vector<int> code = prufer_encode(g); // 1. 调用：编码，长度 n-2
auto edges = prufer_decode(code); // 2. 调用：解码回边集
```

样例：prufer_encode(4 点星形树) -> {0, 0}; prufer_decode({0, 0}) -> 3 条边。

【传参要求（照这个传不会错）】

- prufer_encode(g): g = 树的无向邻接表 $\text{vector}<\text{vector}<\text{int}>>$ ，点从 0 编号； $n \leq 2$ 返回空序列；返回长度 $n-2$ 的 $\text{vector}<\text{int}>$ 。
- prufer_decode(code): code = Pruefer 序列；点数自动取 code.size()+2 ；返回 $n-1$ 条边的 $\text{vector}<\text{pair}<\text{int}, \text{int}>>$ （顺序不定）。
- 编码/解码都要求输入是一棵树（无环连通）；复杂度 $O(n \log n)$ 。

【API / 入口函数（赛时只认这里列的名字）】

- prufer_decode(const $\text{vector}<\text{int}>\& \text{code}$) -> 输入 Pruefer 序列，返回原树的边集。
- prufer_encode(const $\text{vector}<\text{vector}<\text{int}>>\& g$) -> 输入一棵树的邻接表，返回其 Pruefer 序列（点从 0 编号）。

【改板时先认这几个量】

- parent: 编码时叶子唯一剩下的邻点（度仍大于 0 的那个）。
- g: 邻接表。

典题模型：Luogu P6086 Pruefer 序列；凯莱公式；森林连通方案计数；随机生成带标号树。

```
// 输入一棵树的邻接表，返回其 Pruefer 序列（点从 0 编号）。
vector<int> prufer_encode(const vector<vector<int>>& g) {
    const int n = (int)g.size();
    if (n <= 2) return {};
    vector<int> degree(n);
    priority_queue<int, vector<int>, greater<int>> leaves;
    for (int u = 0; u < n; ++u) {
        degree[u] = (int)g[u].size();
        if (degree[u] == 1) leaves.push(u);
    }

    vector<int> code;
    code.reserve(n - 2);
    for (int step = 0; step < n - 2; ++step) {
        int leaf = leaves.top(); leaves.pop();
        int parent = -1;
        for (int v : g[leaf]) if (degree[v] > 0) {
            parent = v;
            break;
        }
        code.push_back(parent);
        degree[leaf] = 0;
        if (--degree[parent] == 1) leaves.push(parent);
    }
    return code;
}

// 输入 Pruefer 序列，返回原树的边集。
vector<pair<int, int>> prufer_decode(const vector<int>& code) {
    const int n = (int)code.size() + 2;
    vector<int> degree(n, 1);
    for (int v : code) ++degree[v];

    priority_queue<int, vector<int>, greater<int>> leaves;
    for (int v = 0; v < n; ++v) if (degree[v] == 1) leaves.push(v);
```

```

vector<pair<int, int>> edges;
edges.reserve(n - 1);
for (int v : code) {
    int leaf = leaves.top(); leaves.pop();
    edges.push_back({leaf, v});
    degree[leaf] = 0;
    if (--degree[v] == 1) leaves.push(v);
}
int a = leaves.top(); leaves.pop();
int b = leaves.top();
edges.push_back({a, b});
return edges;
}

```

扩展中国剩余定理 exCRT

赛时先看

题目信号：多个周期同步、多个模数约束。

本质：解多个同余方程 $x \equiv a_i \pmod{m_i}$ ，模数可不互质。

接法：把每个条件写成 $x \equiv a[i] \pmod{m[i]}$ ，全部放进数组后调用 `exCRT(a,m,ans,mod)`。返回 `false` 就无解；返回 `true` 时通解是 $x \equiv \text{ans} \pmod{\text{mod}}$ ，题目要最小正解时若 `ans==0` 通常输出 `mod`。

复杂度判定： $O(k \log M)$ 。

维护的量：`ans`（已合并部分的同余剩余）、`mod`（已合并部分的模数 lcm）；每并入一个方程更新一次。

警告：无解条件是 $(a_2 - a_1) \% \gcd(m_1, m_2) \neq 0$ 。合并时中间乘法可能超过 `i64`，用 `i128`。

【最小完整示例（先抄这一段就能跑）】

题目：解同余组 $x \equiv 2 \pmod{3}$ 且 $x \equiv 3 \pmod{5}$ 。

```

vector<i64> a = {2, 3}, m = {3, 5};
i64 ans, mod;
if (exCRT(a, m, ans, mod)) {
    cout << ans << ' ' << mod << '\n'; // 通解 x ≡ ans (mod mod)
} else {
    cout << -1 << '\n'; // 无解
}

```

样例：`exCRT({2,3},{3,5}) -> ans=8, mod=15`（8 是最小非负解）。

【传参要求（照这个传不会错）】

- `exCRT(a, m, ans, mod)`: `a/m` 为等长 `vector<i64>`（第 `i` 个方程 $x \equiv a[i] \pmod{m[i]}$ ，`m[i] >= 1`，不要求互质）。
- 返回 `bool`: `true` 时 `ans` 为最小非负解、`mod` 为合并后的模数（lcm），通解 $x \equiv \text{ans} \pmod{\text{mod}}$ ；`false` 无解。
- 求最小正整数解：若 `ans == 0` 输出 `mod`；合并过程内部乘法用 `i128`，不会溢出。

```

bool crt_merge(i64 a1, i64 m1, i64 a2, i64 m2,
               i64& a, i64& m) {
    i64 x, y;
    i64 g = exgcd(m1, m2, x, y);
    if ((a2 - a1) % g != 0) return false;
    i64 mod = m2 / g;
    i64 t = (i128)((a2 - a1) / g) * x % mod;
    i64 lcm = m1 / g * m2;
    a = (a1 + (i128)m1 * t) % lcm;
    if (a < 0) a += lcm;
    m = lcm;
    return true;
}

bool exCRT(const vector<i64>& a, const vector<i64>& m,
           i64& ans, i64& mod) {
    ans = 0;
    mod = 1;
    for (int i = 0; i < (int)a.size(); ++i) {
        if (!crt_merge(ans, mod, a[i], m[i], ans, mod)) return false;
    }
    return true;
}

```

线性同余：模 2^{64} 最小解

赛时先看

题目信号：64 位无符号整数自然溢出；模数是 2^{64} ；要求最小 `x`。

本质：求 $a \cdot x = b \pmod{2^{64}}$ 的最小非负解，或判断无解。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接调用本节列出的函数即可。

复杂度判定：常数级（约 6 轮 Newton 迭代，每轮正确位数翻倍）。

维护的量：`v`（`a` 中因子 2 的个数）、`odd`（约掉 2 后的奇数部分）、`res`（最小非负解）。

警告：`a=0` 单独处理；令 `g=lowbit(a)`，必须有 `g | b`；除掉 `g` 后，`a/g` 是奇数，可以在 $2^{(64-v2(g))}$ 下求逆。

【最小完整示例（先抄这一段就能跑）】

题目：求 $3x \equiv 6 \pmod{2^{64}}$ 的最小非负解（ui64 全程自然溢出运算）。

```
auto r = solve_linear_congruence_mod_2_64(3, 6);
if (r) cout << *r << '\n'; // 有解：输出最小非负解
else cout << "no solution\n"; // 无解
```

样例：`solve_linear_congruence_mod_2_64(3, 6) -> 2`；`solve_linear_congruence_mod_2_64(4, 3) -> 无解`。

【传参要求（照这个传不会错）】

- `solve_linear_congruence_mod_2_64(a, b)`：`a`、`b` 为 ui64（无符号 64 位，模数固定为 2^{64} ，不要传负数）。
- 返回 `optional<ui64>`：有解时为最小非负解（ $[0, 2^{64})$ 内）；`nullopt` 无解。
- `a = 0` 时只有 `b = 0` 有解（返回 0）；一般情形先看 `lowbit(a) | b` 是否成立，不成立即无解。

```
ui64 inverse_odd_mod_2_64(ui64 a) {
    // a 必须是奇数；Newton 迭代每轮让正确二进制位数翻倍。
    ui64 x = 1;
    for (int i = 0; i < 6; i++) x *= 2 - a * x;
    return x;
}

optional<ui64> solve_linear_congruence_mod_2_64(ui64 a, ui64 b) {
    if (a == 0) {
        if (b == 0) return 0;
        return nullopt;
    }
    int v = __builtin_ctzll(a);
    if (v > 0) {
        ui64 low_mask = (1ULL << v) - 1;
        if ((b & low_mask) != 0) return nullopt;
    }
    ui64 odd = a >> v;
    ui64 rhs = b >> v;
    ui64 res = inverse_odd_mod_2_64(odd) * rhs;
    int bits = 64 - v;
    if (bits < 64) res &= (1ULL << bits) - 1;
    return res; // 在  $[0, 2^{64})$  的所有解中取最小值。
}
```

AtCoder floor_sum

赛时先看

题目信号：整除求和、格点计数、坐标压缩后的数学求和。

本质：计算 $\sum_{i=0}^{n-1} \text{floor}((a \cdot i + b)/m)$ 。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接调用本节列出的函数即可。

复杂度判定： $O(\log \max(a, m))$ 。

维护的量：`ans`（答案累加）；循环中维护 $n/m/a/b$ 的对称化简（取模 + 交换）。

警告：这是 AtCoder Library 经典函数，参数可为非负整数。

【最小完整示例（先抄这一段就能跑）】

题目：求 $\sum_{i=0}^4 \text{floor}((2i+1)/3)$ 。

```
i64 ans = floor_sum(4, 3, 2, 1); // floor_sum(n, m, a, b): i 取 0..n-1
cout << ans << '\n';
```

样例：`floor_sum(4, 3, 2, 1) -> 4`（ $0+1+1+2$ ）。

【传参要求（照这个传不会错）】

- `floor_sum(n, m, a, b)`：`n` = 项数（`i` 取 $0..n-1$ ）；`m` = 模数（ ≥ 1 ）；`a`、`b` = 一次式系数。
- 全部为 i64 非负整数；返回 $\sum_{i=0}^{n-1} \text{floor}((a \cdot i + b)/m)$ （i64）。
- 复杂度 $O(\log \max(a, m))$ ；代码内部会先把 `a`、`b` 对 `m` 归约，不用手动处理。

```
i64 floor_sum(i64 n, i64 m, i64 a, i64 b) {
```



```

i64 ans = 0;
while (true) {
    if (a >= m) {
        ans += (n - 1) * n * (a / m) / 2;
        a %= m;
    }
    if (b >= m) {
        ans += n * (b / m);
        b %= m;
    }
    i64 y_max = a * n + b;
    if (y_max < m) break;
    n = y_max / m;
    b = y_max % m;
    swap(m, a);
}
return ans;
}

```

BSGS 离散对数

赛时先看

题目信号：指数在模意义下作为未知量。

本质：求最小非负整数 x 使 $a^x \equiv b \pmod{\text{mod}}$ 。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接调用本节列出的函数即可。

复杂度判定： $O(\sqrt{\text{mod}})$ 。

维护的量： m (`ceil(sqrt(mod))` 分块大小)；`table` (前段 a^j 出现的最小 j 哈希表)；`cur` (后段候选值，每次乘 `inv_factor` 前进)。

警告：基础版要求 $\text{gcd}(a, \text{mod})=1$ ；非互质要用 `exBSGS`。

【最小完整示例（先抄这一段就能跑）】

题目：求 $2^x \equiv 7 \pmod{13}$ 的最小非负整数解。

```

i64 x = bsgs(2, 7, 13); // 1. 调用：求  $a^x \equiv b \pmod{\text{mod}}$  的最小非负  $x$ 
cout << x << '\n'; // 2. 取结果：-1 表示无解

```

样例：`bsgs(2, 7, 13) -> 11` ($2^{11} = 2048 \equiv 7 \pmod{13}$)；`bsgs(2, 3, 13) -> 4` ($2^4 = 16 \equiv 3 \pmod{13}$)。

【传参要求（照这个传不会错）】

- `bsgs(a, b, mod)`： a = 底数、 b = 目标值、 mod = 模数，均为 `i64`， $\text{mod} \geq 1$ ；内部先做 $a \% \text{mod}$ 、 $b \% \text{mod}$ 。
- 返回最小非负整数 x 使 $a^x \equiv b \pmod{\text{mod}}$ ；无解返回 `-1`。
- 前提：必须 $\text{gcd}(a, \text{mod}) == 1$ (后半段要求逆)； $\text{mod} == 1$ 或 $b \equiv 1 \pmod{\text{mod}}$ 时返回 `0`。
- 不互质时翻下一节 `exbsgs`；依赖 `mod_pow_bsgs` 与 `inv_general`，抄板时一起抄。

```

i64 mod_pow_bsgs(i64 a, i64 b, i64 mod) {
    i64 res = 1 % mod;
    a %= mod;
    while (b) {
        if (b & 1) res = (i128)res * a % mod;
        a = (i128)a * a % mod;
        b >>= 1;
    }
    return res;
}

i64 bsgs(i64 a, i64 b, i64 mod) {
    a %= mod;
    b %= mod;
    if (mod == 1) return 0;
    if (b == 1 % mod) return 0;

    i64 m = (i64)ceil(sqrt((long double)mod));
    unordered_map<i64, i64> table;
    i64 e = 1;
    for (i64 j = 0; j < m; ++j) {
        if (!table.count(e)) table[e] = j;
        e = (i128)e * a % mod;
    }

    i64 factor = mod_pow_bsgs(a, m, mod);
    i64 inv_factor = inv_general(factor, mod);
    if (inv_factor == -1) return -1;

    i64 cur = b;
    for (i64 i = 0; i <= m; ++i) {

```

```

    if (table.count(cur)) {
        i64 x = i * m + table[cur];
        if (mod_pow_bsgs(a, x, mod) == b) return x;
    }
    cur = (i128)cur * inv_factor % mod;
}
return -1;
}

```

exBSGS

赛时先看

题目信号：离散对数，模数不保证质数。

本质：求 $a^x \equiv b \pmod{m}$ ，允许 a 与 m 不互质。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接调用本节列出的函数即可。

复杂度判定： $O(\sqrt{m})$ 。

维护的量：`k/add`（约分阶段累积的系数与已加步数）；`table`（BSGS 前段哈希表）；`cur`（每次乘 `inv_factor` 前进的候选值）。

警告：先不断约掉 `gcd(a,m)`；约不掉后转普通 BSGS。

【最小完整示例（先抄这一段就能跑）】

题目：求 $2^x \equiv 4 \pmod{6}$ 的最小非负解（ a 与模数不互质）。

```

i64 x = exbsgs(2, 4, 6); // 1. 调用：自动先约 gcd(a,m) 再转 BSGS
cout << x << '\n'; // 2. 取结果：-1 表示无解

```

样例：`exbsgs(2, 4, 6) -> 2` ($2^2=4$)；`exbsgs(5, 3, 8) -> -1`（无解）。

【传参要求（照这个传不会错）】

- `exbsgs(a, b, mod)`：`a`、`b`、`mod` 为 `i64`，`mod >= 1`；内部先 `a %= mod`。
- 返回最小非负整数 x 使 $a^x \equiv b \pmod{mod}$ ；无解返回 `-1`。
- `mod == 1 || b == 1` 时返回 `0`；`gcd(a, mod) == 1` 的场景也可直接用同节 `bsgs_coprime`。

【改板时先认这几个量】

- `g`：每轮约分时的 `gcd(a, mod)`，`b % g != 0` 即无解。
- `cur`：BSGS 中每次乘 `inv_factor` 前进的当前候选值。

```

i64 bsgs_coprime(i64 a, i64 b, i64 mod) {
    a %= mod; b %= mod;
    if (b == 1 % mod) return 0;
    i64 m = (i64)ceil(sqrt((long double)mod));
    unordered_map<i64, i64> table;
    i64 e = 1;
    for (i64 j = 0; j < m; ++j) {
        if (!table.count(e)) table[e] = j;
        e = (i128)e * a % mod;
    }
    i64 factor = mod_pow(a, m, mod);
    i64 inv_factor = inv_general(factor, mod);
    i64 cur = b;
    for (i64 i = 0; i <= m; ++i) {
        if (table.count(cur)) return i * m + table[cur];
        cur = (i128)cur * inv_factor % mod;
    }
    return -1;
}

i64 exbsgs(i64 a, i64 b, i64 mod) {
    a %= mod; b %= mod;
    if (mod == 1 || b == 1) return 0;
    i64 k = 1, add = 0, g;
    while ((g = gcd(a, mod)) > 1) {
        if (b % g) return -1;
        b /= g;
        mod /= g;
        k = (i128)k * (a / g) % mod;
        add++;
        if (k == b) return add;
    }
    i64 inv_k = inv_general(k, mod);
    i64 res = bsgs_coprime(a, (i128)b * inv_k % mod, mod);
    return res == -1 ? -1 : res + add;
}

```

原根 Primitive Root

赛时先看

题目信号：需要模质数的生成元。

本质：NTT、自定义质数模下求原根。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接调用本节列出的函数即可。

复杂度判定：试候选，通常很快。

维护的量：`factors` (`mod-1` 的不同质因子表)；`x` (试除后的剩余部分)；`g` (从小到大的候选原根)。

警告：只适用于质数模。

【最小完整示例（先抄这一段就能跑）】

题目：NTT 前求质数 998244353 的原根。

```
i64 g = primitive_root(998244353); // 1. 调用：mod 传质数
cout << g << '\n'; // 2. 取结果：该质数的最小原根
```

样例：`primitive_root(998244353) -> 3` (NTT 常用模数的原根)；`primitive_root(7) -> 3` ($3^1..3^6$ 恰好覆盖 $1..6$)。

【传参要求（照这个传不会错）】

- `primitive_root(mod)`：mod 必须是质数 (`i64, mod >= 2`)。
- 返回最小原根 `g` ($2 \leq g < \text{mod}$ ；`mod == 2` 时返回 1)，满足 $g^1..g^{(\text{mod}-1)}$ 恰好覆盖全部非零剩余。
- `mod` 为合数时本板不适用；依赖 `mod_pow` (快速幂)，抄板时一起抄。

```
i64 primitive_root(i64 mod) {
    if (mod == 2) return 1;
    vector<i64> factors;
    i64 x = mod - 1;
    for (i64 p = 2; p * p <= x; ++p) {
        if (x % p == 0) {
            factors.push_back(p);
            while (x % p == 0) x /= p;
        }
    }
    if (x > 1) factors.push_back(x);
    for (i64 g = 2; ; ++g) {
        bool ok = true;
        for (i64 p : factors) {
            if (mod_pow(g, (mod - 1) / p, mod) == 1) {
                ok = false;
                break;
            }
        }
        if (ok) return g;
    }
}
```

二次剩余 Tonelli-Shanks

赛时先看

题目信号：模意义开平方；多项式开方、几何模运算、数论构造。

本质：求 $x^2 \equiv n \pmod{p}$ 的解，`p` 为奇质数。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接调用本节列出的函数即可。

复杂度判定： $O(\log^2 p)$ 。

维护的量：`q/s` ($p-1 = 2^s * q$ 的拆分)；`z` (非二次剩余)、`c`、`x` (当前根)、`t`、`m` (迭代变量)。

警告：先用欧拉判别检查是否有解；`p % 4 == 3` 有快捷公式。

【最小完整示例（先抄这一段就能跑）】

题目：求 $x^2 \equiv 5 \pmod{41}$ 的一个根。

```
i64 x = tonelli_shanks(5, 41); // 1. 调用：求  $x^2 \equiv n \pmod{p}$  的一个解
cout << x << '\n'; // 2. 取结果：-1 表示 n 不是二次剩余
```

样例：`tonelli_shanks(5, 41) -> 13` ($13^2=169 \equiv 5 \pmod{41}$ ，另一个根是 28)。

【传参要求（照这个传不会错）】

- `tonelli_shanks(n, p)`：`n` = 被开方数 (`i64`，非负)；`p` 必须是奇质数 (`i64`，`p != 2`)。

- 返回满足 $x^2 \equiv n \pmod{p}$ 的一个解，另一个解是 $p-x$ ； n 非二次剩余返回 -1 （欧拉判别已内置）。
- $n == 0$ 返回 0 ； $p \% 4 == 3$ 走快捷公式 $n^{(p+1)/4}$ ； p 为合数时本板不适用。

```
i64 qpow_mod(i64 a, i64 e, i64 mod) {
    i64 r = 1 % mod;
    while (e) {
        if (e & 1) r = (i128)r * a % mod;
        a = (i128)a * a % mod;
        e >>= 1;
    }
    return r;
}

i64 tonelli_shanks(i64 n, i64 p) {
    n %= p;
    if (n == 0) return 0;
    if (p == 2) return n;
    if (qpow_mod(n, (p - 1) / 2, p) != 1) return -1;
    if (p % 4 == 3) return qpow_mod(n, (p + 1) / 4, p);
    i64 q = p - 1, s = 0;
    while ((q & 1) == 0) q >>= 1, s++;
    i64 z = 2;
    while (qpow_mod(z, (p - 1) / 2, p) != p - 1) z++;
    i64 c = qpow_mod(z, q, p);
    i64 x = qpow_mod(n, (q + 1) / 2, p);
    i64 t = qpow_mod(n, q, p);
    i64 m = s;
    while (t != 1) {
        i64 i = 1, tt = (i128)t * t % p;
        while (tt != 1) {
            tt = (i128)tt * tt % p;
            i++;
        }
        i64 b = qpow_mod(c, 1LL << (m - i - 1), p);
        x = (i128)x * b % p;
        c = (i128)b * b % p;
        t = (i128)t * c % p;
        m = i;
    }
    return x; // 另一个解是 p-x
}
```

Miller-Rabin 素性测试

赛时先看

题目信号： n 可到 $1e18$ ，不能筛。

本质：判断 $i64$ 范围内大数是否为质数。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接调用本节列出的函数即可。

复杂度判定： $O(\log n * \text{常数})$ 。

维护的量： d/s （把 $n-1$ 拆成 $2^s * d$ ）； a （固定测试底数表，覆盖全部 64 位整数）； x （ $a^d \bmod n$ 迭代平方的中间值）。

警告：固定测试底数可覆盖 64 位整数。

【最小完整示例（先抄这一段就能跑）】

题目：判断大数 n 是否为质数（ n 太大不能筛）。

```
bool ok = is_prime_u64(1000000000000000009ULL); // 1. 调用: n 传 ui64 (无符号)
cout << (ok ? "prime" : "composite") << '\n'; // 2. 取结果: true = 质数
```

样例：`is_prime_u64(1000000007) -> true` ($1e9+7$ 是质数)；`is_prime_u64(1000000008) -> false` (偶数)。

【传参要求（照这个传不会错）】

- `is_prime_u64(n)`： n = 待判数（`ui64`，范围 $0 \sim 2^{64}-1$ ；负数不要传）。
- 返回 `bool`：`true` 表示质数； $n < 2$ 返回 `false`。
- 固定 7 组测试底数即可覆盖全部 64 位整数，无需随机底数。
- 依赖 `mod_mul_u64`（`u128` 防溢出乘法）与 `mod_pow_u64`，抄板时一起抄；只从 `solve()` 调 `is_prime_u64`。

```
using u128 = __uint128_t;

ui64 mod_mul_u64(ui64 a, ui64 b, ui64 mod) {
    return (u128)a * b % mod;
}

ui64 mod_pow_u64(ui64 a, ui64 b, ui64 mod) {
```

```

    ui64 res = 1;
    while (b) {
        if (b & 1) res = mod_mul_u64(res, a, mod);
        a = mod_mul_u64(a, a, mod);
        b >>= 1;
    }
    return res;
}

bool is_prime_u64(ui64 n) {
    if (n < 2) return false;
    for (ui64 p : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL, 19ULL, 23ULL, 29ULL, 31ULL, 37ULL}) {
        if (n % p == 0) return n == p;
    }
    ui64 d = n - 1, s = 0;
    while ((d & 1) == 0) d >>= 1, s++;
    for (ui64 a : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 179526502ULL}) {
        if (a % n == 0) continue;
        ui64 x = mod_pow_u64(a, d, n);
        if (x == 1 || x == n - 1) continue;
        bool ok = false;
        for (ui64 r = 1; r < s; ++r) {
            x = mod_mul_u64(x, x, n);
            if (x == n - 1) {
                ok = true;
                break;
            }
        }
        if (!ok) return false;
    }
    return true;
}

```

Pollard-Rho 因式分解

赛时先看

题目信号: $n \leq 1e18$, 需要质因数分解, 试除太慢。

本质: 分解 $i64$ 范围大整数。

接法: 把本节 struct/class/函数组 整段抄到 `solve()` 上面; `solve()` 里直接调用本节列出的函数即可。

复杂度判定: 期望较快。

维护的量: `fac` (结果容器, 递归填入质因子); `rng` (随机源); `d` (Floyd 判圈找到的非平凡因子)。

警告: 随机算法, 失败时换参数重试; 先 Miller-Rabin 判断质数。

【最小完整示例 (先抄这一段就能跑)】

题目: 分解 $n = 8051$ ($= 83 \times 97$) 的全部质因子。

```

vector<ui64> fac;
factor_u64(8051, fac);           // 1. 调用: 质因子递归填入 fac (无序)
sort(fac.begin(), fac.end());    // 2. 按需排序后输出
for (ui64 p : fac) cout << p << ' ';

```

样例: `factor_u64(8051, fac) -> {83, 97}`。

【传参要求 (照这个传不会错)】

- `factor_u64(n, fac)`: n 为 `ui64` (质数、合数都行); `fac` 传空 `vector<ui64>`, 函数把质因子 (含重复) 追加进去, 顺序不定。
- `pollard_rho(n)`: 内部用, n 必须是合数 (质数请先过 `is_prime_u64`); 返回一个非平凡因子。
- 依赖上节 Miller-Rabin 的 `is_prime_u64`、`mod_mul_u64`; n 为 1 时 `factor_u64` 直接返回。
- 期望复杂度亚指数级, $n \leq 1e18$ 最稳; 极少数情况随机失败, 多跑一次即可。

```

mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

ui64 pollard_rho(ui64 n) {
    if (n % 2 == 0) return 2;
    if (n % 3 == 0) return 3;
    while (true) {
        ui64 c = uniform_int_distribution<ui64>(1, n - 1)(rng);
        ui64 x = uniform_int_distribution<ui64>(0, n - 1)(rng);
        ui64 y = x, d = 1;
        auto f = [&](ui64 v) { return (mod_mul_u64(v, v, n) + c) % n; };
        while (d == 1) {
            x = f(x);
            y = f(f(y));
            ui64 diff = x > y ? x - y : y - x;
            d = gcd(diff, n);
        }
    }
}

```

```

    }
    if (d != n) return d;
}
}

void factor_u64(ui64 n, vector<ui64>& fac) {
    if (n == 1) return;
    if (is_prime_u64(n)) {
        fac.push_back(n);
        return;
    }
    ui64 d = pollard_rho(n);
    factor_u64(d, fac);
    factor_u64(n / d, fac);
}

```

11 线性代数、多项式与生成函数

矩阵快速幂、线性方程、马尔可夫期望、行列式、插值、FFT/NTT/FWT、形式幂级数和线性递推放在这里。

矩阵快速幂

赛时先看

题目信号：题面出现“线性递推/状态转移重复 K 次”且 K 可达 $1e18$ ，如 Fibonacci 第 n 项、固定规则的转移计数、马尔可夫过程期望；每一步转移规则完全相同。看到“大次数幂 + 固定转移”就先想矩阵快速幂。

本质：把“走一步”写成一次矩阵乘法：状态向量 `state` 按列向量存，转移矩阵 `A` 满足 `state_next = A * state`；走 K 步就是乘 A^K ，快速幂把 K 次乘法压成 $O(\log K)$ 次矩阵乘法。

复杂度判定： $O(k^3 \log n)$ ， k 是状态数； $k \leq 100$ 、 $n \leq 1e18$ 时约 60 轮矩阵乘，稳； k 到 300+ 需考虑稀疏矩阵或线性递推优化（BM / 特征多项式）。

维护的量：矩阵元素 `a[i][j]`（0-based）；列向量约定 `state_next = A * state; mod 模数`。

接法：只要转移矩阵每一步都一样，且要跳很多步，就可以矩阵快速幂。先确定状态向量顺序，例如 Fibonacci 用 `[F_i, F_{i-1}]`；再写矩阵让它乘一次等于走一步；最后算 `mat_pow(transition, steps) * initial_vector`。

警告：初始向量顺序要和矩阵定义一致（列向量约定下是 `state_next = A * state`）；`A` 必须是方阵才能做 `mat_pow(A, k)`；指数 $k=0$ 返回单位矩阵。

【最小完整示例（先抄这一段就能跑）】

题目：求 Fibonacci 第 n 项 mod $1e9+7$ ($n \leq 1e18$)。

```

Matrix A(2, 2, mod); // 1. 结构体定义: Matrix(行, 列, 模数)
A.a[0][0] = 1; A.a[0][1] = 1; // 转移矩阵: state_{i+1} = A * state_i
A.a[1][0] = 1; A.a[1][1] = 0; // state_i = [F_i; F_{i-1}]
Matrix P = mat_pow(A, n); // 2. 调用: 矩阵快速幂 A^n
cout << P.a[1][0] << '\n'; // 3. 取结果: F_n (设 F_1 = 1, F_0 = 0)

```

样例： $n = 10 \rightarrow$ 输出 55。

【传参要求（照这个传不会错）】

- `Matrix(n, m, mod)`：构造全 0 矩阵；`mat_pow(A, k)` 要求 `A` 是方阵。
- 列向量约定：状态 `state_next = A * state`；写转移矩阵前先固定状态向量顺序。
- `A.a[i][j]`：0-based 下标访问元素； $k = 0$ 返回单位矩阵。
- 模数任意，乘法内部用 `i64` ($\text{mod} \leq 1e9$ 时相乘不爆)。
- 转移“每步相同 + 步数巨大 ($1e18$)”的递推/计数都能套这个思路。

【不会用就照抄】

```

Matrix A(...);
Matrix R = mat_pow(A, k);

```

- 这份 `Matrix` 是普通矩阵乘法，常用约定是 列向量：`state_next = A * state`。若你自己改成行向量，转移矩阵要整体转置思考。
- `A` 必须是方阵才能做 `mat_pow(A, k)`；指数 $k=0$ 返回单位矩阵。

【API / 入口函数（赛时只认这里列的名字）】

- `Matrix(n, m, mod) \rightarrow` 建立 $n \times m$ 零矩阵。
- `A.a[i][j]` $\rightarrow$ 直接填写矩阵元素，内部下标 0-based。

- `mat_pow(A,k)` -> 求方阵 A^k ; $k=0$ 返回单位阵。

【抄板清单（照着做就行）】

1. 抄哪段：整个 `Matrix` 结构体 + `mat_pow` 函数。
2. 构造：`Matrix A(n, n, mod);` 建 n 阶方阵；`Matrix init(n, 1, mod)` 建列向量初值。
3. 填矩阵：`A.a[i][j] = x;`，按“`state_next[i]` 由哪些 `state[j]` 贡献”来填。
4. 调用：`Matrix R = mat_pow(A, k) * init;`，答案在 `R.a[i][0]`。

【改造点（按题目改这几处）】

- 模数：构造矩阵时传题面模数（非质数也能用，矩阵乘法只用加乘取模，不需要逆元）。
- 递推系数转矩阵：如 $F_i = c_1 * F_{i-1} + c_2 * F_{i-2}$ ，第一行填 `[c1, c2]`，其余行做“移位”（`a[i][i-1]=1`）。
- 加速递推的矩阵构造：状态含常数项、前缀和、多维 DP 时，把所有需要量都塞进状态向量，让每一步只是线性组合。
- “恰好 K 步”与“至多 K 步”：矩阵构造不同，后者常加吸收状态；带 `min/max` 的“最短路式”转移翻 C 章 `min-plus` 矩阵快速幂。
- 行向量习惯：若坚持行向量，转移矩阵整体转置思考，别和列向量混用。

【核心逻辑（改代码时别破坏）】

- 固定线性转移写成矩阵 A 后，走 k 步就是乘 A^k 。
- 快速幂逻辑与普通整数幂完全一样，只是乘法换成矩阵乘法。

【改板时先认这几个量】

- `a[i][j]`：矩阵第 i 行第 j 列（0-based）； n/m 行列数；`mod` 取模数。
- 乘法内层跳过 `a[i][k]==0`，稀疏矩阵时更快。

```
// 维护的量：a[i][j] 矩阵元素（0-based）；n/m 行列数；mod 模数。
// 约定：列向量 state_next = A * state; mat_pow(A,k) 返回  $A^k$ ,  $k=0$  时为单位阵。
struct Matrix {
    int n, m;
    i64 mod;
    vector<vector<i64>>> a;

    Matrix(int n_, int m_, i64 mod_, bool ident = false)
        : n(n_), m(m_), mod(mod_), a(n_, vector<i64>(m_, 0)) {
        if (ident) {
            // 单位阵：对角线置 1，作快速幂的初始结果
            for (int i = 0; i < min(n, m); ++i) a[i][i] = 1 % mod;
        }
    }

    static Matrix identity(int n, i64 mod) {
        return Matrix(n, n, mod, true);
    }

    Matrix operator*(const Matrix& o) const {
        Matrix res(n, o.m, mod);
        for (int i = 0; i < n; ++i) {
            for (int k = 0; k < m; ++k) if (a[i][k]) {
                for (int j = 0; j < o.m; ++j) {
                    // 第 i 行乘第 k 列累加进 (i, j); i128 防中间乘爆
                    res.a[i][j] = (res.a[i][j] + (i128)a[i][k] * o.a[k][j]) % mod;
                }
            }
        }
        return res;
    }
};

Matrix mat_pow(Matrix base, i64 exp) {
    Matrix res = Matrix::identity(base.n, base.mod); // 结果初始化为单位阵
    while (exp) {
        if (exp & 1) res = res * base; // 二进制位为 1 时累乘进结果
        base = base * base; // base 自乘，对应指数翻倍
        exp >>= 1;
    }
    return res;
}
```

有限状态转移矩阵快速幂

赛时先看

题目信号： n 可达 $1e9$ 或更大；转移只依赖有限个状态；每一步规则相同。

本质：状态数小、步数巨大时，用矩阵快速幂推进 DP 或自动机。

复杂度判定： $O(S^3 \log n)$ ， S 是状态数。

维护的量：MatVV 即 `vector<vector<i64>>`；`a[i][j]` 表示从状态 j 一步转移到状态 i ；三个函数只认列向量约定 `next = A * cur`。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；`solve()` 里直接调用本节列出的函数即可。

警告：矩阵 `a[i][j]` 表示从 j 转移到 i ，这样 `next = A * cur`；不要把行列方向写反。

【最小完整示例（先抄这一段就能跑）】

题目：求 Fibonacci 第 n 项 ($F_1 = 1, F_0 = 0$) mod $1e9+7$ ($n \leq 1e18$)。

```
MatVV A(2, vector<i64>(2, 0));
A[0][0] = 1; A[0][1] = 1; // 转移: next[i] = Σ_j A[i][j] * cur[j]
A[1][0] = 1; // 状态 [F_i, F_{i-1}] -> [F_{i+1}, F_i]
vector<i64> cur = {1, 0}; // 初始向量: F_1 = 1, F_0 = 0
auto P = mat_pow_mod(A, 9, 1000000007LL); // 求 A^9
auto res = mat_apply_mod(P, cur, 1000000007LL); // A^9 * cur = [F_10, F_9]
cout << res[0] << '\n'; // 第 10 项
```

样例： $n = 10 \rightarrow$ 输出 55 (mod $1e9+7$ 仍是 55)。

【传参要求（照这个传不会错）】

- `A[i][j]`：转移矩阵元素，0-based；含义是“从状态 j 走一步到状态 i ”，保证 `next = A * cur`（列向量）。
- `mat_mul_mod(a, b, mod)`：两个 n 阶方阵相乘，返回 $n \times n$ 结果；行列必须都是 n 。
- `mat_pow_mod(a, e, mod)`：方阵 a 的 e 次幂 ($e = 0$ 返回单位阵)， e 可达 $1e18$ ；`mod` 任意（非质数也能用）。
- `mat_apply_mod(a, v, mod)`：返回 $a * v$ ， v 是长度 n 的列向量，结果长度 n 。
- 乘完记得 `mat_apply_mod(P, cur, mod)` 得到最终状态，答案按你定义的向量位置取。

```
using MatVV = vector<vector<i64>>;

MatVV mat_mul_mod(const MatVV& a, const MatVV& b, i64 mod) {
    int n = (int)a.size();
    MatVV c(n, vector<i64>(n, 0));
    for (int i = 0; i < n; i++) {
        for (int k = 0; k < n; k++) {
            if (a[i][k] == 0) continue;
            for (int j = 0; j < n; j++) {
                c[i][j] = (c[i][j] + (i128)a[i][k] * b[k][j]) % mod;
            }
        }
    }
    return c;
}

MatVV mat_pow_mod(MatVV a, i64 e, i64 mod) {
    int n = (int)a.size();
    MatVV r(n, vector<i64>(n, 0));
    for (int i = 0; i < n; i++) r[i][i] = 1 % mod;
    while (e) {
        if (e & 1) r = mat_mul_mod(a, r, mod);
        a = mat_mul_mod(a, a, mod);
        e >>= 1;
    }
    return r;
}

vector<i64> mat_apply_mod(const MatVV& a, const vector<i64>& v, i64 mod) {
    int n = (int)a.size();
    vector<i64> res(n, 0);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            res[i] = (res[i] + (i128)a[i][j] * v[j]) % mod;
        }
    }
    return res;
}
```

容斥原理：二进制枚举

赛时先看

题目信号：多个限制条件，交集容易算，并集不好直接算。

本质：统计满足至少一个条件的对象数量。

复杂度判定： $O(2^m)$ ， m 是条件个数，一般 $m \leq 20$ 才枚举，更大要用容斥+其他技巧。

维护的量： n （计数范围右端点）； p （除数集合）； ans （最终并集大小）； $mask$ 枚举时用 `bits` 判奇偶决定加还是减。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接调用本节列出的函数即可。

警告：奇数个集合加，偶数个集合减。

【最小完整示例（先抄这一段就能跑）】

题目：求 $1..n$ 中能被 2, 3, 5 至少一个整除的个数。

```
i64 n = 100;
vector<i64> p = {2, 3, 5};           // 除数集合（不要求互质，内部用 lcm）
i64 ans = count_divisible_by_any(n, p); // 容斥枚举  $2^3$  个子集
cout << ans << '\n';               // 74
```

样例： $n = 100$, $p = \{2, 3, 5\}$ $\rightarrow$ 输出 74。

【传参要求（照这个传不会错）】

- n ：计数范围 $[1, n]$ 的右端点， n 可达 $1e18$ （返回 i64）。
- p ：除数集合， $m = p.size() < 62$ ($1LL \ll m$ 不溢出)；内部按 `lcm(p[i]` 的子集 `<= n` 才计贡献，乘积溢出会跳过该子集。
- 返回值： $1..n$ 中能被 p 中至少一个数整除的个数（整数，不取模）。
- 若 p 含重复数或非质数也能正确计算，不必预处理成质数。

```
i64 count_divisible_by_any(i64 n, const vector<i64>& p) {
    int m = (int)p.size();
    assert(m < 62);
    i64 ans = 0;
    for (int mask = 1; mask < (1LL << m); ++mask) {
        i128 l = 1;
        int bits = 0;
        bool overflow = false;
        for (int i = 0; i < m; ++i) {
            if (mask >> i & 1) {
                bits++;
                l = l / gcd((i64)l, p[i]) * p[i];
                if (l > n) {
                    overflow = true;
                    break;
                }
            }
        }
        if (overflow) continue;
        if (bits & 1) ans += n / (i64)l;
        else ans -= n / (i64)l;
    }
    return ans;
}
```

高斯消元：实数线性方程组

赛时先看

题目信号：概率期望方程、电路、线性代数建模。

本质：解 n 个未知数的线性方程组。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；优先调用下面 API 列出的入口，其余 helper 默认不要从 `solve()` 直接调。

复杂度判定： $O(n^3)$ 。

维护的量： a (n 行 $n+1$ 列增广矩阵，传值拷贝可随便改)；`where[col]` (第 `col` 列主元所在行，`-1` 表示自由元)；`ans` (解向量)。

警告：浮点比较用 EPS；无解/无穷多解要按题意处理。

【最小完整示例（先抄这一段就能跑）】

题目：解 $x + y = 3$, $2x - y = 0$ 。

```
vector<vector<double>> a = {{1, 1, 3}, {2, -1, 0}}; // 增广矩阵 [系数 | 常数]
vector<double> ans; // 输出解向量
int ret = gauss(a, ans); // 0 无解 / 1 唯一解 / 2 无穷多解
cout << ret << ' ' << ans[0] << ' ' << ans[1] << '\n'; // 1 1 2
```

样例：上面方程组 $\rightarrow$ 输出 1 1 2 (唯一解 $x=1$, $y=2$)。

【传参要求（照这个传不会错）】

- a ： n 行 $n+1$ 列增广矩阵，`a[i][n]` 是第 i 个方程的常数项；按值传递，原矩阵不会被改。

- ans: 输出参数, 长度 n 的解向量; 没被选为主元的自由元位置填 0。
- 返回值: 0 无解、1 唯一解、2 无穷多解 (判断阈值内部用 $\text{EPS} = 1\text{e-}9$ 与 $1\text{e-}7$ 回代校验)。
- 解的精度: 常规 $1\text{e-}6$ 判题没问题; 要更高精度改大 EPS 或换 long double 版本。

【API / 入口函数 (赛时只认这里列的名字)】

- gauss(vector<vector<double>> a, vector<double>& ans) -> a 是 n 行 $n+1$ 列增广矩阵。返回 0 无解, 1 唯一解, 2 无穷多解。

```
const double EPS = 1e-9;

// a 是 n 行 n+1 列增广矩阵。返回 0 无解, 1 唯一解, 2 无穷多解。
int gauss(vector<vector<double>> a, vector<double>& ans) {
    int n = (int)a.size();
    int m = (int)a[0].size() - 1;
    vector<int> where(m, -1);
    for (int col = 0, row = 0; col < m && row < n; ++col) {
        int sel = row;
        for (int i = row; i < n; ++i) {
            if (fabs(a[i][col]) > fabs(a[sel][col])) sel = i;
        }
        if (fabs(a[sel][col]) < EPS) continue;
        swap(a[sel], a[row]);
        where[col] = row;

        double div = a[row][col];
        for (int j = col; j <= m; ++j) a[row][j] /= div;
        for (int i = 0; i < n; ++i) {
            if (i == row) continue;
            double factor = a[i][col];
            for (int j = col; j <= m; ++j) a[i][j] -= factor * a[row][j];
        }
        row++;
    }

    ans.assign(m, 0);
    for (int i = 0; i < m; ++i) {
        if (where[i] != -1) ans[i] = a[where[i]][m];
    }
    for (int i = 0; i < n; ++i) {
        double sum = 0;
        for (int j = 0; j < m; ++j) sum += ans[j] * a[i][j];
        if (fabs(sum - a[i][m]) > 1e-7) return 0;
    }
    for (int i = 0; i < m; ++i) if (where[i] == -1) return 2;
    return 1;
}
```

单纯形法：线性规划最大化

赛时先看

题目信号：变量是实数；约束和目标都是线性的；题面出现资源分配、连续配比、混合策略、最大/最小线性收益。若变量必须是整数，单纯形不能直接解决整数规划。

本质：求 $\max c^T x$, 满足 $Ax \leq b, x \geq 0$ 的连续线性规划；可区分最优、无界、无可行解。

接法：饮料配方/资源约束下最大收益；零和博弈的混合策略；线性不等式下的最大面积或最大价值。

复杂度判定：单纯形理论最坏指数级，但竞赛随机/普通数据通常很快；变量、约束在数百规模较常见。

维护的量： $D((m+2) \times (n+2))$ 单纯形表； B/N (基变量/非基变量编号)； m/n (约束数/变量数)。

警告：返回 INF 表示无界，返回 -INF 表示无可行解；浮点比较用 EPS。最小化可对目标系数取反，再把答案取反。

【最小完整示例 (先抄这一段就能跑)】

题目：求 $\max x + y$, 约束 $2x + y \leq 10, x + 2y \leq 8, x, y \geq 0$ 。

```
vector<vector<long double>> A = {{2, 1}, {1, 2}}; // 约束矩阵 A (m 行 n 列)
vector<long double> b = {10, 8}; // 约束右端  $A \cdot x \leq b$ 
vector<long double> c = {1, 1}; // 目标系数, 最大化  $c \cdot x$ 
vector<long double> x; // 输出最优解
LPSolver lp(A, b, c);
long double ans = lp.solve(x); // 最优值: x[0], x[1] 是最优点
cout << fixed << setprecision(6) << ans << '\n'; // 6.000000
```

样例：上面线性规划 -> 最优值 6 ($x=4, y=2$)。

【传参要求 (照这个传不会错)】

- A : $m \times n$ 约束矩阵; b : 长度 m 的右端向量; c : 长度 n 的目标系数。三者都必须是非负/任意实数 (内部自动处理)。
- `solve(x)`: 执行主算法; 返回最大目标值; x 被填成最优解 (长度 n)。
- 返回值 `INF` (`1e100`) 说明目标无界; `-INF` 说明无可行解。
- 约束形式固定为 $A \cdot x \leq b$, $x \geq 0$; 要 $\geq$ 就两边乘 `-1`; 要最小化就把 c 取反、答案取反。

【API / 入口函数 (赛时只认这里列的名字)】

- `solve(vector<long double>& x) ->` 执行主算法并返回答案

典型模型: 饮料配方/资源约束下最大收益; 零和博弈的混合策略; 线性不等式下的最大面积或最大价值。

```
struct LPSolver {
    static constexpr long double EPS = 1e-12L;
    static constexpr long double INF = 1e100L;
    int m, n;
    vector<int> B, N;
    vector<vector<long double>> D;

    // A 是 m*n, 约束 A*x <= b; 最大化 c*x。
    LPSolver(const vector<vector<long double>>& A, const vector<long double>& b,
             const vector<long double>& c)
        : m((int)b.size()), n((int)c.size()), B(m), N(n + 1), D(m + 2, vector<long double>(n + 2)) {
        for (int i = 0; i < m; ++i) for (int j = 0; j < n; ++j) D[i][j] = A[i][j];
        for (int i = 0; i < m; ++i) {
            B[i] = b[i];
            D[i][n] = -1;
            D[i][n + 1] = b[i];
        }
        for (int j = 0; j < n; ++j) {
            N[j] = j;
            D[m][j] = -c[j];
        }
        N[n] = -1;
        D[m + 1][n] = 1;
    }

    void pivot(int r, int s) {
        long double inv = 1 / D[r][s];
        for (int i = 0; i < m + 2; ++i) if (i != r) {
            for (int j = 0; j < n + 2; ++j) if (j != s) D[i][j] -= D[r][j] * D[i][s] * inv;
        }
        for (int j = 0; j < n + 2; ++j) if (j != s) D[r][j] *= inv;
        for (int i = 0; i < m + 2; ++i) if (i != r) D[i][s] *= -inv;
        D[r][s] = inv;
        swap(B[r], N[s]);
    }

    bool simplex(int phase) {
        int x = phase == 1 ? m + 1 : m;
        while (true) {
            int s = -1;
            for (int j = 0; j <= n; ++j) {
                if (phase == 2 && N[j] == -1) continue;
                if (s == -1 || D[x][j] < D[x][s] - EPS ||
                    (fabs1(D[x][j] - D[x][s]) <= EPS && N[j] < N[s])) s = j;
            }
            if (D[x][s] >= -EPS) return true;
            int r = -1;
            for (int i = 0; i < m; ++i) if (D[i][s] > EPS) {
                if (r == -1) r = i;
                else {
                    long double lhs = D[i][n + 1] / D[i][s];
                    long double rhs = D[r][n + 1] / D[r][s];
                    if (lhs < rhs - EPS || (fabs1(lhs - rhs) <= EPS && B[i] < B[r])) r = i;
                }
            }
            if (r == -1) return false;
            pivot(r, s);
        }
    }

    long double solve(vector<long double>& x) {
        int r = 0;
        for (int i = 1; i < m; ++i) if (D[i][n + 1] < D[r][n + 1]) r = i;
        if (D[r][n + 1] < -EPS) {
            pivot(r, n);
            if (!simplex(1) || D[m + 1][n + 1] < -EPS) return -INF; // 无可行解
            if (fabs1(D[m + 1][n + 1]) > EPS) return -INF;
            for (int i = 0; i < m; ++i) if (B[i] == -1) {
                int s = 0;

```

```

        for (int j = 1; j <= n; ++j) {
            if (D[i][j] < D[i][s] - EPS ||
                (fabsl(D[i][j] - D[i][s]) <= EPS && N[j] < N[s])) s = j;
        }
        pivot(i, s);
    }
}
if (!simplex(2)) return INF; // 无界
x.assign(n, 0);
for (int i = 0; i < m; ++i) if (B[i] < n) x[B[i]] = D[i][n + 1];
return D[m][n + 1];
}
};

```

GF(2) 高斯消元与异或方程组

赛时先看

题目信号：约束是若干变量 xor 后等于 0/1；图的度数奇偶、开关灯、异或构造。

本质：解异或方程组、求秩、判断是否有解、计算自由元数量。

复杂度判定： $O(n * m^2 / \text{word})$ 用 bitset 常数较好；下方模板为 $O(\text{rows} * \text{vars}^2 / 64)$ 级别。

维护的量： $a[i]$ (第 i 行 bitset, $[0, \text{nvar})$ 存系数, $a[i][\text{nvar}]$ 存常数)； row (已消出的主元行数 = 秩)。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接调用本节列出的函数即可。

警告：增广列放在 `nvar`；出现全 0 系数但常数为 1 即无解。

【最小完整示例（先抄这一段就能跑）】

题目：判断异或方程组 $x_0 \oplus x_2 = 1$, $x_0 \oplus x_1 \oplus x_2 = 0$, $x_1 = 1$ 的秩与解数。

```

const int MAXV = 10; // 变量个数上界，模板参数
vector<bitset<MAXV + 1>> a;
a.push_back(bitset<MAXV + 1>(0b1101)); // 二进制串右起第 i 位是 x_i, 第 nvar 位是常数
a.push_back(bitset<MAXV + 1>(0b0111)); // x0^x1^x2 = 0
a.push_back(bitset<MAXV + 1>(0b1010)); // x1 = 1
int rank = gauss_xor<MAXV>(a, 3); // 3 个变量；返回秩，-1 表示无解
cout << rank << '\n'; // 秩 = 2，自由元 3-2 = 1 个

```

样例：上面方程组 -> 输出 2 (有解，解数 $2^1 = 2$)。

【传参要求（照这个传不会错）】

- `MAXV`：模板参数，变量个数的上界；每个方程用 `bitset<MAXV + 1>`，多出的 1 位存常数。
- `a`：行数 = 方程数； $a[i][j]$ ($0 \leq j < \text{nvar}$) 是 x_j 的系数 (0/1)， $a[i][\text{nvar}]$ 是方程右端常数。
- `nvar`：实际变量个数， $\leq \text{MAXV}$ ；注意 `nvar` 同时也是增广列下标。
- 返回值：方程组有解时返回秩 `rank`，解的数量是 $2^{(\text{nvar} - \text{rank})}$ ；无解返回 `-1`。
- 消元后主元行 $a[i][\text{nvar}]$ 可直接读出对应变量的值。

```

template <int MAXV>
int gauss_xor(vector<bitset<MAXV + 1>>& a, int nvar) {
    int n = (int)a.size();
    int row = 0;
    for (int col = 0; col < nvar && row < n; col++) {
        int pivot = -1;
        for (int i = row; i < n; i++) {
            if (a[i][col]) {
                pivot = i;
                break;
            }
        }
        if (pivot == -1) continue;
        swap(a[row], a[pivot]);
        for (int i = 0; i < n; i++) {
            if (i != row && a[i][col]) a[i] ^= a[row];
        }
        row++;
    }
    for (int i = row; i < n; i++) {
        bool all_zero = true;
        for (int j = 0; j < nvar; j++) {
            if (a[i][j]) {
                all_zero = false;
                break;
            }
        }
        if (all_zero && a[i][nvar]) return -1; // 无解。
    }
    return row; // rank 为秩，解的数量为  $2^{(\text{nvar} - \text{rank})}$ 。
}

```

模意义高斯消元：唯一解

赛时先看

题目信号：答案是有理数取模；方程形如 $x_i = c + \sum p_{ij} x_j$ ；模数是质数。

本质：模质数意义下解线性方程组，常用于期望、概率、计数线性方程。

复杂度判定： $O(n^3)$ 。

维护的量： a (n 行 $n+1$ 列增广矩阵)； $where[col]$ (第 col 列主元所在行， -1 为自由元)； ans (解向量)。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接调用本节列出的函数即可。

警告：只有主元非零才能求逆；若题目保证唯一解可以直接返回，否则要额外处理无解/多解。

【最小完整示例（先抄这一段就能跑）】

题目：模 5 下解 $2x + y = 3, x + y = 2$ 。

```
vector<vector<i64>> a = {{2, 1, 3}, {1, 1, 2}}; // n 行 n+1 列增广矩阵
vector<i64> ans = gauss_mod_unique(a, 5); // mod 必须是质数
cout << ans[0] << ' ' << ans[1] << '\n'; // 1 1
```

样例：上面方程组 $\rightarrow$ 输出 1 1 ($x=1, y=1$ 代入两个方程均成立)。

【传参要求（照这个传不会错）】

- a : n 行 $n+1$ 列增广矩阵， $a[i][n]$ 是第 i 个方程的常数项；值可以为负（内部自动转正取模）。
- mod : 质数模数（内部用费马小定理求 $mod-2$ 次幂当逆元，合数模数不能用本函数）。
- 返回值：长度 n 的解向量 ans ，全部在 $[0, mod)$ ；本函数假设唯一解，自由元位置填 0。
- 配套的 `mod_pow_prime(a, e, mod)` 是内部快速幂，普通题目不需要直接调用。

```
i64 mod_pow_prime(i64 a, i64 e, i64 mod) {
    i64 r = 1 % mod;
    a %= mod;
    while (e) {
        if (e & 1) r = (i128)r * a % mod;
        a = (i128)a * a % mod;
        e >>= 1;
    }
    return r;
}

vector<i64> gauss_mod_unique(vector<vector<i64>> a, i64 mod) {
    int n = (int)a.size();
    vector<int> where(n, -1);
    int row = 0;
    for (int col = 0; col < n && row < n; col++) {
        int pivot = -1;
        for (int i = row; i < n; i++) {
            if ((a[i][col] % mod + mod) % mod != 0) {
                pivot = i;
                break;
            }
        }
        if (pivot == -1) continue;
        swap(a[row], a[pivot]);
        where[col] = row;

        i64 inv = mod_pow_prime((a[row][col] % mod + mod) % mod, mod - 2, mod);
        for (int j = col; j <= n; j++) a[row][j] = a[row][j] * inv % mod;
        for (int i = 0; i < n; i++) {
            if (i == row) continue;
            i64 factor = a[i][col] % mod;
            if (factor < 0) factor += mod;
            if (factor == 0) continue;
            for (int j = col; j <= n; j++) {
                a[i][j] = (a[i][j] - factor * a[row][j]) % mod;
                if (a[i][j] < 0) a[i][j] += mod;
            }
        }
        row++;
    }

    vector<i64> ans(n, 0);
    for (int col = 0; col < n; col++) {
        if (where[col] != -1) ans[col] = a[where[col]][n] % mod;
    }
    return ans;
}
```

吸收马尔可夫链期望方程

赛时先看

题目信号：每轮随机转移；问期望多少轮结束；状态数可控；样例解释里有 $E = 1 + pE$ 。

本质：有限状态随机过程，求到达吸收态的期望轮数。

复杂度判定：建方程 $O(S^2)$ ，消元 $O(S^3)$ 。

维护的量： $trans[i]$ （状态 i 到各非吸收状态的转移 $\{to, 概率\}$ ）； a ($E[i] = 1 + \sum p_{ij} * E[j]$ 的增广矩阵)。

警告：只把非吸收状态编号进方程；转移到吸收态不需要加变量。依赖上一节 `gauss_mod_unique` 解方程。

【最小完整示例（先抄这一段就能跑）】

题目：两状态赌博，状态 0 以 1/2 结束、1/2 去状态 1；状态 1 以 1/2 去状态 0、1/2 结束。求从各状态出发的期望轮数。

```
vector<vector<pair<int, i64>>> trans(2);
trans[0].push_back({1, 499122177}); // 状态 0 -> 状态 1, 概率 1/2 (模 998244353)
trans[1].push_back({0, 499122177}); // 状态 1 -> 状态 0, 概率 1/2
// 转向“结束/吸收态”的概率一律不写入 trans
vector<i64> E = absorbing_expectation(trans); // E[i] = 从状态 i 出发的期望轮数
cout << E[0] << ' ' << E[1] << '\n'; // 2 2
```

样例：上面转移 -> 输出 2 2（两状态期望都等于 2）。

【传参要求（照这个传不会错）】

- `trans`：长度 s = 非吸收状态个数；`trans[i]` 的元素是 $\{to, p\}$ ， p 为模 998244353 下的转移概率。
- 转移到吸收态（游戏结束）的边不要写入 `trans[i]`；本模板每轮自动 +1 计期望轮数。
- 返回值：长度 s 的 $E[i]$ ，即从状态 i 出发到吸收的期望轮数（模 998244353）。
- 依赖上一节 `gauss_mod_unique` 与本节开头常量 `EXPECT_MOD`；模数不可自行更换。

```
const i64 EXPECT_MOD = 998244353;

// trans[i] = {to, prob_mod}, i/to 都是非吸收状态编号。
// 若有概率转向吸收态，直接不写入 trans[i]。
vector<i64> absorbing_expectation(
    const vector<vector<pair<int, i64>>>& trans
) {
    int s = (int)trans.size();
    vector<vector<i64>> a(s, vector<i64>(s + 1, 0));
    for (int i = 0; i < s; i++) {
        a[i][i] = 1;
        a[i][s] = 1;
        for (auto [to, p] : trans[i]) {
            a[i][to] = (a[i][to] - p) % EXPECT_MOD;
            if (a[i][to] < 0) a[i][to] += EXPECT_MOD;
        }
    }
    return gauss_mod_unique(a, EXPECT_MOD);
}
```

典题：标号图度数奇偶计数

赛时先看

题目信号：图的边任意选，条件只限制每个点度数的奇偶；答案对任意 `mod` 取模。

本质：计算 n 个点的简单标号无向图中，所有点度数奇偶性全部相同的图数量。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；`solve()` 里直接调用本节列出的函数即可。

复杂度判定： $O(\log n)$ 。

维护的量：`edges`（完全图边数 $n(n-1)/2$ ）；`exponent`（`edges-(n-1)`， $GF(2)$

秩差）；`even_degree_graphs`（全偶度数图数量）。

警告：完全图关联矩阵在 $GF(2)$ 下秩为 $n-1$ ；全奇度数只有 n 为偶数时可行。

【最小完整示例（先抄这一段就能跑）】

题目： $n=4$ 个点的标号无向图，所有点度数奇偶性相同的图数 `mod 1e9+7`。

```
i64 ans = count_same_parity_degree_graph(4, 1000000007); // 1. 调用：点数 n、模数 mod
cout << ans << '\n'; // 2. 取结果：mod 后答案
```

样例：`count_same_parity_degree_graph(4, 1000000007)` -> 16（全偶 8 + 全奇

8）；`count_same_parity_degree_graph(3, 1000000007)` -> 2（空图与三角形，全奇不可能）。

【传参要求（照这个传不会错）】

- `count_same_parity_degree_graph(n, mod)`: n = 点数 ($i64$, $n \geq 1$); mod = 任意模数 ($i64$, 合数也行)。
- 返回 n 个点的简单标号无向图中, 所有点度数奇偶性相同 (全偶或全奇) 的图数 `mod mod`。
- `mod == 1` 返回 0; n 为奇数时全奇不可能, 答案只含全偶 (乘 2 的分支跳过)。
- 原理: 完全图关联矩阵 $GF(2)$ 下秩 $n-1$, 全偶图数 = $2^{(edges-(n-1))}$, n 为偶数时全奇图数同样多。

```
i64 pow_mod_any(i64 a, i64 e, i64 mod) {
    i64 r = 1 % mod;
    a %= mod;
    while (e) {
        if (e & 1) r = (i128)r * a % mod;
        a = (i128)a * a % mod;
        e >>= 1;
    }
    return r;
}

i64 count_same_parity_degree_graph(i64 n, i64 mod) {
    if (mod == 1) return 0;
    i64 edges = n * (n - 1) / 2;
    i64 exponent = edges - (n - 1);
    i64 even_degree_graphs = pow_mod_any(2, exponent, mod);
    if (n % 2 == 0) return even_degree_graphs * 2 % mod; // 全偶或全奇。
    return even_degree_graphs; // 全奇不可能。
}
```

行列式取模：任意模数版

赛时先看

题目信号：要求矩阵行列式，模数不保证质数。

本质：Matrix-Tree、线性代数计数、格点相关计数。

复杂度判定： $O(n^3 \log V)$, V 是元素值域。

维护的量： a ($n \times n$ 方阵, 传值拷贝内部随便消); `ans` (行列式值, 含行交换符号, 最后转正)。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面; `solve()` 里直接调用本节列出的函数即可。

警告：欧几里得消元过程中要处理行交换符号; 最后答案转正。

【最小完整示例（先抄这一段就能跑）】

题目：求 `det([[1,2],[3,4]]) mod 1e9+7` (模数不是质数也照样算)。

```
vector<vector<i64>> a = {{1, 2}, {3, 4}}; // n x n 方阵
i64 det = determinant_any_mod(a, 1000000007LL); // 任意模数, 无需逆元
cout << det << '\n'; // 1*4-2*3 = -2 ≡ 1000000005
```

样例：上面矩阵 $\rightarrow$ 输出 `1000000005` (即 $-2 \bmod 1e9+7$)。

【传参要求（照这个传不会错）】

- a : $n \times n$ 方阵, 元素任意整数均可 (内部自动取模); 按值传递, 原矩阵不被改。
- `mod`: 任意模数 (不要求质数, 全程只用加减乘和整除, 不需要逆元)。
- 返回值: `det(a) mod mod`, 已转正落在 $[0, mod)$; 消元中若主元变 0 直接返回 0。
- 复杂度 $O(n^3 \log V)$: n 到几百可用; Matrix-Tree 定理、格点计数都能套。

```
i64 determinant_any_mod(vector<vector<i64>> a, i64 mod) {
    int n = (int)a.size();
    i64 ans = 1;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            while (a[j][i]) {
                i64 t = a[i][i] / a[j][i];
                for (int k = i; k < n; k++) {
                    a[i][k] = (a[i][k] - t * a[j][k]) % mod;
                    swap(a[i][k], a[j][k]);
                }
                ans = -ans;
            }
        }
        if (a[i][i] == 0) return 0;
        ans = ans * a[i][i] % mod;
    }
    return (ans % mod + mod) % mod;
}
```

康托展开

赛时先看

题目信号：问一个排列是第几小，或第 k 个排列。

复杂度判定：朴素 $O(n^2)$ ，树状数组可优化到 $O(n \log n)$ 。

维护的量： $\text{fac}[i]$ （阶乘，未取模）； $\text{used}[x]$ （数字 x 是否已用）； rank （累加的 0-based 排名）。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接调用本节列出的函数即可。

警告：排名通常有 0-based 和 1-based 两种，按题意调整。阶乘未取模， $n \geq 21$ 时 `i64`

溢出；需要取模请自行对阶乘取模。

【最小完整示例（先抄这一段就能跑）】

题目：求排列 $[2, 1, 3]$ 在 $1..n$ 全排列中的排名（0-based）。

```
vector<int> p = {2, 1, 3};           // 必须是 1..n 的排列（元素从 1 开始）
i64 rank = cantor_rank_zero_based(p); // 0-based 排名
cout << rank << '\n';              // [1, 2, 3] 是 0, [2, 1, 3] 是 2
```

样例： $p = [2, 1, 3] \rightarrow$ 输出 2（全排列字典序： $[1, 2, 3]=0$ ， $[1, 3, 2]=1$ ， $[2, 1, 3]=2$ ）。

【传参要求（照这个传不会错）】

- p ：一个 $1..n$ 的排列，长度 n ，元素范围必须是 $1..n$ （代码用 `used[x]` 查 $1..n$ ）。
- 返回值：0-based 排名，即比它小的排列个数；题目要 1-based 排名就 $+1$ 。
- 复杂度朴素 $O(n^2)$ ， n 到 $1e5$ 用树状数组优化； $n \geq 21$ 阶乘溢出 `i64`（要取模得自己把 `fac` 改成取模）。
- 逆操作（第 k 个排列）本节没有，可拿 BIT + 二分现写。

```
i64 cantor_rank_zero_based(const vector<int>& p) {
    int n = (int)p.size();
    vector<i64> fac(n + 1, 1);
    for (int i = 1; i <= n; ++i) fac[i] = fac[i - 1] * i;

    i64 rank = 0;
    vector<int> used(n + 1, 0);
    for (int i = 0; i < n; ++i) {
        int smaller_unused = 0;
        for (int x = 1; x < p[i]; ++x) if (!used[x]) smaller_unused++;
        rank += smaller_unused * fac[n - 1 - i];
        used[p[i]] = 1;
    }
    return rank;
}
```

Lagrange 插值：连续点 $0..n-1$

赛时先看

题目信号：答案是低次多项式，给出前若干项，求第 k 项。

本质：已知多项式在 $0..n-1$ 的值，求 $f(x)$ 。

复杂度判定： $O(n)$ （预处理前缀/后缀积 + 阶乘逆元后一次线性求和）。

维护的量： pre/suf （ $\prod(x-i)$ 的前缀/后缀积）； fac/ifac （阶乘与逆元）； ans （逐项累加的插值结果）。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接调用本节列出的函数即可。

警告：模数要为质数； $x < n$ 时直接返回。

【最小完整示例（先抄这一段就能跑）】

题目：已知 $f(x) = x^2$ 在 $0..3$ 的值，求 $f(5) \bmod 1e9+7$ 。

```
vector<i64> y = {0, 1, 4, 9};           // f(0), f(1), f(2), f(3)
i64 val = lagrange_consecutive(y, 5, 1000000007LL); // 求 f(5), x 可到 1e18
cout << val << '\n';                  // 25
```

样例： $y = \{0, 1, 4, 9\}$ 求 $f(5) \rightarrow$ 输出 25。

【传参要求（照这个传不会错）】

- y ：长度 n ，依次是 $f(0), f(1), \dots, f(n-1)$ 的值（ n 为插值点数，通常取“次数 + 1”）。
- x ：要求值的点，`i64`，可达 $1e18$ ；若 $x < n$ 直接返回 $y[x]$ 。
- mod ：质数模数（内部对阶乘求逆元，合数模数不可用）。
- 返回值： $f(x) \bmod \text{mod}$ ，在 $[0, \text{mod})$ 。

- 依赖前置函数 `mod_pow`（第一节已有）；`n` 到 `1e6` 可用。

```
i64 lagrange_consecutive(const vector<i64>& y, i64 x, i64 mod) {
    int n = (int)y.size();
    if (x < n) return y[(int)x] % mod;
    vector<i64> pre(n + 1, 1), suf(n + 1, 1), fac(n + 1, 1), ifac(n + 1, 1);
    for (int i = 0; i < n; ++i) pre[i + 1] = pre[i] * ((x - i) % mod + mod) % mod;
    for (int i = n - 1; i >= 0; --i) suf[i] = suf[i + 1] * ((x - i) % mod + mod) % mod;
    for (int i = 1; i <= n; ++i) fac[i] = fac[i - 1] * i % mod;
    ifac[n] = mod_pow(fac[n], mod - 2, mod);
    for (int i = n; i >= 1; --i) ifac[i - 1] = ifac[i] * i % mod;
    i64 ans = 0;
    for (int i = 0; i < n; ++i) {
        i64 num = pre[i] * suf[i + 1] % mod;
        i64 den = ifac[i] * ifac[n - 1 - i] % mod;
        if ((n - 1 - i) & 1) den = (mod - den) % mod;
        ans = (ans + y[i] % mod * num % mod * den) % mod;
    }
    return ans;
}
```

FFT：任意整数卷积

赛时先看

题目信号：`c[k] = sum a[i] * b[k-i]`，模数不是 NTT 友好质数，或题目要求普通整数答案。

本质：两个整数系数多项式卷积。NTT 受模数限制；当题目需要实数近似或普通整数卷积时可使用 FFT。

复杂度判定： $O(n \log n)$ ，`n` 为补到 2 的幂的长度。

维护的量：`fa/fb`（补零到 2 的幂后的系数数组，`fa` 复用为结果）；`c`（`llround` 取整后的整数卷积）。

接法：多项式乘法、字符串匹配的卷积化、骰子点数分布合并。

警告：系数和长度过大时浮点误差会累积；模意义卷积优先用 NTT，任意模数再考虑三模 NTT + CRT。

【最小完整示例（先抄这一段就能跑）】

题目：计算整数卷积 `[1,2,3] * [4,5]`。

```
vector<i64> a = {1, 2, 3}, b = {4, 5};
vector<i64> c = convolution_fft(a, b); // c[k] = Σ a[i] * b[k-i]
for (i64 v : c) cout << v << ' '; // 4 13 22 15
cout << '\n';
```

样例：`[1,2,3] * [4,5] → 输出 4 13 22 15`。

【传参要求（照这个传不会错）】

- `a, b`：长度 `n, m` 的整数系数数组，可为负数；空数组返回空结果。
- 返回值：长度 `n + m - 1` 的 `vector<i64>`，`c[k] = Σ a[i]*b[k-i]`（`llround` 取整，不做任何取模）。
- 精度：建议系数绝对值之和在 `1e15` 以内（double 53 位有效数字）；超了换三模 NTT + CRT。
- `fft(a, invert)` 是底层函数，普通题目只调 `convolution_fft`。

【改板时先认这几个量】

- `fa/fb`：输入系数补到 2 的幂长度后的数组（`fa` 复用为结果）。
- `bit`：位逆序置换的局部变量。

典题模型：多项式乘法、字符串匹配的卷积化、骰子点数分布合并。

```
using cd = complex<double>;
const double PI = acos(-1.0);

void fft(vector<cd>& a, bool invert) {
    int n = (int)a.size();
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <= 1) {
        double ang = 2 * PI / len * (invert ? -1 : 1);
        cd wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; ++j) {
                cd u = a[i + j], v = a[i + j + len / 2] * w;
                a[i + j] = u + v;
                a[i + j + len / 2] = u - v;
            }
            w = w * wlen;
        }
    }
}
```

```

        w *= wlen;
    }
}
}
if (invert) for (cd& x : a) x /= n;
}

vector<i64> convolution_fft(const vector<i64>& a, const vector<i64>& b) {
    if (a.empty() || b.empty()) return {};
    int need = (int)a.size() + (int)b.size() - 1;
    int n = 1;
    while (n < need) n <= 1;
    vector<cd> fa(n), fb(n);
    for (int i = 0; i < (int)a.size(); ++i) fa[i] = (double)a[i];
    for (int i = 0; i < (int)b.size(); ++i) fb[i] = (double)b[i];
    fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; ++i) fa[i] *= fb[i];
    fft(fa, true);
    vector<i64> c(need);
    for (int i = 0; i < need; ++i) c[i] = llround(fa[i].real());
    return c;
}

```

NTT 卷积

赛时先看

题目信号: $c[k] = \sum a[i] * b[k-i]$, $O(nm)$ 太慢。

本质: 快速多项式乘法、卷积、生成函数基础。

复杂度判定: $O(n \log n)$, n 为补到 2 的幂的长度, 上限 2^{23} 。

维护的量: a/b (补零后的系数数组, 逐点相乘后 a 复用为结果); $\text{NTT_MOD} = 998244353$ 、 $\text{NTT_G} = 3$ 。

接法: 把本节 struct/class/函数组 整段抄到 `solve()` 上面; 优先调用下面 API 列出的入口, 其余 helper 默认不要从 `solve()` 直接调。

警告: 这里模数固定 998244353, 原根 3。

【最小完整示例 (先抄这一段就能跑)】

题目: 计算模 998244353 下的卷积 $[1, 2, 3] * [4, 5]$ 。

```

vector<int> a = {1, 2, 3}, b = {4, 5}; // 系数先规范到 [0, 998244353)
vector<int> c = convolution(a, b);    // c[k] = Σ a[i]*b[k-i] (mod 998244353)
for (int v : c) cout << v << ' ';    // 4 13 22 15
cout << '\n';

```

样例: $[1, 2, 3] * [4, 5] \rightarrow$ 输出 4 13 22 15。

【传参要求 (照这个传不会错)】

- a, b : 长度 n, m 的 `vector<int>` 系数, 取值先规范到 $[0, 998244353)$; 空数组返回空结果。
- 返回值: 长度 $n + m - 1$ 的 `vector<int>`, 结果已对 998244353 取模。
- 长度限制: $n + m - 1 \leq 2^{23}$ (本模数支持的最大 2 次幂长度); 超长换 FWT 思路或分块卷积。
- `ntt(a, invert)` 是底层函数, 普通卷积题只调 `convolution`。

【不会用就照抄】

```
auto c = convolution(a, b); // c[k] = sum a[i] * b[k-i]
```

- 先确认模板模数 (常见 998244353) 和原根; 不要直接换成任意模数。
- 输入系数先规范到模数范围。

【API / 入口函数 (赛时只认这里列的名字)】

- `convolution(a, b)` $\rightarrow$ 返回卷积系数; 结果长度 `a.size()+b.size()-1`。
- `ntt(a, invert)` $\rightarrow$ 底层正/逆变换; 普通卷积题直接调用 `convolution`, 不要手动碰。

【核心逻辑 (改代码时别破坏)】

- 把系数值表示转换为点值表示, 逐点相乘, 再逆变换回系数。
- 长度补到不小于 $n+m-1$ 的 2 的幂; 这版固定模 998244353、原根 3。

```

const int NTT_MOD = 998244353;
const int NTT_G = 3;

int mod_pow_int(int a, int e) {
    i64 r = 1, x = a;

```

```

while (e) {
    if (e & 1) r = r * x % NTT_MOD;
    x = x * x % NTT_MOD;
    e >>= 1;
}
return (int)r;
}

void ntt(vector<int>& a, bool invert) {
    int n = (int)a.size();
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        int wlen = mod_pow_int(NTT_G, (NTT_MOD - 1) / len);
        if (invert) wlen = mod_pow_int(wlen, NTT_MOD - 2);
        for (int i = 0; i < n; i += len) {
            i64 w = 1;
            for (int j = 0; j < len / 2; ++j) {
                int u = a[i + j];
                int v = (int)(a[i + j + len / 2] * w % NTT_MOD);
                a[i + j] = u + v < NTT_MOD ? u + v : u + v - NTT_MOD;
                a[i + j + len / 2] = u - v >= 0 ? u - v : u - v + NTT_MOD;
                w = w * wlen % NTT_MOD;
            }
        }
    }
    if (invert) {
        int inv_n = mod_pow_int(n, NTT_MOD - 2);
        for (int& x : a) x = (int)(1LL * x * inv_n % NTT_MOD);
    }
}

vector<int> convolution(vector<int> a, vector<int> b) {
    if (a.empty() || b.empty()) return {};
    int need = (int)a.size() + (int)b.size() - 1;
    int n = 1;
    while (n < need) n <<= 1;
    a.resize(n);
    b.resize(n);
    ntt(a, false);
    ntt(b, false);
    for (int i = 0; i < n; ++i) a[i] = (int)(1LL * a[i] * b[i] % NTT_MOD);
    ntt(a, true);
    a.resize(need);
    return a;
}

```

形式幂级数核心：inv / ln / exp / sqrt / pow / divmod

赛时先看

题目信号：题目直接要求多项式初等函数；生成函数可整理成 $F=\exp(G)$ 、 $F * G=1$ 、 $F^2=G$ ；需要长度 $1e5$ 以上的卷积并继续做除法；或者需要从复杂递推中抽出有限项生成函数。

本质：在模 998244353 下，对截断到 x^n 的多项式/形式幂级数做快速运算。包含 NTT 卷积、牛顿迭代求逆和开方、对数、指数、非负整数幂、多项式除法与取模。

接法：多项式求逆/开根/ln/exp 模板题；有标号组合结构的 EGF；将递推转为生成函数后取前 n 项；需要快速除以一組线性因子的多项式题。

复杂度判定：卷积 $O(n \log n)$ ；inv / ln / exp / sqrt / pow 为 $O(n \log^2 n)$ 的稳健版本；多项式 divmod 为 $O(n \log^2 n)$ 。NTT 长度不能超过 2^{23} 。

维护的量：Poly（系数向量，Poly[i] 是 x^i 的系数）； n （要保留的系数个数）；FPS_MOD=998244353（固定模数）。

警告：所有函数的第二参数 n 都是“要保留的系数个数”，不是最高次数。每次运算后结果会截断到 x^n 。形式幂级数不是普通实函数，ln/exp 的常数项限制不能忽略。任意模数请使用 FFT 或多模 NTT + CRT，不要直接改 MOD。

【最小完整示例（先抄这一段就能跑）】

```

// 题目：求  $F(x)=1+x$  的逆的前 4 项 ( $1/(1+x)=1-x+x^2-x^3+\dots$ )。
Poly f = {1, 1}; // f[0]=1 非零，可求逆
Poly g = fps_inv(f, 4); // g = {1, 998244352, 1, 998244352}，即  $1-x+x^2-x^3$ 
// 验证：样例 g = {1, 998244352, 1, 998244352} (mod 998244353)

```

【传参要求（照这个传不会错）】

- fps_inv(f, n): f 非空且 $f[0] \neq 0$; $n \geq 1$ 是保留系数个数；返回 f 在 mod x^n 下的逆。

- `fps_log(f, n)`: 要求 `f[0] == 1`; 返回 $\ln(f) \bmod x^n$ 。
- `fps_exp(f, n)`: 要求 `f[0] == 0`; 返回 $\exp(f) \bmod x^n$ 。
- `fps_sqrt_nonzero_const(f, n)`: 要求 `f[0] != 0` 且常数项有平方根（无平方根 `assert` 失败）; 返回 $\bmod x^n$ 意义下的一个平方根。
- `fps_pow(f, k, n)`: `k` 为非负整数 (`i64`) ; 若首位非零项次数 $\ast k \geq n$ 返回全 0 的 `Poly(n)`。
- `fps_divmod(f, g) / fps_mod(f, g)`: `g` 非零; 返回 {商, 余数} 或余数。
- 所有函数的第二参数 `n` 是系数个数而不是最高次数; 模数固定 `998244353`, 任意模数不要改 `FPS_MOD`。

【API / 入口函数（赛时只认这里列的名字）】

- `fps_divmod(Poly f, Poly g) ->` 返回 `f/g` 的商和余数; 要求 `g` 非零。
- `fps_exp(const Poly& f, int n) ->` 返回 $\exp(f) \bmod x^n$; 要求 `f[0] == 0`。
- `fps_inv(const Poly& f, int n) ->` 返回 `f` 在 $\bmod x^n$ 意义下的逆; 要求 `f[0] != 0`。
- `fps_log(const Poly& f, int n) ->` 返回 $\ln(f) \bmod x^n$; 要求 `f[0] == 1`。
- `fps_pow(Poly f, i64 k, int n) ->` 返回非负整数 `k` 下的 $f^k \bmod x^n$ 。
- `fps_sqrt_nonzero_const(const Poly& f, int n) ->` 返回 `f` 在 $\bmod x^n$ 意义下的一个平方根; 要求 `f[0] != 0` 且常数项有平方根。

【改板时先认这几个量】

- `Poly`: `vector<int>` 的别名, 所有多项式都以它表示。
- `FPS_MOD`: 固定模数 `998244353`, 原根 `FPS_G=3`。
- `n`: 所有函数第二参数, 表示要保留的系数个数。

使用前提: 模数固定为 `998244353`, 原根为 `3`。 `fps_inv` 要求常数项非零; `fps_log` 要求 `f[0]=1`; `fps_exp` 要求 `f[0]=0`; `fps_sqrt_nonzero_const` 要求常数项存在平方根且非零; `fps_pow` 的指数为非负整数。

典型模型: 多项式求逆/开根/ $\ln/\exp$ 模板题; 有标号组合结构的 EGF; 将递推转为生成函数后取前 `n` 项; 需要快速除以一组线性因子的多项式题。

```
using Poly = vector<int>;
constexpr int FPS_MOD = 998244353;
constexpr int FPS_G = 3;

int fps_powmod(i64 a, i64 e) {
    i64 r = 1;
    while (e) {
        if (e & 1) r = r * a % FPS_MOD;
        a = a * a % FPS_MOD;
        e >>= 1;
    }
    return (int)r;
}

void fps_ntt(Poly& a, bool invert) {
    int n = (int)a.size();
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <= 1) {
        int wlen = fps_powmod(FPS_G, (FPS_MOD - 1) / len);
        if (invert) wlen = fps_powmod(wlen, FPS_MOD - 2);
        for (int i = 0; i < n; i += len) {
            i64 w = 1;
            for (int j = 0; j < len / 2; ++j) {
                int u = a[i + j];
                int v = (int)(a[i + j + len / 2] * w % FPS_MOD);
                a[i + j] = u + v < FPS_MOD ? u + v : u + v - FPS_MOD;
                a[i + j + len / 2] = u - v >= 0 ? u - v : u - v + FPS_MOD;
                w = w * wlen % FPS_MOD;
            }
        }
    }
    if (invert) {
        int inv_n = fps_powmod(n, FPS_MOD - 2);
        for (int& x : a) x = (int)(1LL * x * inv_n % FPS_MOD);
    }
}

Poly fps_mul(Poly a, Poly b) {
    if (a.empty() || b.empty()) return {};
}
```

```

    if (min(a.size(), b.size()) <= 32) {
        Poly c(a.size() + b.size() - 1);
        for (int i = 0; i < (int)a.size(); ++i)
            for (int j = 0; j < (int)b.size(); ++j)
                c[i + j] = (c[i + j] + 1LL * a[i] * b[j]) % FPS_MOD;
        return c;
    }
    int need = (int)a.size() + (int)b.size() - 1;
    int n = 1;
    while (n < need) n <= 1;
    assert(n <= (1 << 23));
    a.resize(n); b.resize(n);
    fps_ntt(a, false); fps_ntt(b, false);
    for (int i = 0; i < n; ++i) a[i] = (int)(1LL * a[i] * b[i] % FPS_MOD);
    fps_ntt(a, true);
    a.resize(need);
    return a;
}

Poly fps_derivative(const Poly& f) {
    if (f.size() <= 1) return {};
    Poly g(f.size() - 1);
    for (int i = 1; i < (int)f.size(); ++i) g[i - 1] = (int)(1LL * i * f[i] % FPS_MOD);
    return g;
}

Poly fps_integral(const Poly& f) {
    int n = (int)f.size();
    Poly inv(n + 1), g(n + 1);
    if (n >= 1) inv[1] = 1;
    for (int i = 2; i <= n; ++i) inv[i] = (int)(1LL * (FPS_MOD - FPS_MOD / i) * inv[FPS_MOD % i] % FPS_MOD);
    for (int i = 0; i < n; ++i) g[i + 1] = (int)(1LL * f[i] * inv[i + 1] % FPS_MOD);
    return g;
}

// 返回 f 在 mod x^n 意义下的逆; 要求 f[0] != 0。
Poly fps_inv(const Poly& f, int n) {
    assert(!f.empty() && f[0] != 0 && n >= 1);
    Poly g{fps_powmod(f[0], FPS_MOD - 2)};
    for (int m = 1; m < n; m <= 1) {
        int need = min(n, m << 1);
        Poly cut(f.begin(), f.begin() + min((int)f.size(), need));
        Poly h = fps_mul(cut, g);
        h.resize(need);
        for (int& x : h) x = x ? FPS_MOD - x : 0;
        h[0] = (h[0] + 2) % FPS_MOD;
        g = fps_mul(g, h);
        g.resize(need);
    }
    return g;
}

// 返回 ln(f) mod x^n; 要求 f[0] == 1。
Poly fps_log(const Poly& f, int n) {
    assert(!f.empty() && f[0] == 1 && n >= 1);
    Poly g = fps_mul(fps_derivative(f), fps_inv(f, n));
    g.resize(max(0, n - 1));
    g = fps_integral(g);
    g.resize(n);
    return g;
}

// 返回 exp(f) mod x^n; 要求 f[0] == 0。
Poly fps_exp(const Poly& f, int n) {
    assert((f.empty() || f[0] == 0) && n >= 1);
    Poly g{1};
    for (int m = 1; m < n; m <= 1) {
        int need = min(n, m << 1);
        Poly lg = fps_log(g, need);
        Poly h(need);
        for (int i = 0; i < need; ++i) {
            int fi = i < (int)f.size() ? f[i] : 0;
            h[i] = fi - lg[i];
            if (h[i] < 0) h[i] += FPS_MOD;
        }
        h[0] = (h[0] + 1) % FPS_MOD; // 牛顿迭代公式中的 1 + f - ln(g)。
        g = fps_mul(g, h);
        g.resize(need);
    }
    return g;
}

int fps_mod_sqrt(int a) { // Tonelli-Shanks; 不存在平方根时返回 -1。
    if (a == 0) return 0;
    if (fps_powmod(a, (FPS_MOD - 1) / 2) != 1) return -1;

```



```

int q = FPS_MOD - 1, s = 0;
while ((q & 1) == 0) q >>= 1, ++s;
int z = 2;
while (fps_powmod(z, (FPS_MOD - 1) / 2) != FPS_MOD - 1) ++z;
i64 c = fps_powmod(z, q), x = fps_powmod(a, (q + 1) / 2), t = fps_powmod(a, q);
for (int m = s; t != 1; ) {
    int i = 1;
    i64 tt = t * t % FPS_MOD;
    while (tt != 1) tt = tt * tt % FPS_MOD, ++i;
    i64 b = fps_powmod(c, 1LL << (m - i - 1));
    x = x * b % FPS_MOD;
    c = b * b % FPS_MOD;
    t = t * c % FPS_MOD;
    m = i;
}
return (int)x;
}

// 返回 f 在 mod x^n 意义下的一个平方根; 要求 f[0] != 0 且常数项有平方根。
Poly fps_sqrt_nonzero_const(const Poly& f, int n) {
    assert(!f.empty() && f[0] != 0 && n >= 1);
    int root = fps_mod_sqrt(f[0]);
    assert(root != -1);
    Poly g{root};
    const int inv2 = (FPS_MOD + 1) / 2;
    for (int m = 1; m < n; m <= 1) {
        int need = min(n, m << 1);
        Poly cut(f.begin(), f.begin() + min((int)f.size(), need));
        Poly div = fps_mul(cut, fps_inv(g, need));
        div.resize(need);
        g.resize(need);
        for (int i = 0; i < need; ++i) g[i] = (int)(1LL * (g[i] + div[i]) * inv2 % FPS_MOD);
    }
    return g;
}

// 返回非负整数 k 下的 f^k mod x^n。
Poly fps_pow(Poly f, i64 k, int n) {
    assert(k >= 0 && n >= 1);
    if (k == 0) { Poly one(n); one[0] = 1; return one; }
    int first = 0;
    while (first < (int)f.size() && f[first] == 0) ++first;
    if (first == (int)f.size() || (i128)first * k >= n) return Poly(n);
    int shift = (int)(first * k);
    int need = n - shift;
    int c = f[first], inv_c = fps_powmod(c, FPS_MOD - 2);
    Poly h;
    for (int i = first; i < (int)f.size() && (int)h.size() < need; ++i)
        h.push_back((int)(1LL * f[i] * inv_c % FPS_MOD));
    h.resize(need);
    Poly l = fps_log(h, need);
    int km = (int)(k % FPS_MOD);
    for (int& x : l) x = (int)(1LL * x * km % FPS_MOD);
    Poly ans = fps_exp(l, need);
    int scale = fps_powmod(c, k);
    for (int& x : ans) x = (int)(1LL * x * scale % FPS_MOD);
    Poly out(n);
    for (int i = 0; i < need; ++i) out[i + shift] = ans[i];
    return out;
}

void fps_trim(Poly& f) { while (!f.empty() && f.back() == 0) f.pop_back(); }

// 返回 f/g 的商和余数; 要求 g 非零。
pair<Poly, Poly> fps_divmod(Poly f, Poly g) {
    fps_trim(f); fps_trim(g);
    assert(!g.empty());
    if (f.size() < g.size()) return {{}, f};
    int qn = (int)f.size() - (int)g.size() + 1;
    Poly rf = f, rg = g;
    reverse(rf.begin(), rf.end());
    reverse(rg.begin(), rg.end());
    Poly q = fps_mul(rf, fps_inv(rg, qn));
    q.resize(qn);
    reverse(q.begin(), q.end());

    Poly product = fps_mul(q, g);
    Poly r(f.size());
    for (int i = 0; i < (int)f.size(); ++i) {
        int sub = i < (int)product.size() ? product[i] : 0;
        r[i] = f[i] - sub;
        if (r[i] < 0) r[i] += FPS_MOD;
    }
    r.resize(g.size() - 1);
    fps_trim(r);
}

```

```

    return {q, r};
}

Poly fps_mod(Poly f, const Poly& g) { return fps_divmod(std::move(f), g).second; }

```

积树：多点求值 $f(x_i)$

赛时先看

题目信号： m 个询问点都要代入同一个高次多项式；点是任意模数值而不是连续 $0..n-1$ （连续点可优先翻 Lagrange）；或要求后续做快速插值。

本质： 给一个多项式和很多互异点 x_i ，同时计算所有

$f(x_i)$ 。本实现利用积树和上面的多项式取模，适合点数较大时替代逐点 $O(nm)$ 代入。

接法： 多项式在大量随机点的值；积树/余数树模板；快速插值的前半部分；求 $f(a_i)$ 并再做组合/CRT。

复杂度判定： $O(m \log^2 m)$ 量级（包含多项式除法），空间 $O(m \log m)$ 。

维护的量： x （询问点集，构造时已正规化到 $[0, \text{MOD})$ ）；**product**（乘积树，**product[p]** 存节点区间对应的 $\prod (X - x_i)$ ）；**answer**（每个点的 $f(x_i)$ ）。

警告： 这份实现假定所有 x_i 互异；重复点在求值本身没问题，但如果之后复用到插值会出错。**points** 为空时直接返回空。多项式系数与点都必须先正规化到 $[0, \text{MOD})$ 。

【最小完整示例（先抄这一段就能跑）】

```

// 题目：求  $f(x)=x^2+1$  在  $x=0, 1, 2$  三点的值。
MultipointEvaluation ev({0, 1, 2}); // 构造乘积树，点集内部自动取模
vector<int> ans = ev.evaluate({1, 0, 1}); //  $f=\{1, 0, 1\}$ ，输出  $\text{ans}=\{1, 2, 5\}$ 
// 验证：样例  $f(0)=1, f(1)=2, f(2)=5$ 

```

【传参要求（照这个传不会错）】

- **MultipointEvaluation(points):** points 为询问点 x_i （要求互异；重复点求值没问题但不可再复用去插值）；points 为空时 evaluate 返回空。
- **evaluate(f):** f 为 Poly 系数（ $f[i]$ 是 x^i 的系数）；返回 $\text{vector}<\text{int}>$ ， $\text{ans}[i] = f(x[i])$ ，长度等于点数。
- 点与系数在构造/内部都会按 $\% \text{FPS_MOD}$ 正规化，传负数也能跑；依赖上一节的 **fps_mod**。

典型模型：多项式在大量随机点的值；积树/余数树模板；快速插值的前半部分；求 $f(a_i)$ 并再做组合/CRT。

```

struct MultipointEvaluation {
    int n;
    vector<int> x, answer;
    vector<Poly> product;

    explicit MultipointEvaluation(vector<int> points) : n((int)points.size()), x(std::move(points)) {
        for (int& v : x) {
            v %= FPS_MOD;
            if (v < 0) v += FPS_MOD;
        }
        product.resize(max(1, 4 * n));
        answer.resize(n);
        if (n) build(1, 0, n);
    }

    void build(int p, int l, int r) {
        if (r - l == 1) {
            product[p] = {(FPS_MOD - x[l]) % FPS_MOD, 1}; // 叶子多项式为  $X - x[l]$ 。
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m);
        build(p * 2 + 1, m, r);
        product[p] = fps_mul(product[p * 2], product[p * 2 + 1]);
    }

    void solve(int p, int l, int r, const Poly& remainder) {
        if (r - l == 1) {
            answer[l] = remainder.empty() ? 0 : remainder[0];
            return;
        }
        int m = (l + r) / 2;
        solve(p * 2, l, m, fps_mod(remainder, product[p * 2]));
        solve(p * 2 + 1, m, r, fps_mod(remainder, product[p * 2 + 1]));
    }

    vector<int> evaluate(const Poly& f) {
        if (!n) return {};
        solve(1, 0, n, fps_mod(f, product[1]));
        return answer;
    }
}

```

};

积树：快速插值 $f(x_i)=y_i$

赛时先看

题目信号：给了许多离散点的函数值，要求恢复系数多项式；横坐标任意且数量达到 $1e5$ ；需要根据点值重建生成函数或再做多项式运算。

本质：给定互异点 x_i 与对应值

y_i ，构造唯一的次数小于点数的多项式。它和多点求值互为对偶，适合点数大、横坐标不连续的插值。

接法：任意点快速插值模板题；从多组函数值重建多项式；多项式求值/插值的往返变换。

复杂度判定： $O(n \log^2 n)$ ，空间 $O(n \log n)$ 。依赖上一节的 `MultipointEvaluation` 与 FPS 核心，必须按顺序粘贴。

维护的量：`tree`（复用多点求值的乘积树 $M(x)=\prod(x-x_i)$ ）；`weight`（每点权重 $y_i / M'(x_i)$ ）；`interpolate` 返回唯一插值多项式。

警告：所有 x_i 必须两两不同，否则 $M'(x_i)=0$ 无法求逆。返回多项式次数严格小于点数；若只需要 $0..n-1$ 连续点处的单个值，优先使用前册 Lagrange，常数更小。

【最小完整示例（先抄这一段就能跑）】

```
// 题目：过三点 (0,1)、(1,2)、(2,5) 还原多项式。
MultipointInterpolation it({0, 1, 2}); // 先建乘积树
Poly f = it.interpolate({1, 2, 5}); // 返回 f={1, 0, 1}，即 x^2+1
// 验证：样例 f={1, 0, 1}，代回三点均吻合
```

【传参要求（照这个传不会错）】

- `MultipointInterpolation(points)`： x_i 必须两两不同（否则 $M'(x_i)=0$ 求逆 `assert` 失败）。
- `interpolate(y)`： $y[i]$ 是 $f(\text{tree}.x[i])$ 的函数值，`y.size()` 必须等于点数；返回次数严格小于点数的 `Poly`。
- 依赖「积树：多点求值」与「形式幂级数核心」的 `fps_derivative` / `fps_mul` / `fps_powmod`，必须按顺序粘贴。

【API / 入口函数（赛时只认这里列的名字）】

- `interpolate(const vector<int>& y) -> y[i]` 是 $f(\text{tree}.x[i])$ ；所有 x_i 必须两两不同。返回 `Poly`。

典题模型：任意点快速插值模板题；从多组函数值重建多项式；多项式求值/插值的往返变换。

```
struct MultipointInterpolation {
    MultipointEvaluation tree; // 复用它的乘积树 M(x)=prod(x-x_i)。
    vector<int> weight;

    explicit MultipointInterpolation(vector<int> points) : tree(std::move(points)) {}

    Poly merge(int p, int l, int r) {
        if (r - l == 1) return {weight[l]};
        int m = (l + r) / 2;
        Poly left = merge(p * 2, l, m);
        Poly right = merge(p * 2 + 1, m, r);
        Poly a = fps_mul(left, tree.product[p * 2 + 1]);
        Poly b = fps_mul(right, tree.product[p * 2]);
        if (a.size() < b.size()) a.resize(b.size());
        for (int i = 0; i < (int)b.size(); ++i) {
            a[i] += b[i];
            if (a[i] >= FPS_MOD) a[i] -= FPS_MOD;
        }
        return a;
    }

    // y[i] 是 f(tree.x[i])；所有 x_i 必须两两不同。
    Poly interpolate(const vector<int>& y) {
        int n = tree.n;
        assert((int)y.size() == n);
        if (n == 0) return {};
        vector<int> denom = tree.evaluate(fps_derivative(tree.product[1]));
        weight.resize(n);
        for (int i = 0; i < n; ++i) {
            assert(denom[i] != 0);
            int yi = y[i] % FPS_MOD;
            if (yi < 0) yi += FPS_MOD;
            weight[i] = (int)(1LL * yi * fps_powmod(denom[i], FPS_MOD - 2) % FPS_MOD);
        }
        return merge(1, 0, n);
    }
};
```

FWT：AND / OR 卷积

赛时先看

题目信号：状态由 bitmask 表示，合并时用按位 OR/AND 而非 XOR；数组长度是 2^m 。

本质：计算 $c[k] = \sum_{i \mid j = k} a[i]b[j]$ 或 $c[k] = \sum_{i \& j = k} a[i]b[j]$ 。

接法：两个集合任选一个元素后，问并集掩码或交集掩码的方案数。

复杂度判定： $O(m \cdot 2^m)$ 。

维护的量：a、b（原数组，值在 $[0, \text{mod})$ ）；n（补齐后的 2 的幂长度）；mod（模数）。

警告：OR 与 AND 的正反变换方向不同；必须在模数下规范化负数。XOR 卷积见下一节 FWHT。

【最小完整示例（先抄这一段就能跑）】

```
// 题目：a={1,2}、b={2,3}，求 OR 卷积与 AND 卷积（长度自动补到 2）。
vector<i64> c1 = or_convolution({1, 2}, {2, 3}, 998244353); // c1={2, 13}
vector<i64> c2 = and_convolution({1, 2}, {2, 3}, 998244353); // c2={9, 6}
// 验证：c1[k]=Σ_{i|j=k} a[i]b[j]；c2[k]=Σ_{i&j=k} a[i]b[j]
```

【传参要求（照这个传不会错）】

- `or_convolution(a, b, mod)` / `and_convolution(a, b, mod)`: 返回 $c[k] = \sum_{i \mid j = k} a[i]b[j]$ （或 $i \& j = k$ ）；内部自动把长度补齐到 2 的幂。
- `fwt_or(a, mod, inverse)` / `fwt_and(a, mod, inverse)`: 原地变换；`inverse=true` 做逆变换（已含归一化），一般不单独用。
- 数组元素必须在 $[0, \text{mod})$ ，mod 传质数（如 998244353）；类型是 `vector<i64>`，乘法会取模不会溢出。

典题模型：两个集合任选一个元素后，问并集掩码或交集掩码的方案数。

```
void fwt_or(vector<i64>& a, i64 mod, bool inverse) {
    int n = (int)a.size();
    for (int len = 1; len < n; len <= 1) {
        for (int i = 0; i < n; i += len < 1) {
            for (int j = 0; j < len; ++j) {
                i64& x = a[i + j];
                i64& y = a[i + j + len];
                y = (y + (inverse ? -x : x)) % mod;
                if (y < 0) y += mod;
            }
        }
    }
}

void fwt_and(vector<i64>& a, i64 mod, bool inverse) {
    int n = (int)a.size();
    for (int len = 1; len < n; len <= 1) {
        for (int i = 0; i < n; i += len < 1) {
            for (int j = 0; j < len; ++j) {
                i64& x = a[i + j];
                i64& y = a[i + j + len];
                x = (x + (inverse ? -y : y)) % mod;
                if (x < 0) x += mod;
            }
        }
    }
}

vector<i64> or_convolution(vector<i64> a, vector<i64> b, i64 mod) {
    int n = 1;
    while (n < (int)max(a.size(), b.size())) n <= 1;
    a.resize(n); b.resize(n);
    fwt_or(a, mod, false);
    fwt_or(b, mod, false);
    for (int i = 0; i < n; ++i) a[i] = a[i] * b[i] % mod;
    fwt_or(a, mod, true);
    return a;
}

vector<i64> and_convolution(vector<i64> a, vector<i64> b, i64 mod) {
    int n = 1;
    while (n < (int)max(a.size(), b.size())) n <= 1;
    a.resize(n); b.resize(n);
    fwt_and(a, mod, false);
    fwt_and(b, mod, false);
    for (int i = 0; i < n; ++i) a[i] = a[i] * b[i] % mod;
    fwt_and(a, mod, true);
    return a;
}
```

FWHT：XOR 卷积

赛时先看

题目信号：子集异或卷积、按 xor 合并状态。

本质：计算 $c[k] = \sum_{i \text{ xor } j = k} a[i] b[j]$ 。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接调用本节列出的函数即可。

复杂度判定： $O(n \log n)$ ， n 为 2 的幂。

维护的量： a 、 b （原数组，值在 $[0, \text{mod})$ ）； n （2 的幂长度）； mod （模数，逆变换要乘 inv_n ）。

警告：逆变换时每项乘 inv_n 。依赖 `mod_pow`，来自第 10 章通用辅助函数。

【最小完整示例（先抄这一段就能跑）】

```
// 题目：a={1,2}、b={2,3}，求 XOR 卷积。
vector<i64> c = xor_convolution({1, 2}, {2, 3}, 998244353); // c={8, 7}
// 验证：c[0]=1*2+2*3=8，c[1]=1*3+2*2=7
```

【传参要求（照这个传不会错）】

- `xor_convolution(a, b, mod)`：返回 $c[k] = \sum_{i \text{ xor } j = k} a[i]b[j]$ ；长度自动补齐到 2 的幂。
- `fwht_xor(a, mod, invert)`：原地变换；`invert=true` 时自动乘 inv_n ，一般不单独用。
- 数组元素必须在 $[0, \text{mod})$ ；依赖 `mod_pow(n, mod-2, mod)`；与「XOR 随机变量分布」节的同名 `fwht_xor` 签名不同，混抄时二选一或改名。

```
void fwht_xor(vector<i64>& a, i64 mod, bool invert) {
    int n = (int)a.size();
    for (int len = 1; len < n; len <= 1) {
        for (int i = 0; i < n; i += len < 1) {
            for (int j = 0; j < len; ++j) {
                i64 x = a[i + j], y = a[i + j + len];
                a[i + j] = (x + y) % mod;
                a[i + j + len] = (x - y + mod) % mod;
            }
        }
    }
    if (invert) {
        i64 inv_n = mod_pow(n, mod - 2, mod);
        for (auto& x : a) x = x * inv_n % mod;
    }
}

vector<i64> xor_convolution(vector<i64> a, vector<i64> b, i64 mod) {
    int n = 1;
    while (n < (int)max(a.size(), b.size())) n <= 1;
    a.resize(n); b.resize(n);
    fwht_xor(a, mod, false);
    fwht_xor(b, mod, false);
    for (int i = 0; i < n; ++i) a[i] = a[i] * b[i] % mod;
    fwht_xor(a, mod, true);
    return a;
}
```

XOR 随机变量分布：FWHT 合并

赛时先看

题目信号：每个变量独立取值，条件是 $x_1 \text{ xor } x_2 \text{ xor } \dots \text{ xor } x_n = \text{target}$ 。

本质：多个独立随机变量，求它们异或和等于目标值的概率或方案数。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接调用本节列出的函数即可。

复杂度判定：值域大小为 $V=2^k$ 时， $O(n V \log V)$ ；适合值域较小或可压缩的版本。

维护的量：`upper`（各变量取值上界 a_i ）； V （值域大小，2 的幂）；`prod`（FWHT 变换域上的累积乘积）。

警告：概率要乘每个变量取值数量的逆元；FWHT 乘完后要做逆变换。依赖 `pow_mod_any`（本章前文定义）；`MOD` 在本节代码内自行定义。注意本节 `fwht_xor` 与「FWHT：XOR 卷积」节同名函数签名不同，混抄时二选一或改名。

【最小完整示例（先抄这一段就能跑）】

```
// 题目：X∈[0,1]、Y∈[0,1] 独立均匀，求 X xor Y 的分布（概率模 MOD）。
vector<i64> dist = xor_distribution_uniform_ranges({1, 1}); // dist={499122177, 499122177}
// 验证：样例 dist[0]=dist[1]=499122177，即概率各 1/2 (mod 998244353)
```

【传参要求（照这个传不会错）】

- `xor_distribution_uniform_ranges(upper)`：`upper[i]` 是第 i 个变量的取值上界（取值 $0..a_i$ 均匀，各变量独立），元素为非负整数。
- 返回值：长度 V 的 `vector<i64>`，`dist[t]` = 异或和恰为 t 的概率（模 `MOD`）， V 由内部自动取最小的 2 的幂。
- 概率已在函数内乘过各变量取值数量的逆元，外面不要再除；依赖 `pow_mod_any`（本章前文定义）。

【改板时先认这几个量】

- `dist`: 单个变量的取值分布数组（长度 `V`）。
- `mx`: `upper` 中的最大值，决定值域大小 `V`。

```
const i64 MOD = 998244353;

void fwht_xor(vector<i64>& a, bool inverse) {
    int n = (int)a.size();
    for (int len = 1; len < n; len <= 1) {
        for (int i = 0; i < n; i += len < 1) {
            for (int j = 0; j < len; j++) {
                i64 x = a[i + j], y = a[i + j + len];
                a[i + j] = (x + y) % MOD;
                a[i + j + len] = (x - y + MOD) % MOD;
            }
        }
    }
    if (inverse) {
        i64 inv_n = pow_mod_any(n, MOD - 2, MOD);
        for (i64& x : a) x = x * inv_n % MOD;
    }
}

vector<i64> xor_distribution_uniform_ranges(const vector<int>& upper) {
    int mx = 0;
    for (int x : upper) mx = max(mx, x);
    int V = 1;
    while (V <= mx) V <= 1;
    vector<i64> prod(V, 1);
    for (int a : upper) {
        vector<i64> dist(V, 0);
        i64 inv = pow_mod_any(a + 1, MOD - 2, MOD);
        for (int x = 0; x <= a; x++) dist[x] = inv;
        fwht_xor(dist, false);
        for (int i = 0; i < V; i++) prod[i] = prod[i] * dist[i] % MOD;
    }
    fwht_xor(prod, true);
    return prod;
}
```

Berlekamp-Massey + Kitamasa

赛时先看

题目信号: 给出序列前项，猜/求最短线性递推；组合计数高阶递推。

本质: 从前若干项推线性递推，求第 `n` 项。

接法: 把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；`solve()` 里直接调用本节列出的函数即可。

复杂度判定: BM $O(k^2)$ ，求第 `n` 项 $O(k^2 \log n)$ 。

维护的量: `s`（已知前项序列）；`rec`（BM 求出的最短递推系数，`rec.size()` 即阶数 `k`）；`init`（递推初项）。

警告: 模数通常要求质数；输入项至少给足 `2k` 比较稳。

【最小完整示例（先抄这一段就能跑）】

```
// 题目: Fibonacci 前 6 项 0,1,1,2,3,5, 求第 10 项（下标从 0 计）。
vector<i64> rec = berlekamp_massey({0, 1, 1, 2, 3, 5}, 998244353); // rec={1, 1}
i64 ans = linear_recurrence_nth({0, 1}, rec, 10, 998244353); // ans=55
// 验证: 样例 ans=55 (F10)
```

【传参要求（照这个传不会错）】

- `berlekamp_massey(s, mod)`: `s` 为已知前项，长度至少给足 `2k` 较稳；返回最短递推 `rec`（满足 $s[n] = \sum rec[i-1] * s[n-i]$ ）；`mod` 通常要求质数。
- `linear_recurrence_nth(init, rec, n, mod)`: `init` 为前 `k` 项（`init[i]` = 序列第 `i` 项，长度 = `rec.size()`）；`n` 为 0-indexed 项号，`n < init.size()` 时直接返回 `init[n]`；返回第 `n` 项（i64）。
- `combine_rec` 是内部函数，不用直接调。

```
vector<i64> berlekamp_massey(const vector<i64>& s, i64 mod) {
    vector<i64> C{1}, B{1};
    i64 b = 1;
    int m = 1;
    for (int n = 0; n < (int)s.size(); ++n) {
        i64 d = 0;
        for (int i = 0; i < (int)C.size(); ++i) d = (d + C[i] * s[n - i]) % mod;
        if (d == 0) {
            m++;
            continue;
        }
    }
}
```

```

vector<i64> T = C;
i64 coef = d * mod_pow(b, mod - 2, mod) % mod;
if ((int)C.size() < (int)B.size() + m) C.resize(B.size() + m, 0);
for (int i = 0; i < (int)B.size(); ++i) {
    C[i + m] = (C[i + m] - coef * B[i]) % mod;
    if (C[i + m] < 0) C[i + m] += mod;
}
if (2 * ((int)T.size() - 1) <= n) {
    B = T;
    b = d;
    m = 1;
} else {
    m++;
}
}
C.erase(C.begin());
for (auto& x : C) x = (mod - x) % mod;
return C; // 递推式: s[n] = sum C[i-1] * s[n-i]。
}

vector<i64> combine_rec(vector<i64> a, vector<i64> b, const vector<i64>& rec, i64 mod) {
    int k = (int)rec.size();
    vector<i64> tmp(2 * k, 0);
    for (int i = 0; i < k; ++i) for (int j = 0; j < k; ++j) tmp[i + j] = (tmp[i + j] + a[i] * b[j]) % mod;
    for (int i = 2 * k - 2; i >= k; --i) {
        for (int j = 1; j <= k; ++j) tmp[i - j] = (tmp[i - j] + tmp[i] * rec[j - 1]) % mod;
    }
    tmp.resize(k);
    return tmp;
}

i64 linear_recurrence_nth(const vector<i64>& init, const vector<i64>& rec, i64 n, i64 mod) {
    int k = (int)rec.size();
    if (n < (int)init.size()) return init[n] % mod;
    vector<i64> pol(k), e(k);
    pol[0] = 1;
    if (k == 1) e[0] = rec[0];
    else e[1] = 1;
    while (n) {
        if (n & 1) pol = combine_rec(pol, e, rec, mod);
        e = combine_rec(e, e, rec, mod);
        n >>= 1;
    }
    i64 ans = 0;
    for (int i = 0; i < k; ++i) ans = (ans + pol[i] * init[i]) % mod;
    return ans;
}

```

Bostan-Mori: 有理生成函数第 n 项

赛时先看

题目信号: 序列满足线性递推但项数极大; 已经得到生成函数分子分母; Kitamasa 的 $O(k^2 \log n)$ 不够, 且模数可用 998244353; 题目问第 n 项而不是前 n 项。

本质: 求有理生成函数 $P(x) / Q(x)$ 的 $[x^n]$ 系数。尤其适合常系数线性递推、 n 可到 $1e18$ 、递推阶数在 $1e5$ 量级的场景。

接法: 常系数递推第 10^{18} 项; 有理生成函数系数; 大阶 Fibonacci 类推广; 线性递推与 NTT 结合题。

复杂度判定: $O(k \log k \log n)$, $k = \deg(Q)$; 依赖上文形式幂级数核心的 Poly / fps_mul / fps_mod / fps_powmod, 并固定模数 998244353。

维护的量: p 、 q (生成函数 $P(x)/Q(x)$ 的分子分母, 内部保证 $\deg P < \deg Q$); n (要取的第 n 项, 可到 $1e18$)。

警告: $Q[0]$ 必须非零。按“递推系数构造 Q ”时, 若 $a_n = c_1 a_{n-1} + \dots + c_k a_{n-k}$, 则 $Q = 1 - c_1 x - \dots - c_k x^k$ 。传入前把 P 、 Q 都规范化到模数范围。

【最小完整示例 (先抄这一段就能跑)】

```

// 题目:  $a_n = 2a_{n-1}$ ,  $a_0=1$ , 求  $a_{100}$  (即  $1/(1-2x)$  的  $[x^{100}]$ )。
Poly Q = {1, FPS_MOD - 2}; //  $Q = 1 - 2x$ 
int ans = bostan_mori_nth({1}, Q, 100); // ans = 882499742  $2^{100} \bmod 998244353$ 
// 验证: 样例 ans = 882499742

```

【传参要求 (照这个传不会错)】

- `bostan_mori_nth(p, q, n)`: n 为非负整数 (可到 $1e18$); q 非空且 $q[0] \neq 0$; p 会自动先 `fps_mod(p, q)` (保证 $\deg P < \deg Q$); 返回 $[x^n] P(x)/Q(x)$ (int)。
- 构造 Q : 递推 $a_n = c_1 a_{n-1} + \dots + c_k a_{n-k} \rightarrow Q = \{1, -c_1, \dots, -c_k\} \pmod{FPS_MOD}$; P 按代码下方注释由前 k 项展开。
- 依赖形式幂级数核心 (`fps_mul / fps_mod / fps_powmod`), 固定模数 998244353, 不要改 `FPS_MOD`。

【API / 入口函数（赛时只认这里列的名字）】

- `boston_mori_nth(Poly p, Poly q, i64 n) ->` 依赖上文形式幂级数核心（`fps_mul/fps_mod/fps_powmod`）；返回 $[x^n] P(x)/Q(x)$ 。

典题模型：常系数递推第 10^{18} 项；有理生成函数系数；大阶 Fibonacci 类推广；线性递推与 NTT 结合题。

```
// 依赖上文形式幂级数核心（fps_mul/fps_mod/fps_powmod）；返回 [x^n] P(x)/Q(x)。
int boston_mori_nth(Poly p, Poly q, i64 n) {
    assert(n >= 0 && !q.empty() && q[0] != 0);
    p = fps_mod(std::move(p), q); // 该方法要求 deg(P) < deg(Q)。
    while (n > 0) {
        if (p.empty()) return 0;
        Poly q_neg = q;
        for (int i = 1; i < (int)q_neg.size(); i += 2) {
            if (q_neg[i]) q_neg[i] = FPS_MOD - q_neg[i]; // 公式：Q(-x)。
        }
        Poly numerator = fps_mul(p, q_neg);
        Poly denominator = fps_mul(q, q_neg); // Q(x)Q(-x)，只剩偶次幂。

        Poly next_p, next_q;
        int parity = (int)(n & 1);
        for (int i = parity; i < (int)numerator.size(); i += 2) next_p.push_back(numerator[i]);
        for (int i = 0; i < (int)denominator.size(); i += 2) next_q.push_back(denominator[i]);
        p.swap(next_p);
        q.swap(next_q);
        n >>= 1;
    }
    return p.empty() ? 0 : (int)(1LL * p[0] * fps_powmod(q[0], FPS_MOD - 2) % FPS_MOD);
}

// 对于 a_n=c[0]a_{n-1}+...+c[k-1]a_{n-k}，先构造：
// 公式：Q=-c[0],-c[1],...,-c[k-1] (mod FPS_MOD)。
// 对 0<=i<k, P[i]=a[i]-sum_{j=1..i} c[j-1]*a[i-j],
// 然后调用 boston_mori_nth(P, Q, n)。
```

自适应辛普森积分：连续函数数值积分

赛时先看

题目信号：题面给出函数公式，要求曲线下面积、概率密度积分、旋转体体积分等；误差要求类似 $1e-6$ 。

本质：计算连续函数在 $[1, r]$ 上的数值积分。适合函数光滑、无法直接求原函数、答案允许浮点误差的题。

接法：把题目函数写进 `lambda f`，调用

`adaptive_simpson(f, l, r, eps)`。若积分区间很长且函数振荡，先按关键点拆成多段分别积分。

复杂度判定：取决于函数形状和误差要求，通常递归次数不大。

维护的量：`f`（被积函数）；`l`、`r`（积分区间）；`eps`（精度要求）；`max_depth`（递归深度上限）。

警告：函数不连续、尖峰很多或区间极大时辛普森可能不稳定；递归层数要限制，避免极端数据爆栈。

【最小完整示例（先抄这一段就能跑）】

```
// 题目：求 sin(x) 在 [0, pi] 上的积分（真实值 2），误差要求 1e-6。
auto f = [](double x) { return sin(x); };
double ans = adaptive_simpson(f, 0.0, 3.141592653589793, 1e-6); // ans ≈ 2.000000
// 验证：样例 ans ≈ 2.0（误差 < 1e-6）
```

【传参要求（照这个传不会错）】

- `adaptive_simpson(f, l, r, eps = 1e-8, max_depth = 30)`：`f` 为 `double(double)` 的可调用对象（`lambda`/函数指针都行）；`l < r`；返回 $[l, r]$ 上积分的 `double` 近似值，误差量级约 `eps`。
- `simpson(f, l, r)`：单段辛普森公式，自适应递归内部用，一般不直接调。
- 函数不光滑/尖峰多/区间很长时先按关键点拆段，或调大 `max_depth`；注意深度上限防爆栈。

```
template <class F>
double simpson(const F& f, double l, double r) {
    double m = (l + r) / 2;
    return (f(l) + 4 * f(m) + f(r)) * (r - l) / 6;
}

template <class F>
double adaptive_simpson_dfs(const F& f, double l, double r, double whole,
                           double eps, int depth) {
    double m = (l + r) / 2;
    double left = simpson(f, l, m);
    double right = simpson(f, m, r);
    double delta = left + right - whole;
    if (depth <= 0 || fabs(delta) <= 15 * eps) {
        return left + right + delta / 15;
    }
}
```

```

    }
    return adaptive_simpson_dfs(f, l, m, left, eps / 2, depth - 1)
        + adaptive_simpson_dfs(f, m, r, right, eps / 2, depth - 1);
}

template <class F>
double adaptive_simpson(const F& f, double l, double r,
                        double eps = 1e-8, int max_depth = 30) {
    return adaptive_simpson_dfs(f, l, r, simpson(f, l, r), eps, max_depth);
}

```

12 计算几何、扫描线与空间结构

几何先放点线面基础，再放多边形、凸包、闵可夫斯基和、旋转卡壳、圆、半平面交和空间查询；扫描线典题放在本章末尾。

点与向量基础

赛时先看

题目信号：坐标、点线圆、多边形、凸包。

本质：几何题的基础类型。

接法：几何题先复制这一段基础类型，后面的凸包、线段、圆都会用到

Point/dot/cross/dist。整数坐标但会做距离和角度时用 **double**；如果只比较方向和面积，改成 **i64** 叉积会更稳。

复杂度判定：0(1)。

维护的量：Point 存二维坐标 **x/y** (**double**)；**EPS=1e-9** 配合 **sgn** 统一做浮点比较。

警告：浮点比较用 EPS，不要直接 **==**。

【最小完整示例（先抄这一段就能跑）】

题目：给两点坐标，求距离与叉积（判断转向）。

```

Point a{0, 0}, b{3, 4};
printf("%.10f\n", dist(a, b)); // 距离 = 5
printf("%.10f\n", cross(a, b)); // 叉积 = 0 (a、b 共线)

```

样例输出：5.0000000000；0.0000000000。

【传参要求（照这个传不会错）】

- Point(x, y): 坐标, double; 点与向量都用它表示, 支持 + - * / 运算。
- dot(a, b): 点积, 返回 double。
- cross(a, b): 二维叉积, 返回 double; >0 表示 b 在 a 的逆时针方向。
- cross(a, b, c): (b-a)×(c-a), 判断三点转向; >0 为逆时针。
- norm2(a) / norm(a) / dist(a,b): 模长平方 / 模长 / 两点距离, 均返回 double。
- sgn(x): x>EPS 返回 1, x<-EPS 返回 -1, 否则 0。

```

const double EPS = 1e-9;
const double PI = acos(-1.0);

int sgn(double x) {
    return (x > EPS) - (x < -EPS);
}

struct Point {
    double x = 0, y = 0;
    Point() = default;
    Point(double x_, double y_) : x(x_), y(y_) {}
    Point operator+(const Point& o) const { return {x + o.x, y + o.y}; }
    Point operator-(const Point& o) const { return {x - o.x, y - o.y}; }
    Point operator*(double k) const { return {x * k, y * k}; }
    Point operator/(double k) const { return {x / k, y / k}; }
    bool operator<(const Point& o) const {
        if (sgn(x - o.x) != 0) return x < o.x;
        if (sgn(y - o.y) != 0) return y < o.y;
        return false;
    }
    bool operator==(const Point& o) const {
        return sgn(x - o.x) == 0 && sgn(y - o.y) == 0;
    }
};

double dot(Point a, Point b) { return a.x * b.x + a.y * b.y; }
double cross(Point a, Point b) { return a.x * b.y - a.y * b.x; }
double cross(Point a, Point b, Point c) { return cross(b - a, c - a); }

```

```
double norm2(Point a) { return dot(a, a); }
double norm(Point a) { return sqrt(norm2(a)); }
double dist(Point a, Point b) { return norm(a - b); }
```

三维点、向量、平面与体积

赛时先看

题目信号：坐标出现 (x,y,z) ，题面提到平面法向量、四面体/体积、直线射线与平面相交、是否共面。

本质：处理点到平面距离、直线和平面交点、夹角、四面体体积、共面判断等三维几何基本操作。

接法：四面体 ABCD 的体积为 `tetrahedron_volume6(A,B,C,D) / 6`；点 Q 到平面 ABC 的距离中，令 `Plane3 pl{A, cross3(B-A, C-A)}` 后调用 `distance_to_plane(Q, pl)`。若求线段和三角形/平面是否相交，先用 `line_plane_intersection` 求参数 `t`，再判断 `0 <= t <= 1` 与交点是否在三角形内。

复杂度判定：每个基本操作 $O(1)$ 。

维护的量：`Point3` 存 (x,y,z) ；`Plane3` 维护平面上一点 `p` 与法向量 `n`；`EPS3=1e-10` 用于三维浮点判断。

警告：浮点几何统一用 `EPS` 判断零；法向量不能是零向量；直线与平面平行时没有唯一交点。体积公式的标量三重积有正负，真正体积要取绝对值再除以 6。

【最小完整示例（先抄这一段就能跑）】

题目：求四面体 ABCD 的体积，以及点 Q 到平面 ABC 的距离。

```
Point3 A{0,0,0}, B{1,0,0}, C{0,1,0}, D{0,0,1}, Q{1,1,1};
printf("%.10f\n", tetrahedron_volume6(A,B,C,D) / 6); // 体积 = 1/6
Plane3 pl{A, cross3(B-A, C-A)}; // 平面 z=0
printf("%.10f\n", distance_to_plane(Q, pl)); // 距离 = 1
```

样例输出：0.1666666667；1.0000000000。

【传参要求（照这个传不会错）】

- `dot3(a,b)`：三维点积，返回 `double`。
- `cross3(a,b)`：三维叉积，返回 `Point3`（法向量）。
- `norm3(a) / norm2_3(a)`：模长 / 模长平方，返回 `double`。
- `scalar_triple(a,b,c)`：标量三重积 `dot3(a, cross3(b,c))`，有正负，返回 `double`。
- `Plane3{p, n}`：`p` 为平面上一点，`n` 为非零法向量。
- `signed_distance_to_plane(q, pl) / distance_to_plane(q, pl)`：点到平面有符号 / 无符号距离。
- `line_plane_intersection(a, v, pl, out)`：直线 `a+t*v` 与平面交点，平行返回 `false`，交点写入 `out`。
- `tetrahedron_volume6(a,b,c,d)`：四面体体积的 6 倍（已取绝对值），再 /6 得体积。
- `coplanar(a,b,c,d)`：四点共面返回 `true`。

【API / 入口函数（赛时只认这里列的名字）】

- `line_plane_intersection(Point3 a, Point3 v, const Plane3& pl, Point3& out) ->` 直线为 `a+t*v`；若直线与平面平行则返回 `false`。

【改板时先认这几个量】

- `p`：平面上的一个点。
- `n`：非零法向量。

```
const double EPS3 = 1e-10;

struct Point3 {
    double x = 0, y = 0, z = 0;
    Point3() = default;
    Point3(double x_, double y_, double z_) : x(x_), y(y_), z(z_) {}
    Point3 operator+(const Point3& o) const { return {x + o.x, y + o.y, z + o.z}; }
    Point3 operator-(const Point3& o) const { return {x - o.x, y - o.y, z - o.z}; }
    Point3 operator*(double k) const { return {x * k, y * k, z * k}; }
    Point3 operator/(double k) const { return {x / k, y / k, z / k}; }
};

double dot3(Point3 a, Point3 b) {
    return a.x * b.x + a.y * b.y + a.z * b.z;
}

Point3 cross3(Point3 a, Point3 b) {
    return {a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y - a.y * b.x};
}

double norm2_3(Point3 a) { return dot3(a, a); }
double norm3(Point3 a) { return sqrt(norm2_3(a)); }
double scalar_triple(Point3 a, Point3 b, Point3 c) { return dot3(a, cross3(b, c)); }
```

```

struct Plane3 {
    Point3 p; // 平面上的一个点。
    Point3 n; // 非零法向量。
};

double signed_distance_to_plane(Point3 q, const Plane3& p1) {
    return dot3(q - p1.p, p1.n) / norm3(p1.n);
}

double distance_to_plane(Point3 q, const Plane3& p1) {
    return fabs(signed_distance_to_plane(q, p1));
}

// 直线为 a+t*v; 若直线与平面平行则返回 false。
bool line_plane_intersection(Point3 a, Point3 v, const Plane3& p1, Point3& out) {
    double den = dot3(v, p1.n);
    if (fabs(den) <= EPS3) return false;
    double t = dot3(p1.p - a, p1.n) / den;
    out = a + v * t;
    return true;
}

double tetrahedron_volume6(Point3 a, Point3 b, Point3 c, Point3 d) {
    return fabs(scalar_triple(b - a, c - a, d - a));
}

bool coplanar(Point3 a, Point3 b, Point3 c, Point3 d) {
    return fabs(scalar_triple(b - a, c - a, d - a)) <= EPS3;
}

```

典型模型：四面体 $ABCD$ 的体积为 $\text{tetrahedron_volume6}(A,B,C,D) / 6$ ；点 Q 到平面 ABC 的距离中，令 $\text{Plane3 } p1\{A, \text{cross3}(B-A, C-A)\}$ 后调用 $\text{distance_to_plane}(Q, p1)$ 。若求线段和三角形/平面是否相交，先用 $\text{line_plane_intersection}$ 求参数 t ，再判断 $0 \leq t \leq 1$ 与交点是否在三角形内。

六边形网格 cube 坐标

赛时先看

题目信号：坐标满足 $x+y+z=0$ ；每个点有 6 个邻居；题面给三维坐标描述六边形网格。

本质：六边形网格距离、邻居、环、翻转周围一圈等实现。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；`solve()` 里先构造对象，再调用其公开接口。

复杂度判定：邻居 $O(1)$ ，半径为 r 的环 $O(r)$ 。

维护的量：`Hex` 维护三元坐标 $x/y/z$ （保证 $x+y+z=0$ ）；`HEX_DIR` 是六个方向的常量表。

警告：读入时断言 $x+y+z=0$ ；六边形距离是三维差绝对值之和除以 2。

【最小完整示例（先抄这一段就能跑）】

题目：求六边形网格上两点的距离，以及某点的邻居数。

```

Hex a{0, 0, 0}, b{2, -1, -1};
printf("%lld\n", hex_distance(a, b)); // 距离 = 2
auto ns = hex_neighbors(a);
printf("%zu\n", ns.size());          // 邻居数 = 6

```

样例输出：2；6。

【传参要求（照这个传不会错）】

- `Hex{x, y, z}`：三元坐标，`i64`，必须满足 $x+y+z=0$ 。
- `HEX_DIR`：`array<Hex,6>`，下标 `0..5` 依次是六个方向。
- `hex_distance(a, b)`：返回 `i64` 距离， $(\text{abs}(dx)+\text{abs}(dy)+\text{abs}(dz))/2$ 。
- `hex_neighbors(c)`：返回 `vector<Hex>`，共 6 个邻居。
- `hex_ring(c, radius)`：返回 `vector<Hex>`，半径 `radius` 的环；`radius=0` 时只含 `c` 自己。

```

struct Hex {
    i64 x = 0, y = 0, z = 0;

    bool operator==(const Hex& other) const {
        return x == other.x && y == other.y && z == other.z;
    }

    Hex operator+(const Hex& other) const {
        return {x + other.x, y + other.y, z + other.z};
    }

    Hex operator*(i64 k) const {
        return {x * k, y * k, z * k};
    }
}

```

```

    }
};

const array<Hex, 6> HEX_DIR = {{
    {0, -1, 1},
    {1, -1, 0},
    {1, 0, -1},
    {0, 1, -1},
    {-1, 1, 0},
    {-1, 0, 1},
}};

i64 hex_distance(Hex a, Hex b) {
    return (llabs(a.x - b.x) + llabs(a.y - b.y) + llabs(a.z - b.z)) / 2;
}

vector<Hex> hex_neighbors(Hex c) {
    vector<Hex> res;
    for (auto d : HEX_DIR) res.push_back(c + d);
    return res;
}

vector<Hex> hex_ring(Hex c, i64 radius) {
    vector<Hex> res;
    if (radius == 0) {
        res.push_back(c);
        return res;
    }
    Hex cur = c + HEX_DIR[4] * radius;
    for (int side = 0; side < 6; side++) {
        for (int step = 0; step < radius; step++) {
            res.push_back(cur);
            cur = cur + HEX_DIR[side];
        }
    }
    return res;
}

```

线段相交与点到线段距离

赛时先看

题目信号：线段、相交、距离、投影。

本质：判断路径碰撞、几何约束、最近距离。

接法：判断两条线段是否碰撞用 `segment_intersect(a1,a2,b1,b2)`；求点到线段最短距离用 `distance_point_to_segment(p,a,b)`。题目如果是整数网格且只问是否相交，注意端点接触通常也算相交，模板已经包含边界。

复杂度判定： $O(1)$ 。

维护的量：无额外结构，只依赖基础 `Point` 与 `dot/cross/norm/sgn`、`EPS`。

警告：共线重叠要用 `on_segment` 处理。

【最小完整示例（先抄这一段就能跑）】

题目：判断两条线段是否相交；求点 P 到线段 AB 的最短距离。

```

Point A{0,0}, B{2,2}, C{0,2}, D{2,0}, P{1,0};
printf("%d\n", segment_intersect(A, B, C, D)); // 相交 = 1
printf("%.10f\n", distance_point_to_segment(P, A, B)); // 距离 = sqrt(2)/2

```

样例输出：1；0.7071067812。

【传参要求（照这个传不会错）】

- `on_segment(p, a, b)`: p 是否在线段 ab 上（含端点），返回 `bool`。
- `segment_intersect(a1, a2, b1, b2)`: 两线段 $a1a2$ 、 $b1b2$ 是否相交（含端点接触与共线重叠），返回 `bool`。
- `distance_point_to_line(p, a, b)`: 点 p 到直线 ab 的距离，返回 `double`；要求 a 、 b 不重合。
- `distance_point_to_segment(p, a, b)`: 点 p 到线段 ab 的最短距离，返回 `double`（已处理垂足落在线段外的情况）。

```

bool on_segment(Point p, Point a, Point b) {
    return sgn(cross(a, b, p)) == 0 && sgn(dot(p - a, p - b)) <= 0;
}

bool segment_intersect(Point a1, Point a2, Point b1, Point b2) {
    double c1 = cross(a1, a2, b1), c2 = cross(a1, a2, b2);
    double c3 = cross(b1, b2, a1), c4 = cross(b1, b2, a2);
    if (sgn(c1) * sgn(c2) < 0 && sgn(c3) * sgn(c4) < 0) return true;
    return on_segment(b1, a1, a2) || on_segment(b2, a1, a2) ||
           on_segment(a1, b1, b2) || on_segment(a2, b1, b2);
}

```

```
double distance_point_to_line(Point p, Point a, Point b) {
    return fabs(cross(b - a, p - a)) / norm(b - a);
}

double distance_point_to_segment(Point p, Point a, Point b) {
    if (sgn(dot(b - a, p - a)) < 0) return dist(p, a);
    if (sgn(dot(a - b, p - b)) < 0) return dist(p, b);
    return distance_point_to_line(p, a, b);
}
```

多边形面积与点包含

赛时先看

题目信号：围栏、区域、内部/外部判断。

本质：多边形面积、判断点在多边形内。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；`solve()` 里直接调用本节列出的函数即可。

复杂度判定： $O(n)$ 。

维护的量：只维护输入多边形顶点序列（按序的 `vector<Point>`，`poly` / `p`）。

警告：`polygon_area2` 返回两倍有向面积（真实面积需除以 2）；`point_in_polygon` 对边界返回 1、内部 2、外部 0，判断时要单独区分边界。

【最小完整示例（先抄这一段就能跑）】

题目：求三角形面积，并判断点是否在三角形内部。

```
vector<Point> tri{{0,0},{2,0},{0,2}};
printf("%.10f\n", polygon_area2(tri) / 2); // 面积 = 2
printf("%d\n", point_in_polygon({1,1}, tri)); // 内部 = 2
printf("%d\n", point_in_polygon({3,3}, tri)); // 外部 = 0
```

样例输出：2.0000000000; 2; 0。

【传参要求（照这个传不会错）】

- `polygon_area2(p)`: `p` 为按序顶点（`vector<Point>`，顺逆皆可）；返回 2 倍有向面积（`double`），真实面积取 `fabs(结果)/2`。
- `point_in_polygon(q, poly)`: `q` 为查询点，`poly` 为按序顶点（顺逆皆可）；返回 0=外部、1=边界（含顶点与边上）、2=内部。

```
double polygon_area2(const vector<Point>& p) {
    double s = 0;
    int n = (int)p.size();
    for (int i = 0; i < n; ++i) s += cross(p[i], p[(i + 1) % n]);
    return s;
}

int point_in_polygon(Point q, const vector<Point>& poly) {
    bool in = false;
    int n = (int)poly.size();
    for (int i = 0, j = n - 1; i < n; j = i++) {
        Point a = poly[j], b = poly[i];
        if (on_segment(q, a, b)) return 1; // 点在边界上。
        bool cross_ray = ((a.y > q.y) != (b.y > q.y)) &&
            (q.x < (b.x - a.x) * (q.y - a.y) / (b.y - a.y) + a.x);
        if (cross_ray) in = !in;
    }
    return in ? 2 : 0; // 0 表示外部，2 表示内部。
}
```

简单多边形耳切三角剖分

赛时先看

题目信号：多边形是简单多边形但不一定凸；后续算法只会处理三角形或凸多边形；`n` 不大， $O(n^3)$ 可接受。

本质：把简单多边形拆成 `n-2` 个三角形，常用于把非凸多边形问题转成三角形对/凸多边形问题。

接法：每轮枚举连续三点（`prev`, `cur`, `next`），若它是凸角，且其它点不在该三角形内部，就切下这个三角形并删除 `cur`。

复杂度判定：朴素耳切 $O(n^3)$ ；精细维护可做到 $O(n^2)$ 。

维护的量：维护存活顶点下标表 `alive` 与结果 `triangles`（`n-2` 个三角形）。

警告：输入点最好统一成逆时针；实现只检查凸角与三角形内无其他顶点；对简单多边形该条件等价于耳点判定；共线点会让实现变麻烦，必要时先删除连续共线点。

【最小完整示例（先抄这一段就能跑）】

题目：把凸四边形剖成三角形，输出三角形个数。

```
vector<EarPoint> poly{{0,0},{2,0},{2,2},{0,2}};
auto t = ear_clipping_triangulation(poly);
printf("%zu\n", t.size()); // 三角形数 = n-2 = 2
```

样例输出：2。

【传参要求（照这个传不会错）】

- EarPoint(x, y): 顶点, long double, 读入用 long double 类型。
- ear_clipping_triangulation(poly): 输入简单多边形顶点 (vector<EarPoint>, 顺序任意, 内部自动翻为逆时针); 返回 vector<array<EarPoint,3>> 共 n-2 个三角形; 要求 n>=3 且无重复点、无连续共线点, 否则 assert 失败。
- point_in_triangle_strict(p, a, b, c): p 是否严格在三角形 abc 内部, 返回 bool (不统计边界)。
- polygon_signed_area2(p): 2 倍有向面积, 负值表示顺时针。

```
struct EarPoint {
    long double x = 0, y = 0;
    EarPoint() = default;
    EarPoint(long double x_, long double y_) : x(x_), y(y_) {}
    EarPoint operator+(const EarPoint& o) const { return {x + o.x, y + o.y}; }
    EarPoint operator-(const EarPoint& o) const { return {x - o.x, y - o.y}; }
};

long double cross(EarPoint a, EarPoint b) { return a.x * b.y - a.y * b.x; }
long double cross(EarPoint a, EarPoint b, EarPoint c) { return cross(b - a, c - a); }

long double polygon_signed_area2(const vector<EarPoint>& p) {
    long double s = 0;
    for (int i = 0; i < (int)p.size(); ++i) s += cross(p[i], p[(i + 1) % p.size()]);
    return s;
}

bool point_in_triangle_strict(EarPoint p, EarPoint a, EarPoint b, EarPoint c) {
    long double c1 = cross(a, b, p);
    long double c2 = cross(b, c, p);
    long double c3 = cross(c, a, p);
    return c1 > 1e-12L && c2 > 1e-12L && c3 > 1e-12L;
}

vector<array<EarPoint, 3>> ear_clipping_triangulation(vector<EarPoint> poly) {
    int n = (int)poly.size();
    assert(n >= 3);
    if (polygon_signed_area2(poly) < 0) reverse(poly.begin(), poly.end());
    vector<int> alive(n);
    iota(alive.begin(), alive.end(), 0);
    vector<array<EarPoint, 3>> triangles;

    while ((int)alive.size() > 3) {
        bool cut = false;
        int m = (int)alive.size();
        for (int i = 0; i < m && !cut; ++i) {
            int ia = alive[(i - 1 + m) % m];
            int ib = alive[i];
            int ic = alive[(i + 1) % m];
            EarPoint a = poly[ia], b = poly[ib], c = poly[ic];
            if (cross(a, b, c) <= 1e-12L) continue;
            bool has_inside = false;
            for (int id : alive) {
                if (id == ia || id == ib || id == ic) continue;
                if (point_in_triangle_strict(poly[id], a, b, c)) {
                    has_inside = true;
                    break;
                }
            }
            if (!has_inside) {
                triangles.push_back({a, b, c});
                alive.erase(alive.begin() + i);
                cut = true;
            }
        }
        assert(cut); // 若失败, 通常是有重复点、共线点或输入不是简单多边形。
    }
    triangles.push_back({poly[alive[0]], poly[alive[1]], poly[alive[2]]});
    return triangles;
}
```

典题：本场 K 《Geometry Textbook》。先对两个简单多边形分别耳切，枚举三角形对；每对三角形的闵可夫斯基和是凸多边形，再求这些凸多边形的面积并。

Andrew 凸包

赛时先看

题目信号：最大距离、包围所有点、凸多边形。

本质：点集最外层边界。

接法：点集外壳、最远点对、凸多边形处理前，先

`convex_hull(points)`。返回点按凸包顺序排列，不重复首点；如果返回点数小于 3，很多多边形算法要单独处理。

复杂度判定： $O(n \log n)$ 。

维护的量：维护凸壳栈 `h`（下凸壳 + 上凸壳拼接）；返回值即按凸包顺序排列的顶点。

警告：如果要保留边上的共线点，把 `<= 0` 改成 `< 0`。

【最小完整示例（先抄这一段就能跑）】

题目：给 4 个点求凸包，输出凸包点数。

```
vector<Point> pts{{0,0},{1,0},{0,1},{1,1}};
auto hull = convex_hull(pts);
printf("%zu\n", hull.size()); // 凸包点数 = 4
```

样例输出：4。

【传参要求（照这个传不会错）】

- `convex_hull(p)`：输入任意点集（`vector<Point>`，可含重复点，内部自动去重）；返回 `vector<Point>`，逆时针凸包顶点，首点不重复；点数 `<=1` 时原样返回；返回点数 `<3` 时不是多边形，后续多边形算法需单独处理。

【不会用就照抄】

```
auto hull = convex_hull(points);
```

- 先看本节是否保留共线边界点；会影响周长、点数、后续旋转卡壳。
- 点类型和叉积类型不要临时改窄，坐标大时用 `i64/i128`。

【API / 入口函数（赛时只认这里列的名字）】

- `convex_hull(vector<Point> p)` -> Andrew 凸包 返回 `vector<Point>`。

【核心逻辑（改代码时别破坏）】

- 先按坐标排序；分别维护下凸壳和上凸壳。
- 出现非期望转向就弹栈；`<=0` 会丢掉边上共线点，改成 `<0` 可保留。

```
vector<Point> convex_hull(vector<Point> p) {
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()), p.end());
    if (p.size() <= 1) return p;

    vector<Point> h;
    for (auto pt : p) {
        while (h.size() >= 2 && sgn(cross(h[h.size()-2], h.back(), pt)) <= 0) h.pop_back();
        h.push_back(pt);
    }
    int lower = (int)h.size();
    for (int i = (int)p.size() - 2; i >= 0; --i) {
        while ((int)h.size() > lower && sgn(cross(h[h.size()-2], h.back(), p[i])) <= 0) h.pop_back();
        h.push_back(p[i]);
    }
    h.pop_back();
    return h;
}
```

凸多边形闵可夫斯基和

赛时先看

题目信号：题目给出两个凸包，出现“所有两点坐标相加/相减”“边按极角合并”“凸多边形平移后是否相交”等描述；若要判断 `A` 与 `B` 的交叠，常构造 `A + (-B)` 再判断原点位置。

本质：求两个凸多边形的和集 $A + B = \{a + b \mid a \in A, b \in B\}$ ；

常用来把两个物体的相对运动、凸多边形碰撞或“两个凸集合点对差”转成一次凸包操作。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；外部只调用下面 API

列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定：输入顶点按逆时针顺序给出时为 $O(n + m)$ ；若还需要先求凸包，合计 $O((n + m) \log(n + m))$ 。

维护的量：维护 `IPoint` (`i64` 整数坐标) 与 `i128` 叉积 `cross_i`；结果和集顶点序列 `ans`。

警告：代码使用整数坐标和 `i128`

叉积，避免乘法溢出；输入必须是非退化、严格凸的凸包，顶点可从任意位置开始。若输入只是散点，先用 Andrew 凸包；若题目允许边上连续共线点，最好先删去共线中间点。

【最小完整示例（先抄这一段就能跑）】

题目：三角形与三角形求闵可夫斯基和，输出和集顶点数。

```
vector<IPoint> a{{0,0},{2,0},{0,2}};
vector<IPoint> b{{0,0},{1,0},{0,1}};
auto s = minkowski_sum_convex(a, b);
printf("%zu\n", s.size()); // 和集顶点数 = 6 (六边形)
```

样例输出：6。

【传参要求（照这个传不会错）】

- `IPoint(x, y)`: 整数坐标，`i64`。
- `minkowski_sum_convex(a, b)`: 输入两个严格凸、非退化凸多边形顶点（`vector<IPoint>`，顺序任意，内部自动调 `normalize_convex_polygon`）；返回 `vector<IPoint>` 逆时针和集顶点；单个点（退化）也支持。
- `normalize_convex_polygon(p)`: 把多边形翻为逆时针，并让最低、再最靠左的顶点做 0 号位置，返回 `vector<IPoint>`。
- `polygon_area2_i(p)`: 2 倍有向面积，`i128`。
- `cross_i(a, b)`: `i128` 叉积，防溢出。

【API / 入口函数（赛时只认这里列的名字）】

- `minkowski_sum_convex(vector<IPoint> a, vector<IPoint> b) ->`
求两个凸多边形（非退化、严格凸）的闵可夫斯基和，返回顶点序列。返回前会自动调 `normalize_convex_polygon` 规范化输入。
- `normalize_convex_polygon(vector<IPoint> p) ->` 把多边形转成逆时针，并把最低、再最靠左的点旋到 0 号位置。返回 `vector<IPoint>`。

```
struct IPoint {
    i64 x = 0, y = 0;
    IPoint() = default;
    IPoint(i64 x_, i64 y_) : x(x_), y(y_) {}
    IPoint operator+(const IPoint& o) const { return {x + o.x, y + o.y}; }
    IPoint operator-(const IPoint& o) const { return {x - o.x, y - o.y}; }
    bool operator==(const IPoint& o) const { return x == o.x && y == o.y; }
};

i128 cross_i(IPoint a, IPoint b) {
    return (i128)a.x * b.y - (i128)a.y * b.x;
}

i128 polygon_area2_i(const vector<IPoint>& p) {
    i128 s = 0;
    for (int i = 0; i < (int)p.size(); ++i) s += cross_i(p[i], p[(i + 1) % p.size()]);
    return s;
}

// 把多边形转成逆时针，并把最低、再最靠左的点旋到 0 号位置。
vector<IPoint> normalize_convex_polygon(vector<IPoint> p) {
    if (p.size() > 1 && p.front() == p.back()) p.pop_back();
    if (p.size() <= 1) return p;
    if (polygon_area2_i(p) < 0) reverse(p.begin(), p.end());
    int start = 0;
    for (int i = 1; i < (int)p.size(); ++i) {
        if (p[i].y < p[start].y || (p[i].y == p[start].y && p[i].x < p[start].x)) {
            start = i;
        }
    }
    rotate(p.begin(), p.begin() + start, p.end());
    return p;
}

vector<IPoint> minkowski_sum_convex(vector<IPoint> a, vector<IPoint> b) {
    a = normalize_convex_polygon(a);
    b = normalize_convex_polygon(b);
    if (a.empty() || b.empty()) return {};
    if (a.size() == 1) {
        for (auto& p : b) p = p + a[0];
        return b;
    }
    if (b.size() == 1) {
        for (auto& p : a) p = p + b[0];
    }
}
```

```

    return a;
}

int n = (int)a.size(), m = (int)b.size();
vector<IPoint> ans;
ans.reserve(n + m);
ans.push_back(a[0] + b[0]);
int i = 0, j = 0;
while (i < n || j < m) {
    IPoint da = (i < n ? a[(i + 1) % n] - a[i] : IPoint{});
    IPoint db = (j < m ? b[(j + 1) % m] - b[j] : IPoint{});
    if (j == m || (i < n && cross_i(da, db) > 0)) {
        ans.push_back(ans.back() + da);
        ++i;
    } else if (i == n || cross_i(da, db) < 0) {
        ans.push_back(ans.back() + db);
        ++j;
    } else {
        ans.push_back(ans.back() + da + db);
        ++i;
        ++j;
    }
}
ans.pop_back(); // 最终累加得到的点应等于 ans[0]。
return ans;
}

```

典型模型：若 B 中每个点取相反数并保持逆时针顺序，就得到 $A + (-B)$ 。原点在其中（边界也算）等价于 A 、 B 有公共点；这是凸多边形相交和“相对位移可行性”最常用的变形。点在凸多边形内可直接接本册已有的 `point_in_convex_polygon`。

旋转卡壳：凸包直径

赛时先看

题目信号：点集最大距离，先求凸包再处理。

本质：凸包上最远点对。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；`solve()` 里直接调用本节列出的函数即可。

复杂度判定：凸包后 $O(n)$ 。

维护的量：维护对踵点指针 j 与当前最优距离平方 `ans`。

警告：输入必须是凸包顺序。`convex_diameter2` 返回的是距离平方，输出时需开根。

【最小完整示例（先抄这一段就能跑）】

题目：求凸包上最远点对的距离（已先求好凸包）。

```

vector<Point> p{{0,0},{4,0},{4,3}};
printf("%.10f\n", sqrt(convex_diameter2(p))); // 直径 = 5

```

样例输出：5.0000000000。

【传参要求（照这个传不会错）】

- `convex_diameter2(p)`：输入凸包顶点（`vector<Point>`，逆时针、首点不重复）；返回最远点对距离的平方（`double`），输出时 `sqrt`； $n \leq 1$ 返回 0， $n = 2$ 返回两点距离平方。

```

double convex_diameter2(const vector<Point>& p) {
    int n = (int)p.size();
    if (n <= 1) return 0;
    if (n == 2) return norm2(p[0] - p[1]);
    double ans = 0;
    int j = 1;
    for (int i = 0; i < n; ++i) {
        int ni = (i + 1) % n;
        while (fabs(cross(p[ni] - p[i], p[(j + 1) % n] - p[i])) >
                fabs(cross(p[ni] - p[i], p[j] - p[i])) + EPS) {
            j = (j + 1) % n;
        }
        ans = max(ans, norm2(p[i] - p[j]));
        ans = max(ans, norm2(p[ni] - p[j]));
    }
    return ans;
}

```

最近点对

赛时先看

题目信号： n 到 $1e5$ ，问最近距离，暴力不行。

本质：平面点集中最近两点距离。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接调用本节列出的函数即可。

复杂度判定： $O(n \log n)$ 。

维护的量：维护归并缓冲 `tmp` 与递归区间 `[l, r]`；全程只传距离平方避免开根误差。

警告：返回距离平方更稳；有重点时答案为 0。

【最小完整示例（先抄这一段就能跑）】

题目：求平面点集中最近点对的距离。

```
vector<Point> p{{0,0},{3,4},{1,0},{100,100}};
printf("%.10f\n", sqrt(closest_pair(p))); // 最近距离 = 1
```

样例输出：1.0000000000。

【传参要求（照这个传不会错）】

- `closest_pair(p)`：输入任意点集（`vector<Point>`，内部会排序，不修改外部语义）；返回最近点对距离平方（`double`），输出时 `sqrt`；有重复点返回 0； $n \leq 1$ 返回 `1e100`。
- `closest_pair_rec(p, tmp, l, r)`：内部递归函数，处理 `[l, r]` 区间，`tmp` 为同大小缓冲；不要从 `solve()` 直接调用，一律走 `closest_pair`。

```
double closest_pair_rec(vector<Point>& p, vector<Point>& tmp, int l, int r) {
    if (r - l <= 3) {
        double best = 1e100;
        for (int i = l; i < r; ++i) {
            for (int j = i + 1; j < r; ++j) best = min(best, norm2(p[i] - p[j]));
        }
        sort(p.begin() + l, p.begin() + r, [](Point a, Point b) { return a.y < b.y; });
        return best;
    }
    int m = (l + r) >> 1;
    double midx = p[m].x;
    double best = min(closest_pair_rec(p, tmp, l, m), closest_pair_rec(p, tmp, m, r));
    merge(p.begin() + l, p.begin() + m, p.begin() + m, p.begin() + r, tmp.begin(),
        [](Point a, Point b) { return a.y < b.y; });
    copy(tmp.begin(), tmp.begin() + (r - l), p.begin() + l);

    int cnt = 0;
    for (int i = l; i < r; ++i) {
        if ((p[i].x - midx) * (p[i].x - midx) >= best) continue;
        for (int j = cnt - 1; j >= 0 && (p[i].y - tmp[j].y) * (p[i].y - tmp[j].y) < best; --j) {
            best = min(best, norm2(p[i] - tmp[j]));
        }
        tmp[cnt++] = p[i];
    }
    return best;
}

double closest_pair(vector<Point> p) {
    sort(p.begin(), p.end(), [](Point a, Point b) {
        if (sgn(a.x - b.x) != 0) return a.x < b.x;
        return a.y < b.y;
    });
    vector<Point> tmp(p.size());
    return closest_pair_rec(p, tmp, 0, (int)p.size());
}
```

动态 K-D Tree：二维加点与矩形权值和

赛时先看

题目信号：操作被答案异或或加密，无法离线；每次只有一个点增加权值，但查询是 `[x1,x2] × [y1,y2]` 的矩形和；坐标可到 `1e9` 或更大。

本质：强制在线维护二维带权点，支持插入/给某坐标加权，以及查询轴对齐矩形内的权值和。比二维 BIT、树套树更适合坐标范围极大且不能离线离散化的题。

接法：初始全零平面，在线执行 `1 x y a` 表示 `(x,y)` 增加 `a`，执行 `2 x1 y1 x2 y2`

询问矩形和。若每个操作参数都要与上一次答案异或，二维 BIT/CDQ 无法离线，这棵 K-D Tree 就可直接套用。

复杂度判定：替罪羊重构保证树高均摊 $O(\log n)$ ；矩形查询依靠包围盒剪枝，均摊通常很快，但理论最坏仍可能 $O(n)$ 。这是 K-D Tree 的本质限制。

维护的量：维护节点池 `tr`（0 号为空节点）与树根 `root`；每节点维护子树权值和 `sum`、包围盒 `min_x/max_x/min_y/max_y`、子树大小 `sz`。

警告：这里同坐标插入会累加权值，不新增结点；查询端点是闭区间；权值和、坐标都使用 `i64`。模板只支持插入，不支持删除；有删除时需加懒删除计数并将“有效点比例过低”也作为重构条件。

【最小完整示例（先抄这一段就能跑）】

题目：在线给点加权，询问矩形权值和。

```
KDTree2D kd;
kd.add_point(1, 1, 5);
kd.add_point(2, 3, 7);
printf("%lld\n", kd.rectangle_sum(0, 0, 2, 2)); // 闭区间 [0,2]x[0,2] 的和 = 5
```

样例输出：5。

【传参要求（照这个传不会错）】

- `KDTree2D kd`：直接默认构造，内部从空树开始。
- `add_point(x, y, val)`：坐标为 `i64`（`1e9` 也可直接用）；在 `(x,y)` 累加 `val`，同坐标不新增结点。
- `rectangle_sum(x1, y1, x2, y2)`：闭区间矩形 `[x1,x2] × [y1,y2]` 的权值和，返回 `i64`；端点大小关系随意，内部自动 `swap` 纠正。
- 内部 `build` / `insert` / `query` 是递归实现细节，不要从 `solve()` 直接调。

【API / 入口函数（赛时只认这里列的名字）】

- `add_point(i64 x, i64 y, i64 val)` → 在 `(x,y)` 累加权值 `val`（同坐标不新增结点）。
- `rectangle_sum(i64 x1, i64 y1, i64 x2, i64 y2)` → 闭区间矩形权值和 返回 `i64`。
- 内部 `build` / `insert` / `query` 是递归实现细节，不要从 `solve()` 直接调。

【改板时先认这几个量】

- `sz`：子树大小。
- `tr`：树节点池（`tr[0]` 表示空节点）。
- `root`：树根节点编号。
- `depth`：深度。
- `sum`：子树权值和（含自身）。

```
struct KDTree2D {
    static constexpr double ALPHA = 0.75;

    struct Node {
        i64 x, y, val, sum;
        i64 min_x, max_x, min_y, max_y;
        int ch[2] = {0, 0};
        int sz = 1;
    };

    vector<Node> tr = {Node{}}; // 0 号下标表示空节点。
    int root = 0;

    int size_of(int u) const { return u ? tr[u].sz : 0; }
    i64 sum_of(int u) const { return u ? tr[u].sum : 0; }

    int new_node(i64 x, i64 y, i64 val) {
        tr.push_back({x, y, val, val, x, x, y, y});
        return (int)tr.size() - 1;
    }

    void pull(int u) {
        Node& p = tr[u];
        p.sz = 1;
        p.sum = p.val;
        p.min_x = p.max_x = p.x;
        p.min_y = p.max_y = p.y;
        for (int v : p.ch) {
            if (!v) continue;
            p.sz += tr[v].sz;
            p.sum += tr[v].sum;
            p.min_x = min(p.min_x, tr[v].min_x);
            p.max_x = max(p.max_x, tr[v].max_x);
            p.min_y = min(p.min_y, tr[v].min_y);
            p.max_y = max(p.max_y, tr[v].max_y);
        }
    }

    void flatten(int u, vector<int>& ids) {
        if (!u) return;
        flatten(tr[u].ch[0], ids);
        ids.push_back(u);
        flatten(tr[u].ch[1], ids);
    }
}
```

```

int build(vector<int>& ids, int l, int r, int depth) {
    if (l >= r) return 0;
    int mid = (l + r) >> 1;
    int axis = depth & 1;
    nth_element(ids.begin() + l, ids.begin() + mid, ids.begin() + r, [&](int a, int b) {
        if (axis == 0) {
            if (tr[a].x != tr[b].x) return tr[a].x < tr[b].x;
            return tr[a].y < tr[b].y;
        }
        if (tr[a].y != tr[b].y) return tr[a].y < tr[b].y;
        return tr[a].x < tr[b].x;
    });
    int u = ids[mid];
    tr[u].ch[0] = build(ids, l, mid, depth + 1);
    tr[u].ch[1] = build(ids, mid + 1, r, depth + 1);
    pull(u);
    return u;
}

bool unbalanced(int u) const {
    return max(size_of(tr[u].ch[0]), size_of(tr[u].ch[1])) > ALPHA * tr[u].sz;
}

void rebuild(int& u, int depth) {
    vector<int> ids;
    ids.reserve(tr[u].sz);
    flatten(u, ids);
    u = build(ids, 0, (int)ids.size(), depth);
}

void insert(int& u, i64 x, i64 y, i64 val, int depth) {
    if (!u) {
        u = new_node(x, y, val);
        return;
    }
    if (tr[u].x == x && tr[u].y == y) {
        tr[u].val += val;
        pull(u);
        return;
    }
    int axis = depth & 1;
    bool go_right = axis == 0 ? make_pair(x, y) > make_pair(tr[u].x, tr[u].y)
        : make_pair(y, x) > make_pair(tr[u].y, tr[u].x);
    // 不要把 tr[u].ch[...] 按引用传入: 深层递归里的 push_back
    // 可能导致 tr 重新分配, 使该引用失效。
    int child = tr[u].ch[go_right];
    insert(child, x, y, val, depth + 1);
    tr[u].ch[go_right] = child;
    pull(u);
    if (unbalanced(u)) rebuild(u, depth);
}

void add_point(i64 x, i64 y, i64 val) {
    insert(root, x, y, val, 0);
}

bool disjoint(int u, i64 x1, i64 y1, i64 x2, i64 y2) const {
    return tr[u].max_x < x1 || x2 < tr[u].min_x || tr[u].max_y < y1 || y2 < tr[u].min_y;
}

bool covered(int u, i64 x1, i64 y1, i64 x2, i64 y2) const {
    return x1 <= tr[u].min_x && tr[u].max_x <= x2 && y1 <= tr[u].min_y && tr[u].max_y <= y2;
}

i64 query(int u, i64 x1, i64 y1, i64 x2, i64 y2) const {
    if (!u || disjoint(u, x1, y1, x2, y2)) return 0;
    if (covered(u, x1, y1, x2, y2)) return tr[u].sum;
    i64 ans = (x1 <= tr[u].x && tr[u].x <= x2 && y1 <= tr[u].y && tr[u].y <= y2) ? tr[u].val : 0;
    return ans + query(tr[u].ch[0], x1, y1, x2, y2) + query(tr[u].ch[1], x1, y1, x2, y2);
}

i64 rectangle_sum(i64 x1, i64 y1, i64 x2, i64 y2) const {
    if (x1 > x2) swap(x1, x2);
    if (y1 > y2) swap(y1, y2);
    return query(root, x1, y1, x2, y2);
}
};

```

典型模型：初始全零平面，在线执行 $1\ x\ y\ a$ 表示 (x, y) 增加 a ，执行 $2\ x1\ y1\ x2\ y2$

询问矩形和。若每个操作参数都要与上一次答案异或，二维 BIT/CDQ 无法离线，这棵 K-D Tree 就可直接套用。

直线与圆交点

赛时先看

题目信号：圆、半径、直线穿过圆、切线。

本质：圆相关几何基础。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里先构造对象，再调用其公开接口。

复杂度判定： $O(1)$ 。

维护的量：维护 `Line{p, v}`（参数式直线：点 + 方向向量）与 `Circle{o, r}`（圆心 + 半径）。

警告：先判断点到圆心距离和半径关系。

【最小完整示例（先抄这一段就能跑）】

题目：求直线与圆的交点个数。

```
Line L{{0, 0}, {1, 0}}; // 直线 y = 0
Circle C{{0, 0}, 2}; // 圆心原点, 半径 2
auto res = line_circle_intersection(L, C);
printf("%zu\n", res.size()); // 交点个数 = 2
```

样例输出：2。

【传参要求（照这个传不会错）】

- `Line{p, v}`: `p` 为直线上一点, `v` 为非零方向向量, 直线为 $p + t \cdot v$ 。
- `Circle{o, r}`: `o` 为圆心, `r` 为半径 (double)。
- `line_circle_intersection(line, c)`: 返回 `vector<Point>`; 0 个=不相交, 1 个=相切 (返回切点), 2 个=相交 (两个交点)。

```
struct Line {
    Point p, v; // 直线参数式 p + t * v。
};

struct Circle {
    Point o;
    double r;
};

vector<Point> line_circle_intersection(Line line, Circle c) {
    Point p = line.p;
    Point v = line.v;
    Point foot = p + v * (dot(c.o - p, v) / norm2(v));
    double d2 = norm2(foot - c.o);
    if (sgn(d2 - c.r * c.r) > 0) return {};
    if (sgn(d2 - c.r * c.r) == 0) return {foot};
    double len = sqrt(c.r * c.r - d2) / norm(v);
    return {foot - v * len, foot + v * len};
}
```

向量旋转与直线交点

赛时先看

题目信号：旋转、反射、两条直线交点。

本质：角度变换、求两直线交点。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接调用本节列出的函数即可。

复杂度判定： $O(1)$ 。

维护的量：无额外结构；`rotate` 返回旋转后的点，`line_intersection` 把交点写进 `out` 并返回是否相交。

警告：平行时没有唯一交点。

【最小完整示例（先抄这一段就能跑）】

题目：(1, 0) 绕原点逆时针转 90° ；直线 $(0, 0) + t \cdot (1, 1)$ 与 $(1, 0) + s \cdot (0, 1)$ 求交点。

```
Point a = rotate({1, 0}, acos(-1.0) / 2); // (1, 0) 逆时针转 90°, 得 (0, 1)
Point out; // 交点写进 out
bool ok = line_intersection({0, 0}, {1, 1}, // 直线1: 过 (0, 0)、方向 (1, 1)
                           {1, 0}, {0, 1}, out); // 直线2: 过 (1, 0)、方向 (0, 1)
printf("%d %.1f %.1f\n", ok, out.x, out.y); // 交点为 (1, 1)
```

样例输出：1 1.0 1.0。

【传参要求（照这个传不会错）】

- `rotate(a, rad)`: `a` 为原向量 `Point{x, y}`; `rad` 为弧度制转角 (正 = 逆时针); 返回旋转后的 `Point`。
- `line_intersection(p, v, q, w, out)`: `p/q` 为两条直线上各一点, `v/w` 为对应方向向量 (`Point`, 非零); `out` 为 `Point&` 引用, 相交时写入交点; 返回 `bool`, `true` = 有唯一交点, `false` = 平行 (此时 `out` 不变)。

- 直线是参数式 $p + t \cdot v$; `cross(v, w) == 0` 表示平行, 没有唯一交点。

```
Point rotate(Point a, double rad) {
    double c = cos(rad), s = sin(rad);
    return {a.x * c - a.y * s, a.x * s + a.y * c};
}

bool line_intersection(Point p, Point v, Point q, Point w, Point& out) {
    double d = cross(v, w);
    if (sgn(d) == 0) return false;
    double t = cross(q - p, w) / d;
    out = p + v * t;
    return true;
}
```

一维移动墙反弹事件模拟

赛时先看

题目信号: 两条边界位置是一次函数; 碰撞只改变速度符号; 数据较小或用于验证解析式。

本质: 小球在一维线性移动边界之间反弹, 碰到边界后速度取反, 求某个时刻位置。

复杂度判定: 0 (碰撞次数)。

维护的量: t (当前时间)、 x (当前位置)、 v (当前速度); 每步取最早碰撞时刻 `best_dt` 与撞墙方向 `hit`, 撞墙后把 x 贴墙并 $v = -v$ 。

警告: 该模板按题意“速度直接取反”, 不做物理中的相对速度反射; 如果碰撞次数可能极大, 需要另推周期/展开公式。内部以 $2e6$ 步为上限, 超出会静默返回当前位置; 起点贴墙 (`dt` 近似 0) 的场景需自行特判。

【最小完整示例 (先抄这一段就能跑)】

题目: 小球从 $x=0$ 以 $v=1$ 向右, 左右墙固定 (左 0 右 10), 求 $t=12$ 时位置。

```
double ans = simulate_moving_walls(0, 1, 0, 10, 0, 0, 12);
printf("%.1f\n", ans); // t=10 碰右墙弹回, t=12 时 x=8
```

样例输出: 8.0。

【传参要求 (照这个传不会错)】

- x : 起点位置 (double)。
- v : 初速度 (double, 向右为正)。
- `left0` / `right0`: $t=0$ 时左右墙位置 (double, 须 `left0 < right0`)。
- `left_speed` / `right_speed`: 左右墙的匀速速度 (double, 向右为正, 可负可 0)。
- `end_time`: 模拟截止时刻 (double, 须 ≥ 0)。
- 返回值: double, `end_time` 时刻小球位置; 内部最多 $2e6$ 步, 超限静默返回当前位置。

```
double simulate_moving_walls(
    double x,
    double v,
    double left0,
    double right0,
    double left_speed,
    double right_speed,
    double end_time
) {
    const double EPS = 1e-12;
    double t = 0;
    for (int steps = 0; steps < 2000000 && t + EPS < end_time; steps++) {
        double left = left0 + left_speed * t;
        double right = right0 + right_speed * t;
        double best_dt = end_time - t;
        int hit = 0;

        double den_left = v - left_speed;
        if (den_left < -EPS) {
            double dt = (left - x) / den_left;
            if (dt > EPS && dt < best_dt) {
                best_dt = dt;
                hit = -1;
            }
        }

        double den_right = v - right_speed;
        if (den_right > EPS) {
            double dt = (right - x) / den_right;
            if (dt > EPS && dt < best_dt) {
                best_dt = dt;
                hit = 1;
            }
        }

        x = left + best_dt * v;
        v = -v;
        t += best_dt;
    }
    return x;
}
```

```

    }

    x += v * best_dt;
    t += best_dt;
    if (hit == 0) break;
    if (hit == -1) x = left0 + left_speed * t;
    else x = right0 + right_speed * t;
    v = -v;
}
return x;
}

```

极坐标圆弧图最短路：关键角度建图

注意：本节仅记录思路与建图要点，无直接可抄代码；赛时需按题面现场建图 + Dijkstra。

赛时先看

题目信号：给若干圆弧 $(r, \theta_{l1}, \theta_{r1})$ ；同一圆弧可沿角度走，同一角度可沿半径走；起点终点在极坐标上；若把每个角度与每条圆弧都连边，图会变成 $O(n^2)$ 。

本质：把圆弧和径向移动的连续几何问题离散成 $O(n)$ 个关键点，再跑 Dijkstra。

接法：按半径从外到内扫描圆弧，用“区间取 max 覆盖 + 单点查询”维护当前角度最外层弧。每条弧的两个端点向外查询最近可达弧，加入关键点。随后按同一角度连径向边，按同一圆弧上相邻关键角连弧长边，最后 Dijkstra。

复杂度判定：关键点和边 $O(n)$ ，用线段树找外层/内层相邻弧 $O(n \log U)$ ，Dijkstra $O(n \log n)$ 。

警告：跨越 θ 角度的圆弧要拆成两段；只有圆弧端点、起点角、终点角以及端点径向碰到的第一条弧是关键点；径向边只连同一角度下相邻半径的点。

典题：本场 B《Cast off as Cast》。角度值离散在

$0..99999$ ，线段树维护角度区间当前最外层圆弧，最终图规模线性。

点在凸多边形内

赛时先看

题目信号：凸多边形固定，查询点很多。

本质：多次判断点是否在凸包内。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接调用本节列出的函数即可。

复杂度判定： $O(\log n)$ 。

维护的量：无额外结构；只读凸包顶点数组 `p`（逆时针、无重复点）与查询点 `q`。

警告：要求凸包点按逆时针且无重复点；边界按这里返回 `true`。

【最小完整示例（先抄这一段就能跑）】

题目：逆时针凸包 $(0,0), (4,0), (4,3), (0,3)$ ，查询 $(1,1)$ 与 $(5,2)$ 是否在凸包内。

```

vector<Point> hull = {{0,0}, {4,0}, {4,3}, {0,3}}; // 逆时针、无重复点
printf("%d %d\n",
    point_in_convex_polygon(hull, {1,1}), // 内部 → 1
    point_in_convex_polygon(hull, {5,2})); // 外部 → 0

```

样例输出：1 0。

【传参要求（照这个传不会错）】

- `p`: `vector<Point>`，凸包顶点，必须按逆时针顺序且无重复点；`n=1` 时按点重合判，`n=2` 时按 `on_segment` 判线段。
- `q`: 查询点 `Point{x,y}`，坐标任意。
- 返回值: `bool`, `true` = 在凸包内部或边界上（边界返回 `true`）。
- 依赖本文件同一套 `Point`、`cross(a,b,c)`、`sgn`、`on_segment`。

```

bool point_in_convex_polygon(const vector<Point>& p, Point q) {
    int n = (int)p.size();
    if (n == 1) return q == p[0];
    if (n == 2) return on_segment(q, p[0], p[1]);
    if (sgn(cross(p[1] - p[0], q - p[0])) < 0) return false;
    if (sgn(cross(p[n-1] - p[0], q - p[0])) > 0) return false;

    int l = 1, r = n - 1;
    while (r - l > 1) {
        int mid = (l + r) >> 1;
        if (sgn(cross(p[mid] - p[0], q - p[0])) >= 0) l = mid;
        else r = mid;
    }
    return sgn(cross(p[l], p[(l+1)%n], q)) >= 0;
}

```

Pick 定理

赛时先看

题目信号：多边形顶点都是整数坐标，问内部/边界格点数。

本质：整数坐标简单多边形的格点数量关系。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接调用本节列出的函数即可。

复杂度判定： $O(n)$ 。

维护的量：`interior_lattice_points` 内部维护 `a2`（两倍面积 $2S$ ）与 `b`（边界格点数 B ），按 Pick 公式 $I = (2S - B + 2) / 2$ 返回内部格点数。

警告： $2S = 2I + B - 2$ ，所以 $I = (2S - B + 2) / 2$ 。

【最小完整示例（先抄这一段就能跑）】

题目：三角形 $(0, 0), (4, 0), (0, 3)$ ，求内部与边界格点数。

```
vector<pair<i64, i64>> tri = {{0,0}, {4,0}, {0,3}};
printf("%lld %lld\n",
    interior_lattice_points(tri), // 内部格点数 I
    boundary_lattice_points(tri)); // 边界格点数 B
```

样例输出：3 8。

【传参要求（照这个传不会错）】

- `p`: `vector<pair<i64,i64>>`，简单多边形顶点（整数坐标，按顺序，顺时针皆可）。
- `boundary_lattice_points(p)`: 返回边界格点数 B （`i64`），每条边贡献 $\gcd(|dx|, |dy|)$ 。
- `area2_lattice_polygon(p)`: 返回两倍面积 $2S = |\sum(x_1*y_2 - x_2*y_1)|$ （`i64`）。
- `interior_lattice_points(p)`: 返回内部格点数 $I = (2S - B + 2) / 2$ （`i64`）。
- `gcd_abs(a,b)`: 内部辅助，取 $|a|, |b|$ 的 $\gcd$ （`i64`）。

```
i64 gcd_abs(i64 a, i64 b) {
    return std::gcd(llabs(a), llabs(b));
}

i64 boundary_lattice_points(const vector<pair<i64, i64>>& p) {
    int n = (int)p.size();
    i64 b = 0;
    for (int i = 0; i < n; ++i) {
        auto [x1, y1] = p[i];
        auto [x2, y2] = p[(i + 1) % n];
        b += gcd_abs(x1 - x2, y1 - y2);
    }
    return b;
}

i64 area2_lattice_polygon(const vector<pair<i64, i64>>& p) {
    int n = (int)p.size();
    i64 s = 0;
    for (int i = 0; i < n; ++i) {
        auto [x1, y1] = p[i];
        auto [x2, y2] = p[(i + 1) % n];
        s += x1 * y2 - x2 * y1;
    }
    return llabs(s);
}

i64 interior_lattice_points(const vector<pair<i64, i64>>& p) {
    i64 a2 = area2_lattice_polygon(p);
    i64 b = boundary_lattice_points(p);
    return (a2 - b + 2) / 2;
}
```

两圆交点

赛时先看

题目信号：两个圆的公共点。

本质：圆与圆相交、几何构造。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接调用本节列出的函数即可。

复杂度判定： $O(1)$ 。

维护的量：无；按圆心距 d 分类——外离/内含/同心返回空， $h2 == 0$ 相切返回 1 点，否则沿 `dir/perp` 构造 2 个交点。

警告：相离、内含、同心重合都没有两个普通交点。

【最小完整示例（先抄这一段就能跑）】

题目：圆 A 心 (0,0) 半径 2，圆 B 心 (3,0) 半径 2，求两圆交点。

```
vector<Point> res = circle_circle_intersection({{0,0}, 2}, {{3,0}, 2});
printf("%zu %.1f %.1f\n", res.size(), res[0].x, res[0].y); // 交点 (1.5, ±1.32)
```

样例输出：2 1.5 1.3。

【传参要求（照这个传不会错）】

- a / b: Circle{o, r}, o 为圆心 Point, r 为半径 (double, ≥ 0)。
- 返回值: vector<Point>; 空 = 相离/内含/同心重合, 1 个 = 相切 (返回切点), 2 个 = 相交 (两个交点)。
- 依赖 dist(a,b)、sgn 与本文件同一套 Point/Circle 定义。

```
vector<Point> circle_circle_intersection(Circle a, Circle b) {
    double d = dist(a.o, b.o);
    if (sgn(d) == 0) return {};
    if (sgn(d - (a.r + b.r)) > 0) return {};
    if (sgn(d - fabs(a.r - b.r)) < 0) return {};

    double x = (d * d - b.r * b.r + a.r * a.r) / (2 * d);
    double h2 = a.r * a.r - x * x;
    if (sgn(h2) < 0) return {};
    Point dir = (b.o - a.o) / d;
    Point base = a.o + dir * x;
    if (sgn(h2) == 0) return {base};
    Point perp{-dir.y, dir.x};
    double h = sqrt(h2);
    return {base + perp * h, base - perp * h};
}
```

半平面交

赛时先看

题目信号：二维不等式、多个有向直线左侧区域求交。

本质：求若干半平面的交，多边形裁剪、线性约束可行域。

接法：把本节 struct/class/函数组 整段抄到 solve() 上面；solve() 里先构造对象，再调用其公开接口。

复杂度判定：排序后 $O(n \log n)$ 。

维护的量：lines 为半平面（直线上一点 p + 方向 v，保留左侧）；双端队列 q 维护仍有效的直线、p 维护相邻直线交点，最终 p 即交区域顶点。

警告：直线方向要统一为保留左侧；平行线只保留更靠内的一条；最后还要处理队首队尾。结果点数 ≤ 2 时返回空向量，表示无界区域或空交，按题意自行处理。

【最小完整示例（先抄这一段就能跑）】

题目： $x \geq 0$ 、 $y \geq 0$ 、 $x+y \leq 4$ 三个半平面求交（每个半平面取方向向量左侧）。

```
vector<HalfPlaneLine> lines = {
    {{0,0}, {0,-1}}, // 过 (0,0) 方向向下：左侧即  $x \geq 0$ 
    {{0,0}, {1,0}}, // 过 (0,0) 方向向右：左侧即  $y \geq 0$ 
    {{4,0}, {-1,1}}, // 过 (4,0) 方向 (-1,1)：左侧即  $x+y \leq 4$ 
};
auto poly = half_plane_intersection(lines);
printf("%zu\n", poly.size()); // 交区域为三角形，3 个顶点
```

样例输出：3。

【传参要求（照这个传不会错）】

- lines: vector<HalfPlaneLine>, 每个为 {p, v}: p 为直线上一点, v 为方向向量 (HPoint, double), 半平面取 v 左侧 ($\text{cross}(v, \text{点}-p) \geq 0$)。
- ang 由函数内自动按 $\text{atan2}(v.y, v.x)$ 填好并排序，调用时不用管。
- 返回值: vector<HPoint>, 交区域的顶点 (逆时针); 点数 ≤ 2 返回空 (无界或空交), 按题意自行处理。
- 坐标跨度很大时注意 EPS_HP = $1e-10$ 的精度；平行线只需保留最靠内的一条。

```
const double EPS_HP = 1e-10;

struct HPoint {
    double x, y;
    HPoint operator+(const HPoint& o) const { return {x + o.x, y + o.y}; }
    HPoint operator-(const HPoint& o) const { return {x - o.x, y - o.y}; }
    HPoint operator*(double k) const { return {x * k, y * k}; }
};
```

```
double cross(HPoint a, HPoint b) { return a.x * b.y - a.y * b.x; }

struct HalfPlaneLine {
    HPoint p, v;
    double ang;
    bool operator<(const HalfPlaneLine& other) const {
        if (fabs(ang - other.ang) > EPS_HP) return ang < other.ang;
        return cross(v, other.p - p) < 0;
    }
};

HPoint line_intersection(const HalfPlaneLine& a, const HalfPlaneLine& b) {
    HPoint u = b.p - a.p;
    double t = cross(u, b.v) / cross(a.v, b.v);
    return a.p + a.v * t;
}

bool outside(const HalfPlaneLine& l, HPoint p) {
    return cross(l.v, p - l.p) < -EPS_HP;
}

vector<HPoint> half_plane_intersection(vector<HalfPlaneLine> lines) {
    for (auto& l : lines) l.ang = atan2(l.v.y, l.v.x);
    sort(lines.begin(), lines.end());
    deque<HalfPlaneLine> q;
    deque<HPoint> p;
    for (auto l : lines) {
        if (!q.empty() && fabs(cross(q.back().v, l.v)) < EPS_HP) {
            if (outside(l, q.back().p)) q.back() = l;
            continue;
        }
        while (!p.empty() && outside(l, p.back())) {
            p.pop_back();
            q.pop_back();
        }
        while (!p.empty() && outside(l, p.front())) {
            p.pop_front();
            q.pop_front();
        }
        if (!q.empty()) p.push_back(line_intersection(q.back(), l));
        q.push_back(l);
    }
    while (!p.empty() && outside(q.front(), p.back())) {
        p.pop_back();
        q.pop_back();
    }
    if (q.size() <= 2) return {};
    p.push_back(line_intersection(q.back(), q.front()));
    return vector<HPoint>(p.begin(), p.end());
}
```

曼哈顿距离变换

赛时先看

题目信号：距离为 $|x_1 - x_2| + |y_1 - y_2|$ ，需要全局最远或多次查询。

本质：求平面点集曼哈顿距离最大值。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接调用本节列出的函数即可。

复杂度判定：最大距离 $O(n)$ 。

维护的量：四种符号组合 (sx, sy) 下投影 $sx*x + sy*y$ 的最小值 mn 与最大值 mx ；答案取 $mx - mn$ 的最大者。

警告：二维曼哈顿最大距离等于四种符号组合投影最大差。

【最小完整示例（先抄这一段就能跑）】

题目：点集 $\{(0, 0), (5, 3), (1, 7)\}$ 中两点的最大曼哈顿距离。

```
vector<pair<i64, i64>> pts = {{0,0}, {5,3}, {1,7}};
printf("%lld\n", max_manhattan_distance(pts)); // 四种符号组合投影最大差
```

样例输出：8。

【传参要求（照这个传不会错）】

- `pts`: `vector<pair<i64, i64>>`，平面点集（至少 1 个点；空集返回 0）。
- 返回值：i64，最大曼哈顿距离；原理 $|x_1 - x_2| + |y_1 - y_2| = \max\{sx, sy \in \{-1, 1\}\} (sx*x_1 + sy*y_1) - (sx*x_2 + sy*y_2)$ 。
- 只能求最大：求“最小”曼哈顿距离不能这样算，需换平面最近点对等做法。

【改板时先认这几个量】

- `mn`: 某符号组合下投影的最小值。
- `mx`: 某符号组合下投影的最大值。

```
i64 max_manhattan_distance(const vector<pair<i64, i64>>& pts) {
    i64 ans = 0;
    for (int sx : {-1, 1}) {
        for (int sy : {-1, 1}) {
            i64 mn = (1LL << 62), mx = -(1LL << 62);
            for (auto [x, y] : pts) {
                i64 v = sx * x + sy * y;
                mn = min(mn, v);
                mx = max(mx, v);
            }
            ans = max(ans, mx - mn);
        }
    }
    return ans;
}
```

曼哈顿距离最小生成树

赛时先看

题目信号: 很多平面点、连通总代价最小、距离是曼哈顿距离; 完全图 $O(n^2)$ 建边太慢。

本质: 平面点集的边权为 $|x_1 - x_2| + |y_1 - y_2|$ 时, 利用四个方向扫描生成 $O(n)$ 条候选边, 再 Kruskal。

接法: 城市坐标给定, 修最短总长度的只能横竖走的道路网络。

复杂度判定: 候选边 $O(n)$, 总复杂度 $O(n \log n)$ 。

维护的量: 候选边集合 `edges{u,v,w}` (4 轮坐标变换生成 $O(n)$ 条); `DSUForMST` 并查集 (`p` 父节点、`sz` 集合大小) 按边权升序 Kruskal 累加 `ans`。

警告: 变换顺序为每轮取反 `x`, 两轮后交换 `x,y`; 候选边可能重复, Kruskal 自然会过滤。

【最小完整示例（先抄这一段就能跑）】

题目: 城市 (0, 0)、(10, 0)、(0, 10) 修只能横竖走的道路, 求连通所有城市的最短总长。

```
vector<pair<i64, i64>> cities = {{0,0}, {10,0}, {0,10}};
printf("%lld\n", manhattan_mst(cities)); // 两条边 10 + 10
```

样例输出: 20。

【传参要求（照这个传不会错）】

- `manhattan_mst(point)`: `point` 为 `vector<pair<i64,i64>>`, 城市坐标 ($n \geq 1$); 返回 `i64` 总代价; $n = 1$ 时返回 0。
- `DSUForMST(n)`: n 为点数; `find(x)` 查代表元 (路径压缩, 返回 `int`); `unite(a,b)` 按 `sz` 大小合并, 返回 `bool` (是否真的合并)。
- 内部自动做 4 轮坐标变换生成候选边, 重复边由 Kruskal 过滤, 调用时无需去重。

【API / 入口函数（赛时只认这里列的名字）】

- `manhattan_mst(vector<pair<i64,i64>> points)` -> 求曼哈顿距离下的最小生成树总代价, 返回 `i64`。
- `find(int x)` -> 查代表元/查找结果 返回 `int`。
- `unite(int a, int b)` -> 合并两个集合 返回 `bool`。

【改板时先认这几个量】

- `sz`: 集合/子树大小。
- `dsu`: 并查集实例 (`find/unite` 的调用对象)。
- `it`: `active` 中的迭代器, 用于枚举候选边。

典题模型: 城市坐标给定, 修最短总长度的只能横竖走的道路网络。

```
struct DSUForMST {
    vector<int> p, sz;
    explicit DSUForMST(int n) : p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a; sz[a] += sz[b];
        return true;
    }
};
```

```

i64 manhattan_mst(const vector<pair<i64, i64>>& point) {
    struct P { i64 x, y; int id; };
    struct E {
        int u, v;
        i64 w;
        bool operator<(const E& other) const { return w < other.w; }
    };
    int n = (int)point.size();
    vector<P> p(n);
    for (int i = 0; i < n; ++i) p[i] = {point[i].first, point[i].second, i};
    vector<E> edges;

    for (int swap_xy = 0; swap_xy < 2; ++swap_xy) {
        for (int flip_x = 0; flip_x < 2; ++flip_x) {
            sort(p.begin(), p.end(), [](const P& a, const P& b) {
                return a.x + a.y != b.x + b.y ? a.x + a.y < b.x + b.y : a.x < b.x;
            });
            map<i64, P> active;
            for (const P& now : p) {
                auto it = active.lower_bound(-now.y);
                while (it != active.end()) {
                    const P old = it->second;
                    if (now.x - now.y < old.x - old.y) break;
                    i64 w = llabs(point[now.id].first - point[old.id].first)
                        + llabs(point[now.id].second - point[old.id].second);
                    edges.push_back({now.id, old.id, w});
                    it = active.erase(it);
                }
                active[-now.y] = now;
            }
            for (P& q : p) q.x = -q.x;
        }
        for (P& q : p) swap(q.x, q.y);
    }

    sort(edges.begin(), edges.end());
    DSUForMST dsu(n);
    i64 ans = 0;
    for (const E& e : edges) if (dsu.unite(e.u, e.v)) ans += e.w;
    return ans;
}

```

最小圆覆盖：随机增量

赛时先看

题目信号：平面点集最小覆盖圆；点数较大，要求浮点答案。

本质：求覆盖所有点的最小圆。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里先构造对象，再调用其公开接口。

复杂度判定：期望 $O(n)$ 。

维护的量：`CircleH c{o, r}` 为当前覆盖前 i 个点的最小圆；加点点若在圆外，用两点定圆 `circle_from_two`、三点定圆 `circle_from_three` 逐层重建。

警告：随机打乱；判断点是否在圆内要加 EPS。

【最小完整示例（先抄这一段就能跑）】

题目：点 $(0, 0)$ 、 $(2, 0)$ 、 $(0, 2)$ 的最小覆盖圆。

```

vector<MecPoint> pts = {{0,0}, {2,0}, {0,2}};
CircleH c = minimum_enclosing_circle(pts); // 内部已随机打乱
printf("%.2f %.2f %.2f\n", c.o.x, c.o.y, c.r); // 圆心 (1,1), 半径  $\sqrt{2}$ 

```

样例输出：1.00 1.00 1.41。

【传参要求（照这个传不会错）】

- `minimum_enclosing_circle(p)`: p 为 `vector<MecPoint>` ($\{x, y\}$, double 坐标)；返回 `CircleH{o, r}` (o 圆心、 r 半径)，期望 $O(n)$ ，内部自动打乱。
- `circle_from_two(a, b)`：以 a 、 b 为直径端点作圆；`circle_from_three(a, b, c)`： a 、 b 、 c 外接圆（三点共线时 $\det=0$ 会除零，需保证输入不共线）。
- `in_circle(c, p)`：点 p 是否在圆 c 内（含 $1e-9$ 容差）。

```

struct MecPoint {
    double x, y;
    MecPoint() = default;
    MecPoint(double x_, double y_) : x(x_), y(y_) {}
    MecPoint operator+(const MecPoint& o) const { return {x + o.x, y + o.y}; }
    MecPoint operator-(const MecPoint& o) const { return {x - o.x, y - o.y}; }
    MecPoint operator*(double k) const { return {x * k, y * k}; }
}

```



```

MecPoint operator/(double k) const { return {x / k, y / k}; }
};

struct CircleH {
    MecPoint o;
    double r;
};

double dist(MecPoint a, MecPoint b) {
    return hypot(a.x - b.x, a.y - b.y);
}

CircleH circle_from_two(MecPoint a, MecPoint b) {
    return {(a.x + b.x) / 2, (a.y + b.y) / 2, dist(a, b) / 2};
}

CircleH circle_from_three(MecPoint a, MecPoint b, MecPoint c) {
    double a1 = b.x - a.x, b1 = b.y - a.y;
    double c1 = c.x - a.x, d1 = c.y - a.y;
    double e1 = (a1 * (a.x + b.x) + b1 * (a.y + b.y)) / 2;
    double f1 = (c1 * (a.x + c.x) + d1 * (a.y + c.y)) / 2;
    double det = a1 * d1 - b1 * c1;
    MecPoint o((d1 * e1 - b1 * f1) / det, (-c1 * e1 + a1 * f1) / det);
    return {o, dist(o, a)};
}

bool in_circle(CircleH c, MecPoint p) {
    return dist(c.o, p) <= c.r + 1e-9;
}

CircleH minimum_enclosing_circle(vector<MecPoint> p) {
    shuffle(p.begin(), p.end(), mt19937(chrono::steady_clock::now().time_since_epoch().count()));
    CircleH c{{0, 0}, -1};
    for (int i = 0; i < (int)p.size(); i++) {
        if (c.r >= 0 && in_circle(c, p[i])) continue;
        c = {p[i], 0};
        for (int j = 0; j < i; j++) {
            if (in_circle(c, p[j])) continue;
            c = circle_from_two(p[i], p[j]);
            for (int k = 0; k < j; k++) {
                if (!in_circle(c, p[k])) c = circle_from_three(p[i], p[j], p[k]);
            }
        }
    }
    return c;
}

```

扫描线 + 线段树：矩形面积并

赛时先看

题目信号：多个矩形，问覆盖面积，坐标大，矩形数量可到 $1e5$ 。

本质：求多个轴平行矩形覆盖的总面积。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定： $O(n \log n)$ 。

维护的量：事件表 `events{x, y1, y2, delta}`（按 x 升序）；线段树 `cover[p]`（区间被覆盖层数）与 `len[1]`（ y 方向被覆盖总长）；答案按 `len[1] * Δx` 逐段累加。

警告：对 y 坐标离散化后，线段树维护的是区间 `[ys[1], ys[r+1])`；事件按 x 排序。

【最小完整示例（先抄这一段就能跑）】

题目：矩形 $(0,0)-(2,2)$ 与 $(1,1)-(3,3)$ 的覆盖面积（重合部分只算一次）。

```

vector<array<double, 4>> rects = {{0,0,2,2}, {1,1,3,3}};
printf("%.1f\n", rectangle_union_area(rects)); // 4 + 4 - 1 = 7

```

样例输出：7.0。

【传参要求（照这个传不会错）】

- `rectangle_union_area(rects)`：唯一外部入口；`rects` 为 `vector<array<double,4>>`，每个矩形 $\{x1, y1, x2, y2\}$ （角点顺序任意，内部自动交换； $x1==x2$ 或 $y1==y2$ 的退化矩形自动忽略）；返回 `double` 覆盖面积。
- 内部 `SweepEvent{x, y1, y2, delta}`：`delta = +1` 左边界入、`-1` 右边界出；事件按 x 升序扫描。
- 不要直接调 `AreaUnionSeg::add` 的递归参数，只调 `rectangle_union_area`。

【API / 入口函数（赛时只认这里列的名字）】

- `rectangle_union_area(rects)` -> 唯一外部入口; `rects` 每个元素为 `{x1,y1,x2,y2}`, 返回所有轴平行矩形覆盖面积。

```
struct SweepEvent {
    double x, y1, y2;
    int delta;
    bool operator<(const SweepEvent& other) const { return x < other.x; }
};

struct AreaUnionSeg {
    vector<int> cover;
    vector<double> len, ys;
    int n; // 基本小区间数量为 ys.size()-1。

    AreaUnionSeg(const vector<double>& ys_) {
        ys = ys_;
        n = (int)ys.size() - 1;
        cover.assign(4 * n + 4, 0);
        len.assign(4 * n + 4, 0);
    }

    void pull(int p, int l, int r) {
        if (cover[p] > 0) len[p] = ys[r + 1] - ys[l];
        else if (l == r) len[p] = 0;
        else len[p] = len[p << 1] + len[p << 1 | 1];
    }

    void add(int p, int l, int r, int ql, int qr, int delta) {
        if (ql <= l && r <= qr) {
            cover[p] += delta;
            pull(p, l, r);
            return;
        }
        int mid = (l + r) >> 1;
        if (ql <= mid) add(p << 1, l, mid, ql, qr, delta);
        if (qr > mid) add(p << 1 | 1, mid + 1, r, ql, qr, delta);
        pull(p, l, r);
    }
};

double rectangle_union_area(vector<array<double, 4>> rects) {
    vector<SweepEvent> events;
    vector<double> ys;
    for (auto [x1, y1, x2, y2] : rects) {
        if (x1 == x2 || y1 == y2) continue;
        if (x1 > x2) swap(x1, x2);
        if (y1 > y2) swap(y1, y2);
        events.push_back({x1, y1, y2, 1});
        events.push_back({x2, y1, y2, -1});
        ys.push_back(y1);
        ys.push_back(y2);
    }
    if (events.empty()) return 0;
    sort(events.begin(), events.end());
    sort(ys.begin(), ys.end());
    ys.erase(unique(ys.begin(), ys.end()), ys.end());

    AreaUnionSeg seg(ys);
    double ans = 0;
    for (int i = 0; i < (int)events.size(); ++i) {
        if (i > 0) ans += seg.len[1] * (events[i].x - events[i - 1].x);
        int l = int(lower_bound(ys.begin(), ys.end(), events[i].y1) - ys.begin());
        int r = int(lower_bound(ys.begin(), ys.end(), events[i].y2) - ys.begin()) - 1;
        if (l <= r) seg.add(1, 0, seg.n - 1, l, r, events[i].delta);
    }
    return ans;
}
```

F 动态规划与状态优化

13 动态规划基础与状态设计

背包、序列、区间、状压、轮廓线、数位、概率期望和小状态 MDP 放在这里，优先解决“状态怎么设”。

0/1 背包

赛时先看

题目信号：题面说“每个物品选/不选一次”“每种最多用一次”，容量上限 W 到几十万，问容量限制下的最大价值。看到“每个物品只有一次机会”就是 0/1 背包。

本质：容量维倒序滚动数组；外层遍历物品，内层容量从大到小推，保证 $dp[j-w]$ 还是上一轮没选过当前物品的旧值，实现“每件最多选一次”。

复杂度判定： $O(nW)$ 时间、 $O(W)$ 空间； $W \leq 1e5 \sim 1e6$ 常用， W 上千万或更大就要换做法（价值维、倍增等）。

维护的量： $dp[j]$ （容量不超过 j 时的最大价值）。

接法：把每个物品的花费放进 $weight[i]$ ，收益放进 $value[i]$ ，容量是 W ；调用后 $dp[j]$ 表示容量不超过 j 的最大收益，通常答案是 $dp[W]$ 。

警告：0/1 背包容量必须倒序循环；改成正序就变成完全背包。

【最小完整示例（先抄这一段就能跑）】

题目： n 个物品（第 i 个花费 $w[i]$ 、价值 $v[i]$ ，每个最多选一次），容量 W ，求最大总价值。

```
vector<int> weight(n + 1);
vector<i64> value(n + 1);
for (int i = 1; i <= n; ++i) cin >> weight[i] >> value[i];
auto dp = zero_one_knapsack(W, weight, value); // 1. 调用：物品数组 1-indexed
cout << dp[W] << '\n'; // 2. 取结果：容量 W 的最大价值
```

样例：物品 (2,3) (3,4) (4,5)， $W = 5 \rightarrow$ 输出 7（选前两个）。

【传参要求（照这个传不会错）】

- `zero_one_knapsack(W, weight, value)`： W = 容量上限；`weight/value` 必须 1-indexed（长度 $n+1$ ，`[0]` 不用）。
- 返回值：`vector<i64> dp`； $dp[j]$ = 容量不超过 j 的最大价值，答案取 $dp[W]$ 。
- 每个物品只能选一次；可重复选换 `complete_knapsack`；每件有限个换 `multiple_knapsack`。
- 物品数量多但 W 巨大时 $O(nW)$ 会超时，考虑价值维或二进制优化。

【不会用就照抄】

```
// dp[j]：容量不超过 j 的最大价值；外层物品、内层容量倒序
vector<i64> dp(W + 1, 0);
for (auto [w, v] : items)
    for (int j = W; j >= w; --j)
        dp[j] = max(dp[j], dp[j - w] + v);
```

【抄板清单（照着做就行）】

1. 抄哪段：整个 `zero_one_knapsack` 函数。
2. 构造：`weight[i]`、`value[i]` 按题面填好， W 是容量上限。
3. 调用：`auto dp = zero_one_knapsack(W, weight, value);`
4. 取结果：答案通常就是 $dp[W]$ （容量不超过 W 的最大价值）。

【改造点（按题目改这几处）】

- 价值可能为负：初值全 0 会把负价值吞掉，把 `dp` 初值改成 `-INF`（如 `LLONG_MIN / 4`），只有 `dp[0] = 0`。
- 要求恰好装满：同样把初值改成 `-INF`、`dp[0] = 0`，答案取 $dp[W]$ ，不可达状态保持 `-INF`。
- 需要输出方案：加一个 `pre[j]` 数组记录 `dp[j]` 最后被哪个物品更新，算完后倒序还原选中的物品。
- W 很大（ $1e6+$ ）：空间 $O(W)$ 吃紧时先看内存限制； W 上亿考虑价值维背包或其他做法。

【API / 入口函数（赛时只认这里列的名字）】

- `zero_one_knapsack(int W, const vector<int>& weight, const vector<i64>& value)` $\rightarrow$ 0/1 背包 返回 `vector<i64>`。

【核心逻辑（改代码时别破坏）】

- 每件物品只能用一次，所以容量必须倒序，保证本轮不会再次使用当前物品。

```
// 维护的量：dp[j] = 容量不超过 j 时的最大价值。
// 不变量：处理完前 i 个物品后，dp[j] 只由前 i 个物品构成，且每件最多用一次。
vector<i64> zero_one_knapsack(int W, const vector<int>& weight, const vector<i64>& value) {
    vector<i64> dp(W + 1, 0);
    for (int i = 0; i < (int)weight.size(); ++i) {
        // 容量倒序：dp[j - weight[i]] 还是上一轮的旧值，当前物品不会被重复选
        for (int j = W; j >= weight[i]; --j) {
            dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
        }
    }
    return dp;
}
```

完全背包

赛时先看

题目信号：硬币不限量、材料可重复使用。

本质：每个物品可选无限次。

接法：硬币、材料、技能可重复使用时用完全背包。正序循环代表当前物品可以被重复使用；如果你把它写成倒序，就会错误地变成每种只能选一次。

复杂度判定： $O(nW)$ 。

维护的量： $dp[j]$ （容量不超过 j 时的最大价值）。

警告：完全背包容量正序循环。

【最小完整示例（先抄这一段就能跑）】

题目： n 种物品（第 i 种花费 $w[i]$ 、价值 $v[i]$ ，每种不限量），容量 W ，求最大总价值。

```
vector<int> weight(n + 1);
vector<i64> value(n + 1);
for (int i = 1; i <= n; ++i) cin >> weight[i] >> value[i];
auto dp = complete_knapsack(W, weight, value); // 1. 调用：物品数组 1-indexed
cout << dp[W] << '\n'; // 2. 取结果：容量 W 的最大价值
```

样例：物品 (2,3) (3,4) (4,5)， $W = 6 \rightarrow$ 输出 9（三个 (2,3)）。

【传参要求（照这个传不会错）】

- `complete_knapsack(W, weight, value)`： W = 容量上限；`weight/value` 必须 1-indexed（长度 $n+1$ ，`[0]` 不用）。
- 返回值：`vector<i64> dp`； $dp[j]$ = 容量不超过 j 的最大价值，答案取 $dp[W]$ 。
- 每种物品可取任意次；只能取一次换 `zero_one_knapsack`；每种有限个换 `multiple_knapsack`。

【不会用就照抄】

```
for (auto [w, v] : items)
    for (int j = w; j <= W; ++j)
        dp[j] = max(dp[j], dp[j - w] + v);
```

- 完全背包容量正序；和 0/1 背包最核心的区别就是循环方向。

【API / 入口函数（赛时只认这里列的名字）】

- `complete_knapsack(int W, const vector<int>& weight, const vector<i64>& value)` $\rightarrow$ 完全背包 返回 `vector<i64>`。

【核心逻辑（改代码时别破坏）】

- 每件物品可无限次，所以容量正序，让本轮新状态继续转移到更大容量。

```
vector<i64> complete_knapsack(int W, const vector<int>& weight, const vector<i64>& value) {
    vector<i64> dp(W + 1, 0);
    for (int i = 0; i < (int)weight.size(); ++i) {
        for (int j = weight[i]; j <= W; ++j) {
            dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
        }
    }
    return dp;
}
```

多重背包：二进制拆分

赛时先看

题目信号：物品数量有限但可能很大。

本质：每种物品最多选 $cnt[i]$ 次。

接法：每种物品给了数量 $cnt[i]$ 时用这个模板。它把 cnt 拆成 $1, 2, 4, \dots$ 若干包，再跑 0/1 背包；题目容量很大、数量也很大时，这个版本可能不够快，翻“多重背包单调队列优化”。

复杂度判定： $O(W * \sum \log cnt[i])$ 。

维护的量： $dp[j]$ （容量不超过 j 时的最大价值）；辅助 nw/nv （把 $cnt[i]$ 按 $1, 2, 4, \dots$ 拆出的新物品包）。

警告：拆分后按 0/1 背包倒序做。

【最小完整示例（先抄这一段就能跑）】

题目： n 种物品（第 i 种重量 $w[i]$ 、价值 $v[i]$ 、最多 $cnt[i]$ 个），容量 W ，求最大总价值。

```
vector<int> weight(n + 1), cnt(n + 1);
vector<i64> value(n + 1);
for (int i = 1; i <= n; ++i) cin >> weight[i] >> value[i] >> cnt[i];
auto dp = multiple_knapsack(W, weight, value, cnt); // 1. 调用: 三个数组 1-indexed
cout << dp[W] << '\n'; // 2. 取结果: 容量 W 的最大价值
```

样例: 物品 (1,2,2) (2,3,2) (重量, 价值, 个数), $W = 5 \rightarrow$ 输出 8 ((1,2) 一个 + (2,3) 两个)。

【传参要求 (照这个传不会错)】

- `multiple_knapsack(W, weight, value, cnt)`: W = 容量上限; 三个数组等长、1-indexed (长度 $n+1$, $[0]$ 不用)。
- 返回值: `vector<i64> dp`; $dp[j]$ = 容量不超过 j 的最大价值, 答案取 $dp[W]$ 。
- $cnt[i]$ = 第 i 种物品最多能拿的个数; 内部拆成 1,2,4,... 再跑 0/1 背包。
- W 和 $cnt[i]$ 都很大、 $O(W \log cnt)$ 过不了时, 换 `multiple_knapsack_monotone_queue`。

```
vector<i64> multiple_knapsack(int W, vector<int> weight, vector<i64> value, vector<int> cnt) {
    vector<int> nw;
    vector<i64> nv;
    for (int i = 0; i < (int)weight.size(); ++i) {
        int c = cnt[i];
        for (i64 k = 1; c > 0; k <= 1) {
            int take = (int)min(k, (i64)c);
            nw.push_back((int)((i64)weight[i] * take));
            nv.push_back(value[i] * (i64)take);
            c -= take;
        }
    }
    return zero_one_knapsack(W, nw, nv);
}
```

多重背包单调队列优化: 数量很大、容量也大

赛时先看

题目信号: 多重背包, W 很大、物品数量上限也大, 且题目需要 $O(nW)$ 级别而非 $O(nW \log count)$; 重量为正整数。

本质: 每种物品最多取 $count[i]$ 个, 重量/价值均为整数; 二进制拆分的 $\log count$

仍然不够快时, 把同余类转化为滑动窗口最大值。

复杂度判定: $O(nW)$ 时间、 $O(W)$ 空间。

维护的量: $dp[j]$ (容量不超过 j 时的最大价值); 同余类滑窗 `candidates` (存 $(t, previous[r+t*w] - t*v)$, 值单调递减)。

警告: 同一类物品更新时必须从旧数组 `previous` 转移, 不能原地污染; 对固定余数 r , 位置 $r + t*w$ 的候选窗口只允许前 $count$ 步。若题目是“恰好容量”, 初始值和不可达状态应改为 $-INF$, 不能直接用全零。

【最小完整示例 (先抄这一段就能跑)】

题目: n 种物品 (第 i 种重量 $w[i]$ 、价值 $v[i]$ 、最多 $count[i]$ 个), 容量 W , 求最大总价值。

```
vector<int> weight(n + 1), count(n + 1);
vector<i64> value(n + 1);
for (int i = 1; i <= n; ++i) cin >> weight[i] >> value[i] >> count[i];
auto dp = multiple_knapsack_monotone_queue(W, weight, value, count); // 1. 调用: 数组 1-indexed
cout << dp[W] << '\n'; // 2. 取结果: 容量 W 的最大价值
```

样例: 物品 (2,3,3) (3,4,2) (重量, 价值, 个数), $W = 6 \rightarrow$ 输出 9 (三个 (2,3))。

【传参要求 (照这个传不会错)】

- `multiple_knapsack_monotone_queue(W, weight, value, count)`: W = 容量上限; 三个数组等长、1-indexed (长度 $n+1$, $[0]$ 不用)。
- 返回值: `vector<i64> dp`; $dp[j]$ = 容量不超过 j 的最大价值, 答案取 $dp[W]$ 。
- 要求重量是正整数 ($w \geq 1$); 要“恰好装满”时把 dp 初值改成 $-INF$ 、只留 $dp[0] = 0$ 。

```
vector<i64> multiple_knapsack_monotone_queue(
    int W, const vector<int>& weight, const vector<i64>& value, const vector<int>& count
) {
    vector<i64> dp(W + 1, 0); // 至多容量 W, 允许不装满。
    for (int item = 0; item < (int)weight.size(); ++item) {
        int w = weight[item], c = count[item];
        i64 v = value[item];
        vector<i64> previous = dp;
        for (int remainder = 0; remainder < w && remainder <= W; ++remainder) {
            deque<pair<int, i64>> candidates; // (t, previous[r+t*w] - t*v), 值单调递减。
            for (int t = 0, position = remainder; position <= W; ++t, position += w) {
                i64 key = previous[position] - (i64)t * v;
                while (!candidates.empty() && candidates.back().second <= key) {
                    candidates.pop_back();
                }
            }
        }
    }
}
```

```

    }
    candidates.push_back({t, key});
    while (candidates.front().first < t - c) candidates.pop_front();
    dp[position] = candidates.front().second + (i64)t * v;
}
}
return dp;
}

```

分组背包

赛时先看

题目信号：题面说“每组选一个/最多一个”，如套餐、课程类别。

本质：物品分成若干组，每组最多选一个。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O(\text{总物品数} * W)$ 。

维护的量：`dp[j]`（容量不超过 `j` 时的最大价值）；`old`（上一组处理完的 `dp` 快照，保证每组最多选一个）。

警告：每组内要从上一组状态转移，容量倒序或复制旧数组。

【最小完整示例（先抄这一段就能跑）】

题目：`m` 组物品，第 `i` 组是若干 `(w, val)` 对，每组最多选一个，容量 `W`，求最大总价值。

```

vector<vector<pair<int, i64>>> groups(m);
for (int i = 0; i < m; ++i) {
    int k; cin >> k;
    groups[i].resize(k);
    for (auto& [w, val] : groups[i]) cin >> w >> val;
}
auto dp = group_knapsack(W, groups); // 1. 调用: groups 0-indexed
cout << dp[W] << '\n';           // 2. 取结果: 容量 W 的最大价值

```

样例：组 1 `{(2,3),(3,4)}`、组 2 `{(2,5)}`，`W = 4` → 输出 8（组 1 选 `(2,3)`，组 2 选 `(2,5)`）。

【传参要求（照这个传不会错）】

- `group_knapsack(W, groups)`：`W` = 容量上限；`groups[i]` 是第 `i` 组的所有物品 `{w, val}` 对，0-indexed。
- 返回值：`vector<i64> dp`；`dp[j]` = 容量不超过 `j` 的最大价值，答案取 `dp[W]`。
- 每组最多选一个；若要求每组“至少选一个”，把转移里的 `dp[j]` 换成只从 `old[j-w]+val` 更新（去掉跳过整组的选项）。

```

vector<i64> group_knapsack(int W, const vector<vector<pair<int, i64>>>& groups) {
    vector<i64> dp(W + 1, 0);
    for (auto group : groups) {
        vector<i64> old = dp;
        for (auto [w, val] : group) {
            for (int j = w; j <= W; ++j) {
                dp[j] = max(dp[j], old[j - w] + val);
            }
        }
    }
    return dp;
}

```

- 核心逻辑：实现用 `old` 数组保存上一组结果，正序更新 `dp`；一组里最多选一个。

二维费用背包

赛时先看

题目信号：每个物品消耗两类资源。

本质：有两个容量限制，如重量和体积。

复杂度判定： $O(nAB)$ 。

维护的量：`dp[a][b]`（资源一不超过 `a`、资源二不超过 `b` 时的最大价值）。

警告：0/1 情况两个维度都倒序。

【最小完整示例（先抄这一段就能跑）】

题目：`n` 个物品（第 `i` 个耗资源一 `ca[i]`、资源二 `cb[i]`、价值 `val[i]`，各至多一次），资源一上限 `A`、资源二上限 `B`，求最大总价值。

```

vector<int> ca(n + 1), cb(n + 1);

```

```
vector<i64> val(n + 1);
for (int i = 1; i <= n; ++i) cin >> ca[i] >> cb[i] >> val[i];
auto dp = two_cost_knapsack(A, B, ca, cb, val); // 1. 调用：三个数组 1-indexed
cout << dp[A][B] << '\n'; // 2. 取结果：两类资源都不超 A、B
```

样例：物品 (1,1,3) (1,1,4) (1,2,5)（费用一，费用二，价值），A = 2, B = 2 -> 输出 7（选前两个）。

【传参要求（照这个传不会错）】

- `two_cost_knapsack(A, B, ca, cb, val)`: A/B = 两类资源的容量上限；三个物品数组等长、1-indexed（长度 `n+1`，`[0]` 不用）。
- 返回值：vector<vector<i64>> dp; `dp[a][b]` = 资源一 $\leq a$ 、资源二 $\leq b$ 的最大价值，答案取 `dp[A][B]`。
- 每件最多选一次（两维都倒序）；可重复选就把两维都改正序。
- 注意空间：(A+1)*(B+1) 个 i64, A*B 很大时先看内存限制。

```
vector<vector<i64>> two_cost_knapsack(int A, int B,
                                     const vector<int>& ca,
                                     const vector<int>& cb,
                                     const vector<i64>& val) {
    vector<vector<i64>> dp(A + 1, vector<i64>(B + 1, 0));
    for (int i = 0; i < (int)val.size(); ++i) {
        for (int a = A; a >= ca[i]; --a) {
            for (int b = B; b >= cb[i]; --b) {
                dp[a][b] = max(dp[a][b], dp[a - ca[i]][b - cb[i]] + val[i]);
            }
        }
    }
    return dp;
}
```

背包方案数

赛时先看

题目信号：问有多少种选法，常见硬币、子集和。

本质：统计凑出容量/金额的方案数。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()`

里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O(nW)$ 。

维护的量：`dp[j]`（凑出恰好 j 的方案数，`dp[0] = 1`）。

警告：0/1 背包倒序，完全背包正序；按题意处理取模。

【最小完整示例（先抄这一段就能跑）】

题目： n 种硬币（第 i 种面值 `w[i]`，可重复用），凑出金额 W 有多少种方案，模 `mod`。

```
vector<int> coin(n + 1);
for (int i = 1; i <= n; ++i) cin >> coin[i];
auto dp = complete_count(W, coin, mod); // 1. 调用：硬币不限量用 complete_count（正序）
cout << dp[W] << '\n'; // 2. 取结果：凑出恰好 W 的方案数模 mod
```

样例：硬币 {1,2,5}, $W = 5$ -> 输出 4 (5、2+2+1、2+1+1+1、1+1+1+1+1)。

【传参要求（照这个传不会错）】

- `complete_count(W, coin, mod)`: W = 目标金额；coin 1-indexed（长度 `n+1`，`[0]` 不用）；`mod` = 取模数。
- 返回值：vector<i64> dp; `dp[j]` = 凑出恰好 j 的方案数模 `mod`，答案取 `dp[W]`。
- 硬币不限量：用 `complete_count`；每枚只能用一次（子集和）：用 `zero_one_count(W, coin, mod)`。
- 循环顺序“外层硬币、内层金额”数的是组合（顺序不区分）；交换两层就变成排列计数，结果不同。

```
vector<i64> zero_one_count(int W, const vector<int>& weight, i64 mod) {
    vector<i64> dp(W + 1, 0);
    dp[0] = 1;
    for (int w : weight) {
        for (int j = W; j >= w; --j) {
            dp[j] = (dp[j] + dp[j - w]) % mod;
        }
    }
    return dp;
}

vector<i64> complete_count(int W, const vector<int>& coin, i64 mod) {
    vector<i64> dp(W + 1, 0);
    dp[0] = 1;
    for (int w : coin) {

```



```

    for (int j = w; j <= W; ++j) {
        dp[j] = (dp[j] + dp[j - w]) % mod;
    }
}
return dp;
}

```

整数分拆：把 n 写成若干正整数之和

赛时先看

题目信号：硬币面值恰为 $1..n$ 、每种可无限取、组合顺序不区分；这正是完全背包的“先枚举面值，再枚举和”。

本质：计算无序正整数分拆数，例如 $4 = 4 = 3+1 = 2+2 = 2+1+1 = 1+1+1+1$ ，共 5 种。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()`

里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O(n^2)$ 时间、 $O(n)$ 空间。

维护的量：`dp[x]` (x 的无序正整数分拆数，`dp[0] = 1`)。

警告：循环顺序决定是否把排列重复计数。必须外层枚举 `part`，内层正序枚举 `sum`；`dp[0]=1` 表示“凑成 0 的空方案”。

【最小完整示例（先抄这一段就能跑）】

题目：把 n 写成若干个正整数的和（顺序不区分），求分拆数模 `mod`。

```

int n, mod;
cin >> n >> mod;
auto dp = integer_partition_counts(n, mod); // 1. 调用：mod 可省略（默认全局 modn）
cout << dp[n] << '\n'; // 2. 取结果：n 的分拆数模 mod

```

样例： $n = 4 \rightarrow$ 输出 5 (4、3+1、2+2、2+1+1、1+1+1+1)。

【传参要求（照这个传不会错）】

- `integer_partition_counts(n, mod = modn)`： n = 要拆的数；`mod` = 取模数，缺省用全局 `modn`。
- 返回值：`vector<int> dp` (长度 $n+1$)；`dp[x]` = 整数 x 的无序分拆数（模 `mod`），答案取 `dp[n]`。
- 等价于“面值 $1..n$ 各无限个的完全背包”；外层 `part`、内层正序 `sum` 的顺序不能换，换了会把排列也算进去。
- n 很大时 $O(n^2)$ 会超时；带额外限制（每部分不超过 k 、必须恰好 m 个等）需自己改造。
- `sum`：当前正在凑的总和。
- `dp`：DP 状态。

```

vector<int> integer_partition_counts(int n, int mod = modn) {
    vector<int> dp(n + 1);
    dp[0] = 1 % mod;
    for (int part = 1; part <= n; ++part) {
        for (int sum = part; sum <= n; ++sum) {
            dp[sum] += dp[sum - part];
            if (dp[sum] >= mod) dp[sum] -= mod;
        }
    }
    return dp; // dp[x] 是 x 的无序正整数分拆数。
}

```

有上界的方案计数：分糖果 / 分球到各人

赛时先看

题目信号：每个人分到的数量有上限，物品/糖果必须刚好分完； n 约百、`total` 可到 $1e5$ ，朴素枚举每人取多少会超时。

本质：有 n 个位置，第 i 个取 $0..limit[i]$ 个，恰好凑总和 `total`，求方案数模 `mod`。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()`

里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O(n * total)$ 时间、 $O(total)$ 空间。

维护的量：`dp[s]` (前若干个人分完、恰好凑出 s 的方案数)；`window_sum` (滑动窗口内 `dp` 的和，即当前人可取范围的转移和)。

警告：转移是 `next[s] = sum(dp[s-x])`，

$0 \leq x \leq limit$)，用滑动窗口维护而非原地更新；窗口左端移出时要补回模数。经典练习：AtCoder Educational DP M - Candies。

【最小完整示例（先抄这一段就能跑）】

题目： n 个人，第 i 人最多拿 `limit[i]` 个糖果，恰好分完 `total` 个，方案数模 `mod`。

```
vector<int> limit(n);
for (int i = 0; i < n; ++i) cin >> limit[i];
int ans = count_bounded_sum_ways(limit, total, mod); // 1. 调用: limit 0-indexed
cout << ans << '\n'; // 2. 取结果: 方案数模 mod
```

样例: `limit = {2, 1, 2}`, `total = 3` -> 输出 `5`。

【传参要求（照这个传不会错）】

- `count_bounded_sum_ways(limit, total, mod = mod7)`: `limit` 0-indexed, `limit[i]` = 第 `i` 个人最多能拿的个数; `total` = 必须恰好凑出的总和; `mod` = 取模数, 缺省用全局 `mod7`。
- 返回值: `int`, 恰好凑出 `total` 的方案数模 `mod`。
- 每人“恰好分完”隐含每人都要参与: 本模板允许每人拿 `0..limit[i]` 个, 也就是允许有人拿 `0` 个; 要求每人至少 `1` 个时先给 `total` 减掉 `n`。
- `sum`: 当前枚举的总和。
- `dp`: DP 状态。

```
int count_bounded_sum_ways(const vector<int>& limit, int total, int mod = mod7) {
    vector<int> dp(total + 1);
    dp[0] = 1 % mod;
    for (int upper : limit) {
        vector<int> next(total + 1);
        int window_sum = 0;
        for (int sum = 0; sum <= total; ++sum) {
            window_sum += dp[sum];
            if (window_sum >= mod) window_sum -= mod;
            if (sum - upper - 1 >= 0) {
                window_sum -= dp[sum - upper - 1];
                if (window_sum < 0) window_sum += mod;
            }
            next[sum] = window_sum;
        }
        dp.swap(next);
    }
    return dp[total];
}
```

依赖背包：树上选课骨架

赛时先看

题目信号: 课程先修、附件依赖、树形依赖选择。

本质: 选某个物品前必须选它的父物品。

接法: 把本节 `struct/class/函数组` 整段抄到 `solve()` 上面; `solve()`

里先构造对象, 再按下方“不会用就看这里”调用公开接口。

复杂度判定: 常见 $O(nw^2)$ 。

维护的量: `dp[u]` (以 `u` 为根的子树、容量不超过 `j`

的最大价值); `sz[u]` (子树已占容量); `child/cost/value` (依赖树与物品数据)。

警告: 合并子树时容量倒序, 避免重复使用同一子树。构造后需手动填写 `cost/value/child` 再对根调用

`dfs`; 森林要对每个根分别调用。

【最小完整示例（先抄这一段就能跑）】

题目: `n` 门课, 第 `i` 门要先修 `fa[i]` (`0` 表示无先修), 花费 `cost[i]`、价值 `value[i]`, 容量 `W`, 求最大总价值。

```
DependentKnapsack dk(n, W); // 1. 建对象: n 个节点、容量 W
vector<int> fa(n + 1);
for (int i = 1; i <= n; ++i) {
    cin >> fa[i] >> dk.cost[i] >> dk.value[i]; // 2. 手动填 cost/value (1-indexed)
    if (fa[i]) dk.child[fa[i]].push_back(i); // 3. 填依赖: i 挂到 fa[i] 下
}
int ans = 0;
for (int r = 1; r <= n; ++r) // 4. 森林: 每个根 (fa[r]==0) 各 dfs 一次
    if (fa[r] == 0) {
        dk.dfs(r);
        for (int j = 0; j <= W; ++j) ans = max(ans, dk.dp[r][j]); // 5. 取各根 dp 的最大值
    }
cout << ans << '\n';
```

样例: `(fa,cost,value)=(0,2,5) (1,2,6) (1,3,7)`, `W = 4` -> 输出 `11` (选 `1` 和 `2`)。

【传参要求（照这个传不会错）】

- 构造 `DependentKnapsack dk(n, W)`: `n` = 节点数 (1-indexed), `W` = 容量上限; 成员全部公开, 直接填。

- 填 `dk.cost[i]`、`dk.value[i]`：依赖关系用 `dk.child[fa].push_back(i)`；无先修的节点是根。
- 调用 `dk.dfs(根)`：每个根都要调用一次，子节点在递归里自动处理；答案对所有根取 `dp[根][j]` 的最大值。
- `cost[i] > W` 的节点实际选不了，但 `dp[u][j]` 会保持 `-INF`，不影响其他分支。
- `sz`：集合/子树大小。
- `dp`：DP 状态。

```
struct DependentKnapsack {
    int n, W;
    vector<vector<int>> child;
    vector<int> cost, value, sz;
    vector<vector<i64>> dp;

    DependentKnapsack(int n, int W) : n(n), W(W), child(n + 1), cost(n + 1), value(n + 1), sz(n + 1), dp(n + 1) {}

    void dfs(int u) {
        dp[u].assign(W + 1, -(1LL << 60));
        for (int j = cost[u]; j <= W; ++j) dp[u][j] = value[u];
        sz[u] = cost[u];
        for (int v : child[u]) {
            dfs(v);
            vector<i64> ndp = dp[u];
            for (int j = W; j >= cost[u]; --j) {
                for (int k = 0; k + j <= W; ++k) {
                    ndp[j + k] = max(ndp[j + k], dp[u][j] + dp[v][k]);
                }
            }
            dp[u].swap(ndp);
        }
    }
};
```

LIS 最长严格上升子序列

赛时先看

题目信号：题面问“最长严格递增子序列长度”“按原顺序挑数、要求递增、最多挑几个”， $n \leq 1e5+$ ； n 一大就不能 $O(n^2)$ 暴力，直接想 `lis_strict`。

本质：`d` 维护“长度为 `len` 的上升子序列的最小末尾值”（`d[len-1]`）；新数 `x` 二分找它该替换哪个长度，要么把最小末尾变小给后面留余地，要么把最长链延长一格。

复杂度判定： $O(n \log n)$ 时间、 $O(n)$ 空间； n 到 $1e6$ 也能过。

维护的量：`d`（`d[len-1]` = 长度为 `len` 的严格上升子序列的最小末尾）。

接法：题面说“按原顺序选一些数，要求严格递增，问最多选几个”就调用

`lis_strict(a)`；它只返回长度，不返回选了哪些位置。

警告：严格上升用 `lower_bound`；非降用 `upper_bound`；`d` 是辅助结构，本身不是答案子序列。

【最小完整示例（先抄这一段就能跑）】

题目： n 个数，求最长严格上升子序列的长度（ $n \leq 1e5$ ）。

```
vector<int> a(n + 1);
for (int i = 1; i <= n; ++i) cin >> a[i];
int len = lis_strict(a); // 1. 调用：a 需 1-indexed (a[0] 不用)
cout << len << '\n'; // 2. 取结果：最长严格上升子序列长度
```

样例：`a = [3,1,4,1,5,9,2,6]` → 输出 4（如 1, 4, 5, 9 或 3, 4, 5, 9）。

【传参要求（照这个传不会错）】

- `lis_strict(a)`：`a` 1-indexed（长度 `n+1`）；返回 `int` 最长严格上升长度。
- 只返回长度，不返回选了哪些数；要方案翻“LIS：恢复”节，要计数翻“LIS 计数”节。
- 非降（允许相等）：把实现里的 `lower_bound` 改成 `upper_bound`。

【抄板清单（照着做就行）】

1. 抄哪段：整个 `lis_strict` 函数。
2. 构造：把原数组直接放进 `a`。
3. 调用：`int len = lis_strict(a);`
4. 取结果：`len` 就是最长严格上升子序列长度，直接输出。

【改造点（按题目改这几处）】

- 严格 / 非降：严格递增用 `lower_bound`；允许相等（非降）把 `lower_bound` 改成 `upper_bound`。

- 要输出方案：翻本节“LIS：恢复一组最长严格上升子序列”（`longest_increasing_subsequence`）。
- 要数方案数：翻本节“LIS 计数”（树状数组版，按原下标区分方案）。
- 元素类型不是 `int`：把 `vector<int>` 换成 `vector<i64>` 等对应类型即可。

```
// 维护的量：d[len-1] = 长度为 len 的严格上升子序列的最小末尾值。
// 不变量：d 严格递增，这是二分（lower_bound）能用的前提。
int lis_strict(const vector<int>& a) {
    vector<int> d;
    for (int x : a) {
        // 严格上升找第一个 >= x 的位置替换；改 upper_bound 即允许相等（非降）
        auto it = lower_bound(d.begin(), d.end(), x);
        if (it == d.end()) d.push_back(x);
        else *it = x;
    }
    return (int)d.size();
}
```

LIS：恢复一组最长严格上升子序列

赛时先看

题目信号：题目不仅问最长长度，还要求输出选了哪些元素；或答案要沿着一条严格递增链继续处理。

本质：除了长度，还要输出一组方案、记录链上的原下标，或把 LIS 当作后续构造的骨架。

接法：需要输出一条递增链时用这个模板。`result.indices` 是原数组下标，默认 `0-based`；如果题目要求输出位置从 `1` 开始，输出时 `+1`。如果只要长度，用上一小节的 `lis_strict` 更简洁。

复杂度判定： $O(n \log n)$ ，额外空间 $O(n)$ 。

维护的量：`tails_value/tails_index`（每个长度的最小结尾值及其原下标）；`previous[i]`（`i` 在最优链上的前驱下标）。

警告：`tails_value[len - 1]` 只表示“长度为 `len`

的最小结尾值”，它本身不是答案；恢复必须额外保存每个位置的前驱 `previous[i]`。严格上升用

`lower_bound`，非降子序列改成 `upper_bound`。

【最小完整示例（先抄这一段就能跑）】

题目：`n` 个数（ $n \leq 1e5$ ），输出一组最长严格上升子序列（长度 + 选中的值）。

```
vector<i64> a(n);
for (int i = 0; i < n; ++i) cin >> a[i]; // a 是 0-indexed
auto res = longest_increasing_subsequence(a); // 1. 调用
cout << res.length << '\n'; // 2. 取结果：最长长度
for (i64 x : res.values) cout << x << ' '; // 3. 取结果：选中的元素值
cout << '\n'; // 下标要 1-based 输出时 res.indices 各项 +1
```

样例：`a = [3,1,4,1,5,9,2,6]` → `length = 4`，一组答案 `{1,4,5,9}`（下标 `1,2,4,5`）。

【传参要求（照这个传不会错）】

- `longest_increasing_subsequence(a)`：`a` `0-indexed`（内部从 `0` 遍历到 `size()-1`），元素类型 `i64`，可为负。
- 返回值 `LISResult`：`length` = 最长长度；`indices` = 被选元素在原数组中的下标（`0-based`），递增；`values` = 对应元素值。
- 严格上升用 `lower_bound`；非降（允许相等）把 `lower_bound` 改成 `upper_bound`。
- 只问长度用上一节 `lis_strict` 更简洁；`a` 为空时返回 `length = 0`。

```
struct LISResult {
    int length = 0;
    vector<int> indices; // 原数组中被选元素的下标，递增。
    vector<i64> values;
};

LISResult longest_increasing_subsequence(const vector<i64>& a) {
    int n = (int)a.size();
    vector<i64> tails_value;
    vector<int> tails_index, previous(n, -1);

    for (int i = 0; i < n; ++i) {
        int pos = (int)(lower_bound(tails_value.begin(), tails_value.end(), a[i])
            - tails_value.begin());
        if (pos > 0) previous[i] = tails_index[pos - 1];
        if (pos == (int)tails_value.size()) {
            tails_value.push_back(a[i]);
            tails_index.push_back(i);
        } else {
            tails_value[pos] = a[i];
            tails_index[pos] = i;
        }
    }
}
```

```

    }

    LISResult result;
    result.length = (int)tails_value.size();
    for (int at = result.length == 0 ? -1 : tails_index.back();
        at != -1; at = previous[at]) {
        result.indices.push_back(at);
    }
    reverse(result.indices.begin(), result.indices.end());
    for (int index : result.indices) result.values.push_back(a[index]);
    return result;
}

```

LIS 计数：最长严格上升子序列有多少条

赛时先看

题目信号：题目问最长递增链的数量、最长嵌套方案数，值域很大但只涉及大小比较；要同时维护“最长长度”和“达到该长度的计数”。

本质：求 LIS 的长度及“按原下标不同”计算的方案数。

接法：如果只问 LIS 长度，翻本节上一个小节

lis_strict：如果问“最长递增链有多少条”，用这个树状数组版本。读每个数 x 时，只能接在所有 $< x$ 的最优链后面，所以查询 `rank-1`；得到新链后更新 `rank`。

复杂度判定：离散化加树状数组， $O(n \log n)$ 。

维护的量：`tree` (Fenwick 每个节点存 `LISCount{length, ways}`，按排名合并最优)；`values` (离散化后的值，用于把 x 转成排名)。

警告：本实现的计数按下标区分，值相同但位置不同的子序列算不同方案。严格上升必须查询排名 $< \text{rank}$ ，不能查 $\leq$

`rank`；空数组按唯一空子序列返回 `{0,1}`。

【最小完整示例（先抄这一段就能跑）】

题目： n 个数，求最长严格上升子序列的长度和条数（模 `mod`）。

```

vector<i64> a(n);
for (int i = 0; i < n; ++i) cin >> a[i];
auto res = count_longest_increasing_subsequences(a, mod); // 1. 调用：a 0-indexed
cout << res.length << ' ' << res.ways << '\n'; // 2. 取结果：长度 与 方案数模 mod

```

样例：`a = [1,3,2,4]` → 输出 `3 2` (`1,2,4` 与 `1,3,4`)。

【传参要求（照这个传不会错）】

- `count_longest_increasing_subsequences(a, mod = modn)`: `a` 0-indexed (长度 n)，元素 `i64`、可为负可重复；`mod` = 取模数，缺省用全局 `modn`。
- 返回值 `LISCount{length, ways}`: `length` = LIS 长度；`ways` = 达到该长度的方案数模 `mod`。
- 计数按下标区分：值相同但位置不同算不同方案；`a` 为空返回 `{0, 1}`。
- 只问长度用 `lis_strict`；`FenwickLISCount` 的 `merge/query/update` 是内部方法，不要直接调用。

【API / 入口函数（赛时只认这里列的名字）】

- `count_longest_increasing_subsequences(a, mod = modn)` → 主入口：求 LIS 长度及方案数，返回 `LISCount{length, ways}`。内部 `FenwickLISCount` 的 `merge/query/update` 只是内部方法，不要直接调。

```

struct LISCount {
    int length = 0;
    int ways = 0;
};

struct FenwickLISCount {
    int n, mod;
    vector<LISCount> tree;

    FenwickLISCount(int n, int mod) : n(n), mod(mod), tree(n + 1) {}

    LISCount merge(LISCount a, LISCount b) const {
        if (a.length != b.length) return a.length > b.length ? a : b;
        if (a.length == 0) return {0, 0}; // 查询空前缀时，稍后人为补一条空链。
        return {a.length, (a.ways + b.ways) % mod};
    }

    void update(int index, LISCount value) {
        for (; index <= n; index += index & -index) tree[index] = merge(tree[index], value);
    }

    LISCount query(int index) const { // [1, index] 的最优 (长度, 方案数)

```

```

        LISCount result;
        for (; index > 0; index -= index & -index) result = merge(result, tree[index]);
        return result;
    }
};

LISCount count_longest_increasing_subsequences(const vector<i64>& a, int mod = modn) {
    if (a.empty()) return {0, 1 % mod};
    vector<i64> values = a;
    sort(values.begin(), values.end());
    values.erase(unique(values.begin(), values.end()), values.end());

    FenwickLISCount bit((int)values.size(), mod);
    for (i64 x : a) {
        int rank = (int)(lower_bound(values.begin(), values.end(), x) - values.begin()) + 1;
        LISCount best = bit.query(rank - 1); // 严格小于 x。
        int ways = best.length == 0 ? 1 % mod : best.ways;
        bit.update(rank, {best.length + 1, ways});
    }
    return bit.query((int)values.size());
}

```

概念辨析：子段、子序列与子串

赛时先看

题目信号：数组题若要求选中的元素下标连续，是子段；若只要求保留原相对顺序、可以删除中间元素，是子序列。字符串中的连续片段叫子串，和数组子段同样要求连续。

本质：避免把名称相近、但约束完全不同的模型套错。

接法：读到“连续”两个字，先想子段/子串；读到“删除若干但相对顺序不变”，先想子序列。数组连续最大和翻 Kadane；字符串连续匹配翻 KMP/哈希/SAM；不连续公共部分翻 LCS/序列自动机。这个判断错了，后面代码再对也会 WA。

复杂度判定：这是术语判断，不需要额外算法；读题时先确认“连续”与“可删除”的含义，再选对应模板。

警告：Kadane 解决的是最大子段和（Maximum Subarray / 最大连续子数组和），不是最大子序列和。若题目允许任意跳过元素、只要求和最大：允许空选时取所有正数之和；必须非空且全为非正时取最大元素。只有附加长度、数量、单调性或其他约束时，才会形成真正的子序列 DP。为了照顾常见口头误称，网页搜索“最大子序列和”仍会带你来到本节，但打印和展示标签只使用正确术语“最大子段和”。

带权区间选择：最大权不相交任务

赛时先看

题目信号：任务、演出、订单、预约、机器排程等都有时间区间和收益；允许一个任务恰好在另一个结束时开始。

本质：从很多有开始、结束、收益的任务中选若干个，不重叠地取得最大总收益，并恢复选中的任务。

接法：把每个任务读成 {start,end,weight,id}，全部丢给

maximum_weight_non_overlapping_intervals。如果题目区间是闭区间 [l,r]

且两个任务不能共享端点，就把兼容条件理解成 previous.end < current.start，可以通过把 end 预处理成 r+1 转成半开区间。

复杂度判定：排序加二分为 $O(n \log n)$ 。

维护的量：dp[i]（前 i 个按结束时间排序的任务的最大收益）；previous_count[i]/take[i]（记录每个任务的兼容前驱与“是否选它”，用于恢复方案）。

警告：这里统一使用半开区间 [start, end)，所以 previous.end <= current.start 才兼容。不要把“按开始时间贪心”错当成带权版本；带权时贪心通常不成立。

【最小完整示例（先抄这一段就能跑）】

题目：n 个任务，每个有开始、结束、收益，选互不重叠的任务使总收益最大，并输出选中的任务编号。

```

vector<WeightedInterval> intervals(n);
for (int i = 0; i < n; ++i) {
    cin >> intervals[i].start >> intervals[i].end >> intervals[i].weight;
    intervals[i].id = i + 1; // 1. 记住原题编号
}
auto res = maximum_weight_non_overlapping_intervals(intervals); // 2. 调用：0-indexed
cout << res.max_weight << '\n'; // 3. 取结果：最大总收益
for (int id : res.chosen_ids) cout << id << ' '; // 4. 取结果：选中的任务编号
cout << '\n';

```

样例：任务 (1,3,2) (2,5,4) (4,6,4) -> 输出 6，选中 1 3。

【传参要求（照这个传不会错）】

- `maximum_weight_non_overlapping_intervals(intervals)`: `intervals` 0-indexed, 每个元素填 `start/end/weight/id` 四个字段。
- 返回值 `WeightedIntervalResult`: `max_weight` = 最大总收益; `chosen_ids` = 选中的 `id` 列表 (按输入顺序, 升序)。
- 区间是半开 `[start, end)`: 上一个的 `end <=` 下一个的 `start` 才兼容; 题面给闭区间 `[l,r]` 且端点不能共享时, 把 `end` 存成 `r+1`。
- 只问最大值不必填 `id` (保持默认 0 即可); 要恢复方案则必须给每个任务填不同的 `id`。带权问题贪心不成立, 必须用这个 DP。
- `id`: 原题的任务编号, 便于恢复答案。
- `dp`: DP 状态。

```
struct WeightedInterval {
    i64 start, end, weight;
    int id; // 原题的任务编号, 便于恢复答案。
};

struct WeightedIntervalResult {
    i64 max_weight = 0;
    vector<int> chosen_ids;
};

WeightedIntervalResult maximum_weight_non_overlapping_intervals(
    vector<WeightedInterval> intervals
) {
    sort(intervals.begin(), intervals.end(), [](const auto& a, const auto& b) {
        if (a.end != b.end) return a.end < b.end;
        return a.start < b.start;
    });
    int n = (int)intervals.size();
    vector<i64> ends(n);
    for (int i = 0; i < n; ++i) ends[i] = intervals[i].end;

    vector<i64> dp(n + 1, 0); // 前 i 个按结束时间排序的任务的最优收益。
    vector<int> previous_count(n + 1);
    vector<char> take(n + 1, false);
    for (int i = 1; i <= n; ++i) {
        // 前 i-1 个任务中, 结束时间 <= 当前开始时间的任务数量。
        int compatible = (int)(upper_bound(ends.begin(), ends.begin() + i - 1,
                                           intervals[i - 1].start) - ends.begin());

        previous_count[i] = compatible;
        i64 use_current = dp[compatible] + intervals[i - 1].weight;
        if (use_current > dp[i - 1]) {
            dp[i] = use_current;
            take[i] = true;
        } else {
            dp[i] = dp[i - 1];
        }
    }

    WeightedIntervalResult result;
    result.max_weight = dp[n];
    for (int i = n; i > 0; i--) {
        if (take[i]) {
            result.chosen_ids.push_back(intervals[i - 1].id);
            i = previous_count[i];
        } else {
            --i;
        }
    }
    reverse(result.chosen_ids.begin(), result.chosen_ids.end());
    return result;
}
```

最大子段和：无限制、环形、恰好 K 与长度区间 [L,R]

赛时先看

题目信号: 题面出现“最大连续子数组和 / 最大连续收益 /

最大连续得分”, 元素可正可负。若要求“连续”, 用本节; 若允许跳过元素, 则是

LIS、背包或其他“子序列”问题, 不能套 Kadane。长度恰好为 `K` 时用定长滑动窗口; 长度范围为 `[L,R]`

时, 对每个右端点减去可行左端点中的最小前缀和。

本质: 在数组中选一个非空连续子段, 使元素和最大。给出无限制长度的 Kadane、环形数组、长度恰好为 `K`、长度至多为 `K`, 以及长度限制在 `[L,R]` 的完整版本。

维护的量: 只读原数组 `a`; 长度区间版另维护前缀和 `prefix` 与存左端点下标的单调队列 `candidates`; 恰好 `K` 版维护定长窗口和 `window`。

接法：无限制长度用 `maximum_subarray_sum_nonempty`；环形数组用 `maximum_circular_subarray_sum_nonempty`；恰好 K 个连续元素用 `maximum_subarray_sum_exactly_k`；长度在 $[L,R]$ 用 `maximum_subarray_sum_length_between`；如果题目说“最多 K /至少 K ”，分别把区间写成 $[1,K]$ 或 $[K,n]$ 。所有函数都默认必须选非空连续段。

复杂度判定：所有函数均为 $O(n)$ 时间。无限制、环形、恰好 K 使用 $O(1)$ 额外空间；长度区间版使用 $O(n)$ 前缀和与 $O(R-L+1)$ 单调队列空间。

警告：以下均要求选非空子段，且长度参数须满足 $1 \leq K \leq n$ 或 $1 \leq L \leq R \leq n$ 。全负环形数组不能靠 `total - min_subarray` 凑答案（那会选到空段），必须直接返回最大元素。前缀和 `prefix[i]` 表示前 i 个元素之和；固定右端点 `right` 时，子段 $[left, right)$ 的和是 `prefix[right] - prefix[left]`。

【最小完整示例（先抄这一段就能跑）】

- 题目：数组 $[-2, 1, -3, 4, -1, 2, 1, -5, 4]$ 的最大连续子段和家族。
- 调用（抄进 `solve()` 直接跑）：

```
vector<i64> a = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
i64 p = maximum_subarray_sum_nonempty(a); // 无限制: 6
i64 c = maximum_circular_subarray_sum_nonempty(a); // 环形: 6
i64 k = maximum_subarray_sum_exactly_k(a, 3); // 恰好 3 个: 5
i64 b = maximum_subarray_sum_length_between(a, 2, 4); // 长度 [2,4]: 6
cout << p << " " << c << " " << k << " " << b << "\n";
```

- 样例数值：6 6 5 6。

【传参要求（照这个传不会错）】

- `maximum_subarray_sum_nonempty(a)`：a 原数组（下标 $0..n-1$ ，允许负数，非空）；返回 `i64` 最大非空连续子段和。
- `maximum_circular_subarray_sum_nonempty(a)`：同 a；返回环形最大子段和；全负时直接返回最大元素（不能靠 `total - min` 凑）。
- `maximum_subarray_sum_exactly_k(a, k)`：k 窗口长度，要求 $1 \leq k \leq n$ ；返回恰好 k 个连续元素的最大和。
- `maximum_subarray_sum_length_between(a, min_len, max_len)`：`min_len/max_len` 长度上下界，要求 $1 \leq \text{min_len} \leq \text{max_len} \leq n$ ；返回长度在 $[\text{min_len}, \text{max_len}]$ 的最大和。
- `maximum_subarray_sum_at_most_k(a, k)` / `maximum_subarray_sum_at_least_k(a, k)`：内部就是 $[1,k]$ / $[k,n]$ 的 `length_between` 特例；要求 $1 \leq k \leq n$ 。

【API / 入口函数（赛时只认这里列的名字）】

- `maximum_subarray_sum_nonempty(const vector<i64>& a)` -> 无限制长度的 Kadane，必选非空子段。返回 `i64`。
- `maximum_circular_subarray_sum_nonempty(const vector<i64>& a)` -> 环形数组最大子段和，必选非空子段。返回 `i64`。
- `maximum_subarray_sum_length_between(const vector<i64>& a, int min_len, int max_len)` -> 长度范围 $[\text{min_len}, \text{max_len}]$ 的单调队列版，`at_least/at_most` 内部都复用它。返回 `i64`。
- `maximum_subarray_sum_at_least_k(const vector<i64>& a, int k)` -> “长度至少 K ”也常见：范围 $[K, n]$ 。返回 `i64`。
- `maximum_subarray_sum_at_most_k(const vector<i64>& a, int k)` -> “长度至多 K ”就是长度范围 $[1, K]$ 的特例。返回 `i64`。
- `maximum_subarray_sum_exactly_k(const vector<i64>& a, int k)` -> 恰好选 K 个连续元素：窗口 $[\text{right}-K, \text{right})$ 唯一确定。返回 `i64`。

检索提示：若网页里输入“最大子序列和”，本节也会被命中以纠正这个常见口头名称；正式术语和打印标签仍应写作“最大子段和”。

```
i64 maximum_subarray_sum_nonempty(const vector<i64>& a) {
    assert(!a.empty());
    i64 current = a[0], answer = a[0];
    for (int i = 1; i < (int)a.size(); ++i) {
        current = max(a[i], current + a[i]);
        answer = max(answer, current);
    }
    return answer;
}

i64 maximum_circular_subarray_sum_nonempty(const vector<i64>& a) {
    assert(!a.empty());
    i64 total = accumulate(a.begin(), a.end(), 0LL);
    i64 max_end = a[0], max_sum = a[0];
    i64 min_end = a[0], min_sum = a[0];
    for (int i = 1; i < (int)a.size(); ++i) {
        max_end = max(a[i], max_end + a[i]);
```

```

        max_sum = max(max_sum, max_end);
        min_end = min(a[i], min_end + a[i]);
        min_sum = min(min_sum, min_end);
    }
    if (max_sum < 0) return max_sum; // 否则 total - min_sum 会错误地选空段。
    return max(max_sum, total - min_sum);
}

// 恰好选 K 个连续元素：窗口 [right-K, right) 唯一确定。
i64 maximum_subarray_sum_exactly_k(const vector<i64>& a, int k) {
    int n = (int)a.size();
    assert(1 <= k && k <= n);
    i64 window = accumulate(a.begin(), a.begin() + k, 0LL);
    i64 answer = window;
    for (int right = k; right < n; ++right) {
        window += a[right] - a[right - k];
        answer = max(answer, window);
    }
    return answer;
}

// 选连续子段长度在 [min_len, max_len] 中。
i64 maximum_subarray_sum_length_between(
    const vector<i64>& a, int min_len, int max_len
) {
    int n = (int)a.size();
    assert(1 <= min_len && min_len <= max_len && max_len <= n);
    vector<i64> prefix(n + 1);
    for (int i = 0; i < n; ++i) prefix[i + 1] = prefix[i] + a[i];

    deque<int> candidates; // 前缀和单调递增；存可作为左端点的下标。
    i64 answer = -(1LL << 62);
    for (int right = 1; right <= n; ++right) {
        // 新加入 left = right - min_len, 保证长度不短于 min_len。
        int entering = right - min_len;
        if (entering >= 0) {
            while (!candidates.empty() && prefix[candidates.back()] >= prefix[entering]) {
                candidates.pop_back();
            }
            candidates.push_back(entering);
        }
        // left < right - max_len 时, 长度已经超过 max_len。
        while (!candidates.empty() && candidates.front() < right - max_len) {
            candidates.pop_front();
        }
        if (!candidates.empty()) {
            answer = max(answer, prefix[right] - prefix[candidates.front()]);
        }
    }
    return answer;
}

// “长度至多 K” 就是长度范围 [1, K] 的特例。
i64 maximum_subarray_sum_at_most_k(const vector<i64>& a, int k) {
    return maximum_subarray_sum_length_between(a, 1, k);
}

// “长度至少 K” 也常见：范围 [K, n]。
i64 maximum_subarray_sum_at_least_k(const vector<i64>& a, int k) {
    return maximum_subarray_sum_length_between(a, k, (int)a.size());
}

```

相邻区间合并：石子合并最小代价

赛时先看

题目信号：石子、文件、括号段、相邻颜色块等只能按原顺序合并；合并后的代价依赖整段权重。

本质：一排相邻块只能合并相邻两段，每次代价等于新段的总权重，求总成本最小。

维护的量：`prefix`（前缀和，供 `range_sum` 算区间代价）；`dp[1][r]`（合并闭区间 `[1, r]` 的最小代价，0-indexed）。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；`solve()`

里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定：朴素 $O(n^3)$ ，空间 $O(n^2)$ 。若题目满足 Knuth 优化的四边形不等式/决策单调性，再翻 Knuth 模板降至 $O(n^2)$ 。

警告：本模板是直线，不是环。环形石子合并要把数组复制一遍并枚举长度为 `n` 的起点；转移顺序是先对所有分割取最小值，再在末尾加上整段和。

约定：`range_sum(1, r)` 表示闭区间 `[1, r]` 的元素和（前缀和实现）。

【最小完整示例（先抄这一段就能跑）】

- 题目：石子堆重量 [4, 1, 5, 2]，相邻两堆才能合并，求最小总代价。
- 调用（抄进 `solve()` 直接跑）：

```
vector<i64> weight = {4, 1, 5, 2};
i64 ans = minimum_adjacent_merge_cost(weight);
cout << ans << "\n";
```

- 样例数值：24。

【传参要求（照这个传不会错）】

- `minimum_adjacent_merge_cost(weight)`: `weight` 各堆石子重量，下标 `0..n-1`（允许任意值）；返回 `i64` 直线合并的最小总代价；`n <= 1` 时返回 `0`。环形问题把数组复制一份，对每个长度为 `n` 的窗口取最小值。

```
i64 minimum_adjacent_merge_cost(const vector<i64>& weight) {
    int n = (int)weight.size();
    if (n <= 1) return 0;
    vector<i64> prefix(n + 1);
    for (int i = 0; i < n; ++i) prefix[i + 1] = prefix[i] + weight[i];
    auto range_sum = [&](int l, int r) { // 闭区间 [l, r]。
        return prefix[r + 1] - prefix[l];
    };

    const i64 INF64 = (1LL << 62);
    vector<vector<i64>> dp(n, vector<i64>(n));
    for (int len = 2; len <= n; ++len) {
        for (int l = 0; l + len <= n; ++l) {
            int r = l + len - 1;
            dp[l][r] = INF64;
            for (int mid = l; mid < r; ++mid) {
                dp[l][r] = min(dp[l][r], dp[l][mid] + dp[mid + 1][r]);
            }
            dp[l][r] += range_sum(l, r);
        }
    }
    return dp[0][n - 1];
}
```

矩阵链乘：相乘顺序的最小标量乘法次数

赛时先看

题目信号：`Ai` 的尺寸为 `dimension[i] x`

`dimension[i+1]`，所有矩阵都要按原顺序相乘，但括号位置可变；代价由两个子结果的尺寸相乘决定。

本质：给定一系列维度相容的矩阵，选择括号化方式，使乘法标量次数最少，并恢复一种括号方案。

维护的量：`dp[l][r]`（子链 `Al..Ar` 的最少标量乘法次数）；`split[l][r]`（最优分割点，用来恢复括号方案）。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；`solve()`

里先构造对象，再按下方“不会用就看这里”调用公开接口。

复杂度判定： $O(n^3)$ 时间、 $O(n^2)$ 空间，其中 `n = dimension.size()-1`。

警告：第 `mid` 个分割是 `[l, mid]` 与 `[mid + 1, r]`，乘法代价是 `dimension[l] * dimension[mid + 1] * dimension[r + 1]`。若维度和成本可能超过 `i64`，把 `i64` 改成 `i128`。

【最小完整示例（先抄这一段就能跑）】

- 题目：矩阵尺寸 [10, 20, 50, 1]（A:10x20、B:20x50、C:50x1），求最优括号化。
- 调用（抄进 `solve()` 直接跑）：

```
vector<i64> dimension = {10, 20, 50, 1};
MatrixChainResult res = matrix_chain_multiplication(dimension);
cout << res.min_multiplications << "\n";
cout << res.parenthesization << "\n";
```

- 样例数值：1200 与 (A1 x (A2 x A3))。

【传参要求（照这个传不会错）】

- `matrix_chain_multiplication(dimension)`: `dimension` 长度 `n+1`，第 `i` 个矩阵 (`i = 0..n-1`) 尺寸是 `dimension[i] x dimension[i+1]`；返回 `MatrixChainResult`: `.min_multiplications` (`i64` 最少标量乘法次数)、`.parenthesization`（括号方案，`A1` 表示第 1 个矩阵）。

```
struct MatrixChainResult {
    i64 min_multiplications = 0;
    string parenthesization;
};

MatrixChainResult matrix_chain_multiplication(const vector<i64>& dimension) {
    int n = (int)dimension.size() - 1;
```

```

if (n <= 0) return {0, ""};
vector<vector<i64>> dp(n, vector<i64>(n));
vector<vector<int>> split(n, vector<int>(n, -1));
const i64 INF64 = (1LL << 62);

for (int len = 2; len <= n; ++len) {
    for (int l = 0; l + len <= n; ++l) {
        int r = l + len - 1;
        dp[l][r] = INF64;
        for (int mid = l; mid < r; ++mid) {
            i64 candidate = dp[l][mid] + dp[mid + 1][r]
                + dimension[l] * dimension[mid + 1] * dimension[r + 1];
            if (candidate < dp[l][r]) {
                dp[l][r] = candidate;
                split[l][r] = mid;
            }
        }
    }
}

function<string(int, int)> build = [&](int l, int r) -> string {
    if (l == r) return "A" + to_string(l + 1);
    int mid = split[l][r];
    return "(" + build(l, mid) + " x " + build(mid + 1, r) + ")";
};
return {dp[0][n - 1], build(0, n - 1)};
}

```

回文串最少分割：每段都是回文

赛时先看

题目信号：每一段都要回文、允许在任意位置切开，目标是段数/切割次数最少。若 n 只有几千，预处理回文区间后做前缀 DP 足够。

本质：把一个字符串切成若干个回文子串，求最少切割次数并恢复一种切分。

维护的量：`is_palindrome[l][r]` (`s[l..r]` 是否回文)；`parts[end]` (前 `end` 个字符的最少段数)；`previous[end]` (最优切点，用于恢复分段)。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；`solve()` 里先构造对象，再按下方“不会用就看这里”调用公开接口。

复杂度判定：时间 $O(n^2)$ ，空间 $O(n^2)$ 。

警告：`parts[i]` 表示前 i 个字符的最少“段数”，不是最少切割次数；最后答案要减一。空串这里返回 0 刀、空方案。

【最小完整示例（先抄这一段就能跑）】

- 题目：“aab” 切成若干回文段的最少切割次数。
- 调用（抄进 `solve()` 直接跑）：

```

string s = "aab";
PalindromicPartitionResult res = minimum_palindromic_partition(s);
cout << res.min_cuts << "\n";

```

- 样例数值：1（切一刀：aa | b）。

【传参要求（照这个传不会错）】

- `minimum_palindromic_partition(s)`：s 原字符串（下标 $0..n-1$ ）；返回 `PalindromicPartitionResult`：.min_cuts (int 最少切割次数，空串为 0)、.segments (切出的回文段，按原顺序)。

```

struct PalindromicPartitionResult {
    int min_cuts = 0;
    vector<string> segments;
};

PalindromicPartitionResult minimum_palindromic_partition(const string& s) {
    int n = (int)s.size();
    if (n == 0) return {0, {}};
    vector<vector<char>> is_palindrome(n, vector<char>(n));
    for (int l = n - 1; l >= 0; --l) {
        for (int r = l; r < n; ++r) {
            is_palindrome[l][r] = s[l] == s[r]
                && (r - l <= 2 || is_palindrome[l + 1][r - 1]);
        }
    }

    const int INF32 = 1e9;
    vector<int> parts(n + 1, INF32), previous(n + 1, -1);
    parts[0] = 0;
}

```

```

for (int end = 1; end <= n; ++end) {
    for (int start = 0; start < end; ++start) {
        if (is_palindrome[start][end - 1] && parts[start] + 1 < parts[end]) {
            parts[end] = parts[start] + 1;
            previous[end] = start;
        }
    }
}

PalindromicPartitionResult result;
result.min_cuts = parts[n] - 1;
for (int end = n; end > 0; end = previous[end]) {
    result.segments.push_back(s.substr(previous[end], end - previous[end]));
}
reverse(result.segments.begin(), result.segments.end());
return result;
}

```

区间 DP 骨架

赛时先看

题目信号：答案定义在 $[1, r]$ ；最后一步是把区间拆成两段或合并两段。

本质：合并区间、删除区间、两端收缩。

维护的量： $dp[l][r]$ （闭区间 $[l, r]$ 的答案，1-indexed）； a （原数组， $a[0]$ 是占位、实际元素从下标 1 开始）。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()`

里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定：常见 $O(n^3)$ 。

警告：按区间长度从小到大枚举。这是骨架：代码没有转移代价、恒返回

0，需要自己补转移代价与目标统计；直接照抄会得到无意义结果。

约定：`int n = (int)a.size() - 1; // 1-based, 下标从 1 开始`

【最小完整示例（先抄这一段就能跑）】

- 题目：骨架演示， $a = \{\text{占位}, 1, 2, 3\}$ ，跑完看枚举顺序（转移代价需自己补）。
- 调用（抄进 `solve()` 直接跑）：

```

vector<int> a = {0, 1, 2, 3}; // a[0] 占位，下标从 1 开始。
i64 ans = interval_dp_example(a);
cout << ans << "\n";

```

- 样例数值：0（骨架无转移代价，恒 0）。

【传参要求（照这个传不会错）】

- `interval_dp_example(a)`： a 长度 $n+1$ ， $a[0]$ 占位、 $a[1..n]$ 是实际元素（代码内 $n = a.size()-1$ ）；返回 `i64`。骨架没有转移代价、恒返回 $dp[1][n] = 0$ ，必须自己补转移代价与目标统计。

```

const i64 DP_INF = (1LL << 60);

i64 interval_dp_example(const vector<int>& a) {
    int n = (int)a.size() - 1; // 1-based, 下标从 1 开始。
    vector<vector<i64>> dp(n + 2, vector<i64>(n + 2, 0));

    for (int len = 2; len <= n; ++len) {
        for (int l = 1; l + len - 1 <= n; ++l) {
            int r = l + len - 1;
            dp[l][r] = DP_INF;
            for (int k = l; k < r; ++k) {
                dp[l][r] = min(dp[l][r], dp[l][k] + dp[k + 1][r]);
            }
        }
    }
    return dp[1][n];
}

```

Knuth 优化 DP

赛时先看

题目信号： $dp[l][r] = \min(dp[l][k] + dp[k][r] + w(l, r))$ ，且最优断点单调。

本质：区间 DP 中转移点具有单调性，常见于合并石子、最优二叉搜索树。

维护的量： $dp[l][r]$ （闭区间 $[l, r]$ 的最小合并代价）； $opt[l][r]$ （最优断点，只扫 $[opt[l][r-1], opt[l+1][r]]$ ）； pre （前缀和算区间代价）。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；优先调用下面 API 列出的入口，其余 helper 默认不要从 `solve()` 直接调。

复杂度判定：从 $O(n^3)$ 优化到 $O(n^2)$ 。

警告：需要满足四边形不等式和包含单调；区间端点定义要统一为开区间或闭区间。

约定：闭区间写法：合并石子，代价为区间和

【最小完整示例（先抄这一段就能跑）】

- 题目：石子堆（1-indexed）[4, 1, 5, 2]，相邻合并的最小总代价。
- 调用（抄进 `solve()` 直接跑）：

```
vector<int> a = {0, 4, 1, 5, 2}; // a[0] 占位，下标从 1 开始。
i64 ans = knuth_merge_stones(a);
cout << ans << "\n";
```

- 样例数值：24。

【传参要求（照这个传不会错）】

- `knuth_merge_stones(a)`: `a` 长度 `n+1`, `a[0]` 占位、`a[1..n]` 是石子重量（代码内 `n = a.size()-1`）；返回 `i64` 最小合并代价。要求代价满足四边形不等式与包含单调，否则答案可能不是最优。

【API / 入口函数（赛时只认这里列的名字）】

- `knuth_merge_stones(const vector<int>& a) ->` 闭区间写法：合并石子，代价为区间和。 返回 `i64`。

```
// 闭区间写法：合并石子，代价为区间和。
i64 knuth_merge_stones(const vector<int>& a) {
    int n = (int)a.size() - 1;
    vector<i64> pre(n + 1);
    for (int i = 1; i <= n; i++) pre[i] = pre[i - 1] + a[i];
    auto cost = [&](int l, int r) {
        return pre[r] - pre[l - 1];
    };
    const i64 INF = (1LL << 62);
    vector<dp>(n + 2, vector<i64>(n + 2, 0));
    vector<opt>(n + 2, vector<int>(n + 2, 0));
    for (int i = 1; i <= n; i++) opt[i][i] = i;
    for (int len = 2; len <= n; len++) {
        for (int l = 1; l + len - 1 <= n; l++) {
            int r = l + len - 1;
            dp[l][r] = INF;
            int from = opt[l][r - 1];
            int to = opt[l + 1][r];
            for (int k = from; k <= to; k++) {
                i64 val = dp[l][k] + dp[k + 1][r] + cost(l, r);
                if (val < dp[l][r]) {
                    dp[l][r] = val;
                    opt[l][r] = k;
                }
            }
        }
    }
    return dp[1][n];
}
```

状压 DP：TSP 骨架

赛时先看

题目信号：`n <= 20`，状态是一个集合。

本质：小规模集合状态，访问/选择哪些点。

维护的量：`dp[mask][u]`（已访问集合为 `mask`、当前停在 `u` 的最短路径长度）；`w`（`w[u][v]` 是 `u→v` 的边权）。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；`solve()`

里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O(2^n \cdot n^2)$ 。

警告：数组大小是 `1<<n`，注意内存。

【最小完整示例（先抄这一段就能跑）】

- 题目：4 个城市、从 0 出发走完所有城市的最短路径（不要求回起点）。
- 调用（抄进 `solve()` 直接跑）：

```
vector<vector<i64>> w = {
    {0, 10, 15, 20},
    {10, 0, 35, 25},
    {15, 35, 0, 30},
    {20, 25, 30, 0}
};
```

```
i64 ans = tsp_min_path(w, 0); // start 默认 0, 可省略。
cout << ans << "\n";
```

- 样例数值: 65 (0 -> 1 -> 3 -> 2)。

【传参要求 (照这个传不会错)】

- `tsp_min_path(w, start = 0)`: `w` 是 $n \times n$ 邻接矩阵 (`w[u][v]` = `u` 到 `v` 的边权, 下标 `0..n-1`, 无边给大数); `start` 起点城市, 默认 0, 范围 `[0, n)`; 返回 `i64` 从 `start` 出发、恰好访问每个点一次的最短路径长度 (不回起点); `n` 别太大 ($1 \ll n$ 的数组要开得下, $n \leq 20$ 左右)。

```
i64 tsp_min_path(const vector<vector<i64>>& w, int start = 0) {
    int n = (int)w.size();
    const i64 INF = (1LL << 60);
    vector<vector<i64>> dp(1 << n, vector<i64>(n, INF));
    dp[1 << start][start] = 0;

    for (int mask = 0; mask < (1 << n); ++mask) {
        for (int u = 0; u < n; ++u) {
            if (dp[mask][u] == INF) continue;
            for (int v = 0; v < n; ++v) {
                if (mask >> v & 1) continue;
                dp[mask | (1 << v)][v] = min(dp[mask | (1 << v)][v], dp[mask][u] + w[u][v]);
            }
        }
    }
    return *min_element(dp[(1 << n) - 1].begin(), dp[(1 << n) - 1].end());
}
```

SOS DP: 子集和变换

赛时先看

题目信号: 状态是 bitmask; 转移或查询形如 $\sum f[T], T \text{ subset } S$ 。

本质: 对所有集合 `S`, 快速求所有子集/超集贡献和。

维护的量: `f` (长度 $1 \ll n$ 的数组, 原地 zeta 累加); `bit/s` (按位分层、按掩码枚举的循环变量)。

接法: 把本节 `struct/class/函数组` 整段抄到 `solve()` 上面; 优先调用下面 API 列出的入口, 其余 helper 默认不要从 `solve()` 直接调。

复杂度判定: $O(n 2^n)$ 。

警告: 子集和与超集和循环方向不同; 数组长度是 $1 \ll n$ 。

【最小完整示例 (先抄这一段就能跑)】

- 题目: $n = 2, f = \{0, 1, 2, 3\}$ (下标即掩码), 求每个掩码的子集和 / 超集和。
- 调用 (抄进 `solve()` 直接跑):

```
int n = 2;
vector<i64> f = {0, 1, 2, 3};
vector<i64> sub = subset_sum_transform(f, n); // 子集和
vector<i64> sup = superset_sum_transform(f, n); // 超集和
```

- 样例数值: `sub = {0, 1, 2, 6}`, `sup = {6, 4, 5, 3}`。

【传参要求 (照这个传不会错)】

- `subset_sum_transform(f, n)`: `f` 长度必须恰好 $1 \ll n$ (下标 `0..2^n-1` 是掩码); `n` 位数; 返回新数组 `g`, $g[S] = \sum f[T], T \text{ 遍历 } S \text{ 的所有子集}$ 。`f` 按值传参、原数组不被破坏。
- `superset_sum_transform(f, n)`: 同上; 返回 $g[S] = \sum f[T], T \text{ 取所有以 } S \text{ 为子集的超集 (即 } S \subseteq T)$ 。

【API / 入口函数 (赛时只认这里列的名字)】

- `subset_sum_transform(vector<i64> f, int n) -> g[S] = sum f[T], T 是 S 的子集`。返回 `vector<i64>`。
- `superset_sum_transform(vector<i64> f, int n) -> g[S] = sum f[T], S 是 T 的子集`。返回 `vector<i64>`。

【改板时先认这几个量】

- `f`: 输入数组, 原地做子集/超集 zeta 累加。
- `n`: 位数, 数组长度 $1 \ll n$ 。
- `s`: 掩码循环变量。

```
// g[S] = sum f[T], T 是 S 的子集。
vector<i64> subset_sum_transform(vector<i64> f, int n) {
    for (int bit = 0; bit < n; bit++) {
        for (int s = 0; s < (1 << n); s++) {
            if (s & (1 << bit)) f[s] += f[s ^ (1 << bit)];
        }
    }
}
```



```

    }
    return f;
}

// g[S] = sum f[T], S 是 T 的子集。
vector<i64> superset_sum_transform(vector<i64> f, int n) {
    for (int bit = 0; bit < n; bit++) {
        for (int s = 0; s < (1 << n); s++) {
            if (!(s & (1 << bit))) f[s] += f[s | (1 << bit)];
        }
    }
    return f;
}

```

快速子集卷积 Fast Subset Convolution

赛时先看

题目信号：状态是位集；要把一个集合拆成两个互不相交部分，并累加两个函数的乘积；朴素枚举子集是 $O(3^n)$ ，但 n 已经到 $18 \sim 22$ 左右。

本质：计算不相交子集卷积 $h[S] = \sum_{T \subseteq S, T \cap S^c = \emptyset} f[T] * g[S \setminus T]$ ；按集合大小分层，用 zeta/逆 zeta 变换把复杂度从 $O(3^n)$ 降到 $O(n^2 2^n)$ 。

维护的量： $F[k][mask]/G[k][mask]$ （按大小 k 分层的 zeta

表）； $H[k][mask]$ （大小维卷积后的结果表）； mod （模数，默认 $\leq 1e9+7$ 保证 i64 乘法安全）。

接法：计数每个点集拆成两个独立结构的方案；连通子图 DP 中按不交集合并；集合划分、带颜色集合的组合 DP。

复杂度判定： $O(n^2 2^n)$ 时间、 $O(n 2^n)$ 空间。若 $n \leq 17$ 且常数小，朴素 $O(3^n)$

有时更短更快；本模板适合位数更大、或需要多次复用卷积实现的场景。

警告：不要把它和 OR 卷积混淆。答案中是 $S \setminus T$ ，所以两部分天然不交；若题目允许重叠，通常应考虑 OR 卷积。`__builtin_popcount(mask)` 的参数是 `unsigned int`，因此 n 不应大到让 $1 \ll n$ 溢出。

【最小完整示例（先抄这一段就能跑）】

- 题目： $n = 2$ ， $f = \{1, 2, 4, 8\}$ 、 $g = \{1, 1, 1, 1\}$ ，求不相交子集卷积。
- 调用（抄进 `solve()` 直接跑）：

```

i64 mod = 1000000007;
vector<i64> f = {1, 2, 4, 8}, g = {1, 1, 1, 1};
vector<i64> h = fast_subset_convolution(f, g, mod);

```

- 样例数值： $h = \{1, 3, 5, 15\}$ 。

【传参要求（照这个传不会错）】

- `fast_subset_convolution(f, g, mod)`： f 、 g 长度必须相等且都是 2^n （内部用 `__builtin_ctz` 推出 n ）； mod 模数，要求 $mod \leq 1e9+7$ 量级保证乘法不溢出；返回 `vector<i64> h`， $h[S] = \sum_{T \subseteq S, T \cap S^c = \emptyset} f[T] * g[S \setminus T] \mod mod$ ，两部分天然不交。

【改板时先认这几个量】

- f 、 g ：两个输入函数，长度均为 2^n 。
- mod ：模数，默认不大于 $1e9+7$ 以保证 i64 乘法安全。
- n ：位数。

$h[S] = \sum_{T \subseteq S, T \cap S^c = \emptyset} f[T] * g[S \setminus T]$ 。这不是 OR/AND/XOR 卷积：这里要求两个子集不交且并集正好为 S 。

使用： f 和 g 的长度必须都是 2^n ，系数在 mod 下运算；本实现默认 mod 不大于 $1e9+7$ ，以保证 i64 乘法安全。

```

// 返回 h[S] = sum_{T subseteq S} f[T] * g[S \setminus T] (mod mod)
vector<i64> fast_subset_convolution(
    const vector<i64>& f,
    const vector<i64>& g,
    i64 mod
) {
    const int N = (int)f.size();
    assert(N == (int)g.size() && N > 0 && (N & (N - 1)) == 0);
    const int n = __builtin_ctz((unsigned)N);

    vector<vector<i64>> F(n + 1, vector<i64>(N));
    vector<vector<i64>> G(n + 1, vector<i64>(N));
    vector<vector<i64>> H(n + 1, vector<i64>(N));

    // 按集合大小分层，再分别做子集 zeta 变换。
    for (int mask = 0; mask < N; ++mask) {
        int k = __builtin_popcount((unsigned)mask);
        F[k][mask] = (f[mask] % mod + mod) % mod;
    }
}

```

```

    G[k][mask] = (g[mask] % mod + mod) % mod;
}
for (int k = 0; k <= n; ++k) {
    for (int bit = 0; bit < n; ++bit) {
        for (int mask = 0; mask < N; ++mask) if (mask & (1 << bit)) {
            F[k][mask] += F[k][mask ^ (1 << bit)];
            if (F[k][mask] >= mod) F[k][mask] -= mod;
            G[k][mask] += G[k][mask ^ (1 << bit)];
            if (G[k][mask] >= mod) G[k][mask] -= mod;
        }
    }
}

// 对每个 S，把“大小”这一维做普通卷积。
for (int mask = 0; mask < N; ++mask) {
    for (int k = 0; k <= n; ++k) {
        for (int left = 0; left <= k; ++left) {
            H[k][mask] = (H[k][mask]
                + F[left][mask] * G[k - left][mask]) % mod;
        }
    }
}

// 逆 zeta 变换后，取与 |S| 同层的值。
for (int k = 0; k <= n; ++k) {
    for (int bit = 0; bit < n; ++bit) {
        for (int mask = 0; mask < N; ++mask) if (mask & (1 << bit)) {
            H[k][mask] -= H[k][mask ^ (1 << bit)];
            if (H[k][mask] < 0) H[k][mask] += mod;
        }
    }
}

vector<i64> h(N);
for (int mask = 0; mask < N; ++mask) {
    h[mask] = H[__builtin_popcount((unsigned)mask)][mask];
}
return h;
}

```

小 k 枚举 + 区间行 DP：无左移网格避障

赛时先看

题目信号：障碍数量 $k \leq 10 \sim 20$ ；每个障碍列不同；方案按 bitmask 输出；不能向左。

本质：网格只能向右/上/下走，少量障碍每个有二选一限制，对所有选择方案求最短路。

维护的量：obs_id[c]/obs_row[c]（列 c

的障碍编号与行号）；dp[row]（当前列走到各行的最少步数）；ans[mask]（绕行方案 mask 的最短路，-1 表示不可行）。

复杂度判定： $O(2^k * m * n)$ 。

警告：某列选择“从上方绕过”时可用行是 $[1, r-1]$ ，从下方绕过时是

$[r+1, n]$ ；从一行走到下一列时，前后两列的行号都必须在各自允许区间内。

【最小完整示例（先抄这一段就能跑）】

- 题目：3 x 4 网格，唯一障碍在第 2 行第 2 列，求两个绕行方案各自的最短路。
- 调用（抄进 solve() 直接跑）：

```

int n = 3, m = 4;
vector<pair<int, int>> obstacles = {{2, 2}}; // {行, 列}，列严格递增。
vector<i64> ans = min_steps_for_all_obstacle_masks(n, m, obstacles);

```

- 样例数值：ans = {3, 3}（两种绕行方案都需 3 步）。

【传参要求（照这个传不会错）】

- min_steps_for_all_obstacle_masks(n, m, obstacles): n/m 网格行数/列数（1-indexed，范围 1..n、1..m）；obstacles 是 {行, 列} 列表（行、列均从 1 开始，且列坐标必须严格递增）；返回 vector<i64>（长度 $1 \leq k$, $k = \text{obstacles.size}()$ ）：ans[mask] 是“第 i 个障碍按 mask 第 i 位 0=上绕/1=下绕”的最短步数，方案不可行为 -1。

```

vector<i64> min_steps_for_all_obstacle_masks(
    int n,
    int m,
    const vector<pair<int, int>>& obstacles // {行, 列}，列坐标严格递增。
) {
    const i64 INF = (1LL << 60);
    int k = (int)obstacles.size();

```

```

vector<int> obs_id(m + 1, -1), obs_row(m + 1, -1);
for (int i = 0; i < k; i++) {
    auto [r, c] = obstacles[i];
    obs_id[c] = i;
    obs_row[c] = r;
}

auto interval = [&](int col, int mask) -> pair<int, int> {
    int id = obs_id[col];
    if (id == -1) return {1, n};
    int r = obs_row[col];
    if (((mask >> id) & 1) == 0) return {1, r - 1};
    return {r + 1, n};
};

vector<i64> ans(1 << k, -1);
for (int mask = 0; mask < (1 << k); mask++) {
    vector<i64> dp(n + 1, INF);
    auto [l1, r1] = interval(1, mask);
    if (l1 > r1) continue;
    for (int r = l1; r <= r1; r++) dp[r] = 0;

    bool ok = true;
    for (int col = 2; col <= m; col++) {
        auto [l, r] = interval(col, mask);
        if (l > r) {
            ok = false;
            break;
        }
        vector<i64> enter(n + 1, INF), ndp(n + 1, INF);
        for (int row = 1; row <= r; row++) {
            if (dp[row] < INF) enter[row] = dp[row] + 1;
        }

        i64 best = INF;
        for (int row = 1; row <= r; row++) {
            best = min(best, enter[row] - row);
            ndp[row] = min(ndp[row], best + row);
        }
        best = INF;
        for (int row = r; row >= 1; row--) {
            best = min(best, enter[row] + row);
            ndp[row] = min(ndp[row], best - row);
        }
        dp.swap(ndp);
    }
    if (!ok) continue;
    i64 best = *min_element(dp.begin() + 1, dp.end());
    if (best < INF) ans[mask] = best;
}
return ans;
}

```

轮廓线 DP：骨牌覆盖

赛时先看

题目信号： $n \times m$ 网格，较小的一维 ≤ 12 ；每格和左/上/右/下局部相关。

本质：网格按行逐格推进，统计铺砖、连通性、障碍路径等状态压缩问题。

维护的量： $dp[mask]$ （当前轮廓线状态的铺法数， ndp 是推进一格后的新表）；内部自动让较小维度作状态维 m 。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；优先调用下面 API 列出的入口，其余 helper 默认不要从 `solve()` 直接调。

复杂度判定：骨牌覆盖约 $O(n \cdot m \cdot 2^m)$ 。

警告：让 m 为较小维度；本模板只覆盖空棋盘铺满方案，障碍格、连通性等扩展需要自己加在逐格转移的判断里。

【最小完整示例（先抄这一段就能跑）】

- 题目：2 x 3 空棋盘用 1x2 骨牌铺满的方案数。
- 调用（抄进 `solve()` 直接跑）：

```

i64 ways = domino_tiling(2, 3); // 内部自动 swap 成 n=3, m=2。
cout << ways << "\n";

```

- 样例数值：3。

【传参要求（照这个传不会错）】

- `domino_tiling(n, m)`： n/m 棋盘行数/列数（ ≥ 1 ，函数内部自动让较小的作为状态维 m ）；返回 `i64` 空棋盘用 1x2 骨牌铺满的方案数；要求状态维 m 别太大（数组长 $1 < m$ ，约 $m \leq 20$ ）。

【API / 入口函数（赛时只认这里列的名字）】

- `domino_tiling(int n, int m) ->` 统计 $n*m$ 空棋盘用 $1*2$ 骨牌铺满的方案数。返回 `i64`。

```
// 统计 n*m 空棋盘用 1*2 骨牌铺满的方案数。
i64 domino_tiling(int n, int m) {
    if (m > n) swap(n, m);
    vector<i64> dp(1 << m), ndp(1 << m);
    dp[0] = 1;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            fill(ndp.begin(), ndp.end(), 0);
            for (int mask = 0; mask < (1 << m); mask++) {
                i64 ways = dp[mask];
                if (!ways) continue;
                if (mask & (1 << j)) {
                    ndp[mask ^ (1 << j)] += ways;
                } else {
                    // 竖放，向下一行占同一列。
                    ndp[mask | (1 << j)] += ways;
                    // 横放，占当前行下一列。
                    if (j + 1 < m && !(mask & (1 << (j + 1)))) {
                        ndp[mask | (1 << (j + 1))] += ways;
                    }
                }
            }
        }
        dp.swap(ndp);
    }
    return dp[0];
}
```

数位 DP 骨架

赛时先看

题目信号：数字范围很大，限制和每一位数字有关。

本质：统计 $[1, n]$ 中数位和能被 3 整除的数的个数。

维护的量：`s` (`n` 的十进制串)；`memo` (非 `tight` 分支下 $(pos, sum, started)$ 的记忆化)；`dfs` 的四参 `pos/sum/started/tight`。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；`solve()`

里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。骨架的统计条件埋在代码里：`base case` 是 `started && sum % 3 == 0` (数位和 $\bmod 3 == 0$)，换题要改 `base case` 与记忆化参数。

复杂度判定： 0 (位数 $\times$ 状态数 $\times$ 转移数)；`memo` 可用数组替换 `map` 提速。

警告：`tight` 表示是否贴上界；`started` 处理前导零。

【最小完整示例（先抄这一段就能跑）】

- 题目： $[1, 123]$ 中数位和能被 3 整除的数的个数。
- 调用（抄进 `solve()` 直接跑）：

```
i64 ans = solve_digit_dp(123);
cout << ans << "\n";
```

- 样例数值：41。

【传参要求（照这个传不会错）】

- `solve_digit_dp(n)`：`n` 上界 ($n \geq 1$ ，统计 $[1, n]$)；返回 `i64` 满足“数位和 $\% 3 == 0$ ”的个数。换题只改 `base case` (`pos == s.size()` 处的判定条件) 与记忆化状态参数，`tight/started` 逻辑保留。

```
i64 solve_digit_dp(i64 n) {
    string s = to_string(n);
    map<tuple<int, int, int>, i64> memo;

    function<i64(int, int, int, bool)> dfs = [&](int pos, int sum, int started, bool tight) -> i64 {
        if (pos == (int)s.size()) {
            return started && (sum % 3 == 0);
        }
        if (!tight) {
            auto key = make_tuple(pos, sum, started);
            if (memo.count(key)) return memo[key];
        }
        int limit = tight ? s[pos] - '0' : 9;
        i64 res = 0;
        for (int d = 0; d <= limit; ++d) {
            int ns = started || d != 0;
            res += dfs(pos + 1, (sum + (ns ? d : 0)) % 3, ns, tight && d == limit);
        }
    };
    return dfs(0, 0, 0, true);
}
```

```

    if (!tight) memo[make_tuple(pos, sum, started)] = res;
    return res;
};

return dfs(0, 0, 0, true);
}

```

概率 DP：独立事件分布

赛时先看

题目信号：每枚硬币、每道题、每个开关独立成功，成功概率可以不同；题目问成功个数的分布或“正面数大于反面数”。

本质：若干独立 Bernoulli 事件依次发生，求恰好发生 k 次、至少发生某次数，或多数/少数发生的概率。

接法：读入每枚硬币正面概率数组 p 后调用 `probability_more_heads_than_tails(p)`；若问恰好 k 次成功，对 `independent_success_distribution(p)` 的返回值取 $dp[k]$ ，至少 k 次则累加 $dp[k..n]$ 。

复杂度判定： $O(n^2)$ 时间， $O(n)$ 空间。

维护的量： $dp[j]$ （已处理事件中恰有 j 次成功的概率，下标 $0..n$ ）。

警告：一维数组必须 j 倒序，否则同一个事件会被算多次；概率答案用浮点数，通常误差 $1e-9$ 足够；事件不独立时不能使用本块。

【最小完整示例（先抄这一段就能跑）】

三枚硬币，正面概率分别为 0.5、0.6、0.3。

```

vector<long double> p = {0.5L, 0.6L, 0.3L};
vector<long double> dp = independent_success_distribution(p); // dp[k] = 恰好 k 次成功
long double more = probability_more_heads_than_tails(p); // 正面多于反面
// 样例输出: dp = {0.14, 0.41, 0.36, 0.09}, more ≈ 0.45

```

【传参要求（照这个传不会错）】

- `independent_success_distribution(success_prob)`: `success_prob[i]` 是第 i 个事件（0-based, $0 \leq i < n$ ）的成功概率， $0 \leq p \leq 1$ （断言）；返回长度 $n+1$ 的数组， $dp[k]$ = 恰好 k 次成功的概率（long double）。
- `probability_more_heads_than_tails(head_prob)`: `head_prob[i]` 是第 i 枚硬币（0-based）的正面概率；返回正面数严格大于反面数的概率（long double）。各事件必须独立。

状态: $dp[j]$ 表示已处理的事件中，恰有 j 次成功的概率。新事件成功率为 p 时，倒序转移 $dp[j+1] += dp[j]*p$ 、 $dp[j] *= 1-p$ 。

```

vector<long double> independent_success_distribution(const vector<long double>& success_prob) {
    int n = (int)success_prob.size();
    vector<long double> dp(n + 1);
    dp[0] = 1;
    for (int i = 0; i < n; ++i) {
        long double p = success_prob[i];
        assert(0 <= p && p <= 1);
        for (int successes = i; successes >= 0; --successes) {
            dp[successes + 1] += dp[successes] * p;
            dp[successes] *= 1 - p;
        }
    }
    return dp; // dp[k] = 恰好 k 次成功的概率
}

long double probability_more_heads_than_tails(const vector<long double>& head_prob) {
    vector<long double> dp = independent_success_distribution(head_prob);
    long double answer = 0;
    for (int heads = (int)head_prob.size() / 2 + 1;
         heads <= (int)head_prob.size(); ++heads) {
        answer += dp[heads];
    }
    return answer;
}

```

典题模型：AtCoder DP Contest I - Coins。读入每枚硬币正面概率后调用

`probability_more_heads_than_tails(p)`；若问至少 k 次成功，则累加 $dp[k..n]$ 。

期望 DP：含自环的移项公式

赛时先看

题目信号：题面出现“随机抽到空位置就重来”“本轮可能没有任何进展”“直到完成为止的平均次数”，并能把未来状态写成 E 。

本质：随机过程每轮付出代价，问到终点的期望轮数/期望代价；当前状态可能以一定概率仍留在原状态。

复杂度判定：消掉一个状态自身的转移是 $O(1)$ ；整题复杂度取决于其余状态是否能按拓扑顺序

DP，或是否需要高斯消元。

维护的量： $E[s]$ （当前状态期望）、自环概率 p_self 、本轮代价 $step_cost$ 、其余状态贡献 $other$ 。

警告： $p_self = 1$ 且尚未终止意味着期望可能无穷；若转移之间不再能按拓扑顺序求值，不能硬套本公式，应翻 E 章「吸收马尔可夫链期望方程」小节的高斯消元模板；不要漏掉本轮已经花掉的代价（公式里的 $cost$ ）。

【最小完整示例（先抄这一段就能跑）】

抽卡：每轮花 1 次， $1/3$ 概率抽到重复卡留在原状态（自环）， $2/3$ 概率进入下一阶段（该阶段期望还需 4 次）。

```
long double ans = expectation_remove_self_loop(1.0L, 1.0L / 3, (2.0L / 3) * 4.0L);
// 样例输出: ans ≈ 5.5 (= (1 + 8/3) / (1 - 1/3))
```

【传参要求（照这个传不会错）】

- $step_cost$ ：本轮已经花掉的代价（轮数/次数），非负。
- $self_probability$ ：留在原状态的概率 p_self ，范围 $0 \leq p_self < 1$ （断言）；等于 1 且未终止时期望可能无穷。
- $other$ ：其余状态的贡献和 $\sum(p_i * E[to_i])$ ，先用题目的 DP 把所有非自环转移算好再传入。
- 返回：消去自环后的期望 $(step_cost + other) / (1 - self_probability)$ ($long\ double$)。

公式：若 $E[s] = cost + p_self * E[s] + \sum(p_i * E[to_i])$ ，则

$E[s] = (cost + \sum(p_i * E[to_i])) / (1 - p_self)$ 。

```
// 已知所有其他状态的期望贡献 other = sum(p_i * E[to_i]) 时，消掉自环。
long double expectation_remove_self_loop(
    long double step_cost, long double self_probability, long double other
) {
    assert(0 <= self_probability && self_probability < 1);
    return (step_cost + other) / (1 - self_probability);
}
```

典题：随机吃寿司到空盘的期望步数

赛时先看

题目信号：对象同质、状态只依赖于“还剩 1、2、3 个的对象分别有多少”，随机选到已经完成的对象会形成自环。这个模型同样适用于随机抽盒子、随机处理不同剩余容量任务。

本质：有 n 个盘子，每轮等概率选一个盘子；若盘中有寿司则吃掉一个。给定每盘初始 $1/2/3$ 个寿司，求全部吃完的期望轮数。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()`

里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O(n^3)$ 时间与空间； $n=300$ 时 `double` 数组约 220 MB，确认内存限制后再使用。

维护的量： $E[a][b][c]$ （还剩 $1/2/3$ 个寿司的盘子数分别为 a, b, c 时吃完的期望轮数， $a+b+c \leq n$ ）。

警告：分子是总盘数 n ，不是 s ；循环顺序要让右侧状态先计算；这里用 `double` 节省内存，若 n 更小可改 `long double`。

【最小完整示例（先抄这一段就能跑）】

三个盘子，初始寿司数分别为 1、2、3。

```
vector<int> plates = {1, 2, 3};
double ans = expected_sushi_steps(plates);
// 样例输出: ans ≈ 11.0 (AtCoder DP Contest J 第 1 个样例)
```

【传参要求（照这个传不会错）】

- `plates`：`plates[i]` 是第 i 个盘子（0-based）的初始寿司数，只允许 $1/2/3$ （断言）；盘子数 n 任意，但状态空间 $O(n^3)$ ， n 大时注意内存。
- 返回：全部吃完的期望轮数（`double`）；想要更高精度可把函数内的 `double` 换成 `long double`。

状态： $E[a][b][c]$ ，分别有 a, b, c 个盘子还剩 $1, 2, 3$ 个寿司。非空盘数为 $s=a+b+c$ ，移项后：

$E[a][b][c] = (n + aE[a-1][b][c] + bE[a+1][b-1][c] + cE[a][b+1][c-1]) / s$ 。

```
double expected_sushi_steps(const vector<int>& plates) {
    int n = (int)plates.size();
    vector<int> count(4);
    for (int x : plates) {
        assert(1 <= x && x <= 3);
        count[x]++;
    }
}
```

```

int width = n + 1;
auto index = [width](int a, int b, int c) -> size_t {
    return ((size_t)a * width + b) * width + c;
};
vector<double> dp((size_t)width * width * width, 0);
auto get = [&](int a, int b, int c) -> double& {
    return dp[index(a, b, c)];
};

for (int c = 0; c <= n; ++c) {
    for (int b = 0; b + c <= n; ++b) {
        for (int a = 0; a + b + c <= n; ++a) {
            int nonempty = a + b + c;
            if (nonempty == 0) continue;
            double value = n;
            if (a) value += a * get(a - 1, b, c);
            if (b) value += b * get(a + 1, b - 1, c);
            if (c) value += c * get(a, b + 1, c - 1);
            get(a, b, c) = value / nonempty;
        }
    }
}
return get(count[1], count[2], count[3]);
}

```

典题：AtCoder DP Contest J - Sushi。直接读入盘子数组后输出

`expected_sushi_steps(a)`；状态压缩的关键是只统计剩余数量的频数，不区分盘子编号。

树上随机游走：到根的期望步数

赛时先看

题目信号：图是一棵树、每一步均匀随机走边、目标是固定根。树边会来回走，但不需要高斯消元。

本质：无向树上每步等概率走向一个相邻点，求每个点首次走到根的期望步数。

接法：建无向树邻接表（0-based）后调用 `expected_hit_root_on_uniform_tree(graph, root)`，返回各点首次走到根的期望步数。

复杂度判定： $O(n)$ 时间和空间。

维护的量：`parent`（DFS 父节点）、`order`（遍历序）、`subtree_size`（子树大小）、`expectation`（答案数组）。

警告：这是无向树且每条边等概率的特例；如果边有转移概率、图有环或目标不止一个，转为期望方程/高斯消元。

【最小完整示例（先抄这一段就能跑）】

链 0-1-2，根为 0。

```

vector<vector<int>> g(3);
g[0].push_back(1); g[1].push_back(0);
g[1].push_back(2); g[2].push_back(1);
vector<long double> ans = expected_hit_root_on_uniform_tree(g, 0);
// 样例输出：ans = {0, 3, 4}（链上深度 d 的点为 d^2，可自查）

```

【传参要求（照这个传不会错）】

- `graph`：无向树邻接表（0-based），`graph[u]` 存 `u` 的所有邻居；必须是树（否则断言失败），点数为 `graph.size()`。
- `root`：目标根节点下标，默认 0，范围 $0 \leq \text{root} < n$ （断言）。
- 返回：长度 `n` 的 `long double` 数组，`ans[u]` = 从 `u` 首次随机游走到 `root` 的期望步数。

【改板时先认这几个量】

- `parent`：DFS 时记录的树上父节点。
- `order`：遍历顺序（BFS 序）。
- `subtree_size`：子树大小。
- `expectation`：答案数组，各点首次到根的期望步数。

结论：设 `sz[u]` 是以 `root` 为根时 `u` 子树大小，则从 `u` 首次走到父亲的期望步数为 $2 * \text{sz}[u] - 1$ 。所以 $E[\text{root}] = 0$ ， $E[u] = E[\text{parent}[u]] + 2 * \text{sz}[u] - 1$ 。

```

vector<long double> expected_hit_root_on_uniform_tree(
    const vector<vector<int>>& graph, int root = 0
) {
    int n = (int)graph.size();
    assert(0 <= root && root < n);
    vector<int> parent(n, -1), order{root};
    for (int i = 0; i < (int)order.size(); ++i) {

```



```

    int u = order[i];
    for (int v : graph[u]) {
        if (v == parent[u]) continue;
        assert(parent[v] == -1); // 这里要求输入是一棵树。
        parent[v] = u;
        order.push_back(v);
    }
}
assert((int)order.size() == n);

vector<int> subtree_size(n, 1);
for (int i = n - 1; i > 0; --i) {
    subtree_size[parent[order[i]]] += subtree_size[order[i]];
}

vector<long double> expectation(n);
for (int i = 1; i < n; ++i) {
    int u = order[i];
    expectation[u] = expectation[parent[u]] + 2 * subtree_size[u] - 1;
}
return expectation;
}

```

典题模型：给一棵无向树和根

0，每次随机走到相邻点，问从各点回到根的平均步数。建邻接表后调用本函数；链上深度 d 的点答案为 d^2 ，可用来手算检查。

均匀集卡与线性期望速查

赛时先看

题目信号：问“平均多少次”，总量能拆成很多指示变量之和，或已经收集 i 种时，得到新种类的概率为 $(kinds-i)/kinds$ 。

本质：从 $kinds$

种等概率卡片中反复抽，问集齐全部种类的期望抽取次数；或随机排列、随机染色中，求很多局部事件数量之和的期望。

接法：抽卡类型等概率时直接调用 `uniform_coupon_collector_expectation(k)`；随机排列的期望逆序对数调用 `expected_inversions_of_uniform_permutation(n)`。

复杂度判定：两者均为 $O(kinds)$ 或 $O(1)$ 。

维护的量：无状态结构；只用标量 $kinds$ （卡牌种类数）或 n （排列长度）。

警告：集卡公式要求每一种卡出现概率相同且独立；非均匀抽取不能把总种类数直接代入。线性期望虽然不要求各指示变量独立，但每个指示变量的期望与取值都要算对。

【最小完整示例（先抄这一段就能跑）】

6 种等概率卡片集齐的期望次数；长度 4 的随机排列期望逆序对数。

```

long double cards = uniform_coupon_collector_expectation(6);
long double invs = expected_inversions_of_uniform_permutation(4);
// 样例输出: cards ≈ 14.7 (= 6*(1+1/2+...+1/6)), invs = 3 (= 4*3/4)

```

【传参要求（照这个传不会错）】

- `uniform_coupon_collector_expectation(kinds)`: $kinds$ 是卡牌种类数, $kinds \geq 0$ (断言)，要求每种卡抽取概率相等；返回集齐全部种类的期望抽取次数 (`long double`)。
- `expected_inversions_of_uniform_permutation(n)`: n 是排列长度 (i64), $n \geq 0$ (断言)；返回均匀随机排列的期望逆序对数 $n*(n-1)/4$ (`long double`)。

结论：均匀集卡期望为 $kinds * (1 + 1/2 + \dots + 1/kinds)$ ；随机排列的期望逆序对数为 $n*(n-1)/4$ 。线性期望不需要事件互相独立。

```

long double uniform_coupon_collector_expectation(int kinds) {
    assert(kinds >= 0);
    long double answer = 0;
    for (int collected = 0; collected < kinds; ++collected) {
        answer += (long double)kinds / (kinds - collected);
    }
    return answer;
}

long double expected_inversions_of_uniform_permutation(i64 n) {
    assert(n >= 0);
    return (long double)n * (n - 1) / 4;
}

```

典型模型：抽卡类型等概率时直接调用

`uniform_coupon_collector_expectation(k)`：若各类概率不相等，不能代此公式，需要按状态做期望 DP 或容斥。随机排列的每一对元素以 $1/2$ 概率构成逆序，因此即使所有对之间相关，期望仍可逐对相加。

有限状态零和 MDP：值迭代差分收敛外推

赛时先看

题目信号：状态数几十到几百；每一步 Alice 取 `max`、Bob 取 `min`，之后随机转移；`k` 可到 $1e9$

或更大；矩阵快速幂不容易，因为有策略选择的非线性 `max/min`。

本质：小状态、超大轮数的随机零和博弈。先做足够多轮值迭代，若有限状态链不可约且非周期， $V_t(s) - V_{t-1}(s)$ 会趋于同一个长期平均收益 `g`，大轮数可用 $V_T(s) + (k-T)g$ 外推。

接法：把一轮最优转移写成 `bellman(state, previous_values)`；跑 `warmup` 轮；用所有状态的平均差分估计 `gain`；若 `k <= warmup` 直接查表，否则外推。

复杂度判定： $O(T_0 * \text{状态数} * \text{单次 Bellman 代价})$ 预处理，每个询问 $O(1)$ 。

维护的量：`dp[t][s]`（第 `t` 轮状态 `s` 的值）、`gain`（平均差分估计的长期收益）、`warmup`（预热轮数）。

警告：这是数值外推模板，不是严格适用于所有 MDP。要有强连通/不可约、非周期等稳定性；`T0`

要足够大并用样例或误差要求检验。输出一般用 `long double`。

【最小完整示例（先抄这一段就能跑）】

2 个状态：状态 0 每轮稳定 +1 分，状态 1 每轮 +0 分，无随机转移。

```
struct Bell {
    long double operator()(int s, const vector<long double>& prev) const {
        return s == 0 ? prev[0] + 1.0L : prev[1] + 0.0L;
    }
};
AverageRewardValueIteration<Bell> it;
it.init(2, 1000); // 状态数 2, 预热 1000 轮
it.run(Bell{});
long double v = it.value(1000000000LL, 0); // 第 1e9 轮状态 0 的值
// 样例输出: v ≈ 1e9 (每轮稳定收益 1, 外推值与真实值一致)
```

【传参要求（照这个传不会错）】

- `init(states, rounds)`: `states` 为状态数（下标 $0..states-1$ ）；`rounds` 为预热轮数 `warmup`，要足够大（如 $1e4$ ）。
- `run(bellman)`: `bellman(s, prev)` 给出第 `t` 轮状态 `s` 的最优转移值，`prev` 是第 `t-1` 轮全部状态的值（0-based）；跑完后内部自动把所有状态末轮差分平均成 `gain`。
- `value(rounds, state)`: `rounds` 为轮数（`long long`，可到 $1e18$ ）；`state` 为状态下标， $0 \leq state < states$ （断言）；返回 `long double`: `rounds <= warmup` 直接查表，否则 `dp[warmup][state] + (rounds - warmup) * gain` 外推。

【API / 入口函数（赛时只认这里列的名字）】

- `init(int states, int rounds)` -> 初始化/清空结构。
- `run(bellman)` -> 跑 `warmup` 轮值迭代，并用所有状态的平均差分估计长期收益 `gain`。
- `value(long long rounds, int state)` -> 查询第 `rounds` 轮状态 `state` 的值： `rounds <= warmup` 直接查表，否则用 `gain` 线性外推。返回 `long double`。

```
template <class Bellman>
struct AverageRewardValueIteration {
    int state_count = 0;
    int warmup = 0;
    vector<vector<long double>> dp;
    long double gain = 0;

    AverageRewardValueIteration() = default;
    AverageRewardValueIteration(int states, int rounds) { init(states, rounds); }

    void init(int states, int rounds) {
        state_count = states;
        warmup = rounds;
        dp.assign(warmup + 1, vector<long double>(state_count, 0));
        gain = 0;
    }

    void run(Bellman bellman) {
        for (int t = 1; t <= warmup; ++t) {
            for (int s = 0; s < state_count; ++s) {
                dp[t][s] = bellman(s, dp[t - 1]);
            }
        }
    }
};
```

```

    }
}
gain = 0;
for (int s = 0; s < state_count; ++s) {
    gain += dp[warmup][s] - dp[warmup - 1][s];
}
gain /= state_count;

long double value(long long rounds, int state) const {
    assert(0 <= state && state < state_count);
    if (rounds <= warmup) return dp[(int)rounds][state];
    return dp[warmup][state] + (rounds - warmup) * gain;
}
};

```

典题：本场 H 《Rock-Paper-Scissors Master》。双方手牌状态仅 $10 \times 10 = 100$ ，Bellman 转移枚举 Alice 出牌、Bob 出牌和双方补牌；预处理 $T0=10000$ 后对 $k>10000$ 线性外推。

14 DP 优化与高级状态模型

单调队列、斜率优化、Li Chao、分治优化、SMAWK、WQS、Slope Trick 集中放在这里。

单调队列优化 DP

赛时先看

题目信号： $dp[i] = \max(dp[j]) + val[i]$ ，且 j 在 $[i-k, i-1]$ 。

本质： 转移从滑动窗口中取最大/最小。

接法： 把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()`

里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O(n)$ 。

维护的量： $dp[i]$ （前 i 项的最优值）；双端队列 q （存候选下标，按 $dp[j]$ 从大到小）。

警告： 先弹过期，再取队首转移。队列为空时 $dp[i]$ 保持负无穷，后续状态不可达时注意别把 `NEG` 状态带进转移。

约定： `val` 为 1-indexed（长度 $n+1$ ），`val[0]` 不用。

【最小完整示例（先抄这一段就能跑）】

`val = {0, 3, 5, 1}`（1-indexed），窗口 $k = 2$ 。

```

vector<i64> val = {0, 3, 5, 1}; // val[0] 不用
vector<i64> dp = window_dp_max(val, 2);
// 样例输出: dp = {0, 3, 8, 9}

```

【传参要求（照这个传不会错）】

- `val`：长度 $n+1$ （1-indexed），`val[i]`（ $1 \leq i \leq n$ ）是第 i 个位置的值，`val[0]` 不参与。
- k ：转移窗口大小， $dp[i]$ 只能从 $j \in [i-k, i-1]$ 转移； $k \geq 1$ 。
- 返回：长度 $n+1$ 的 `i64` 数组， $dp[i] = \max(dp[j]) + val[i]$ ， $dp[0] = 0$ ；无合法转移的位置保持 `NEG`（不可达）。

```

vector<i64> window_dp_max(const vector<i64>& val, int k) {
    int n = (int)val.size() - 1;
    const i64 NEG = -(1LL << 60);
    vector<i64> dp(n + 1, NEG);
    deque<int> q;
    dp[0] = 0;
    q.push_back(0);

    for (int i = 1; i <= n; ++i) {
        while (!q.empty() && q.front() < i - k) q.pop_front();
        if (!q.empty()) dp[i] = dp[q.front()] + val[i];
        while (!q.empty() && dp[q.back()] <= dp[i]) q.pop_back();
        q.push_back(i);
    }
    return dp;
}

```

斜率优化：单调队列凸包

赛时先看

题目信号： 把转移式展开后，候选 j 变成一堆直线。

本质： 转移形如 $dp[i] = \min(k_j * x_i + b_j)$ 。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部函数参数直接当题面参数。

复杂度判定：斜率单调且查询单调时 $O(n)$ 。

维护的量：双端队列 `q`（存按斜率递增加入的下凸包直线 `Line{k,b}`）。

警告：如果斜率或查询不单调，改用 Li Chao Tree。

【最小完整示例（先抄这一段就能跑）】

依次加入 `y = 1x+3`、`y = 2x+1`、`y = 3x-2`，查询最小值。

```
HullMin h;
h.add_line(1, 3);
h.add_line(2, 1);
h.add_line(3, -2);
i64 best = h.query(1); // x=1 处取值 4、3、1
// 样例输出: best = 1
```

【传参要求（照这个传不会错）】

- `add_line(k, b)`: 加入直线 $y = k \cdot x + b$ (`i64`)；要求斜率 `k` 按加入顺序单调不减（本模板维护最小值下凸包）。
- `query(x)`: 查询所有已加入直线在 `x` 处的最小值 (`i64`)；要求查询点 `x` 单调不减。
- 返回: `i64` 最小值；斜率或查询不单调时结果错误，改用 Li Chao Tree。

【API / 入口函数（赛时只认这里列的名字）】

- `add_line(i64 k, i64 b) ->` 加入一条直线
- `query(i64 x) ->` 查询 返回 `i64`。

```
struct Line {
    i64 k, b;
    i64 eval(i64 x) const { return k * x + b; }
};

bool bad(const Line& a, const Line& b, const Line& c) {
    return (i128)(b.b - a.b) * (b.k - c.k) >=
        (i128)(c.b - b.b) * (a.k - b.k);
}

struct HullMin {
    deque<Line> q;

    void add_line(i64 k, i64 b) {
        Line cur{k, b};
        while (q.size() >= 2 && bad(q[q.size() - 2], q.back(), cur)) q.pop_back();
        q.push_back(cur);
    }

    i64 query(i64 x) {
        while (q.size() >= 2 && q[0].eval(x) >= q[1].eval(x)) q.pop_front();
        return q.front().eval(x);
    }
};
```

Li Chao Tree

赛时先看

题目信号：斜率不单调、查询 `x` 不单调，普通凸包队列不能用。

本质：动态加入直线，查询某个 `x` 处最小值。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部函数参数直接当题面参数。

复杂度判定：插入/查询 $O(\log X)$ 。

维护的量：线段树节点 `tr[p]`（每节点存区间中点更优的一条直线，含 `lc/rc/has`）；建树值域 `[L, R]`。

警告：需要知道 `x` 的值域；若 `x` 是离散集合，可以写离散 Li Chao。

【最小完整示例（先抄这一段就能跑）】

值域 `[-5, 5]`，加入 `y = 2x+1` 与 `y = -x+9`，查询 `x = 3` 处最小值。

```
LiChao lc(-5, 5);
lc.add_line(2, 1);
lc.add_line(-1, 9);
i64 best = lc.query(3); // 2*3+1 = 7 -1*3+9 = 6
// 样例输出: best = 6
```

【传参要求（照这个传不会错）】

- 构造 `LiChao lc(XL, XR)`: 整数查询值域 `[XL, XR]`，之后所有 `x` 必须落在其中；本版维护最小值。
- `add_line(k, b)`: 加入直线 $y = k * x + b$ (i64)，斜率任意、可乱序、可重复。
- `query(x)`: `XL <= x <= XR`；返回所有已加入直线在 `x` 处的最小值 (i64)，一条直线都没有时返回 `INF = 1LL << 62`。

【不会用就照抄】

```
LiChao lc(XL, XR);
lc.add_line(k, b);
auto best = lc.query(x);
```

- 先确认模板维护最大值还是最小值。
- `x` 的允许范围必须落在建树值域 `[XL, XR]`。

【API / 入口函数（赛时只认这里列的名字）】

- `LiChao lc(XL, XR)` -> 建立整数 `x` 值域 `[XL, XR]`，本版维护最小值。
- `lc.add_line(k, b)` -> 加入直线 $y = kx + b$ 。
- `lc.query(x)` -> 查询所有已加入直线在 `x` 处的最小值。

【核心逻辑（改代码时别破坏）】

- 每个节点保留在区间中点更优的直线；另一条只可能在左半或右半继续有机会。
- 查询沿 `x` 所在根到叶路径取最优值；无需斜率单调。

```
struct LiChao {
    struct Line {
        i64 k, b;
        i64 get(i64 x) const { return k * x + b; }
    };

    struct Node {
        Line line;
        int lc = 0, rc = 0;
        bool has = false;
    };

    vector<Node> tr;
    i64 L, R;
    const i64 INF = (1LL << 62);

    LiChao(i64 L_, i64 R_) : L(L_), R(R_) {
        tr.push_back(Node{});
        tr.push_back(Node{});
    }

    int new_node() {
        tr.push_back(Node{});
        return (int)tr.size() - 1;
    }

    void add_line(Line nw, int p, i64 l, i64 r) {
        if (!tr[p].has) {
            tr[p].line = nw;
            tr[p].has = true;
            return;
        }
        i64 mid = (l + r) >> 1;
        Line lo = tr[p].line, hi = nw;
        if (lo.get(mid) > hi.get(mid)) swap(lo, hi);
        tr[p].line = lo;
        if (l == r) return;
        if (lo.get(l) > hi.get(l)) {
            if (!tr[p].lc) tr[p].lc = new_node();
            add_line(hi, tr[p].lc, l, mid);
        } else if (lo.get(r) > hi.get(r)) {
            if (!tr[p].rc) tr[p].rc = new_node();
            add_line(hi, tr[p].rc, mid + 1, r);
        }
    }

    i64 query(i64 x, int p, i64 l, i64 r) const {
        if (!p) return INF;
        i64 ans = tr[p].has ? tr[p].line.get(x) : INF;
        if (l == r) return ans;
        i64 mid = (l + r) >> 1;
        if (x <= mid) return min(ans, query(x, tr[p].lc, l, mid));
        return min(ans, query(x, tr[p].rc, mid + 1, r));
    }
};
```

```

}

void add_line(i64 k, i64 b) { add_line({k, b}, 1, L, R); }
i64 query(i64 x) const { return query(x, 1, L, R); }
};

```

分治优化 DP

赛时先看

题目信号: `dp[g][i] = min(prev[k] + cost(k+1,i))`，并且决策点随 `i` 不下降。

本质: 分组 DP，最优决策点单调。

接法: 每层分组把 `prev/cur` 准备好，调用 `compute_dc(1, n, 0, n - 1, prev, cur, cost)`；`cost(1, r)` 是闭区间组代价，写成 `lambda` 传入。

复杂度判定: $O(G \ n \log \ n)$ 。

维护的量: `prev`（上一层 DP，只读）；`cur`（当前层，递归内逐段填，`cur[0]` 由外部初始化）；`opt_l/opt_r`（当前段决策点允许区间）；`mid/opt`（段中点与中点处最优决策点）。

警告: 必须确认决策单调，否则会错。

【最小完整示例（先抄这一段就能跑）】

```

// 示例: n=5 个元素分 G=2 组，组代价 cost(l,r)=段长，求最小总代价。
// 真正用时把 cost 换成题目自己的闭区间代价，并保证决策点单调。
int n = 5, G = 2;
vector<i64> prev(n + 1, 0), cur(n + 1);
auto cost = [](int l, int r) { return (i64)(r - l + 1); };
for (int g = 1; g <= G; ++g) {
    fill(cur.begin(), cur.end(), (1LL << 60));
    cur[0] = 0;
    compute_dc(1, n, 0, n - 1, prev, cur, cost);
    prev.swap(cur);
}
cout << prev[n] << '\n'; // 5（每段代价加起来就是 n）

```

- 样例：输出 5。

【传参要求（照这个传不会错）】

- `prev`: 上一层 DP 数组（`vector<i64>`，下标 `0..n`）；`cur`: 当前层输出，长度与 `prev` 相同，`cur[0]` 需外部初始化（求最小为 0，求最大为 `-INF`）。
- `cost(1, r)`: 闭区间 `[1, r]` 的组代价，返回 `i64`；内部按 `cost(k+1, mid)` 调用。
- 入口: `compute_dc(1, n, 0, n - 1, prev, cur, cost)`，每层分组调用一次后 `prev.swap(cur)`。
- 前提: 最优决策点随 `i` 单调不降，否则答案错误。

```

// 维护的量: prev（上一层 DP，只读）；cur（当前层，递归中逐段填）；opt_l/opt_r（决策点允许区间）；
// mid/opt（段中点与中点处的最优决策点）。
// 不变量: 进入 compute_dc(l, r, opt_l, opt_r) 时，段内所有位置的最优决策点都落在 [opt_l, opt_r]，
// 且随下标单调不降，所以左右半段可以各自收窄决策区间。
template <class Cost>
void compute_dc(int l, int r, int opt_l, int opt_r,
               const vector<i64>& prev,
               vector<i64>& cur,
               Cost cost) {
    if (l > r) return;
    int mid = (l + r) >> 1;
    pair<i64, int> best = {(1LL << 60), -1};
    // 只扫 [opt_l, min(mid-1, opt_r)]：决策单调让每层总扫描量只有 O(n)
    for (int k = opt_l; k <= min(mid - 1, opt_r); ++k) {
        best = min(best, {prev[k] + cost(k + 1, mid), k});
    }
    cur[mid] = best.first;
    int opt = best.second;
    compute_dc(l, mid - 1, opt_l, opt, prev, cur, cost); // 左半段决策点不超过 opt
    compute_dc(mid + 1, r, opt, opt_r, prev, cur, cost); // 右半段决策点不小于 opt
}

```

SMAWK：全单调矩阵每行最优值

赛时先看

题目信号: `opt[i]` 随行号不下降；转移能看成矩阵 `A[i][j]` 的每行最小值；题目进一步给出 Monge 性/四边形不等式/凸代价，或要求 `min-plus/max-plus` 卷积；分治优化的 $O(n \log n)$ 仍不够。

本质: 不显式建出矩阵，在一个全单调（totally monotone）矩阵中求每一行的最优列。它是“决策单调 DP”的线性级别工具，适合 `n*m` 已经无法枚举、但可以 $O(1)$ 比较两个候选的情况。

接法：把候选比较写成 `better(row, candidate, current)` 后调用 `smawk_row_minimum(n, m, better)` 获取每行最优列；典型适用场景见下方典型模型。

复杂度判定：比较次数 $O(n+m)$ ，其中矩阵有 n 行、 m 列。前提是矩阵满足全单调性；代码不验证此前提。

维护的量：`reduced`（压缩后的候选列表，长度不超过行数）；`odd_answer`（奇数行的答案，递归求得）；`answer`（每行最优列，最终全部填满）。

警告：`better(row, candidate, current)` 必须表示“candidate 严格更优”，相等时返回 `false`，这样会保留更小列号。列编号必须按升序传入。一般的“最优决策单调”不自动推出可用 SMAWK，先确认全单调/Monge 条件。

【最小完整示例（先抄这一段就能跑）】

```
// 全单调矩阵 A[i][j] = (i-j)^2 (4 行 5 列)，求每行最小值的列。
auto opt = smawk_row_minimum(4, 5, [&](int i, int a, int b) {
    return (i64)(i - a) * (i - a) < (i64)(i - b) * (i - b); // 严格更优返回 true
});
for (int x : opt) cout << x << ' '; // 0 1 2 3 (第 i 行最优列是 i)
```

• 样例：输出 0 1 2 3。

【传参要求（照这个传不会错）】

- `smawk_row_minimum(n, m, better)`: n 行、 m 列；`better(row, candidate, current)` 表示 `candidate` 列在 `row` 行严格优于 `current` 列（相等返回 `false`，会保留更小列号）；返回 `vector<int>`，第 i 个元素是第 i 行的最优列。
- 前提：矩阵必须全单调/Monge（代码不验证）；列号按升序传入。
- `smawk_rows(rows, cols, better)`: 底层函数，行列都以升序 `vector<int>` 传入；一般只用 `smawk_row_minimum`。

典型模型：Monge 数组行最小值；凸费用的 min-plus 卷积；区间 DP 或分组 DP 的更高阶优化；运输/匹配代价矩阵的批量最优决策。

```
// 维护的量: reduced (候选列压缩表, 长度不超过行数); odd_answer (奇数行答案, 递归求得);
// answer (每行最优列; 偶数行在相邻奇数行答案之间收窄范围后扫描)。
// 不变量: 每轮调用后, 任何行的最优列都仍落在 reduced 中; 奇数行答案已定, 偶数行只需
// 在左右相邻奇数行答案之间的列里扫描 (全单调性保证答案必在该区间内)。
// better(row, candidate_col, current_col) 必须表示候选列严格更优。
// rows 和 cols 都按升序排列; 返回每一行的最优列。
template <class Better>
vector<int> smawk_rows(const vector<int>& rows, const vector<int>& cols,
    const Better& better) {
    if (rows.empty()) return {};

    // 把候选列缩到不超过行数。
    vector<int> reduced;
    for (int col : cols) {
        while (!reduced.empty()) {
            int row = rows[(int)reduced.size() - 1];
            if (better(row, col, reduced.back())) reduced.pop_back(); // 新列优于旧列才淘汰
            else break;
        }
        if ((int)reduced.size() < (int)rows.size()) reduced.push_back(col);
    }

    vector<int> odd_rows;
    for (int i = 1; i < (int)rows.size(); i += 2) odd_rows.push_back(rows[i]);
    vector<int> odd_answer = smawk_rows(odd_rows, reduced, better);

    vector<int> answer(rows.size(), -1);
    for (int i = 0; i < (int)odd_answer.size(); ++i) answer[2 * i + 1] = odd_answer[i];

    // 在相邻奇数行答案之间, 只扫描必要的列范围。
    for (int i = 0; i < (int)rows.size(); i += 2) {
        int left = 0, right = (int)reduced.size() - 1;
        if (i > 0) left = (int)(lower_bound(reduced.begin(), reduced.end(), answer[i - 1]) - reduced.begin());
        if (i + 1 < (int)rows.size()) {
            right = (int)(lower_bound(reduced.begin(), reduced.end(), answer[i + 1]) - reduced.begin());
        }
        answer[i] = reduced[left];
        for (int j = left + 1; j <= right; ++j) {
            if (better(rows[i], reduced[j], answer[i])) answer[i] = reduced[j];
        }
    }
    return answer;
}

// 维护的量: rows/cols (0..n-1, 0..m-1 的行/列号列表, 供底层 smawk_rows 使用)。
// 不变量: 返回第 i 行的最优列; 比较全部交给 better。
template <class Better>
vector<int> smawk_row_minimum(int n, int m, Better better) {
    vector<int> rows(n), cols(m);
```



```

    iota(rows.begin(), rows.end(), 0);
    iota(cols.begin(), cols.end(), 0);
    return smawk_rows(rows, cols, better);
}

// 以 A[i][j] 为例的调用方式:
// 示例: auto opt = smawk_row_minimum(n, m, [&](int i, int a, int b) {
// 示例: return value(i, a) < value(i, b); // strict: ties keep the smaller column.
// 示例: });

```

WQS 二分

赛时先看

题目信号：目标是最大/最小权值，同时限制选取个数；选取个数随惩罚值单调变化。

本质：要求“恰好选 k 个”但普通 DP 多一维很慢，通过给每个选择加惩罚项把数量约束二分掉。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API

列出的接口，不要把内部函数参数直接当题面参数。

复杂度判定： $O(\log V * \text{check})$ 。

维护的量：`WqsResult{value, count}`（当前 λ 下的最优值与对应选取个数）；二分边界 `lo/hi`。

警告：`check(lambda)` 必须同时返回最优值和选取数量；最后答案要把惩罚项加/减回去。

【最小完整示例（先抄这一段就能跑）】

数组 `{3, 5, 1, 8, 2}`，恰好选 2 个互不相邻元素。

```

vector<i64> a = {3, 5, 1, 8, 2};
i64 ans = maximum_sum_exactly_k_non_adjacent(a, 2);
// 样例输出: ans = 13 (选 a[1]=5 与 a[3]=8)

```

【传参要求（照这个传不会错）】

- `maximum_sum_exactly_k_non_adjacent(a, k)`：主入口。`a` 是原数组（0-based），元素 `a[i]` 不能为 `LLONG_MIN`（断言）；`k` 为恰好选取的个数， $0 \leq k \leq (n+1)/2$ （越界断言）；返回恰好选 k 个互不相邻元素的最大和（`i64`）， $k = 0$ 时返回 0。
- `check_non_adjacent(a, lambda)`：判定器，仅供主入口内部调用。`lambda` 是每个被选元素减去的惩罚；返回 `WqsResult{value, count}`：`value` 为已减惩罚的最优值，`count` 为达到该值的选取个数（同值时偏向多选，保证数量随 λ 单调）。

【API / 入口函数（赛时只认这里列的名字）】

- `maximum_sum_exactly_k_non_adjacent(const vector<i64>& a, int k)` → 主入口，先看这个：恰好选 k 个互不相邻元素的最大和，内部二分惩罚并反复调用 `check`。返回 `i64`。
- `check_non_adjacent(const vector<i64>& a, i64 lambda)` → 给定 λ 的判定器，仅供 `maximum_sum_exactly_k_non_adjacent` 内部使用，不要从 `solve()` 直接调。返回 `WqsResult`。

```

struct WqsResult {
    i64 value; // 已减去 lambda * count 后的最优值。
    int count;
};

WqsResult wqs_better(WqsResult a, WqsResult b) {
    if (a.value != b.value) return a.value > b.value ? a : b;
    return a.count > b.count ? a : b; // 同值时偏向选得更多，保证数量关于 lambda 单调。
}

// 完整典型模型：从数组中恰好选 k 个互不相邻元素，最大化元素和。
// 对每个被选元素减 lambda，check 同时维护最优值和选中个数。
WqsResult check_non_adjacent(const vector<i64>& a, i64 lambda) {
    const i64 NEG = -(1LL << 60);
    WqsResult skip{0, 0};
    WqsResult take{NEG, INT_MIN / 2};
    for (i64 x : a) {
        WqsResult next_take{skip.value + x - lambda, skip.count + 1};
        WqsResult next_skip = wqs_better(skip, take);
        skip = next_skip;
        take = next_take;
    }
    return wqs_better(skip, take);
}

i64 maximum_sum_exactly_k_non_adjacent(const vector<i64>& a, int k) {
    int n = (int)a.size();
    assert(0 <= k && k <= (n + 1) / 2);
    if (k == 0) return 0;
}

```

```

i64 bound = 1;
for (i64 x : a) {
    assert(x != LLONG_MIN); // llabs(LLONG_MIN) 无法用 i64 表示。
    bound = max(bound, llabs(x) + 1);
}
i64 lo = -bound, hi = bound;
while (lo < hi) {
    i64 mid = lo + (hi - lo + 1) / 2;
    if (check_non_adjacent(a, mid).count >= k) lo = mid;
    else hi = mid - 1;
}
WqsResult result = check_non_adjacent(a, lo);
return result.value + lo * k; // 把恰好 k 份惩罚加回。
}

```

Slope Trick: 分段线性凸函数 DP

赛时先看

题目信号: DP 状态形如 $dp[i][x]$, x

是整数位置/高度/时间; 转移可写成对前一层取一个区间最小值; 代价是绝对值、单边折线或凸函数; 直接枚举 x 太大。

本质: 维护一元凸函数的最小值与最优区间, 特别适合不断叠加

$\max(a-x, 0)$ 、 $\max(x-a, 0)$ 、 $|x-a|$, 以及做“相邻状态变化范围限制”的 DP。

接法: 按 DP 转移逐层调用下方 API 的 `add_a_minus_x/add_x_minus_a/shift/prefix_min/suffix_min`

维护凸函数; 典型应用见下方典题模型。

复杂度判定: 每个加折线操作 $O(\log n)$, 平移和取前缀/后缀最小值 $O(1)$ (清空堆除外)。空间 $O(操作数)$ 。

维护的量: `min_f` (当前函数全局最小值); L/R (左/右折点堆, 真实折点 = 堆内值 +

`add_l/add_r`); `add_l/add_r` (两堆整体懒平移量)。

警告: `shift(a,b)` 的语义是 $g(x)=\min_{\{x-b \leq y \leq x-a\}} f(y)$, 要求 $a \leq b$ 。 `prefix_min()` 是

$g(x)=\min_{\{y \leq x\}} f(y)$, 而 `suffix_min()` 是

$g(x)=\min_{\{y \geq x\}} f(y)$, 不要写反。该模板维护的是凸的分段线性函数, 不能直接拿去处理任意函数。

约定: 本模板维护一元凸的分段线性函数, 初始代表恒为 0 的函数; 各操作的语义见下方 API 与代码前说明。

【最小完整示例 (先抄这一段就能跑)】

```

// 把序列改成非降序列的最小绝对偏差代价。
vector<i64> a = {1, 4, 3, 2, 5};
SlopeTrick st;
for (i64 x : a) {
    st.add_x_minus_a(x); // 限制当前值 >= 上一段的值
    st.prefix_min();      // 取前缀最小值, 保证结果非降
}
cout << st.min_value() << '\n'; // 2 (改成 1 3 3 3 5)

```

• 样例: 输出 2。

【传参要求 (照这个传不会错)】

- `add_a_minus_x(a)`: 加代价项 $f(x) += \max(a-x, 0)$; `add_x_minus_a(a)`: 加代价项 $f(x) += \max(x-a, 0)$; 两者连用即加 $|x-a|$ 。
- `shift(a, b)`: 转移 $g(x) = \min_{\{x-b \leq y \leq x-a\}} f(y)$, 要求 $a \leq b$ 。
- `prefix_min()`: 转移 $g(x) = \min_{\{y \leq x\}} f(y)$; `suffix_min()`: 转移 $g(x) = \min_{\{y \geq x\}} f(y)$; 不要写反。
- `min_value()`: 当前函数全局最小值; `argmin()`: 所有最优 x 构成的闭区间 (无界时端点取 $\pm INF$)。

【API / 入口函数 (赛时只认这里列的名字)】

- `add_a_minus_x(i64 a)` -> 加入代价项 $f(x) += \max(a-x, 0)$ 。
- `add_x_minus_a(i64 a)` -> 加入代价项 $f(x) += \max(x-a, 0)$ 。
- `prefix_min()` -> 转移为 $g(x)=\min_{\{y \leq x\}} f(y)$ 。
- `shift(i64 a, i64 b)` -> 转移为 $g(x)=\min_{\{x-b \leq y \leq x-a\}} f(y)$, 其中 $a \leq b$ 。
- `suffix_min()` -> 转移为 $g(x)=\min_{\{y \geq x\}} f(y)$ 。

【改板时先认这几个量】

- `add_l/add_r`: 左右两个堆各自的整体懒平移。
- `L`: 存左折点, 真实值 = `raw` + `add_l`。
- `R`: 右折点。

使用: 初始代表恒为 0 的函数。 `add_a_minus_x(a)` 加 $\max(a-x, 0)$; `add_x_minus_a(a)` 加 $\max(x-a, 0)$; 两者一起就是加 $|x-a|$ 。 `min_value()` 给函数全局最小值, `argmin()` 给所有最优 x 构成的闭区间。

典题模型：把序列改成非降/非升，代价为绝对偏差；限制相邻高度变化不超过 D 的最小代价；区间最小转移 + 单点绝对值罚分；OI Wiki Slope Trick 专题的序列修复与安全高度模型。

```
// 维护的量: min_f (当前分段线性凸函数的最小值); L/R (左/右折点堆, 真实折点 = 堆内值 + add_l/add_r);
// add_l/add_r (两个堆的整体懒平移量)。
// 不变量: 函数是凸的分段线性函数; 按真实值排序时 L 中所有折点 <= R 中所有折点,
// 全局最小值在闭区间 [top_l, top_r] 上取到, min_f 就是它的值。
struct SlopeTrick {
    static constexpr i64 INF = (1LL << 60);

    i64 min_f = 0; // 当前函数的全局最小值
    i64 add_l = 0, add_r = 0; // 两个堆的整体懒平移
    priority_queue<i64> L; // 存左折点, 真实值 = raw + add_l
    priority_queue<i64, vector<i64>, greater<i64>> R; // 右折点

    bool empty_l() const { return L.empty(); }
    bool empty_r() const { return R.empty(); }
    i64 top_l() const { return L.top() + add_l; }
    i64 top_r() const { return R.top() + add_r; }
    void push_l(i64 x) { L.push(x - add_l); }
    void push_r(i64 x) { R.push(x - add_r); }
    i64 pop_l() { i64 x = top_l(); L.pop(); return x; }
    i64 pop_r() { i64 x = top_r(); R.pop(); return x; }

    // 加入代价项 f(x) += max(a-x, 0)。
    void add_a_minus_x(i64 a) {
        if (!empty_r() && top_r() < a) {
            min_f += a - top_r();
            i64 r = pop_r();
            push_r(a);
            push_l(r); // 最小值上移 a-top_r, 原右折点转成左折点
        } else {
            push_l(a); // 最小值区间覆盖到 a: 只新增左折点, min_f 不变
        }
    }

    // 加入代价项 f(x) += max(x-a, 0)。
    void add_x_minus_a(i64 a) {
        if (!empty_l() && top_l() > a) {
            min_f += top_l() - a;
            i64 l = pop_l();
            push_l(a);
            push_r(l);
        } else {
            push_r(a);
        }
    }

    void add_abs(i64 a) { // 加入绝对值代价项 f(x) += |x-a|。
        add_a_minus_x(a);
        add_x_minus_a(a);
    }

    // 转移为 g(x)=min_{x-b<=y<=x-a} f(y), 其中 a<=b。
    void shift(i64 a, i64 b) {
        assert(a <= b);
        add_l += a; // 懒平移: 真实折点 = 堆内值 + 懒标记
        add_r += b;
    }

    // 转移为 g(x)=min_{y<=x} f(y)。
    void prefix_min() { while (!R.empty()) R.pop(); }
    // 转移为 g(x)=min_{y>=x} f(y)。
    void suffix_min() { while (!L.empty()) L.pop(); }

    i64 min_value() const { return min_f; }
    pair<i64, i64> argmin() const {
        return {empty_l() ? -INF : top_l(), empty_r() ? INF : top_r()};
    }
};
```

最大子段和与最大子矩阵

赛时先看

题目信号：连续子数组/子矩阵最大和。这里的“子段/子数组”要求连续；允许跳过元素的是子序列，不适用 Kadane。

本质：Kadane 算法（最大子段和）和二维压缩。

接法：一维直接 `max_subarray_sum(a)`；二维把矩阵丢给

`max_submatrix_sum(mat)`；全负数组/矩阵时答案取最大元素，两者都已处理。

复杂度判定：一维 $O(n)$ ，二维 $O(n^2 m)$ 。

维护的量：一维 `cur`（以当前元素结尾的最大子段和）/`best`（历史最大）；二维另加 `col[j]`（当前上下边界间第 `j` 列的列和）。

警告：全负数时答案应为最大元素，不是 0。

【最小完整示例（先抄这一段就能跑）】

```
// 一维最大子段和与二维最大子矩阵和（元素可为负，全负时取最大元素）。
vector<i64> a = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
cout << max_subarray_sum(a) << '\n'; // 6
vector<vector<i64>> mat = {
    {1, -2, 3},
    {-4, 5, 6},
    {7, -8, 9}
};
cout << max_submatrix_sum(mat) << '\n'; // 18（取整个矩阵）
```

- 样例：输出 6 与 18。

【传参要求（照这个传不会错）】

- `max_subarray_sum(a)`：a 非空（内部断言）；返回 `i64` 最大非空连续子段和；全负数时返回最大元素。
- `max_submatrix_sum(a)`：a 为非空 `vector<vector<i64>>`；返回 `i64` 最大非空子矩阵和；内部对每对上下边界跑 Kadane ($O(n^2 m)$)。
- 两者都要求“连续”，允许跳过元素的问题不适用。

```
// 维护的量：cur（以当前元素结尾的最大子段和）；best（扫描至今的最大子段和）。
// 不变量：处理完 a[0..i] 后，best 就是这段的最大非空子段和；cur < 0 时前缀被丢弃、重开一段。
i64 max_subarray_sum(const vector<i64>& a) {
    assert(!a.empty());
    i64 best = a[0], cur = 0;
    for (i64 x : a) {
        cur = max(x, cur + x); // 要么从 x 重开一段，要么接上前面
        best = max(best, cur);
    }
    return best;
}

// 维护的量：col[j]（当前 [top, bot] 行区间内第 j 列的列和）；ans（历史最大子矩阵和）。
// 不变量：固定 top/bot 后，问题退化成在 col 上求一维最大子段和。
i64 max_submatrix_sum(const vector<vector<i64>>& a) {
    assert(!a.empty() && !a[0].empty());
    int n = a.size(), m = a[0].size();
    i64 ans = LLONG_MIN;
    for (int top = 0; top < n; ++top) {
        vector<i64> col(m, 0);
        for (int bot = top; bot < n; ++bot) {
            for (int j = 0; j < m; ++j) col[j] += a[bot][j]; // 把 bot 行并入当前列和
            ans = max(ans, max_subarray_sum(col));
        }
    }
    return ans;
}
```

全 1 最大矩形

赛时先看

题目信号：网格只含 0/1，问最大全 1 子矩形。

本质：01 矩阵中最大全 1 矩形面积。

接法：把网格按 `vector<string>` 传给 `maximal_all_one_rectangle(grid)` 一行出答案；单条直方图题也可以单独用 `largest_rectangle_histogram_zero_based(h)`。

复杂度判定： $O(nm)$ 。

维护的量：`h[j]`（以当前行为底边时第 `j` 列连续 1 的高度）；`st`（高度不减的下标栈）；`ans`（扫描中维护的最大面积）。

警告：每行当作直方图底边，用单调栈。

【最小完整示例（先抄这一段就能跑）】

```
// 01 网格中最大全 1 矩形面积。
vector<string> grid = {
    "10100",
    "10111",
    "11111",
    "10010"
};
cout << maximal_all_one_rectangle(grid) << '\n'; // 6
```

- 样例：输出 6（第 3 行起第 2..4 列构成 2x3 的矩形）。

【传参要求（照这个传不会错）】

- `maximal_all_one_rectangle(grid)`: `grid` 为 `vector<string>`，每行等长、只含 '0'/'1'；返回 `int` 最大全 1 矩形面积（空网格返回 0）。
- `largest_rectangle_histogram_zero_based(h)`: `h` 为 0-indexed 柱高数组 (`vector<int>`)，返回 `int` 直方图最大矩形面积；单独柱状图题也能用。
- `maximal_all_one_rectangle` 内部调用 `largest_rectangle_histogram_zero_based`，两个函数一起抄。

```
// 维护的量: st (高度单调不减的下标栈); ans (当前最大矩形面积)。
// 不变量: 弹出高度 h[i] 时, 它的左右边界是栈内前一个元素与 i, 贡献面积 h[i]*(i-left-1) 即最优。
int largest_rectangle_histogram_zero_based(const vector<int>& h) {
    vector<int> st;
    int n = h.size(), ans = 0;
    for (int i = 0; i <= n; ++i) {
        int cur = (i == n ? 0 : h[i]); // 末尾补 0 强制把栈清空, 确保所有高度都被计算
        while (!st.empty() && h[st.back()] >= cur) {
            int height = h[st.back()];
            st.pop_back();
            int left = st.empty() ? -1 : st.back();
            ans = max(ans, height * (i - left - 1)); // 以 height 为高的矩形宽度
        }
        st.push_back(i);
    }
    return ans;
}

// 维护的量: h[j] (以当前行为底边时第 j 列连续 1 的高度); ans (历史最大面积)。
// 不变量: h 就是直方图高度; 逐行累加高度后套用直方图模板, 答案即最大全 1 矩形。
int maximal_all_one_rectangle(const vector<string>& grid) {
    if (grid.empty()) return 0;
    int n = grid.size(), m = grid[0].size();
    vector<int> h(m, 0);
    int ans = 0;
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) h[j] = (grid[i][j] == '1' ? h[j] + 1 : 0); // 遇 0 断高
        ans = max(ans, largest_rectangle_histogram_zero_based(h));
    }
    return ans;
}
```

G 交互、博弈、杂项与近期模型

15 交互、通信与特殊评测

2019 年以后越来越常见的交互、通信、特殊评测和新奇构造模型放在这里。先看协议和信息量，再看 checker 与构造自检。

2019–2026 XCPC 题源审查清单

赛时先看

题目信号：题面不是单纯套传统算法，而是出现“输出任意合法方案”“special judge”“construct any”“checker”“通信”“交互”“两份程序”“给出一个满足性质的对象”“评测只检查性质”等描述。

本质：把 2019 年以后区域赛、EC-Final、World Finals、CCPC

网络赛/分站/总决赛、邀请赛里容易漏进板子的模型集中记录，后续补题时按这个清单反查。

复杂度判定：这是审查清单，不是单个算法；比赛时按题面关键词跳到下面对应模板。

警告：不要把“有趣”误当“能在比赛乱搞”。正式赛只使用题面允许的输出自由度；所谓 checker 思维是为了做构造、自检和边界验证，不是攻击评测系统。

- 2019–2026 公开题源优先查 ICPC Archives、ICPC World Finals Past Problems、Codeforces Gym、QOJ/Universal Cup、XCPCIO、VJudge 题单、CCPC 官网和站点公开题解。
- 看到 `any valid output`：先写本地 checker，再写构造；先让 checker 能抓自己的错误。
- 看到交互/通信：先算信息量下界，再看查询次数/通信容量是否暗示二分、分治、编码、哈希摘要或纠错。
- 看到“几何构造 + special judge”：优先考虑标准构型、组合块拼接、低维投影、数值容差和本地验证。
- 看到“题面形式怪，但数据很小”：考虑打表、搜索构造、SAT/2-SAT、最大流可行性、随机化找方案。

参考来源：ICPC Archives、ICPC World Finals Past Problems、XCPCIO、VJudge 2018–2024/2025
 题单整理、Codeforces Gym 104022、USACO Guide Interactive and Communication Problems。

特殊评测与 checker 思维：输出任意合法方案

赛时先看

题目信号：题面说“如果有多种答案输出任意一种”“It can be shown that a solution exists”“special judge”“print any valid arrangement/model/list”。

本质：处理构造题、几何输出题、方案恢复题和特殊评测题。核心不是猜 checker

漏洞，而是把题面性质翻译成本地检查器，保证自己输出真的满足条件。

复杂度判定：取决于本地 checker，常见 $O(n^2)$ 或 $O(m \log n)$ ；只在本地调试使用，不提交或只保留轻量断言。

警告：赛场上不要利用未公开系统漏洞；允许利用的是题面给出的自由度。浮点 special judge

要严格按题面容差验证，整数构造要检查范围、重复、连通性、度数、计数。

- 第一步：把输出约束逐条列成 `check_*` 函数。
- 第二步：对样例输出、手造边界、随机小数据跑 checker。
- 第三步：构造程序每输出一个方案前，在本地 `#ifdef LOCAL` 调 checker。
- 第四步：如果 checker 比构造还难，先写暴力枚举小规模和 checker 互相对拍。

本地 checker 骨架：构造题输出自检

赛时先看

题目信号：构造题 WA 但不知道哪条约束错；题面输出不是唯一答案；样例输出只是一个可行方案。

本质：为“输出任意合法方案”写本地验证器。可以检查整数、排列、范围、不重复、图边、几何距离等约束。

接法：把自己的输出保存成字符串或文件，用 `LocalChecker ck(out)` 读取；每读一个值都用 `read_int/read_double`，随后 `require` 检查性质。提交前可以用 `#ifdef LOCAL` 包住。

复杂度判定：按检查逻辑而定；本骨架读输出文本并提供通用断言。

维护的量：`ss`（剩余待读的输出流）；`errors`（已收集的违规信息，`finish` 时统一打印）。

警告：本地 checker 的容差和题面必须一致；下标、范围、重复、输出行数要先检查，再检查高级性质。

【最小完整示例（先抄这一段就能跑）】

题目：构造题输出 n 个数的排列，把程序生成的输出字符串交给 checker 逐条检查。

```
LocalChecker ck(out);           // 1. 用程序构造出的输出文本初始化
for (int i = 0; i < n; ++i) {   // 2. 逐个读入并核对范围
    ck.read_int(1, n, "p[i]");  // 值必须在 [1, n]
}
ck.require(cnt == n, "数量不对"); // 3. 按题面性质任意加 require
ck.finish();                    // 4. 收尾：查多余 token、汇总报错
```

样例：输出 3 1 2 通过；输出 3 3 2 报错“排列中元素重复或缺失”。

【传参要求（照这个传不会错）】

- `LocalChecker ck(out)`：`out` = 待自检的完整输出字符串（包含所有行）。
- `read_int(low, high, name)`：读一个整数；`low/high` = 允许闭区间；`name` 只用于报错文案；缺值或越界会记录错误并返回 `low`。
- `read_double(low, high, name)`：同上，读浮点数。
- `read_permutation(n, low = 1)`：读 n 个数，须是 $[low, low+n-1]$ 的一个排列，返回 `vector<int>`。
- `require(ok, message)`：`ok` 为 `false` 时把 `message` 记入错误。
- `finish()`：最后必调；流里还有多余 token 或有任何错误则 `assert(false)` 并打印全部错误。

```
struct LocalChecker {
    stringstream ss;
    vector<string> errors;

    LocalChecker(const string& output) : ss(output) {}

    void require(bool ok, const string& message) {
        if (!ok) errors.push_back(message);
    }

    long long read_int(long long low, long long high, const string& name) {
        long long x;
        if (!(ss >> x)) {
            errors.push_back("缺少整数: " + name);
        }
    }
};
```



```

        return low;
    }
    require(low <= x && x <= high, name + " 超出范围");
    return x;
}

double read_double(double low, double high, const string& name) {
    double x;
    if (!(ss >> x)) {
        errors.push_back("缺少浮点数: " + name);
        return low;
    }
    require(low <= x && x <= high, name + " 超出范围");
    return x;
}

vector<int> read_permutation(int n, int low = 1) {
    vector<int> p(n), seen(n, 0);
    for (int i = 0; i < n; ++i) {
        p[i] = (int)read_int(low, low + n - 1, "permutation item");
        int id = p[i] - low;
        if (0 <= id && id < n) seen[id]++;
    }
    for (int i = 0; i < n; ++i) require(seen[i] == 1, "排列中元素重复或缺失");
    return p;
}

void finish() {
    string extra;
    if (ss >> extra) errors.push_back("输出末尾有多余 token: " + extra);
    if (!errors.empty()) {
        for (auto& e : errors) cerr << "[checker] " << e << '\n';
        assert(false);
    }
}
};

```

典题记录：2020 ICPC 银川 H Absolute Space

赛时先看

题目信号：题面说“create any possible model”，输出浮点坐标，checker 只检查距离关系和范围。

本质：记录一种非常典型的“构造对象 + special judge 检查性质”的题型。题目要求给定 $n \leq 10$ ，输出不超过 100 个三维点，使每个点恰好有 n 个距离为 1 的邻点，且任意两点距离至少 0.01。

接法：先把构造出的点扔进 `check_absolute_space(points, n)`；如果是打表构造，每个 n 的表都跑一遍；如果是随机/搜索构造，找到方案后固定输出，别把随机留到提交里。

复杂度判定：本地验证 $O(m^2)$ ，其中 $m \leq 100$ 。

维护的量： p （构造出的点集，长度 m ）；`need_degree`（每个点应有的距离 1 邻点数）。

警告：不要用肉眼看图当证明；必须按题面容差检查“距离在 (0.999999, 1.000001) 的邻点数”。浮点输出位数要足够，点间最小距离限制也要检查。

【最小完整示例（先抄这一段就能跑）】

题目：构造 ≤ 100 个三维点（坐标 $[-100, 100]$ ），使每个点恰有 n 个距离约 1 的邻点且点距 ≥ 0.01 。

```

vector<Point3D> points = my_build(n);           // 1. 自己的构造函数产出点集
if (!check_absolute_space(points, n)) {         // 2. 本地校验
    /* 重新构造或换一组表 */
}
cout << points.size() << '\n';                 // 3. 校验通过再按题面格式输出

```

样例： $n=1$ 时两个相距 1 的点通过；三点成边长为 1 的三角形时每个点度数为 2 也通过。

【传参要求（照这个传不会错）】

- `check_absolute_space(p, need_degree)`： p = 点集，长度 m 须满足 $1 \leq m \leq 100$ ；`need_degree` = 每个点应有的距离 1 邻点数。
- 校验内容：坐标均在 $[-100, 100]$ ；任意两点距离 ≥ 0.01 ；每点邻点数（距离在 (0.999999, 1.000001) 内）恰为 `need_degree`。
- 返回值：全部满足返回 `true`，任一违反返回 `false`。
- `dist3(a, b)`：返回两点欧氏距离（已开根号）。

```

struct Point3D {
    double x, y, z;
};

```



```
double dist3(Point3D a, Point3D b) {
    double dx = a.x - b.x;
    double dy = a.y - b.y;
    double dz = a.z - b.z;
    return sqrt(dx * dx + dy * dy + dz * dz);
}

bool check_absolute_space(const vector<Point3D>& p, int need_degree) {
    int m = (int)p.size();
    if (m < 1 || m > 100) return false;
    for (int i = 0; i < m; ++i) {
        if (p[i].x < -100 || p[i].x > 100) return false;
        if (p[i].y < -100 || p[i].y > 100) return false;
        if (p[i].z < -100 || p[i].z > 100) return false;
    }
    for (int i = 0; i < m; ++i) {
        int degree = 0;
        for (int j = 0; j < m; ++j) {
            if (i == j) continue;
            double d = dist3(p[i], p[j]);
            if (d < 0.01) return false;
            if (0.99999 < d && d < 1.000001) degree++;
        }
        if (degree != need_degree) return false;
    }
    return true;
}
```

多数关系构造：McGarvey 投票序列

赛时先看

题目信号：题目要求输出很多个排列；每条约束形如“a 在超过半数排列中排在 b 前”；约束之间允许成环。

本质：给定若干约束 a 必须在多数投票中战胜

b，构造一组排列作为投票，使每条指定边都满足多数关系。适合“输出若干个排列，使指定 pair 的相对顺序超过一半”的构造题。

接法：把题面约束读成 wins.push_back({a,b})，调用 mcgarvey_votes(n, wins)，输出返回的每个排列。如果题面限制票数，先检查 2*m 是否不超过限制。

复杂度判定： $O(mn)$ 生成，其中 n 是候选数，m 是指定胜负关系数。若 $n \leq 50$ ， $m \leq n(n-1)/2$ ，输出 2m 个排列很安全。

维护的量：votes（生成的票序列，每行一个排列）；wins（指定胜负的约束对列表）。

警告：边是有向关系，不要求传递；构造的票数为偶数时，“超过一半”对指定边是 2 票优势，对未指定边两个方向各一票、相互抵消。

【最小完整示例（先抄这一段就能跑）】

题目：n 个候选人，给定 m 条“a 在多数票中排在 b 前”的约束，输出若干排列作为投票。

```
vector<pair<int,int>> wins = {{1, 3}, {3, 2}}; // 1. 收集约束：1 胜 3、3 胜 2
auto votes = mcgarvey_votes(n, wins); // 2. 生成 2*m 张票
for (auto& v : votes) { // 3. 每行输出一个排列
    for (int x : v) cout << x << ' ';
    cout << '\n';
}
```

样例：n=3、约束 (1,3) 得两张票 1 3 2、2 1 3，指定边 2:0 胜出。

【传参要求（照这个传不会错）】

- mcgarvey_votes(n, wins): n = 候选人数，编号 1..n; wins = 约束列表，每个 (a,b) 表示 a 须在多数票中排在 b 前，a/b 必须在 [1,n]。
- 返回值：vector<vector<int>>，每行一个排列；总票数 = 2*m（约束为空时返回一个 1..n 的排列）。
- 若题面限票数，先确认 2*m 不超限。

```
vector<vector<int>> mcgarvey_votes(int n, const vector<pair<int, int>>& wins) {
    vector<vector<int>> votes;
    for (auto [a, b] : wins) {
        vector<int> rest;
        for (int x = 1; x <= n; ++x) {
            if (x != a && x != b) rest.push_back(x);
        }

        // 第一张票：a 在 b 前，其他人按正序放后面。
        vector<int> first;
        first.push_back(a);
        first.push_back(b);
```

```

for (int x : rest) first.push_back(x);

// 第二张票：其他人反序放前面，最后仍然保持 a 在 b 前。
// 这样除 (a,b) 外的相对贡献两两抵消。
vector<int> second;
for (int i = (int)rest.size() - 1; i >= 0; --i) second.push_back(rest[i]);
second.push_back(a);
second.push_back(b);

votes.push_back(first);
votes.push_back(second);
}
if (votes.empty()) {
    vector<int> id(n);
    iota(id.begin(), id.end(), 1);
    votes.push_back(id);
}
return votes;
}

```

镜面反射网格：状态图 + 0-1 BFS

赛时先看

题目信号：网格里有 `.`、`#`、`/`、`\`、起点和方向；问能否逃出、最少删除几个镜子、最少改几个格子。

本质：处理小球/光线在网格中按方向运动，遇到 `/`、`\`、`#` 改变方向，并且可以付出代价删除镜子或改变格子的题。

接法：把每个状态设为 (r, c, dir) ；代价 0 的自然反射走队首，代价 1 的“删除镜子后直行”走队尾；最后取所有逃出边界的最小代价。

复杂度判定：状态数 $4 \times n \times m$ ，0-1 BFS 为 $O(nm)$ 。

维护的量： $dist[r][c][d]$ （状态 $(r, c, 方向)$ 的最短代价，0-1 BFS 的核心表）； dr/dc （四个方向的位移）。

警告：状态必须包含方向；遇到边界通常直接逃出；遇到墙是反弹还是不能走，要按题面调整。若题目要求输出具体操作方案，需要在本模板基础上加前驱恢复。

【最小完整示例（先抄这一段就能跑）】

题目： $n \times m$ 网格含 `#`（墙、反弹）与 `/`、`\`（镜子），从 (sr, sc) 任意方向出发，求最少删几个镜子能逃出边界。

```

MirrorGrid01BFS solver(grid);           // 1. 传入 vector<string> 网格
int ans = solver.shortest_destroy_to_escape(sr, sc); // 2. 起点坐标 (0-indexed)
cout << (ans >= (int)1e9 ? -1 : ans) << '\n'; // 3. INF 表示逃不出去

```

样例： 2×2 全是 `.`，起点 $(0, 0)$ ：原方向直行 0 代价逃出，答案为 0。

【传参要求（照这个传不会错）】

- `MirrorGrid01BFS(grid)`：`grid` = 网格字符串数组（`vector<string>`），自动取 n = 行数、 m = 列数。
- `shortest_destroy_to_escape(sr, sc)`： sr/sc = 起点，0-indexed，范围 $[0, n-1] \times [0, m-1]$ 。
- 返回值：逃出边界所需最少删除的镜子数；无法逃出返回 `INF = 1e9`。
- 方向约定：0 上 / 1 右 / 2 下 / 3 左；遇 `#` 原路反弹代价 0，遇镜子反射代价 0、删镜直穿代价 1。

【改板时先认这几个量】

- `dist`：每个方向状态的最短代价。
- `cur`：当前状态的最短代价。

```

struct MirrorGrid01BFS {
    int n, m;
    vector<string> grid;
    const int INF = 1e9;
    int dr[4] = {-1, 0, 1, 0};
    int dc[4] = {0, 1, 0, -1};

    MirrorGrid01BFS(vector<string> grid_) : grid(grid_) {
        n = (int)grid.size();
        m = n ? (int)grid[0].size() : 0;
    }

    int reflect(int dir, char ch) const {
        if (ch == '#') return dir ^ 2;           // 整块墙：原路反弹。
        static int slash[4] = {1, 0, 3, 2};
        static int backslash[4] = {3, 2, 1, 0};
        if (ch == '/') return slash[dir];
        if (ch == '\\') return backslash[dir];
        return dir;
    }

    int shortest_destroy_to_escape(int sr, int sc) {

```

```

vector<vector<array<int, 4>>> dist(n, vector<array<int, 4>>(m));
for (int i = 0; i < n; ++i) {
    for (int j = 0; j < m; ++j) {
        dist[i][j] = {INF, INF, INF, INF};
    }
}
deque<tuple<int, int, int>> dq;
for (int d = 0; d < 4; ++d) {
    dist[sr][sc][d] = 0;
    dq.push_back({sr, sc, d});
}
int ans = INF;
while (!dq.empty()) {
    auto [r, c, d] = dq.front();
    dq.pop_front();
    int cur = dist[r][c][d];
    int nr = r + dr[d], nc = c + dc[d];
    if (nr < 0 || nr >= n || nc < 0 || nc >= m) {
        ans = min(ans, cur);
        continue;
    }
    char ch = grid[nr][nc];
    if (ch == '#') {
        int nd = reflect(d, ch);
        if (cur < dist[r][c][nd]) {
            dist[r][c][nd] = cur;
            dq.push_front({r, c, nd});
        }
    } else if (ch == '/' || ch == '\\') {
        int nd = reflect(d, ch);
        if (cur < dist[nr][nc][nd]) {
            dist[nr][nc][nd] = cur;
            dq.push_front({nr, nc, nd});
        }
    } else if (cur + 1 < dist[nr][nc][d]) {
        dist[nr][nc][d] = cur + 1;
        dq.push_back({nr, nc, d}); // 删除镜子后按原方向穿过。
    }
} else {
    if (cur < dist[nr][nc][d]) {
        dist[nr][nc][d] = cur;
        dq.push_front({nr, nc, d});
    }
}
}
return ans;
};

```

计数构造工具：三角数和矩形数贪心拆分

赛时先看

题目信号：题面给一个很大的 K ，要求构造恰好 K 个“矩形/路径/子序列/交点/逆序对”，并保证存在解。

本质：很多构造题要求输出一个图/网格/排列，使某类子结构数量恰好为 K 。常见套路是找一种独立块，其贡献是 $C(x, 2)$ 、 $C(x, 2) * C(y, 2)$ 或三角数，然后把 K 拆成若干块贡献之和。

接法：先写 `choose2_floor(k)` 找最大 x 使 $C(x, 2) \leq k$ ；如果一个块贡献 $C(h, 2) * C(w, 2)$ ，可以枚举较小维度再贪心另一维。

复杂度判定：贪心拆分一般 $O(\sqrt{K})$ 或 $O(\text{块数} * \log K)$ 。

维护的量：`blocks`（拆分出的独立块大小列表，每块贡献 $C(x, 2)$ ）； k （剩余待拆的贡献）。

警告：块和块之间必须互不影响；网格构造通常要用空行/空列隔开；输出尺寸上限要边构造边检查。

【最小完整示例（先抄这一段就能跑）】

题目：构造若干互不相交的块，使某类子结构总数为 K ；每块贡献 $C(x, 2)$ 。

```

i64 K = 11;
vector<i64> blocks = split_into_triangular_numbers(K); // 1. 拆成 {5, 2}：10+1=11
auto [h, w] = best_rectangle_block(K, 100); // 2. 矩形块：贡献 C(h, 2)*C(w, 2)
// 3. 按 blocks / (h, w) 画块，块之间用空行/空列隔开

```

样例： $K=11$ 拆成 $\{5, 2\}$ ($C(5, 2)=10$ 、 $C(2, 2)=1$ ，合计 11)。

【传参要求（照这个传不会错）】

- `choose2(x)`：返回 $C(x, 2) = x * (x - 1) / 2$ 。
- `choose2_floor(limit)`：返回最大的 x 使 $C(x, 2) \leq \text{limit}$ ($\text{limit}=0$ 时返回 1)。
- `split_into_triangular_numbers(k)`： k = 需要的总贡献（正整数）；返回若干 x ，满足 $\text{sum}(C(x, 2)) == k$ 。

- `best_rectangle_block(k, max_side)`: `k` = 总贡献、`max_side` = 允许的最大边长；返回 `{h, w}` 使 $C(h,2) \cdot C(w,2) \leq k$ 且尽量大；无可行块返回 `{0,0}`。

```
i64 choose2(i64 x) {
    return x * (x - 1) / 2;
}

i64 choose2_floor(i64 limit) {
    i64 l = 0, r = 2;
    while (choose2(r) <= limit) r <= 1;
    while (l + 1 < r) {
        i64 mid = (l + r) >> 1;
        if (choose2(mid) <= limit) l = mid;
        else r = mid;
    }
    return l;
}

vector<i64> split_into_triangular_numbers(i64 k) {
    vector<i64> blocks;
    while (k > 0) {
        i64 x = choose2_floor(k);
        blocks.push_back(x);
        k -= choose2(x);
    }
    return blocks; // 每个 x 代表一个贡献 C(x, 2) 的独立块。
}

pair<i64, i64> best_rectangle_block(i64 k, i64 max_side) {
    pair<i64, i64> best = {0, 0};
    i64 best_value = 0;
    for (i64 h = 2; h <= max_side; ++h) {
        i64 a = choose2(h);
        if (a > k) break;
        i64 w = min(max_side, choose2_floor(k / a));
        i64 value = a * choose2(w);
        if (value > best_value) {
            best_value = value;
            best = {h, w};
        }
    }
    return best;
}
```

近年构造题自检：随机验证与边界打印

赛时先看

题目信号：构造函数返回方案；题面有范围、唯一性、连通性、数量等一堆输出约束。

本质：构造题最怕“想法对但某个边界输出坏了”。本模板提供一个轻量思路：生成方案后立即调用 checker，小规模再用随机测试跑很多次。

接法：把 `build_one_case` 和 `check_one_case` 换成题目自己的构造与检查；提交前删除或包住 `stress_construct`。

复杂度判定：本地调试使用，不提交或用 `#ifdef LOCAL` 包住。

维护的量：`rng`（随机源）；`input/output`（每个 case 的输入与方案，供 checker 验证）。

警告：随机测试要覆盖最小值、最大值、空约束、全约束、奇偶边界、浮点容差边界。

【最小完整示例（先抄这一段就能跑）】

题目：构造题本地自检——随机生成小规模输入，构造方案后立刻用 checker 验证。

```
auto build = [&](mt19937& rng) { return Case::random(rng); }; // 1. 随机造小输入
auto check = [&](Case in, Case::Output out) { return my_check(in, out); }; // 2. 校验
stress_construct(1000, build, check); // 3. 跑 1000 轮，失败即 assert
```

样例：1000 轮全过说明随机小数据下构造合法；失败时 `stderr` 打印 case 编号与输入/输出。

【传参要求（照这个传不会错）】

- `stress_construct(rounds, build_one_case, check_one_case)`: `rounds` = 随机测试轮数。
- `build_one_case(rng)`: 接收 `mt19937&`，返回输入结构体；该结构体需有 `solve()`（产出方案）和 `debug_print(ostream&)`（出错时打印）。
- `check_one_case(input, output)`: 接收输入与 `input.solve()` 的结果，返回 `bool` 表示方案是否合法。
- 失败时：`cerr` 打印“构造自检失败”与 case 编号、输入、输出，然后 `assert(false)`。
- 提交前把调用删除或包进 `#ifdef LOCAL`。

```
template <class Build, class Check>
```

```
void stress_construct(int rounds, Build build_one_case, Check check_one_case) {
    mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
    for (int tc = 1; tc <= rounds; ++tc) {
        auto input = build_one_case(rng);
        auto output = input.solve();
        if (!check_one_case(input, output)) {
            cerr << "构造自检失败, case = " << tc << '\n';
            input.debug_print(cerr);
            output.debug_print(cerr);
            assert(false);
        }
    }
}
```

交互题总策略：协议、次数与故障处理

赛时先看

题目信号：题面出现 `interactive`、`query`、`flush`、`You may ask at most Q queries`、`print ?`、`print ! answer`。

本质：给交互题建立统一检查表。交互题不是普通输入输出，程序要一边输出询问、一边读评测器回答。

复杂度判定：核心复杂度看询问次数；读到限制后先估算 $\log n$ 、 $n \log n$ 、 $\sqrt{n}$ 、 n 哪个量级能过。

警告：每次询问后必须刷新输出；读到非法回答或 `-1`

要立即退出；不要提前读不存在的输入；不要把调试输出写到标准输出。

- 查询次数约等于 $\log n$ ：优先二分、三分、分治。
- 查询次数约等于 $n \log n$ ：可能是交互排序、分治恢复排列、归并。
- 查询次数约等于 n 或 $2n$ ：考虑每个元素一次/两次信息，或者利用异或、前缀、差分。
- 回答可能有噪声：重复询问取多数，或设计纠错码。
- 本地调试：写一个模拟 `interactor`，用函数代替真实查询。

交互题通用协议：query / answer / flush

赛时先看

题目信号：题面让你输出 `? ...` 得到回答，最后输出 `! ...`。

本质：封装交互题常见输出格式，减少忘记 `flush` 和查询次数超限的问题。

接法：把 `ask_int` 的输出内容改成题目要求的 `query` 格式；把 `answer` 改成题目最终答案格式。

复杂度判定：每次查询 $O(\text{输出长度} + \text{读入回答})$ 。

维护的量：`query_count`（已用询问次数）；`query_limit`（询问上限，超限即 `assert`）。

警告：`endl` 会刷新但慢，`'\n' << flush` 更明确；读到 `-1` 通常代表格式错或次数爆了，必须 `exit(0)`。

【最小完整示例（先抄这一段就能跑）】

题目：交互题每次输出 `? x y` 并读回答，最后输出 `! ans`，询问上限 100 次。

```
InteractiveIO io(100);           // 1. 传询问上限，超限自动 assert
int resp = io.ask_int({x, y});   // 2. 输出 "? x y"（内部已 flush）并读回答
io.answer(ans);                 // 3. 输出 "! ans" 并结束程序
```

样例：`ask_int({1, 2})` 输出 `? 1 2`；回答 `-1` 时程序内部直接 `exit(0)`。

【传参要求（照这个传不会错）】

- `InteractiveIO(limit = INT_MAX)`：`limit` = 询问上限；每次 `ask_int` 前断言 `query_count < query_limit`。
- `ask_int(args)`：`args` = 放进 `?` 后的整数列表；返回评测器回答的整数；读到 `-1` 或输入结束自动 `exit(0)`。
- `answer(value)`：输出 `! value`（任意可打印类型）并 `exit(0)`。
- 每次询问内部已 `flush`，不要再手动刷新。

```
struct InteractiveIO {
    int query_count = 0;
    int query_limit = INT_MAX;

    InteractiveIO(int limit = INT_MAX) : query_limit(limit) {}

    int ask_int(const vector<int>& args) {
        assert(query_count < query_limit);
        ++query_count;
        cout << "?";
        for (int x : args) cout << ' ' << x;
        cout << '\n' << flush; // 交互题必须刷新输出。

        int response;
        if (!(cin >> response)) exit(0); // 评测器结束或协议错误。
    }
};
```

```

    if (response == -1) exit(0);    // 多数交互题用 -1 表示非法询问。
    return response;
}

template <class T>
void answer(const T& value) {
    cout << "!" << value << '\n' << flush;
    exit(0);
}
};

```

交互二分：隐藏下标或阈值

赛时先看

题目信号：交互题允许你问 `? mid`，评测器回答“答案在左边还是右边”（大小/是否可行）；题面询问上限接近 `ceil(log2 n)`。

本质：隐藏对象有单调性质（答案 $\leq mid$ 是否成立随 mid 单调），用 $O(\log n)$ 次询问把答案区间对半压缩。

复杂度判定： $O(\log n)$ 次查询；若上限是 $\log_2 n$ 的两倍左右，可能是“先二分定位 + 一次确认”，检查是否真的要多问。

维护的量：二分边界 l/r ；每次询问后按回答收缩一半。

接法：把 `predicate(mid)` 换成一次真实 `ask`，含义是“答案是否 $\leq mid$ ”，然后 `interactive_first_true(n, ask)`。

警告：交互二分不要多问一次确认；边界 $[l, r]$ 写成闭区间更稳；如果回答含噪声，套多数投票（本册交互容错节）。

【最小完整示例（先抄这一段就能跑）】

题目：交互题，评测器藏了一个答案在 $[1, n]$ ，每次问 `? mid` 回答 `0/1`（`1` 表示答案 $\leq mid$ ），最多问 `ceil(log2 n)` 次。

```

auto ask = [&](int mid) {
    cout << "? " << mid << '\n' << flush;    // 1. 输出询问（必须 flush）
    int resp;
    cin >> resp;                               // 2. 读回答
    return resp == 1;                           // 含义：答案 <= mid
};
int ans = interactive_first_true(n, ask);       // 3. 调用：得到最小可行答案
cout << "!" << ans << '\n' << flush;          // 4. 输出最终答案

```

样例：答案藏为 `42`， $n = 100$ ；交互器会返回 `1` 当且仅当 $mid \geq 42$ ，程序输出 `! 42`。

【传参要求（照这个传不会错）】

- `interactive_first_true(n, predicate)`： n = 答案上界（答案范围 $[1, n]$ ）。
- `predicate(mid)`：你的询问封装，返回 `bool` = “答案是否 $\leq mid$ ”；必须单调。
- 返回值：最小的满足条件的整数，直接输出。
- 每次询问后必须 `flush`（`'\n' << flush`）；回答 `-1` 表示询问非法，立即 `exit(0)`。
- 问“最大可行值”时把判定方向反过来并自行改成上取中位数二分。

【抄板清单（照着做就行）】

- 抄哪段：`interactive_first_true` 模板函数。
- 构造：写 `auto ask = & { ... 真实询问, 返回 bool ... }`；（`bool` 含义 = “答案 $\leq mid$ ”）。
- 调用：`int ans = interactive_first_true(n, ask);`
- 取结果：`ans` 即最小可行下标；记得 `flush` 由你的询问函数负责。

【改造点（按题目改这几处）】

- 询问格式：`cout << "? " << mid << '\n' << flush;` 后 `cin >> resp;`，把回答转成 `bool`。
- 问最大值：把判定改成“答案 $\geq mid$ ”，用 `max_true` 版二分（取上中位 $(l+r+1)/2$ ）。
- 回答是 `0/1` 之外的比较：把 `resp >= mid` 之类的条件折进判定函数。
- 有噪声：对同一个 `mid` 重复问奇数遍取多数。

【核心逻辑（改代码时别破坏）】

- 判定必须单调：`ask(mid)` 为真时答案 $\leq mid$ ，否则答案 $> mid$ 。
- 闭区间 $[l, r]$ ，`l` 必须是一个“答案 $\leq l$ ”必假的下界。

【改板时先认这几个量】

- `predicate(mid)`：真实询问的封装，返回“答案是否 $\leq mid$ ”。


```
// 交互二分：在 [1, n] 里找最小的 x 使 predicate(x) 为真；predicate 单调。
// predicate(mid) 内部必须完成一次真实的交互询问（输出？并读入回答）。
template <class Predicate>
int interactive_first_true(int n, Predicate predicate) {
    int l = 1, r = n;
    while (l < r) {
        int mid = (l + r) >> 1; // 闭区间下取中，保证不会死循环。
        if (predicate(mid)) r = mid; // 答案 <= mid, 收缩右边界。
        else l = mid + 1; // 答案 > mid, 收缩左边界。
    }
    return l;
}
```

交互比较排序：用查询当比较器

赛时先看

题目信号：题面允许询问 `compare(a,b)`；询问上限接近 $n \log n$ ；最终要输出一个排列。

本质：评测器能回答两个元素的大小、优先级、祖先关系或相对顺序时，用稳定排序/归并排序恢复隐藏排列。

接法：把 `less_than(a,b)` 换成真实查询；如果回答“a 是否排在 b 前”，归并时直接用。

复杂度判定： $O(n \log n)$ 次比较询问。

维护的量：a（待排序的元素列表）；`less_than`（比较器，内部封装一次真实询问）。

警告：C++ `sort` 的比较器有过多副作用或不满足严格弱序时会出事；交互比较更推荐自己写归并排序，询问次数稳定。

【最小完整示例（先抄这一段就能跑）】

题目：交互题允许问两个元素谁在前（约 $n \log n$ 次），要求恢复隐藏排列。

```
auto less_than = [&](int x, int y) { // 1. 把真实询问封装成 bool 比较器
    cout << "? " << x << " < " << y << "\n" << flush;
    int r; cin >> r; // r==1 表示 x 排在 y 前
    return r == 1;
};
vector<int> perm = interactive_merge_sort(a, less_than); // 2. 归并排序恢复
for (int x : perm) cout << x << " "; // 3. 输出排列
```

样例： $n=4$ 、初始 $\{4,2,3,1\}$ ，按真实比较恢复为 $\{1,2,3,4\}$ 。

【传参要求（照这个传不会错）】

- `interactive_merge_sort(a, less_than)`：a = 初始元素列表（元素类型随意）；返回按 `less_than` 排好序的 `vector<int>`。
- `less_than(x, y)`：返回 `bool` = “x 应排在 y 前”；每次调用必须恰好完成一次真实询问；须是严格弱序。
- 询问次数：稳定 $O(n \log n)$ ，不会像 `std::sort` 那样最坏退化。

```
template <class Less>
vector<int> interactive_merge_sort(vector<int> a, Less less_than) {
    if ((int)a.size() <= 1) return a;
    int mid = (int)a.size() / 2;
    vector<int> left(a.begin(), a.begin() + mid);
    vector<int> right(a.begin() + mid, a.end());
    left = interactive_merge_sort(left, less_than);
    right = interactive_merge_sort(right, less_than);

    vector<int> res;
    int i = 0, j = 0;
    while (i < (int)left.size() || j < (int)right.size()) {
        if (j == (int)right.size()) res.push_back(left[i++]);
        else if (i == (int)left.size()) res.push_back(right[j++]);
        else if (less_than(left[i], right[j])) res.push_back(left[i++]);
        else res.push_back(right[j++]);
    }
    return res;
}
```

交互容错：重复询问多数投票

赛时先看

题目信号：题面出现 `probability`、`lie`、`malfunction`、`random response`、回答不一定可靠。

本质：处理评测器可能随机出错、隐藏对象可能有噪声、或题面允许若干次错误回答的交互模型。

接法：把 `ask_once` 换成一次真实查询；若返回值只有 0/1，用 `majority_binary_query`。

复杂度判定：把原查询次数乘以重复次数 `repeat`。

维护的量：`ones / cnt`（重复询问中各回答的计数，用多数/众数当最终答案）。

警告：`repeat` 必须是奇数；如果询问上限很紧，不能无脑重复，要用纠错码或贝叶斯更新。

【最小完整示例（先抄这一段就能跑）】

题目：交互回答可能随机出错，对同一问题重复问奇数遍取多数作为最终回答。

```
auto ask_once = [&]() { return ask_q() == 1 ? 1 : 0; }; // 1. 单次真实查询转 0/1
int vote = majority_binary_query(ask_once, 3); // 2. 重复 3 遍取 0/1 多数
int val = majority_value_query(ask_once, 5); // 3. 或重复 5 遍取众数
```

样例：三次回答 1,0,1 -> 多数为 1；五次回答 2,5,2,3,2 -> 众数为 2。

【传参要求（照这个传不会错）】

- majority_binary_query(ask_once, repeat = 3): ask_once() = 一次真实询问的封装，返回 0 或 1；repeat 必须是奇数；返回 0/1 多数。
- majority_value_query(ask_once, repeat = 5): ask_once() 返回任意整数值；返回 repeat 次中出现次数最多的值。
- 两个函数都不再修改你的询问封装；返回值都是 int。

```
template <class AskOnce>
int majority_binary_query(AskOnce ask_once, int repeat = 3) {
    assert(repeat % 2 == 1);
    int ones = 0;
    for (int i = 0; i < repeat; ++i) ones += ask_once();
    return ones * 2 > repeat;
}

template <class AskOnce>
int majority_value_query(AskOnce ask_once, int repeat = 5) {
    map<int, int> cnt;
    int best_value = 0, best_count = -1;
    for (int i = 0; i < repeat; ++i) {
        int x = ask_once();
        if (++cnt[x] > best_count) {
            best_count = cnt[x];
            best_value = x;
        }
    }
    return best_value;
}
```

通信题信息量估算与设计清单

赛时先看

题目信号：题面出现 Alice/Bob、encoder/decoder、send_message、receive_message、bit_set、two separate programs。

本质：通信题通常要写两个函数/两份程序：发送者看见一部分信息，接收者看见另一部分信息，通过有限通信恢复答案。

复杂度判定：先看通信容量，不先看时间复杂度。能传 B 位，就最多区分 2^B

种消息；答案空间或不确定性必须被压到这个范围内。

警告：不要把所有数据都传过去；通信顺序、编号、共享随机种子、是否有反馈通道必须按题面确认。

- 要比较/恢复数值：考虑进制分组、前缀编码、二分最高不同位。
- 要传集合：考虑 bitset、Bloom/hash 摘要、XOR 和、模数摘要。
- 有一位/少量错误：考虑奇偶校验、Hamming 码、重复码。
- 输出是交互式 API：先用普通函数模拟 Alice/Bob，再改成题面 API。

通信题 bit packing：定长整数编码

赛时先看

题目信号：每个数范围小，比如 $< 2^k$ ；要传很多个数但直接传 int 太浪费。

本质：把若干个小整数按固定位宽打包成 uint64_t 数组，适合通信容量按 bit 或 machine word 计的题。

接法：发送方 pack_fixed_width(a, bits)；接收方 unpack_fixed_width(words, n, bits)。

复杂度判定：编码/解码都是 $O(\text{元素数} \times \text{位宽} / 64)$ 。

维护的量：w（打包后的 64 位字数组）；pos/id/off（每个元素在 bit 流里的位置、所在字、字内偏移）。

警告：位宽要能覆盖最大值；跨 uint64_t 边界时要分两段写入；通信题里双方必须约定同一顺序和位宽。

【最小完整示例（先抄这一段就能跑）】

题目：Alice 把 n 个小于 2^{bits} 的数传给 Bob，通信按 64 位字计数。

```
vector<unsigned long long> a = {3, 1, 2}; // 1. 待传数据，各元素 < 2^bits
auto words = pack_fixed_width(a, 2); // 2. 编码：压成 ceil(6/64)=1 个字
```

```
auto back = unpack_fixed_width(words, 3, 2); // 3. 解码: 还原 n=3 个元素
assert(back == a); // 4. 校验一致
```

样例: $a=\{3,1,2\}$ 、 $bits=2 \rightarrow$ 1 个 word (值 39), 解回 $\{3,1,2\}$ 完全一致。

【传参要求 (照这个传不会错)】

- `pack_fixed_width(a, bits)`: a = 待编码元素 (每个须 $0 \leq a[i] < 2^{bits}$); $bits$ 范围 $[1, 63]$; 返回数组, 长度 `ceil(n*bits/64)`。
- `unpack_fixed_width(w, n, bits)`: w = `pack_fixed_width` 的输出; n = 元素个数; $bits$ 必须与编码方一致; 返回还原的 n 个元素。
- 双方必须约定同一 $bits$ 和元素顺序; 跨 64 位边界的元素由函数自动分两段读写, 无需手动处理。

```
vector<unsigned long long> pack_fixed_width(const vector<unsigned long long>& a, int bits) {
    assert(1 <= bits && bits <= 63);
    int total_bits = (int)a.size() * bits;
    vector<unsigned long long> w((total_bits + 63) / 64, 0);
    unsigned long long mask = (bits == 64 ? ~0ULL : ((1ULL << bits) - 1));
    for (int i = 0; i < (int)a.size(); ++i) {
        unsigned long long x = a[i] & mask;
        int pos = i * bits;
        int id = pos >> 6;
        int off = pos & 63;
        w[id] |= x << off;
        if (off + bits > 64) w[id + 1] |= x >> (64 - off);
    }
    return w;
}

vector<unsigned long long> unpack_fixed_width(const vector<unsigned long long>& w, int n, int bits) {
    assert(1 <= bits && bits <= 63);
    vector<unsigned long long> a(n);
    unsigned long long mask = (1ULL << bits) - 1;
    for (int i = 0; i < n; ++i) {
        int pos = i * bits;
        int id = pos >> 6;
        int off = pos & 63;
        unsigned long long x = w[id] >> off;
        if (off + bits > 64) x |= w[id + 1] << (64 - off);
        a[i] = x & mask;
    }
    return a;
}
```

通信题 XOR 翻一位传目标

赛时先看

题目信号: 有 0/1 状态数组; 允许翻转恰好一个位置; 接收者看最终状态并要恢复某个编号。

本质: 发送者只能改变一个位置/一位, 但接收者要得到目标编号 `target`。利用 XOR 自反性: $x \oplus y \oplus x = y$ 。

接法: 当前所有为 1 的位置异或和为 `cur`, 想让接收者读出 `target`, 就翻 `cur ^ target` 这个位置。

复杂度判定: $O(n)$ 。

维护的量: `bit` (0/1 状态数组); `cur` (所有为 1 位置的下标异或和)。

警告: 位置编号必须覆盖目标范围, 通常要求 n 是 2 的幂或至少大于目标上界; 如果编号从 1 开始, 要双方统一。

【最小完整示例 (先抄这一段就能跑)】

题目: n 个 0/1 位 (0-indexed), 发送者只翻一位, 接收者读最终状态里 1 位下标异或得到 `target`。

```
vector<int> bit(8, 0); bit[1] = bit[2] = 1; // 1. 初始状态, cur = 1^2 = 3
int pos = choose_flip_position(bit, 5); // 2. 应翻位置 = 3^5 = 6
bit[pos] ^= 1; // 3. 翻这一位
int got = decode_xor_message(bit); // 4. 接收者读出 5
assert(got == 5);
```

样例: $bit=\{0,1,1,0,0,0,0,0\}$ 、 $target=5 \rightarrow$ 翻位置 6, 接收者异或得 5。

【传参要求 (照这个传不会错)】

- `xor_state(bit)`: bit = 0/1 数组 (任意长度); 返回所有为 1 位置的下标异或和。
- `choose_flip_position(bit, target)`: bit = 当前状态; $target$ = 想让接收者读出的编号; 返回 `cur ^ target` 作为要翻转的位置。
- 前置条件: 必须满足 $0 \leq cur \oplus target < bit.size()$ (内部 `assert` 会检查), 所以数组长度要大于所有可能的 `target`。
- `decode_xor_message(bit_after_flip)`: 接收方读最终状态, 返回异或和即 `target`。

- 下标统一 0-indexed；若题面编号从 1 开始，双方先各自减 1。

【改板时先认这几个量】

- bit: 0/1 状态数组（每位可翻转一次）。
- cur: 当前所有 1 位的异或和。

```
int xor_state(const vector<int>& bit) {
    int x = 0;
    for (int i = 0; i < (int)bit.size(); ++i) {
        if (bit[i] x ^= i;
    }
    return x;
}

int choose_flip_position(vector<int> bit, int target) {
    int cur = xor_state(bit);
    int pos = cur ^ target;
    assert(0 <= pos && pos < (int)bit.size());
    return pos;
}

int decode_xor_message(const vector<int>& bit_after_flip) {
    return xor_state(bit_after_flip);
}
```

通信题 Hamming(7, 4)：一位纠错

赛时先看

题目信号：题面有传输错误、翻转一位、最多一位 corrupted、需要恢复原消息。

本质：每 4 位数据编码成 7 位，能纠正任意 1 位错误。适合通信/交互里“可能有一位坏掉”的小块编码。

接法：发送方对每个 4-bit 块调用 hamming74_encode；接收方先 hamming74_decode 自动修一位，再拼回原消息。

复杂度判定：每 4 位一组，0（消息长度）。

维护的量：b[1..7]（1-indexed 码字位）；syndrome（校验子，用于定位出错位）。

警告：本模板使用 1-index 位置逻辑，校验位在 1、2、4，数据位在 3、5、6、7；返回的数据低 4 位是原消息。

【最小完整示例（先抄这一段就能跑）】

题目：每 4 位消息编码成 7 位传输，允许 1 位翻转，接收端自动纠错还原。

```
int data4 = 11; // 1. 4 位数据 (0..15)
int code = hamming74_encode(data4); // 2. 发送: 11 -> 码字 85(1010101)
code ^= (1 << 3); // 3. 模拟传输中翻 1 位 (第 4 位)
int got = hamming74_decode(code); // 4. 接收: 自动修 1 位并还原
assert(got == 11);
```

样例：data4=11 -> code=85，翻任意 1 位后 decode 仍还原 11。

【传参要求（照这个传不会错）】

- hamming74_encode(data4): data4 = 4 位数据 (0..15)；返回 7 位码字（用低 7 位），按位打包成 int。
- hamming74_decode(code7): code7 = 收到的码字（可有 1 位错误）；自动定位并纠正后返回 4 位原数据 (0..15)。
- 位置逻辑 1-indexed: 校验位 1/2/4，数据位 3/5/6/7；翻转超过 1 位无法保证纠正。
- 多个 4 位块逐个调用即可，函数无内部状态。

```
int hamming74_encode(int data4) {
    vector<int> b(8, 0); // 使用 1..7。
    b[3] = (data4 >> 0) & 1;
    b[5] = (data4 >> 1) & 1;
    b[6] = (data4 >> 2) & 1;
    b[7] = (data4 >> 3) & 1;
    b[1] = b[3] ^ b[5] ^ b[7];
    b[2] = b[3] ^ b[6] ^ b[7];
    b[4] = b[5] ^ b[6] ^ b[7];
    int code = 0;
    for (int i = 1; i <= 7; ++i) code |= b[i] << (i - 1);
    return code;
}

int hamming74_decode(int code7) {
    vector<int> b(8, 0);
    for (int i = 1; i <= 7; ++i) b[i] = (code7 >> (i - 1)) & 1;
    int syndrome = 0;
    if ((b[1] ^ b[3] ^ b[5] ^ b[7]) != 0) syndrome |= 1;
    if ((b[2] ^ b[3] ^ b[6] ^ b[7]) != 0) syndrome |= 2;
    if ((b[4] ^ b[5] ^ b[6] ^ b[7]) != 0) syndrome |= 4;
```

```

if (1 <= syndrome && syndrome <= 7) b[syndrome] ^= 1;
int data4 = 0;
data4 |= b[3] << 0;
data4 |= b[5] << 1;
data4 |= b[6] << 2;
data4 |= b[7] << 3;
return data4;
}

```

通信题随机摘要：集合一致性校验

赛时先看

题目信号：Alice 和 Bob 各有一个集合/序列，不能全传，要求判断是否相同或找差异。

本质：用少量哈希摘要判断两个集合/多重集是否一致，或者定位“只差一个元素”的情况。常用于通信题、分布式校验和构造自检。

接法：双方使用同一个 `splitmix64_hash(value)` 作为元素随机权，集合摘要用 XOR；只差一个元素时，两个摘要异或可以得到差异元素的哈希。

复杂度判定： $O(n)$ 计算摘要。

维护的量：`xr`（异或摘要，只差一个元素时异或差值就是该元素的哈希）；`sum`（哈希和，区分重复元素）；`sum_sq`（哈希平方和，和相同但重复次数不同也能区分）。

警告：随机哈希有极小碰撞概率；题目要求确定性时应改用多模数哈希，只有题目允许随机化才可直接用；多重集要同时维护和维护和、平方和或计数摘要。

【最小完整示例（先抄这一段就能跑）】

题目：Alice 和 Bob 各有一个集合（元素可重复），只传摘要判断是否相同：

```

SetSketch a, b;
for (auto v : setA) a.add(v); // 1. 双方用同一个 splitmix64_hash 作为元素权
for (auto v : setB) b.add(v);
bool same = a.probably_equal(b); // 2. 调用：true = 两个集合大概率相同
// 只差一个元素时：a.xr ^ b.xr 就是差异元素的哈希，可直接查它是什么。

```

【传参要求（照这个传不会错）】

- `splitmix64_hash(x)`: `x` 是元素值（unsigned long long，整数直接传，字符/字符串先转成整数编码）；返回该元素的随机哈希。
- `add(x)`: `x` 为 unsigned long long；把 `splitmix64_hash(x)` 并入三个摘要（重复加同一个值会重复计入，天然支持多重集）。
- `probably_equal(other)`: `other` 是另一个 `SetSketch`；三个摘要（`xr`、`sum`、`sum_sq`）全部相等才返回 `true`。

【API / 入口函数（赛时只认这里列的名字）】

- `add(unsigned long long x)` -> 加入一个元素/贡献

```

unsigned long long splitmix64_hash(unsigned long long x) {
    x += 0x9e3779b97f4a7c15ULL;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
    x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
    return x ^ (x >> 31);
}

struct SetSketch {
    unsigned long long xr = 0;
    unsigned long long sum = 0;
    unsigned long long sum_sq = 0;

    void add(unsigned long long x) {
        unsigned long long h = splitmix64_hash(x);
        xr ^= h;
        sum += h;
        sum_sq += h * h;
    }

    bool probably_equal(const SetSketch& other) const {
        return xr == other.xr && sum == other.sum && sum_sq == other.sum_sq;
    }
};

```

16 博弈、随机、STL 与杂项速查

最后放博弈、随机数、日期、STL 和一些不适合塞进前面章节但比赛里常用的速查项。

SG 函数

赛时先看

题目信号：两人轮流操作，不能操作者输；局面能拆成独立子游戏。

本质：公平组合游戏，多堆独立游戏异或。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定：状态数乘转移数。

维护的量：`sg[x]`（局面 x 的 SG 值）；`nxt`（ x 的所有后继 SG 值，供 `mex` 用）。

警告：SG 为 0 是必败态；多个子游戏 SG 异或为 0 是必败。

【最小完整示例（先抄这一段就能跑）】

```
// 局面最大 n=100，每次只能取 {1,3,4} 个；sg[x]=0 必败、非 0 必胜。
vector<int> sg = sg_take_away(100, {1, 3, 4});
bool win = (sg[5] != 0); // 单堆 5 个：sg[5]=3 → 先手胜
// 多堆独立子游戏：把各堆 sg 异或，非 0 先手胜（SG 定理）。
// 样例：sg[7]=0 → 7 个石子先手必败。
```

【传参要求（照这个传不会错）】

- `mex(vals)`：传入所有后继 SG 值（`vector<int>`），返回最小的未出现非负整数（`int`）。
- `sg_take_away(max_n, moves)`：`max_n` 是最大石子数（下标 $1..max_n$ 都算，0 是必败空局）；`moves` 是每步可取数量集合（`vector<int>`，如 $\{1,3,4\}$ ）；返回 `vector<int> sg`，`sg[x] != 0` 表示石子数为 x 时先手必胜。

【改板时先认这几个量】

- `sg`：每个局面的 SG 值数组。
- `nxt`：当前局面的后继 SG 值列表。

```
int mex(vector<int> vals) {
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());
    int g = 0;
    for (int x : vals) {
        if (x == g) g++;
        else if (x > g) break;
    }
    return g;
}

vector<int> sg_take_away(int max_n, const vector<int>& moves) {
    vector<int> sg(max_n + 1);
    for (int x = 1; x <= max_n; ++x) {
        vector<int> nxt;
        for (int mv : moves) {
            if (x >= mv) nxt.push_back(sg[x - mv]);
        }
        sg[x] = mex(nxt);
    }
    return sg;
}
```

Nim 游戏

赛时先看

题目信号：若干堆，每次选一堆取任意正数，不能取者输。

本质：经典取石子游戏。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O(n)$ 。

维护的量： x （全部堆的异或和，非 0 即先手胜）；`piles` 只读不改。

警告：异或和为 0 是先手必败。

【最小完整示例（先抄这一段就能跑）】

```
// 各堆石子数按题面顺序填进数组，个数不限、每堆可到  $1e18$ 。
bool win = first_win_nim({3, 4, 5}); //  $3^4^5=2 \neq 0 \rightarrow$  先手胜
// 样例： $\{3,4,5\} \rightarrow true$ ； $\{3,4,7\} \rightarrow 3^4^7=0 \rightarrow false$ （先手必败）。
```

【传参要求（照这个传不会错）】

- `first_win_nim(piles)`: `piles` 是各堆石子数的 `vector<i64>`（每堆 $0..1e18$ 均可，堆数不限，允许 0 的空堆）；返回 `bool`: `true` 先手必胜，`false` 先手必败。

```
bool first_win_nim(const vector<i64>& piles) {
    i64 x = 0;
    for (i64 v : piles) x ^= v;
    return x != 0;
}
```

巴什博弈 / Bachet：一堆每次取 $1..k$

赛时先看

题目信号：题面只有一堆；每次至少取 1 个、最多取 k 个；没有其他限制。

本质：一堆石子有 n 个，两人轮流取 $1..k$ 个，不能取者输。最基础的取石子周期模型。

接法：如果先手必胜，第一步取 $n \% (k+1)$ 个；之后对手取 x 个，你取 $k+1-x$ 个，把局面重新送回倍数。

复杂度判定： $O(1)$ 。

维护的量：`take`（先手第一步取 $n \% (k+1)$ 个）；之后每回合保持“你取 + 对手取 = $k+1$ ”即可。

警告：正常规则下 $n \% (k+1) == 0$ 是先手必败；如果题面说“取最后一个输”，这是反常规则，不能直接套。

【最小完整示例（先抄这一段就能跑）】

```
// 一堆 n 个，每次取 1..k 个，取最后者胜；先手必胜时一步送入必败态。
bool win = first_win_bash(10, 3); // 10 % 4 = 2 != 0 -> 先手胜
i64 take = bash_first_take(10, 3); // 第一步取 2，之后对手取 x 你补 4-x
// 样例：n=10,k=3 -> win=true, take=2; n=12,k=3 -> win=false, take=0（必败）。
```

【传参要求（照这个传不会错）】

- `first_win_bash(n, k)`: n 石子总数、 k 每次最多可取数（`i64`, $k \geq 1$ ）；返回 `bool`: `true` 先手必胜。
- `bash_first_take(n, k)`: 参数同上；返回先手第一步应取数量（`i64`, 范围 $1..k$ ）；返回 0 表示当前已是必败态、没有必胜第一步。

```
bool first_win_bash(i64 n, i64 k) {
    // 正常规则：每次取 1..k，不能取者输。
    return n % (k + 1) != 0;
}

i64 bash_first_take(i64 n, i64 k) {
    // 返回先手第一步应该取多少；0 表示当前就是必败态。
    i64 take = n % (k + 1);
    return take;
}
```

Nim 必胜一步：把异或和打成 0

赛时先看

题目信号：若干堆石子；一次选一堆取任意正数；题目问胜负并要求输出操作。

本质：不仅判断 Nim 胜负，还要输出先手第一步取哪一堆、取到多少。

接法：若 `xor_sum == 0` 无必胜一步；否则调用 `nim_winning_move`，输出堆编号和要取走的数量 `old - target`。

复杂度判定： $O(n)$ 。

维护的量：`xr`（全部堆异或和）；`NimMove{index, target, take}`（改哪堆、改成几、取走几个）。

警告：找到最高位后，不是随便取；要选一堆 `a[i]` 使 $(a[i] \wedge xor_sum) < a[i]$ ，把它改成这个值。

【最小完整示例（先抄这一段就能跑）】

```
// 题目要输出“取哪一堆、取多少”时用；只判胜负用「Nim 游戏」那节。
NimMove mv = nim_winning_move({3, 4, 5}); // xr=2: 3^2=1 < 3 -> 改 a[0]
printf("取第 %d 堆，取走 %lld\n", mv.index, mv.take);
// 样例：{3,4,5} -> mv = {index=0, target=1, take=2}; {3,4,7} -> index=-1（必败）。
```

【传参要求（照这个传不会错）】

- `nim_winning_move(piles)`: `piles` 各堆石子数（`vector<i64>`）；返回 `NimMove{index, target, take}`: `index` 是 0-index 堆编号（-1 表示当前必败、无必胜一步）；`target` 是这堆应改成的新数量（异或值替换后必小于原值）；`take = 原数 - target` 是应取走的数量，直接输出即可。

```
struct NimMove {
    int index = -1; // 0-index 堆编号；-1 表示当前是必败态。
    i64 target = 0; // 把这一堆变成 target。
    i64 take = 0; // 从这一堆取走 take。
```



```
};

NimMove nim_winning_move(const vector<i64>& piles) {
    i64 xr = 0;
    for (i64 x : piles) xr ^= x;
    if (xr == 0) return {};
    for (int i = 0; i < (int)piles.size(); ++i) {
        i64 target = piles[i] ^ xr;
        if (target < piles[i]) {
            return {i, target, piles[i] - target};
        }
    }
    return {}; // 理论上不会走到这里。
}
```

反常 Nim / Misere Nim: 取最后一个输

赛时先看

题目信号: Nim 规则几乎一样, 但题面写“取最后一个的人输”或“last move loses”。

本质: 若干堆, 每次从一堆取任意正数, 但取走最后一枚石子的玩家失败。

接法: 先统计 $\text{big} = \text{count}(a[i] > 1)$ 和 $\text{ones} = \text{count}(a[i] == 1)$; $\text{big} == 0$ 走奇偶特判, 否则看总异或。

复杂度判定: $O(n)$ 。

维护的量: big (>1 的堆数)、 ones ($=1$ 的堆数)、 xr (全部堆异或和)。

警告: 只要存在大于 1 的堆, 就按普通 Nim 的异或和判断; 所有堆都为 0/1 时, 奇数个 1 是先手必败, 偶数个 1 是先手必胜。

【最小完整示例 (先抄这一段就能跑)】

```
// 取最后一个石子的人输; 其余规则与普通 Nim 相同。
bool win1 = first_win_misere_nim({1, 1, 1, 1}); // 全 1: 偶数个 → 先手胜
bool win2 = first_win_misere_nim({2, 2});      // 有 >1 的堆: 异或 2^2=0 → 败
// 样例: {1, 1, 1, 1} → true; {1, 1, 1} → false; {2, 2} → false。
```

【传参要求 (照这个传不会错)】

- `first_win_misere_nim(piles)`: `piles` 各堆石子数的 `vector<i64>` (堆数不限, 值可到 $1e18$); 返回 `bool`: `true` 先手必胜。内部自动区分“全 0/1 奇偶特判”与“存在大堆走普通异或”两种情况, 不需要手动分支。

```
bool first_win_misere_nim(const vector<i64>& piles) {
    int big = 0, ones = 0;
    i64 xr = 0;
    for (i64 x : piles) {
        xr ^= x;
        if (x > 1) big++;
        if (x == 1) ones++;
    }
    if (big == 0) return ones % 2 == 0;
    return xr != 0;
}
```

Moore's Nim-k: 一次最多操作 k 堆

赛时先看

题目信号: 题面明确“每次可以同时从不超过 k 堆中取石子”。

本质: 普通 Nim 的推广。每回合可以选择至少 1 堆、至多 k 堆, 每堆各取任意正数, 取走最后者胜。

接法: 如果任意一位计数模 $k+1$ 不为 0, 先手必胜; 全为 0 则必败。

复杂度判定: $O(n \log A)$ 。

维护的量: $\text{cnt}[b]$ (全部堆中第 b 个二进制位为 1 的个数); 胜负只看 $\text{cnt}[b] \% (k+1)$ 是否全为 0。

警告: 不是把所有堆异或; 要对每一个二进制位统计 1 的个数, 并对 $k+1$ 取模。 $k=1$ 时退化为普通 Nim。

【最小完整示例 (先抄这一段就能跑)】

```
// 每回合至少取 1 堆、至多同时操作 k 堆, 每堆可各取任意正数, 取最后者胜。
bool win = first_win_moore_nim_k({1, 2, 3}, 2); // 位计数模 3 非全 0 → 先手胜
bool lose = first_win_moore_nim_k({1, 2, 3}, 1); // k=1 退化普通 Nim: 1^2^3=0 → 败
// 样例: {1, 2, 3}, k=2 → true; {1, 2, 3}, k=1 → false。
```

【传参要求 (照这个传不会错)】

- `first_win_moore_nim_k(piles, k)`: `piles` 各堆石子数 (`vector<ui64>`, 每堆可到 $1e18$ 以上); k 每回合最多同时操作的堆数 (`int`, $1 \leq k \leq \text{堆数}$, $k=1$ 就是普通 Nim); 返回 `bool`: `true` 先手必胜。

```
bool first_win_moore_nim_k(const vector<ui64>& piles, int k) {
    vector<int> cnt(64, 0);
```



```

for (ui64 x : piles) {
    for (int b = 0; b < 64; ++b) {
        if ((x >> b) & 1ULL) cnt[b]++;
    }
    for (int b = 0; b < 64; ++b) {
        if (cnt[b] % (k + 1) != 0) return true;
    }
    return false;
}

```

阶梯 Nim: 只看奇数层异或

赛时先看

题目信号: 石子不是直接消失, 而是向前一个位置移动; 题面像“台阶”“楼梯”“从 i 移到 $i-1$ ”。

本质: 有 n 个位置, 每次可以把第 $i>1$ 堆的若干石子移到 $i-1$, 或从第 1 堆取走若干, 不能操作输。

接法: 把所有 1-index 奇数位置的石子数异或, 非 0 先手胜。

复杂度判定: $O(n)$ 。

维护的量: xr (1-index 奇数位置石子数的异或和; 0-index 版看偶数下标)。

警告: 如果位置从 1 开始, 只异或奇数位置; 如果数组从 0 开始, 奇数位置对应下标偶数。

约定: $a[1..n]$ 有效; $a[0]$ 不用。

【最小完整示例 (先抄这一段就能跑)】

```

// 1-index: a[1..n] 有效, a[0] 随便填 0 占位; 石子只能往编号小的位置移。
bool win = first_win_staircase_nim_1indexed({0, 1, 2, 3}); // 1^3=2 → 先手胜
// 0-index: 直接看偶数下标 (等价于 1-index 的奇数位)。
bool win2 = first_win_staircase_nim_0indexed({1, 2, 3}); // 1^3=2 → 先手胜
// 样例: {0, 1, 2, 3} → true; {0, 2, 0, 2} → 2^2=0 → false。

```

【传参要求 (照这个传不会错)】

- `first_win_staircase_nim_1indexed(a)`: a 长度至少 $n+1$, $a[1..n]$ 是每层石子数 (`vector<i64>`), $a[0]$ 必须存在但内容随意 (代码从下标 1 起跳); 返回 `bool`: 1-index 奇数位置异或非 0 即先手胜。
- `first_win_staircase_nim_0indexed(a)`: $a[0..n-1]$ 直接是石子数 (`vector<i64>`); 返回 `bool`: 偶数下标异或非 0 即先手胜。

```

bool first_win_staircase_nim_1indexed(const vector<i64>& a) {
    // a[1..n] 有效; a[0] 不用。
    i64 xr = 0;
    for (int i = 1; i < (int)a.size(); i += 2) xr ^= a[i];
    return xr != 0;
}

bool first_win_staircase_nim_0indexed(const vector<i64>& a) {
    i64 xr = 0;
    for (int i = 0; i < (int)a.size(); i += 2) xr ^= a[i];
    return xr != 0;
}

```

Fibonacci Nim: 最小 Zeckendorf 项

赛时先看

题目信号: 题面出现“下一次最多取上一次的两倍”“first move cannot take all”。

本质: 一堆 n 个石子。第一步不能取完; 之后每次最多取上一次对手取走数量的 2 倍, 取走最后者胜。

接法: 开局若不是 Fibonacci 数, 第一步取 `min_zeckendorf_part(n)`。这一步不会取完, 并能把局面送到必败态。

复杂度判定: $O(\log n)$ 。

维护的量: f (Zeckendorf 基 $1, 2, 3, 5, \dots$); `smallest` (n 的 Zeckendorf 分解最小项, 即开局第一步应取量)。

警告: 开局 n 是 Fibonacci 数时先手必败; 一般状态 (`remain`, `quota`) 是否必胜, 看 `quota` 是否不小于 `remain` 的 Zeckendorf 分解中最小 Fibonacci 项。

【最小完整示例 (先抄这一段就能跑)】

```

// 开局: n 个石子, 第一步不能取完, 之后最多取对手上次取量的 2 倍, 取最后者胜。
bool win = first_win_fibonacci_nim_start(10); // 10 不是 Fibonacci 数 → 先手胜
i64 take = fibonacci_nim_first_take(10); // 第一步取 Zeckendorf 最小项 2
bool mid = win_fibonacci_state(10, 3); // 中间局面 (剩余10, 上限3) → 胜
// 样例: n=10 → win=true, take=2; n=8 (Fibonacci 数) → win=false, take=0。

```

【传参要求 (照这个传不会错)】

- `fibonacci_basis(n)`: 返回 $\leq$ 相关上界的 Fibonacci 基 (`vector<i64>`, 1, 2, 3, 5, ...; 内部函数, 一般不用手动调)。
- `min_zeckendorf_part(n)`: 返回 n 的 Zeckendorf 分解中最小项 (`i64`, ≥ 1)。
- `is_fibonacci_number_for_game(n)`: 返回 `bool`, n 是否为 (该基下的) Fibonacci 数。
- `first_win_fibonacci_nim_start(n)`: 开局 n 时先手是否必胜, 返回 `bool`。
- `fibonacci_nim_first_take(n)`: 返回开局第一步应取数量 (`i64`); `0` 表示先手必败。
- `win_fibonacci_state(remain, quota)`: `remain` 剩余石子数、`quota` 当前最多可取的个数; 返回 `bool`: `quota` $\geq$ Zeckendorf 最小项 时胜。

```
vector<i64> fibonacci_basis(i64 n) {
    // 使用 1, 2, 3, 5, ..., 避免 1 重复, 适合 Zeckendorf 分解。
    vector<i64> f = {1, 2};
    while (f.back() <= n - f[(int)f.size() - 2]) {
        f.push_back(f.back() + f[(int)f.size() - 2]);
    }
    return f;
}

i64 min_zeckendorf_part(i64 n) {
    vector<i64> f = fibonacci_basis(n);
    i64 smallest = 0;
    for (int i = (int)f.size() - 1; i >= 0; --i) {
        if (f[i] <= n) {
            n -= f[i];
            smallest = f[i];
        }
    }
    return smallest;
}

bool is_fibonacci_number_for_game(i64 n) {
    vector<i64> f = fibonacci_basis(n);
    return binary_search(f.begin(), f.end(), n);
}

bool first_win_fibonacci_nim_start(i64 n) {
    return !is_fibonacci_number_for_game(n);
}

i64 fibonacci_nim_first_take(i64 n) {
    // 返回 0 表示先手必败; 否则返回第一步应取数量。
    if (!first_win_fibonacci_nim_start(n)) return 0;
    return min_zeckendorf_part(n);
}

bool win_fibonacci_state(i64 remain, i64 quota) {
    // 普通中间局面: 当前最多能取 quota 个。
    if (quota >= remain) return true;
    return quota >= min_zeckendorf_part(remain);
}
```

Wythoff 博弈: 两堆 + 黄金分割

赛时先看

题目信号: 只有两堆; 允许“取一堆”或“两堆取一样多”。

本质: 两堆石子 (a, b), 每次可以从一堆取任意正数, 或从两堆同时取相同正数, 取走最后者胜。

接法: 只判胜负时调用 `first_win_wythoff(a, b)`; 需要输出一步时调用 `wythoff_move_to_losing`, 返回的 `pair` 是取完后两堆数量。

复杂度判定: 判断 $O(\log A)$, 给出一步转移 $O(\log A)$ 。

维护的量: $d = b - a$ (两堆差); `wythoff_lower(d)` ($= \text{floor}(d \cdot \phi)$ 的整数精确值, 即必败态小堆数量)。

警告: 必败态不是异或为 0, 而是设 $a \leq b$, $d = b - a$, 满足 $a = \text{floor}(d \cdot \phi)$ 。下面用整数平方根算 `floor(d * phi)`, 避免浮点误差; 要求 $d \leq 8e18$, 足够覆盖常见 $1e18$ 级题目。

【最小完整示例 (先抄这一段就能跑)】

```
// 两堆 (a, b): 可任取一堆任意正数, 或两堆同时取相同正数, 取最后者胜。
bool win = first_win_wythoff(3, 5); // d=2, floor(2*phi)=3=a → 必败
auto p = wythoff_move_to_losing(4, 6); // 必胜态: 一步取到必败态 {3, 5}
// 样例: first_win_wythoff(3, 5)=false; wythoff_move_to_losing(4, 6) → {3, 5}。
```

【传参要求 (照这个传不会错)】

- `first_win_wythoff(a, b)`: 两堆石子数 (`ui64`, 顺序无所谓, 内部会 `swap` 成 $a \leq b$); 返回 `bool`: `true` 先手必胜。

- `wythoff_move_to_losing(a, b)`: 参数同上; 返回 `pair<ui64, ui64>`: 取完后两堆的新数量 (可能是两堆同减, 也可能只减一堆); 已处于必败态时原样返回。
- `wythoff_p_position_with_value(value, max_d)`: 找一个包含 `value` 的必败态, 返回 `pair<ui64, ui64>`; 找不到返回 `{ULLONG_MAX, ULLONG_MAX}` (内部函数, 一般不用手动调)。
- 注意: $d = b - a$ 须满足 $d \leq 8e18$ (内部有断言)。

```
using u128 = __uint128_t;

ui64 isqrt_u128(u128 x) {
    ui64 lo = 0, hi = ULLONG_MAX;
    while (lo < hi) {
        ui64 mid = lo + (hi - lo + 1) / 2;
        if ((u128)mid * mid <= x) lo = mid;
        else hi = mid - 1;
    }
    return lo;
}

ui64 wythoff_lower(ui64 d) {
    // 公式: floor(d*phi)=floor((d+sqrt(5*d*d))/2)。
    if (d == 0) return 0;
    assert(d <= 8000000000000000ULL);
    u128 x = (u128)5 * d * d;
    ui64 s = isqrt_u128(x);
    return (ui64)((u128)d + s) / 2;
}

bool first_win_wythoff(ui64 a, ui64 b) {
    if (a > b) swap(a, b);
    ui64 d = b - a;
    return a != wythoff_lower(d);
}

pair<ui64, ui64> wythoff_p_position_with_value(ui64 value, ui64 max_d) {
    // 找一个包含 value 的必败态; 没有则返回 {ULLONG_MAX, ULLONG_MAX}。
    ui64 lo = 1, hi = max_d;
    while (lo <= hi) {
        ui64 mid = lo + (hi - lo) / 2;
        ui64 x = wythoff_lower(mid);
        if (x == value) return {x, x + mid};
        if (x < value) lo = mid + 1;
        else hi = mid - 1;
    }
    lo = 1, hi = max_d;
    while (lo <= hi) {
        ui64 mid = lo + (hi - lo) / 2;
        ui64 x = wythoff_lower(mid), y = x + mid;
        if (y == value) return {x, y};
        if (y < value) lo = mid + 1;
        else hi = mid - 1;
    }
    return {ULLONG_MAX, ULLONG_MAX};
}

pair<ui64, ui64> wythoff_move_to_losing(ui64 a, ui64 b) {
    if (a > b) swap(a, b);
    ui64 d = b - a;
    ui64 x = wythoff_lower(d), y = x + d;
    if (a == x) return {a, b}; // 已经是必败态。
    if (a > x) return {x, y}; // 两堆同时取 a-x。
    auto target = wythoff_p_position_with_value(a, d - 1);
    if (target.first != ULLONG_MAX) return target; // 只减少另一堆。
    return {0, 0}; // 兜底: 正常必胜态不会需要。
}
```

Multi-SG: 一步把一个子游戏拆成多个子游戏

赛时先看

题目信号: 题面说“切一刀后左右两段继续游戏”“删掉一个点后分裂成若干块”“一个状态会变成多个独立状态”。

本质: 某个堆/区间/连通块经过一步操作后, 会拆成若干个互不影响的小游戏, 后继 SG 是这些新子游戏 SG 的异或。

接法: 把 `gen_parts(x)` 写成“状态 `x` 一步后可能变成哪些子状态列表”, 再调用这份模板。

复杂度判定: $O(\text{状态数} * \text{每个状态可拆方案数} * \text{每个方案块数})$ 。

维护的量: `sg[x]` (状态 `x` 的 SG 值); `next_sg` (每种拆法对应的子游戏异或值列表)。

警告: 后继不是 `sg[next]`, 而是所有拆出子局面的 SG 异或; 如果拆出空块, 通常忽略空块。

【最小完整示例 (先抄这一段就能跑)】

```
// 例: 一堆 x 个, 一次可把 x 拆成 (i, x-i) 两堆继续玩; gen_parts 返回所有拆法。
```

```
auto gen_parts = [](int x) {
    vector<vector<int>> parts;
    for (int i = 1; i < x; ++i) parts.push_back({i, x - i});
    return parts;
};
vector<int> sg = sg_multi_game(10, gen_parts);
// 样例：拆两堆规则下 sg[5]=0 → 5 个石子先手必败。
```

【传参要求（照这个传不会错）】

- `mex_values(vals)`: 传入后继 SG 值列表 (`vector<int>`)，返回最小的未出现非负整数 (`int`)。
- `sg_multi_game(max_state, gen_parts)`: `max_state` 最大状态编号（下标 `1..max_state` 都算，`0` 视为必败空块）；`gen_parts(x)` 是自定义函数/仿函数：入参状态 `x` (`int`)，返回 `vector<vector<int>>`，其中每个内层 `vector` 是一种拆法、元素是拆出的子状态编号 (`≤ 0` 的空块自动忽略)；返回 `vector<int> sg`，`sg[x] != 0` 先手必胜。

```
int mex_values(vector<int> vals) {
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());
    int g = 0;
    for (int x : vals) {
        if (x == g) g++;
        else if (x > g) break;
    }
    return g;
}

template <class GenParts>
vector<int> sg_multi_game(int max_state, GenParts gen_parts) {
    vector<int> sg(max_state + 1, 0);
    for (int x = 1; x <= max_state; ++x) {
        vector<int> next_sg;
        for (const vector<int>& parts : gen_parts(x)) {
            int xr = 0;
            for (int p : parts) {
                if (p > 0) xr ^= sg[p];
            }
            next_sg.push_back(xr);
        }
        sg[x] = mex_values(next_sg);
    }
    return sg;
}
```

树的删边游戏：Green Hackenbush 常用版

赛时先看

题目信号：题面是树；操作是删除一条边/切断一个子树；被切下来的部分不再参与游戏。

本质：有根树上每次删一条边，同时删掉这条边下面的整个子树；不能操作输。

接法：以地面/根为 `root`，DFS 求 `sg[u] = xor(sg[v] + 1)`；根的 SG 非 0 先手胜。

复杂度判定： $O(n)$ 。

维护的量：`g`（邻接表）；`sg[u]`（以 `u` 为根的子树删边游戏 SG 值）。

警告：每条儿子边贡献是 `sg[child] + 1`，不是单纯 `sg[child]`。多个儿子独立，整体异或。

【最小完整示例（先抄这一段就能跑）】

```
// 树上每次删一条边并丢掉下面整个子树，不能操作者输：根 SG 非 0 先手胜。
TreeCutGame tcg(3);
tcg.add_edge(1, 2);
tcg.add_edge(1, 3);
bool win = tcg.first_win(1); // 两个叶子 sg=0 → sg[1]=(0+1)^(0+1)=0 → 败
// 样例：星形 1-2、1-3 → first_win(1)=false；再连一条 2-4 → sg[1]=(1+1)^1=3 → true。
```

【传参要求（照这个传不会错）】

- `TreeCutGame(n)` / `init(n_)`: `n` 是点数（编号 `1..n`，`1-index`）；构造即初始化，清空邻接表与 `sg`。
- `add_edge(u, v)`: 加一条无向边，`u/v` 都是 `1-index` 节点编号。
- `first_win(root)`: `root` 是树根编号（默认 `1`）；内部 DFS 后返回 `bool`: `true` 表示根的 SG 非 0、先手必胜。
- `sg`（公有成员）：DFS 结束后 `sg[u]` 存每个点子树的 SG 值，需要时可以自行读取。

【API / 入口函数（赛时只认这里列的名字）】

- `add_edge(int u, int v)` → 加入一条边
- `init(int n_)` → 初始化/清空结构

【改板时先认这几个量】

- **g**: 邻接表。
- **parent**: DFS 的父节点参数。
- **root**: 树根（从哪个点开始 DFS）。

```
struct TreeCutGame {
    int n;
    vector<vector<int>> g;
    vector<int> sg;

    TreeCutGame(int n = 0) { init(n); }

    void init(int n_) {
        n = n_;
        g.assign(n + 1, {});
        sg.assign(n + 1, 0);
    }

    void add_edge(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void dfs(int u, int parent) {
        sg[u] = 0;
        for (int v : g[u]) {
            if (v == parent) continue;
            dfs(v, u);
            sg[u] ^= (sg[v] + 1);
        }
    }

    bool first_win(int root = 1) {
        dfs(root, 0);
        return sg[root] != 0;
    }
};
```

有向图游戏：胜 / 负 / 平局三态

赛时先看

题目信号: 状态图不是 DAG; 可以循环; 题面问 Win/Lose/Draw。

本质: 棋子在有向图上移动, 无法移动者输; 图可能有环, 因此除了先手胜/负, 还可能平局。

接法: 把每个局面编号成点, 边表示一步操作; 调用 `solve_directed_graph_game`, 返回 `1=Win, -1=Lose, 0=Draw`。

复杂度判定: $O(n+m)$ 。

维护的量: `state[u]` (1 胜 / -1 败 / 0 平局或未知); `degree[u]` (还有几个后继的状态未定)。

警告: 从终止状态反推。正常规则下出度为 0

是必败; 一个状态只要能走到必败就是必胜; 所有后继都是必胜才是必败; 剩下未定的是平局。

【最小完整示例（先抄这一段就能跑）】

```
// 节点 0-index, g[u] 存 u 一步能走到的点; 走不了的人输, 有环时可能平局。
vector<vector<int>> g = {{1}, {2}, {}}; // 0 → 1 → 2, 2 是死点
vector<int> state = solve_directed_graph_game(g);
// 样例: 链 0→1→2 → state = {-1, 1, -1} (0 败、1 胜、2 败)。
```

【传参要求（照这个传不会错）】

- `solve_directed_graph_game(g)`: `g` 是 `vector<vector<int>>` 邻接表, 节点编号 0-index (0..n-1), `g[u]` 存 `u` 可一步到达的节点列表 (重复边无影响); 返回 `vector<int> state`, 长度 = 节点数: `state[u] = 1` 表示先手必胜, `-1` 表示先手必败, `0` 表示平局 (总能在环里绕、走不到死点)。

```
vector<int> solve_directed_graph_game(const vector<vector<int>>& g) {
    int n = (int)g.size();
    vector<vector<int>> rg(n);
    vector<int> degree(n), state(n, 0); // 1 表示必胜, -1 表示必败, 0 表示平局或未知。
    queue<int> q;
    for (int u = 0; u < n; ++u) {
        degree[u] = (int)g[u].size();
        if (degree[u] == 0) {
            state[u] = -1;
            q.push(u);
        }
        for (int v : g[u]) rg[v].push_back(u);
    }
    while (!q.empty()) {
```

```

int v = q.front();
q.pop();
for (int p : rg[v]) {
    if (state[p] != 0) continue;
    if (state[v] == -1) {
        state[p] = 1;
        q.push(p);
    } else {
        if (--degree[p] == 0) {
            state[p] = -1;
            q.push(p);
        }
    }
}
}
return state;
}

```

二分图博弈：起点是否为最大匹配关键点

赛时先看

题目信号：题面是二分图；每步移动并删除当前点；结论和最大匹配有关。

本质：棋子在二分图上，玩家每次沿边移动到邻点，并删除刚离开的点；无路可走者输。起点若在所有最大匹配中都被匹配，先手胜，否则先手败。

接法：先建二分图，算原图最大匹配 `base`；查询起点 `v` 时，删掉 `v` 后重算，若匹配数 `< base` 则先手胜。

复杂度判定：单次查询用本模板重算一次匹配， $O(E \sqrt{V})$ ；多起点大量查询时应使用 Dulmage-Mendelsohn 分解优化。

维护的量：`g`（左部邻接表）；`m1/mr`（左右部当前匹配边）；`dist`（HK 的 BFS 分层距离）；`nL/nR`（左右部点数）。

警告：不是看“当前某个最大匹配是否匹配起点”，而是看“所有最大匹配是否都匹配起点”。等价于删掉起点后最大匹配大小是否下降。

【最小完整示例（先抄这一段就能跑）】

题目：棋子在二分图上沿边移动并删掉刚离开的点，无路可走者输；判断从起点出发先手是否必胜。

```

HopcroftKarpGame game(nL, nR);          // 左部 nL 个点、右部 nR 个点
for (int i = 0; i < m; ++i) {
    int l, r; cin >> l >> r;
    game.add_edge(l, r);                // 左 l 连右 r, 0-based
}
bool win = game.first_win_start_left(1); // 先手从左部点 1 出发
cout << (win ? "First" : "Second") << '\n';

```

样例：仅边 左0-右0，起点 左0 $\rightarrow$ `win = true`（先手胜）。

【传参要求（照这个传不会错）】

- `init(int left_size, int right_size)`: 先调用（或构造时传参），编号 `0..left_size-1 / 0..right_size-1`。
- `add_edge(int l, int r)`: `l` 范围 `[0, nL)`，`r` 范围 `[0, nR)`，0-based；每条边调一次。
- `first_win_start_left(int l)`: 起点在左部，`l` 范围 `[0, nL)`；返回 `true` 先手胜、`false` 后手胜。
- `first_win_start_right(int r)`: 起点在右部，`r` 范围 `[0, nR)`；返回 `true` 先手胜、`false` 后手胜。

【API / 入口函数（赛时只认这里列的名字）】

- `add_edge(int l, int r)` $\rightarrow$ 加入一条边
- `init(int left_size, int right_size)` $\rightarrow$ 初始化/清空结构

【改板时先认这几个量】

- `g`: 左部邻接表。
- `dist`: Hopcroft-Karp 的 BFS 距离（增广层数）。

```

struct HopcroftKarpGame {
    int nL = 0, nR = 0;
    vector<vector<int>> g;
    vector<int> dist, m1, mr;

    HopcroftKarpGame(int left_size = 0, int right_size = 0) {
        init(left_size, right_size);
    }

    void init(int left_size, int right_size) {
        nL = left_size;
        nR = right_size;
        g.assign(nL, {});
    }
}

```

```

void add_edge(int l, int r) {
    // l 的范围为 [0, nL), r 的范围为 [0, nR)。
    g[l].push_back(r);
}

bool bfs(int banned_l, int banned_r) {
    queue<int> q;
    dist.assign(nL, -1);
    for (int i = 0; i < nL; ++i) {
        if (i == banned_l) continue;
        if (ml[i] == -1) {
            dist[i] = 0;
            q.push(i);
        }
    }
    bool found = false;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : g[u]) {
            if (v == banned_r) continue;
            int to = mr[v];
            if (to == -1) found = true;
            else if (to != banned_l && dist[to] == -1) {
                dist[to] = dist[u] + 1;
                q.push(to);
            }
        }
    }
    return found;
}

bool dfs(int u, int banned_l, int banned_r) {
    for (int v : g[u]) {
        if (v == banned_r) continue;
        int to = mr[v];
        if (to == -1 || (to != banned_l && dist[to] == dist[u] + 1 && dfs(to, banned_l, banned_r))) {
            ml[u] = v;
            mr[v] = u;
            return true;
        }
    }
    dist[u] = -1;
    return false;
}

int max_matching_without(int banned_l = -1, int banned_r = -1) {
    ml.assign(nL, -1);
    mr.assign(nR, -1);
    int matching = 0;
    while (bfs(banned_l, banned_r)) {
        for (int i = 0; i < nL; ++i) {
            if (i != banned_l && ml[i] == -1 && dfs(i, banned_l, banned_r)) {
                matching++;
            }
        }
    }
    return matching;
}

bool first_win_start_left(int l) {
    int base = max_matching_without();
    return max_matching_without(l, -1) < base;
}

bool first_win_start_right(int r) {
    int base = max_matching_without();
    return max_matching_without(-1, r) < base;
}
};

```

随机数

赛时先看

题目信号：需要打乱、随机选 pivot、Pollard-Rho。

本质：随机化算法、造数据、哈希种子。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()` 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O(1)$ 。

维护的量：无额外结构；只维护全局随机引擎 `rng` (`mt19937`) / `rng64` (`mt19937_64`)，种子取自系统时钟。

警告：不要每次调用都重新构造 `rng`。

【最小完整示例（先抄这一段就能跑）】

题目：生成闭区间随机数，并原地打乱数组。

```
int x = rand_int(1, 6); // 1. [1,6] 闭区间随机整数，含两端
i64 y = rand_ll(1, 1000000000000000LL); // 2. [1,1e18] 闭区间随机 i64
shuffle(a.begin(), a.end(), rng); // 3. 原地打乱数组，直接传全局 rng
```

样例：`rand_int(1, 6)` 输出 $1 \sim 6$ 中的随机一个数；每次运行结果都不同。

【传参要求（照这个传不会错）】

- `rand_int(1, r)`：闭区间 $[1, r]$ 的随机整数，含两端；返回 `int`。
- `rand_ll(1, r)`：闭区间 $[1, r]$ 的随机整数，含两端；返回 `i64` (`r` 大到 $9e18$ 也没问题)。
- `rng` / `rng64`：全局随机引擎，可直接传给 `shuffle(a.begin(), a.end(), rng)`；全局只有一个，不要在函数里重复构造。

```
mt19937 rng((unsigned)chrono::steady_clock::now().time_since_epoch().count());
mt19937_64 rng64(chrono::steady_clock::now().time_since_epoch().count());

int rand_int(int l, int r) {
    return uniform_int_distribution<int>(l, r)(rng);
}

i64 rand_ll(i64 l, i64 r) {
    return uniform_int_distribution<i64>(l, r)(rng64);
}
```

爬山法：邻域贪心近似搜索

赛时先看

题目信号：解空间连续或离散但很大；目标函数能快速计算；题面允许近似答案（浮点、带误差、special judge），例如找平面上到若干点距离和最小的点（费马点类）。

本质：从一个初始解出发，每步在邻域内生成候选，只接受更优的解，迭代固定步数后返回最优。

接法：把 `neighbor(cur, rng)` 改成自己的扰动生成（连续解用正态/均匀小扰动，离散解用交换/翻转一位）；`score(x)` 越小越好。多随机起点各爬一遍再取全局最优，效果远好于单起点。

复杂度判定： 0 （步数 * 单次评分）。

维护的量：`best`（当前最优解，也是返回值）；`best_score`（当前最优评分，`score` 越小越好）；`rng`（随机引擎，喂给 `neighbor` 做扰动）；`climb_steps`（迭代步数，默认 10000）。

警告：会陷入局部最优；解的维度大或目标函数有很多局部极小值时先想模拟退火；评分函数必须能快速重算，否则步数开不大。

【最小完整示例（先抄这一段就能跑）】

题目：平面上 n 个点，求到它们欧氏距离和最小的点（输出坐标，允许近似）。直接抄下面的费马点写法：

```
vector<pair<double, double>> pts; // 读入 n 个点的坐标
auto score = [&](const pair<double, double>& p) {
    double s = 0;
    for (auto [x, y] : pts) s += hypot(p.first - x, p.second - y);
    return s;
};
auto neighbor = [&](const pair<double, double>& cur, mt19937& rng) {
    uniform_real_distribution<double> d(-1.0, 1.0); // 步长 1.0, 可随手调
    return pair<double, double>{cur.first + d(rng), cur.second + d(rng)};
};
auto ans = hill_climb(pts[0], score, neighbor, 50000); // 调用：爬 5e4 步
cout << fixed << setprecision(6) << ans.first << " " << ans.second << "\n";
```

【传参要求（照这个传不会错）】

- `hill_climb(start, score, neighbor, climb_steps)`：
- `start`：初始解（任意类型 `State`，结构体、`pair`、`vector` 都行）。
- `score(x)`：目标函数，返回 `double`，越小越好；只用于比较，量级无所谓。
- `neighbor(cur, rng)`：在 `cur` 附近生成一个候选解，`rng` 是 `mt19937&`，直接用它扰动，不要自己另造随机引擎。
- `climb_steps`：迭代步数，默认 10000；时限松可加大。
- 返回值：全程 `score` 最小的那个解（`State`）。

```
// 爬山法: score 越小越好; neighbor(cur, rng) 生成一个邻域候选。
// 多起点: 对若干个随机 start 各跑一次 hill_climb, 取 score 最小者。
template <class State, class Score, class Neighbor>
State hill_climb(State start, Score score, Neighbor neighbor, int climb_steps = 10000) {
    mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
    State best = start;
    double best_score = score(start);
    for (int step = 0; step < climb_steps; ++step) {
        State cand = neighbor(best, rng);
        double s = score(cand);
        if (s < best_score) {
            best_score = s;
            best = cand;
        }
    }
    return best;
}
```

模拟退火 Simulated Annealing

赛时先看

题目信号: 爬山容易卡局部最优; 题目允许近似解或答案带误差 (浮点几何最优位置、带权点集选点、排列/划分近似优化); 题面出现“误差不超过”“近似”“special judge 浮点”。

本质: 爬山的基础上, 按温度概率接受更差的解来跳出局部最优; 温度从高到低, 接受概率 $\exp(-\delta/t)$, 让搜索先“乱逛”后“收敛”。

复杂度判定: $O(\log_{\{\text{cooling}\}}(T_{\min}/T_0) * \text{单次评分})$; 评分 $O(n)$ 时把总迭代控制在 $1e5 \sim 1e6$ 级别。若题目要求精确答案 (误差 $1e-9$ 且数据特殊构造), 不要用随机近似, 先想三分/凸优化/精确公式。

维护的量: 当前解 `cur`、当前最优 `best`、当前温度 `t` (T_0 起按 `cooling` 衰减到 $T_{\min}$)。

接法: 把 `neighbor(cur, t, rng)` 改成自己的扰动生成 (扰动半径随 `t` 缩小), `score(x)` 越小越好, 然后 `simulated_annealing(start, score, neighbor, T0, T_min, cooling)`。

警告: 输出必须按题面精度格式化; 比赛固定随机种子可复现; 多随机起点各跑一次取最优, 效果远好于单起点。

【最小完整示例 (先抄这一段就能跑)】

题目: 平面 n 个点, 求到它们欧氏距离和最小的点 (输出坐标, 允许误差)。直接抄下面的费马点示例即可:

```
vector<pair<double, double>> pts;
for (int i = 0; i < n; ++i) { double x, y; cin >> x >> y; pts.push_back({x, y}); }
AnnealPoint ans = fer_mat_point_sa(pts); // 1. 调用: 返回近似最优坐标
cout << fixed << setprecision(6) << ans.x << ' ' << ans.y << '\n'; // 2. 按题面精度输出
```

【传参要求 (照这个传不会错)】

- `simulated_annealing(start, score, neighbor, T0, T_min, cooling)`: 通用模板。
- `start`: 初始解; `score(x)`: 目标函数, 越小越好。
- `neighbor(cur, t, rng)`: 生成邻域候选 (扰动幅度随温度 `t` 缩小)。
- `T0/T_min/cooling`: 起止温度与降温系数 ($0.98 \sim 0.999$)。
- 返回值: 过程中见过的最优解 (`score` 最小者)。
- `fer_mat_point_sa(pts, restarts=8)`: 费马点专用封装; `pts` 为点坐标列表。
- 输出前按题面精度 `fixed << setprecision(...)`; 多起点取最优。
- 题目要求精确答案时不要用随机近似。

【抄板清单 (照着做就行)】

- 抄哪段: `simulated_annealing` 模板函数 (需要坐标扰动示例时连 `AnnealPoint/fer_mat_point_sa` 一起抄)。
- 构造: 定义 `score(x)` (越小越好) 与 `neighbor(cur, t, rng)` (在当前解附近按温度扰动)。
- 调用: `auto best = simulated_annealing(start, score, neighbor, 10000, 1e-8, 0.999);`
- 取结果: `score(best)` 或 `best` 本身按题面输出; 多起点取最优。

【改造点 (按题目改这几处)】

- 扰动生成: 连续解用 `normal_distribution(0, t)`; 离散解用“随机交换/翻转一位”, 半径或步长随 `t` 缩小。
- 参数: `T0 = 1e3 ~ 1e5`、`T_min = 1e-8 ~ 1e-12`、`cooling = 0.98 ~ 0.999` (时限紧取小, 时限松取大)。
- 起点: 从题面给的或随机点起步; `restarts` 个起点各跑一遍取最优。
- 输出精度: `cout << fixed << setprecision(...)` 按题面位数输出。

【核心逻辑 (改代码时别破坏)】

- 接受条件: `delta <= 0` 必接受, 否则以 `exp(-delta / t)` 概率接受——这是跳出局部最优的关键。
- `best` 记录的是过程中见过的最优解, 不是最终 `cur`。

【改板时先认这几个量】

- `cur`: 当前解。
- `best`: 历史最优解 (返回值)。
- `t`: 当前温度, 控制扰动幅度与接受概率。

```
// 模拟退火: score 越小越好; 返回过程中见过的最优解。
// neighbor(cur, t, rng): 在当前温度 t 下生成邻域候选。
template <class State, class Score, class Neighbor>
State simulated_annealing(State start, Score score, Neighbor neighbor,
    double T0 = 10000, double T_min = 1e-8, double cooling = 0.999) {
    mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
    uniform_real_distribution<double> unit(0.0, 1.0);
    State cur = start, best = start;
    double cur_score = score(cur), best_score = cur_score;
    for (double t = T0; t > T_min; t *= cooling) { // 温度按几何级数下降。
        State cand = neighbor(cur, t, rng);
        double s = score(cand);
        double delta = s - cur_score;
        // 更优必接受; 更差以概率 exp(-delta/t) 接受, 温度越低越难接受。
        if (delta <= 0 || unit(rng) < exp(-delta / t)) {
            cur = cand;
            cur_score = s;
            if (s < best_score) { // 随时记录历史最优, 避免退火后期把最优解丢掉。
                best_score = s;
                best = cand;
            }
        }
    }
    return best;
}

// 典题示例: 平面 n 个点, 求到它们欧氏距离和最小的近似点 (费马点类)。
struct AnnealPoint {
    double x, y;
};

AnnealPoint fer_mat_point_sa(const vector<pair<double, double>>& pts, int restarts = 8) {
    auto score = [&](const AnnealPoint& p) { // 距离和: 越小越好。
        double s = 0;
        for (auto [x, y] : pts) {
            double dx = p.x - x, dy = p.y - y;
            s += sqrt(dx * dx + dy * dy);
        }
        return s;
    };
    auto neighbor = [&](const AnnealPoint& p, double t, auto& rng) { // 扰动半径随温度缩小。
        normal_distribution<double> nd(0.0, t);
        return AnnealPoint{p.x + nd(rng), p.y + nd(rng)};
    };
    mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
    uniform_int_distribution<i64> pick(-1000000, 1000000);
    AnnealPoint best{0, 0};
    double best_score = 1e300;
    for (int r = 0; r < restarts; ++r) { // 多随机起点取最优。
        AnnealPoint start{pick(rng) / 1e3, pick(rng) / 1e3};
        AnnealPoint got = simulated_annealing(start, score, neighbor, 1000, 1e-7, 0.998);
        double s = score(got);
        if (s < best_score) {
            best_score = s;
            best = got;
        }
    }
    return best;
}
```

典题模型: 坐标范围大的几何最优位置、带权聚类中心、近似排列优化 (配合邻域交换)。如果题面要求精确答案 (误差 $1e-9$ 且数据是特殊构造), 不要用随机近似, 先想三分、凸优化或精确公式。

归并排序求逆序对

赛时先看

题目信号: 问至少相邻交换多少次, 或逆序数量。

本质: 统计逆序对, 不依赖值域。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；`solve()` 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O(n \log n)$ 。

维护的量：`a`（正在被排序的区间，传值进入时调用方数组不受影响）；`tmp`（合并用临时数组）；`ans`（累计的逆序对数）。

警告：相等元素不算逆序时，合并条件用 `<=`。

【最小完整示例（先抄这一段就能跑）】

题目：数组 `[3, 1, 4, 2]` 有多少个逆序对（逆序对 = 前大后小的数对）。

```
vector<i64> a = {3, 1, 4, 2};           // 1. 原数组, 0-based
i64 ans = count_inversions_merge(a);    // 2. 传值调用, 返回后 a 仍保持原样
cout << ans << '\n';                 // 3. 输出逆序对数
```

样例：输出 `3`（逆序对为 `(3, 1)`、`(3, 2)`、`(4, 2)`）。

【传参要求（照这个传不会错）】

- `count_inversions_merge(a)`：传整个原数组 `vector<i64>`（值传递，原数组不会被改）；返回 `i64` 逆序对数。
- `merge_count(a, tmp, l, r)`：内部递归函数，区间是 `[l, r)` 半开（左闭右开），默认不要从 `solve()` 直接调。
- 下标从 0 开始；相等元素不算逆序（合并条件用的是 `<=`）；`n` 到 `1e6` 都能跑。

```
i64 merge_count(vector<i64>& a, vector<i64>& tmp, int l, int r) {
    if (r - l <= 1) return 0;
    int m = (l + r) >> 1;
    i64 ans = merge_count(a, tmp, l, m) + merge_count(a, tmp, m, r);
    int i = l, j = m, k = 1;
    while (i < m || j < r) {
        if (j == r || (i < m && a[i] <= a[j])) tmp[k++] = a[i++];
        else {
            tmp[k++] = a[j++];
            ans += m - i;
        }
    }
    for (int t = l; t < r; ++t) a[t] = tmp[t];
    return ans;
}

i64 count_inversions_merge(vector<i64> a) {
    vector<i64> tmp(a.size());
    return merge_count(a, tmp, 0, (int)a.size());
}
```

五张牌牌型比较：德州扑克基础评估

赛时先看

题目信号：题目给扑克牌花色和点数，要求比较五张牌；需要枚举未知牌后的最优/必胜/平局。

本质：评估五张牌牌型并比较大小，适合小规模枚举暗牌、补牌、换牌后的胜负。

接法：把每张牌转成 `(rank, suit)`，调用 `evaluate_five_cards` 得到可比较的向量；枚举双方可能暗牌后用返回值比较。

复杂度判定：单手牌评估 $O(5 \log 5)$ ；枚举暗牌按可选牌数量额外乘上常数。

维护的量：`PokerCard{rank, suit}`（每张牌的点数 2..14、花色 0..3）；`PokerScore{type, key}`（牌型级别 0..8 与同牌型比较键 `key`）。

警告：A2345 是最小顺子；同牌型比较先比主要牌，再比踢脚。题面如果有七选五，要枚举 C(7,5) 个五张组合取最大。

【最小完整示例（先抄这一段就能跑）】

题目：比较两手牌：A K Q J 9 同花 vs A2345 同花顺，谁更大。

```
vector<PokerCard> a = {{14, 0}, {13, 0}, {12, 0}, {11, 0}, {9, 0}}; // 1. 同花: A K Q J 9
vector<PokerCard> b = {{14, 1}, {2, 1}, {3, 1}, {4, 1}, {5, 1}};    // 2. A2345 同花顺 (最小顺子)
PokerScore sa = evaluate_five_cards(a);
PokerScore sb = evaluate_five_cards(b); // 3. 分别评估
cout << (sa < sb) << '\n'; // 4. type 8 同花顺 > type 5 同花
```

样例：输出 `1`（A2345 同花顺更大，牌型级别 `8 > 5`）。

【传参要求（照这个传不会错）】

- `PokerCard`：rank 点数 2..14（A 记 14、10 记 10）；suit 花色 0..3，只要能区分四种花色即可。
- `evaluate_five_cards(cards)`：传 5 张牌（`vector<PokerCard>`，顺序任意）；返回 `PokerScore`。

- `PokerScore.type`: 0=高牌、1=一对、2=两对、3=三条、4=顺子、5=同花、6=葫芦、7=四条、8=同花顺，越大越强；`key` 从主牌到踢脚降序。
- 比较：直接写 `sa < sb / sa == sb`；A2345 这种最小顺子/同花顺时 `key[0]` 是 5。
- 七选五的题：枚举 `C(7,5)` 种组合各评估一次取最优。

```
struct PokerCard {
    int rank = 0; // 点数范围 2..14, 其中 A 记作 14。
    int suit = 0; // 说明: 0..3
};

struct PokerScore {
    int type = 0;
    vector<int> key;
    bool operator<(const PokerScore& other) const {
        if (type != other.type) return type < other.type;
        return key < other.key;
    }
    bool operator==(const PokerScore& other) const {
        return type == other.type && key == other.key;
    }
};

PokerScore evaluate_five_cards(vector<PokerCard> cards) {
    sort(cards.begin(), cards.end(), [](const PokerCard& a, const PokerCard& b) {
        return a.rank < b.rank;
    });
    vector<int> ranks;
    for (auto c : cards) ranks.push_back(c.rank);

    bool flush = true;
    for (int i = 1; i < 5; ++i) flush &= cards[i].suit == cards[0].suit;

    bool straight = false;
    int straight_high = ranks[4];
    if (ranks == vector<int>{2, 3, 4, 5, 14}) {
        straight = true;
        straight_high = 5;
    } else {
        straight = true;
        for (int i = 1; i < 5; ++i) straight &= ranks[i] == ranks[i - 1] + 1;
    }
    if (straight && flush) return {8, {straight_high}};

    map<int, int, greater<int>> count;
    for (int r : ranks) count[r]++;
    vector<pair<int, int>> groups;
    for (auto [rank, cnt] : count) groups.push_back({cnt, rank});
    sort(groups.begin(), groups.end(), [](auto a, auto b) {
        if (a.first != b.first) return a.first > b.first;
        return a.second > b.second;
    });

    if (groups[0].first == 4) return {7, {groups[0].second, groups[1].second}};
    if (groups[0].first == 3 && groups[1].first == 2) return {6, {groups[0].second, groups[1].second}};
    if (flush) {
        sort(ranks.rbegin(), ranks.rend());
        return {5, ranks};
    }
    if (straight) return {4, {straight_high}};
    if (groups[0].first == 3) {
        vector<int> key{groups[0].second};
        for (auto [cnt, rank] : groups) if (cnt == 1) key.push_back(rank);
        return {3, key};
    }
    if (groups[0].first == 2 && groups[1].first == 2) {
        int high_pair = max(groups[0].second, groups[1].second);
        int low_pair = min(groups[0].second, groups[1].second);
        int kicker = groups[2].second;
        return {2, {high_pair, low_pair, kicker}};
    }
    if (groups[0].first == 2) {
        vector<int> key{groups[0].second};
        for (auto [cnt, rank] : groups) if (cnt == 1) key.push_back(rank);
        return {1, key};
    }
    sort(ranks.rbegin(), ranks.rend());
    return {0, ranks};
}
```

典题：2026 牛客多校 Round1 J 《Show

Hand》。先枚举法国赌神暗牌和自己的暗牌，用牌型评估器找出各自最优/次优暗牌，再做必胜/必败/平局分类。

Josephus 约瑟夫问题

赛时先看

题目信号：经典报数出圈。

本质：环上每次数 k 删除一个人，求最后剩下的位置。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；`solve()` 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O(n)$ 。

维护的量：`ans`（当前圈长 i 时最后幸存者的 0-based 位置），从 $i=2$ 迭代到 n 递推。

警告：这个函数返回 0-based 位置，输出 1-based 要加 1。

【最小完整示例（先抄这一段就能跑）】

题目：7 个人围成环报数，每数到 3 出圈，问最后剩下第几个人。

```
i64 n = 7, k = 3; // 1. 7 个人，每数到 3 出圈
i64 pos0 = josephus_zero_based(n, k); // 2. 返回 0-based 位置
cout << pos0 + 1 << '\n'; // 3. 输出 1-based 记得加 1
```

样例：输出 4（0-based 位置是 3，+1 后是第 4 个人）。

【传参要求（照这个传不会错）】

- `josephus_zero_based(n, k)`: n 个人 ($n \geq 1$) 围成环，每数到 k 出圈；返回最后幸存者的 0-based 位置 ($0..n-1$)。
- 时间 $O(n)$ ， n 到 $1e7$ 都能跑； n 到 $1e9$ 以上时不要用本模板，要翻快跳版 Josephus。
- 输出前想清楚题面要 0-based 还是 1-based：要 1-based 就在返回值上加 1。

```
i64 josephus_zero_based(i64 n, i64 k) {
    i64 ans = 0;
    for (i64 i = 2; i <= n; ++i) ans = (ans + k) % i;
    return ans;
}
```

日期：基姆拉尔森公式

赛时先看

题目信号：日历、星期几、日期推算。

本质：根据年月日求星期。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；优先调用下面 API 列出的入口，其余 helper 默认不要从 `solve()` 直接调。

复杂度判定： $O(1)$ 。

维护的量：无额外结构；只做一次公式换算（内部把 1、2 月转成上一年的 13、14 月再套基姆拉尔森公式）。

警告：1 月和 2 月按上一年的 13、14 月处理。

【最小完整示例（先抄这一段就能跑）】

题目：2000 年 1 月 1 日和 2026 年 8 月 12 日分别是星期几。

```
int w1 = weekday(2000, 1, 1); // 1. 直接传年月日
int w2 = weekday(2026, 8, 12); // 2. 1、2 月不用自己处理，函数内部会转
cout << w1 << ' ' << w2 << '\n'; // 3. 输出 0=Sunday, ..., 6=Saturday
```

样例：输出 6 3（2000-01-01 是星期六；2026-08-12 是星期三）。

【传参要求（照这个传不会错）】

- `weekday(y, m, d)`: y 年、 m 月 ($1..12$)、 d 日（按当月天数）；返回 $0=Sunday, 1=Monday, \dots, 6=Saturday$ 。
- 1 月和 2 月自动按上一年 13、14 月处理，调用方不要自己减年。
- 任意合理公历日期都行，无特殊下界；返回 w 一定落在 $[0, 6]$ 。

【API / 入口函数（赛时只认这里列的名字）】

- `weekday(int y, int m, int d)` → 返回 $0=Sunday, 1=Monday, \dots, 6=Saturday$ 。

```
// 返回 0=Sunday, 1=Monday, ..., 6=Saturday
int weekday(int y, int m, int d) {
    if (m == 1 || m == 2) {
        m += 12;
        y--;
    }
```



```

}
int c = y / 100;
int yy = y % 100;
int w = (yy + yy / 4 + c / 4 - 2 * c + 26 * (m + 1) / 10 + d - 1) % 7;
return (w + 7) % 7;
}

```

STL 常用函数速查

赛时先看

题目信号：排序、去重、二分、排列、累加。

本质：比赛时避免重复造轮子。

复杂度判定：按函数而定。

维护的量：无；速查表本身不建结构，注意 `unique` 必须配 `erase` 才真正改变容器长度。

警告：`unique` 只移动元素，不会改变容器长度，后面要 `erase`。

【最小完整示例（先抄这一段就能跑）】

题目：数组去重后二分查找，求第一个 ≥ 3 的下标。

```

vector<int> a = {3, 1, 2, 1, 3}; // 1. 原数组
sort(a.begin(), a.end());
a.erase(unique(a.begin(), a.end()), a.end()); // 2. 原地去重后 a = {1, 2, 3}
bool ok = binary_search(a.begin(), a.end(), 2); // 3. 二分查找，找到返回 true
int pos = lower_bound(a.begin(), a.end(), 3) - a.begin(); // 4. 第一个 >=3 的 0-based 下标

```

样例：a 变成 {1,2,3}, ok = true, pos = 2。

【传参要求（照这个传不会错）】

- `sort(a.begin(), a.end())`：升序排序；要降序加第三参 `greater<int>()`。
- `erase(unique(a.begin(), a.end()), a.end())`：原地去重，返回后容器长度才真正变小；前提是先排序。
- `binary_search(a.begin(), a.end(), x)`：x 是否出现（前提已排序），返回 `bool`。
- `lower_bound(a.begin(), a.end(), x) - a.begin()`：第一个 $\geq x$ 的 0-based 下标；`upper_bound` 是第一个 $> x$ 的下标。
- `iota(a.begin(), a.end(), 0)`：从 0 开始递增填充；`accumulate(a.begin(), a.end(), 0LL)`：求和，累加初值写 `0LL` 才保 i64。
- `next_permutation / prev_permutation`：原地换成下一个/上一个字典序排列，返回 `bool` 表示是否还有；`shuffle(a.begin(), a.end(), rng)`：按全局 `rng` 原地打乱。

【API / 入口函数（赛时只认这里列的名字）】

- `erase(unique(a.begin(), a.end()), a.end())` → 删除 `unique` 去重后末尾的多余元素，完成原地去重。

```

sort(a.begin(), a.end());
a.erase(unique(a.begin(), a.end()), a.end());

bool ok = binary_search(a.begin(), a.end(), x);
int pos1 = lower_bound(a.begin(), a.end(), x) - a.begin(); // 第一个 >= x 的位置。
int pos2 = upper_bound(a.begin(), a.end(), x) - a.begin(); // 第一个 > x 的位置。

iota(a.begin(), a.end(), 0);
i64 sum = accumulate(a.begin(), a.end(), 0LL);

next_permutation(a.begin(), a.end());
prev_permutation(a.begin(), a.end());

shuffle(a.begin(), a.end(), rng);

```

17 近期训练赛模型补充

近期多校、萌新赛和补题时抽出的短模型放在这里。它们不是独立大专题，但很适合赛后复盘和网络赛前快速扫一遍。

赛后模型总览：2026 牛客暑期多校训练营 1

赛时先看

题目信号：刚 VP / 补 2026 牛客暑期多校训练营 1，想知道每题该沉淀到哪个专题。

本质：把本场题目映射到可复用板子，补题时先从这里定位。

复杂度判定：这是索引页，不涉及算法复杂度。

警告：A/E/F/G 更偏签到、构造或观察；真正建议长期复用的是 B/C/H/I/J/K/L 的模型。

题型归档：

- A：字符串位置奇偶规则检查。
- E/F：排列差值贡献与全体加同余不变量。
- G：构造题，把二维密铺的点放到两个平面。
- B：极坐标圆弧/径向关键点建图 + Dijkstra。
- C：按权增量加入点，隐式 Kruskal 重构树维护吞并阈值。
- H：有限状态零和 MDP，值迭代到稳定后按平均收益外推大轮数。
- I：找一条“端点权不同且在同一 DSU 内”的非树边制造好坏，再 BFS 定向。
- J：枚举暗牌 + 五张牌牌型比较 + 简单 minmax 分类。
- K：简单多边形耳切三角剖分，三角形对的闵可夫斯基和并集面积。
- L：AC 自动机批量收集所有询问串出现位置，再按出现位置分段统计所有覆盖区间的最大和与总和。

牛客多校 Round1 具体模型索引

赛时先看

题目信号：补牛客暑期多校第一场但想按题号找到对应板子；或者只记得题号，不记得模型名。

本质：把 Round1 每个值得沉淀的题目指向正文中已经归类的板子。正文仍按知识点存放模板，本节只做赛后复盘索引，避免同一份代码在打印版重复出现。

复杂度判定：索引页不涉及算法复杂度。

警告：Round1 的具体算法模板已经归入对应知识点章节，本节不要再重复复制代码；真正写题时去对应模板页抄。

题号到模板：

- B：极坐标圆弧图最短路：关键角度建图。
- C：隐式 Kruskal 重构树：增量网格连通块吞并阈值。
- H：有限状态零和 MDP：值迭代差分收敛外推。
- I：非树边造好坏 + BFS 定向构造。
- J：五张牌牌型比较：德州扑克基础评估。
- K：简单多边形耳切三角剖分；三角形对闵可夫斯基和面积并可再翻“凸多边形闵可夫斯基和”和几何面积模板。
- L：AC 自动机：批量收集所有模式串出现位置；典题：子串出现覆盖的所有区间最大权和与总权和。

补题建议：先按这张表定位模板，再回题面重写“题面对象 $\rightarrow$ 模板参数”的翻译。C/L/K

是最值得复盘的三题，分别对应动态图结构、字符串出现位置统计和计算几何拆分。

赛后模型总览：2026 牛客暑期多校训练营 2

赛时先看

题目信号：只记得题号或题面关键词，不确定应该翻数学、图论、构造、DP 还是随机近似。

本质：补 Round2 时先按题号定位模型，再去下方模板或全册对应专题查实现。

复杂度判定：索引页不涉及算法复杂度。

警告：J 是重实现题，不建议赛场临时复刻；C 是分类讨论题，建议用小范围暴力打表反查边界；A/G/K 是“先找结构再压缩规模”的好题。

题号到模型：

- A 《Annoying Traffic》：网格期望等待，沿对角线转化为带吸收边界的随机游走，组合数求首次撞边贡献。
- B 《Bitwise Maximization》：两个集合异或之和最大化；总异或固定，剩余位用线性基最大异或。
- C 《Competition: Winning Streaks》：羽毛球比分连胜长度最大/最小，按普通区、终局、超分区分段讨论。
- D 《Delivering Newspapers》：固定 1 到 2 的路径，把挂树按层生长计数，标号组合 DP。
- E 《Easy Puzzle》：网格画同心矩形边，偶数短边时拆成若干矩形块。
- F 《Fabulous Tree》：边权等于点权差绝对值，树形 DP 维护根相对值域。
- G 《GCD Graph》：质数间隙很小，远处答案只分 $1/2$ ，近处用短区间 DP。
- H 《Hyperspace Pairing》：超立方删两点后按 Hamming 距离 2 配对，奇偶性判定 + 位重映射构造。
- I 《Imperfect Dot Sums and Cross Sums》：允许 10% 相对误差，按查询向量方向分组，用前缀和近似回答。
- J 《Just Round It》：随机位四舍五入期望，连续 4 进位链转生成函数系数，NTT 批量求。

- K《Kindergarten》：只有少量特殊边可变，普通边全同权；全源 BFS 后压缩到特殊端点 + 查询端点上跑小图 Dijkstra。
- L《Lazy Shuffling》：排列逆序对约束变成偏序图，状压 DP 计数拓扑序。
- M《Maybe Connected》：给定边数最大化“连通但未直接相连”点对，大连通块 + 孤点最优。
- N《Narrow to Median》：排序后选择一个长度 k 子序列替换为中位数，前缀和枚举中位位置。

两集合异或和最大：总 XOR 固定位 + 线性基

赛时先看

题目信号：每个元素二选一进入两个集合之一；两个集合 xor 记为 x, y ，必有 $x \oplus y = \text{total_xor}$ ；要求最大化 $x+y$ 。

本质：把数组分成两个集合，使两个集合的异或和之和最大。

接法：每个数先 $a[i] \&= \sim \text{total_xor}$ ，只保留可争取的位；线性基求这些数能异或出的最大值 best ，答案为 $\text{total_xor} + 2 * \text{best}$ 。

复杂度判定： $O(n \log V)$ 。

维护的量： $\text{basis}[b]$ （线性基第 b 位主元）； total_xor （全局异或，固定贡献）。

警告： total_xor 中为 1 的位对 $x+y$ 贡献固定；只有 total_xor 中为 0 的位可通过让 x, y 同时为 1 多贡献一倍。

【最小完整示例（先抄这一段就能跑）】

题目：把 $\{2, 5, 6, 7\}$ 分成两堆，最大化两堆异或和之和。

```
vector<int> a = {2, 5, 6, 7};
printf("%lld\n", max_two_subset_xor_sum(a)); // total_xor=6 固定，剩余位取线性基最大
```

样例输出：8

【传参要求（照这个传不会错）】

- a ：原数组，元素 $0..2^{30}$ ，下标 0 起，长度任意。返回：最大 $x+y$ （long long）。

【API / 入口函数（赛时只认这里列的名字）】

- $\text{insert}(\text{int } x)$ -> 把一个数插入线性基
- $\text{query_max}()$ -> 查询线性基能异或出的最大值，返回 int 。

```
struct XorBasisMax {
    static constexpr int LOG = 30;
    int basis[LOG]{};

    void clear() {
        memset(basis, 0, sizeof(basis));
    }

    void insert(int x) {
        for (int b = LOG - 1; b >= 0; --b) {
            if ((x >> b) & 1) == 0) continue;
            if (basis[b] x ^= basis[b];
            else {
                basis[b] = x;
                return;
            }
        }
    }

    int query_max() const {
        int res = 0;
        for (int b = LOG - 1; b >= 0; --b) {
            res = max(res, res ^ basis[b]);
        }
        return res;
    }
};

long long max_two_subset_xor_sum(vector<int> a) {
    int total_xor = 0;
    for (int x : a) total_xor ^= x;

    XorBasisMax lb;
    for (int x : a) lb.insert(x & ~total_xor);

    long long best = lb.query_max();
    return total_xor + 2LL * best;
}
```

典题：Round2 B 《Bitwise Maximization》。如果题目变成“分成两堆后最大化按位 OR/AND”，这套 xor 固定思路不能直接用。

逆序约束拓扑序计数：状压 DP

赛时先看

题目信号： $n \leq 22$ ；所有约束都是若干二元大小关系；两种相反方向的偏序答案一一对应。

本质：给一个排列 p ，统计有多少个排列 A 能让打乱前后逆序对差的绝对值最大。

接法：对每个逆序对 (i, j) ，固定其中一种方向，把 j 作为 i 的前置条件。 $dp[S]$ 表示已经放入集合 S 的拓扑序数量。

复杂度判定： $O(n \cdot 2^n)$ ，空间 $O(2^n)$ 。

维护的量： $need[u]$ (u 必须晚于这些元素放置的掩码)； $dp[mask]$ (已放入集合 $mask$ 的拓扑序数量)。

警告：若 p 本身没有逆序对，每个 A 都满足，答案是 $n!$ ；否则两个相反方向不重叠，最后乘 2。

【最小完整示例（先抄这一段就能跑）】

题目： $p = \{2, 1, 3\}$ ，统计打乱后使逆序对差绝对值最大的排列数（1-based 排列）。

```
printf("%d\n", count_lazy_shuffling_orders({2, 1, 3})); // 约束 1 排在 2 后，dp 计数乘 2
```

样例输出：6

【传参要求（照这个传不会错）】

- p : $1..n$ 的排列，下标 0 起， $n \leq 22$ 。返回：方案数 mod 998244353； p 已升序（无逆序对）时返回 $n!$ 。

【改板时先认这几个量】

- nxt : 转移/子节点。
- dp : DP 状态。

```
int count_lazy_shuffling_orders(const vector<int>& p) {
    const int MOD = 998244353;
    int n = (int)p.size();

    bool identity = true;
    for (int i = 0; i < n; ++i) {
        if (p[i] != i + 1) identity = false;
    }
    if (identity) {
        long long fac = 1;
        for (int i = 1; i <= n; ++i) fac = fac * i % MOD;
        return (int)fac;
    }

    vector<int> need(n, 0);
    for (int i = 0; i < n; ++i) {
        for (int j = i + 1; j < n; ++j) {
            if (p[i] > p[j]) need[i] |= 1 << j;
        }
    }

    int full = 1 << n;
    vector<int> dp(full, 0);
    dp[0] = 1;
    for (int mask = 0; mask < full; ++mask) {
        for (int u = 0; u < n; ++u) {
            if ((mask >> u) & 1) continue;
            if ((need[u] & mask) != need[u]) continue;
            int nxt = mask | (1 << u);
            dp[nxt] += dp[mask];
            if (dp[nxt] >= MOD) dp[nxt] -= MOD;
        }
    }
    return 2LL * dp[full - 1] % MOD;
}
```

典题：Round2 L 《Lazy Shuffling》。看到 $n \leq 22$ 且要数所有满足偏序关系的排列，直接往拓扑序状压 DP 想。

中位数替换最大和：排序 + 前缀和枚举中位

赛时先看

题目信号：只关心被选子序列的数值集合，不关心原下标；操作后贡献只由中位数和被替换元素原和决定。

本质：选一个长度为 k 的子序列，把这 k 个数全部替换成该子序列中位数，最大化最终数组和。

接法：排序后枚举中位位置 i 。令 $left=(k-1)/2$, $right=k-left$, 被替换原和是最小的 $left$ 个数加上从 i 开始的 $right$ 个数。

复杂度判定：排序 $O(n \log n)$, 枚举 $O(n)$ 。

维护的量： $pref[i]$ (排序后前 i 个元素前缀和); $best_delta$ (替换前后最大增量)。

警告：偶数 $k=2m$ 时中位数是中间两数平均, 替换后总和是

$m*(a[i]+a[i+1])$; 选择的较小元素应尽量小, 较大元素应贴着右中位数。

【最小完整示例（先抄这一段就能跑）】

题目: $\{1,2,3,4\}$ 选长度 $k=3$ 的子序列全部替换成中位数, 最大化总和。

```
vector<int> a = {1, 2, 3, 4};
printf("%lld\n", maximize_sum_after_median_replace(a, 3)); // 替换 {1, 3, 4} 成 {3, 3, 3} 得 11
```

样例输出: 11

【传参要求（照这个传不会错）】

- a : 原数组 (会被排序, 任意 int), 下标 0 起。 k : 替换长度, $1 \leq k \leq n$ 。返回: 替换后的最大数组和 ($long\ long$)。

```
long long maximize_sum_after_median_replace(vector<int> a, int k) {
    sort(a.begin(), a.end());
    int n = (int)a.size();

    long long total = 0;
    vector<long long> pref(n + 1, 0);
    for (int i = 0; i < n; ++i) {
        total += a[i];
        pref[i + 1] = pref[i] + a[i];
    }

    int left = (k - 1) / 2;
    int right = k - left;
    long long best_delta = LLONG_MIN;

    for (int i = left; i + right <= n; ++i) {
        long long before = pref[left] + (pref[i + right] - pref[i]);
        long long after;
        if (k & 1) after = 1LL * a[i] * k;
        else after = 1LL * (a[i] + a[i + 1]) * (k / 2);
        best_delta = max(best_delta, after - before);
    }
    return total + best_delta;
}
```

典题: Round2 N 《Narrow to

Median》。如果题目要求最小和, 把“尽量小/大”的选择方向反过来重新推, 别直接取负。

最大连通非边点对: 大块加孤点

赛时先看

题目信号：边数给定, 问最多有多少点对连通但不是相邻; 连通块大小可自由设计。

本质：在 n 个点、 m 条无重边无自环的无向图中, 最大化“同连通块但没有直接连边”的点对数。

接法： m 条边最多让 $\min(n, m+1)$ 个点连成一个块。该块内共有 $C(x, 2)$ 对点, 其中 m 对被直接连边占掉。

复杂度判定： $O(1)$ 。

维护的量：无额外结构; 只算连通块大小 $x = \min(n, m+1)$ 。

警告：最优结构是一个尽可能大的连通块加若干孤点, 不是把边平均分散到多个块里。

【最小完整示例（先抄这一段就能跑）】

题目: $n=5$ 个点、 $m=3$ 条边, 最大化“连通但未直接连边”的点对数。

```
printf("%lld\n", max_connected_non_adjacent_pairs(5, 3)); // 4 点一块用 3 条边: C(4, 2)-3
```

样例输出: 3

【传参要求（照这个传不会错）】

- n : 点数 (≥ 1)。 m : 无重边无自环的边数 ($0 \leq m \leq n*(n-1)/2$)。返回: 最大“连通但非相邻”点对数 ($long\ long$)。

```
long long max_connected_non_adjacent_pairs(long long n, long long m) {
    long long x = min(n, m + 1);
    return x * (x - 1) / 2 - m;
}
```

典题：Round2 M 《Maybe Connected》。若题目额外要求整图连通，那么 $m < n - 1$ 不可行，公式也要重推。

GCD 图最短路：质数间隙 + 容斥 + 短 DP

赛时先看

题目信号： n 到 $1e7$ ；若起点附近存在一个质数，路径最多两步；真正需要 DP 的只有 n 前面最后一个质数到 n 的短间隙。

本质：在正整数图上， $u < v$ 的边权为 $\gcd(u, v)$ ，求区间内所有起点到固定 n 的最短路和。

接法：预处理最小质因子表 spf 。 $\text{prefix_sum_to_target}(\text{target}, \text{up})$ 返回 $\sum_{x=1..up} \text{dis}(x, n)$ ，区间答案即 $\text{range_sum}(1, r, \text{target})$ 。

复杂度判定：筛 $O(M \log \log M)$ ；单次约 $O(2^{\omega(n)} + \text{gap}^2 \log n)$ ， gap 为 n 与前一个质数的距离。

维护的量： spf （最小质因子表，判质数/分解质因子）； $\text{dp}[i]$ （从 $\text{target}-i$ 到 target 的最短路）。

警告：对小于前一个质数 p 的 x ， $\text{dis}(x, n)$ 为 1 当且仅当 $\gcd(x, n)=1$ ，否则为 2；短区间 DP 要从大到小对应到 $\text{target}-i$ 。

【最小完整示例（先抄这一段就能跑）】

题目：1..10 所有点到终点 10 的最短路之和（ $u < v$ 边权 $\gcd(u, v)$ ）。

```
GcdGraphDistanceSum g;
g.init_sieve(10); // 筛到 10, 须 >= 目标与查询上界
printf("%d\n", g.range_sum(1, 10, 10)); // 质数 7 之后短间隙 DP, 其余距离只可能是 1 或 2
```

样例输出：14

【传参要求（照这个传不会错）】

- $\text{init_sieve}(\text{max_value})$ ：筛上界，须 $\geq$ 所有 target 与查询上界。
- $\text{range_sum}(l, r, \text{target})$ ： $l \leq r$ ， $1 \leq l, r, \text{target} \leq \text{max_value}$ ，闭区间。返回： $\sum_{x=l..r} \text{dis}(x, \text{target})$ (int)。

【API / 入口函数（赛时只认这里列的名字）】

- $\text{range_sum}(\text{int } l, \text{int } r, \text{int } \text{target}) \rightarrow$ 查询闭区间和 返回 int。

【改板时先认这几个量】

- spf ：最小质因子表。
- dp ：短间隙 DP 数组。

```
struct GcdGraphDistanceSum {
    const int INF = 1000000000;
    vector<int> spf;

    void init_sieve(int max_value) {
        spf.resize(max_value + 1);
        iota(spf.begin(), spf.end(), 0);
        for (int i = 2; i <= max_value; ++i) {
            if (spf[i] == i) {
                for (long long j = i; j <= max_value; j += i) spf[j] = i;
            }
        }
    }

    int count_coprime_leq(int x, int up) const {
        if (up <= 0) return 0;
        vector<int> primes;
        int t = x;
        while (t > 1) {
            int p = spf[t];
            primes.push_back(p);
            while (t % p == 0) t /= p;
        }

        int ans = 0;
        int k = (int)primes.size();
        for (int mask = 0; mask < (1 << k); ++mask) {
            int cur = up;
            for (int i = 0; i < k; ++i) {
                if ((mask >> i) & 1) cur /= primes[i];
            }
            if (__builtin_popcount((unsigned)mask) & 1) ans -= cur;
            else ans += cur;
        }
        return ans;
    }
};
```

```

int prefix_sum_to_target(int target, int up) const {
    if (up <= 0) return 0;

    int smaller_prime = target - 1;
    while (smaller_prime >= 2 && spf[smaller_prime] != smaller_prime) --smaller_prime;

    if (up < smaller_prime) {
        return 2 * up - count_coprime_leq(target, up);
    }

    int ans = 2 * (smaller_prime - 1) - count_coprime_leq(target, smaller_prime - 1);
    int gap = target - smaller_prime;
    vector<int> dp(gap + 1, INF);
    dp[0] = 0; // dp[i] 表示从 target-i 到 target 的最短路。

    for (int i = 1; i <= gap; ++i) {
        for (int j = 0; j < i; ++j) {
            dp[i] = min(dp[i], dp[j] + gcd(target - i, target - j));
        }
        if (target - i <= up) ans += dp[i];
    }
    return ans;
}

int range_sum(int l, int r, int target) const {
    return prefix_sum_to_target(target, r) - prefix_sum_to_target(target, l - 1);
}
};

```

典题：Round2 G 《GCD Graph》。这题的关键不是 Dijkstra，而是用质数把大范围压成“绝大多数只可能是 1 或 2”。

超立方删两点 Hamming-2 配对

赛时先看

题目信号：点是二进制向量；配对要求 Hamming 距离为偶数；允许整体异或和维度位重排。

本质：从 $0..2^n-1$ 中删去两个点 a, b ，构造剩余点的完美匹配，使每对点恰好有两个二进制位不同。

接法：先把 $a \text{ xor } b$ 的 1 位重排到低位，构造删 0 和 2^k-1 的标准配对，再把位映射和异或 a 还原。

复杂度判定： $O(2^n)$ 。

维护的量： $\text{mate}[x]$ （标准配对表， x 的配对点）； mapped （把 $a \text{ xor } b$ 的 1 位压到低位的位重排映射）。

警告：Hamming 距离 2 不改变 popcount 奇偶性，所以删去的两个点必须 popcount 奇偶性相同；构造前可整体异或 a ，把删点变成 0 和 $a \text{ xor } b$ 。

【最小完整示例（先抄这一段就能跑）】

题目： $n=2$ 维超立方删掉点 0,3，给剩余点做 Hamming 距离 2 的完美配对。

```

auto pairs = pair_hypercube_minus_two(2, 0, 3);
for (auto [u, v] : pairs) printf("(%d,%d) ", u, v); // 剩余 1、2 恰好距离 2

```

样例输出：(1,2)

【传参要求（照这个传不会错）】

- n ：维度，点集 $0..2^n-1$ 。 a, b ：删去的两个点， $0 \leq a, b < 2^n$ ； $\text{popcount}(a \oplus b)$ 为奇数时返回空。返回：完美配对列表（每对 Hamming 距离 2，pair 内升序）。

【改板时先认这几个量】

- sz ：当前已生成的掩码集合大小。
- bit ：遍历的维度位。
- cur ：当前标准配对用的掩码值。

```

vector<pair<int, int>> pair_hypercube_minus_two(int n, int a, int b) {
    int diff = a ^ b;
    if (__builtin_popcount((unsigned)diff) & 1) return {};

    int full = 1 << n;
    vector<int> mate(full);
    for (int x = 0; x < full; ++x) mate[x] = x ^ 3;
    mate[0] = 0;
    mate[3] = 3;

    int k = __builtin_popcount((unsigned)diff);
    for (int bits = 2; bits < k; bits += 2) {
        int cur = (1 << bits) - 1;

```

```

    mate[cur] = cur << 1;
    mate[cur << 1] = cur;
    mate[(cur << 1) ^ 3] = cur << 2;
    mate[cur << 2] = (cur << 1) ^ 3;
    mate[(cur << 2) ^ 3] = (cur << 2) ^ 3;
}

vector<int> mapped = {0};
for (int bit = 0; bit < n; ++bit) {
    if ((diff >> bit) & 1) {
        int sz = (int)mapped.size();
        for (int i = 0; i < sz; ++i) mapped.push_back(mapped[i] ^ (1 << bit));
    }
}
for (int bit = 0; bit < n; ++bit) {
    if (((diff >> bit) & 1) == 0) {
        int sz = (int)mapped.size();
        for (int i = 0; i < sz; ++i) mapped.push_back(mapped[i] ^ (1 << bit));
    }
}

vector<pair<int, int>> ans;
for (int i = 0; i < full; ++i) {
    if (mate[i] > i) ans.push_back({mapped[i] ^ a, mapped[mate[i]] ^ a});
}
return ans;
}

```

典题：Round2 H 《Hyperspace Pairing》。如果删点 popcount 奇偶性不同，两个奇偶类剩余数量不同，直接无解。

交通期望：对角线随机游走首次撞边

赛时先看

题目信号：每个控制点只有横向/纵向两种状态；斜走永远不等待；最优策略会尽量靠近 $i=j$ 的零等待对角线。

本质：在特殊交通网格中，从右下到左上，最小化期望等待时间。基础行走时间固定，只需求额外等待期望。

接法：若一边已经为 0，额外等待是单方向等待概率乘步数；否则把较大差值一侧映射成首次撞到边界的组合数求和。

复杂度判定：预处理组合数 $O(\max(n+m))$ ，单次 $O(|n-m| + \log \text{MOD})$ 。

维护的量：fac/ifac（阶乘与逆元表，供 C）；p1/p2（两个方向概率， $p1=p/q$ ）。

警告：最后答案要加固定走边数 $2n+2m-2$ ；代码里先令 $n--,m--$ 进入控制点坐标； p/q 和 $1-p/q$ 的方向在两侧是对称互换的。

【最小完整示例（先抄这一段就能跑）】

题目：2x2 网格、双向等概率（ $p=q$ ），求右下到左上的期望总时间。

```

TrafficExpectation t;
t.init(10); // 组合数预处理到 10 (须 >= n+m)
printf("%d\n", t.solve(2, 2, 1, 1)); // 等概率无等待，只剩固定步数 2n+2m-2

```

样例输出：6

【传参要求（照这个传不会错）】

- `init(max_n)`: 组合数上界，须 $\geq n+m$ 。
- `solve(n, m, p, q)`: n,m 网格行列 (≥ 1)； p,q 方向概率 ($1 \leq p \leq q < \text{MOD}$ ，模意义可逆)。返回：期望总时间 mod 998244353 (int)。

【API / 入口函数（赛时只认这里列的名字）】

- `C(int n, int k)` $\rightarrow$ 组合数 $C(n,k)$ ，返回 int。
- `init(int max_n)` $\rightarrow$ 初始化/清空结构
- `solve(int n, int m, int p, int q)` $\rightarrow$ 执行主算法并返回答案

```

struct TrafficExpectation {
    static constexpr int MOD = 998244353;
    vector<int> fac, ifac;

    long long mod_pow(long long a, long long e) const {
        long long r = 1;
        while (e > 0) {
            if (e & 1) r = r * a % MOD;
            a = a * a % MOD;
            e >>= 1;
        }
        return r;
    }
}

```



```

void init(int max_n) {
    fac.assign(max_n + 1, 1);
    ifac.assign(max_n + 1, 1);
    for (int i = 1; i <= max_n; ++i) fac[i] = 1LL * fac[i - 1] * i % MOD;
    ifac[max_n] = mod_pow(fac[max_n], MOD - 2);
    for (int i = max_n; i >= 1; --i) ifac[i - 1] = 1LL * ifac[i] * i % MOD;
}

int C(int n, int k) const {
    if (k < 0 || k > n) return 0;
    return 1LL * fac[n] * ifac[k] % MOD * ifac[n - k] % MOD;
}

int solve(int n, int m, int p, int q) const {
    long long ans = 2LL * (n + m) - 2;
    int p1 = 1LL * p * mod_pow(q, MOD - 2) % MOD;
    int p2 = (MOD + 1 - p1) % MOD;

    --n;
    --m;
    if (n == 0) return (ans + 1LL * m * p1) % MOD;
    if (m == 0) return (ans + 1LL * n * p2) % MOD;

    auto calc_side = [&](int equal_part, int extra, int keep_prob, int hit_prob) {
        long long cur = 1;
        long long res = 0;
        for (int i = 0; i <= extra; ++i) {
            res += 1LL * (extra - i) * cur % MOD * C(equal_part - 1 + i, i) % MOD;
            res %= MOD;
            cur = cur * keep_prob % MOD;
        }
        return res * mod_pow(hit_prob, equal_part + 1) % MOD;
    };

    if (n > m) ans += calc_side(m, n - m, p1, p2);
    else ans += calc_side(n, m - n, p2, p1);
    return ans % MOD;
}
};

```

典题：Round2 A 《Annoying Traffic》。如果遇到类似“二维 DP 看起来很大但有一条零代价对角线”，优先尝试按差值压缩。

树上绝对差赋点权：根相对值域 DP

赛时先看

题目信号：边约束是 $\text{abs}(\text{val}[u] - \text{val}[v]) = w$ ；点权整体平移不影响合法性；只关心最大值减最小值。

本质：给树边权，为每棵子树赋点权，使每条边权等于两端点权差绝对值，求子树内点权极差最小值。

接法： $\text{dp}[u][i]$ 表示设 u 权值为 V ，子树最小值不低于 $V-i$ 时，子树最大值超过 V 的最小值。

复杂度判定： $O(nW)$ ，其中 W 是最大边权，本题总规模保证可过。

维护的量： g （邻接表）； $\text{dp}[i]$ （父权为 V 、子树最低不低于 $V-i$ 时，子树高出 V 的最小量）； $\text{ans}[u]$ （ u 子树的最小极差）。

警告：最大极差最小值不超过 $2W-1$ ，DP 数组长度用 $2 * \text{max_edge}$ ；子节点可以比父亲小 w 或大 w ，两种转移都要取。

【最小完整示例（先抄这一段就能跑）】

题目：两个点连边权 3，赋点权使 $\text{abs}(\text{val}[u] - \text{val}[v]) = 3$ ，最小化整棵树极差。

```

FabulousTreeDP t;
t.init(2);
t.add_edge(0, 1, 3);
vector<int> ans = t.solve();
printf("%d\n", ans[0]); // 整棵树极差最小 3；叶节点 1 的子树极差 0

```

样例输出：3

【传参要求（照这个传不会错）】

- `init(n_)`：点数。
- `add_edge(u, v, w)`： u, v 节点下标 0 起 ($0..n-1$)， w 边权 (≥ 1)。
- `solve()`：返回 $\text{ans}[u]$ 以 u 为根子树的最小极差 (`vector<int>`)。

【API / 入口函数（赛时只认这里列的名字）】

- `add_edge(int u, int v, int w)` -> 加入一条边
- `init(int n_)` -> 初始化/清空结构

- `solve()` -> 执行主算法并返回答案

【改板时先认这几个量】

- `g`: 邻接表。
- `parent`: DFS 的父节点参数。
- `dp`: DP 状态。

```
struct FabulousTreeDP {
    int n = 0, bound = 0;
    vector<vector<pair<int, int>>> g;
    vector<int> ans;

    void init(int n_) {
        n = n_;
        bound = 0;
        g.assign(n, {});
    }

    void add_edge(int u, int v, int w) {
        g[u].push_back({v, w});
        g[v].push_back({u, w});
        bound = max(bound, 2 * w);
    }

    vector<int> dfs(int u, int parent) {
        vector<int> dp(bound, 0);
        for (auto [v, w] : g[u]) {
            if (v == parent) continue;
            vector<int> child = dfs(v, u);
            vector<int> best_from_child(bound, bound);

            for (int i = 0; i < bound; ++i) {
                best_from_child[max(0, i - w)] = min(best_from_child[max(0, i - w)], child[i] + w);
                if (i + w < bound) {
                    best_from_child[i + w] = min(best_from_child[i + w], max(0, child[i] - w));
                }
            }
            for (int i = 1; i < bound; ++i) {
                best_from_child[i] = min(best_from_child[i], best_from_child[i - 1]);
            }
            for (int i = 0; i < bound; ++i) {
                dp[i] = max(dp[i], best_from_child[i]);
            }
        }

        for (int i = 0; i < bound; ++i) ans[u] = min(ans[u], dp[i] + i);
        return dp;
    }

    vector<int> solve() {
        if (bound == 0) bound = 1;
        ans.assign(n, bound);
        dfs(0, -1);
        return ans;
    }
};
```

典题: Round2 F 《Fabulous Tree》。这类“绝对值边约束”先固定一个根值，把所有状态都变成相对根的上下界。

少量特殊边动态最短路：普通图全源 BFS + 小图 Dijkstra

赛时先看

题目信号: 可变边很少；非特殊边权相同；查询端点很多但每次最短路只需要经过特殊边端点和查询端点。

本质: 无向连通图中，大多数边权都是同一个 T ，只有 $k \leq 50$ 条特殊边权会被修改。每次修改后回答多组点对最短路。

接法: 普通边组成的图上从每个点 BFS，距离乘 T 。查询时在压缩点集上同时考虑完全图普通距离边和特殊边。

复杂度判定: 预处理普通边全源 BFS $O(nm)$ ；每个点对查询在 $O(k^2)$ 小图上跑朴素 Dijkstra。

维护的量: `normal_dis[u][v]`（普通边全源距离）；`special_weight[id]`（特殊边当前权值）；`special_nodes`（特殊边端点集）。

警告: 压缩点集要包含所有特殊边端点以及本次的 `a, b`；普通边路径距离来自预处理 `dis[u][v]`，特殊边用当前权值。

约定: `vector<Edge> edges`；// 0-indexed，下标从 0 开始

【最小完整示例（先抄这一段就能跑）】

题目: 3 点 2 边，普通边权 5，边 0 设为特殊边权 1，求 0 到 2 最短路并动态改权。

```
FewSpecialEdgesShortestPath sp;
sp.init(3, 5, {{0, 1}, {1, 2}}, {{0, 1}}); // (边 id, 特殊权), 权 0 表示普通边
```

```
printf("%lld\n", sp.query(0, 2)); // 0->1 走特殊边 1, 1->2 走普通边 5
sp.update_special_edge(0, 100); // 动态改特殊边权
printf("%lld\n", sp.query(0, 2)); // 绕回普通边 5+5=10
```

样例输出: 6 和 10

【传参要求（照这个传不会错）】

- `init(n_, normal_w_, all_edges, special_id_weight)`: `n_` 点数; `normal_w_` 普通边权; `all_edges` 全部边 `{u,v}` 0-indexed; `special_id_weight` 为 (边 id, 特殊权) 列表, 特殊权 ≥ 1 (0 表示普通边)。
- `update_special_edge(id, w)`: 把 id 号边 (须为特殊边) 权改成 `w` (≥ 1)。
- `query(s, t)`: `s, t` 0-indexed 点对。返回: 最短路 (long long, 不可达为 `4e18`)。

【API / 入口函数（赛时只认这里列的名字）】

- `query(int s, int t) ->` 查询 返回 long long。

【改板时先认这几个量】

- `edges`: 0-indexed, 下标从 0 开始。
- `dist`: 距离。

```
struct FewSpecialEdgesShortestPath {
    struct Edge {
        int u, v;
    };

    int n = 0;
    long long normal_w = 0;
    vector<Edge> edges; // 0-indexed, 下标从 0 开始。
    vector<int> special_weight;
    vector<vector<int>> special_edge_ids_at;
    vector<vector<long long>> normal_dis;
    vector<int> special_nodes;

    void init(int n_, long long normal_w_, const vector<Edge>& all_edges,
              const vector<pair<int, int>>& special_id_weight) {
        n = n_;
        normal_w = normal_w_;
        edges = all_edges;
        int m = (int)edges.size();
        special_weight.assign(m, 0);
        vector<int> is_special_endpoint(n, 0);

        for (auto [id, w] : special_id_weight) {
            special_weight[id] = w;
            is_special_endpoint[edges[id].u] = 1;
            is_special_endpoint[edges[id].v] = 1;
        }

        vector<vector<int>> normal_adj(n);
        special_edge_ids_at.assign(n, {});
        for (int id = 0; id < m; ++id) {
            auto [u, v] = edges[id];
            if (special_weight[id] == 0) {
                normal_adj[u].push_back(v);
                normal_adj[v].push_back(u);
            } else {
                special_edge_ids_at[u].push_back(id);
                special_edge_ids_at[v].push_back(id);
            }
        }

        const long long INF = (long long)4e18;
        normal_dis.assign(n, vector<long long>(n, INF));
        for (int s = 0; s < n; ++s) {
            queue<int> q;
            normal_dis[s][s] = 0;
            q.push(s);
            while (!q.empty()) {
                int u = q.front();
                q.pop();
                for (int v : normal_adj[u]) {
                    if (normal_dis[s][v] == INF) {
                        normal_dis[s][v] = normal_dis[s][u] + normal_w;
                        q.push(v);
                    }
                }
            }
        }

        for (int i = 0; i < n; ++i) {
```

```

        if (is_special_endpoint[i]) special_nodes.push_back(i);
    }
}

void update_special_edge(int id, int w) {
    special_weight[id] = w;
}

long long query(int s, int t) const {
    const long long INF = (long long)4e18;
    vector<int> nodes = special_nodes;
    nodes.push_back(s);
    nodes.push_back(t);
    sort(nodes.begin(), nodes.end());
    nodes.erase(unique(nodes.begin(), nodes.end()), nodes.end());

    vector<long long> dist(n, INF);
    vector<int> used(n, 0);
    dist[s] = 0;

    for (int step = 0; step < (int)nodes.size(); ++step) {
        int u = -1;
        for (int v : nodes) {
            if (!used[v] && (u == -1 || dist[v] < dist[u])) u = v;
        }
        if (u == -1 || dist[u] == INF) break;
        used[u] = 1;

        for (int v : nodes) {
            if (!used[v]) dist[v] = min(dist[v], dist[u] + normal_dis[u][v]);
        }
        for (int id : special_edge_ids_at[u]) {
            int v = edges[id].u ^ edges[id].v ^ u;
            if (!used[v]) dist[v] = min(dist[v], dist[u] + special_weight[id]);
        }
    }
    return dist[t];
}
};

```

典题：Round2

K 《Kindergarten》。少量特殊边/点的题，经常能先把“大图任意普通路径”预处理成压缩图上的完全边。

允许误差的向量询问：按方向分组 + 整数前缀近似

赛时先看

题目信号：题面允许相对误差；查询向量随机；表达式可写成 $|V_i| |T| (|\sin \theta| + |\cos \theta|)$ ，方向相近时比例接近。

本质：多次查询 $\sum (|V_i \cdot T| + |V_i \times T|)$ ，允许较大相对误差，且查询向量随机。

接法：每轮拿一个未回答查询作为代表方向，和它夹角足够小的查询分到同一组。对代表方向预处理数组前缀，回答这一组。

复杂度判定：期望 $O((n+q) * \text{轮数})$ ，轮数由方向覆盖精度决定。

维护的量：pending（未回答查询集合）；pref[i]（代表方向下整数前缀和，每项乘 2^{20} ）；ans[id]（查询答案）。

警告：浮点前缀和作差可能误差不好控；标程把每项乘 2^{20} 后存整数前缀，再按查询向量长度缩放。

【最小完整示例（先抄这一段就能跑）】

题目：向量 $(1,0,0)$ 、 $(0,1,0)$ ，用查询向量 $(1,1,0)$ 求 $[1,2]$ 的 $|\text{dot}| + |\text{cross}|$ 和。

```

vector<Vec3LL> vecs = {{0, 0, 0}, {1, 0, 0}, {0, 1, 0}}; // 下标从 1 起，vecs[0] 占位
vector<array<int, 5>> q = {{1, 1, 0, 1, 2}}; // {x, y, z, l, r}
auto ans = answer_random_direction_queries(vecs, q);
printf("%.6Lf\n", ans[0]); // 每项 |dot|+|cross| = 2，两项合计 4

```

样例输出：4.000000

【传参要求（照这个传不会错）】

- vecs: 向量数组，1-indexed（vecs[0] 占位不用），vecs[i].x/y/z 为坐标分量。
- raw_queries: 每项 $\{x,y,z,l,r\}$ = 查询向量分量 + 闭区间 $[l,r]$ ， $1 \leq l \leq r \leq n$ 。返回：ans[i] 为近似和（long double，允许相对误差）。

```

struct Vec3LL {
    long long x, y, z;
};

long double norm_vec(Vec3LL a) {

```

```

    return sqrtl((long double)a.x * a.x + (long double)a.y * a.y + (long double)a.z * a.z);
}

long double approximate_dot_cross_sum(Vec3LL v, Vec3LL t) {
    long double dot = (long double)v.x * t.x + (long double)v.y * t.y + (long double)v.z * t.z;
    long double cx = (long double)v.y * t.z - (long double)v.z * t.y;
    long double cy = (long double)v.z * t.x - (long double)v.x * t.z;
    long double cz = (long double)v.x * t.y - (long double)v.y * t.x;
    return fabs1(dot) + sqrtl(cx * cx + cy * cy + cz * cz);
}

vector<long double> answer_random_direction_queries(
    const vector<Vec3LL>& vecs,
    const vector<array<int, 5>>& raw_queries
) {
    int n = (int)vecs.size() - 1;
    int q = (int)raw_queries.size();

    vector<int> left(q), right(q);
    vector<Vec3LL> query_vec(q);
    for (int i = 0; i < q; ++i) {
        auto [x, y, z, l, r] = raw_queries[i];
        query_vec[i] = {x, y, z};
        left[i] = l;
        right[i] = r;
    }

    vector<int> pending(q);
    iota(pending.begin(), pending.end(), 0);
    vector<long double> ans(q, 0);

    while (!pending.empty()) {
        int rep_id = pending[0];
        Vec3LL rep = query_vec[rep_id];
        vector<int> group, rest;

        for (int id : pending) {
            Vec3LL cur = query_vec[id];
            long double dot = (long double)cur.x * rep.x + (long double)cur.y * rep.y + (long double)cur.z * rep.z;
            long double cos2 = dot * dot / (norm_vec(cur) * norm_vec(cur)) / (norm_vec(rep) * norm_vec(rep));
            if (cos2 > 0.9911) group.push_back(id);
            else rest.push_back(id);
        }

        const long double SCALE = (1 << 20);
        vector<long long> pref(n + 1, 0);
        for (int i = 1; i <= n; ++i) {
            long double value = approximate_dot_cross_sum(vecs[i], rep) / norm_vec(rep);
            pref[i] = pref[i - 1] + (long long)(value * SCALE);
        }

        for (int id : group) {
            long double base = (long double)(pref[right[id]] - pref[left[id] - 1]) / SCALE;
            ans[id] = base * norm_vec(query_vec[id]);
        }
        pending.swap(rest);
    }
    return ans;
}

```

典题：Round2 I 《Imperfect Dot Sums and Cross Sums》。只有题面允许误差并且数据随机时才这么写；精确题不要用近似分组。

网格画同心矩形：偶短边拆块构造

赛时先看

题目信号：要求输出一组边；每个点度数为 0 或 2，意味着染色边是一堆环；矩形边界天然满足点度数条件。

本质：在 $n*m$ 个格子的网格图边上染色，使每个点 incident 染色边数为 0 或 2，且坏格子数不超过 5。

接法：若短边为奇数，直接一层层画同心矩形。若短边为偶数，把长方形拆成正方形和奇短边长方形，再分别画。

复杂度判定：输出边数 $O(nm)$ 。

维护的量：edges（染色边列表，每条为两个格点编号）；无其他结构。

警告：函数里的坐标是格点坐标，范围 $0..n$ 、 $0..m$ ；输出点编号是 $(n+1)*(m+1)$ 个格点，不是 $n*m$ 个格子。

约定：return $i * (m + 1) + j + 1$ ；// 格点 (i, j) 的 1-based 编号

【最小完整示例（先抄这一段就能跑）】

题目：3x3 网格，画出全部同心矩形边（短边为奇数直接画）。

```
auto e = construct_easy_puzzle_edges(3, 3);
```

```
printf("%zu\n", e.size()); // 外圈 12 条 + 内圈 4 条, 每个格点度数 0 或 2
```

样例输出: 16

【传参要求（照这个传不会错）】

- n : 行格子数; m : 列格子数 (均 ≥ 1)。返回: 染色边列表, 点编号 $i*(m+1)+j+1$, 格点 i in $[0,n]$ 、 j in $[0,m]$ (1-based)。

```
vector<pair<int, int>> construct_easy_puzzle_edges(int n, int m) {
    auto id = [&](int i, int j) {
        return i * (m + 1) + j + 1; // 格点 (i, j) 的 1-based 编号。
    };

    vector<pair<int, int>> edges;
    auto draw_rectangle = [&](auto&& self, int x1, int xr, int y1, int yr) -> void {
        if (xr - x1 <= 0 || yr - y1 <= 0) return;
        for (int i = x1; i < xr; ++i) {
            edges.push_back({id(i, y1), id(i + 1, y1)});
            edges.push_back({id(i, yr), id(i + 1, yr)});
        }
        for (int j = y1; j < yr; ++j) {
            edges.push_back({id(x1, j), id(x1, j + 1)});
            edges.push_back({id(xr, j), id(xr, j + 1)});
        }
        self(self, x1 + 1, xr - 1, y1 + 1, yr - 1);
    };

    if (min(n, m) & 1) {
        draw_rectangle(draw_rectangle, 0, n, 0, m);
    } else if (n > m) {
        int x = n, y = m;
        while (x > 2 * y) {
            draw_rectangle(draw_rectangle, x - y, x, 0, y);
            x -= y + 1;
        }
        if (x == y || x == y + 1) draw_rectangle(draw_rectangle, 0, x, 0, y);
        else if (x % 2 == 0) {
            draw_rectangle(draw_rectangle, 0, y, 0, y);
            draw_rectangle(draw_rectangle, y + 1, x, 0, y);
        } else {
            int v = x / 2;
            if (!(v & 1)) --v;
            draw_rectangle(draw_rectangle, 0, v, 0, y);
            draw_rectangle(draw_rectangle, v + 1, x, 0, y);
        }
    } else {
        int x = n, y = m;
        while (y > 2 * x) {
            draw_rectangle(draw_rectangle, 0, x, y - x, y);
            y -= x + 1;
        }
        if (y == x || y == x + 1) draw_rectangle(draw_rectangle, 0, x, 0, y);
        else if (y % 2 == 0) {
            draw_rectangle(draw_rectangle, 0, x, 0, x);
            draw_rectangle(draw_rectangle, 0, x, x + 1, y);
        } else {
            int v = y / 2;
            if (!(v & 1)) --v;
            draw_rectangle(draw_rectangle, 0, x, 0, v);
            draw_rectangle(draw_rectangle, 0, x, v + 1, y);
        }
    }
    return edges;
}
```

典题: Round2 E 《Easy Puzzle》。构造图上度数全为偶数时, 先想“输出若干个不相交或嵌套的环”。

羽毛球比分连胜: 普通区与超分区分段

赛时先看

题目信号: 得分过程只增加不减少; 终止条件为“至少 k 分且领先 2 分”; 普通区和 deuce 超分区规则不同。

本质: 已知比赛从比分 (x_1, y_1) 到 (x_2, y_2) , 求第一位选手在这段过程中的最长连续得分的最小值和最大值。

复杂度判定: $O(1)$, 但分类多。

维护的量: dx/dy (第一/第二人得分增量); 分段后每段的 mn/mx (最长连胜的最小/最大值)。

警告: 题解建议先写小范围暴力搜索对拍, 再补分类边界; 尤其是从 $(k-1, k-1)$ 进入超分区时, 要用某次第二个人得分把序列切成两段。

【最小完整示例（先抄这一段就能跑）】

题目：比分从 $(0,0)$ 到 $(10,4)$ （全程未到 k ），求第一人最长连胜的最小/最大值。

```
long long dx = 10, dy = 4;
long long mn = (dx + dy) / (dy + 1); // ceil(dx/(dy+1)): dy 次对方得分把连胜隔开
long long mx = dx; // 最大: 全部连在一起
printf("%lld %lld\n", mn, mx);
```

样例输出: 2 10

【传参要求（照这个传不会错）】

- 本节无封装函数，直接用公式分段： $dx = x_2 - x_1$ 、 $dy = y_2 - y_1$ （比分起点/终点， $x_1 \leq x_2$ ）。
- 全程未到 k ： $mn = \text{ceil}(dx/(dy+1))$ 、 $mx = dx$ ；起点已超分或穿过超分区：按「赛时先看」分支枚举中转比分合并，返回值是 (mn, mx) 。
- 若全程没到 k ，第一人得分 dx 用第二人 dy 个得分隔开，最小最长段为 $\text{ceil}(dx/(dy+1))$ ，最大为 dx 。
- 若起点已在超分区，第一人连续 3 分通常不可能，除非最后第一人赢；最大值受总得分和领先差共同限制。
- 穿过普通区到超分区时，枚举中转比分，把前后两段的答案取 $\max$ 合并。

典题：Round2 C《Competition: Winning

Streaks》。这题更像“分类讨论工程题”，模板价值在分段方法，不在死背所有分支。

路径挂树按层生长计数 DP

赛时先看

题目信号：树上两点路径固定，其他点都挂在路径某个点或挂树上；距离到路径只按层数增加；还要处理标号选择。

本质：固定树中 $\text{dis}(1,2)=i$ ，统计路径外挂树对某个距离频次数组的贡献均值。

接法：设 $dp0[x][y]$ 为有 x 棵挂树、总点数 y 的方案数， $dp1[x][y]$

为对应答案和。每向外扩一层，枚举新一层的点数并乘组合数、父亲选择数和距离权值变化。

复杂度判定： $O(n^3)$ 。

警告：路径上的常数距离部分可最后整体乘 $P^{\{\text{dis}(1,2)\}}$ ；DP

转移中新增一层要选标号并给旧根选父亲，所以有组合数和 x^x 。

典题：Round2 D《Delivering Newspapers》。看到“固定主路径 +

路径外森林”的树计数，先把路径外点按到主路径的距离分层。

随机四舍五入期望：连续 4 进位链与生成函数

赛时先看

题目信号：四舍五入会把后缀清零， $0..3$ 直接舍， $5..8$ 直接进；真正复杂的是 $444...45$ 这种进位链。

本质：一个不含数码 9 的数，随机选择数位四舍五入 k 次，求最终数字期望。

复杂度判定：题解做法 $O(n \log n + n \log \text{MOD})$ 。

警告：最高位被操作后可能产生新的一位 10^n ；模意义下待定系数分解可能出现重根，不能直接套普通部分分式。

- 按“操作过的最高位”分组，概率是 $i^k - (i-1)^k$ 这类形式。
- 普通位 $0..3$ 和 $5..8$ 只需前缀数值、后缀 10^i 。
- 连续 4 后接大数时，进位概率等价于若干几何级数乘积的系数。
- 同一段连续 4 可转成卷积： $(x+j-1)^k / x!$ 与 $(-1)^x / x!$ ，用 NTT 批量求。
- 若最高位产生的分母在模意义下重根，要额外处理 $(1-kx)^2$ 的系数。

典题：Round2 J《Just Round

It》。这类题要先把“随机过程”按最后一次关键事件拆开，再把多次等待转成生成函数。

赛后模型总览：2026 牛客暑期多校训练营 3

赛时先看

题目信号：只记得题号或题面关键词，不确定该翻哪个算法专题。

本质：补 Round3 时先用本节定位题号对应的模型，再去下面具体板子或全册对应专题查代码。

复杂度判定：索引页不涉及算法复杂度。

警告：A/B/G/K/L 是短模型，建议直接熟练；D/H/J/M 更偏建模题，补题时要重点复盘“为什么能转成这个模型”。

题号到模型：

- A《比特掩码》：相邻二进制位状态计数，全体 AND/OR/XOR 后维护 1 段数量。

- B《再买一瓶》：Raney 引理 / cycle lemma，花光预算的合法随机序列计数。
- C《蛋糕店》：删支配订单、二分答案、前缀分批 DP，优化可用指针 + 单调结构 + 线段树。
- D《最长的连续的一》：DAG 拓扑序中的连续段，三组划分转最大权闭合图 / 最小割。
- E《扫雷》：两行条带构造，按列 mask DP 统计合法地雷方案数，离线搜索打表。
- F《Not Aqre 2》：窄行大列网格染色，按“颜色相等关系形状”压缩状态后矩阵快速幂。
- G《矩阵标记》：同值点按出现行压缩，前后缀列界推出可标记矩形，二维差分。
- H《季节》：周期序列线性空间，分圆多项式 / 线性递推判定最小周期上界。
- I《交换大师》：至多一次交换，受影响边局部化，按斜率集合维护历史最优。
- J《树.zip》：新父亲必须是原树祖先，约束最深祖先优先，用可并堆自底向上合并要求。
- K《转向导航》：三点叉积判左转、右转、直行。
- L《登山对决》：严格升高移动形成 DAG，按高度降序做普通图上胜负 DP。
- M《漫游者》：树上无立即回头随机游走，有向边状态期望，后序求线性关系、前序代回。

位对计数：全体按位操作后维护 1 段数量

赛时先看

题目信号：题目要统计“二进制里有几段连续的

1”；操作是对全部元素统一按位运算；单独改每个数会超时，但每一位的变化规则相同。

本质：维护一组整数，支持把所有数同时做 $\& x$ 、 $| x$ 、 $\wedge x$ ，每次询问所有数二进制表示中的连续 1 段总数。

接法：对每个相邻位 $(i, i+1)$ 维护四种状态数量。连续 1 段数量等于所有满足 $\text{bit}_i=1, \text{bit}_{i+1}=0$ 的位置数。

复杂度判定：初始化 $O(30n)$ ，每次操作 $O(30 * 4)$ ，询问 $O(30)$ 。

维护的量： $\text{cnt}[i][\text{low}][\text{high}]$ ：第 i 位取 low、第 $i+1$ 位取 high 的元素个数； $\text{LOG}=30$ ，第 30 位固定看成 0。

警告：额外补一个第 30 位为 0，用来统计最高位的 1 段结尾；如果题目值域超过 2^{30} ，把 LOG 改大，并把第 LOG 位补成 0。

【最小完整示例（先抄这一段就能跑）】

题目： n 个数、 q 次全体按位运算，每次询问所有数二进制里连续 1 段的段数。

```
BitRunCounter brc;
brc.init(a);           // a 为 vector<int>, a[i] 是第 i 个数，下标 0..n-1
brc.apply_all(1, x);   // 全体 &= x; type=1/2/3 分别对应 &、|、^
long long ans = brc.answer(); // 当前所有数的二进制连续 1 段总数
```

样例： $a=\{3,5\}$ 时 $\text{answer}()=3$ ； $\text{apply_all}(3,2)$ 后 $\text{answer}()=2$ 。

【传参要求（照这个传不会错）】

- $\text{init}(\text{const vector<int>\& a})$ ： $a[i]$ 为第 i 个数，下标 $0..n-1$ ，值域 $[0, 2^{30})$ ；无返回值。
- $\text{apply_all}(\text{int type}, \text{int x})$ ：type 取 1/2/3 ($\&$ 、 $|$ 、 $\wedge$)； x 与 $a[i]$ 同值域，若题面值域更大需先调大 LOG；无返回值。
- $\text{answer}()$ ：无参数；返回 long long，为当前所有数的连续 1 段总数。

【API / 入口函数（赛时只认这里列的名字）】

- $\text{init}(\text{const vector<int>\& a}) \rightarrow$ 初始化/清空结构

【改板时先认这几个量】

- low：当前低位（第 i 位）的旧 bit 值。
- nxt：按位操作后的新状态计数数组。

```
struct BitRunCounter {
    static constexpr int LOG = 30; // 处理 0..29 位；第 30 位固定看成 0。
    long long cnt[LOG][2][2]{};

    int apply_one_bit(int old_bit, int type, int x_bit) const {
        // type=1 表示 & x, type=2 表示 | x, type=3 表示 ^ x。
        if (type == 1) return old_bit & x_bit;
        if (type == 2) return old_bit | x_bit;
        return old_bit ^ x_bit;
    }

    void init(const vector<int>\& a) {
        memset(cnt, 0, sizeof(cnt));
        for (int x : a) {
            for (int i = 0; i < LOG; ++i) {
                int low = (x >> i) & 1;
                int high = (i + 1 == LOG ? 0 : ((x >> (i + 1)) & 1));
```

```

        cnt[i][low][high]++;
    }
}

void apply_all(int type, int x) {
    for (int i = 0; i < LOG; ++i) {
        long long nxt[2][2] = {};
        int xb0 = (x >> i) & 1;
        int xb1 = (i + 1 == LOG ? 0 : ((x >> (i + 1)) & 1));
        for (int a = 0; a <= 1; ++a) {
            for (int b = 0; b <= 1; ++b) {
                int na = apply_one_bit(a, type, xb0);
                int nb = apply_one_bit(b, type, xb1);
                nxt[na][nb] += cnt[i][a][b];
            }
        }
        memcpy(cnt[i], nxt, sizeof(nxt));
    }
}

long long answer() const {
    long long res = 0;
    for (int i = 0; i < LOG; ++i) res += cnt[i][1][0];
    return res;
}
};

```

典题：Round3 A 《比特掩码》。如果题面改成只问某个区间，通常要在线段树节点里维护同样的 `cnt[i][2][2]`。

Raney 引理：花光预算的合法序列计数

赛时先看

题目信号：有 m 步、其中 w 步是特殊事件；最终余额恰好到 0；要求中途余额始终大于 0；把序列反转取负后，每步都不超过 1。

本质：统计由两类步长组成的序列，要求总和固定，并且所有非空前缀和都为正。常见于“过程中不能破产 / 不能提前归零”的计数概率题。

接法：中奖概率 $p=a/b$ ，不中奖概率 $q=(b-a)/b$ ，答案为 $n * \text{inv}(m) * C(m,w) * p^w * q^{(m-w)}$ 。

复杂度判定：预处理组合数 $O(\max m)$ ，单次 $O(\log \text{MOD})$ 。

维护的量：`fac/ifac`：0..n 的阶乘与阶乘逆元，供 $C(n,k)$ $O(1)$ 查询；除此之外无其它长期状态。

警告：先判断中奖次数是否唯一合法： $m < n$ 或 $(m-n) \% c \neq 0$ 直接为 0；Raney 引理给的是合法线性序列数 $n / m * C(m,w)$ 。

【最小完整示例（先抄这一段就能跑）】

题目： m 次抽奖、每次以 a/b 概率赢，全程余额不为负且最终恰好为 0 的合法序列概率。

```

Comb998 comb;
comb.init(m); // 先预处理到最大步数 m 的组合数
int ans = bottle_raney_answer(n, m, c, a, b, comb);
printf("%d\n", ans); // 概率 mod 998244353

```

样例：`bottle_raney_answer(2,4,1,1,2,comb)` 返回 `811073537`（即 $3/16 \bmod 998244353$ ）。

【传参要求（照这个传不会错）】

- n ：最终净余额；需满足 $m \geq n$ 且 $(m-n) \% c == 0$ ，否则函数直接返回 0；`int`。
- m ：总步数；`int`；`comb.init(m)` 必须先按此值预处理。
- c ：每个特殊事件对余额的净变化量；`int`。
- a / b ：特殊事件概率 a/b （内部自动求模逆元），不中奖概率为 $(b-a)/b$ ；`int`。
- `comb`：已 `init` 的 `Comb998`；函数返回 `int`：概率 $\bmod 998244353$ 。

【API / 入口函数（赛时只认这里列的名字）】

- `C(int n, int k)` $\rightarrow$ 组合数 $C(n,k)$ ，返回 `int`。
- `init(int n)` $\rightarrow$ 初始化/清空结构

```

struct Comb998 {
    static constexpr int MOD = 998244353;
    vector<int> fac, ifac;

    long long mod_pow(long long a, long long e) const {
        long long r = 1;
        while (e > 0) {
            if (e & 1) r = r * a % MOD;

```

```

        a = a * a % MOD;
        e >>= 1;
    }
    return r;
}

void init(int n) {
    fac.assign(n + 1, 1);
    ifac.assign(n + 1, 1);
    for (int i = 1; i <= n; ++i) fac[i] = 1LL * fac[i - 1] * i % MOD;
    ifac[n] = mod_pow(fac[n], MOD - 2);
    for (int i = n; i >= 1; --i) ifac[i - 1] = 1LL * ifac[i] * i % MOD;
}

int C(int n, int k) const {
    if (k < 0 || k > n) return 0;
    return 1LL * fac[n] * ifac[k] % MOD * ifac[n - k] % MOD;
}
};

int bottle_raney_answer(int n, int m, int c, int a, int b, const Comb998& comb) {
    const int MOD = Comb998::MOD;
    if (m < n || (m - n) % c != 0) return 0;
    int win = (m - n) / c;
    if (win < 0 || win > m) return 0;

    long long inv_b = comb.mod_pow(b, MOD - 2);
    long long p = 1LL * a * inv_b % MOD;
    long long q = 1LL * (b - a) * inv_b % MOD;

    long long ans = n % MOD;
    ans = ans * comb.mod_pow(m, MOD - 2) % MOD;
    ans = ans * comb.C(m, win) % MOD;
    ans = ans * comb.mod_pow(p, win) % MOD;
    ans = ans * comb.mod_pow(q, m - win) % MOD;
    return (int)ans;
}

```

典题：Round3 B《再买一瓶》。看到“合法循环移位个数正好等于总和”的味道，优先想 Raney 引理。

DAG 连续全 1 段：三组划分转最大权闭合图

赛时先看

题目信号：要求在某个拓扑序中出现“连续的一段”；边约束使得段内集合必须序凸；点分为“段前、段中、段后”三类。

本质：在 DAG 的所有拓扑序里，最大化某个连续段内满足条件的点数，并构造达到最优的拓扑序。

接法：令 $s_i = [\text{group}_i \geq 1]$, $t_i = [\text{group}_i \geq 2]$, 则 $s_i \geq t_i$ 。最大化组 1 数量就是最大化 $s_i - t_i$ ，用最大权闭合图表达点权和蕴含约束。

复杂度判定：建最大权闭合图后跑最大流。若源点正权总和为 $O(n)$ ，Dinic 通常足够。

维护的量：每个点的一对状态 s_i/t_i （是否属于段内/段后），以及最大权闭合图网络（源点→正权点、负权点→汇点、蕴含边容量无穷）与跑完流后的残量网络。

警告：对每条边 $u \rightarrow v$ ，三组编号必须满足 $\text{group}[u] \leq \text{group}[v]$ ；标号为 0 的点不能分到组 1；最后还要按组号优先做一次拓扑排序来构造序列。

建模清单：

- 点权：选择 s_i 给 +1，选择 t_i 给 -1，于是只有 $s_i=1, t_i=0$ 贡献 1。
- 自身约束： $t_i \rightarrow s_i$ 。
- 边约束： $u \rightarrow v$ 推出 $s_u \rightarrow s_v$ 和 $t_u \rightarrow t_v$ 。
- 标号 0：禁止组 1，可加无限容量约束逼迫 s_i 与 t_i 同时选或同时不选。
- 残量网络源点侧读出变量取值，再按 $\text{group}=0,1,2$ 做拓扑排序。

典题：Round3 D《最长的连续的一》。如果目标从“点数最大”改成“权值最大”，把 +1/-1 换成对应点权即可。

两行扫雷条带 DP：按列 mask 统计方案数

赛时先看

题目信号：高度很小，数字约束只依赖相邻几列；题目要求构造“恰好 x 种方案”的局面， x 范围不大，可以搜索后打表。

本质：给一个两行扫雷条带，第一行全是 *，第二行由数字和 * 组成，统计有多少种合法地雷分布。可以作为构造题的离线搜索校验器。

接法：枚举第二行字符串，用下面函数算方案数；搜到 $x=1..300$ 的短条带后，比赛代码直接输出打表结果。

复杂度判定：宽度为 w 时 $O(w * 4^3)$ ，实际就是常数很小的线性 DP。

维护的量： $dp[older][prev]$ ：前两列掩码分别为 $older/prev$ 时的合法方案数；每列枚举 cur 后滚动成 ndp 。

警告：第二行是数字时，该格一定不是雷；数字只看左、中、右三列里的相邻格，自己的格子不计入。

【最小完整示例（先抄这一段就能跑）】

题目：两行条带、第二行 $bottom$ 由数字/* 组成，统计合法地雷分布方案数。

```
string bottom = "0*0";
long long cnt = count_minesweeper_strip(bottom); // bottom 下标 0..w-1, 可含 '*' 与数字
printf("%lld\n", cnt);                          // 方案数, 可能很大用 long long
```

样例：count_minesweeper_strip("0*0")=1（所有格无雷，只有 1 种）；("1*1")=3。

【传参要求（照这个传不会错）】

- $bottom$ ：第二行字符串，长度 w ，下标 $0..w-1$ ；数字 '0'..'9' 表示该格必须无雷且周围雷数恰为其值，'*' 表示该格任意（第一行不读入，函数自动枚举）；返回 long long：合法方案数。

【改板时先认这几个量】

- cur ：当前列的掩码枚举值。
- dp ：DP 状态。

```
long long count_minesweeper_strip(const string& bottom) {
    int w = (int)bottom.size();

    auto is_digit = [&](int col) {
        return 0 <= col && col < w && isdigit((unsigned char)bottom[col]);
    };
    auto top_mine = [](int mask) { return mask & 1; };
    auto bottom_mine = [](int mask) { return (mask >> 1) & 1; };

    auto digit_ok = [&](int col, int left_mask, int mid_mask, int right_mask) {
        if (!is_digit(col)) return true;
        if (bottom_mine(mid_mask)) return false;
        int mines = 0;
        mines += top_mine(left_mask) + top_mine(mid_mask) + top_mine(right_mask);
        mines += bottom_mine(left_mask) + bottom_mine(right_mask);
        return mines == bottom[col] - '0';
    };

    long long dp[4][4] = {};
    dp[0][0] = 1; // 还没放任何列时，虚拟的第 -2、第 -1 列都是空列。

    for (int col = 0; col <= w; ++col) {
        long long ndp[4][4] = {};
        for (int older = 0; older < 4; ++older) {
            for (int prev = 0; prev < 4; ++prev) {
                if (!dp[older][prev]) continue;
                for (int cur = 0; cur < 4; ++cur) {
                    if (col == w && cur != 0) continue; // 最右侧虚拟空列。
                    if (col < w && is_digit(col) && bottom_mine(cur)) continue;
                    if (col >= 1 && !digit_ok(col - 1, older, prev, cur)) continue;
                    ndp[prev][cur] += dp[older][prev];
                }
            }
        }
        memcpy(dp, ndp, sizeof(dp));
    }

    long long ans = 0;
    for (int a = 0; a < 4; ++a) ans += dp[a][0];
    return ans;
}
```

典题：Round3

E《扫雷》。这个板子本身是“计数器”，不是最终构造；构造题要在赛前或程序启动时搜索字符串并保存答案。

矩阵同值分割：出现行前后缀列界 + 二维差分

赛时先看

题目信号：矩阵按值分组；同一个值在若干行出现；需要找相邻出现行之间能被覆盖的行列区域。

本质：对矩阵中每个相同值独立处理，判断哪些矩形会被该值的上下两部分出现位置共同覆盖，最后用二维差分累计被标记次数。

接法：对某个值的出现行 `row[0..k-1]`，做 `pref_min_col[t]` 和 `suf_max_col[t]`。每个分割 `t` 若 `pref_min_col[t] < suf_max_col[t+1]`，就在差分数组中给矩形 `[row[t], row[t+1]] * [L, R]` 加一。

复杂度判定：设矩阵大小为 `n*m`，总复杂度 $O(nm \log(nm) + nm)$ ，若值域可直接桶排可做到近似 $O(nm)$ 。

维护的量：`pos[value][row]`：该值在每行的最左列 `mn` 与最右列 `mx`；每行出现行序列的前缀最小列

`pref_min`、后缀最大列 `suf_max`；累计覆盖次数的二维差分 `diff`。

警告：每个值只需要记每一行的最左和最右出现列；分割点在相邻出现行之间；题解条件是 $L < R$ ，不是 $L \leq R$ 。

【最小完整示例（先抄这一段就能跑）】

题目：1-indexed 矩阵，统计每个格子被“同值上/下两部分的出现位置”共同覆盖的次数。

```
vector<vector<int>> cover = mark_same_value_rectangles(a);
// a 是 (n+1)x(m+1)，有效数据在 a[1..n][1..m]；cover[i][j] 为 (i,j) 的覆盖次数
if (cover[i][j] > 0) { /* (i,j) 至少被一个值覆盖 */ }
```

样例：`a[1..2][1..2]` 全为 1 时，`cover[1][1]=cover[1][2]=cover[2][1]=cover[2][2]=1`。

【传参要求（照这个传不会错）】

- `a`：(n+1) x (m+1) 的矩阵，下标 1..n 行、1..m 列有效（函数内部从 `a[1]` 开始读，`a[0]` 行与各行的 0 列不读）；返回 `vector<vector<int>>`：与 `a` 同尺寸的 `cover`，`cover[i][j]` 是 1-indexed 的覆盖次数。

【改板时先认这几个量】

- `mx`：该行最右出现列。
- `mn`：该行最左出现列。

```
vector<vector<int>> mark_same_value_rectangles(const vector<vector<int>>& a) {
    int n = (int)a.size() - 1;
    int m = (int)a[1].size() - 1;

    // value -> vector of (row, min_col, max_col)。先临时用 map 合并同一行。
    map<int, map<int, pair<int, int>>> pos;
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            auto& p = pos[a[i][j]][i];
            if (p.first == 0) p = {j, j};
            else {
                p.first = min(p.first, j);
                p.second = max(p.second, j);
            }
        }
    }

    vector<vector<int>> diff(n + 3, vector<int>(m + 3, 0));
    auto add_rect = [&](int x1, int y1, int x2, int y2) {
        diff[x1][y1]++;
        diff[x2 + 1][y1]--;
        diff[x1][y2 + 1]--;
        diff[x2 + 1][y2 + 1]++;
    };

    for (auto& [value, rows_map] : pos) {
        int k = (int)rows_map.size();
        if (k <= 1) continue;

        vector<int> row, mn, mx;
        for (auto& [r, p] : rows_map) {
            row.push_back(r);
            mn.push_back(p.first);
            mx.push_back(p.second);
        }

        vector<int> pref_min(k), suf_max(k);
        for (int i = 0; i < k; ++i) {
            pref_min[i] = (i == 0 ? mn[i] : min(pref_min[i - 1], mn[i]));
        }
        for (int i = k - 1; i >= 0; --i) {
            suf_max[i] = (i == k - 1 ? mx[i] : max(suf_max[i + 1], mx[i]));
        }

        for (int t = 0; t + 1 < k; ++t) {
            int L = pref_min[t];
            int R = suf_max[t + 1];
            if (L < R) add_rect(row[t], L, row[t + 1], R);
        }
    }

    vector<vector<int>> cover(n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; ++i) {
```

```

    for (int j = 1; j <= m; ++j) {
        diff[i][j] += diff[i - 1][j] + diff[i][j - 1] - diff[i - 1][j - 1];
        cover[i][j] = diff[i][j];
    }
    return cover;
}

```

典题：Round3 G《矩阵标记》。如果题目只问是否被至少一个值标记，把 `cover[i][j] > 0` 当布尔值即可。

一次交换最大邻差：局部贡献函数扫描

赛时先看

题目信号：目标是 $\sum |a[i]-a[i-1]|$ ；操作是交换两个元素；直接枚举交换点 $O(n^2)$

超时：每个位置只和左右邻居有关。

本质：最大化序列相邻绝对差之和，允许交换两个位置一次。非相邻交换只影响四条边，可写成两个位置局部贡献函数的组合。

复杂度判定：相邻交换 $O(n)$ ；非相邻交换可按有限斜率集合扫描做到 $O(n * K^2)$ ，其中 K 为局部绝对值函数拆出的斜率数量，通常是常数。

维护的量：原始答案 `base`；每个位置 `p` 的局部贡献函数

$g_p(x) = |x - a[p-1]| + |x - a[p+1]|$ （端点只含一个邻居）；扫描右端点时历史左端点集合的「斜率-截距」最优值。

警告：相邻位置交换和非相邻位置交换的受影响边不同，要分开算；扫描右端点 `j` 时，历史集合只能加入 `i <= j-2`，否则会把相邻交换混进去。

- 先算原始答案 `base`。
- 枚举相邻交换，直接重算受影响的至多三条边。
- 对非相邻交换，把位置 `p` 的贡献写成 $g_p(x) = |x - a[p-1]| + |x - a[p+1]|$ ，端点只有一个邻居。
- $g_p(x)$ 是若干条直线的最大值。枚举右端点实际放入的值，在历史左端点集合里查 `best_slope_intercept + slope * value`。

典题：Round3

I《交换大师》。这题比板子更重要的是“只重算受影响边”的习惯；很多交换/替换/删除一项的题都靠这个降维。

折线路径转向判定：叉积符号

赛时先看

题目信号：题面直接问 `LEFT / RIGHT / STRAIGHT`；坐标是整数；只涉及相邻三个点的方向。

本质：给连续三个点，判断从第一段走到第二段是左转、右转还是直行。

接法：按路径顺序读点，对每个 `i=2..n-1` 调用 `turn(a[i-1], a[i], a[i+1])`。

复杂度判定：每个转向 $O(1)$ 。

维护的量：无额外结构；只维护相邻三个点 `a, b, c` 与叉积 $z = \text{cross}(a, b, c)$ 。

警告：向量应写成 `b-a` 和 `c-b`；叉积为正是左转，负是右转，零是直行。坐标到 `1e9` 时叉积要用 `long long`。

【最小完整示例（先抄这一段就能跑）】

题目：按顺序读 `n` 个点，对每三个相邻点输出转向是 `LEFT / RIGHT / STRAIGHT`。

```

PointLL a, b, c;
cin >> a.x >> a.y >> b.x >> b.y >> c.x >> c.y;
string d = turn_direction(a, b, c); // 从 a→b 再到 c 的转向
cout << d << "\n";

```

样例：`a=(0,0), b=(1,0), c=(1,1) → LEFT; c=(1,-1) → RIGHT; c=(2,0) → STRAIGHT`。

【传参要求（照这个传不会错）】

- `a`：路径上的前一个点；`b`：当前点；`c`：下一个点。均为 `PointLL (long long x, y)`，坐标绝对值 `1e9` 内安全，`x, y` 需先赋值再调用；`turn_direction` 返回 `string`：“LEFT”/“RIGHT”/“STRAIGHT”；`cross(a, b, c)` 返回 `long long` 叉积值。

```

struct PointLL {
    long long x, y;
};

long long cross(PointLL a, PointLL b, PointLL c) {
    long long x1 = b.x - a.x;
    long long y1 = b.y - a.y;
    long long x2 = c.x - b.x;
    long long y2 = c.y - b.y;
    return x1 * y2 - y1 * x2;
}

```



```

}

string turn_direction(PointLL a, PointLL b, PointLL c) {
    long long z = cross(a, b, c);
    if (z > 0) return "LEFT";
    if (z < 0) return "RIGHT";
    return "STRAIGHT";
}

```

典题：Round3 K 《转向导航》。如果题面说“顺时针/逆时针”，注意坐标系是否仍然是数学坐标系。

严格升高移动博弈：按高度反向拓扑

赛时先看

题目信号：每次移动后高度严格上升，天然无环；不能移动的人输；多次询问起点胜负。

本质：网格上只能走到严格更高的相邻格，判断从每个格出发的普通博弈胜负。

接法：把每个格子看成 DAG 点。win[u] = exists(v higher neighbor of u && !win[v])。

复杂度判定：排序 $O(nm \log(nm))$ ，转移 $O(nm)$ ，每次询问 $O(1)$ 。

维护的量：win[i][j]：从格 (i,j) 出发的胜负态（0 必败 / 1 必胜），按高度从高到低逐个填充。

警告：要按高度从高到低算，这样“更高邻居”的胜负已经知道；存在一个后继为必败态，当前就是必胜态。

【最小完整示例（先抄这一段就能跑）】

题目：网格每次只能走到严格更高的相邻格，不能走者输；输出每个格的胜负态并回答起点询问。

```

vector<vector<int>> win = increasing_grid_game(h);
// h 是 (n+1)x(m+1) 的 1-indexed 高度矩阵，win[i][j] 是 (i,j) 的胜负态
if (win[sx][sy]) puts("First"); else puts("Second");

```

样例：2x2 高度 {2,1;1,0}（1-indexed）时，win[1][1]=0、win[1][2]=1、win[2][1]=1、win[2][2]=0。

【传参要求（照这个传不会错）】

- h: (n+1) x (m+1) 的高度矩阵，有效数据在 h[1..n][1..m]（函数从 h[1] 起读，0 行/0 列不读）；返回 vector<vector<int>>：与 h 同尺寸的 win，win[i][j] {0,1}，1-indexed，为从 (i,j) 出发的胜负态。

```

vector<vector<int>> increasing_grid_game(const vector<vector<int>>& h) {
    int n = (int)h.size() - 1;
    int m = (int)h[1].size() - 1;
    vector<tuple<int, int, int>> cells;
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) cells.push_back({h[i][j], i, j});
    }
    sort(cells.rbegin(), cells.rend()); // 高度从大到小。

    vector<vector<int>> win(n + 1, vector<int>(m + 1, 0));
    int dx[4] = {1, -1, 0, 0};
    int dy[4] = {0, 0, 1, -1};

    for (auto [height, x, y] : cells) {
        for (int dir = 0; dir < 4; ++dir) {
            int nx = x + dx[dir], ny = y + dy[dir];
            if (nx < 1 || nx > n || ny < 1 || ny > m) continue;
            if (h[nx][ny] > h[x][y] && !win[nx][ny]) {
                win[x][y] = 1;
            }
        }
    }
    return win;
}

```

典题：Round3 L 《登山对决》。若移动规则从“更高”变成“更低”，排序方向反过来。

树上约束重连：最深要求祖先 + 可并堆

赛时先看

题目信号：所有约束都在一棵原树的祖先关系里；某个点必须挂到若干候选祖先中“最深的那个”才最优；子树需求需要向上合并。

本质：构造一棵新树，要求每个点的新父亲必须是原树严格祖先，并满足若干“v 仍为 u 的祖先”约束，同时最小化新树深度和。

复杂度判定：用左偏树 / 可并堆维护每个点的要求祖先集合， $O((n+q) \log q)$ 。

维护的量：每个点 x 的要求祖先堆 need[x]（按原树深度为关键字，堆顶是最深必须祖先）；新父亲数组 parent_new[x]；DFS 回溯时儿子堆合并后的剩余要求。

警告：处理顺序是原树自底向上；如果 $need[x]$ 为空， x 的新父亲可以直接接到根；取出最深要求祖先 d 后，要弹掉重复的 d ，剩余要求并入 $need[d]$ 。

- 对约束 (u,v) ，把 v 按原树深度作为关键字放进 $need[u]$ 。
- DFS 回溯到点 x 时，先把儿子的堆合进来。
- $need[x]$ 堆顶是当前最深的必须祖先 d ，令 $parent_new[x]=d$ 。
- 弹掉所有等于 d 的要求，剩余堆并入 $need[d]$ 。
- 最后按新父亲计算深度和。

典题：Round3 J《树.zip》。这类题别急着写倍增，核心是“最深约束优先”这个贪心。

有向边状态随机游走：树上无立即回头期望

赛时先看

题目信号：走到点 u 后不能立刻回到

pre ，除非没有其他邻居；目标点吸收；树结构使每个子树只通过一条边和外界相连。

本质：在树上随机游走，状态与“当前点 + 上一个点”有关，求到目标点的期望步数。

复杂度判定：树上两遍 DFS， $O(n)$ 。

维护的量：每个非根点 u 的一次函数系数 a_u/b_u （把“从父亲进入 u 的期望”写成 $F_u = a_u \cdot B_u + b_u$ ，其中 B_u 是“从 u 回父亲之后”的外部期望）；前序代回后每条有向边 $(pre \rightarrow u)$ 的真实期望值。

警告：状态不是点，而是有向边 $(pre \rightarrow u)$ ；对子树可以先写成 $F_u = a_u \cdot B_u + b_u$ ，其中 B_u 是从 u 走回父亲方向后的外部期望。

- 以目标点 t 为根。
- 后序处理每个非根点 u ，把“从父亲进入 u 后的期望”表示成关于“从 u 回父亲后的期望”的一次函数。
- 根的外部期望为 0，再前序把每条边的真实期望代回。
- 起点为 s 时，若 $s=t$ 答案为 0；否则答案是第一步 1 加上从相邻点状态出发的期望平均。

典题：Round3

M《漫游者》。遇到“不能回到上一点”的随机游走，先把状态改成有向边，不要只给每个点设一个期望。

赛后模型总览：2026 牛客暑期多校训练营 4

赛时先看

题目信号：只记得题号或题面关键词，不确定应该翻构造、DP、图论还是几何。

本质：补 Round4 时先按题号定位模型，再去下方模板或全册对应专题查实现。

复杂度判定：索引页不涉及算法复杂度。

警告：G/L 是重实现题，比赛时更需要复盘维护量是否值得；A/C/E/F/H/I/J/K 都能沉淀成相对稳定的模板。

题号到模型：

- A《Sixteen》：固定 16 阶矩阵，把行列式变成小系数线性表示，构造给定巨大整数。
- B《Quadratic Residue》：令 $p+q$ 为完全平方数，同时满足两个二次剩余条件。
- C《Retest Queue》：测试点排序，子集 DP + 高维前缀和预处理“能通过当前前缀的程序代价”。
- D《The Game》：循环最小表示下的双人对抗排列构造。
- E《DPRS》：强连通带权有向图价值可只枚举边；单条边降权询问用原全源最短路 $O(m)$ 回答。
- F《23 Subsequences》：合法子序列增长至少翻倍，最大长度约 60；按长度逐层求最早结束位置。
- G《Fading Memories》：区间消除长度由极值折线描述，线段树节点维护四条折线并可合并。
- H《String》：字符串按 $s+t < t+s$ 排序，修改只会把最靠前非 a 改成 a ，DP 状态只留全 a 前缀长度和后缀。
- I《Rounddog》：模式串无 border，循环串出现次数为 0/1/至少 2 三类讨论。
- J《Walk》：最短操作数下的栈式访问，BFS 全源最短路 + 区间 DP 求最小最大栈深。
- K《Decomposition Trees》：任意点分中心的点分树数量，统计根到固定点路径长度并树上背包。
- L《Geometry》：凸多边形缩放内切圆覆盖带权点，边 Voronoi 图 + 二分 + 区间扫描。

16 阶行列式构造：小元素矩阵表示任意目标值

赛时先看

题目信号：题面要求输出任意矩阵 / 图 / 序列，使某个代数值等于目标；目标值很大但单个元素范围很小。

本质：构造一个 16×16 的整数矩阵，元素范围在 $[-16, 16]$ 内，使行列式等于给定整数 x 。

接法：直接复制 `construct_det_16(x)`。输出时先输出 16，再输出矩阵。

复杂度判定：每个目标常数级搜索，约 $16 * 33$ 个候选。

维护的量：递归搜索中的系数数组 `coef[k]` 与剩余目标 `target`；预处理的 31 的幂 `pow31`；除此之外无其它长期结构。

警告：输入的 `x` 可能超过 `long long`，要用 `__int128`

读：构造出的矩阵元素不能越界；这是特化构造，别拿去求普通行列式。

【最小完整示例（先抄这一段就能跑）】

题目：输入可能超 `long long` 的整数 `x`，输出一个 16×16 矩阵使行列式等于 `x`（元素范围 $[-16, 16]$ ）。

```
i128 x = read_i128_string(s);          // s 为读入的十进制串，支持负号
auto mat = construct_det_16(x);         // mat 是 16x16，元素在 [-16, 16]
printf("16\n");                        // 先输出 16，再逐行输出 mat 的每个元素
```

样例：`read_i128_string("-123") = -123; construct_det_16(5)` 返回行列式为 5 的 16×16 矩阵。

【传参要求（照这个传不会错）】

- `read_i128_string(const string& s)`: `s` 为十进制整数字符串，下标 `0..len-1`，可带前导 `-`；返回 `i128` (`__int128_t`)。
- `construct_det_16(i128 x)`: `x` 为目标行列式值（把上面读到的数直接传入即可，可为负或 0）；返回 `vector<vector<int>>`： 16×16 矩阵，元素在 $[-16, 16]$ ，行列式恰为 `x`。

```
using i128 = __int128_t;

i128 abs_i128(i128 x) {
    return x < 0 ? -x : x;
}

i128 read_i128_string(const string& s) {
    bool neg = false;
    int p = 0;
    if (!s.empty() && s[0] == '-') neg = true, p = 1;
    i128 x = 0;
    for (; p < (int)s.size(); ++p) x = x * 10 + (s[p] - '0');
    return neg ? -x : x;
}

bool det16_dfs(int k, i128 target, const vector<i128>& pow31, vector<int>& coef) {
    if (k == 1) {
        if (-16 <= target && target <= 16) {
            coef[0] = (int)target;
            return true;
        }
        return false;
    }

    i128 weight = 16 * pow31[k - 2];
    vector<pair<i128, int>> candidate;
    for (int v = -16; v <= 16; ++v) {
        i128 rem = target - (i128)v * weight;
        if (rem % 15 != 0) continue;
        i128 nxt = rem / 15;
        i128 max_possible = 16 * pow31[k - 2];
        if (abs_i128(nxt) <= max_possible) candidate.push_back({abs_i128(nxt), v});
    }

    sort(candidate.begin(), candidate.end());
    for (auto [unused_abs, v] : candidate) {
        coef[k - 1] = v;
        if (det16_dfs(k - 1, (target - (i128)v * weight) / 15, pow31, coef)) {
            return true;
        }
    }
    return false;
}

vector<vector<int>> construct_det_16(i128 x) {
    const int N = 16;
    vector<i128> pow31(N + 1, 1);
    for (int i = 1; i <= N; ++i) pow31[i] = pow31[i - 1] * 31;

    vector<int> coef(N, 0);
    bool ok = det16_dfs(N, x, pow31, coef);
    assert(ok);

    vector<vector<int>> a(N, vector<int>(N, 0));
    for (int i = 0; i < N - 1; ++i) {
        for (int j = 0; j <= i; ++j) {
            a[i][j] = ((i + j) % 2 == 0 ? 16 : -16);
        }
    }
}
```

```

    a[i][i + 1] = 15;
}

for (int j = 0; j < N; ++j) {
    int sign = ((N + j + 1) % 2 == 0 ? 1 : -1);
    a[N - 1][j] = sign * coef[j];
}
return a;
}

```

典题：Round4 A 《Sixteen》。构造题先找“目标量对某些变量是线性的”这一层，剩下就是小范围表示。

平方和二次剩余构造：让 $p+q$ 成为平方数

赛时先看

题目信号：两个模数互相作为剩余；条件看似对称；可以把两边都改写成同一个数 $p+q$ 。

本质：给定正整数 p ，构造 x_1, x_2, q ，满足 $x_1^2 \equiv p \pmod{q}$ 且 $x_2^2 \equiv q \pmod{p}$ 。

接法：枚举平方数 $s = x^2 > p$ ，令 $q = s - p$ 。因为 $p \equiv s \pmod{q}$ 且 $q \equiv s \pmod{p}$ ，直接取 $x_1 = x \% q$ 、 $x_2 = x \% p$ 。

复杂度判定：从 $\text{ceil}(\sqrt{p})$ 往上试，实际很快。

维护的量：无额外结构；只维护当前枚举的平方根 x 与算出的 $q = x^2 - p$ ，四个整除/非零条件一起判。

警告：题面要求 $1 \leq x_1 < q$ 、 $1 \leq x_2 < p$ ，所以 $x \% q$ 和 $x \% p$ 不能为 0； q 可能到 $1e12$ ，用 `long long`。

【最小完整示例（先抄这一段就能跑）】

题目：给 p ，构造 x_1, x_2, q 使 $x_1^2 \equiv p \pmod{q}$ 且 $x_2^2 \equiv q \pmod{p}$ 。

```

auto [x1, x2, q] = construct_quadratic_residue(p); // p 为 int，结果自动满足 1<=x1<q, 1<=x2<p
printf("%lld %lld %lld\n", x1, x2, q);

```

样例：`construct_quadratic_residue(5)` 返回 $(3, 3, 4)$ ($3^2 \equiv 5 \pmod{4}$ 且 $3^2 \equiv 4 \pmod{5}$)。

【传参要求（照这个传不会错）】

- p ：给定的正整数，`int` (q 可能到 $1e12$ ，函数内部用 `long long`)；返回 `tuple<long long, long long, long long>`: $(x_1 = x \% q, x_2 = x \% p, q)$ ，满足两个同余式与取值范围约束。

```

tuple<long long, long long, long long> construct_quadratic_residue(int p) {
    long long x = sqrt((long double)p) + 1;
    while (true) {
        long long q = x * x - p;
        if (q > 0 && q % p != 0 && x % p != 0 && x % q != 0) {
            return {x % q, x % p, q};
        }
        ++x;
    }
}

```

典题：Round4 B 《Quadratic

Residue》。这类构造题不要只盯着“平方剩余算法”，先看能否手造一个共同平方数。

重测队列：子集 DP + 高维前缀和

赛时先看

题目信号：测试点数量 $m \leq$

20；要排列所有测试点；某个程序会在第一个失败点停止；状态自然是“已经排好的测试点集合”。

本质：所有程序使用同一个测试点顺序；每个程序遇到第一个不通过测试点就停止。求测试点排序使总时间最小。

接法：先把每个程序能通过的测试点集合记为 `passMask`，把它在每个测试点的耗时加到 `cost[j][passMask]`。对每个 j 做超集和，得到所有 $S \subset \text{passMask}$ 的贡献。

复杂度判定： $O(m^2 2^m + n m)$ ，空间 $O(m 2^m)$ 。

维护的量：`cost[j][S]`（已排完集合 S 后仍通过的程序在测试点 j 上的耗时和，超集和刷成 $S \subset \text{passMask}$ 的总和）；`dp[mask]`（排完集合 `mask` 的最小总耗时）。

警告：`cost[j][S]` 表示“已经排完集合 S 后，仍能通过 S 的程序跑测试点 j 的总时间”；不是只看 S 中最后一个测试点。

【最小完整示例（先抄这一段就能跑）】

```

// 题目：m<=20 个测试点重排，n 个程序遇首个不通过点即停，求最小总耗时。
vector<vector<long long>> d = {{1, 2}, {3, 1}}; // d[i][j]: 程序 i 在测试点 j 的耗时
vector<string> verdict = {"AA", "RA"}; // verdict[i][j]='A': 程序 i 能通过测试点 j
long long ans = minimum_retest_time(d, verdict); // 最小总耗时，直接输出
// 样例：测试点 1 排在 2 前时总耗时 6，ans = 6

```

【传参要求（照这个传不会错）】

- `d`: n 行 m 列耗时矩阵, `d[i][j]` 是程序 i 跑测试点 j 的耗时 (0-indexed), 值可到 $1e18$ 。
- `verdict`: n 个长度 m 的字符串, 'A' 表示能通过该测试点, 其余字符 (如 'R') 表示不通过。
- 返回: `long long`, 最优重排下的总耗时 (各程序跑到首个失败点即停的时间之和)。

【改板时先认这几个量】

- `bit`: 按位遍历的循环变量。
- `dp`: DP 状态。

```
long long minimum_retest_time(const vector<vector<long long>>& d, const vector<string>& verdict) {
    int n = (int)d.size();
    int m = (int)d[0].size();
    int B = 1 << m;
    vector<long long> cost((long long)m * B, 0);

    for (int i = 0; i < n; ++i) {
        int pass_mask = 0;
        for (int j = 0; j < m; ++j) {
            if (verdict[i][j] == 'A') pass_mask |= 1 << j;
        }
        for (int j = 0; j < m; ++j) {
            cost[(long long)j * B + pass_mask] += d[i][j];
        }
    }

    // 超集和: 处理后 cost[j][S] = 所有 passMask 包含 S 的程序在 j 上的耗时和。
    for (int j = 0; j < m; ++j) {
        long long* a = cost.data() + (long long)j * B;
        for (int bit = 0; bit < m; ++bit) {
            for (int mask = 0; mask < B; ++mask) {
                if ((mask & (1 << bit)) == 0) a[mask] += a[mask | (1 << bit)];
            }
        }
    }

    const long long INF = (long long)4e18;
    vector<long long> dp(B, INF);
    dp[0] = 0;
    for (int mask = 0; mask < B; ++mask) {
        int rest = (B - 1) ^ mask;
        while (rest) {
            int bit = __builtin_ctz(rest);
            int nmask = mask | (1 << bit);
            dp[nmask] = min(dp[nmask], dp[mask] + cost[(long long)bit * B + mask]);
            rest &= rest - 1;
        }
    }
    return dp[B - 1];
}
```

典题: Round4 C 《Retest Queue》。如果题面从 “Accepted/Rejected” 换成 “能否通过某条件集合”, 核心仍是 `passMask` 的超集和。

循环最小排列对抗构造: The Game

赛时先看

题目信号: 最终评价函数是 “循环移位的字典序最小表示”; 排列中最小值 1 一定是最小循环移位开头; 1 出现前后的选择有不同战略意义。

本质: 两个玩家轮流写一个排列, 最终看所有循环移位中字典序最小者, 求双方最优下的最终结果。

接法: 按题解结论直接构造。

复杂度判定: 每组 $O(n)$ 。

维护的量: `ans` (构造出的最终 $f(p)$ 序列); $m = n/2$ (两半分界, 奇偶公式在此分叉)。

警告: 偶数和奇数公式不同; 输出的是最终的 $f(p)$, 不是双方实际落子的原排列。

【最小完整示例（先抄这一段就能跑）】

```
// 题目: 两人轮流写排列 p, 最终取字典序最小的循环移位, 构造这个 f(p)。
int n = 6; // 排列长度, 奇偶走不同构造分支
vector<int> ans = game_lexicographically_min_rotation_result(n);
// 样例: n=6 -> {1, 4, 3, 5, 2, 6}; n=5 -> {1, 4, 2, 3, 5}
```

【传参要求（照这个传不会错）】

- `n`: 排列长度, $n \geq 1$; $n = 1$ 时直接返回 `{1}`。

- 返回: `vector<int>`, 长度 `n`, 即最优博弈结果 `f(p)`, 按序输出即可; 不是双方实际落子的排列。

```
vector<int> game_lexicographically_min_rotation_result(int n) {
    vector<int> ans;
    if (n == 1) return {1};

    if (n % 2 == 0) {
        int m = n / 2;
        ans.push_back(1);
        for (int i = 1; i <= m; ++i) {
            ans.push_back(m + i);
            if (i < m) ans.push_back(m + 1 - i);
        }
    } else {
        int m = n / 2;
        ans.push_back(1);
        for (int i = 1; i <= m; ++i) {
            ans.push_back(m + 3 - i);
            if (i == 1) ans.push_back(2);
            else ans.push_back(m + i + 1);
        }
    }
    return ans;
}
```

典题: Round4 D 《The Game》。博弈构造题经常先证明某一位的上下界相同, 再继续固定前缀。

最短路比值只枚举边: DPRS 降边权询问

赛时先看

题目信号: 最大值看似要枚举所有点对, 但图中边权全正; 修改只有单条边降权; `n,m,q` 总量约 2000。

本质: 强连通带权有向图中, 价值定义为 $\max \text{dis}(i,j)/\text{dis}(j,i)$ 。支持一次询问临时降低一条边权, 求新价值。

接法: 原图全源 Dijkstra 得到 `d`。询问把 `a→b` 从 `w` 降到 `x`, 新距离 $\text{dx}(u,v)=\min(\text{d}(u,v), \text{d}(u,a)+x+\text{d}(b,v))$, 然后枚举每条边的贡献。

复杂度判定: 预处理 $O(nm \log n)$; 每次询问 $O(m)$ 。

维护的量: `edges` (1-indexed 边表, `edges[0]` 占位); `g` (邻接表); `dis[s][t]` (原图全源最短路, `INF = 1LL<<62`)。

警告: 价值只需枚举边 `e=(u,v,w)` 的 $\text{dis}(v,u)/w$; 降权后任意最短路至多使用被修改边一次。

约定: `vector<Edge> edges;` // 1-indexed, `edges[0]` 不用

【最小完整示例 (先抄这一段就能跑)】

```
// 题目: 强连通带权有向图, 每次把第 id 条边临时降权, 求  $\max \text{dis}(i,j)/\text{dis}(j,i)$ 。
DPRS solver; // 先构造结构
solver.init(n, input_edges); // 顶点 1..n, 边表从下标 1 开始
long double val = solver.query_decrease_edge(id, new_w); // 第 id 条边降为 new_w 后查询
// 样例: 三角环 1→2(10), 2→3(1), 3→1(1), query_decrease_edge(1,3) 返回 4
```

【传参要求 (照这个传不会错)】

- `init(n_, input_edges)`: `n_` 为点数 (顶点 `1..n_`); `input_edges` 是 `vector<DPRS::Edge>`, 下标从 1 开始 (`edges[0]` 占位不用), 每条边 `{u,v,w}` 权全正。
- `query_decrease_edge(id, new_w)`: `id` 为被降权边下标 ($1 \leq id < \text{edges.size}()$); `new_w` 为降权后的新边权, 满足 $0 < new_w < \text{edges}[id].w$ 。
- 返回: `long double`, 降权后的最大不对称度 (枚举每条边取 $\text{dis}(v,u)/w_e$ 的最大值)。

【API / 入口函数 (赛时只认这里列的名字)】

- `init(int n_, const vector<Edge>& input_edges)` → 初始化/清空结构

【改板时先认这几个量】

- `edges`: 1-indexed, `edges[0]` 不用。
- `g`: 邻接表。

```
struct DPRS {
    struct Edge {
        int u, v;
        long long w;
    };

    int n = 0;
    vector<Edge> edges; // 1-indexed, edges[0] 不用。
    vector<vector<pair<int, long long>>> g;
    vector<vector<long long>> dis;
```

```

static constexpr long long INF = (1LL << 62);

void init(int n_, const vector<Edge>& input_edges) {
    n = n_;
    edges = input_edges;
    g.assign(n + 1, {});
    for (int i = 1; i < (int)edges.size(); ++i) {
        g[edges[i].u].push_back({edges[i].v, edges[i].w});
    }
    dis.assign(n + 1, vector<long long>(n + 1, INF));
    for (int s = 1; s <= n; ++s) dijkstra(s);
}

void dijkstra(int s) {
    priority_queue<pair<long long, int>, vector<pair<long long, int>>, greater<pair<long long, int>>> pq;
    dis[s][s] = 0;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [du, u] = pq.top();
        pq.pop();
        if (du != dis[s][u]) continue;
        for (auto [v, w] : g[u]) {
            if (dis[s][v] > du + w) {
                dis[s][v] = du + w;
                pq.push({dis[s][v], v});
            }
        }
    }
}

long double query_decrease_edge(int id, long long new_w) const {
    int a = edges[id].u;
    int b = edges[id].v;
    long double ans = 0;
    for (int i = 1; i < (int)edges.size(); ++i) {
        int u = edges[i].u;
        int v = edges[i].v;
        long long denominator = (i == id ? new_w : edges[i].w);

        long long back = dis[v][u];
        if (dis[v][a] < INF && dis[b][u] < INF) {
            back = min(back, dis[v][a] + new_w + dis[b][u]);
        }
        ans = max(ans, (long double)back / (long double)denominator);
    }
    return ans;
}
};

```

典题：Round4 E 《DPRS》。这个结论也适合记成一句话：最大往返不对称度由某条边的反向最短路除以边权达到。

23 子序列：长度很小的区间最长合法子序列

赛时先看

题目信号：每一步至少翻倍，值域最多 $1e18$ ，所以最长长度不超过约 60；询问很多但长度层数很少。

本质：给定序列 a ，多次询问区间 $[l, r]$ 内最长子序列长度，要求相邻选择值满足 $2*b[i-1] \leq b[i] \leq 3*b[i-1]$ 。

接法： $end_len[i]$ 表示以 i

为第一个元素、长度为当前层的合法子序列最早能在哪个位置结束。每加一层，用线段树按值域查 $[2a[i], 3a[i]]$ 内的最小结束位置。

复杂度判定：设最大可行长度为 $K \leq 60$ ，预处理并回答所有询问 $O(K(n \log n + q))$ 。

维护的量： $prev/cur[i]$ （长为当前层的合法子序列在 i

处开始的最早结束位置）； $suffix_best[i]$ （后缀最早结束位置的最小值）； seg （按值域存最小结束位置的线段树）。

警告：子序列不是子段，可以跳着选；为了保证下标递增，扫 i 时线段树里只能放 $j > i$ 的位置。

【最小完整示例（先抄这一段就能跑）】

```

// 题目：多次询问区间 [l,r] 内最长合法子序列，相邻元素满足 2*b[i-1] <= b[i] <= 3*b[i-1]。
vector<long long> a = {0, 1, 2, 6}; // a[0] 占位，序列从 a[1] 开始
vector<pair<int, int>> queries = {{1, 3}}; // 询问闭区间，1-indexed
vector<int> ans = answer_23_subsequence_queries(a, queries);
// 样例：a={1,2,6} 可选出 1->2->6，询问 [1,3] 得 ans[0] = 3

```

【传参要求（照这个传不会错）】

- a ：长度 $n+1$ 的数组， $a[0]$ 占位不用，真实序列在 $a[1..n]$ ，值可到 $1e18$ 。
- $queries$ ： q 个 $\{l, r\}$ 闭区间， $1 \leq l \leq r \leq n$ 。
- 返回： $vector<int>$ ，长度 q ， $ans[id]$ 是第 id 个询问的最长合法子序列长度（至少 1）。

【API / 入口函数（赛时只认这里列的名字）】

- `init(int m)` -> 初始化/清空结构
- `query_min(int l, int r)` -> 查询闭区间最小值 返回 `int`。

```
struct RangeMinPointUpdate {
    int n = 1;
    static constexpr int INF = 1000000000;
    vector<int> t;

    void init(int m) {
        n = 1;
        while (n < m) n <= 1;
        t.assign(2 * n, INF);
    }

    void reset() {
        fill(t.begin(), t.end(), INF);
    }

    void update_min(int p, int value) {
        int x = p + n - 1;
        t[x] = min(t[x], value);
        for (x >= 1; x; x >= 1) t[x] = min(t[x << 1], t[x << 1 | 1]);
    }

    int query_min(int l, int r) const {
        if (l > r) return INF;
        int res = INF;
        l += n - 1;
        r += n - 1;
        while (l <= r) {
            if (l & 1) res = min(res, t[l++]);
            if (!(r & 1)) res = min(res, t[r--]);
            l >>= 1;
            r >>= 1;
        }
        return res;
    }
};

vector<int> answer_23_subsequence_queries(const vector<long long>& a, const vector<pair<int, int>>& queries) {
    int n = (int)a.size() - 1;
    int q = (int)queries.size();
    vector<long long> vals(a.begin() + 1, a.end());
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());

    int M = (int)vals.size();
    vector<int> pos(n + 1), left_id(n + 1), right_id(n + 1);
    for (int i = 1; i <= n; ++i) {
        pos[i] = lower_bound(vals.begin(), vals.end(), a[i]) - vals.begin() + 1;
        left_id[i] = lower_bound(vals.begin(), vals.end(), 2 * a[i]) - vals.begin() + 1;
        right_id[i] = upper_bound(vals.begin(), vals.end(), 3 * a[i]) - vals.begin();
    }

    vector<int> ans(q, 1);
    vector<int> prev(n + 2), cur(n + 2), suffix_best(n + 3);
    for (int i = 1; i <= n; ++i) prev[i] = i;

    RangeMinPointUpdate seg;
    seg.init(M);
    int max_len = min(n, 63); // 1e18 下最多约 60 层。

    for (int len = 2; len <= max_len; ++len) {
        seg.reset();
        fill(cur.begin(), cur.end(), RangeMinPointUpdate::INF);

        for (int i = n; i >= 1; --i) {
            cur[i] = seg.query_min(left_id[i], right_id[i]);
            seg.update_min(pos[i], prev[i]);
        }

        int best = RangeMinPointUpdate::INF;
        for (int i = n; i >= 1; --i) {
            best = min(best, cur[i]);
            suffix_best[i] = best;
        }
        if (best >= RangeMinPointUpdate::INF) break;

        for (int id = 0; id < q; ++id) {
            auto [l, r] = queries[id];
            if (suffix_best[l] <= r) ans[id] = len;
        }
    }
}
```



```

    }
    prev.swap(cur);
  }
  return ans;
}

```

典题：Round4 F 《23 Subsequences》。如果比例条件换成 $b[i] \geq c \cdot b[i-1]$ ，最大长度仍可用 $\log_c V$ 控制。

拼接字符串加前缀改 a：只留 a 前缀长度和后缀

赛时先看

题目信号：允许把字符改小；最优修改一定从左到右把非 a 改成 a；剩余未修改字符串要按 $s+t < t+s$ 排序。

本质：若干字符串可任意重排拼接，再把至多 k 个字符改成任意小写字母。对每个 k 求字典序最小结果。

接法：状态 (used_cost, has_partial) 存二元组（全 a 前缀长度，后缀字符串）。比较时先让前缀更长，再让后缀字典序更小。

复杂度判定：总长 $L \leq 500$ 时，直接存字符串状态 $O(nL^2)$ 。

维护的量：dp[cost][used]（状态二元组：pref 全 a 前缀长度 + suf 后缀字符串，used 标记是否已选部分修改串）；answer[k]（至多 k 次修改的最优结果）。

警告：最终结构是“若干完全变成 a 的字符串 + 至多一个部分修改字符串 + 未修改后缀”；部分修改字符串只允许选一次。

【最小完整示例（先抄这一段就能跑）】

```

// 题目：字符串可任意重排拼接，至多改 k 个字符为 'a'，对每个 k 求字典序最小结果。
vector<string> s = {"bc", "aa"}; // 任意小写字母串集合
vector<string> ans = best_strings_after_a_changes(s); // ans[k]: 花至多 k 次修改的最优串
// 样例：s={"bc","aa"} -> ans[0]="aabc", ans[1]="aaac"

```

【传参要求（照这个传不会错）】

- s: 若干小写字母串（可重复、任意顺序），总长 $L \leq 500$ ；按值传入即可（内部会排序，不影响调用方副本）。
- 返回：vector<string>，长度 $L+1$ ，answer[k] ($k = 0..L$) 是至多改 k 个字符的字典序最小结果。

【改板时先认这几个量】

- cur: 当前处理的字符串。
- nxt: 转移后的新状态。
- dp: DP 状态。

```

struct StringState {
    int pref = -1;
    string suf;
    bool ok = false;
};

bool better_string_state(const StringState& a, const StringState& b) {
    if (!a.ok) return false;
    if (!b.ok) return true;
    if (a.pref != b.pref) return a.pref > b.pref;
    return a.suf < b.suf;
}

void relax_string_state(StringState& a, const StringState& b) {
    if (better_string_state(b, a)) a = b;
}

vector<string> best_strings_after_a_changes(vector<string> s) {
    sort(s.begin(), s.end(), [](const string& a, const string& b) {
        return a + b < b + a;
    });

    int L = 0;
    for (auto& x : s) L += (int)x.size();
    vector<array<StringState, 2>> dp(L + 1, ndp(L + 1));
    dp[0][0] = {0, "", true};

    for (const string& cur : s) {
        for (int j = 0; j <= L; ++j) ndp[j] = {StringState(), StringState()};

        int full_cost = 0;
        for (char c : cur) full_cost += (c != 'a');
        vector<int> pre_cost(cur.size() + 1, 0);
        for (int i = 0; i < (int)cur.size(); ++i) pre_cost[i + 1] = pre_cost[i] + (cur[i] != 'a');

        for (int cost = 0; cost <= L; ++cost) {
            for (int used = 0; used <= 1; ++used) {

```

```

        if (!dp[cost][used].ok) continue;

        if (cost + full_cost <= L) {
            StringState nxt = dp[cost][used];
            nxt.pref += (int)cur.size();
            relax_string_state(ndp[cost + full_cost][used], nxt);
        }

        StringState keep = dp[cost][used];
        keep.suf += cur;
        relax_string_state(ndp[cost][used], keep);

        if (used == 0) {
            for (int p = 0; p < (int)cur.size(); ++p) {
                if (cur[p] == 'a') continue;
                int add = pre_cost[p];
                if (cost + add > L) continue;
                StringState part;
                part.ok = true;
                part.pref = dp[cost][0].pref + p;
                part.suf = cur.substr(p) + dp[cost][0].suf;
                relax_string_state(ndp[cost + add][1], part);
            }
        }
    }
    dp.swap(ndp);
}

vector<string> answer(L + 1);
string best;
for (int k = 0; k <= L; ++k) {
    for (int used = 0; used <= 1; ++used) {
        if (!dp[k][used].ok) continue;
        string cur = string(dp[k][used].pref, 'a') + dp[k][used].suf;
        if (best.empty() || cur < best) best = cur;
    }
    answer[k] = best;
}
return answer;
}

```

典题：Round4 H 《String》。如果题目只问某一个 k ，也可以跑到 L 后取 `answer[k]`，实现更省心。

无 border 循环串出现计数：Rounddog 三分类

赛时先看

题目信号：模式串没有 border；环形串里某个长度为 m 的模式出现次数只可能属于三类：0 次、1 次、至少 2 次。

本质：给定环形字符串 S 和特殊模式 $T_k = \text{"Rounddog"} + k \text{ 个 'g'}$ ，统计有多少个循环移位包含该模式。

接法：数出现次数 cnt ，答案 $\text{cnt}=0 \rightarrow 0$ ， $\text{cnt}=1 \rightarrow n-m+1$ ， $\text{cnt} \geq 2 \rightarrow n$ 。

复杂度判定：朴素检查 $O(n * m)$ ，本题 $k \leq 100$ 可过；通用写法可替换为 KMP。

维护的量：`pattern`（ $\text{"Rounddog"} + k \text{ 个 'g'}$ ）；`doubled`（ $S + S[0..m-2]$ ）；`cnt`（起点 $0..n-1$ 中的出现次数，数到 2 就停）。

警告：只需要在 $S + S[0..m-2]$ 中检查起点 $0..n-1$ ；如果出现两次及以上，所有循环移位都包含一次。

【最小完整示例（先抄这一段就能跑）】

```

// 题目：环形串 S 的 n 个循环移位里有多少个包含模式 "Rounddog" + k 个 'g'。
string S = "Rounddog"; // 环形串内容 (0-indexed)
int k = 1; // 模式串尾部 'g' 的个数
int ans = count_rounddog_rotations(S, k); // 三分类：0 / n-m+1 / n
// 样例：S="Rounddog", k=1 -> 模式恰出现 1 次, ans = n-m+1 = 1

```

【传参要求（照这个传不会错）】

- S ：环形串内容，长度 n 。
- k ：模式串尾部 'g' 的数量， $k \geq 0$ ；模式总长 $m = 7 + k$ 。
- 返回：int：出现 0 次返回 0；恰 1 次返回 $n - m + 1$ ；至少 2 次返回 n ； $m > n$ 时返回 0。

```

int count_rounddog_rotations(const string& S, int k) {
    int n = (int)S.size();
    string pattern = string("Rounddog") + string(k, 'g');
    int m = (int)pattern.size();
    if (m > n) return 0;

    string doubled = S + S.substr(0, m - 1);
    int cnt = 0;
    for (int start = 0; start < n; ++start) {

```

```

bool ok = true;
for (int j = 0; j < m; ++j) {
    if (doubled[start + j] != pattern[j]) {
        ok = false;
        break;
    }
}
if (ok && ++cnt >= 2) break;
}

if (cnt == 0) return 0;
if (cnt == 1) return n - m + 1;
return n;
}

```

典题：Round4 I 《Rounddog》。若换成一般模式串，先用 KMP 判断 border；有 border 时不能直接套三分类公式。

栈式最短路访问：Walk 区间 DP

赛时先看

题目信号：操作过程像 DFS 栈；最少操作数意味着相邻特殊点之间必须走最短路；还要优化最大栈深。

本质：用栈在无向图上移动；可压入栈顶邻点，也可弹出栈顶并访问新的栈顶。要求按序访问特殊点，在总操作数最少的前提下最小化最大栈大小。

接法： $f[u][1][r]$ 表示以 u 为当前不可弹出的栈顶，访问 $x_1..x_r$ 后回到 u 的最小最大相对栈深； $g[u][1][r]$ 表示最后不要求回到 u 。

复杂度判定：全源 BFS $O(nm)$ ；DP 约 $O(mL^2+nL^3)$ ，适合 $n \leq 200, L \leq 60$ 。

维护的量： $dis[s][t]$ （全源最短路）； $f[u][1][r]$ （以 u 为栈顶、访问完 $x_1..x_r$ 并回到 u 的最小最大相对栈深）； $g[u][1][r]$ （最后不要求回到 u 的版本）。

警告：先合并连续相同特殊点； $on(a,b;u,v)$ 表示边 $u \rightarrow v$ 能在某条 a 到 b 的最短路上。

【最小完整示例（先抄这一段就能跑）】

```

// 题目：无向图上用栈按序访问特殊点序列，最少操作数前提下最小化最大栈大小。
int n = 3; // 顶点数，编号 1..n
vector<vector<int>> adj = {{}, {2}, {1, 3}, {2}}; // 1-indexed 邻接表，adj[0] 不用
vector<int> x = {1, 2, 3}; // 需依次访问的特殊点
int ans = minimum_walk_stack_depth(n, adj, x);
// 样例：链 1-2-3 依次访问 1,2,3 -> ans = 3

```

【传参要求（照这个传不会错）】

- n ：点数，顶点编号 $1..n$ 。
- adj ： $n+1$ 个 `vector` 的邻接表（ $adj[0]$ 不用），无向边要双向加入。
- x ：特殊点访问顺序，元素范围 $[1,n]$ ，可重复（函数会先合并连续相同点）。
- 返回：`int`，最小最大栈大小；去重后只剩 1 个特殊点时返回 1。

```

int minimum_walk_stack_depth(int n, const vector<vector<int>>& adj, vector<int> x) {
    const int INF = 1000000000;

    vector<int> y;
    for (int v : x) {
        if (y.empty() || y.back() != v) y.push_back(v);
    }
    x = y;
    int L = (int)x.size();
    if (L == 1) return 1;

    vector<vector<int>> dis(n + 1, vector<int>(n + 1, INF));
    for (int s = 1; s <= n; ++s) {
        queue<int> q;
        dis[s][s] = 0;
        q.push(s);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int v : adj[u]) {
                if (dis[s][v] == INF) {
                    dis[s][v] = dis[s][u] + 1;
                    q.push(v);
                }
            }
        }
    }

    vector<vector<vector<int>>> f(n + 1, vector<vector<int>>(L, vector<int>(L, INF)));
    vector<vector<vector<int>>> g(n + 1, vector<vector<int>>(L, vector<int>(L, INF)));
    vector<vector<int>> order(L);
}

```

```

for (int l = 0; l < L; ++l) {
    order[l].resize(n);
    iota(order[l].begin(), order[l].end(), 1);
    sort(order[l].begin(), order[l].end(), [&](int a, int b) {
        return dis[a][x[l]] < dis[b][x[l]];
    });
}

for (int len = 1; len <= L; ++len) {
    for (int l = 0; l + len - 1 < L; ++l) {
        int r = l + len - 1;
        for (int u = 1; u <= n; ++u) {
            if (l == r && u == x[l]) f[u][l][r] = g[u][l][r] = 1;

            for (int k = 1; k < r; ++k) {
                if (dis[x[k]][x[k + 1]] == dis[x[k]][u] + dis[u][x[k + 1]]) {
                    f[u][l][r] = min(f[u][l][r], max(f[u][l][k], f[u][k + 1][r]));
                    g[u][l][r] = min(g[u][l][r], max(f[u][l][k], g[u][k + 1][r]));
                }
            }
        }

        for (int u : order[l]) {
            for (int v : adj[u]) {
                if (dis[u][x[l]] == dis[v][x[l]] + 1) {
                    g[u][l][r] = min(g[u][l][r], g[v][l][r] + 1);
                    if (dis[u][x[r]] == dis[v][x[r]] + 1) {
                        f[u][l][r] = min(f[u][l][r], f[v][l][r] + 1);
                    }
                }
            }
        }
    }
}

return g[x[0]][0][L - 1];
}

```

典题：Round4 J 《Walk》。如果题面只要求最少操作数，不要求最小栈深，就直接拼相邻特殊点最短路即可；本题多出的难点是第二优化目标。

任意点分树计数：子树路径长度背包

赛时先看

题目信号：不是标准“重心分治”，中心可以任意选；不同点分树按每个点父亲是否相同区分；需要数所有递归删除点的结构。

本质：一棵树每次在当前连通块任选一个点作为分治中心，统计能产生多少棵不同的点分树。

接法： $f[u][k]$ 表示只考虑原树子树 u ，点分树根到 u 的路径长度恰好为 $k+1$ 的方案数。儿子 v 贡献 t 个路径点时，方案数取后缀和。

复杂度判定： $O(n^2)$ ，空间 $O(n^2)$ 级别以内，适合 $n \leq 3000$ 。

维护的量： $f[u][k]$ （原树子树 u 内、点分树根到 u 路径长恰好 $k+1$ 的方案数）； $sz[u]$ （子树大小）； C （组合数表）。

警告：固定原树根为 1 后，统计的是点分树根到固定点 v 的路径长度；合并儿子贡献时要乘组合数，表示不同子树贡献点交错出现的顺序。

【最小完整示例（先抄这一段就能跑）】

```

// 题目：树每次任选一点当分治中心递归删点，统计能形成多少棵不同的点分树 (mod 1e9+7)。
DecompositionTreeCounter dtc; // 构造计数结构
dtc.init(3); // 点数 3，顶点 1..3
dtc.add_edge(1, 2); // 加无向边（原树）
dtc.add_edge(2, 3);
int ans = dtc.solve(); // 不同点分树数量
// 样例：链 1-2-3 -> ans = 5

```

【传参要求（照这个传不会错）】

- `init(int n_)`: $n_$ 为点数，顶点编号 $1..n_$ ；在加边前调用。
- `add_edge(int u, int v)`: 加入一条无向边， $1 \leq u, v \leq n$ 。
- `solve()`: 执行主算法（固定原树根 1 做 dfs），返回 `int`：模 $1e9+7$ 下的点分树计数。

【API / 入口函数（赛时只认这里列的名字）】

- `add_edge(int u, int v)` -> 加入一条边
- `init(int n_)` -> 初始化/清空结构

- `solve()` -> 执行主算法并返回答案

【改板时先认这几个量】

- `g`: 邻接表。
- `sz`: 集合/子树大小。

```
struct DecompositionTreeCounter {
    static constexpr int MOD = 1000000007;
    int n = 0;
    vector<vector<int>> g;
    vector<vector<int>> f;
    vector<int> sz;
    vector<vector<int>> C;

    void addmod(int& x, long long y) {
        x = (x + y) % MOD;
    }

    void init(int n_) {
        n = n_;
        g.assign(n + 1, {});
        f.assign(n + 1, {});
        sz.assign(n + 1, 0);
        C.assign(n + 1, vector<int>(n + 1, 0));
        for (int i = 0; i <= n; ++i) {
            C[i][0] = C[i][i] = 1;
            for (int j = 1; j < i; ++j) {
                C[i][j] = C[i - 1][j - 1] + C[i - 1][j];
                if (C[i][j] >= MOD) C[i][j] -= MOD;
            }
        }
    }

    void add_edge(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void dfs(int u, int parent) {
        sz[u] = 1;
        vector<int> dp(1, 1);

        for (int v : g[u]) {
            if (v == parent) continue;
            dfs(v, u);

            int m = sz[v];
            vector<int> suffix(m + 1, 0);
            for (int i = m - 1; i >= 0; --i) {
                suffix[i] = suffix[i + 1] + f[v][i];
                if (suffix[i] >= MOD) suffix[i] -= MOD;
            }

            // take[t]: 儿子子树在通向 u 的路径上贡献 t 个点。
            vector<int> take(m + 1, 0);
            take[0] = suffix[0];
            for (int t = 1; t <= m; ++t) take[t] = suffix[t - 1];

            vector<int> ndp(dp.size() + m, 0);
            for (int a = 0; a < (int)dp.size(); ++a) {
                if (dp[a] == 0) continue;
                for (int t = 0; t <= m; ++t) {
                    if (take[t] == 0) continue;
                    long long ways = 1LL * dp[a] * take[t] % MOD * C[a + t][a] % MOD;
                    addmod(ndp[a + t], ways);
                }
            }
            dp.swap(ndp);
            sz[u] += sz[v];
        }
        f[u] = move(dp);
    }

    int solve() {
        dfs(1, 0);
        int ans = 0;
        for (int x : f[1]) addmod(ans, x);
        return ans;
    }
};
```

典题：Round4 K 《Decomposition

Trees》。看到“递归任选一个点删除形成树”时，可以先固定一个原树点，统计分治树根到它的路径。

区间消除长度：极值折线线段树

赛时先看

题目信号：合法删除区间 $[l, r]$

要求内部所有值都在两个端点值之间；询问区间答案和最小值、最大值第一次/最后一次出现位置有关。

本质：维护序列单点修改和区间查询，查询区间经过“端点包住内部值即可删除内部”的最小剩余长度。

复杂度判定：题解做法 $O((n+q) \log^2 n)$ 。

维护的量：线段树节点维护四条 LOW/HIGH 折线

$F(L, \alpha)$ 、 $F(L, \gamma)$ 、 $F(\beta, R)$ 、 $F(\delta, R)$ ，外加区间极值首末位置 $\text{firstMin}/\text{lastMax}/\text{firstMax}/\text{lastMin}$ 。

警告：区间答案可由两种极值方向取最小： $\text{firstMin} \leq \text{lastMax}$ 和 $\text{firstMax} \leq \text{lastMin}$ ；每个线段树节点不是只存答案，还要存四条折线。

- 对区间 $[L, R]$ ，记最小值 mn 、最大值 mx 。
- $\alpha = \text{firstMin}$ ， $\beta = \text{lastMax}$ ， $\gamma = \text{firstMax}$ ， $\delta = \text{lastMin}$ 。
- 若 $\alpha \leq \beta$ ，候选为 $F(L, \alpha) + F(\beta, R)$ 。
- 若 $\gamma \leq \delta$ ，候选为 $F(L, \gamma) + F(\delta, R)$ 。
- 线段树节点维护 $F(L, \alpha)$ 、 $F(L, \gamma)$ 、 $F(\beta, R)$ 、 $F(\delta, R)$ 四条 LOW/HIGH 折线。
- 合并左右儿子时，用可持久化 Treap 支持查找可替代前缀、切掉前缀、拼接序列。

典题：Round4 G 《Fading

Memories》。这个模板工程量很大，适合作为“知道该怎么维护”的复盘卡；真赛场复制请优先参考本场 `stds/G.cpp`。

凸多边形缩放内切圆覆盖：边 Voronoi + 二分

赛时先看

题目信号：圆必须完全位于凸多边形内；圆半径由到所有边的最小距离决定；缩放只会把中轴线和半径同比例放大。

本质：求最小缩放系数 λ ，使凸多边形 λA 内存在一个圆，覆盖给定带权点总权至少 W 。

复杂度判定：预处理边 Voronoi 约 $O(n^2 \log n)$ ；每次二分检查约 $O(nm \log m)$ 。

维护的量：边 Voronoi 的中轴线段（端点 a, b 及端点最大内切圆半径 ra, rb ）；二分变量 λ ；检查时按 t in $[0, 1]$ 维护带权点的覆盖事件。

警告：原点必须严格在凸多边形内部，这样每条边半平面可写成 $u \cdot x \leq h$ 且 $h > 0$ ；固定 λ 时只需考虑极大内切圆的圆心，它一定在边 Voronoi / 中轴线上。

- 对每条边 i 建它的 Voronoi cell：同时满足在原凸多边形内，以及 $d_i(x) \leq d_j(x)$ 。
- 对每个 cell 的边界，如果来自 $d_i = d_j$ ，就是一段中轴线。
- 记录中轴线段两端点 a, b 和端点最大内切圆半径 ra, rb 。
- 二分 λ 。对每条中轴线段参数 t in $[0, 1]$ ，圆心和半径都是一次函数。
- 对每个带权点解一个关于 t 的二次不等式，得到覆盖区间；扫描事件看权值和是否达到 W 。

典题：Round4 L 《Geometry》。如果凸多边形变成圆或矩形，边 Voronoi

可以大幅简化；先看特殊形状再上半平面交。

赛后模型总览：2026 河南萌新联赛第（一）场

赛时先看

题目信号：刚 VP / 补 2026

河南萌新联赛第（一）场；题目明显是基础模型训练，但赛场上容易因为边界和实现细节翻车。

本质：把本场 13 题映射到可复用板子，后续补题时先从这里定位模型。

复杂度判定：这是索引页，不涉及算法复杂度。

警告：A/I 更偏纯模拟或签到；真正建议长期复用的是 B/C/D/E/F/G/H/J/K/L/M 的模型。

题型归档：

- A 《萌新大富翁》：复杂规则模拟，维护行动队列、玩家状态、地产归属和破产释放。
- B 《缝缝补补》：二阶差分 gcd，判断“加减同一个 x 后可变等差数列”。
- C 《最大公约数》：统计 $\text{gcd}(l, r) == \min(l, r)$ 的子数组。

- D《走迷宫》：BFS 状态分层，位置 + 是否已经破墙。
- E《完美灵丝》：恰好 K 种不同元素 + 极差不超过 D ，双指针和单调队列。
- F《闯关游戏》：小根堆 Kahn，输出字典序最小拓扑序。
- G《强迫症》：区间减一到 0 的最少次数，累加正差分。
- H《神秘整数》：所有数模 k 余数相同，转化为差值 gcd 的约数。
- I《AI 训练》：整数模拟 $x = x * 9 / 10$ 。
- J《灵魂之塔》：删除一个元素后相邻奇偶交替。
- K《填魔法》：网格相邻不同染色，逐格 m 进制状态 DP。
- L《小明的丹药 easy》：权值 $1/2$ 的两侧可达和最大公共值。
- M《小明的丹药 hard》：权值 $2/3$ 的两侧可达和最大公共值，等价于小权值有界可达性。

二阶差分 gcd：加减同一个 x 变等差数列

赛时先看

题目信号：操作是“单点加减同一个 x 的任意倍”；最终要求等差数列；问最大 x 或判断某个 x 是否可行。

本质：每个位置都能加减若干次同一个正整数 x ，要求把序列变成等差数列，求最大可行 x 。

接法：如果固定 x ，要求相邻差分在模 x 意义下全相同；移项后就是所有二阶差分都被 x 整除。最大 x 就是二阶差绝对值的 gcd。

复杂度判定： $O(n \log V)$ ，只做 gcd。

维护的量：g（所有二阶差绝对值的 gcd）；循环变量 d2（当前位置的二阶差分 $a[i]-2a[i-1]+a[i-2]$ ）。

警告：不是对一阶差分直接求 gcd，而是对二阶差分 $a[i]-2a[i-1]+a[i-2]$ 求 gcd；若 gcd 为 0，说明原序列已经是等差数列，本题输出 -1。

【最小完整示例（先抄这一段就能跑）】

```
// 题目：每个位置可加减同一个 x 的任意倍，求把序列变等差数列的最大 x。
vector<i64> a = {1, 2, 5}; // 序列（任意长度）
i64 g = second_difference_gcd_for_arithmetic_mod(a);
cout << (g == 0 ? -1 : g) << '\n'; // g==0 说明原序列已是等差数列
// 样例：a={1,2,5} -> g=2; a={1,2,3} -> g=0, 输出 -1
```

【传参要求（照这个传不会错）】

- a：序列（任意长度），元素可为任意整数（函数内部取二阶差分的绝对值求 gcd）。
- 返回：i64，最大可行 x （二阶差绝对值的 gcd）；为 0 表示原序列已是等差数列，按题面输出 -1。

```
i64 second_difference_gcd_for_arithmetic_mod(const vector<i64>& a) {
    i64 g = 0;
    for (int i = 2; i < (int)a.size(); ++i) {
        // 二阶差分为 0 表示局部已经完全符合等差趋势。
        i64 d2 = a[i] - 2LL * a[i - 1] + a[i - 2];
        g = gcd(g, llabs(d2));
    }
    return g; // g == 0 时，整个序列已经是等差数列。
}

// 用法：
// 示例：i64 g = second_difference_gcd_for_arithmetic_mod(a);
// 示例：cout << (g == 0 ? -1 : g) << '\n';
```

典题：2026 河南萌新联赛第（一）场 B《缝缝补补》。

区间 gcd 等于区间最小值计数

赛时先看

题目信号：题面同时定义区间 gcd 和区间最小值，并问二者相等；数组元素为正数； n 到 $5e5/1e6$ ，不能枚举所有区间。

本质：统计正整数数组中满足 $\text{gcd}(a[l..r]) == \min(a[l..r])$ 的连续子数组数量。

接法：对每个位置 i 作为最左最小值，先用单调栈求它能管辖的区间范围；再用 gcd ST 表二分出向左/向右最多能扩到哪里且所有数仍被 $a[i]$ 整除，贡献为左右选择数乘积。

复杂度判定：建 gcd ST 表 $O(n \log n)$ ，每个位置二分左右边界 $O(\log n)$ ，总 $O(n \log n)$ 。

维护的量：st（gcd

稀疏表）；prev_le/next_less（单调栈给出的左右管辖界）；left_good/right_good（二分出的、gcd 仍等于 $a[i]$ 的最远扩展点）。

警告：gcd $\leq \min$ 恒成立，但相等要求区间内所有数都能被最小值整除；最小值重复时必须避免重复计数。这里把每个区间归给“最左侧的最小值位置”：左边界用前一个 $\leq a[i]$ ，右边界用后一个 $< a[i]$ 。

【最小完整示例（先抄这一段就能跑）】

```
// 题目：统计正整数数组中满足 gcd(a[l..r]) == min(a[l..r]) 的连续子数组数量。
vector<int> a = {2, 4, 6}; // 数组 (0-indexed)
i64 ans = count_subarrays_gcd_equal_min(a);
// 样例：a={2,4,6} -> 满足的子数组有 [2], [4], [6], [2,4], [2,4,6] 共 5 个，ans = 5
```

【传参要求（照这个传不会错）】

- a: 正整数数组 (0-indexed, 可为空)；内部 GcdSparseTable 同样按 0-indexed 建表与查询。
- 返回: i64, 满足 gcd(a[l..r]) == min(a[l..r]) 的子数组总数 (可能超出 int 范围, 用 i64)。

【API / 入口函数（赛时只认这里列的名字）】

- build(const vector<int>& a) -> 完成建树或预处理
- query(int l, int r) -> 查询 返回 int。

```
struct GcdSparseTable {
    int n = 0;
    vector<int> lg;
    vector<vector<int>> st;

    GcdSparseTable() = default;
    explicit GcdSparseTable(const vector<int>& a) { build(a); }

    void build(const vector<int>& a) {
        n = (int)a.size();
        lg.assign(n + 1, 0);
        for (int i = 2; i <= n; ++i) lg[i] = lg[i >> 1] + 1;
        st.assign(lg[n] + 1, vector<int>(n));
        st[0] = a;
        for (int k = 1; k < (int)st.size(); ++k) {
            int len = 1 << k;
            for (int i = 0; i + len <= n; ++i) {
                st[k][i] = gcd(st[k - 1][i], st[k - 1][i + (len >> 1)]);
            }
        }
    }

    int query(int l, int r) const {
        assert(0 <= l && l <= r && r < n);
        int k = lg[r - l + 1];
        return gcd(st[k][l], st[k][r - (1 << k) + 1]);
    }
};

i64 count_subarrays_gcd_equal_min(const vector<int>& a) {
    int n = (int)a.size();
    GcdSparseTable gst(a);

    vector<int> prev_le(n), next_less(n), stk;
    for (int i = 0; i < n; ++i) {
        // 栈顶保留离 i 最近的 <= a[i] 的位置，用来保证 i 是最左最小值。
        while (!stk.empty() && a[stk.back()] > a[i]) stk.pop_back();
        prev_le[i] = stk.empty() ? -1 : stk.back();
        stk.push_back(i);
    }

    stk.clear();
    for (int i = n - 1; i >= 0; --i) {
        // 右侧只遇到严格更小才停止；相等的最小值仍归给更左的位置。
        while (!stk.empty() && a[stk.back()] >= a[i]) stk.pop_back();
        next_less[i] = stk.empty() ? n : stk.back();
        stk.push_back(i);
    }

    i64 answer = 0;
    for (int i = 0; i < n; ++i) {
        int v = a[i];

        int lo = prev_le[i] + 1, hi = i, left_good = i;
        while (lo <= hi) {
            int mid = (lo + hi) >> 1;
            if (gst.query(mid, i) == v) {
                left_good = mid;
                hi = mid - 1;
            } else {
                lo = mid + 1;
            }
        }

        lo = i, hi = next_less[i] - 1;
        int right_good = i;
    }
}
```

```

while (lo <= hi) {
    int mid = (lo + hi) >> 1;
    if (gst.query(i, mid) == v) {
        right_good = mid;
        lo = mid + 1;
    } else {
        hi = mid - 1;
    }
}

answer += 1LL * (i - left_good + 1) * (right_good - i + 1);
}
return answer;
}

```

典题：2026 河南萌新联赛第（一）场 C《最大公约数》。

分层状态 BFS：一次破墙网格最短路

赛时先看

题目信号：每一步代价相同；题面说“有一次机会”改变通行规则；到达同一个格子时，是否已经用过机会会影响后续路径。

本质：网格最短路中有一次特殊通行机会，例如最多破一面墙、最多用一次钥匙、最多走一次特殊格。

接法：状态写成 $(x, y, used)$ 。走普通格保持 `used`；走墙且 `used==0` 时转成 `used=1`；答案取终点两层距离的较小值。

复杂度判定：状态数翻倍， $O(nm)$ 。

维护的量：`dist[x][y][used]`：到 (x, y) 且破墙状态为 `used`（0 未破 / 1 已破）的最短步数；`sx, sy` / `tx, ty`：起点/终点坐标。

警告：访问数组必须开第三维 `used`；不能用一个二维 `dist` 把“还没破墙”和“已经破墙”的状态合并。

【最小完整示例（先抄这一段就能跑）】

网格里含 'S' 起点、'T' 终点、'#' 墙，最多破一面墙，求起点到终点的最短步数：

```

vector<string> grid = {"S.#", "...", "#T."};
int ans = shortest_path_break_one_wall(grid); // 入口，返回最短步数
cout << ans << '\n'; // 样例输出：3

```

【传参要求（照这个传不会错）】

- `grid`: n 行字符串（0-indexed），字符只含 'S'/'T'/'#'/ '.'，保证恰好一个 'S' 一个 'T'；'#' 墙每格至多破一次。
- 返回值: `int`，S 到 T 的最短步数；不可达返回 -1。

```

int shortest_path_break_one_wall(const vector<string>& grid) {
    int n = (int)grid.size();
    int m = (int)grid[0].size();
    const int INF = 0x3f3f3f3f;
    vector<vector<int>>(n, vector<int>(m, INF));
    queue<tuple<int, int, int>> q;

    int sx = -1, sy = -1, tx = -1, ty = -1;
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) {
            if (grid[i][j] == 'S') sx = i, sy = j;
            if (grid[i][j] == 'T') tx = i, ty = j;
        }
    }

    dist[sx][sy][0] = 0;
    q.push({sx, sy, 0});
    int dx[4] = {1, -1, 0, 0};
    int dy[4] = {0, 0, 1, -1};

    while (!q.empty()) {
        auto [x, y, used] = q.front();
        q.pop();
        for (int dir = 0; dir < 4; ++dir) {
            int nx = x + dx[dir], ny = y + dy[dir];
            if (nx < 0 || nx >= n || ny < 0 || ny >= m) continue;

            int nused = used;
            if (grid[nx][ny] == '#') {
                if (used) continue;
                nused = 1;
            }
            if (dist[nx][ny][nused] != INF) continue;
            dist[nx][ny][nused] = dist[x][y][used] + 1;
            q.push({nx, ny, nused});
        }
    }
}

```

```

    }
}

int answer = min(dist[tx][ty][0], dist[tx][ty][1]);
return answer == INF ? -1 : answer;
}

```

典题：2026 河南萌新联赛第（一）场 D《走迷宫》。

恰好 K 种且极差不超过 D 的子数组计数

赛时先看

题目信号：右端点右移时，种类数和极差都只会更不满足；左端点右移可以修复约束；问连续区间数量。

本质：统计连续子数组，要求不同值个数恰好为 K ，且区间最大值与最小值之差不超过 D 。

接法：写 `count_at_most(k)` 统计“不同值不超过 k 且极差不超过 D ”的区间数。窗口合法后，每个右端点贡献 `right-left+1`。

复杂度判定：哈希表期望 $O(n)$ ，两个单调队列总 $O(n)$ 。

维护的量：`freq`（窗口内各值出现次数）、`distinct`（当前不同值个数）、`maxq/minq`（窗口最大/最小值的单调队列，存下标）、`left/right`（窗口左右边界）。

警告：恰好 K 不要硬维护一个窗口，改成 `atMost(K) -`

`atMost(K-1)`；收缩左端点时，频次数组和两个单调队列都要同步过期。

【最小完整示例（先抄这一段就能跑）】

统计一个数组中“不同值个数恰好为 K 且最大值减最小值不超过 D ”的连续子数组个数：

```

vector<i64> a = {1, 2, 3, 4};
i64 ans = count_subarrays_exactly_k_distinct_and_range(a, 2, 3); // K=2, D=3
cout << ans << '\n'; // 样例输出: 3

```

【传参要求（照这个传不会错）】

- `a`：原数组，0-indexed，可空。
- `k`：要求的恰好种类数； $k \leq 0$ 时返回 0（内部用 `atMost` 相减实现）。
- `max_diff`：极差上界； $\text{max_diff} < 0$ 时返回 0。
- 返回值：`i64`，满足条件的子数组个数。

```

i64 count_subarrays_at_most_k_distinct_and_range(
    const vector<i64>& a, int k, i64 max_diff
) {
    if (k < 0 || max_diff < 0) return 0;
    unordered_map<i64, int> freq;
    freq.reserve(a.size() * 2 + 1);
    freq.max_load_factor(0.7F);

    deque<int> maxq, minq;
    int left = 0, distinct = 0;
    i64 answer = 0;

    for (int right = 0; right < (int)a.size(); ++right) {
        if (++freq[a[right]] == 1) ++distinct;

        while (!maxq.empty() && a[maxq.back()] <= a[right]) maxq.pop_back();
        while (!minq.empty() && a[minq.back()] >= a[right]) minq.pop_back();
        maxq.push_back(right);
        minq.push_back(right);

        while (distinct > k || a[maxq.front()] - a[minq.front()] > max_diff) {
            if (--freq[a[left]] == 0) {
                freq.erase(a[left]);
                --distinct;
            }
            if (maxq.front() == left) maxq.pop_front();
            if (minq.front() == left) minq.pop_front();
            ++left;
        }

        answer += right - left + 1;
    }
    return answer;
}

i64 count_subarrays_exactly_k_distinct_and_range(
    const vector<i64>& a, int k, i64 max_diff
) {
    return count_subarrays_at_most_k_distinct_and_range(a, k, max_diff)
        - count_subarrays_at_most_k_distinct_and_range(a, k - 1, max_diff);
}

```

}

典题：2026 河南萌新联赛第（一）场 E《完美灵丝》。

字典序最小拓扑排序

赛时先看

题目信号：边 $u \rightarrow v$ 表示 u 必须在 v 前；题面明确要求“编号尽量小”“字典序最小”。

本质：有向依赖图中输出一组合法顺序，并要求字典序最小；若有环则无解。

接法：Kahn 算法的队列换成 `priority_queue<int, vector<int>, greater<int>>`。最终序列长度小于 n 说明有环。

复杂度判定： $O((n+m) \log n)$ 。

维护的量：`indeg[v]` (v 当前剩余入度)、小根堆 `pq` (当前入度为 0 的候选点)、`order` (已确定的输出顺序)。

警告：普通队列只能得到任意拓扑序；要字典序最小必须把所有当前入度为 0 的点放进小根堆。

【最小完整示例（先抄这一段就能跑）】

n 个点 1-indexed，边 $u \rightarrow v$ 表示 u 必须先于 v ，输出字典序最小的拓扑序：

```
int n = 4;
vector<vector<int>> g = {{}, {2, 3}, {4}, {4}, {}}; // 1->2, 1->3, 2->4, 3->4
vector<int> order = lexicographically_smallest_topo(n, g);
for (int x : order) cout << x << ' '; // 样例输出: 1 2 3 4
```

【传参要求（照这个传不会错）】

- n : 点数，编号从 1 到 n 。
- g : 邻接表，`g[u]` 存 u 的所有出边 v ，顶点编号 $1..n$ 。
- 返回值: `vector<int>` 字典序最小拓扑序；若 `order.size() < n` 说明图有环。

【改板时先认这几个量】

- g : 邻接表。
- `indeg`: 入度。

```
vector<int> lexicographically_smallest_topo(int n, const vector<vector<int>>& g) {
    vector<int> indeg(n + 1), order;
    for (int u = 1; u <= n; ++u) {
        for (int v : g[u]) ++indeg[v];
    }

    priority_queue<int, vector<int>, greater<int>> pq;
    for (int i = 1; i <= n; ++i) {
        if (indeg[i] == 0) pq.push(i);
    }

    while (!pq.empty()) {
        int u = pq.top();
        pq.pop();
        order.push_back(u);
        for (int v : g[u]) {
            if (--indeg[v] == 0) pq.push(v);
        }
    }
    return order; // order.size() < n 表示有环。
}
```

典题：2026 河南萌新联赛第（一）场 F《闯关游戏》。

区间减一到 0 的最少操作次数

赛时先看

题目信号：操作是“选一段连续区间，整体加/减 1”；所有数非负；目标全变成 0。

本质：每次把一个连续区间内所有正数同时减少 1，求把整个非负数组变成 0 的最少操作数。

接法：把每个高度看成很多层。若 $a[i] > a[i-1]$ ，多出来的层必须从位置 i 新开操作；否则前面开的操作可以延续过来。

复杂度判定： $O(n)$ 。

维护的量：`previous` (上一个数的高度)、`answer` (已累计的“新开操作”层数，即所有正上升量之和)。

警告：答案可能超过 `int`；只累加正的上升量，不是累加所有差分绝对值。

【最小完整示例（先抄这一段就能跑）】

非负数组，每次选一个连续区间整体减 1，求把整个数组变全 0 的最少操作次数：

```
vector<i64> a = {2, 1, 3, 2};
i64 ans = min_operations_decrement_intervals_to_zero(a);
cout << ans << '\n'; // 样例输出: 4
```

【传参要求（照这个传不会错）】

- a: 非负整数数组，0-indexed，可空。
- 返回值: i64，最少操作次数（可能超过 int）。

```
i64 min_operations_decrement_intervals_to_zero(const vector<i64>& a) {
    i64 answer = 0;
    i64 previous = 0;
    for (i64 current : a) {
        if (current > previous) answer += current - previous;
        previous = current;
    }
    return answer;
}
```

典题：2026 河南萌新联赛第（一）场 G《强迫症》。

所有数模 k 余数相同：枚举差值 gcd 的约数

赛时先看

题目信号：题面出现“所有数除以 k 的余数相同”；要求输出所有合法 k。

本质：给一组整数，找所有 $k > 1$ ，使得它们对 k 取模后的余数相同。

接法：求 $g = \gcd(|a_i - a_0|)$ ，所有 g 的大于 1 的约数就是答案。

复杂度判定：求 gcd $O(n \log V)$ ，枚举约数 $O(\sqrt{g})$ 。

维护的量：g（所有数与 $a[0]$ 差的绝对值 gcd）、answer（g 的大于 1 的约数，升序去重）。

警告：条件不是 k 整除所有数，而是 k 整除所有差值；若差值 gcd 为 1 则无解。若所有数都相同，合法 k 无限多，需要按题面另行特判。

【最小完整示例（先抄这一段就能跑）】

给一组整数，输出所有 $k > 1$ ，使这些数对 k 取模的余数全部相同：

```
vector<i64> a = {2, 5, 8};
vector<i64> ks = same_remainder_moduli(a);
for (i64 k : ks) cout << k << ' '; // 样例输出: 3
```

【传参要求（照这个传不会错）】

- a: 整数数组（可为负，内部取绝对值），0-indexed，非空；传的是拷贝，不改原数组。
- 返回值: vector<i64>，升序去重后的所有合法 k；差值 gcd ≤ 1 时返回空。

```
vector<i64> same_remainder_moduli(vector<i64> a) {
    i64 g = 0;
    for (int i = 1; i < (int)a.size(); ++i) {
        g = gcd(g, labs(a[i] - a[0]));
    }
    if (g <= 1) return {};

    vector<i64> answer;
    for (i64 d = 2; d * d <= g; ++d) {
        if (g % d != 0) continue;
        answer.push_back(d);
        if (d * d != g) answer.push_back(g / d);
    }
    answer.push_back(g);
    sort(answer.begin(), answer.end());
    answer.erase(unique(answer.begin(), answer.end()), answer.end());
    return answer;
}
```

典题：2026 河南萌新联赛第（一）场 H《神秘整数》。

删除一个元素后相邻奇偶交替

赛时先看

题目信号：要求恰好删除一个元素；剩余序列有相邻条件；条件能用前缀合法、后缀合法和中间桥接判断。

本质：统计删掉一个位置后，剩余序列是否满足相邻奇偶性全部不同。

接法：pre[i] 维护 $0..i$ 是否交替，suf[i] 维护 $i..n-1$ 是否交替。枚举删除位置 i，只需检查左段、右段和 i-1 与 i+1 是否能接上。

复杂度判定： $O(n)$ 。

维护的量： `p[i]` (`a[i]` 的奇偶性 0/1)、`pre[i]` (前缀 `0..i` 是否交替)、`suf[i]` (后缀 `i..n-1` 是否交替)。

警告： 删除首尾时没有桥接边；长度 `0/1` 的剩余序列应视为合法。

【最小完整示例（先抄这一段就能跑）】

恰好删除一个元素，使剩余序列相邻元素奇偶性全部不同，统计可删的位置个数：

```
vector<i64> a = {1, 2, 3, 4};
int ans = count_deletions_make_parity_alternating(a);
cout << ans << '\n'; // 样例输出: 2
```

【传参要求（照这个传不会错）】

- `a`：整数数组（可为负，奇偶性按 `|a[i]|` 判），`0-indexed`。
- 返回值：`int`，可删除的位置个数；`n == 1` 时返回 `1`。

```
int count_deletions_make_parity_alternating(const vector<i64>& a) {
    int n = (int)a.size();
    if (n == 1) return 1;

    vector<int> p(n);
    for (int i = 0; i < n; ++i) p[i] = (int)(llabs(a[i]) & 1);

    vector<char> pre(n, true), suf(n, true);
    for (int i = 1; i < n; ++i) {
        pre[i] = pre[i - 1] && (p[i] != p[i - 1]);
    }
    for (int i = n - 2; i >= 0; --i) {
        suf[i] = suf[i + 1] && (p[i] != p[i + 1]);
    }

    int answer = 0;
    for (int i = 0; i < n; ++i) {
        bool left_ok = (i == 0) || pre[i - 1];
        bool right_ok = (i == n - 1) || suf[i + 1];
        bool bridge_ok = (i == 0 || i == n - 1) || (p[i - 1] != p[i + 1]);
        if (left_ok && right_ok && bridge_ok) ++answer;
    }
    return answer;
}
```

典题：2026 河南萌新联赛第（一）场 J《灵魂之塔》。

网格相邻不同染色：逐格 `m` 进制状态 DP

赛时先看

题目信号： 列数很小 `q <= 8`，颜色数小 `m <= 4`，行数可到 `100`；每个格子的合法性只依赖左边和上边。

本质： 统计 `p*q` 网格染色方案，要求上下左右相邻格颜色不同，且首行、末行可能被固定。

接法： 先把固定首行编码成初始状态；从第 2 行第 1

列开始逐格填。每填一个格子，就把状态中对应列替换成新颜色。最后读取固定末行状态的方案数。

复杂度判定： $O(p * q * m * m^q)$ ，空间 $O(m^q)$ 。

维护的量： `power[col]` (m^{col} 位权)、`total_states` (状态总数 m^q)、`dp[state]` (最近一行压缩状态 `state` 的方案数)、`head/tail` (首/末行编码)。

警告： 状态是最近一行的颜色，按 `m`

进制压缩；逐格转移时，当前格的上方颜色是旧状态的本列，左方颜色是已经更新过的前一列。`p == 1` 时直接判断首行是否等于末行。

【最小完整示例（先抄这一段就能跑）】

`p*q` 网格、`m` 种颜色（编号 `0..m-1`），相邻上下左右不同色且首末行固定，统计方案数：

```
GridColoringFixedRows solver(2, 2, 3); // p=2 行, q=2 列, 3 种颜色
int ans = solver.count({0, 1}, {1, 0}); // 首行 [0,1], 末行 [1,0]
cout << ans << '\n'; // 样例输出: 1
```

【传参要求（照这个传不会错）】

- 构造 `GridColoringFixedRows(p, q, colors)`：`p` 行数、`q` 列数（需很小，如 `<= 8`）、`colors` 颜色数（如 `<= 4`）。
- `count(first_row, last_row)`：首行/末行颜色数组，长度必须等于 `q`，且各自内部相邻颜色不能相同（否则断言失败）。
- 返回值：`int`，模 `998244353` 的方案数；`p == 1` 时首行等于末行返回 `1`，否则返回 `0`。

【改板时先认这几个量】

- up: 当前格上方的颜色（旧状态本列）。
- dp: DP 状态。

```
struct GridColoringFixedRows {
    static constexpr int MOD = 998244353;
    int p = 0, q = 0, colors = 0, total_states = 0;
    vector<int> power;

    GridColoringFixedRows(int p_, int q_, int colors_)
        : p(p_), q(q_), colors(colors_), power(q_ + 1, 1) {
        for (int i = 1; i <= q; ++i) power[i] = power[i - 1] * colors;
        total_states = power[q];
    }

    int get_digit(int state, int col) const {
        return state / power[col] % colors;
    }

    int set_digit(int state, int col, int value) const {
        int old = get_digit(state, col);
        return state + (value - old) * power[col];
    }

    int encode(const vector<int>& row) const {
        int state = 0;
        for (int col = 0; col < q; ++col) state += row[col] * power[col];
        return state;
    }

    bool valid_row(const vector<int>& row) const {
        for (int col = 1; col < q; ++col) {
            if (row[col] == row[col - 1]) return false;
        }
        return true;
    }

    int count(const vector<int>& first_row, const vector<int>& last_row) const {
        assert((int)first_row.size() == q && (int)last_row.size() == q);
        assert(valid_row(first_row) && valid_row(last_row));

        int head = encode(first_row);
        int tail = encode(last_row);
        if (p == 1) return head == tail ? 1 : 0;

        vector<int> dp(total_states), ndp(total_states);
        dp[head] = 1;

        for (int row = 2; row <= p; ++row) {
            for (int col = 0; col < q; ++col) {
                fill(ndp.begin(), ndp.end(), 0);
                for (int state = 0; state < total_states; ++state) {
                    if (!dp[state]) continue;
                    int up = get_digit(state, col);
                    int left = (col == 0 ? -1 : get_digit(state, col - 1));
                    for (int color = 0; color < colors; ++color) {
                        if (color == up || color == left) continue;
                        int next_state = set_digit(state, col, color);
                        ndp[next_state] += dp[state];
                        if (ndp[next_state] >= MOD) ndp[next_state] -= MOD;
                    }
                }
                dp.swap(ndp);
            }
        }
        return dp[tail];
    }
};
```

典题：2026 河南萌新联赛第（一）场 K《填魔法》。

两种小权值物品凑最大公共和

赛时先看

题目信号：题面只有两种小权值，例如 $1/2$ 或

$2/3$ ；问“两边一样多且总和最大”；数量上限不大，或权值很小可以枚举目标和。

本质：左右两侧各有两种权值的物品，每种有数量上限；要求两侧凑出相同元素和，且这个公共和最大。

接法：先写 `can_make_with_two_weights(sum, count1, weight1, count2, weight2)`

判断单侧是否可达；从两边总容量较小值往下枚举第一个两侧都可达的公共和。

复杂度判定：枚举目标和 $O(\min(\text{total_left}, \text{total_right}) * \log W)$ ，本模板里 W 极小，实际近似线性。

维护的量： `g`（两种权值的 gcd）、`low/high`（第一种物品用量的可行区间）、`upper`（两侧总容量较小值，枚举起点）。

警告： 答案不一定是两边总容量的较小值，因为奇偶性 / 模数可达性会卡住。例如权值只有 2 时凑不出奇数。

【最小完整示例（先抄这一段就能跑）】

两侧各有两种权值的物品（各给个数与权值），各取若干使两边元素和相等，求最大公共和：

```
i64 ans = maximum_equal_sum_two_sides(
    3, 2, 2, 1, // 左侧: 3 个权值 2, 2 个权值 1
    2, 2, 3, 1); // 右侧: 2 个权值 2, 3 个权值 1
cout << ans << '\n'; // 样例输出: 7
```

【传参要求（照这个传不会错）】

- 左侧四参: `left_a`（权值 `left_weight_a` 的物品个数）、`left_weight_a`、`left_b`（权值 `left_weight_b` 的个数）、`left_weight_b`；右侧四参同理；均为非负 `i64`。
- 返回值: `i64`，两侧可达的最大公共元素和；从总容量较小值往下找第一个两侧都可凑出的值，没有则返回 0。
- 辅助函数 `can_make_with_two_weights(target, cnt_a, weight_a, cnt_b, weight_b)`：判断单侧能否凑出 `target`，一般不用直接调。

【改板时先认这几个量】

- `g`：两种权值的 gcd。
- `low/high`：第一种物品使用数量的可行区间。

```
i64 exgcd_for_inverse(i64 a, i64 b, i64& x, i64& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    i64 x1, y1;
    i64 g = exgcd_for_inverse(b, a % b, x1, y1);
    x = y1;
    y = x1 - a / b * y1;
    return g;
}

i64 inverse_mod_coprime(i64 a, i64 mod) {
    i64 x, y;
    i64 g = exgcd_for_inverse(a, mod, x, y);
    assert(g == 1);
    x %= mod;
    if (x < 0) x += mod;
    return x;
}

bool can_make_with_two_weights(i64 target, i64 cnt_a, i64 weight_a, i64 cnt_b, i64 weight_b) {
    if (target < 0) return false;
    i64 total = cnt_a * weight_a + cnt_b * weight_b;
    if (target > total) return false;

    // 设使用 x 个 weight_a, 则 target - weight_a*x 必须由 weight_b 凑出。
    i64 low = max<i64>(0, (target - cnt_b * weight_b + weight_a - 1) / weight_a);
    i64 high = min(cnt_a, target / weight_a);
    if (low > high) return false;

    i64 g = gcd(weight_a, weight_b);
    if (target % g != 0) return false;

    i64 mod = weight_b / g;
    if (mod == 1) return true;

    i64 a = weight_a / g;
    i64 t = target / g;
    i64 need = t % mod * inverse_mod_coprime(a % mod, mod) % mod;

    // 找到区间 [low, high] 中第一个满足 x == need (mod) 的 x。
    i64 first = low + (need - low % mod + mod) % mod;
    return first <= high;
}

i64 maximum_equal_sum_two_sides(
    i64 left_a, i64 left_weight_a, i64 left_b, i64 left_weight_b,
    i64 right_a, i64 right_weight_a, i64 right_b, i64 right_weight_b
) {
    i64 upper = min(left_a * left_weight_a + left_b * left_weight_b,
        right_a * right_weight_a + right_b * right_weight_b);
    for (i64 target = upper; target >= 0; --target) {
        if (can_make_with_two_weights(target, left_a, left_weight_a, left_b, left_weight_b) &&
```

```
        can_make_with_two_weights(target, right_a, right_weight_a, right_b, right_weight_b)) {
            return target;
        }
    }
    return 0;
}
```

典题：2026 河南萌新联赛第（一）场 L/M《小明的丹药》。easy 调用权值 2/1，hard 调用权值 3/2，最终输出 2 * maximum_equal_sum_two_sides(...)。

赛前题源索引：码蹄杯 2022-2026 真题集

赛时先看

题目信号：准备码蹄杯国赛，想从近年真题中抽出常见算法和题型。

本质：赛前先知道这几年题库覆盖了什么，再按模型扫模板。

复杂度判定：人工扫题用；不是代码模板。

警告：官方题单能确认题号、标题、难度和提交统计；若没有公开题解或完整题面，不要把标题猜测当原题结论。

出处：码蹄集官方真题集抓取于 2026-07-29，官方题库接口为 /exam-back/pc/queryMatBeiQuestionBank.do 与 /exam-back/pc/queryOjProblemByTreeId.do。

年份	官方题库 ID	题量	赛前优先扫的关键词
2026	C98C14523F069FECB0DEED64F00CEAB0	63	图论判定、树上计数、构造、博弈、字符串/序列 DP
2025	305EE97B0D5E361DE6A28CD18C929AF0	90	括号序列、LCM、lowbit、二分图染色、回文、树上加和、期望、路径
2024	C2CBD34082148550EF198C50D10DBDC7	73	二分答案、迷宫、区间数据处理、字符串、图论、异或、构造
2023	16A92C42378232DEB56179D9C70DC45C	54	最短路、组合/计数、密码学、循环、图搜索、博弈
2022	C448715ED43BEA9D2D47CED523050945	80	项链/循环串、马走日、迷宫、水渠规划、铺砖、剪刀石头布

赛前扫题顺序：

1. 先扫本章下面这些短模型，因为它们和官方题单标题或公开题面强相关。
2. 再翻主目录中的对应大章：二分图染色在图论，括号和回文在字符串/DP，区间维护在数据结构，低位运算在位运算。
3. 如果现场读题发现只是换皮，不要急着改模板核心：先改输入转换、建图条件、状态定义和输出。

最大平均子段：长度至少 L 的二分答案

赛时先看

题目信号：题面问“平均值最大”“胜率/密度/单位收益最大”，同时对子段长度有下界；答案是实数或需要扩大若干倍后输出整数。

本质：求一个连续子段的最大平均值，且子段长度必须至少为 L。

接法：把本节 struct/class/函数组 整段抄到 solve() 上面；solve() 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定：0(n log 精度)，常用 60 次二分。

维护的量：pre[i] (a[0..i-1] - avg 的前缀和)、best_left_prefix (允许的最左前缀最小值)、二分边界 lo/hi。

警告：检查函数不是直接看原数组和，而是把每个数减去猜测平均值 mid，判断是否存在长度至少 L 的子段使变换后和非负；pre_min 只能取当前位置左侧 L 个位置及更早的最小前缀。

【最小完整示例（先抄这一段就能跑）】

求长度至少为 L 的连续子段的最大平均值（实数，60 次二分收敛）：

```
vector<i64> a = {1, 2, 3, 4};
double ans = max_average_subarray_at_least_len(a, 2); // 长度至少 2
cout << fixed << setprecision(6) << ans << '\n'; // 样例输出: 3.500000
```

【传参要求（照这个传不会错）】

- a: 非空整数数组，0-indexed。
- min_len: 子段长度下界，要求 1 <= min_len <= n（不满足会断言）。
- iterations: 二分迭代次数，默认 60。
- 返回值: double 最大平均值；要求平均值乘 1000 向下取整输出时用 (long long)(ans * 1000 + 1e-8)。

出处：码蹄杯 2024 官方题单 MC0304 拔河；公开题解将其归为二分最大平均值模型。

```
bool exists_average_at_least(const vector<i64>& a, int min_len, double avg) {
    int n = (int)a.size();
    vector<double> pre(n + 1, 0.0);

    // 把“平均值 >= avg”改写成“存在一段 b[i]=a[i]-avg 的和 >= 0”。
    for (int i = 1; i <= n; ++i) {
        pre[i] = pre[i - 1] + (double)a[i - 1] - avg;
    }

    double best_left_prefix = 0.0;
    for (int r = min_len; r <= n; ++r) {
        // 子段至少 min_len, 所以左端点最多到 r-min_len+1。
        // 前缀下标取到 r-min_len, 表示子段 [1,r] 的 1-1。
        best_left_prefix = min(best_left_prefix, pre[r - min_len]);
        if (pre[r] - best_left_prefix >= -1e-12) return true;
    }
    return false;
}

double max_average_subarray_at_least_len(const vector<i64>& a, int min_len, int iterations = 60) {
    assert(!a.empty());
    assert(1 <= min_len && min_len <= (int)a.size());

    double lo = (double)*min_element(a.begin(), a.end());
    double hi = (double)*max_element(a.begin(), a.end());

    for (int it = 0; it < iterations; ++it) {
        double mid = (lo + hi) / 2.0;
        if (exists_average_at_least(a, min_len, mid)) lo = mid;
        else hi = mid;
    }
    return lo;
}
```

怎么套题：如果题目要求输出平均值乘 1000 后向下取整，就令 $ans = \text{max_average_subarray_at_least_len}(a, L)$ ，最后输出 $(\text{long long})(ans * 1000 + 1e-8)$ 。如果长度是“恰好 L”，不用二分，直接滑动窗口；如果长度是“不超过 L”，这个模板不能直接套，要换成前缀最大/最小的窗口维护。

差分统计区间覆盖次数：重排后最大询问和

赛时先看

题目信号：题面给若干 $[l, r]$ ，每个位置被问到的次数不同；原数组元素可以重新排列；目标是总贡献最大或最小。

本质：有很多区间求和询问，但允许重排原数组，要求所有询问和总和最大。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；`solve()`

里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O((n+q) + n \log n)$ 。

维护的量：`diff[i]`（覆盖次数差分：`diff[1]+=1`、`diff[r+1]-=1`）、`cnt[i]`（位置 i 被询问次数）、`ans`（`i128` 加权和）。

警告：区间是 1-indexed 时差分要开 `n+2`；最大值把大数放到高次数位置，最小值把大数放到低次数位置；乘法可能超过 `i64` 时用 `i128`。

约定：`a` 使用 1-indexed: `a[1..n]` 是原数组，`a[0]` 不用

【最小完整示例（先抄这一段就能跑）】

数组可任意重排， q 个区间询问 $[l, r]$ ，重排后所有询问区间和的总和最大：

```
vector<i64> a = {0, 1, 2, 3}; // 1-indexed: a[1..3] 为 1,2,3
vector<pair<int, int>> q = {{1, 3}, {1, 3}}; // 两次全区间询问
i128 ans = max_total_query_sum_after_rearrange(a, q);
cout << to_string_i128(ans) << '\n'; // 样例输出: 12
```

【传参要求（照这个传不会错）】

- `a`: 1-indexed, `a[0]` 不用, `a[1..n]` 是原数组；函数内部会排序，请传拷贝。
- `queries`: $\{l, r\}$ 列表，要求 $1 \leq l \leq r \leq n$ （不满足会断言）。
- 返回值: `i128` 最大总和；输出必须用 `to_string_i128`。

出处：码蹄杯 2024 官方题单 MC0351 区间询问和；公开题解模型为差分统计覆盖次数 + 排序重排。

```
string to_string_i128(i128 x) {
    if (x == 0) return "0";
```

```

bool neg = x < 0;
if (neg) x = -x;
string s;
while (x > 0) {
    s.push_back(char('0' + x % 10));
    x /= 10;
}
if (neg) s.push_back('-');
reverse(s.begin(), s.end());
return s;
}

i128 max_total_query_sum_after_rearrange(vector<i64> a, const vector<pair<int, int>>& queries) {
    // a 使用 1-indexed: a[1..n] 是原数组, a[0] 不用。
    int n = (int)a.size() - 1;
    vector<i64> diff(n + 2, 0), cnt(n + 1, 0);

    for (auto [l, r] : queries) {
        // 每个询问 [l,r] 会让区间内每个位置的贡献次数 +1。
        assert(1 <= l && l <= r && r <= n);
        diff[l] += 1;
        diff[r + 1] -= 1;
    }

    for (int i = 1; i <= n; ++i) cnt[i] = cnt[i - 1] + diff[i];

    sort(a.begin() + 1, a.end());
    sort(cnt.begin() + 1, cnt.end());

    i128 ans = 0;
    for (int i = 1; i <= n; ++i) {
        ans += (i128)a[i] * cnt[i];
    }
    return ans;
}

```

怎么套题：如果题目说“数组不能重排，只问所有区间总和”，就只保留差分统计次数，再算 `sum a[i]*cnt[i]`，不要排序。如果要求总和最小，把 `a` 升序、`cnt` 降序配对即可。

二分图染色：距离冲突两组分配

赛时先看

题目信号：题面出现“所有点必须分完”“属于同一方的两点必须满足距离/关系限制”“只有两方可选”；冲突关系是无向的。

本质：把点分给两个人/两组，要求同组内任意两点不能冲突。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；`solve()`

里先构造对象，再按下方“不会用就看这里”调用公开接口。

复杂度判定：朴素建边 $O(n^2)$ ，染色 $O(n+m)$ ；若 `n` 很大再考虑网格哈希或扫描线优化建边。

维护的量：`color`（每个点的分组，-1 未分组、0/1 分别两方）；`g`（冲突边邻接表，1-based）；`dist2`（距离平方，与 `limit = d*d` 比较决定是否连边）。

警告：如果同组要求距离 $> d$ ，冲突边条件是距离 $\leq d$ ；坐标到 $1e9$ 时平方必须用 `i128`；图不连通也要逐块染色。

【最小完整示例（先抄这一段就能跑）】

题目：`n` 个哨岗坐标，分给两方，同组欧氏距离必须 $> d$ ，问能否分完（MC0421 分配哨岗）：

```

vector<MatijiPoint> pts;
for (int i = 0; i < n; ++i) { i64 x, y; cin >> x >> y; pts.push_back({x, y}); }
vector<int> color;
bool ok = split_points_into_two_groups_by_distance(pts, d, color); // 1. 调用
cout << (ok ? "YES" : "NO") << '\n'; // 2. true 输出 YES, false 输出 NO

```

【传参要求（照这个传不会错）】

- `split_points_into_two_groups_by_distance(points, d, color)`: `points` 是坐标列表（0-based, `points[i]` 对应输入第 `i+1` 个点）；`d` 是“同组距离必须大于”的阈值（`i64`）；`color` 传出分组结果（1-based 下标，-1 未分组、0/1 两方）；返回 `bool`：能分成两组返回 `true` 且 `color` 有效，否则返回 `false`。
- `color_conflict_graph(n, g, color)`：给现成的冲突图染色；`n` 是点数、`g` 是 1-based 邻接表；返回 `bool` 同上。非距离类冲突题（同色不能同姓/同班/同边相邻）只自己建 `g` 再调它，染色函数不改。

【改板时先认这几个量】

- `g`：邻接表。
- `dist`：距离。

出处：码蹄杯 2025 官方题单 MC0421 分配哨岗；题面为两方分配哨岗且同组欧氏距离必须大于 d 。

```
struct MatijiPoint {
    i64 x, y;
};

bool color_conflict_graph(int n, const vector<vector<int>>& g, vector<int>& color) {
    // color[u] = -1 表示未分组; 0/1 分别表示两方。
    color.assign(n + 1, -1);

    for (int start = 1; start <= n; ++start) {
        if (color[start] != -1) continue;

        queue<int> que;
        color[start] = 0;
        que.push(start);

        while (!que.empty()) {
            int u = que.front();
            que.pop();

            for (int v : g[u]) {
                if (color[v] == -1) {
                    color[v] = color[u] ^ 1;
                    que.push(v);
                } else if (color[v] == color[u]) {
                    return false;
                }
            }
        }
    }
    return true;
}

bool split_points_into_two_groups_by_distance(const vector<MatijiPoint>& points, i64 d, vector<int>& color) {
    int n = (int)points.size();
    vector<vector<int>> g(n + 1);
    i128 limit = (i128)d * d;

    for (int i = 0; i < n; ++i) {
        for (int j = i + 1; j < n; ++j) {
            i128 dx = (i128)points[i].x - points[j].x;
            i128 dy = (i128)points[i].y - points[j].y;
            i128 dist2 = dx * dx + dy * dy;

            // 同组要求“距离必须大于 d”。
            // 所以 dist <= d 的两点不能同组，必须连冲突边。
            if (dist2 <= limit) {
                int u = i + 1;
                int v = j + 1;
                g[u].push_back(v);
                g[v].push_back(u);
            }
        }
    }
    return color_conflict_graph(n, g, color);
}
```

怎么套题：points[i] 按输入顺序存第 $i+1$ 个哨岗；函数返回 true 就输出 YES，返回 false 就输出 NO。如果题目不是距离，而是“同色不能同姓/同班/同边相邻”，只改建 g 的条件，染色函数不改。

最小公倍数安全累积：上界剪枝

赛时先看

题目信号：题面出现周期共同对齐、若干事件同时发生、lcm 可能很大、答案超过 $1e18$ 或超过某个 limit 就不需要继续。

本质：多个数求 LCM，同时判断是否已经超过题目上界，避免 i64 溢出。

接法：把本节 struct/class/函数组 整段抄到 solve() 上面；solve() 里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定： $O(n \log V)$ 。

维护的量：out（当前累积 LCM）、limit（上界）、g（gcd(a,b)）、reduced（a/g，先除再乘防溢出）。

警告：LCM 必须先除 gcd 再乘； $a/g > \text{limit}/b$ 时已经超过上界，不要真的乘；若数组里可能有 0，要按题意单独定义。

【最小完整示例（先抄这一段就能跑）】

给一组数求 LCM，一旦超过上界 limit 立即返回 false，避免 i64 溢出：

```
vector<i64> v = {6, 10, 15};
```

```
i64 ans;
bool ok = lcm_all_limited(v, (i64)1e18, ans);
cout << ok << ' ' << ans << '\n'; // 样例输出: 1 30
```

【传参要求（照这个传不会错）】

- `lcm_all_limited(values, limit, out)`: `values` 任意个整数（含 0，此时 `out=0` 且返回 `true`）、`limit >= 0`；成功返回 `true` 且 `out` 为累积 LCM，超限返回 `false` 且 `out` 无效。
- `lcm_limited(a, b, limit, out)`：单步合并两个数；`a/b` 内部取绝对值，含 0 时 `out=0` 返回 `true`。

出处：码蹄杯 2025 官方题单 MC0406 最小公倍数；该模型也是周期题常用安全工具。

```
bool lcm_limited(i64 a, i64 b, i64 limit, i64& out) {
    // 返回 true: lcm(a,b) 没有超过 limit, 并写入 out。
    // 返回 false: lcm(a,b) 已经超过 limit, out 的值不要再用。
    assert(limit >= 0);
    a = llabs(a);
    b = llabs(b);
    if (a == 0 || b == 0) {
        out = 0;
        return true;
    }

    i64 g = std::gcd(a, b);
    i64 reduced = a / g;
    if (reduced > limit / b) return false;

    out = reduced * b;
    return out <= limit;
}

bool lcm_all_limited(const vector<i64>& values, i64 limit, i64& out) {
    out = 1;
    for (i64 x : values) {
        i64 next_lcm = 0;
        if (!lcm_limited(out, x, limit, next_lcm)) return false;
        out = next_lcm;
    }
    return true;
}
```

怎么套题：如果题目只关心 LCM 是否超过 `1e18`，调用 `lcm_all_limited(a, (i64)4e18, ans)`；如果题目给了目标周期 `M`，把 `limit` 设成 `M`，一旦返回 `false` 就说明后续不会再等于 `M` 以内的答案。

lowbit 拆分与最低位博弈速查

赛时先看

题目信号：题面直接出现 `lowbit`、`x & -x`、二进制最低位、每次加/减最低位、按能被 2^k 整除分类。

本质：围绕最低位的 1 做分层统计、拆数、模拟，或处理每步只能按 `lowbit(x)` 改变数值的小游戏。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；`solve()`

里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定：单次最低位查询 $O(1)$ ；把一个数按 `lowbit` 不断拆到 0 的复杂度是 $O(\text{popcount}(x))$ 。

维护的量：`parts`（按 `lowbit` 拆出的 2 的幂列表）、`lb`（当前最低位 `x & -x`）；博弈只依赖 `popcount(x)` 的奇偶。

警告：`lowbit(0)=0`，不能再拿去做除数；有符号负数的位运算容易出坑，模板统一用 `ui64`；“每步减 `lowbit`”这个博弈只有在题面明确如此时才可用。

【最小完整示例（先抄这一段就能跑）】

`lowbit` 速查：取最低位、最低位指数、按 `lowbit` 拆成 2 的幂、每步只能减 `lowbit` 的博弈胜负：

```
ui64 x = 13; // 13 = 0b1101
cout << lowbit_u64(x) << '\n'; // 样例输出: 1
cout << lowbit_exponent(x) << '\n'; // 样例输出: 0
vector<ui64> parts = split_by_lowbit(x); // [1, 4, 8]
cout << first_win_subtract_lowbit_game(x) << '\n'; // 样例输出: 1 (3 个二进制 1, 先手赢)
```

【传参要求（照这个传不会错）】

- `lowbit_u64(x)`：返回最低位的 2 的幂；`x=0` 时返回 0（不能当除数）。
- `lowbit_exponent(x)`：返回最低位指数 e ($0..63$)；`x=0` 时返回 -1。
- `split_by_lowbit(x)`：返回从低位到高位 2 的幂列表；`x=0` 返回空。
- `first_win_subtract_lowbit_game(x)`：每步只能把 `x` 变成 `x-lowbit(x)`，返回先手是否必胜（`popcount` 为奇数）；仅当每步选择唯一时可用。

出处：码蹄杯 2025 官方题单 MC0407 lowbit神功、MC0437 lowbit博弈；码蹄杯 2024 官方题单 MC0362 异或。

```
ui64 lowbit_u64(ui64 x) {
    return x & (~x + 1);
}

int lowbit_exponent(ui64 x) {
    // 返回 lowbit(x) = 2^e 中的 e。
    // x=0 没有最低位的 1，返回 -1 方便外面判掉。
    if (x == 0) return -1;
    return __builtin_ctzll(x);
}

vector<ui64> split_by_lowbit(ui64 x) {
    // 把 x 拆成若干个 2 的幂，顺序从低位到高位。
    // 例如 x=13(1101) -> [1, 4, 8]。
    vector<ui64> parts;
    while (x) {
        ui64 lb = lowbit_u64(x);
        parts.push_back(lb);
        x -= lb;
    }
    return parts;
}

bool first_win_subtract_lowbit_game(ui64 x) {
    // 仅适用于规则：每一步必须把 x 改成 x-lowbit(x)，不能操作者输。
    // 每一步恰好删除一个二进制 1，所以总步数等于 popcount(x)。
    return (__builtin_popcountll(x) & 1) != 0;
}
```

怎么套题：如果题面是树状数组式 $i += \text{lowbit}(i)$ 或 $i -= \text{lowbit}(i)$ ，翻树状数组；如果题面是按 lowbit 给贡献，先用 $\text{lowbit_exponent}(x)$ 分到 $0..63$ 层；如果是博弈，必须先确认每步选择是否唯一，唯一才可以用上面的奇偶结论。

括号序列：线段树维护总和与前缀最小值

赛时先看

题目信号：（记为 +1，）记为 -1；合法条件是总和为 0 且任意前缀和不小于 0；有单点修改、翻转或区间询问。

本质：动态修改括号串后，判断整串或某个区间是否为合法括号序列。

接法：把本节 struct/class/函数组 整段抄到 `solve()` 上面；外部只调用下面 API 列出的接口，不要把内部递归参数直接当题面参数。

复杂度判定：建树 $O(n)$ ，单点修改和区间查询 $O(\log n)$ 。

维护的量：`s` (1-indexed 括号串副本)；`tr[p]` (线段树第 p 号节点区间的 `BracketNode{sum, min_pref}`：括号和与最小前缀和)。

警告：判断子串合法时，不能直接看全局前缀；要查询区间内部的相对

`sum/min_prefix`：合并两个节点时右段最小前缀要加左段总和。

约定：`str` 使用普通 0-indexed 字符串；模板内部转成 1-indexed；把第 `pos` 个字符改成 `c`，`pos` 是 1-indexed；查询子串 `[l, r]` 的相对总和和相对最小前缀，`l/r` 是 1-indexed

【最小完整示例（先抄这一段就能跑）】

题目：串 `((()())((`，翻转第 8 个字符后问 `[1, 8]` 是否合法括号序列。

```
BracketSegTree seg;
seg.build("((()())(("); // 1. 建树，传普通 0-indexed 字符串
seg.flip(8); // 2. 翻转第 8 个括号 (1-indexed)
cout << seg.valid_range(1, 8) << '\n'; // 3. 问 [1, 8] 是否合法 (1-indexed)
```

样例：输出 `1` (翻转后串变 `((()())()`，合法)。

【传参要求（照这个传不会错）】

- `build(str)`：传 0-indexed 括号串（只含 '(' / ')'）；无返回值，建完树才能查。
- `flip(pos)`：把第 `pos` 个括号翻转 ('(' <-> ')')，`pos` 是 1-indexed (1..n)；`set_char(pos, c)`：把第 `pos` 个字符改成 `c`，同样 1-indexed。
- `query(l, r)`：`l/r` 为 1-indexed ($1 \leq l \leq r \leq n$)，返回 `BracketNode{sum, min_pref}` (区间相对总和、相对最小前缀)。
- `valid_range(l, r)`：问子串 `[l, r]` 是否合法括号序列，返回 `bool`；`valid_whole()`：问整串是否合法。
- 判断区间合法性用的是区间内部的“相对”最小前缀，不能拿全局前缀来比。

【API / 入口函数（赛时只认这里列的名字）】

- `build(const string& str)` → 完成建树或预处理
- `query(int l, int r)` → 查询 返回 `BracketNode`。

【改板时先认这几个量】

- `tr`: 线段树节点数组。
- `sum`: 区间括号和 ((为 +1、) 为 -1)。

出处：码蹄杯 2025 官方题单 MC0402 括号序列；括号序列也是 XCPC 常见动态维护模型。

```
struct BracketNode {
    int sum = 0; // 这一段 '(' 记 +1、')' 记 -1 后的总和。
    int min_pref = 0; // 从这一段左端开始走，过程中出现过的最小前缀和。
};

BracketNode merge_bracket_node(const BracketNode& left, const BracketNode& right) {
    BracketNode res;
    res.sum = left.sum + right.sum;
    res.min_pref = min(left.min_pref, left.sum + right.min_pref);
    return res;
}

struct BracketSegTree {
    int n = 0;
    string s;
    vector<BracketNode> tr;

    int bracket_value(char c) {
        return c == '(' ? 1 : -1;
    }

    void build(const string& str) {
        // str 使用普通 0-indexed 字符串；模板内部转成 1-indexed。
        n = (int)str.size();
        s = " " + str;
        tr.assign(4 * n + 4, {});
        if (n == 0) return;
        build(1, 1, n);
    }

    void build(int p, int l, int r) {
        if (l == r) {
            int v = bracket_value(s[l]);
            tr[p] = {v, min(0, v)};
            return;
        }
        int mid = (l + r) >> 1;
        build(p << 1, l, mid);
        build(p << 1 | 1, mid + 1, r);
        tr[p] = merge_bracket_node(tr[p << 1], tr[p << 1 | 1]);
    }

    void set_char(int pos, char c) {
        // 把第 pos 个字符改成 c, pos 是 1-indexed。
        assert(1 <= pos && pos <= n);
        s[pos] = c;
        set_char(1, 1, n, pos, c);
    }

    void flip(int pos) {
        // 把第 pos 个括号翻转: '(' <-> ')'。
        assert(1 <= pos && pos <= n);
        set_char(pos, s[pos] == '(' ? ')' : '(');
    }

    void set_char(int p, int l, int r, int pos, char c) {
        if (l == r) {
            int v = bracket_value(c);
            tr[p] = {v, min(0, v)};
            return;
        }
        int mid = (l + r) >> 1;
        if (pos <= mid) set_char(p << 1, l, mid, pos, c);
        else set_char(p << 1 | 1, mid + 1, r, pos, c);
        tr[p] = merge_bracket_node(tr[p << 1], tr[p << 1 | 1]);
    }

    BracketNode query(int l, int r) {
        // 查询子串 [l, r] 的相对总和和相对最小前缀, l/r 是 1-indexed。
        assert(1 <= l && l <= r && r <= n);
        return query(1, 1, n, l, r);
    }
};
```

```

}

BracketNode query(int p, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) return tr[p];
    int mid = (l + r) >> 1;
    if (qr <= mid) return query(p << 1, l, mid, ql, qr);
    if (ql > mid) return query(p << 1 | 1, mid + 1, r, ql, qr);
    return merge_bracket_node(
        query(p << 1, l, mid, ql, qr),
        query(p << 1 | 1, mid + 1, r, ql, qr)
    );
}

bool valid_range(int l, int r) {
    BracketNode res = query(1, r);
    return res.sum == 0 && res.min_pref >= 0;
}

bool valid_whole() {
    return n == 0 || (tr[1].sum == 0 && tr[1].min_pref >= 0);
}
};

```

怎么套题：读入字符串 `s` 后 `BracketSegTree seg; seg.build(s);`。翻转第 `pos` 个括号调用 `seg.flip(pos);`。问 `[l,r]` 是否合法调用 `seg.valid_range(l,r)`。如果题目只问静态最长合法括号，翻 D 章 09「括号序列合法性与最长合法括号」的栈/DP 模板。

恢复回文与回文子序列：区间 DP 可恢复方案

赛时先看

题目信号：题面说可把 `?` 替换成字母、要求正反相同；或者允许删除字符，求最长仍为回文的子序列。

本质：处理两类常见回文题：把含 `?` 的字符串恢复成回文；或求最长回文子序列并恢复一个方案。

接法：把本节 `struct/class/函数组` 整段抄到 `solve()` 上面；`solve()`

里直接按下方“不会用就看这里”调用，不需要自己猜内部参数。

复杂度判定：问号恢复 $O(n)$ ；最长回文子序列恢复 $O(n^2)$ 时间和空间。

维护的量：问号恢复无需结构；LPS 用 `dp[i][j]`（子串 `s[i..j]` 的最长回文子序列长度）与 `left/right`（恢复方案的左右两半）。

警告：回文子串必须连续，回文子序列可以跳过字符；`?` 恢复和 LPS 是两个不同模型，不要混用。

【最小完整示例（先抄这一段就能跑）】

题目：把 `a?c?a` 恢复成回文；并求 `bbbab` 的最长回文子序列及一个方案。

```

cout << restore_question_mark_palindrome("a?c?a") << '\n'; // 1. 问号恢复，成对 ? 默认填 'a'
auto [len, pal] = longest_palindromic_subsequence_with_answer("bbbab"); // 2. LPS 长度 + 方案
cout << len << ' ' << pal << '\n'; // 3. 输出

```

样例：第一行 `aacaa`；第二行 `4 bbbb`。

【传参要求（照这个传不会错）】

- `restore_question_mark_palindrome(s, fill = 'a')`：`s` 是含 `?` 的原串（0-indexed）；`fill` 是成对 `?` 的填充字符；返回恢复后的回文串；无解返回空串 `""`。
- `longest_palindromic_subsequence_with_answer(s)`：传原串（0-indexed）；返回 `pair<int, string>`：`first` = 最长回文子序列长度，`second` = 一个具体方案；空串返回 `{0, ""}`。
- 要字典序最小的恢复结果，`fill` 用 `'a'` 即可（题面要求别的字母再改）。
- $n \leq 5000$ 时 LPS 的 $O(n^2)$ 也扛得住；只问长度、不恢复方案的话翻主章节的一维压缩版。

出处：码蹄杯 2025 官方题单 MC0408 恢复回文、MC0429 回文子序列。

```

string restore_question_mark_palindrome(string s, char fill = 'a') {
    int n = (int)s.size();
    for (int l = 0, r = n - 1; l <= r; ++l, --r) {
        if (s[l] != '?' && s[r] != '?' && s[l] != s[r]) return "";
        if (s[l] == '?' && s[r] == '?') {
            s[l] = s[r] = fill;
        } else if (s[l] == '?') {
            s[l] = s[r];
        } else if (s[r] == '?') {
            s[r] = s[l];
        }
    }
    return s;
}

```

```

pair<int, string> longest_palindromic_subsequence_with_answer(const string& s) {
    int n = (int)s.size();
    if (n == 0) return {0, ""};

    vector<vector<int>> dp(n, vector<int>(n, 0));
    for (int i = n - 1; i >= 0; --i) {
        dp[i][i] = 1;
        for (int j = i + 1; j < n; ++j) {
            if (s[i] == s[j]) dp[i][j] = dp[i + 1][j - 1] + 2;
            else dp[i][j] = max(dp[i + 1][j], dp[i][j - 1]);
        }
    }

    string left, right;
    int l = 0, r = n - 1;
    while (l <= r) {
        if (l == r) {
            left.push_back(s[l]);
            break;
        }
        if (s[l] == s[r] && dp[l][r] == dp[l + 1][r - 1] + 2) {
            left.push_back(s[l]);
            right.push_back(s[r]);
            ++l;
            --r;
        } else if (dp[l + 1][r] >= dp[l][r - 1]) {
            ++l;
        } else {
            --r;
        }
    }

    reverse(right.begin(), right.end());
    return {dp[0][n - 1], left + right};
}

```

怎么套题: `restore_question_mark_palindrome` 返回空串表示无解; 如果题目要求字典序最小, 默认把一对 `??` 填成 `'a'` 就是常见做法。LPS 函数会返回长度和一个具体子序列; 如果 `n` 到 `5000` 只要长度, 翻主章节的一维压缩版。

网格寻路：一次破障 BFS

赛时先看

题目信号: 题面是迷宫、地图、入口出口; 有墙/障碍; 允许一次特殊操作穿过或破坏墙; 边权都是 1。

本质: 在网格中从起点走到终点, 允许最多破坏一次障碍, 求最短步数。

接法: 把本节 `struct/class/函数组` 整段抄到 `solve()` 上面; `solve()`

里先构造对象, 再按下方“不会用就看这里”调用公开接口。

复杂度判定: $O(nm)$, 因为每个格子只有“没用破障 / 已用破障”两个状态。

维护的量: `dist[x][y][used]` (每个格子“未破障/已破障”两种状态的最短步数, 初始 `INF`); `GridState{x, y, used, dist}` (BFS 队列元素)。

警告: 访问状态必须包含 `used`, 只用 `vis[x][y]` 会错; 起点和终点坐标要统一成

0-indexed; 如果特殊操作不止一次, 把第二维改成 `0..K`。

约定: 本节的 `shortest_path_break_one_wall_from(grid, start, target)` 需要显式传入 0-indexed

起点终点; 读入地图时把 `S/T` 记成坐标即可。若题面允许破 `K` 次墙, 把 `array<int,2>` 改成 `vector<int>(K+1, INF)`, 状态里的 `used` 范围改成 `0..K`

【最小完整示例（先抄这一段就能跑）】

题目: `3x3` 迷宫, `S` 在 `(0,0)`、`T` 在 `(2,2)`, 中间全是墙, 最多破 1 次墙, 求最短步数。

```

vector<string> grid = {"S.#", "###", "##T"}; // 1. 地图, '#' 是墙
int step = shortest_path_break_one_wall_from(grid, {0, 0}, {2, 2}); // 2. 0-indexed 起点终点
cout << step << '\n'; // 3. 输出最短步数, 不可达输出 -1

```

样例: 输出 4 (`S` → `(0,1)` → 破墙 `(1,1)` → `(2,1)` → `T`)。

【传参要求（照这个传不会错）】

- `shortest_path_break_one_wall_from(grid, start, target)`: `grid` 为 `vector<string>` (`n` 行 `m` 列, `'#'` 是墙、其余可走); `start/target` 是 0-indexed 的 `{x, y}`。
- 返回最短步数 `int`: 走不到返回 `-1`。
- 只允许破 1 次墙; 要破 `K` 次, 把 `array<int,2>` 改成 `vector<int>(K + 1, INF)`, `used` 范围改成 `0..K`。
- 起点终点用 0-indexed: 读入时把 `S/T` 记成坐标再传入。

【改板时先认这几个量】

- `dist`: 每个 (x, y, used) 状态的最短步数。
- `cur`: 当前弹出的 BFS 状态。

出处: 码蹄杯 2025 官方题单 MC0404 寻找出口、MC0469 寻找隐秘路径; 码蹄杯 2022 官方题单 MC0171 迷宫。

```
struct GridState {
    int x, y, used, dist;
};

// 带显式起点/终点参数的版本; 与上方从 S/T 自动识别的版本二选一即可。
int shortest_path_break_one_wall_from(const vector<string>& grid, pair<int, int> start, pair<int, int> target) {
    const int INF = 1e9;
    static const int dx[4] = {1, -1, 0, 0};
    static const int dy[4] = {0, 0, 1, -1};
    int n = (int)grid.size();
    int m = (int)grid[0].size();
    vector<vector<array<int, 2>>> dist(n, vector<array<int, 2>>(m, {INF, INF}));
    queue<GridState> que;

    auto inside = [&](int x, int y) {
        return 0 <= x && x < n && 0 <= y && y < m;
    };

    dist[start.first][start.second][0] = 0;
    que.push({start.first, start.second, 0, 0});

    while (!que.empty()) {
        GridState cur = que.front();
        que.pop();
        if (cur.dist != dist[cur.x][cur.y][cur.used]) continue;
        if (make_pair(cur.x, cur.y) == target) return cur.dist;

        for (int dir = 0; dir < 4; ++dir) {
            int nx = cur.x + dx[dir];
            int ny = cur.y + dy[dir];
            if (!inside(nx, ny)) continue;

            int next_used = cur.used;
            if (grid[nx][ny] == '#') {
                if (next_used == 1) continue;
                next_used = 1;
            }

            if (dist[nx][ny][next_used] > cur.dist + 1) {
                dist[nx][ny][next_used] = cur.dist + 1;
                que.push({nx, ny, next_used, cur.dist + 1});
            }
        }
    }
    return -1;
}
```

怎么套题: 读入地图时把起点 `S` 和终点 `T` 记录成 0-indexed 坐标, 并把它们当普通可走格处理。若题面允许破 `K` 次墙, 把 `array<int, 2>` 改成 `vector<int>(K+1, INF)`, 状态里的 `used` 范围改成 `0..K`。